HZ BOOKS
华章IT

Pearson

Linux/Unix
技术丛书

Linux

Embedded Linux Systems with the Yocto Project

嵌入式Linux系统开发

基于Yocto Project

[美] 鲁道夫·J. 斯特雷夫（Rudolf J. Streif） 著

胥峰 译

机械工业出版社
China Machine Press

Linux/Unix技术丛书

嵌入式Linux系统开发：基于Yocto Project

Embedded Linux Systems with the Yocto Project

（美）鲁道夫·J.斯特雷夫（Rudolf J.Streif） 著

胥峰 译

ISBN：978-7-111-60382-5

本书纸版由机械工业出版社于2018年出版，电子版由华章分社（北京华章图文信息有限公司，北京奥维博世图书发行有限公司）在中华人民共和国境内（不包括中国香港、澳门特别行政区及中国台湾地区）制作与发行。

版权所有，侵权必究

客服热线：+ 86-10-68995265

客服信箱：service@bbbvip.com

官方网址：www.hzmedia.com.cn

新浪微博 @华章数媒

微信公众号 华章电子书（微信号：hzebook）

目录

译者序
序
前言
致谢
第1章　用于嵌入式系统的Linux
1.1　为什么为嵌入式系统选择Linux
1.2　嵌入式Linux形势
1.2.1　嵌入式Linux发行版
1.2.2　嵌入式Linux开发工具
1.3　定制Linux发行版——为什么困难
1.4　关于开源许可的几句话
1.5　组织、相关实体和标准
1.5.1　Linux基金会
1.5.2　Apache软件基金会
1.5.3　Eclipse基金会
1.5.4　Linux标准基
1.5.5　消费电子产品工作组
1.6　总结
1.7　参考文献
第2章　Yocto项目
2.1　启动第一个Yocto项目构建
2.1.1　先决条件
2.1.2　获取Yocto项目工具
2.1.3　设置构建主机
2.1.4　配置构建环境
2.1.5　启动构建
2.1.6　验证构建结果

2.1.7 Yocto项目构建器具
2.2 Yocto项目系列
2.3 历史概览
2.3.1 OpenEmbedded
2.3.2 BitBake
2.3.3 Poky Linux
2.3.4 Yocto项目
2.3.5 OpenEmbedded和Yocto项目的关系
2.4 Yocto项目术语
2.5 总结
2.6 参考文献
第3章 OpenEmbedded构建系统
3.1 构建开源软件包
3.1.1 获取
3.1.2 解压
3.1.3 打补丁
3.1.4 配置
3.1.5 构建
3.1.6 安装
3.1.7 打包
3.2 OpenEmbedded工作流
3.2.1 元数据文件
3.2.2 工作流过程步骤
3.3 OpenEmbedded构建系统架构
3.3.1 构建系统结构
3.3.2 构建环境结构
3.3.3 元数据层结构
3.4 总结

3.5 参考文献
第4章 BitBake构建引擎
4.1 获取和安装BitBake
4.1.1 使用发布快照
4.1.2 克隆BitBake开发仓库
4.1.3 构建和安装BitBake
4.2 运行BitBake
4.2.1 BitBake执行环境
4.2.2 BitBake命令行
4.3 BitBake元数据
4.4 元数据语法
4.4.1 注释
4.4.2 变量
4.4.3 包含
4.4.4 继承
4.4.5 可执行元数据
4.4.6 元数据属性
4.4.7 元数据名（键）扩展
4.5 源下载
4.5.1 使用Fetch类
4.5.2 获取器实现
4.5.3 镜像
4.6 HelloWorld——BitBake方式
4.7 依赖处理
4.7.1 配置
4.7.2 声明依赖
4.7.3 多个提供器
4.8 版本选择

4.9 变体
4.10 默认元数据
4.10.1 变量
4.10.2 任务
4.11 总结
4.12 参考文献
第5章 故障排除
5.1 记日志
5.1.1 日志文件
5.1.2 使用记日志语句
5.2 任务执行
5.2.1 执行特定任务
5.2.2 任务脚本文件
5.3 分析元数据
5.4 开发shell
5.5 依赖性关系图
5.6 调试层
5.7 总结
第6章 Linux系统架构
6.1 Linux或者GNU/Linux?
6.2 Linux系统的剖析
6.3 引导加载程序
6.3.1 引导加载程序的角色
6.3.2 Linux引导加载程序
6.4 内核
6.4.1 主要Linux内核子系统
6.4.2 Linux内核启动
6.5 用户空间

6.6 总结
6.7 参考文献
第7章 构建定制Linux发行版
7.1 核心镜像——Linux发行版蓝图
7.1.1 通过本地配置来扩展核心镜像
7.1.2 用QEMU测试镜像
7.1.3 使用构建历史验证和比较镜像
7.1.4 用菜谱扩展核心镜像
7.1.5 镜像特性
7.1.6 包组
7.2 从头构建镜像
7.3 镜像选项
7.3.1 语言和区域
7.3.2 包管理
7.3.3 镜像大小
7.3.4 根文件系统类型
7.3.5 用户、组和密码
7.3.6 调整根文件系统
7.4 发行版配置
7.4.1 标准发行版策略
7.4.2 Poky发行版策略
7.4.3 发行版特性
7.4.4 系统管理器
7.4.5 默认发行版设置
7.5 外部层
7.6 Hob
7.7 总结
第8章 软件包菜谱

8.1 菜谱布局和惯例
8.1.1 菜谱文件名
8.1.2 菜谱布局
8.1.3 格式指导方针
8.2 写新菜谱
8.2.1 建立菜谱
8.2.2 获取源代码
8.2.3 解压源代码
8.2.4 为源代码打补丁
8.2.5 增加许可信息
8.2.6 配置源代码
8.2.7 编译
8.2.8 安装构建输出
8.2.9 设置系统服务
8.2.10 打包构建输出
8.2.11 定制安装脚本
8.2.12 变体
8.3 菜谱例子
8.3.1 C文件软件包
8.3.2 基于makefile的软件包
8.3.3 基于CMake的软件包
8.3.4 基于GNU Autotools的软件包
8.3.5 外部构建软件包
8.4 devtool
8.4.1 使用devtool的往返开发
8.4.2 用于现有菜谱的工作流
8.5 总结
8.6 参考文献

第9章　内核菜谱
9.1　内核配置
9.1.1　菜单配置
9.1.2　配置片段
9.2　内核补丁
9.3　内核菜谱
9.3.1　从一个Linux内核树构建
9.3.2　从Yocto项目内核仓库构建
9.4　树外模块
9.4.1　开发内核模块
9.4.2　创建用于第三方模块的菜谱
9.4.3　把模块包含在根文件系统中
9.4.4　模块自动加载
9.5　设备树
9.6　总结
9.7　参考文献
第10章　板支持包
10.1　Yocto项目板支持包理念
10.2　用板支持包构建
10.2.1　为BeagleBone构建
10.2.2　外部Yocto项目板支持包
10.3　Yocto项目板支持包内部
10.3.1　许可文件
10.3.2　维护者文件
10.3.3　README文件
10.3.4　README.sources文件
10.3.5　预构建二进制
10.3.6　层配置文件

10.3.7 机器配置文件
10.3.8 类
10.3.9 菜谱文件
10.4 创建Yocto项目板支持包
10.4.1 Yocto项目板支持包工具
10.4.2 用Yocto板支持包工具创建板支持包
10.5 调优
10.6 创建可启动介质镜像
10.6.1 用烹制模式创建镜像
10.6.2 用原始模式创建镜像
10.6.3 kickstart文件
10.6.4 kickstart文件指令
10.6.5 插件
10.6.6 传输镜像
10.7 总结
10.8 参考文献
第11章 应用开发
11.1 Yocto项目ADT内部
11.2 设置Yocto项目ADT
11.2.1 构建工具链安装程序
11.2.2 安装工具链
11.2.3 用工具链工作
11.2.4 目标上执行
11.2.5 远程目标上调试
11.3 构建应用
11.3.1 基于makefile的应用
11.3.2 基于Autotools的应用
11.4 Eclipse集成

11.4.1 安装Eclipse IDE
11.4.2 集成Yocto项目ADT
11.4.3 开发应用
11.4.4 在目标上部署、运行和测试
11.5 使用模拟目标的应用开发
11.5.1 为用QEMU进行应用开发做准备
11.5.2 构建应用并在QEMU中启用它
11.6 总结
11.7 参考文献
第12章 许可和合规
12.1 管理许可
12.1.1 许可追踪
12.1.2 通用许可
12.1.3 商业许可的包
12.1.4 许可部署
12.1.5 黑名单许可
12.1.6 提供许可程序清单和文本
12.2 管理源代码
12.3 总结
12.4 参考文献
第13章 高级主题
13.1 Toaster
13.1.1 Toaster操作模式
13.1.2 Toaster设置
13.1.3 本地Toaster开发
13.1.4 Toaster配置
13.1.5 Toaster生产部署
13.1.6 Toaster Web用户界面

13.2 构建历史
13.2.1 启用构建历史
13.2.2 配置构建历史
13.2.3 推送构建历史到Git仓库服务器
13.2.4 理解构建历史
13.3 源镜像
13.3.1 使用源镜像
13.3.2 设置源镜像
13.4 自动构建器
13.4.1 安装自动构建器
13.4.2 配置自动构建器
13.5 总结
13.6 参考文献
附录A 开源许可协议
附录B 元数据参考

译者序

为什么要翻译这本书

物联网的兴起推动着越来越多的嵌入式设备投入使用，从普通的家用设备到用于提供治安监控、交通流控等公共服务的设备，其中的核心之一就是嵌入式操作系统。而Linux正成为越来越多嵌入式开发者的首选。

市面上不缺乏适用于各种开发语言的开发、编译和打包工具，但在Yocto项目出现前，这些零散的工作需要嵌入式开发者自己串起来以交付最终的嵌入式系统。Yocto项目恰好解决了这些问题，它是很多工具的有机整合，可以让嵌入式开发者更容易且更高效地交付嵌入式系统。

本书就是专注于讲解Yocto项目的实战手册。书中从一开始就强调“动手实验”的重要性，鼓励读者跟着里面的例子动手做起来，最终完全了解整个过程并知道每个步骤需要注意什么。

另外，本书也讲解了如何在嵌入式系统开发和交付过程中管理许可及合规性。在强调知识产权的今天，这一点的重要性也不言而喻。

Yocto项目社区经理、本书英文原版技术审稿人Jeffrey Osier-Mixon在亚马逊网站（https://www.amazon.com/gp/customer-reviews/R20UE23XPGTB7Y/ref=cm_cr_arp_d_rvw_ttl?ie=UTF8&ASIN=0133443248）对本书评论说：“该书非常细致地解释了系统的各个工具和组件是如何相互适应的，从整体构建工具BitBake及其操作的菜谱（元数据）开始，然后深入系统经过的各个过程。最有用的是，它从开发者的角度讲解构建系统，涵盖了开发者在构建过程的各

个环节需要解决的问题……总之，本书是了解使用Yocto项目工具构建用于嵌入式系统的Linux之极佳资源。我非常推荐。”简而言之，本书是一位有着超过20年软件工程实践经验的专家的扛鼎之作，它务实、细致、逻辑清晰。它不仅仅是一本参考书，更是一本实践指南。

本书中文版可能是专注于Yocto项目的先驱性图书，希望能对广大嵌入式系统开发者有所帮助。

读者对象

本书适合以下几类读者阅读：

·嵌入式开发工程师

·嵌入式测试工程师

·嵌入式产品经理

·嵌入式产品合规工程师

·计算机相关专业学生

勘误

虽然译者努力保证本书翻译中不出现错误，但鉴于译者的知识和能力，书中难免出现用词错误、适用性等问题。在此，恳请读者不吝指教，指出错误。请读者发送邮件到xufengnju@163.com帮助译者修正错误。勘误将发布在https://github.com/xufengnju上。

致谢

感谢机械工业出版社华章公司引进了原书的中译本版权，这是本书中文版得以面市的最重要条件。感谢华章公司张志铭和关敏老师，你们专业的编辑能力为本书提供了重要的质量保证。

感谢我的妻子吕宁和可爱的女儿胥欣，谢谢你们的支持和理解。

胥峰

2018年3月

序

嵌入式Linux的形势有点像美国老西部（the Old West）：不同的技术前哨分散在各处，在这些技术前哨之间是贫瘠和往往十分危险的形势。如果想去那里旅行，则需要有充足的储备，要熟悉地形，并且要有可靠的向导。

正如在18世纪中期淘金热期间人们迁移到美国老西部一样，伴随着物联网（Internet of Things，IoT）的热潮，开发者正在进入嵌入式Linux的世界。正如增加的人口为老西部带来法律、秩序和文明一样，重要的新开源软件项目正在为嵌入式Linux带来秩序。

Yocto项目是十分重要的秩序缔造者。它的工具能帮你主要关注设计项目（你想要构建的东西）并且仅仅投入最少的必要时间和精力来把它们整合在一起（如何构建你想要的东西）。

这本书是一个可靠的向导。通过按逻辑排序并带有清晰和完整说明的章节，它将帮助你完成工作并把物联网项目带向市场。如果运气不错，你将会在一路上收获快乐。

享受冒险吧！

Arnold Robbins

Prentice Hall开源软件开发系列编辑

前言

智能家居、智能汽车、智能手机、智能电视、智能恒温器、智能电灯、智能手表、智能洗衣机、智能烘干机、智能冰箱以及智能篮球。欢迎来到智能万物的勇敢新世界。

在日常生活中，我们触及和交互的几乎所有东西中都有嵌入式计算机，它的剧增，使得嵌入式系统工程和嵌入式软件开发受到了瞩目。嵌入式系统隐藏在其用户的直接视线之外，它们缺少那些带有浮华用户界面的网络应用所具有的吸引力，也缺少那些带有动画和逼真图像的计算机游戏所具有的酷酷的感觉。毫不奇怪的是，计算机科学专业的学生和软件开发者几乎不会把嵌入式软件工程当成他们的第一职业选择。然而，“智能万物革命”和物联网正在推动人们对那些能够桥接硬件和软件世界的专业人士的需求。那些使用电路原理图语言和编程语言的专家将受到雇主的追捧。

对于爆炸式增多的嵌入式应用来说，Linux已经成为第一选择。对于这种选择，有很多恰当的理由。我们将在接下来的章节中详细阐述这些理由。经历了作为服务于各种行业的嵌入式软件开发者的旅程，我通过艰辛的方式学习了嵌入式系统的Linux。对几乎任何编程语言来说，都不缺少优秀的开发工具。绝大部分用于Linux的库和应用都会由于它们的工具化而被简单原生地构建。使用内核自有的构建系统，即使从头开始构建Linux内核几乎也是一件轻而易举的事。然而，当需要把所有的东西整合进可启动的系统时，选择就很少了。

通过提供一套全面的、以OpenEmbedded构建系统为中心的集成化工具，Yocto项目弥补了这种鸿沟。在大约几个小时内实现从源代码到可启动的系统——我希望在我开始使用嵌入式Linux的时候会有这样的奢侈。

本书定位

集成多个不同的必要步骤来从头创建完全可用的Linux操作系统栈的构建系统是相当复杂的。本书专注于构建系统本身以及如何有效使用它来构建定制的Linux发行版。本书不是关于嵌入式Linux的教程。虽然第6章解释了Linux系统架构的基础（因为该信息对于理解构建系统如何组合多个不同的组件成为可运行的系统是必要的），但是就嵌入式Linux本身而言，不会深入它的细节。如果你是嵌入式Linux开发新手，那么强烈推荐Christopher Hallinan的好书《Embedded Linux Primer》，该书与本书是同一系列丛书（Prentice Hall开源软件开发系列）。

在本书中，你将会学习OpenEmbedded构建系统如何工作、如何编写菜谱（recipe）来构建你自己的软件组件、如何使用和创建Yocto项目板支持包（Board Support Package，BSP）来支持不同的硬件平台以及如何调试构建失败。你还将学习如何为应用开发构建软件开发包，以及如何为了无缝的往返开发（round-trip development）而把这些开发包和流行的Eclipse集成开发环境（Integrated Development Environment，IDE）集成起来。

本书读者

本书面向那些具有Linux实践知识的软件开发者和程序员。假设你熟悉Linux命令行，可以使用典型的工具（例如Make和C/C++编译器）在Linux系统上构建程序，还可以阅读和理解基本的shell脚本。

构建系统是完全用Python编写的。即使你不是Python专家也可以使用它，并理解它如何工作，但是具备一些核心的Python知识一定是有利的。

本书内容

第1章对采用嵌入式系统的Linux进行简要介绍，为嵌入式Linux的形势和创建定制嵌入式发行版的挑战的概览奠定了基础。

第2章通过用构建系统启动Linux操作系统栈的首次构建来介绍Yocto项目，同时也给出了Yocto项目系列和Yocto项目历史的概览。

第3章解释了构建系统的基础、工作流和架构。

第4章解析了BitBake——处于OpenEmbedded构建系统核心的构建引擎。其中解释了菜谱（recipe）的元数据（metadata）概念、类、配置文件及其语法。BitBake方式的“Hello World”项目说明了构建工作流。通过提供的信息，你会获得用于理解所提供的菜谱和编写自己的菜谱的必要知识。

第5章介绍了可用于对构建问题进行故障排除的工具和机制，并且提供了如何有效使用这些工具的实用建议。

第6章提供了Linux操作系统栈的基础，并解释了不同组件是如何分层的。另外还讨论了内核空间和用户空间的概念以及应用程序如何通过由标准C库提供的系统调用来和Linux内核交互。

第7章细致讲解了如何使用Yocto项目创建定制化的Linux发行版。它以构建系统可用的Linux发行版蓝图以及如何定制它们的简要介绍作为开始。然后演示了如何使用构建系统工具完全从头创建Linux发行版。读完本章以后，你将知道如何构建自己的操作系统镜像。

第8章解释了BitBake菜谱以及如何编写它们来用构建系统构建自己的软件包。这一章提供了你可以尝试的各种真实世界的菜谱例子。

第9章仔细研究了用OpenEmbedded构建系统来构建Linux内核的细节，解释了构建系统工具如何与内核的构建环境交互以设置内核配置和应用补丁。本章以关于构建系统如何处理树外（out-of-tree）内核模块以及合并构建设备树和构建过程的讨论结束。

第10章介绍了构建系统如何支持针对不同硬件的构建——也就是说，为不同CPU架构和系统构建。在介绍了Yocto项目板支持包的概念以后，本章细致讲解了如何使用板支持包构建项目。然后研究了Yocto项目板支持包的内部原理，并用实际的例子来解释如何构建自己的板支持包，你可以将这个例子使用在真实硬件上。本章以为不同硬件配置创建可启动介质镜像结束。

第11章描述了Yocto项目对开发应用的支持，这些应用被用在通过构建系统创建的Linux操作系统栈上。其中提供了关于如何构建那些包含用于往返应用开发的全部必需工具的应用开发工具集（Application Development Toolkit，ADT）的实际操作指南。示例说明了如何通过命令行工具和Eclipse集成开发环境来将应用开发工具集用于应用开发。指南会一步步教你如何远程运行和调试真实硬件目标上的应用。

第12章讨论了对于开源许可合规性的要求以及Yocto项目提供的那些有助于满足要求的工具。

第13章介绍了帮助你把Yocto项目扩展到团队的一些工具。Toaster是基于Web的图形用户界面，它可用于创建能从Web浏览器远程控制的构建系统。构建历史是提供追踪和审计能力的工具。使用源镜像，你可以共享源包来避免重复的下载，并为产品交付控制源版本。最后但并非最不重要的是，Autobuilder为自动化构建、质量保证和发布过程提供了开箱即用的持续构建和集成框架。掌握了本章的知识后，你可以为Yocto项目有效地设置团队环境。

附录包括流行的开源许可，以及以字母排序的构建系统元数据层和机器的参考。

动手操作经验

本书的撰写目的是提供Yocto项目的实际动手操作经验。如果依照并且尝试了相关示例，那么你会收获良多。它们中的大部分可以依赖运行最新Linux发行版的基于x86的工作站来轻松完成（详细的需求会在第2章提供）。为了获得更好的经验，要利用流行的开发板，例如BeagleBone、MinnowBoard　Max或者Wandboard。BeagleBone是优秀的低成本的实验平台。另外两个板提供更好的性能，也让你获得关于多核系统的经验。

分析代码并且尝试理解本书的代码。遵循步骤，然后通过改变设置、应用你自己的配置等来完成自己的设计。这是最好的学习方式，并且我可以告诉你，这也相当有趣。让你自己的第一个Linux发行版工作在你选择的一个硬件上是很棒的体验。

致谢

本书是我第一次尝试撰写的技术书籍。好吧，就此而言，是我人生第一本书。我必须谦逊地承认，我大大低估了开始这样一个项目所需要的精力——花费在实验上的时间、找到最好的让它们工作的方法以及以简洁和可理解的方式文档化所有事物。在这个过程中，要真心感谢很多作者和技术撰稿人的工作——我已经阅读过很多并且正在持续阅读他们的书籍和手册。

首先，对我的家庭，包括我挚爱的妻子Janan和3个非常棒的儿子Dominic、Daniel和Jonas表示感谢。如果没有他们的支持和理解，我是不可能在写作上花费很多时间的。

特别感谢Yocto项目团队。当我接触彼时还是Yocto项目项目经理的Dave Stewart和Yocto项目社区经理的Jeffrey Osier-Mixon时，他们立刻对写作本书的主意表示欢迎并且提供了支持。这个团队的一些人在给予建议和问题解答方面特别有帮助：Beth Flanagan对Autobuilder方面、Belen Barros Pena和Ed Bartosh对Toaster方面以及Paul Eggleton和Khem Raj对我提交到Yocto项目邮件列表里面的问题提供的参考。

特别感谢撰写了《Embedded Linux Primer：A Practical Real-World Approach》（Prentice Hall,2006）的Christopher Hallinan。他写的这本书激发了我创作关于Yocto项目的书的欲望。

特别感谢总策划编辑Debra Williams Cauley，感谢她的指导，特别是在本书写作过程中付出的耐心。本书花费了远超预期的时间，而我是唯一需要为本书错过交稿期限负责的人。

怎么感谢和表扬本书专注的审核团队都不为过，他们是Chris

Zahn、Jeffrey Osier-Mixon、Robert Berger和Bryan Smith。感谢他们的纠错及改进建议，这保证了本书的质量。

我也想感谢Prentice Hall的生产团队——Julie Nahil和Anna Popick，感谢他们的协调和指导工作，特别是Carol Lallier在编辑原稿过程中的勤勉。

还要感谢Linux基金会和Jerry Cooperstein，他们给了我开发Linux基金会的关于Yocto项目的培训课程的机会。没有什么方法比教别人更利于学习。感谢我教过的学生。通过他们的宝贵提问和反馈，我获得了大量对于学生用嵌入式Linux开发产品时遇到的很多不同问题的理解。他们问得最多的问题是：“有没有一本关于Yocto项目的书？”终于，我可以回答：“有的。”

第1章　用于嵌入式系统的Linux

物联网（Internet of Things）正激发着梦想家们的想象灵感，也激发着工程师们的创造性。作为由巨大数量的实时收集、分析和传递数据的互联设备组成的统一计算网络，物联网承载着新的信息技术时代的希望。

组成物联网的设备需要满足一套全新的需求并且提供原来嵌入式系统中不存在的功能性。连通性，包括通过蜂窝数据网络连通，是一个明显的需求，其他还有远程管理、软件配置和更新、能耗效率以及长的寿命，当然还有安全性，仅举几例。

正在改变的嵌入式系统的形势需要新的方法来构建操作这种新的互联硬件的软件栈。

1.1 为什么为嵌入式系统选择Linux

Linux首次亮相是作为英特尔x86架构的PC硬件所用的通用操作系统（General-Purpose Operating System，GPOS）。正如Linux创造者Linus Torvalds在news:comp.os.minix上现在非常有名的帖子中所写的，他明确说道，“我正在做一个（免费）操作系统……它是不可移植的（使用386任务切换等），并且它可能永远不会支持除了AT硬盘以外的硬盘……”

被互联网的兴起所驱动，因为很多理由充分的原因，Linux迅速进化成为网络服务器和联网服务提供基础设施的服务器操作系统。

然而，在三个主要方面，Linux保持了它的通用操作系统的初始特点。这三个方面使得它在初期并没有成为工程师为嵌入式操作系统所定的首要选择：

·文件系统：Linux是基于文件的操作系统，它需要一个在有读和写访问的面向块的大容量存储设备上的文件系统。面向块的大容量存储通常意味着硬盘驱动器具有旋转的盘片，而这对于大部分嵌入式的用例都是不实际的。

·内存管理单元（Memory Management Unit，MMU）：Linux是多任务操作系统。有效的任务切换要求在CPU上运行时，各个进程有自己的可以很容易映射到物理内存的私有内存地址空间。被广泛使用在典型嵌入式应用上的微控制器不提供内存管理单元。

·实时：运行关键应用的嵌入式系统可能会要求在一定误差幅度（margin of error）内保证时间的预测响应，通常称为决定论（determinism）。在其随后的程序或者部分的迭代中时间上的错误数量被称作“抖动”（jitter）。可以绝对保证其执行操作的最大时间的操作系

统被称为“硬实时系统”（hard real-time system）。通常在一定时间内执行操作的操作系统被称为“软实时”（soft real-time）。对Linux来说，虽然有一些提供了实时能力的解决方案（例如最著名的PREEMPT-RT）早在1996年就开发出来了，但是它们仍然没有成为主流Linux内核的一部分。

在过去的几年里，半导体技术的进步帮助克服了这些在嵌入式系统中采用Linux的障碍。在很多消费产品（例如数码相机）中使用到的、随处可见的、廉价的和长期可靠的闪存设备正在为文件系统提供必要的大容量存储。在单一芯片上把一个或者多个通用CPU核心和内存管理单元以及外围设备结合起来的强大的片上系统（System-on-Chip，SoC）设计成为嵌入式系统工程师的处理器的首选，并且在嵌入式应用方面正在逐渐取代微控制器。

今天我们看到了将Linux用于嵌入式设备的爆炸性的增长。几乎每个行业都受到这种趋势的影响。运营商级Linux（Carrier-Grade Linux，CGL）操作系统被用在公共交换的电话网络和全球数据网络的产品中。可能的情况是，你携带了手机，看带有机顶盒的电视和高清电视，通过宽带调制解调器和网络交换机的互联网浏览，通过个人导航设备来寻找路线，以及日常使用其他设备，这些都是由Linux来驱动的。

嵌入式Linux的快速增长存在很多理由。以下列出一些：

·特许权使用费（Royalty）：与传统的专有操作系统不同，Linux可以不用支付任何特许权使用费而部署。

·硬件支持：Linux支持大量硬件设备，包括所有主要和通常使用的CPU架构：在各自32位和64位变体上的ARM、英特尔x86、MIPS以及PowerPC。

·网络：Linux支持大量网络协议。除了普遍的TCP/IP之外，几乎在任何物理介质上的任何其他协议也可被实现。

·模块化：Linux操作系统栈是由很多不同的软件包组成的。工程师们可以定制这个栈来使它精准地适合自己的应用。

·可伸缩性：从只有一个CPU和有限资源的系统，到具有多个多核CPU、大内存占用、多个网络接口以及更多其他特性的系统，Linux都能伸缩适用。

·源代码：Linux内核源代码，以及所有组成Linux操作系统栈的软件包的源代码，都是公开可获取的。

·开发者支持：因为它的开放性，Linux吸引了极大数量的活跃的开发者，那些开发者快速构建了对新硬件的支持。

·商业支持：数量正在增加的硬件和软件厂商，包括所有半导体生产商以及很多独立的软件厂商（Independent Software Vendor，ISV），正在通过产品和服务对Linux提供支持。

·工具化：对软件开发，Linux提供了无数工具。这些工具包括从为几乎任何编程语言提供的编译器到数量逐渐增长的对嵌入式系统开发很重要的分析和性能测量工具。

现在，这些原因和很多其他原因使得Linux成为嵌入式系统工程师的首要选择，并推动着它在消费者产品和专业产品方面的加速应用。

1.2 嵌入式Linux形势

嵌入式系统是多种多样的。种类繁多的硬件不可避免地带来了软件适用性的负担，最显著的是操作系统，操作系统通过它的库和应用程序编程接口（Application Programming Interface，API）提供了对硬件的抽象。没有一劳永逸的方案，而你，作为系统工程师，将必须为嵌入式Linux项目选择一个起点。

在接下来的段落里，我们提供了对嵌入式设备来说最常用的开源项目的概览。除了提到的这些，当然还有一些来自操作系统厂商（Operating System Vendor，OSV）的商业嵌入式Linux产品。

1.2.1 嵌入式Linux发行版

与桌面版和服务器Linux发行版类似，不断发展的嵌入式Linux发行版被社区项目和商业操作系统厂商开发出来。其中一些被定位为针对特定类型的嵌入式系统和设备，而其他的在本质上是更通用的，意在提供一个基础而非完整的系统。

Android

尽管Android（www.android.com、http://developer.android.com和http://source.android.com）的主要目标是手机和平板计算机，但是它正在作为用于所有种类的嵌入式设备的操作系统而逐渐变得流行。这一点儿也不奇怪，因为它的源代码是可以自由获得的，而且包含了带有配置工具的构建系统，该构建系统使得开发者可以改变系统以适应不同的硬件设备。

特别是，如果目标设备使用的是基于ARM的片上系统并且有触摸屏，那么Android是系统工程师们的热门选择，因为对硬件的必要支持是系统的核心部分。对英特尔x86架构的移植确实存在，但是可用的硬件更少，并且开发经常是更加昂贵的。

然而，Android并不能满足嵌入式设备的所有需求。虽然它利用了在Linux操作系统栈中通常见到的Linux内核和其他软件包，但是它的基础架构不同于典型的Linux操作系统栈。Android使用了它自己的由一套精简过的应用程序编程接口组成的C库，其中也有自己的文件系统布局以及其他特定的扩展。这些修改使它并不能直接移植标准Linux软件包到Android。

虽然Android在这个意义上（整个系统的源代码都是可以自由获得

的，在很少限制下可以因为任何目的而被使用、修改和扩展）来说是开源项目，但是开发者不能把他们的变更贡献回Android。谷歌独立控制着这个系统的路线图。基于谷歌的Android发布版的CyanogenMod（www.cyanogenmod.org）社区发行版正在尝试填补这一空白。

尽管如此，Linux内核确实缺少Android的一个重要扩展：电源管理。虽然一些Linux内核开发人员因其过于简单的架构而皱眉，但Android唤醒锁已经成为Linux电源管理的事实标准。

Ångström发行版

Ångström发行版（主页是www.angstrom-distribution.org）因其不断增加的支持开发板（development board）的列表，正在逐渐成为项目的重要资源。Ångström是由一组工作在OpenEmbedded、OpenZaurus和OpenSimpad项目上的开发者启动的社区发行版。Ångström在一开始就使用了OpenEmbedded工具，但是如今正在适应Yocto项目的架构和结构。

OpenWrt

OpenWrt（www.openwrt.org）首次亮相是作为针对路由网络流量的嵌入式设备的开源操作系统，例如宽带调制解调器、路由器、住宅网关和其他客户端设备（Consumer Premises Equipment，CPE）。OpenWrt的核心组件是Linux内核、uClibc和BusyBox。

OpenWrt的早期版本是构建在用于Linksys的WRT54G住宅网关和无线路由器的GPL许可的资源以及用Buildroot所创建的根文件系统上——因此有了OpenWrt这个名字。

OpenWrt支持多种硬件设备和评估板。OpenWrt的核心强项是全面的配置网络技术和协议的能力列表，包括路由、网状网络、防火墙、地

址转换、端口转发、负载均衡以及更多的功能。

虽然OpenWrt的初衷是操作那些通常运行起来不需要频繁人工交互的硬件，但是它提供了高级网页界面来方便地访问众多的配置选项。

对连接能力和远程管理的关注使得OpenWrt成为那些开发联网设备的系统工程师喜欢的选择。带有包管理的可写的文件系统使得它可以简单地增加功能性，即使在系统部署后也可以。

完整Linux发行版的嵌入式版本

对许多成熟的桌面、服务器和云的Linux发行版，针对嵌入式系统的变体现在也可以使用了：

·Debian（www.emdebian.org）

·Fedora（https://fedoraproject.org/wiki/Embedded）

·Gentoo（https://wiki.gentoo.org/wiki/Project:Embedded）

·SUSE（https://tr.opensuse.org/MicroSUSE）

·Ubuntu（https://wiki.ubuntu.com/EmbeddedUbuntu）

对熟悉特定Linux发行版的桌面或者服务器版的系统构建者和开发者来说，使用它的嵌入式变体提供了由熟悉的工具、文件系统布局以及其他所带来的好处。

1.2.2 嵌入式Linux开发工具

除了使用嵌入式Linux发行版，你也可以使用嵌入式Linux开发工具构建自己的定制Linux操作系统栈。这给了你最大的控制和灵活性，但在大部分情况下，这需要更多的努力。

Baserock

Baserock是开源项目，它在一个包中提供了面向Linux发行版的构建系统、开发环境和开发工作流。Baserock的主要特点是：

·作为提供可追踪性的手段，Git作为核心来管理从构建指令到构建制品（artifact）的一切事物。

·内置编译以避免跨构建环境的复杂度。

·跨多个系统的分布式构建使用虚拟机。

当前，Baserock提供针对x86、x86_64和ARMv7架构的构建支持。该项目的主页是http://wiki.baserock.org。

Buildroot

Buildroot是为完整的嵌入式Linux系统所准备的构建系统。它使用GNU Make和一套makefile来创建交叉编译的工具链，有根文件系统、内核镜像和引导加载程序镜像。该项目的主页是http://buildroot.uclibc.org。

Buildroot主要针对小型嵌入式系统，并且支持多种CPU架构。为了启动开发，它限制了配置选项的选择，并且默认了对嵌入式系统来说最

常用的选项：

·uClibc是用以构建交叉编译工具链的目标库。和GNU C库（glibc）相比，uClibc更加紧凑，并且针对小型嵌入式系统做了优化。uClibc支持几乎所有CPU架构和共享的库以及线程化。

·BusyBox是默认的命令行实用程序的集合。

这些默认的设定通常可帮助我们在15到30分钟内用Buildroot来构建基本的嵌入式Linux系统，构建时间取决于构建的主机。然而，这些设定不是绝对的，Buildroot的简单且灵活的结构使其容易理解和扩展。内部的交叉工具链可以用外部的工具链替换，例如crosstool-ng，uClibc可以用其他C库替换。

Buildroot已经支持很多标准的Linux包，例如X.org、GStreamer、DirectFB和Simple DirectMedia Layer（SDL）。交叉工具链可以用来构建额外的包并把这些包包含在根文件系统中。

Buildroot很紧凑并且容易设置。在构建主机上，单一文件（压缩包）的下载和一些额外包的安装就是开始所需要的全部。在解压了压缩包后，make menuconfig命令启动基于文本的用户界面使得能够配置大量被支持的目标和设定其他选项。除了menuconfig之外，Buildroot还提供了gconfig和xconfig，它们是可选的图形用户界面。

通过直接从上游项目下载源代码文件，Buildroot可以从源代码创建所有东西。一个不错的功能是，离线的构建可以通过使用make source下载所有源代码来完成。Buildroot预先获取所有必需文件，然后在不需要再连接到互联网的情况下配置和运行构建。

OpenEmbedded

OpenEmbedded（www.openembedded.org）是构建框架，它包含工具、配置数据和菜谱以创建针对嵌入式设备的Linux发行版。处在OpenEmbedded核心的是管理构建过程的BitBake任务执行器。

从历史上来说，OpenEmbedded是通过合并OpenZaurus项目和来自其他项目（例如Familiar Linux和OpenSIMpad）的贡献而创建出来的。

OpenEmbedded曾经被用来开发大量的开源嵌入式项目，最知名的是致力于为手机交付完整开源软件栈的OpenMoko项目（http://wiki.openmoko.org）。

OpenEmbedded、Yocto项目和Ångström发行版都有相同的根基，它们以各种形式构建在对方之上并且互相补充。在下一章深入Yocto项目细节时，我们将解释共同点和差异。

Yocto项目

Yocto项目当然是本书的主题。它被列在此处以完整嵌入式Linux形势的概览。你可以在https://www.yoctoproject.org找到它的网页。

Yocto项目不是单一的开源项目而是代表了在其保护伞下开发和维护的完整项目家族。本书会描述与Yocto项目相关的很多项目，特别是Poky，Yocto项目的参考发行版，Poky包括了OpenEmbedded构建系统和一整套元数据。

嵌入式Linux的形势是多样的。这个列表并不全面，还有很多其他的开源项目提供了使用Linux开发嵌入式设备的解决方案。在我看来，这里所提到的项目是最活跃和最经常使用到的。在继续阅读之前，你可能希望花一些时间来访问这些项目的网页。它们会帮你理解这些项目的目标和它们之间的比较。

也有一些商业的产品补充嵌入式Linux形势。常见的情况是，这些产品包括交叉开发工具链（cross-development toolchain）、发行版构建器、应用开发集成开发环境以及更多。越来越多的嵌入式系统的操作系统厂商在使用Yocto项目作为上游。它们使用Yocto项目工具来创建产品线。它们中有很多是Yocto项目的成员，并且通过工程化和金融资源来支持Yocto项目。

1.3 定制Linux发行版——为什么困难

我们来面对它——构建和维护操作系统不是简单的任务。需要考虑操作系统的很多不同方面以创建完全功能性的计算机系统。

·引导加载程序（Bootloader）：引导加载程序是第一个软件，它负责初始化硬件、加载操作系统内核到内存中然后启动内核。引导加载程序通常是多级的，其第一级存储在非易失性内存中。第一级然后从挂载的存储（如闪存、硬盘）中加载第二级。以此类推。

·内核：内核，如其名所示，是操作系统的核心。它管理系统的硬件资源，并且通过它的应用程序编程接口向其他软件提供硬件的抽象。内核的主要功能是内存管理、设备管理和响应来自应用软件的系统调用。这些功能如何实现依赖于处理器架构和外围设备以及其他硬件配置。

·设备驱动：设备驱动是内核的一部分。它们通过内核系统调用以结构化的形式向应用软件提供对硬件设备的访问。通过设备驱动，应用软件可以配置硬件、从硬件读取数据或者向硬件写入数据。

·生命周期管理（Life Cycle Management）：从通电到关机，计算机系统呈现多个状态，在这些状态中，它向应用软件提供不同集合的服务。生命周期管理决定什么服务运行在什么状态以及它们需要以什么顺序启动从而维持一致性的操作环境。生命周期管理的一个重要内容也是电源管理，当不需要全部功能时把系统设置成节能模式，而当被请求时又能恢复完全操作模式。

·应用软件管理：应用软件和库组成了在典型系统上安装的软件的大部分，为最终用户提供功能。对一个完整的可操作系统来说，它通常

需要数百到数千个软件包。

Linux和大量的开源软件包都如同构造工具包的组成部分。不幸的是，它更像一个谜而不是乐高。搞清楚不同包之间的依赖、不兼容性和冲突是一个艰巨的任务。一些包甚至提供相同或者相似的功能。选择哪一个？最终，你将必须绘制自己的蓝图从而为嵌入式项目构建Linux发行版。原则上，你有两种方法。

·自顶向下：关于这种方法，你一开始在众多可用的Linux发行版中选择一个，并根据需求，通过增加或移除软件包来定制它。多年前本书作者选择了这种方法（通过运行在x86服务器硬件上的高速图像处理系统）。这是一种可行的方法，并具有吸引力，因为使用经过测试和维护的发行版减轻了一些更繁琐的构建和维护自己发行版的任务。而且你可能会为此获得支持。然而，它可能会限制你对硬件的选择，因为大部分现成的Linux发行版是为x86硬件所构建的。而且，选择正确的发行版来开始以及为了目标设备而裁剪它也不是简单的事情。

·自底向上：自底向上的方法需要从源代码构建你自己定制的Linux发行版，先从引导加载程序和内核开始，然后为目标设备增加软件包来支持应用程序。这种方法可以给你最大的控制权（一般来说你可以学习到很多关于Linux和操作系统的内容），但是它也是具有挑战性的任务。你将不得不在一路上做出许多选择，从选择合适的工具链到设置内核配置选项，再到选择合适的软件包。其中一些选择是互相依赖的，例如工具链和目标库的选择，并且错误的选择会使你很快进入死胡同。在成功地构建和部署了发行版以后，你将面临维护它的重担——为内核和发行版中已经集成的所有其他软件包寻找补丁和安全更新。

这就是Yocto项目的优势所在。通过提供给你完整的工具集和蓝图来帮助你从零开始创建你自己的Linux发行版（以从上游项目来的源代码下载开始），它集合了两种方法的优点。随Yocto项目工具而带的、

针对各种系统的蓝图使你可以在几个小时内构建完整的操作系统栈。你可以从下面这些蓝图中选择：针对带有命令行登录的基本系统而构建目标系统镜像的蓝图、针对移动设备而构建带有图像界面的系统的蓝图、构建Linux标准基础兼容的系统的蓝图以及更多蓝图。

你可以使用这些蓝图作为你自己的发行版的起点并且通过增加或删除软件包来修改它们。本书的剩余章节将带你走完使用Yocto项目工具来构建和定制Linux发行版以及创建你自己的蓝图的整个流程，每次在你构建系统的时候，这都会给你可重复的结果。

1.4 关于开源许可的几句话

当构建基于或者包含开源软件的系统时，你将不可避免地必须注意开源许可。软件的创始作者当然可以自由选择针对他们的成果所倾向的任何许可，这已经造成了很长并且仍在增长的开源许可列表。没有单一的许可，并且不管你喜欢还是不喜欢，你将不得不处理很多的许可。一些开源项目甚至会使用不止一个软件许可。这样的例子如BusyBox。

即使不是最常见的开源许可，也是最常见的开源许可之一，就是GNU通用公共许可（General Public License，GPL）[1]。GPL现在是第3版了，它被广泛地认为是开源许可之母。虽然一些来源把创立于1990年的BSD许可作为第一个开源许可，但是通用公共许可却早于其一年（由Richard Stallman撰写并于1989年发布）。

一个归因于开源许可的流行的错误认识是，开源软件是免费的。然而，GNU通用公共许可的第2段对这种常见的错误理解做了澄清：“当我们说‘free software’的时候，我们说的是自由（freedom），而不是价格。”职业的工程经理可能会完全同意——虽然你可以免费下载开源软件，但是开发和部署基于它的产品通常要花费巨大的工程成本。从这个意义上来说，开源软件和商业软件产品没有什么区别。

相对于商业或者闭源软件许可，开源许可是宽容的，这意味着它们给予你使用和运行软件的自由、学习和修改它的权力以及分发原始代码和修改版的许可。这种广泛的自由使得人们很容易漫不经心地对待开源许可。一句话：不要这样做。开源许可是具有约束力和强制执行力的，正如任何商业许可一样。

大部分开源许可明确约定你在交付基于开源代码的产品时需要遵守一些主要条件：

·归因（Attribution）：作者们必须被归因成作品的创作者。你不能去除源代码的任何作者的任何著作权声明。

·传输（Conveyance）：传输通常是指传送源代码的逐字副本、传送源代码的修改版本以及传送非源代码形式（例如二进制文件或者嵌入在产品中的固件）。在后一种情况下，很多开源许可，包括GNU通用公共许可，要求你传送产品对应的源代码或者辅助的文档。

·派生作品（Derivative Works）：这个通常指的是包含了先前创造的作品的全部或者主要部分的创作物。对于开源软件来说，这确切地意味着什么仍然不明确，因为目前还没有针对它的合法的测试用例。大部分情况下，这意味着修改源代码或在其上增加内容，但是，对一些许可来说，也包括连接甚至在运行时动态地连接到库。在许可的条款下，派生作品的作者被要求以和原始作品完全相同的许可条款分发作品。这使得许可可以自我延续（self-perpetuating）。

本书不是以提供有关开源许可的合法建议为目的而写成的。然而，我们强烈建议你在实际交付产品前认真对待包含在你产品中的软件包所用到的许可。虽然开源许可的法律领域还相当新，但是越来越多的法律专家正在专攻这个领域。如果心存疑虑，请向专家们寻求专业建议。

[1] 关于GPL许可的完整文本，请参考附录A或者查看www.gnu.org/licenses/gpl.html。

1.5 组织、相关实体和标准

随着Linux和开源在计算、通信、消费电子、工业自动化以及众多其他领域的市场份额在继续增加，组织和标准正在逐渐形成，这影响着Linux和开源技术本身的接受度和采用度以及它们所代表的开放协作和创新的原则。本节将介绍一些你可能希望熟悉的组织、实体和标准。

1.5.1 Linux基金会

Linux基金会（Linux Foundation，www.linuxfoundation.org）是“非营利性的致力于推动Linux发展的共同体。创立于2000年，Linux基金会赞助Linux创造者Linus Torvalds的工作并且被来自全世界的领先的技术公司和开发者所支持。”

Linux基金会通过以下方式控制它的成员以及开源社区的资源和贡献：

·促进Linux并提供针对协作和教育的中立环境

·保护和支持Linux开发

·改进Linux作为技术平台

Linux基金会直接赞助Linus Torvalds和其他核心Linux开发者的工作以使得他们可以保持独立并且重点关注于改进Linux。Linux基金会也赞助了一些工作组和协作项目以定义标准和推动Linux在特定领域与行业的发展。其中一些项目在稍后的部分会有简要描述。

1.5.2 Apache软件基金会

超过140个开源软件项目是由Apache软件基金会（Apache Software Foundation，ASF）所拥有的。对于这些项目，Apache软件基金会提供了包含经济支持、知识产权管理和法律支持在内的协作框架。Apache软件基金会的网站可以在www.apache.org找到。

你可能已经熟悉了一些最知名的Apache软件基金会的项目，例如Apache HTTP服务器、针对Java的Ant构建工具、Cassandra云数据库、CloudStack云计算基础设施、Hadoop分布式计算平台、针对Java Servlet和JavaServer网页的Tomcat网站服务器。

所有的Apache项目和在Apache软件基金会庇护下产生的软件都遵从Apache许可的条款。Apache许可的重要属性是，贡献者保留在Apache项目外部因为任何目的而使用他们的原始贡献的全部权利，同时给予Apache软件基金会和项目分发以及在其工作之上构建的权利。

1.5.3 Eclipse基金会

Eclipse项目（www.eclipse.org）是2001年由IBM创建的，目的是围绕Eclipse平台为开发者和软件厂商构建支持的社区。Eclipse平台是作为灵活的针对软件开发工具的集成开发环境框架而开始的。在2004年，Eclipse基金会作为控制该项目资源的法律实体而建立。Eclipse基金会为在其庇护下运营的项目提供信息技术基础设施和知识产权管理，以及支持它们的开发和工程过程来保证项目的透明度与产品质量。

除了Eclipse集成开发环境之外，在Eclipse基金会保护下支持的项目列表包括针对几乎任何编程语言的开发工具、软件和数据建模工具、网站开发工具以及更多。

嵌入式软件开发框架频繁地构建在Eclipse集成开发环境上，以期在同一个集成开发环境内提供方便的包括目标调试和分析的往返开发。Yocto项目提供了可直接从集成开发环境内启用Yocto项目创建的工具链的Eclipse插件。

1.5.4 Linux标准基

正如上一节概述的，有很多方式可以构建Linux操作系统栈。虽然灵活性是好的，但是随之而来的是分裂的负担。Linux标准基（Linux Standard Base，LSB）的目标是为Linux发行版确立一套通用标准。通用标准提供给应用开发者这样的保证——他们在Linux发行版上开发的代码将可以不加额外修改地运行在其他Linux发行版上。

另外，当遇到特定Linux发行版的连续性问题时，Linux标准基给了开发者平和的心态。只要发行版的未来版本和特定的Linux标准基的版本保持兼容，应用将会继续运行在这个发行版的未来版本上。

Linux标准基项目提供了一整套规格说明、文档和工具来检测发行版和特定Linux标准基的兼容性。

虽然应用程序编程接口和应用二进制接口兼容性可能并不需要处在对嵌入式系统工程师来说的列表的顶部，但是从长远来看，让你自己熟悉这些概念和规格说明可能有助于嵌入式项目的进行。即使你不希望第三方开发来向嵌入式平台贡献应用，但类似那些Linux标准基的兼容性考虑无疑支持你产品的平台战略。

Linux标准基是Linux基金会工作组。你可以在www.linuxfoundation.org/collaborate/workgroups/lsb找到它的网站。

1.5.5 消费电子产品工作组

消费电子产品工作组（Consumer Electronics Workgroup）是在Linux基金会庇护下运营的工作组。它的任务是推进Linux在用于消费电子产品的嵌入式系统中的应用以及促进Linux本身的增强。消费电子产品工作组在2003年作为消费电子Linux论坛而开始工作，并且在2010年为了更好地和Linux社区一致而与Linux基金会合并。可以在www.linuxfoundation.org/collaborate/workgroups/celf找到消费电子产品工作组的网站。

消费电子产品工作组的主要活动之一是长期支持计划（Long-Term Support Initiative，LTSI）。长期支持计划的目标是创建和维护稳定的、由相关补丁支持大概2～3年的Linux内核树，而2～3年是例如智能手机、游戏控制端和电视机等消费电子产品通常的生命周期。长期支持计划的细节发布在http://ltsi.linuxfoundation.org中。

1.6 总结

嵌入式Linux正在为许多你日常使用的设备和服务提供动力。通常不被注意到的是，它通过互联网路由器引导数据流，把高清图片放在电视机屏幕上，在导航设备内部为旅行者导航，在智能仪表中测量能源消耗，在路边传感器中收集交通信息以及更多。Linux和开源正在为从互联设备到网络基础设施及数据处理中心的物联网提供动力。本章为即将到来的材料准备了条件，覆盖以下主题：

·工程师视角的嵌入式系统的定义以及把嵌入式产品从设计带到生产中所关联的一系列责任

·致力于针对嵌入式设备的Linux快速采用的技术开发

·嵌入式Linux概览

·与构建和维护操作系统栈相关联的挑战

·针对嵌入式项目的开源许可的重要性

·与嵌入式Linux有关的一些组织和标准

1.7 参考文献

Apache许可，www.apache.org/licenses。

Apache基金会，www.apache.org。

Eclipse基金会，www.eclipse.org。

GNU通用公共许可，www.gnu.org/licenses/gpl.html。

Linux基金会，www.linuxfoundation.org。

Linux标准基，www.linuxfoundation.org/collaborate/workgroups/lsb。

第2章　Yocto项目

Yocto是国际单位制（International System of Units）（简写为SI,来自法语Le Système International d’Unités）指定的测量单元的最小部分。它命名了Yocto项目——用于构建针对嵌入式设备的定制的Linux发行版的一套综合的工具套件、模板和资源。说这个名字是轻描淡写，也确实是轻描淡写。

在本章中，我们通过设置由Yocto项目在Poky参考发行版中提供的OpenEmbedded构建系统，以及构建我们的第一个完全依赖于Poky默认提供的蓝图上的Linux操作系统栈来直接切入主题。我们在本章中执行的任务为接下来的章节布景，在接下来的章节里，我们分析Yocto项目的各个方面，从Poky工作流到OpenEmbedded构建系统（包括BitBake构建引擎），到定制操作系统栈，再到板支持包（Board support package，BSP）和应用开发工具包，以及更多。

我们以Yocto项目和OpenEmbedded的关系以及Yocto项目术语结束本章。

2.1 启动第一个Yocto项目构建

自己动手——或者从做中学习——无疑是获得新技能的最好方式。因此，我们通过构建第一个用于QEMU（Quick Emulator，一个适用于不同CPU架构的通用的开源机器模拟器）的Linux操作系统栈开始。

本章学习如何准备计算机以成为Yocto项目开发主机、获取和安装构建系统、设置和配置构建环境、启动和监控构建过程，以及最后通过在QEMU模拟器中启动新构建的Linux操作系统栈来验证构建结果。

接下来的部分简要描述了对Yocto项目主机来说的硬件和软件先决条件。如果不希望立即设置构建主机，Yocto项目提供构建器具（build appliance），它是在虚拟机中预先配置的系统，使得你能够在不安装任何软件的情况下尝试Yocto项目工具。2.1.7节简要描述了如何用Yocto项目的构建器具来进行测试。

2.1.1 先决条件

你可能已经猜到了：为了用Yocto项目工具构建Linux系统，需要一个运行着Linux的构建主机。

硬件条件

尽管Yocto项目工具具备构建Linux操作系统栈的能力，但是其需要带有x86架构CPU的构建主机。32位和64位CPU都是被支持的。出于吞吐量的原因，带有64位CPU的系统是首选的。Yocto项目的构建系统在可能的情况下利用并行处理（parallel processing）。因此，带有多个CPU或者一个多核CPU的构建主机能极大地减少构建时间。当然，CPU时钟速度也对包能够多快构建出来有影响。

内存也是重要因素。BitBake——Yocto项目的构建引擎，解析数千个“菜谱”（recipe）并且创建带有构建依赖项的缓存。而且，编译器需要内存来存储数据结构和更多其他数据。Yocto项目工具在小于1GB内存的系统上不能运行。推荐的是4GB或者更多内存。

磁盘空间也是考虑因素。当前，完整的构建过程——创建基于X11的带有图形用户界面（Graphical User Interface，GUI）的镜像——消耗大约50GB的磁盘空间。如果你想在将来为更多架构构建或向构建增加更多的包，那么需要额外的空间。推荐的是，系统的硬盘有至少100GB的可用空间。因为大容量的普通硬盘的价格已经是普遍用户可承受的了，推荐你用带有500GB或者更多空间的主机来承载Yocto项目的全部构建环境。

因为构建系统从磁盘读取大量数据并向磁盘写入大量构建输出数据，带有高I/O吞吐率的磁盘也可以极大地加速构建过程。使用固态盘

可以进一步改善构建体验，但是，这些设备，特别是具有大容量的，在成本上是远高于具有旋转盘片的普通磁盘的。不管你正在使用传统的硬盘还是固态盘，额外的性能增加可以通过设置独立磁盘冗余阵列（Redundant Array of Independent Disks，RAID）来实现，例如RAID 0。

互联网连接

可从Yocto项目网站获取的OpenEmbedded构建系统仅仅包含构建系统本身——BitBake和引导它的元数据。对于它将要构建的软件来说，它不包含任何源包。这些源包是在构建运行时按需自动下载下来的。因此，你需要一个接至互联网的在线连接，最好是高速连接。

当然，下载的源包是存储在系统上并且对于将来的构建来说可以重用。你也可以提前下载所有的源包然后在没有接至互联网的连接的情况下离线地构建它们。

软件需求

首先，你需要当前的Linux发行版。随着每次版本发布，Yocto项目正在持续认可越来越多的发行版。使用以下发行版之一的前一个或者当前发布通常没有任何问题：

·CentOS

·Fedora

·openSUSE

·Ubuntu

一般情况下，32位和64位变体都是验证过的。然而，如果硬件支持

64位版本，那么就推荐使用64位版本。可以在位于www.yoctoproject.org/docs/current/ref-manual/ref-manual.html的Yocto项目参考手册中找到所有被支持的发行版的详细列表。

除了Linux发行版，你需要安装一系列软件包以使构建系统运行。我们会在2.1.3节阐述安装。

2.1.2 获取Yocto项目工具

对你来说，获取Yocto项目工具有很多方式，或者更准确地说，Yocto项目参考发行版Poky：

·从Yocto项目网站下载当前发布。

·从发布仓库下载当前发布或者先前发布的版本。

·从自动构建器（Autobuilder）仓库下载一个当前的每夜构建。

·从由Yocto项目Git仓库服务器承载的Poky Git仓库中克隆当前开发分支或者其他分支。

每6个月，在每年4～5月和10～11月时间段内，Yocto项目团队发布构建系统的新的主版本（major version）。所有已发布的Yocto项目工具都经过了多轮的质量保证和测试。它们是稳定的，并且还附带了发布说明和更新过的描述特性的文档集。对Yocto项目新手来说，推荐使用当前的稳定发布版本。

解决了问题但没有增加新特性的次版本（minor version）于6个月发布周期中间的必要时刻提出。因为没有新特性，所以文档通常不会随着次版本的发布而变化。

以前的主版本和次版本被归档并且可以从下载仓库中下载。有时，新的主要发布引入新的层结构、新的配置文件或配置文件中新的配置项。因此，迁移现有的构建环境到更新的发布版本可能需要付出努力。留在前一个版本使得你可以延迟或者彻底取消迁移。

每夜构建追踪Yocto项目Git仓库中代码基的当前开发状态。这些构

建接受了基本的质量保证和自动构建器测试。它们的测试没有像正常的主发布和次发布那样严格，但是你至少可以获取这样的信心——核心功能是可运行的。

从Poky Git仓库克隆当前的开发分支（主干分支）可让你对当前状态的开发工作直接访问。对这个分支的修改除了开发人员在他们提交签字前做过的测试以外，没有任何其他测试。虽然质量通常是高的，并且任何严重的核心功能破坏通常可以在开发人员迁进变更后的短时间内检测出来，但是很有可能，系统不能像期待的那样工作。除非你直接参与Yocto项目开发，否则没有必要直接工作在主干分支上。

除了主干分支，Poky的Git仓库还包含了里程碑分支、对各个版本的开发分支和长的标签（索引到各个分支的特定修订）列表。

在接下来的章节里，我们简要描述如何从各个地方下载Yocto项目的发布。我们也会细致地探索针对Poky的Yocto项目的Git仓库、板支持包、Linux内核以及更多内容。

下载当前的Poky发布版本

导航到https://www.yoctoproject.org/downloads，然后在Poky的最新发布版本处单击。这个链接把你转到详细的、存在指向到各个下载服务器和镜像的链接的下载网站。该网站也包含发布信息和勘误。

下载发布版本会把名字为poky-<codename>-<release>.tar.bz2的Poky参考发行版压缩包放在系统上。

2.1.3 设置构建主机

设置构建主机需要安装额外的软件包。所有4个主流的Linux发行版的包仓库中都包含这些直接可用的包。然而，在作为该发行版的默认配置下哪些包被预先安装方面，这些发行版是有区别的。

安装完额外的包后，需要解压Poky压缩包——它包括所有必要的配置数据、菜谱、便利性脚本和BitBake。

BitBake需要2.6或者2.7主版本的Python。BitBake当前不支持新的Python 3（Python 3引入了打破向后兼容的语言语法和新的库）。

安装额外的软件包

使用什么命令和安装什么额外的包取决于安装在构建主机上的Linux发行版。

为了在CentOS构建主机上安装必要的包，使用程序清单2-1的命令。

程序清单2-1　CentOS

```
user@centos:~$ sudo yum install gawk make wget tar bzip2 gzip \
python unzip perl patch diffutils diffstat git cpp gcc gcc-c++ \
glibc-devel texinfo chrpath socat perl-Data-Dumper \
perl-Text-ParseWords perl-Thread-Queue SDL-devel xterm
```

为了设置Fedora构建主机，执行程序清单2-2中的命令。

程序清单2-2　Fedora

```
user@fedora:~$ sudo dnf install gawk make wget tar bzip2 gzip python \
unzip perl patch diffutils diffstat git cpp gcc gcc-c++ glibc-devel \
texinfo chrpath ccache perl-Data-Dumper perl-Text-ParseWords \
perl-Thread-Queue socat findutils which SDL-devel xterm
```

程序清单2-3显示了针对openSUSE构建主机的安装命令。

程序清单2-3　openSUSE

```
user@opensuse:~$ sudo zypper install python gcc gcc-c++ git chrpath \
make wget python-xml diffstat makeinfo python-curses patch socat \
libSDL-devel xterm
```

在Ubuntu构建主机上，执行程序清单2-4的命令。

程序清单2-4　Ubuntu

```
user@ubuntu:~$ sudo apt-get install gawk wget git-core diffstat \
unzip texinfo gcc-multilib build-essential chrpath socat \
libsdl1.2-dev xterm
```

在安装成功以后，你可能希望验证正确版本的Python被安装了：python--version。输出应该显示2.6或者2.7的主版本号。

安装Poky

安装Poky时仅仅需要解压先前从Yocto项目网站下载下来的压缩包。推荐你在家目录中为所有有关Yocto项目构建的东西创建子目录。程序清单2-5显示了必要的步骤。

程序清单2-5　安装Poky

```
user@buildhost:~$ mkdir ~/yocto
user@buildhost:~$ cd ~/yocto
user@buildhost:~$ tar xvfj <downloadpath>/poky-<codename>-release.tar.bz2
```

现在构建系统已经准备好，可以设置构建环境和创建第一个Linux

操作系统栈了。

2.1.4 配置构建环境

Poky提供了脚本oe-init-build-env来创建新的构建环境。这个脚本设置构建环境的目录结构和初始化核心集合的配置文件。它也设置一系列构建系统所需要的shell变量。你不直接执行oe-init-build-env脚本而是使用source命令来把shell变量设置输出到当前的shell：

```
$ source <pokypath>/oe-init-build-env <builddir>
```

执行这个命令在当前目录中以由参数<builddir>提供的名字创建新的构建环境。你可以省略这个参数，那么脚本会使用默认的build作为名字。在设置了构建环境后，脚本改变目录到构建目录。

使用程序清单2-6所示形式的脚本来创建新的构建环境并且初始化先前创建的现有的构建环境。在创建新的构建环境时，脚本提供给你一些指示。

程序清单2-6　新构建环境设置

```
You had no conf/local.conf file. This configuration file has therefore been
created for you with some default values. You may wish to edit it to use a
different MACHINE (target hardware) or enable parallel build options to take
advantage of multiple cores, for example. See the file for more information, as
common configuration options are commented.

The Yocto Project has extensive documentation about OE including a reference
manual which can be found at:
    http://yoctoproject.org/documentation

For more information about OpenEmbedded see their website:
    http://www.openembedded.org/

You had no conf/bblayers.conf file. The configuration file has been created
for you with some default values. To add additional metadata layers into your
configuration, please add entries to this file.
The Yocto Project has extensive documentation about OE including a reference
manual which can be found at:
    http://yoctoproject.org/documentation

For more information about OpenEmbedded see their website:
    http://www.openembedded.org/

### Shell environment set up for builds. ###

You can now run 'bitbake <target>'

Common targets are:
    core-image-minimal
    core-image-sato
    meta-toolchain
    meta-toolchain-sdk
    adt-installer
    meta-ide-support

You can also run generated qemu images with a command like 'runqemu qemux86'
```

在新创建的构建环境里面，该脚本增加了目录conf并且把两个配置文件放在里面：bblayers.conf和local.conf。我们会在第3章中详细解释bblayers.conf。现在仅仅看看local.conf，它是构建环境的主要配置文件。

在local.conf中，设置各个变量以影响BitBake构建自定义的Linux操作系统栈。你可以修改设置并且可以向文件中加入新的设置以覆盖在其他配置文件中做出的设置。通过贯穿本书的各个例子，我们会解释这种集成以及如何使用它。对于我们的第一个构建来说，我们聚焦在少量设置上，剩余的设置采用默认值。如果在文本编辑器中打开local.conf文件，那么你将发现在程序清单2-7中显示的设置（在众多里面，它包括

从这个程序清单里面清除的注释行）。

程序清单2-7　conf/local.conf

```
BB_NUMBER_THREADS ?= "${@bb.utils.cpu_count()}"
PARALLEL_MAKE ?= "-j ${@bb.utils.cpu_count()}"
MACHINE ??= "qemux86"
DL_DIR ?= "${TOPDIR}/downloads"
SSTATE_DIR ?= "${TOPDIR}/sstate-cache"
TMP_DIR = "${TOPDIR}/tmp"
```

以井字符（#）开头的行是注释。如果有变量设置的行的行首有井字符，那么需要去除井字符以使得设置生效。显示的值是默认值。BitBake使用这些值，即使你没有明确启用它们。在程序清单2-7中显示的变量配置是在创建了新的构建环境以后通常希望改变的变量配置。它们描述在表2-1中。

表2-1　配置变量

变量	默认值	描述
BB_NUMBER_THREADS	${@bb.utils.cpu_count()}	并发的 BitBake 任务的数量
PARALLEL_MAKE	-j ${@bb.utils.cpu_count()}	并发的 Make 进程数量
MACHINE	qemux86	目标机器
DL_DIR	${TOPDIR}/downloads	源下载被存放的目录
SSTATE_DIR	${TOPDIR}/sstate_cache	共享的状态缓存文件的目录
TMP_DIR	${TOPDIR}/tmp	构建输出的目录

两个并发选项BB_NUMBER_THREADS和PARALLEL_MAKE的默认值是以使用所有可用核心的系统中的CPU核心数量为基础自动计算出来的。你可以通过将这些值设置得比你系统中的核心数量少来限制负载。使用比物理核心数更大的值是可能的，但是不加速构建过程。BitBake和Make对应地派生更多线程，但是它们仅仅在CPU核心可用的情况下运行。永远不要忘记变量设置前后的引号。也要注意到，对于PARALLEL_MAKE，应该包含-j，例如“-j　4”，因为这个值会被一字不差地传递到make命令。

设置MACHINE选择BitBake构建Linux操作系统栈所针对的目标机器类型。Poky提供了一系列针对QEMU和一些实际硬件板目标机器的标准机器。板支持包可以提供额外的目标机器。对于我们的第一个构建来说，选择qemux86——带有x86 CPU的模拟的目标机器。

变量DL_DIR告诉BitBake把源下载放在什么地方。默认的设置把文件放在构建环境顶级目录下的downloads目录中。TOPDIR变量包含到构建环境的完整（绝对）路径。源下载可以在多个构建环境之间共享。如果BitBake检测到源下载在下载目录中已经是可用的了，那么BitBake不会再次下载它。因此，推荐你设置DL_DIR变量来指向到构建环境之外的目录路径。当不再需要特定的构建环境时，可以简单地删除它而不会删除所有的源文件下载。

对于SSTATE_DIR也是这样的，它包含到共享状态缓存的路径。OpenEmbedded在处理构建了组成Linux操作系统栈的包所必需的很多任务中产生大量中间输出。和源下载类似，中间输出对将来的构建来说是可以重用的并且在多个构建环境之间共享以加速构建过程。默认情况下，配置把共享状态缓存目录放在构建环境顶级目录下面。建议你把这个设置改成构建环境之外的路径。

变量TMP_DIR包含到BitBake执行所有构建工作和存储构建输出的目录的路径。因为在这个目录下存储的输出是非常特定于构建环境的，所以保留它作为到构建环境的子目录是有意义的。在该目录下存储的数据量可能会最终占用很多GB的硬盘空间，因为它包含解压出来的源下载、交叉编译工具链、编译输出和针对目标机器的内核镜像以及根文件系统。

为了在构建中保留磁盘空间，可以增加

```
INHERIT += rm_work
```

它命令BitBake在构建完包之后删除针对构建包的工作目录。

2.1.5 启动构建

为了启动构建，从构建环境的顶级目录调用BitBake指定构建目标：

```
$ bitbake <build-target>
```

在接下来的章节里，我们将深入到什么是构建目标以及如何使用它们来控制构建输出的细节。对于我们的第一个构建来说，我们使用创建了带有图形用户界面的完整Linux操作系统栈的构建目标。从在前面部分创建和配置过的构建环境的顶级目录中，执行以下命令：

```
$ bitbake core-image-sato
```

core-image-sato目标创建针对移动设备的带有用户界面的根文件系统镜像。依赖于构建硬件和用于下载源文件的互联网的速度，构建可能会花费1到数个小时不等的时间。

也可以命令BitBake先下载所有源而不构建，用以下的命令实现：

```
$ bitbake -c fetchall core-image-sato
```

在下载完成后，可以断开构建系统与互联网的连接，在以后的时间里离线运行构建。

如果BitBake遇到了它无法恢复的错误，那么通常会立即终止构建过程。然而，你可以命令BitBake继续构建，即使它遇到了错误，只要剩余的任务并不被这个错误所阻碍就行：

```
$ bitbake -k core-image-sato
```

-k选项告诉BitBake继续构建直到不依赖这个错误的任务被处理。

2.1.6 验证构建结果

因为我们的目标机器是模拟的系统，所以可以通过启动QEMU模拟器来验证构建结果。对此，Poky提供了一个便利的脚本，该脚本准备QEMU执行环境并用合适的内核和根文件系统镜像来启动模拟器：

```
$ runqemu qemux86
```

在最简单的形式下，runqemu脚本以目标机器名字作为参数被调用。然后，它在构建输出中自动为目标寻找合适的内核和根文件系统镜像。你必须输入系统管理员（或者sudo）密码以使脚本去设置虚拟网络接口。图2-1显示了运行中的系统。

你可以通过单击Utilities屏幕中的Shutdown按钮来终止QEMU虚拟机。其按照关机顺序来恰当地关闭系统。或者是，你可以简单地在你启动QEMU的终端上输入Ctrl-C。

图2-1　带有core-image-sato目标的QEMU

2.1.7 Yocto项目构建器具

如果你仅仅想尝试Yocto项目和Poky而不设置Linux构建主机，那么可以使用Yocto项目构建器具。构建器具是完整的Yocto项目构建主机，包括带有OpenEmbedded构建系统和Poky需要的已安装的软件包的Linux操作系统，被捆绑为虚拟机镜像。它甚至包括了所有的源包下载，能加速第一个构建和使你能够在没有网络连接的情况下离线地构建。

构建器具可从Yocto项目网站https://www.yoctoproject.org/download/build-appliance-0下载。构建器具以ZIP压缩包的形式提供，需要在下载后于系统上把解压。

为了使用构建器具，需要使用在计算机上安装的VMWare Player或者VMWare Workstation。从VMWare网站www.vmware.com的下载部分，可以获取匹配计算机上的操作系统的VMWare Player或者VMWare Workstation。遵循由VMWare提供的安装指南。

一旦安装了VMWare Player或者VMWare Workstation，便可知在https://www.yoctoproject.org/documentation/build-appliance-manual上的构建器具手册提供了详细的、关于如何配置虚拟机和启动构建器具的指南。

如图2-2所示，启动构建器具时直接启动了针对BitBake的Hob图形用户界面。

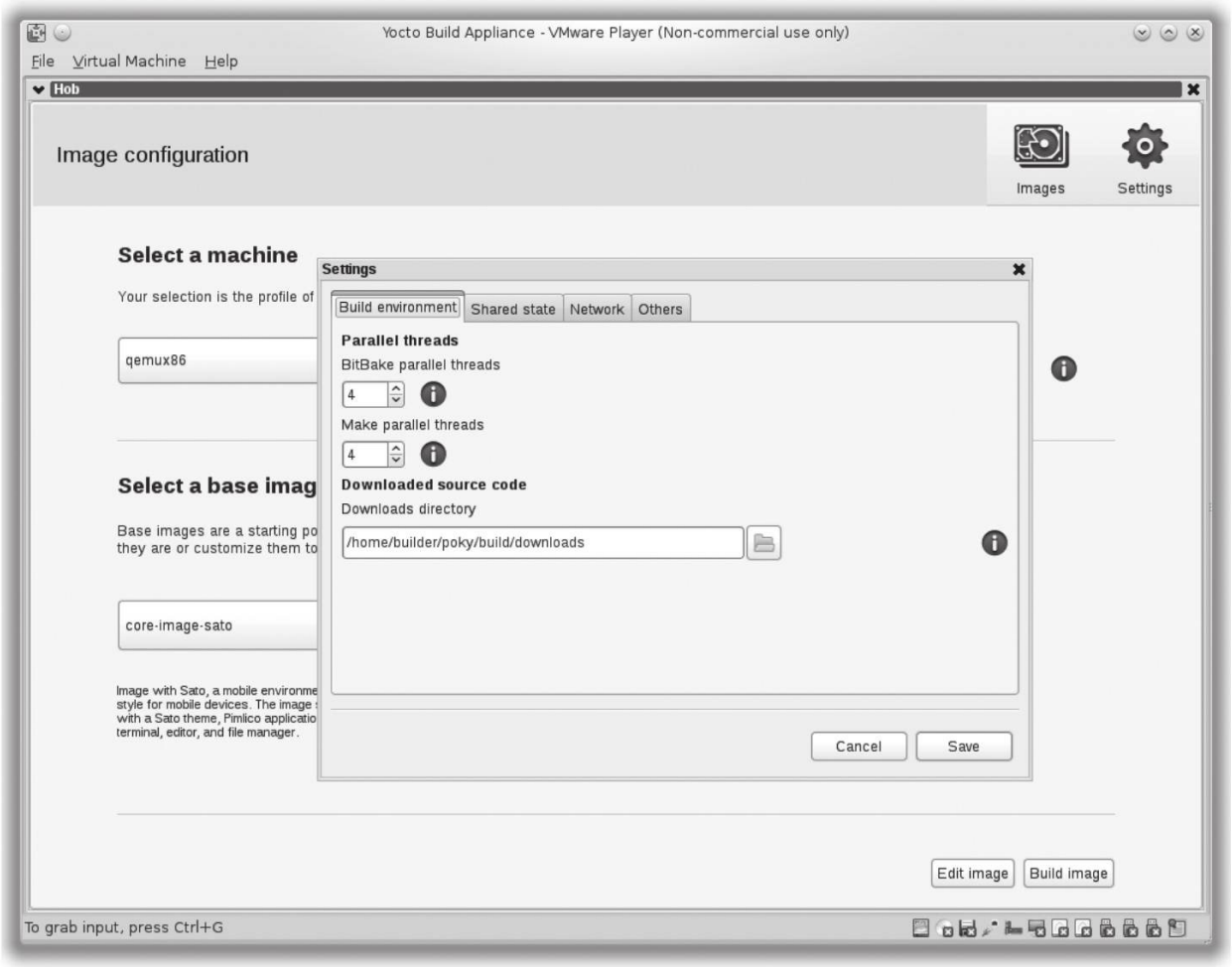

图2-2　Yocto项目构建器具

从下拉框中针对机器选择qemux86、针对基础镜像选择core-image-sato，然后开始构建。取决于主机系统和虚拟机配置，它可能要花费数个小时来构建镜像。你可以从Hob的Log屏幕上观察构建过程。Log屏幕显示了在运行队列中被分成独立任务的那些要构建的包。当前运行的任务被高亮显示。

在构建完成后，你可以直接从Hob中用QEMU模拟器启动镜像。

2.2 Yocto项目系列

Yocto项目不仅仅是单一的开源项目而是在一个名字下集合了多个项目。你已经见过了该项目系列中的最重要的成员：OpenEmbedded构建系统，它包括BitBake、OpenEmbedded核心和Poky（Yocto项目参考发行版）。

基本上，所有系列成员都支持OpenEmbedded构建系统。Yocto项目团队和OpenEmbedded项目（另一个组织）一起维护构建系统。随着构建系统进化，新功能被增加到子项目中。

表2-2提供了作为Yocto项目部分维护的子项目的概览。

表2-2　Yocto项目系列

内部项目	描述
应用开发工具（Application Development Toolkit，ADT）	应用开发工具为用户空间应用程序提供了运行在由 Poky 构建的操作系统栈上的完整的开发环境。应用开发工具包含交叉开发工具链、QEMU 模拟器、Linux 内核和根文件系统镜像。Poky 直接使用它的配置文件从构建环境内创建和打包应用开发工具
AutoBuilder	AutoBuilder 通过使构建系统和 Buildbot 集成来自动化构建。Yocto 项目质量保证团队使用 AutoBuilder 持续集成和用一套标准构建目标回归测试
BitBake	BitBake 是 OpenEmbedded 构建系统的构建引擎。BitBake 是类似 Make 或者 Ant 的构建工具，它聚焦在构建软件包和操作系统栈
构建器具	构建器具是作为 VMWare 虚拟机镜像打包的在 Ubuntu Linux 之上的完整自包含的 Poky 构建系统。它是一个便捷的方法，用来尝试 Poky 而不用安装 Linux 构建主机并且在上面设置 Poky
交叉预先链接（cross-prelink）	共享库的内存地址位置通常是在执行时间内当库被第一次加载进内存时计算出来的。每一次运行依赖于共享库的程序时，加载程序（loader）需要在内存中定位库。因为库可以在内存中移动，所以这个任务造成了性能代价——增加了需要被解析的共享库的数量。预先链接（prelinking）通过预先计算地址来执行可行性程序的动态链接以减少开销。通过模仿运行时链接器，交叉预先链接作为交叉开发工具链的部分执行这个过程
Eclipse 集成开发环境插件	针对 Eclipse 集成开发环境的 Yocto 项目插件把应用开发工具集成进 Eclipse，提供用于用户空间应用的交叉开发工作流。这个插件提供针对将被模拟的或硬件目标所用的交叉工具链以及根文件系统的集成
EGLIBC	EGLIBC 是 GNU C 库（GLIBC）的嵌入式版本。它和 GLIBC 提供了相同的应用程序编程接口（API），并且努力使自己和 GLIBC 在源代码和二进制方面兼容，但是它是针对嵌入式系统优化的。优化包括减少的内存占用、对交叉编译和交叉测试的优化的支持以及可配置的组件。虽然 EGLIBC 已经被合并进了 GLIBC 并且结束了生命周期，但是在这里提到它是为了提供上下文和历史
Hob	Hob 是针对 BitBake 的图形用户界面。Hob 允许从方便的图形用户界面而不是编辑文件来进行构建环境的配置、包选择以及镜像配置。构建过程可以直接从 Hob 启动并且被监控。Hob 是被 OpenEmbedded 核心包含的层。最终，Yocto 项目团队将停止 Hob 而倾向于 Toaster
Matchbox	Matchbox 是定位于嵌入式设备的、基于 X Window 系统的开源窗口管理器。它和传统桌面窗口管理器不同的是，它同一时间仅仅展示一个充满整个屏幕的窗口。这个特性使得它适合小规格设备
OpenEmbedded 核心	OpenEmbedded 核心（OpenEmbedded core）是在 OpenEmbedded 构建系统中的核心的元数据集合。它由在 OpenEmbedded 驱动的系统中共享的 BitBake 层、类、集成和实用脚本的集合组成。OpenEmbedded 核心是由 OpenEmbedded 项目和 Yocto 项目共同维护的
Poky	Poky 是 Yocto 项目的参考发行版。它提供了一套预先配置的嵌入式 Linux 操作系统栈的蓝图作为工作示例，这些蓝图可以用来引导实际系统开发
Pseudo	构建软件包频繁以系统管理员的身份要求系统的根文件系统安装文件、改变文件属主或者访问许可、创建设备节点等操作。Pseudo 是提供虚拟化环境的应用，它允许这种操作可以成功执行，就像用户有系统管理员权限一样，即使他是普通用户

（续）

内部项目	描述
Swabber	大部分软件包被配置成在原生系统上构建。在交叉开发环境中频繁构建它们要求配置变化并且带来了风险——构建使用了来自构建主机的组件而不是来自交叉构建环境的组件。例如，在 make 文件中的路径指向了主机系统的文件而不是在交叉构建环境中的文件。这样的交叉构建的主机污染可以导致难以检测的、不仅仅在构建时而且在执行时的系统失败。Swabber 提供了用于检测越过构建沙盒边界的交叉构建访问的机制
Toaster	Toaster 是针对 BitBake 和构建系统的新的图形用户界面。它是基于网页的，可以被部署成带有远程访问的分布式构建服务

虽然在Yocto项目的子项目间有紧密的集成，但是开发者确保没有交叉的依赖且子项目是可以互操作的，并且其也可以独立于构建系统使用。

2.3 历史概览

OpenEmbedded和Yocto都植根于OpenZaurus项目——致力于提高第一个基于Linux的个人数码助理（personal digital assistant）（夏普Zaurus SL-5000D）代码的开源项目。SL-5000D，第一次发布于2001年，是目标定位于开发者的设备，并且夏普提供了必要工具来修改和更新设备的ROM代码。起初，这个项目聚焦在重新打包现有ROM代码以使得它对开发者更加友好。随着时间推移，这个项目进化了，起初的夏普代码被从源代码构建的基于Debian的Linux发行版完全替换。它迅速发展了构建系统，使得它对于创建新设备的项目和发行版独立的构建系统来说是必要的。OpenEmbedded项目诞生了。

2.3.1 OpenEmbedded

在2003年，OpenEmbedded项目通过整合OpenZaurus项目的成果和来自其他类似目标的嵌入式Linux项目（例如Familiar Linux和OpenSIMpad项目）的贡献而启动。

OpenEmbedded项目维护了构建系统以及描述如何构建软件包和组合操作系统镜像作为通用代码基的元数据。加到元数据目录中的包数量迅速增长到构建5000个包的2100个菜谱。

在2005年，项目团队决定拆分这个项目成BitBake构建系统和OpenEmbedded元数据。

OpenEmbedded从使用它作为构建系统的各种Linux发行版获得了支持。其中就有Ångström发行版、Openmoko、WebOS和其他。商业团体让他们的产品采用该系统，其中有MontaVista软件和OpenedHand——开发了Poky Linux发行版的创业公司。

2.3.2 BitBake

BitBake，位于OpenEmbedded和Yocto项目的Poky参考发行版核心的构建引擎，是从Portage（Gentoo Linux的构建和包管理系统）派生而来。Portage包含两个组件：

·ebuild是处理从源代码构建软件包并安装它们的实际的构建系统。

·emerge是到ebuild的接口以及管理ebuild包仓库、解决依赖性和更多工作的工具。

所有的Portage都是用Python写的。BitBake从Portage进化而来，BitBake为使用原生和交叉开发工具链构建软件包而扩展了Portage、支持多个包管理系统和其他针对交叉构建所必需的其他功能性。

BitBake使用和Portage构建脚本相同的元数据语法，但是引入了新的特性，例如由类提供的继承机制、追加（appending）菜谱和全局配置文件等。

2.3.3 Poky Linux

OpenEmbedded极大地简化了为嵌入式设备（但不限于此）构建Linux操作系统栈的过程。然而，对于修改和调整系统来创建不同的发行版和移植系统到新的硬件来说，它仍然是一个带有相当陡峭的学习曲线的挑战。

软件创业公司OpenedHand为内部使用开发了Poky Linux——多功能的开发平台和针对移动设备的Linux发行版。Poky Linux为该公司应用于嵌入式设备的Matchbox窗口管理器提供了测试平台。Matchbox最显著地被Nokia 770和N800平板设备、Openmoko的Neo1973和“每个儿童一台笔记本”项目（One Laptop Per Child，OLPC）的XO笔记本电脑所使用。

Poky Linux是由OpenEmbedded构建的，它提供了更加直观的方法来为目标设备配置操作系统镜像。它也提供了一些针对易于调整的目标设备镜像的蓝图。因为Poky Linux是开源的，所以它迅速地被其他人采用以便构建嵌入式设备。

英特尔公司在2008年收购了OpenedHand，目的是进一步发展Poky Linux作为用于嵌入式设备的通用的发行版。

2.3.4 Yocto项目

为了构建Poky Linux来支持许多不同的架构和硬件平台，英特尔那时一直在寻找其他的商业团体——特别是其他的半导体生产商和嵌入式Linux公司——来支持这个项目并且为其做贡献。因为英特尔是芯片市场上的主导玩家，所以这证明对英特尔来说，让它的竞争对手和其他公司来支持它改进Poky Linux是困难的。

在2010年，英特尔与Linux基金会接洽，希望在基金会的赞助下，建立中立管理的协作项目，涉及开源社区，特别是OpenEmbedded项目。

Linux基金会在2010年10月26日公开宣布Yocto项目启动。在2011年3月1日，Linux基金会宣布Yocto项目与OpenEmbedded的技术对准（alignment）以及多个公司协作者对该项目的支持。这个声明被2011年4月6日的另一个报道所跟进，主要内容包括Yocto项目指导组（Yocto Project Steering Group）的信息和第一个Yocto项目发布。

2.3.5 OpenEmbedded和Yocto项目的关系

在OpenEmbedded和Yocto项目之间的技术对准对这两个项目都带来了一些主要的改进：

·对准开发（aligned development）：在开源项目中常见的问题是分裂——有相同的根源和类似目标的两个项目派生并且产生隔阂。资源被分割并且最终在两个分支上重复努力，提供类似的功能。最终，用户和支持者被强制在这两个努力之间做出选择。OpenEmbedded和Yocto项目的紧密对准保证了用户可以获得这两个项目的好处。

·BitBake元数据层：元数据层能使菜谱和配置文件逻辑分组成可以被轻松纳入并迁移到不同构建环境的结构。元数据层也简化了依赖管理，这是在构建操作系统栈时的复杂任务。

·OpenEmbedded核心元数据层（openEmbedded core metadata layer）：OpenEmbedded和Yocto项目开发团队同意创建这两个项目共享并且包含所有基准菜谱和配置设定的通用的元数据层。然后，每个项目根据其目标增加额外的元数据层。

虽然OpenEmbedded和Yocto项目之间有紧密的协作，但是这两个项目是独立的实体。它们都是开源项目并且都受到了由开源开发者和商业团体组成的社区的支持。

OpenEmbedded聚焦在尖端技术、菜谱和一大套针对不同硬件平台的板支持包。Yocto项目聚焦在构建系统本身和针对交叉开发的工具化上。Yocto项目的目标是提供强大却简单易用并且测试良好的工具以及核心的元数据集合，以启动嵌入式系统开发。额外的板支持包和其他组件是通过OpenEmbedded和Yocto项目生态系统来提供的。

OpenEmbedded项目也维护了层的索引，这是可检索的层、菜谱和机器的数据库。要寻找构建特定的开源包的菜谱，向层索引输入名字，可能的情况是某个人已经为它创建了菜谱。

2.4 Yocto项目术语

表2-3定义了一套经常和Yocto项目联合使用并且贯穿于Yocto项目的术语。贯穿本书，我们一致性地按照这里提供的定义来使用这些术语。

表2-3 Yocto项目术语

术语	描述
追加文件（append file）	追加文件扩展现有的菜谱。BitBake 逐字追加该追加文件的内容到对应的菜谱，在解析前创建单一文件。追加文件中的变量可以覆盖对应菜谱中定义的相同的变量。追加文件使用 bbappend 后缀
BitBake	作为 OpenEmbedded 构建系统中的构建引擎，BitBake 是任务执行器和调度器。它的输入是元数据（例如配置文件和菜谱），通过这些元数据，BitBake 处理被控制
板支持包（Board Support Package，BSP）	板支持包中的文档、二进制、代码和其他特定于实现的支持数据使一个特定的操作系统运行在特定的目标硬件系统上。有时，板支持包也会包含完整的根文件系统和交叉开发环境，以创建运行在目标硬件平台上的应用程序
类（class）	BitBake 术语中，类是提供逻辑封装和基本继承机制，使得通常用到的模式被定义一次而在多个菜谱中使用的元数据文件。BitBake 类文件使用 bbclass 后缀
配置文件	配置文件是提供影响构建过程的全局定义和变量设置的 BitBake 元数据文件
交叉开发工具链（cross-development toolchain）	交叉开发工具链是一套软件开发工具，它允许针对目标系统的软件开发利用不同的架构而不是开发主机。在这个上下文中，架构是指不同的 CPU 指令集（例如，ARM、MIPS、PowerPC、x86）和不同的位大小（例如，8 位、16 位、32 位和 64 位）。通常，交叉开发工具链包括一个或者多个语言编译器、汇编程序、链接器，以及特定于目标架构的调试器、模拟器和其他工具

（续）

术语	描述
镜像（image）	一个二进制文件，通常是压缩形式的，镜像包含引导加载程序（bootloader）、操作系统内核以及将被复制到存储介质并且目标系统可以由此启动和运行的根文件系统。术语“镜像”也可以仅用于表示操作系统内核（内核镜像）或者根文件系统（根文件系统镜像）
层（layer）	在 BitBake 术语中，层是组织进文件和目录结构的元数据（配置文件、菜谱等）集合。BitBake 可以包含层来扩展它的功能。Yocto 项目板支持包也是作为层来提供的
元数据（metadata）	在 BitBake 术语中，元数据包括所有命令 BitBake 如何执行构建过程的文件。BitBake 元数据包括类、菜谱（和追加文件）以及配置文件
OpenEmbedded 核心	作为在 OpenEmbedded 构建系统中由 OpenEmbedded 和 Yocto 项目共享的一套核心元数据，OpenEmbedded 核心是由 OpenEmbedded 项目和 Yocto 项目共同维护的 BitBake 层
包（package）	包是软件包，其中包含可执行的二进制文件、库、文档、配置信息和其他文件，这些文件遵循“操作系统的包管理系统可以安装或卸载”的特定格式。包通常也包括与包管理系统用来自动解决或通知用户的其他软件包互相依赖和不兼容的相关信息。 Yocto 项目还使用术语“包”来表示用于建立各自的软件包的菜谱和其他元数据。依赖于上下文，该术语指的是实际的软件包或者构建软件包的元数据
包管理系统（package management system）	包管理系统是一套软件工具集合，这些工具用于为计算机操作系统自动化安装、升级、配置和删除软件包的过程。它通常维护计算机上已安装的软件的数据库，包括版本信息、依赖、不兼容性以及更多，以防止由软件不匹配和缺失的先决条件而导致的系统故障
Poky	Poky 是 Yocto 项目的参考发行版，也是由构建系统创建的默认 Linux 发行版的名字。Poky 下载包包含 OpenEmbedded 构建系统和用于创建名字为 Poky 的样例嵌入式发行版的额外的元数据
菜谱（recipe）	菜谱是包含用于 BitBake 构建特定软件包的指令的元数据文件。通过它的指令，菜谱描述从哪里获取源代码、应用什么补丁以及如何应用这些补丁、如何构建二进制和相关文件、如何在目标系统上安装构建的结果、如何创建打包的软件包以及更多。菜谱也描述了在构建中以及运行时对其他软件包的依赖性，因而创建构建过程所需要的菜谱的逻辑层级。菜谱使用 bb 文件后缀
任务（task）	BitBake 菜谱可能含有 BitBake 在构建过程中执行的可执行元数据或者代码。执行步骤可以分组成元数据函数。元数据函数可以通过插入 BitBake 任务列表而被声明为任务
上游（upstream）	在软件开发中，特别是在开源软件开发方面，上游指向发起人，也就是软件的原始作者或者维护者。通常，这个术语被用作限定词，例如上游仓库和上游补丁

2.5 总结

Yocto项目是与嵌入式Linux软件开发相关的项目系列。它的核心是OpenEmbedded构建系统和Poky参考发行版。Poky开始是由OpenedHand作为Poky Linux开发的，然后，它进化成了Yocto项目——在Linux基金会支持下的协作项目。它由企业和独立软件开发者所支持，它将技术与OpenEmbedded对准而形成宽泛的交付于开发嵌入式Linux系统的最先进的工具的社区。

上手Yocto项目和下载构建器具并从VMWare虚拟机管理器中启动它一样简单。虽然构建器具对于严肃的开发来说并不是值得推荐的，但是它提供了对于OpenEmbedded构建系统的好的指导而不需要设置Linux构建主机。

安装可用于Poky的Linux构建主机需要更多的步骤但是避免了虚拟机的开销以及性能影响。

2.6 参考文献

The Linux Foundation,Linux Foundation and Consumer Electronics Linux Forum to Merge,www.linuxfoundation.org/news-media/announcements/2010/10/linux-foundationand-consumer-electronics-linux-forum-merge

The Linux Foundation,Yocto Project Aligns Technology with OpenEmbedded and Gains Corporate Collaborators,www.linuxfoundation.org/news-media/announcements/2011/03/yocto-project-aligns-technology-openembedded-and-gains-corporate-co

第3章　OpenEmbedded构建系统

Poky是Yocto项目的参考发行版。它包含OpenEmbedded构建系统。它提供了构建Linux操作系统栈所要求的全部必要的工具、菜谱和配置数据。正如我们在前面一章中看到的，Poky是打包成简单包的几乎自包含的系统。仅仅有少量额外的组件需要安装在构建主机上以使用Poky。

本章先分析针对开源软件包的典型的工作流。然后解释OpenEmbedded工作流是如何将各个软件包的构建与创建完整的Linux操作系统栈和可启动文件系统镜像的过程集成起来的。有了这些知识，我们就可以详细了解Poky架构及其组件了。

3.1 构建开源软件包

如果你先前曾为Linux主机系统构建过开源软件包，那么你可能已经注意到了，工作流遵循特定的模式。这个工作流的一些步骤你自己执行，而其他的通常是通过某种自动化（例如Make或者其他开源到二进制的构建系统）来实施的。

·获取：获取源代码。

·解压：解压源代码。

·补丁：针对缺陷修复和增加的功能应用补丁。

·配置：依据环境准备构建过程。

·构建：编译和连接。

·安装：复制二进制和辅助文件到它们的目标目录。

·打包：为在其他系统上安装而打包二进制和辅助文件。

如果你正在构建仅仅用在你用来构建的主机系统上的软件包，那么通常你将在系统上安装了二进制以后就停止了。然而，如果你希望为了在其他系统上安装和使用而分发二进制，那么你将也包括打包步骤，它创建可以被包管理系统安装的包。

让我们来看看各个步骤。

3.1.1 获取

所有的一切都是以获取针对软件包的源代码开始。通常，开源软件项目有一个下载区域，从那里源代码、指令、文档以及其他信息可以以包的形式被下载，包通常也是压缩的。理论上听起来很像一个简单的任务，但事实上需要大量对细节的关注。当下载源代码包时，没有通用的要遵循的约定。

当然，每个开源项目有自己的统一资源定位地址（Uniform Resource Locator，URL）来访问它的网站、文件服务器和下载区域。另外，下载可能是通过一个或者更多协议访问的，例如HTTP、HTTPS、FTP、SFTP和其他协议。一些项目可能也提供对已发布版本和来自Git、Subversion、并发版本系统（Concurrent Versions System，CVS；也被称为并发修改系统）等的源代码控制管理系统（Source Control Management，SCM）的源代码开发分支的访问。

通常，从远程位置（例如下载站点或者仓库）获取的源可能由存储在本地文件系统上的补丁和辅助文件来补充。

对于如OpenEmbedded构建系统这样的自动化构建系统来说，这种获取源代码方式的变化性意味着它需要的是灵活，并且其能够为开发者最透明地处理这种变化性。

3.1.2 解压

在源代码被下载后，必须将其解开并且把它从下载位置复制到你将构建它的位置。通常，开源软件包被封装进包，大部分情况下封装成压缩的tar包，但是CPIO和其他序列化多个文件成单一包的格式也是在使用中的。最常用的压缩格式是GZIP和BZIP，但是一些项目使用其他的压缩方案。再一次说明，构建系统必须能够自动检测源包的格式并且使用正确的工具来解压它。

如果源是从源控制管理系统获得的，那么解压包一般意味着把它们从源控制管理系统检出到BitBake构建它们的地方。

3.1.3 打补丁

打补丁是通过增加、删除和变更源文件来修改源代码的过程。有很多理由说明为什么源代码在构建前可能要打补丁：应用缺陷和安全修复、增加功能性、提供配置信息、为交叉编译做出调整等。例如，Linux内核需要提供了数百个用于内核构建系统的配置项的文件，例如目标架构、硬件信息、设备驱动等。

应用补丁可能与复制文件到源代码的目录结构中一样简单。在这种情况下，构建系统当然需要知道把文件复制到哪里去。通常，补丁是使用补丁工具来被应用的，它把用diff工具创建的补丁文件作为输入。diff将原始文件和修改过的文件进行比较并且创建了不但包含变化而且包含如文件的名字和路径、修改的精确位置和上下文等元数据的差异文件。这个文件的格式是标准化的，被称为统一格式（unified format）。使用了统一格式的补丁文件可以包含一次打多个文件补丁的信息，并且它可以增加或者删除完整文件。因为所有关于文件被修改、增加或者删除的信息都是在补丁文件中包含的，所以构建系统不需要知道任何关于要打补丁的源代码的目录结构。

补丁被应用的顺序是很重要的，因为补丁可能是互相依赖的。以正确的顺序应用大量补丁可能是困难的任务。Quilt补丁管理系统通过创建补丁栈来维护顺序从而极大地简化了这个任务。在众多其他功能中，Quilt也允许撤回已经应用的补丁和所有依赖补丁。Quilt是一系列最开始为Linux内核开发而现在也经常被其他开源项目使用的shell脚本。

3.1.4 配置

以源代码的形式提供软件包，服务了这样的目的：用户可以自己针对大范围的目标系统构建软件。随变化性而来的是要求针对软件包的构建环境能够针对目标系统而被正确配置的多样性。精确的配置对于构建主机的CPU架构不同于目标系统的CPU架构这一交叉构建环境是特别重要的。

许多软件包现在为配置而使用GNU构建系统，也被称作Autotools。Autotools是一套目标是使源代码软件包可以移植到很多类UNIX系统的工具。Autotools是反映目标系统和依赖的变化性和多样性的相当复杂的系统。简言之，Autotools从一些列具有特殊源代码体特征的输入文件中创建configure脚本。通过一系列处理步骤，configure创建了特别用于目标系统的makefile。Autotools经常因为难以使用而受到批评。这种困难性，当然，依赖于视角。从用户的视角看，执行单一脚本来配置针对目标系统的源代码包构建环境当然是巨大的收益。希望向他们的软件用户提供便利性的开发者需要理解Autotools的工作方式以及如何正确创建必要的输入文件。而且，它值得努力，并且使用类似针对许多不同目标系统的OpenEmbedded构建系统的自动化构建系统极大地简化构建软件包。

一些软件包使用它们自己的配置系统。在这种情况下，自动化的构建系统需要提供对应调整配置步骤的灵活性。

3.1.5 构建

大部分的软件包使用Make来从源代码构建二进制文件例如可执行程序文件和库以及辅助文件。一些软件包可能使用其他工具，例如CMake或者针对使用了Qt图形库的软件包的qmake。

3.1.6 安装

安装步骤复制二进制、库、文档、配置和其他文件到目标系统的正确位置。程序文件通常被安装进/usr/bin（对于用户程序）和/usr/sbin（对于系统管理程序）。库被复制到/usr/lib以及位于/usr/lib之内的特定于应用的子目录。配置文件通常被安装到/etc。虽然关于在哪里安装特定文件有常用的惯例，但是一些软件开发者有时选择不同的目录来安装属于他们软件包的文件。文件系统层级标准（filesystem Hierarchy Standard，FHS）（https://wiki.linuxfoundation.org/en/FHS）是针对用于UNIX操作系统的文件系统布局的规格说明。

大部分软件包提供作为它们makefile一部分的install目标，它执行安装步骤。正确写下的安装目标使用安装工具来从构建环境中复制文件到它们各自的目标目录。安装工具也可以在复制文件时设置文件属主和权限。

3.1.7 打包

打包是捆绑软件、二进制和辅助文件到单一用于分发和在目标系统上直接安装的包文件的过程。打包可以像压缩的tar包一样简单，然后用户可以在目标系统上进行获取。

为了便利性和可使用性，大部分软件包捆绑它们的文件和安装程序或者包管理系统一起使用。一些系统包含安装软件和软件包并且创建用于自包含安装的可执行文件。其他系统依赖于已经安装在目标系统上的包管理器，并且仅仅捆绑实际的软件和用于包管理器的元数据信息。所有系统的共同点是，它们不但从软件包中复制文件到目标系统而且验证依赖性和系统配置以避免最终导致系统不可操作的不匹配。

Linux系统通常依赖作为发行版部分的包管理系统而不是使用自包含的安装包。好处是，包管理器，作为唯一的实例，维护系统上的软件数据库，并且软件包在尺寸上更小，因为它们不需要包含安装软件。然而，对每个Linux发行版来说，维护者决定它的包管理系统，这需要软件包针对不同的目标系统而被打包多次。

对Linux发行版来说最常用的包管理系统是RPM包管理器（RPM；最初是红帽包管理器）和dpkg（Debian的包管理程序）。对嵌入式设备来说，Itsy包管理系统（ipkg）已经获得了流行。Ipkg是类似dpkg的轻量级系统。Ipkg的开发是被中断的，并且很多一直使用ipkg的嵌入式项目现在在使用opkg，它是由Openmoko项目从ipkg派生出来的。Opkg是以C编写的——它是被Yocto项目积极维护的并且被OpenEmbedded和许多其他项目使用着。

安装和打包不一定是连续的步骤；并且它们也是可选的。如果你正在构建本地使用而非再分发的软件包，那么不需要打包软件。如果你是

包维护者并且创建用于再分发的包，那么你可能不需要执行在构建系统上安装软件包的步骤。

这里简述的步骤本质上是相同的，不管你是在原生地构建软件包还是执行交叉构建。然而，在设置和配置构建环境以及为交叉构建构建包时，你必须要考虑错综复杂的事物。通过本书，我们解决交叉构建软件的复杂性。

3.2 OpenEmbedded工作流

图3-1说明了OpenEmbedded工作流。这个工作流不是BitBake固有的。BitBake完全不建立工作流。工作流和它的配置是被元数据定义的，它被组织成不同类别的文件。

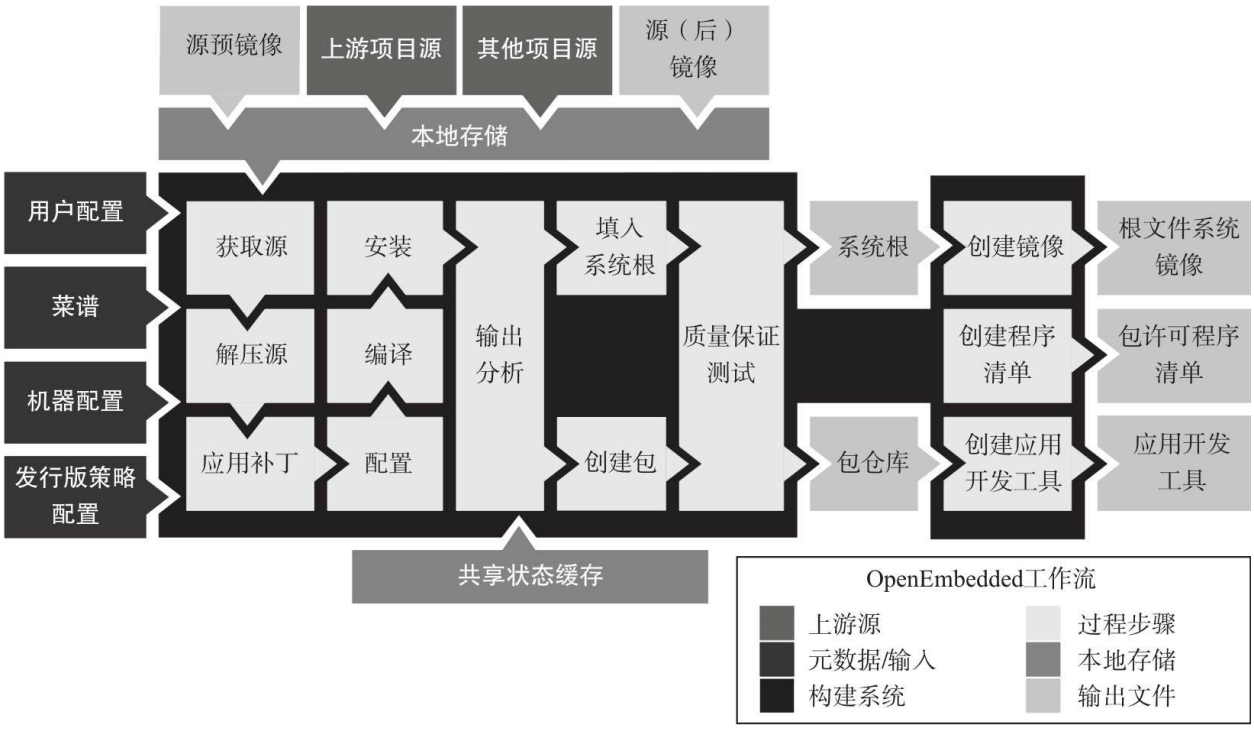

图3-1　OpenEmbedded工作流

3.2.1 元数据文件

元数据文件被划分成两个类别：配置文件和菜谱。

配置文件

配置文件包含简单变量赋值形式的全局构建系统设置。BitBake在全局数据字典中维护变量设定，并且它们可以在任何元数据文件中被访问到。变量可以被设置在配置文件中并且被另外的配置文件覆盖。菜谱可以设置和覆盖变量，但是在菜谱中所做的赋值对于菜谱来说仍然是本地的。BitBake对于赋值元数据变量采用了特殊的语法。赋值和覆盖元数据变量的优先级由多个因素决定，例如层结构、层优先级、文件解析顺序以及赋值语法。我们将在第4章中解释BitBake元数据语法和优先级的细节。

BitBake区分多个不同类型的配置文件，但是都有相同的文件扩展名.conf。

BitBake主配置文件（bitbake.conf）

BitBake的主要配置文件被命名为bitbake.conf。BitBake期望这个文件存在于在它的元数据搜索路径下列出的所有目录中。这个文件包含所有默认配置设置。其他配置文件和菜谱通常根据它们的特定需求而覆盖这个文件中的一些变量设置。

文件bitbake.conf是OpenEmbedded核心（OpenEmbedded Core）元数据层的部分，并且可以在那个层的conf配置文件子目录中找到。

层配置文件（layer.conf）

OpenEmbedded构建系统使用层来组织元数据。层本质上是目录和文件的层级结构。每个层有自己的名字为layer.conf的配置文件。这个文件包含了针对这个层的菜谱文件的路径设置和文件模式。文件layer.conf可以在层的conf子目录中找到。

构建环境层配置（bblayers.conf）

构建环境需要告诉BitBake它需要什么层来针对它的构建过程。文件bblayers.conf提供BitBake关于什么层要包含在构建过程中以及在哪里找到文件路径的信息。每个构建环境有自己的bblayers.conf文件，它可以在构建环境的conf子目录中找到。

构建环境配置（local.conf）

构建环境的本地配置是通过名为local.conf的配置来提供的。文件local.conf包含应用到特定构建环境的设置，例如到下载位置、构建输出和其他文件的路径；针对目标系统的配置设置，例如目标机器、包管理系统以及分发策略；和许多其他设置。文件local.conf可以在构建环境的conf子目录中找到。

发行版配置（<distribution-name>.conf）

发行版配置文件包含反映应用到由OpenEmbedded构建系统构建的特殊发行版的策略的变量设置。对于Poky参考发行版，默认的镜像名字也是Poky，它的配置设置包含在名为poky.conf文件中。发行版策略设置通常包含工具链、C库和发行版名字等。发行版通过设置在构建环境的local.conf文件中的变量DISTRO来被选择。当然，不限于由Poky作为参考所提供的发行版策略。你可以创建你自己的发行版策略文件并且在构建环境中使用它。

发行版配置文件通常在定义发行版的层（例如meta-yocto层）的

conf/distro子目录中找到。

机器配置（<machine-name>.conf）

OpenEmbedded工作流最强力的特性之一是它能够严格区分构建过程中依赖于特殊硬件系统、机器和其架构的部分和不依赖的部分。这种能力极大地简化了板支持包的创建，允许它们仅仅提供依赖于硬件的必要部分，并补充构建系统的机器独立部分。因此，对另外机器构建相同的Linux发行版是如同用另一个板支持包替换板支持包一样简单的。

这个架构的主要部分由机器配置文件组成，该文件包含由构建需要机器特定适配的软件包的菜谱引用的机器相关性的变量设置。机器配置文件是以机器命名的，并且可以在板支持包层的conf/machine子目录中找到。

菜谱

BitBake菜谱形成构建系统的核心，因为它们定义针对软件包的工作流。菜谱包含用于BitBake的关于如何通过实施在3.1节中列出的过程步骤而构建特定软件包的指令。BitBake菜谱以它们的.bb文件扩展名标识。

菜谱包含简单的变量复制和以可执行元数据（executable metadata）形式存在的构建指令，构建指令本质上是执行过程步骤的函数。我们在下一章讨论BitBake内幕时会解释可执行元数据和BitBake任务的细节。

与配置文件相反，所有在菜谱中做出的变量赋值都是仅仅本地于菜谱的。虽然菜谱通常引用在配置文件中做出的变量设置并且有时为了它们的目的而覆盖变量设置，但是所有设置都是保持本地于菜谱的。

许多软件包是以很相似的方式被构建出来的，这种方式使用遵循相

同过程步骤的几乎相同的构建指令。在调整仅仅一些特定于软件包的部分时重复性地复制相同的菜谱将导致大量多余的努力。因此，BitBake提供了类的概念——一种简单的允许菜谱可以容易地在通用工作流中共享的继承机制。类可以被任何BitBake层所定义并且被它们的.bbclass文件扩展所识别出来。

另外的针对菜谱、促进重用的BitBake机制是追加文件（append files）——通过它们的.bbappend文件扩展而被识别出来。追加文件通常由构建在其他层之上的层根据针对它们的特殊需求来优化那些层中包含的菜谱。在大部分情况下，它们会覆盖变量设置或者修改变量设置。追加文件拥有和从其他它们正在追加的层来的核心菜谱相同的基础文件名。

3.2.2 工作流过程步骤

由OpenEmbedded核心元数据层确立和由BitBake执行的工作流本质上遵循在3.1节中列出的步骤。

源获取

菜谱召集源（例如源文件包、补丁和辅助文件）的位置。BitBake可以从构建主机本地或远程地借助网络从外部源仓库获取源。源文件可以以大量不同的格式呈现，例如普通或者压缩的tar包。它们可以借由文件传输协议而被获取，也可以从例如Git、SVN等的源控制管理（Source Control Management，SCM）系统获取[1]。

菜谱通过在变量SRC_URI中包含源文件的统一资源标识符（Uniform Resource Identifier，URI）来指定源文件的位置。在SRC_URI中的统一资源标识符经常指向软件包的上游源仓库，例如文件下载服务器或者上游项目的源控制管理系统。

在尝试从由菜谱的SRC_URI变量指定的上游仓库中下载源软件包之前，BitBake先检查本地的下载目录以确认正确版本的源文件是否已经被获取下来。如果它在本地下载区域找不到源，那么BitBake尝试从一列被称为预先镜像（premirror）（如果它们被配置了）的镜像文件服务器上获取源文件。如果没有任何预先镜像包含必要的文件，那么BitBake尝试实际的、由SRC_URI指定的上游仓库。如果它从那里找不到文件或者如果上游仓库是无法访问的，那么BitBake尝试从第2个列表的镜像服务器下载文件。在本书的上下文中，我们称这些服务器为后镜像（postmirror），虽然在OpenEmbedded术语中，它们仅仅被称为镜像（mirrors）。

Yocto项目维护了具有高可用性的文件服务器，团队把所有上游软件包放在这些服务器上。Poky发行版配置命令BitBake在尝试直接从上游仓库下载文件之前使用Yocto项目镜像。使用Yocto项目镜像使得构建更少地依赖上游文件服务器的可用性。

你也可以通过设置作为你自己构建基础设施一部分的镜像来维护你构建中包含的源的直接控制。

源解压和打补丁

一旦源被下载到本地的下载目录，它们将被解压进本地构建环境中。如果任何补丁被指定成源下载的一部分，那么使用Quilt来应用它们。

通常，源包是不适合于交叉构建的，因此大部分补丁都是集成补丁，修改了使用BitBake进行正确构建的源代码。

配置、编译和安装

通过它的类，OpenEmbedded提供了各种模式来构建标准软件包，例如基于Make的包、基于GNU　Autotools的包和基于CMake的包。这些模式提供了标准化的方法来指定定制化的环境设置。在第8章中，我们将探索用BitBake通过标准模式构建包以及定制化它们的细节。

虽然配置、编译和安装是构建过程中不同的步骤，但它们通常是在相同的类中被解决的，因为它们都包含调用包自己构建系统的部分。

安装步骤是使用pseudo[2]命令来执行的，它允许特殊文件的创建和被正确设置的属主、属组以及其他等权限。所有文件是被安装进私有的、存在于针对特定包的构建环境中的系统根目录中。

输出分析和打包

在输出分析中，由前一个步骤生成和安装的软件按照它的功能被分类成：运行时文件、调试文件、开发文件、文档和语言环境。这允许文件为包管理系统而被拆分进多个物理包中。

在分析之后，包通过使用一个或者多个常见的打包格式RPM、dpkg和ipkg而被创建。

BitBake为在构建环境配置文件local.conf的PACKAGE_CLASSES变量中包含的包管理系统类创建包。虽然BitBake可以为一个或者多个类创建包，但是它只使用第一个列出的类来为发行版创建最终的根文件系统。

镜像创建

针对发行版的根文件系统的各种镜像是使用来自打包步骤的包仓库创建的。使用包管理系统将包从包仓库安装进根文件系统试运行区域。

哪些包被安装进镜像，是由集合了针对基于定义的需求集的工作系统的功能集的镜像菜谱决定的。例如，最小镜像可能包含仅仅足够启动带有最少基础应用集合的命令行操作的包，而带有图形用户界面的镜像可能包含X服务器和许多其他应用包。

镜像创建是由core-image类处理的，其中，它评估变量IMAGE_INSTALL来判断要包含在镜像中的包的列表。

镜像可以以多种格式被创建，包括用于解压在格式化的文件系统中的tar.bz2，以及其他格式，例如ext2、ext3、ext4和jffs，它们可以被直接比特复制到合适的存储设备。

软件开发工具包生成

作为格外的步骤，它不是标准构建过程的一部分，带着创建可启动操作系统栈的目的，软件开发工具包（Software Development Kit，SDK）可以被创建。

软件开发工具包包含了针对开发主机的原生应用，例如交叉工具链、QEMU模拟器和安装脚本。它可能也包含和模拟器一起使用的、基于镜像创建步骤内容的根文件系统镜像。软件开发工具包可以被应用开发者用来创建和测试应用程序，其使用与构建目标系统时相同的环境，而没有实际使用OpenEmbedded构建系统。

软件开发工具包和它的工具可能从命令行直接应用在开发主机上，并且是通过与Eclipse集成开发环境集成而被使用。对于后者来说，Yocto项目提供了用于Eclipse的、可以直接被从Eclipse工作台安装的插件。

[1] 完整的协议和源控制管理包括HTTP、FTP、HTTPS、Git、Subversion（SVN）、Perforce（P4）、Mercurial SCM（Hg）、Bazar（BZR）、CVS、Open Build Service（OSC）、REPO、SSH和SVK。

[2] http://www.yoctoproject.org/tools-resources/projects/pseudo。

3.3 OpenEmbedded构建系统架构

3个基础组件组成了OpenEmbedded构建系统架构：

·构建系统

·构建环境

·元数据层

图3-2描绘了组件及其相互关系。

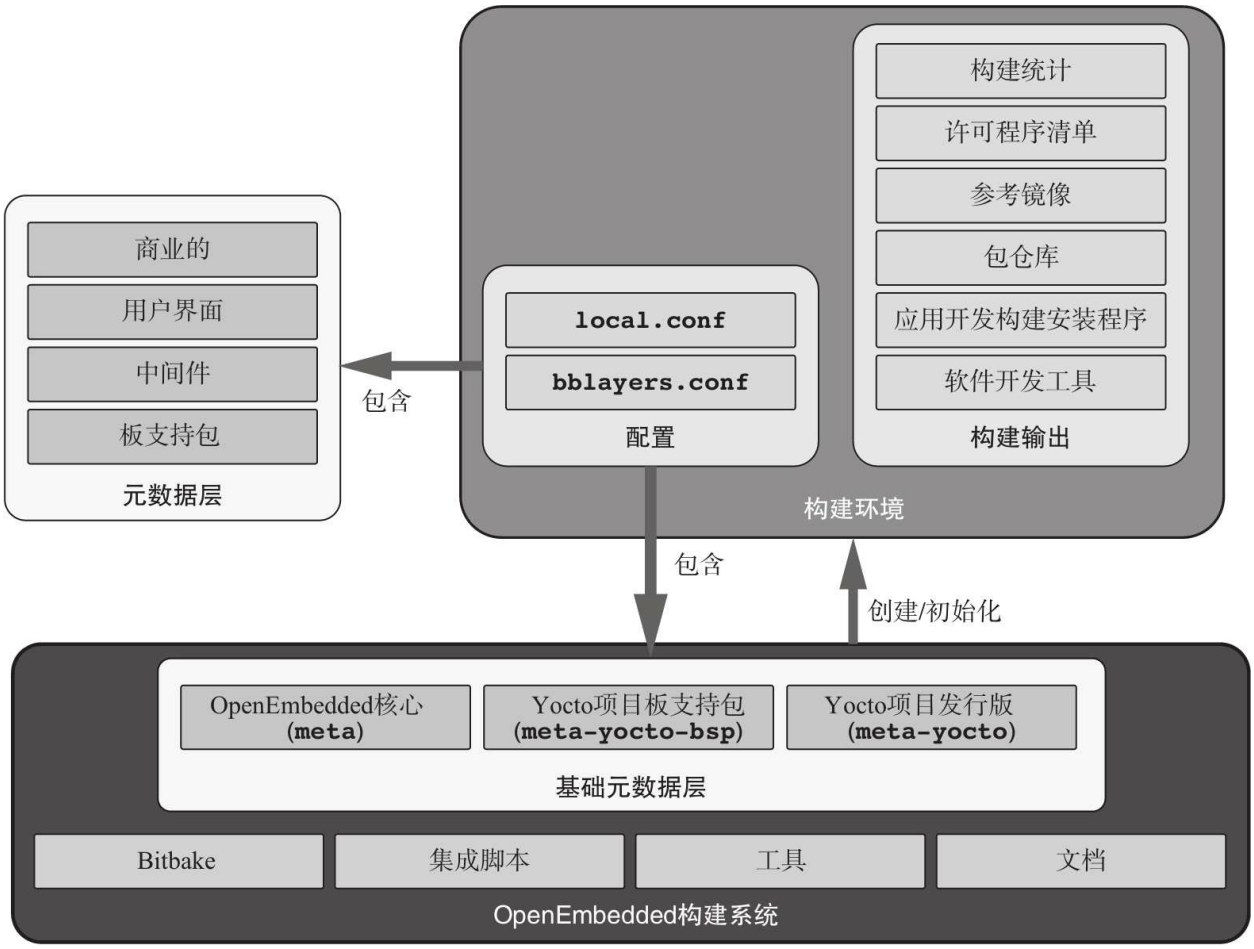

图3-2　Poky架构

除了少量随任何Linux发行版而来的额外软件包以外，OpenEmbedded构建系统为构建嵌入式Linux操作系统栈提供了所有必要组件。包含在内的是：BitBake构建引擎；一套为便利性而提供额外功能性的集成脚本；各种工具；OpenEmbedded核心——BitBake所需要的用以构建镜像的核心元数据集合；DocBook格式的完整的文档。Poky参考发行版也包括了最小集合的基础元数据层：Yocto项目板支持包（meta-yocto-bsp）和Yocto项目发行版（meta-yocto）。

正如我们在第2章中所看到的，脚本oe-init-build-env创建并初始化构建环境。这个脚本是包含在构建系统内的脚本之一。构建系统和构建环境形成1：n的关系：构建系统可以和任何数量的构建环境相关联，但

是构建环境仅仅可以和构建系统相关联。在你一次使用多于一个Yocto项目发布时，这是重要的、你需要知道的限制。你仅仅可以用构建环境和它原来被创建时用到的版本的构建系统。使用不同于原来被用来创建这个构建环境的构建系统来初始化构建环境会导致构建失败。

构建系统总是要包含元数据层，它提供菜谱和配置文件。当你用构建系统的oe-init-build-env脚本创建构建环境的时候，脚本自动设置包含了3个基础层的conf/bblayers.conf文件：meta、meta-yocto-bsp和meta-yocto。这些基础层足够来构建标准的Poky参考发行版。然而，作为嵌入式Linux开发者，你最终希望创建你自己的发行版、增加你自己的软件包并最终为目标硬件提供你自己的板支持包。这个目标是通过包含构建系统的其他元数据层来实现的。

在接下来的部分，我们将更加细节地探索构建系统的结构、构建环境和元数据层。

3.3.1 构建系统结构

不管你以tar包的形式下载Poky发行版并且解压它，还是你直接从Yocto项目Git仓库克隆它，它都会安装由类似程序清单3-1的目录和单一目录中的文件组成的简单结构。

程序清单3-1 OpenEmbedded构建系统结构

```
yocto@yocto-dev:~/yocto$ tree -L 1 poky
poky
├── bitbake
├── documentation
├── LICENSE
├── meta
├── meta-hob
├── meta-selftest
├── meta-skeleton
├── meta-yocto
├── meta-yocto-bsp
├── oe-init-build-env
├── oe-init-build-env-memres
├── README
├── README.hardware
└── scripts

9 directories, 5 files
```

因为OpenEmbedded构建系统是几乎自包含的，并且不安装任何它的组件到你构建主机的系统目录中，所以在相同的构建主机上使用构建系统的多个版本和不同构建环境是相当容易的。这是非常便利的特性，因为你最终用版本的构建系统来开发和维护产品代，同时用更新的版本来开发下一代以利用新的功能性和特性。

BitBake构建引擎，是OpenEmbedded构建系统不可分割的部分。它与构建一起演进，Yocto项目开发者向BitBake增加新功能性以支持构建系统所需要的新特性。结果，BitBake被包含在构建系统中，你可以在bitbake子目录中找到它。需要知道的是，一些Linux发行版包含你可以使用发行版包管理系统安装的BitBake包。如果你已经在开发主机上安

装了BitBake作为发行版的一部分，那么我们建议卸载它，因为它可能干扰包含在构建系统中的版本。构建系统和BitBake是相互匹配的。疏忽地使用不匹配构建系统的BitBake版本可能导致构建失败。

目录bitbake包含包括了BitBake文档和man页面的doc子目录。文档是以DocBook格式写的，并且通过调用make命令，你可以构建PDF和HTML版本的手册。在下一章，我们更进一步看BitBake。

目录documentation包含针对Poky构建系统的文档。以下是你也可以在Yocto项目网站上找到的各种手册：

·应用开发工具集用户指南（adt-manual）

·板支持包开发者指南（bsp-guide）

·开发手册（dev-manual）

·Linux内核开发手册（kernel-dev）

·分析和追踪手册（profile-manual）

·参考手册（ref-manual）

·Yocto项目快速开始（yocto-project-qs）

·Toaster手册（toaster-manual）

各自的手册是以DocBook格式的源在独立的目录中提供的。为了创建PDF和HTML格式的手册，使用命令

```
$ make DOC=<manual>
```

把<manual>替换成目录的名字。例如，

```
$ make DOC=mega-manual
```

创建把所有手册包含进一个文件的“mega-manual”。

LICENSE文件包含了针对Poky构建系统的许可信息。MIT和GPLv2许可混合使用。BitBake是在GPLv2许可下被许可的，并且所有的元数据是在MIT许可下被许可的。如果有任何源代码包含在针对各个菜谱的树中，例如补丁，那么它是以在各个菜谱中阐述的许可被许可的。

对任何文件的许可信息要么是在各自的文件中明确陈述的，要么是在其缺失的情况下默认为GPLv2。

重要的是注意，可能存在并且常见的是，许可一个包的源代码和构建那个包的元数据（菜谱）有差别。一定要保证不要混淆这两者，因为它可能对你正在开发的产品产生影响。

以meta开头的目录都是元数据层：

·meta：OpenEmbedded核心元数据层

·meta-hob：针对BitBake的Hob图形用户界面所使用的元数据层

·meta-selftest：oe-selftest脚本使用的用来测试BitBake的层

·meta-skeleton：用来创建自己的层的模板

·meta-yocto：Yocto项目发行版层

·meta-yocto-bsp：Yocto项目板支持包层

我们将在3.3.3节中解释元数据层的结构。

脚本oe-init-build-env创建和初始化构建环境。它有两种使用方式：

以默认设置来创建空的构建环境和初始化先前被创建的构建环境。我们在第2章用前者的形式创建我们的第一个构建环境。该脚本的命令行是

```
$ oe-init-build-env <buildenv>
```

<buildenv>被替换成构建环境的名字。如果没有构建环境名字提供，那么脚本使用默认的名字构建。脚本使用提供的构建环境名字在当前目录下创建子目录。在那个目录中，它创建了2个被任何构建环境需要的配置文件bblayers.conf和local.conf的名为conf的子目录。在那之后，脚本初始化所有必要shell环境变量并且切换目录到构建环境。

如果构建环境目录已经存在并且是OpenEmbedded构建环境，那么oe-init-build-env仅仅初始化shell环境变量和切换目录。

第2个脚本，oe-init-build-env-memres，也像oe-init-build-env一样创建和初始化构建环境，但是也会启动内存驻留的BitBake服务器，它在TCP端口上监听以接收命令。这容易允许在远程构建服务器上运行BitBake并且从本地系统通过网络控制它。脚本的命令行是：

```
$ oe-init-build-env <buildenv> <port>
```

每一个<port>参数或者<buildenv>和<port>两个参数都可以被省略，在这种情况下，默认的构建和12345被使用。

最后，有scripts子目录，它包含了一套用于和Yocto项目构建一起使用的集成和支持脚本。最常用到的脚本如下：

·bitbake-whatchanged：作为在两次构建之间对元数据做出的变化的结果，列出所有需要被重新构建的组件

·cleanup-workdir：从构建环境中移除废弃包的构建目录

·create-recipe：创建工作于BitBake的菜谱

·hob：启动Hob——用于BitBake的图形用户界面

·runqemu：启动QEMU模拟器

·yocto-bsp：创建Yocto项目板支持包层

·yocto-kernel：配置在Yocto项目板支持包层内的Yocto项目内核菜谱

·yocto-layer：创建工作于BitBake的元数据层

贯穿本书，我们使用来自scripts子目录的这些和其他脚本。在第一次介绍它们的时候，我们会解释其用法。

3.3.2　构建环境结构

OpenEmbedded构建系统在构建环境内执行它的全部工作。构建环境也有特定的布局和结构。布局以及其中的所有目录和文件是由构建系统自动化地创建的。构建环境的目录和文件结构是深度嵌套的。程序清单3-2显示了在构建已经运行后的前2级结构。

程序清单3-2　构建环境结构

```
yocto@yocto-dev:~/yocto$ tree -L 2 build
x86/
├── bitbake.lock
├── cache
│   ├── bb_codeparser.dat
│   ├── bb_persist_data.sqlite3
│   └── local_file_checksum_cache.dat
├── conf
│   ├── bblayers.conf
│   ├── local.conf
│   └── sanity_info
└── tmp
    ├── abi_version
    ├── buildstats
    ├── cache
    ├── deploy
    │   ├── images
    │   ├── licenses
    │   ├── deb
    │   ├── ipk
    │   └── rpm
    ├── log
    ├── qa.log
    ├── saved_tmpdir
    ├── sstate-control
    ├── stamps
    ├── sysroots
    ├── work
    │   ├── all-poky-linux
    │   ├── i586-poky-linux
    │   ├── qemux86-poky-linux
    │   └── x86_64-linux
    └── work-shared
```

新创建的构建环境仅仅包含conf子目录和2个文件：bblayers.conf和local.conf。在第2章中设置和配置我们的第一个构建环境时，我们遇到过后者（此处指文件local.conf）。这个文件包含用于构建环境的所有配置设置。你也可以向其增加本地覆盖来自被包含层中的设置的变量设置。

文件bblayers.conf包含用于构建环境的层设置。列表3-3显示了一个典型的bblayers.conf文件。

程序清单3-3　bblayers.conf

```
# LCONF_VERSION: version number for bblayers.conf
```

```
# It is increased each time build/conf/bblayers.conf
# changes incompatibly
LCONF_VERSION = "6"

BBPATH = "${TOPDIR}"
BBFILES ?= ""

BBLAYERS ?= " \
  /absolute/path/to/poky/meta \
  /absolute/path/to/poky/meta-yocto \
  /absolute/path/to//poky/meta-yocto-bsp \
  "
BBLAYERS_NON_REMOVABLE ?= " \
  /absolute/path/to/poky/meta \
  /absolute/path/to/poky/meta-yocto \
  "
```

在这个文件中最重要的变量是BBLAYERS，它是空格分隔的、到所有该构建环境包含的层的路径列表。这是你可以增加在构建环境中被包含的额外层的地方。这个文件也设置BBPATH成构建环境的顶级目录，并且用空字符串来初始化菜谱文件列表BBFILES。

其他目录和文件是在构建过程中被创建的。所有的构建输出被放进tmp子目录。你可以通过设置在环境的conf/local.conf文件中的TMPDIR变量来配置这个目录。在tmp目录中的构建输出被组织进大量子目录中：

·buildstats：这个子目录存储根据构建目标和目标被构建时的日期/时间戳组织的构建统计。

·cache：当BitBake初步解析元数据时，它分析依赖性和表达式。解析过程的结果被写入缓存。只要元数据没有改变，BitBake在后续运行时直接从这个缓存获取元数据信息。

·deploy：针对部署的构建输出，例如目标文件系统镜像、包仓库和许可信息，被包含在deploy子目录中。

·log：你可以在此找到由cooker进程创建的BitBake日志信息。

·sstate-control：这个子目录包含用于根据架构/目标和任务组织的共享状态缓存的程序清单文件。

·stamps：BitBake把针对每个任务、根据架构/目标组织的完成标签和签名数据以及包名放在这个子目录中。

·sysroots：这个子目录包含根据架构/目标组织的根文件系统。内容包括针对构建主机的，包含交叉工具链、QEMU和在构建过程中用到的很多工具的根文件系统。

·work：在这个目录中，BitBake创建根据架构/目标组织的、它在这里构建实际软件包的子目录。

·work-shared：这个子目录和work类似，但是针对共享的软件包。

也有2个在tmp目录中值得解释的文件：abi_version和saved_tmpdir。前者包含针对tmp目录布局的版本号码。当布局变化时，这个号码增加。这个号码允许验证构建环境是否和构建系统是兼容的。后者包含tmp目录的绝对文件系统路径。很多在tmp目录下的文件包含绝对文件路径。很不幸，这种安排使得目录不可迁移。虽然这种限制可能不够方便，但是saved_tmpdir文件使得你可以方便地检查目录是否已经被从原始位置移出来了。

尽管构建环境，特别是通常位于其中的tmp目录，不可以被简单地迁移，但是BitBake本质上可以从共享状态缓存重新创建tmp目录的所有内容。共享状态缓存存储自其输入元数据（例如任务代码、变量等）创建的签名所识别的任务的中间输出。只要输入不变，那么签名就不变，使得BitBake使用来自共享状态缓存的输出而不是运行任务。这极大地缩短了构建时间，特别是针对那些花费长时间运行的任务，例如配置或者编译。

3.3.3 元数据层结构

元数据层是分组和组织菜谱、类、配置文件和其他元数据到逻辑实体中的容器。层通常构建在其他层之上并且彼此互相扩展。OpenEmbedded核心层形成了针对Poky构建系统的层架构的基础。它提供了针对大部分Linux操作系统栈需要的软件包的核心集合的菜谱，当然包括Linux内核、引导加载程序、图形、网络和许多其他包。OpenEmbedded核心也提供基础类以构建软件包、用包管理系统打包软件、创建文件系统和扩展BitBake功能性。

OpenEmbedded核心层本身，由BitBake、便利性和集成脚本所补充，针对构建用于模拟的设备的Linux操作系统栈来说是足够的。OpenEmbedded核心包含基础镜像目标和针对用于ARM、MIPS、PowerPC、x86和x86_64架构的QEMU模拟器的机器定义。

针对设备操作系统栈的构建环境通常包含其他层，例如用于实际硬件的板支持包层；指定用于用户账号、系统启动等操作系统配置的发行版层；用户界面层和用于用户空间应用、提供设备功能性的应用层。图3-3描述了架构。

图3-3　层架构

层布局

所有层，不管它们服务的目的是什么，都有如程序清单3-4所示的相同的基本结构（目录名字是斜体化的以把它们和文件名区分开）。

程序清单3-4　层布局

```
meta-<layername>
├── conf
│   ├── layer.conf
│   ├── machine
│   │   ├── <machine 1>.conf
│   │   ├── <machine 2>.conf
│   │   ├── ...
│   │   └── <machine m>.conf
│   ├── distro
│   │   ├── <distro 1>.conf
│   │   ├── <distro 2>.conf
│   │   ├── ...
│   │   └── <distro r>.conf
├── classes
│   ├── class<1>.bbclass
```

```
|   ├── class<2>.bbclass
|   ├── ...
|   └── class<1>.bbclass
├── recipes-<category 1>
|   ├── <package a>
|   |   ├── <package a>_<version 1>.bb
|   |   └── <package a>_<version 2>.bb
|   ├── <package b>
|   |   ├── <package b>_<version 1>.bb
|   |   └── <package b>_<version 2>.bb
|   ├── ...
|   └── <package z>
├── recipes-<category 2>
|   └── ...
└── recipes-<category n>
    └── ...
```

元数据层本质上是由目录和文件组成的结构。层顶级目录的名字并不严格；但是，按照惯例，所有的层目录名字以词meta开头、接下来是连字符（-）和层的名字。

每个层必须包含含有该层配置文件layer.conf的conf子目录。BitBake需要这个文件来设置针对元数据文件的路径和搜索模式。程序清单3-5显示了样本的layer.conf文件。

程序清单3-5　layer.conf

```
# Add the layer's directory to BBPATH
BBPATH =. "${LAYERDIR}:"

# Add the layer's recipe files to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"

# Add the name of the layer to the layer collections
BBFILE_COLLECTIONS += "layername"

# Set the recipe file search pattern
BBFILE_PATTERN_layername = "^${LAYERDIR}/"

# Set the priority of this layer
BBFILE_PRIORITY_layername = "5"

# Set version of this layer
# (should only be incremented if changes break compatibility)
LAYERVERSION_layername = "2"
# Specify other layers this layer depends on. This is a white space-
# delimited list of layer names. If this layer depends on a particular
# version of another layer, it can be specified by adding the version
# with a colon to the layer name: e.g., anotherlayer:3.
LAYERDEPENDS_layername = "core"
```

第一个赋值向BBPATH变量增加层的目录。变量LAYERDIR被BitBake扩展成层的权威路径名字。然后，层的菜谱被增加到BBFILES变量。你可以看到，通配符表达式匹配在这层的菜谱目录的布局。用于菜谱和菜谱追加文件的通配符都需要被增加到BBFILES。

BBFILE_COLLECTIONS是由空格分隔的层名字的列表。每一层向列表增加它的名字。BBFILE_PATTERN包含正则表达式来匹配在BBFILES变量中的这一层的菜谱。这个变量是以层为条件的，因此变量名需要以层的名字作为前缀[1]。

因为层互相依赖和扩展，所以处理顺序是重要的。因此，每层通过设置BBFILE_PRIORITY来赋予优先级。层优先级处于1到10之间，1是最低优先级、10是最高优先级。如果2个层使用相同的优先级，那么它们在文件bblayers.conf中的变量BBLAYERS的顺序决定优先级。

可选的是，层也可以通过设置变量LAYERVERSION来定义版本号。层版本可以和LAYERDEPENDS变量一起使用以避免包含层的不兼

容版本。如果层依赖于另外的层，那么这些依赖性可以通过增加这些层到LAYERDEPENDS变量来设置，LAYERDEPENDS包含了空格分隔的层名字的列表。如果依赖性是基于层的特定版本，那么版本号可以通过增加冒号（：）和版本号来指定。

子目录conf可能包含其他文件和目录，特别是distro和machine子目录。这些是可选的。典型的是，仅仅发行版层将包含distro子目录，并且仅仅板支持包层通常包含machine子目录。如果存在，那么这2个子目录的每一个都包含用于发行版和机器配置的文件[2]。

如果层定义自己的类，那么它们位于classes子目录中。

层的菜谱由类别和包分组。类别是一组逻辑上属于一起的包。例如，OpenEmbedded核心元数据层的recipes-connectivity类别包含构建用于网络、电话和其他连接功能软件的包的菜谱。在每个类别子目录中，有针对不同软件包的子目录。这些包子目录包含菜谱、补丁和其他构建软件包所需要的文件。通常，包子目录包含用于构建不同版本的特定包的菜谱。

创建层

使用针对板支持包、应用软件、发行版策略等的层对你自己的项目产生了很多好处。大部分项目从小处开始，但随后越来越多的功能性被增加。即使你一开始仅仅有1个或者2个菜谱，把菜谱放在你自己的层而不是增加它们进OpenEmbedded核心层或者任何Yocto项目层依然是好的实践。

你自己的层从常用菜谱中分离你自己的菜谱，使得对你来说容易从OpenEmbedded构建系统的一个版本迁移到下一个。你仅仅需要用新的构建系统创建新的构建环境，并且在这个构建环境中包含进层。

通过在层中使用bbappend文件，你可以从常用层中调整菜谱而不是复制或者重写它们。考虑下在meta/recipes-kernel/linux中用于构建Linux内核的菜谱。就绝大部分而言，它们已经提供了用于构建内核的所有必要的东西。对你自己的板支持包层，你通常需要仅仅改进几个设置来完全支持目标硬件。不必复制内核菜谱，你可以使用bbappend文件或者包含文件来按照需求定制基础菜谱。

用yocto-layer脚本创建层是简单和直接的。在引用了构建环境后，这个脚本在命令行搜索路径下是直接可用的。简单调用这个脚本如下：

```
$ yocto-layer create <layername>
```

如果你想创建样例菜谱或样例bbappend文件，那么脚本提示你输入新层的优先级，然后在当前目录中创建包含conf/layer.conf文件和样例文件的层。所有你需要做的是，向你构建环境的conf/bblayers.conf文件中的BBLAYERS变量增加层以在构建过程中包含它。当然，你也需要根据你项目需求而修改样例菜谱或新菜谱、配置文件和类。

[1] 我们将在第4章讨论条件变量设置。
[2] 我们将在接下来的章中解释发行版层和板支持包层。

3.4 总结

本章展示了构建系统及其工作流和其组件的概览。

·开源软件包最常见的是遵循标准工作流构建的。

·OpenEmbedded构建系统为数百个从源代码构建出来的包自动化工作流步骤以创建具有完整功能性的Linux操作系统栈。

·BitBake不仅构建软件包而且也打包可以被通用包管理系统dpkg、RPM和ipkg所利用的可执行文件、库、文档等进包中。

·构建系统创建可以被安装在目标系统上的根文件系统镜像。

·可选的是，BitBake可以为开发者构建包含交叉开发工具的软件开发工具包以构建针对目标系统的应用。

·组成架构的3个核心组件是OpenEmbedded构建系统、构建环境和元数据层。

·Poky参考发行版包含OpenEmbedded构建系统和这3个元数据层——OpenEmbedded核心（meta）、Yocto发行版（meta-yocto）和Yocto板支持包（meta-yocto-bsp），这3个元数据层在构建环境被创建的时候自动化地加到了构建环境中。

·层分组元数据进逻辑实体，例如板支持包、发行版层和应用层等。

3.5 参考文献

Yocto项目文档，https://www.yoctoproject.org/documentation/current。

第4章　BitBake构建引擎

BitBake是类似Make和Apache　Ant的构建工具。然而，因为它灵活且可扩展的架构，BitBake的能力远远超过典型软件构建引擎所提供的东西。BitBake的元数据语法不仅支持变量和简单命令的执行，而且可以嵌入整个shell脚本和Python函数。

BitBake源于Portage——Gentoo Linux发行版所使用的构建和包管理系统。BitBake是完全用Python编写的，这使它在大部分情况下是平台独立的——只要平台提供Python运行时环境。

与任何软件构建工具一样，BitBake自己不提供用以构建软件包的功能性。它必须由包含在元数据文件中的构建指令来补充。BitBake元数据文件类似Make的makefiles和Ant的build.xml文件。OpenEmbedded和Yocto项目提供必要的元数据以构建数千个软件包并且将它们集成进包含完整功能的Linux操作系统栈的文件系统镜像中。

4.1 获取和安装BitBake

BitBake是OpenEmbedded构建系统的核心组件，在前面的章节中已有描述。BitBake是Yocto和OpenEmbedded共同维护的Yocto项目的子项目。BitBake源仓库原来是托管在http://developer.berlios.de/projects/bitbake，但是在Yocto项目创建后，活跃的开发分支被移动到位于http://git.openembedded.org/bitbake的OpenEmbedded的Git仓库。

你从Yocto项目网站下载的或者从Yocto项目的Git仓库克隆出来的构建系统总是包含匹配Poky元数据的BitBake版本。因此，在使用Poky时，你不必要担心下载和安装BitBake[1]。然而，如果你有兴趣使用BitBake用于其他项目，例如追踪当前BitBake开发状态，或者期待向BitBake作贡献，那么你可以从OpenEmbedded仓库下载它并且在系统上安装它。

我们举例说明BitBake的下载和安装以便你可以看到BitBake如何工作。使用BitBake进行实验并且学习它的内部工作机制而不用OpenEmbedded构建系统自带的集成是一个有用的设置。

[1] 不鼓励使用非构建系统附带的BitBake版本。BitBake是和提供它的构建系统紧密集成的。

4.1.1 使用发布快照

已发布BitBake版本的压缩tar包可以通过浏览器从OpenEmbedded的Git仓库下载或者使用wget命令：

```
wget https://git.openembedded.org/bitbake/snapshot/bitbake-1.17.0.tar.bz2
```

执行

```
tar xvjf bitbake-1.17.0.tar.bz2 -C ~
```

解压包到home目录中，创建名字为bitbake-1.17.0且包含了BitBake源的子目录。

4.1.2 克隆BitBake开发仓库

如下命令

```
git clone git://git.openembedded.org/bitbake.git ~/bitbake
```

克隆BitBake的Git仓库进home目录的bitbake目录中，并且检出主干（master）分支。和Git仓库一样，主干分支是实际的开发分支。

4.1.3 构建和安装BitBake

BitBake源树包含让你可以预先编译BitBake的Python源模块、创建文档并且最终在系统上安装BitBake的setup.py脚本。

从你已经解压或者克隆BitBake源的目录中，执行

```
./setup.py build
```

以编译BitBake并从DocBook源中创建HTML和PDF格式的文档。编译BitBake要求Python　Lex-Yacc（PLY）包被安装在系统上。几乎所有Linux发行版都提供这个包。执行

```
./setup.py install
```

安装BitBake和它的库到开发系统上的默认Python安装目录中，在大部分Linux发行版上，它是/usr/lib/python<version>/site-packages。安装BitBake当然是可选的，因为你可以直接从它的源目录中使用BitBake。

4.2 运行BitBake

如果你已经用过Make，那么你知道，你可以执行它而不带任何参数，Make在当前目录中寻找被称为GNUmakefile、makefile或者Makefile的文件，并且构建它在文件中找到的默认目标。Ant是相当类似的，因为它在当前目录中搜索名字为build.xml的文件。这两个工具也让你可以显式地指定包含构建指令的文件作为命令行参数。

BitBake本质上以相同的方式工作。然而，BitBake总是必须要用菜谱的基名称（base name）或者目标作为参数而被调用：

```
$ bitbake core-image-minimal
```

在这个例子中，core-image-minimal对应于菜谱core-image-minimal.bb。然而，与Make和Ant不同，BitBake不自动在当前目录中寻找菜谱。BitBake要求你在它可以定位和执行构建指令前设置执行环境。

4.2.1 BitBake执行环境

在启动时，BitBake首先在当前工作目录中寻找conf/bblayers.conf配置文件（见程序清单4-1）。

程序清单4-1 conf/bblayers.conf

```
BBPATH = "${TOPDIR}"
BBFILES ?= ""

BBLAYERS ?= " \
  /path/to/directory/for/layer1 \
  /path/to/directory/for/layer2 \
  /path/to/directory/for/layer3 \
  "
```

虽然这个文件是可选的，但是它出现在任何使用层架构的BitBake构建环境中，它包括所有由Yocto项目工具创建的构建环境。BitBake期望这个文件包含被称为BBLAYERS的变量，BBLAYERS包含指向目录的路径列表，包含在构建环境中的层可以在那里被找到。这些层目录中的每一个都被期望包含名为conf/layer.conf的文件（见程序清单4-2）。

程序清单4-2 conf/layer.conf

```
# We have a conf and classes directory, add to BBPATH
BBPATH .= ":${LAYERDIR}"

# We have recipes-* directories, add to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
        ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "layer1"
BBFILE_PATTERN_layer1 = "^${LAYERDIR}/"
BBFILE_PRIORITY_layer1 = "6"
```

层配置文件conf/layer.conf的目的是正确设置变量BBPATH和BBFILES以使得BitBake可以找到在该层中包含的菜谱、类和配置文件：

·BBPATH：BitBake使用这个变量来在名为classes的子目录中定位类（.bbclass文件）和在名为conf的子目录中定位配置文件（.conf文件）以及子目录等。

·BBFILES：这个变量包含针对菜谱文件的、带有通配符的路径列表。

层通常增加指向它顶级目录的路径到包含在BBPATH中的路径列表中。BitBake在它开始解析某一层中的文件时，会自动地设置变量LAYERDIR为到那个层顶级目录的路径。

层也增加指向它所提供的菜谱文件的路径到包含在BBFILES变量中的文件路径列表中。文件路径代表包含了层的菜谱的目录结构，根据惯例，它是带有2级子目录的布局，正如在第3章中所解释的。

3个变量BBFILE_COLLECTIONS、BBFILE_PATTERN、BBFILE_PRIORITY提供BitBake关于如何定位和在考虑到其他层的情况下如何对待这个层的菜谱的信息：

·BBFILE_COLLECTIONS：包含已配置层的名字的列表。这个列表被BitBake使用以在它的数据目录中找到其他BBFILE_*变量。通常每个层都增加它自己的名字到这个列表。

·BBFILE_PATTERN：告诉BitBake如何从BBFILES的这一层中定位菜谱文件的正则表达式。层所设置的变量值对应于它增加到BBFILES变量的路径。因为所有在BBFILES中的路径都以层的顶级目录开头，所以这个正则表达式反映了那个惯例。变量的名字必须追加下划线（_）和层的名字。

·BBFILE_PRIORITY：向包含在该层中的菜谱赋予优先级。变量的名字必须追加层的名字。

大的优先级值对应于高的优先级。如果带有相同名字的菜谱出现在多于一个层中，那么优先级是特别重要的。在这种情况下，BitBake从带有高优先级的层中选取菜谱，即使包含在低优先级层中的菜谱拥有比高优先级层中的菜谱更新的版本。

优先级也决定了BitBake以什么顺序追加追加文件到菜谱。来自高优先级层的追加文件在来自低优先级层的追加文件之后被增加。

在大部分情况下，你直接从执行或者构建环境的顶级目录中执行BitBake。如果，因为某些原因，从顶级目录执行不是合适的，那么你可以在启动BitBake前设置BBPATH变量：

```
$ BBPATH="/absolute/path/to/build_env" bitbake <target>
```

注意，BitBake期望BBPATH变量包含绝对路径；否则，它找不到配置文件conf/bblayers.conf。

在解析conf/bblayers.conf之后，BitBake寻找配置文件conf/bitbake.conf，它提供了构建系统设置。如果层没有被使用并且因此没有conf/bblayers.conf文件存在，那么BBPATH变量需要进行如前所示的设置，并且conf/bitbake.conf必须包含用于BBFILES的变量赋值。

在定位和解析conf/bitbake.conf以及其他配置文件之后，BitBake定位和解析所有类。至少一个包含在文件base.bbclass中的base类必须存在以使得BitBake正确地操作。这个类提供了基本的功能和任务，包括默认的build任务。

4.2.2 BitBake命令行

用--help选项运行BitBake提供了这个工具的命令行选项的概览（见程序清单4-3）。

程序清单4-3 BitBake命令行选项

```
$ bitbake --help
Usage: bitbake [options] [recipename/target ...]
```

```
   Executes the specified task (default is 'build') for a given set of
   target recipes (.bb files). It is assumed there is a conf/bblayers.conf
   available in cwd or in BBPATH which will provide the layer, BBFILES and
   other configuration information.
Options:
  --version             show program's version number and exit
  -h, --help            show this help message and exit
  -b BUILDFILE, --buildfile=BUILDFILE
                        Execute tasks from a specific .bb recipe directly.
                        WARNING: Does not handle any dependencies from
                        other recipes.
  -k, --continue        Continue as much as possible after an error. While
                        the target that failed and anything depending on it
                        cannot be built, as much as possible will be built
                        before stopping.
  -a, --tryaltconfigs   Continue with builds by trying to use alternative
                        providers where possible.
  -f, --force           Force the specified targets/task to run
                        (invalidating any existing stamp file).
  -c CMD, --cmd=CMD     Specify the task to execute. The exact options
                        available depend on the metadata. Some examples
                        might be 'compile' or 'populate_sysroot' or
                        'listtasks' may give a list of the tasks available.
  -C INVALIDATE_STAMP, --clear-stamp=INVALIDATE_STAMP
                        Invalidate the stamp for the specified task such as
                        'compile' and then run the default task for the
                        specified target(s).
  -r PREFILE, --read=PREFILE
                        Read the specified file before bitbake.conf.
  -R POSTFILE, --postread=POSTFILE
                        Read the specified file after bitbake.conf.
  -v, --verbose         Output more log message data to the terminal.
  -D, --debug           Increase the debug level. You can specify this more
                        than once.
  -n, --dry-run         Don't execute, just go through the motions.
  -S, --dump-signatures
                        Don't execute, just dump out the signature
                        construction information.
  -p, --parse-only      Quit after parsing the BB recipes.
  -s, --show-versions   Show current and preferred versions of all recipes.
  -e, --environment     Show the global or per-package environment complete
                        with information about where variables were
                        set/changed.
  -g, --graphviz        Save dependency tree information for the specified
                        targets in the dot syntax.
  -I EXTRA_ASSUME_PROVIDED, --ignore-deps=EXTRA_ASSUME_PROVIDED
                        Assume these dependencies don't exist and are
                        already provided (equivalent to ASSUME_PROVIDED).
                        Useful to make dependency graphs more appealing.
  -l DEBUG_DOMAINS, --log-domains=DEBUG_DOMAINS
                        Show debug logging for the specified logging
                        domains.
  -P, --profile         Profile the command and save reports.
  -u UI, --ui=UI        The user interface to use (e.g. knotty, hob,
                        depexp).
  -t SERVERTYPE, --servertype=SERVERTYPE
                        Choose which server to use, process or xmlrpc.
  --revisions-changed   Set the exit code depending on whether upstream
                        floating revisions have changed or not.
  --server-only         Run bitbake without a UI, only starting a server
                        (cooker) process.
  -B BIND, --bind=BIND  The name/address for the bitbake server to bind to.
```

```
--no-setscene         Do not run any setscene tasks. sstate will be
                      ignored and everything needed, built.
--remote-server=REMOTE_SERVER
                      Connect to the specified server.
-m, --kill-server     Terminate the remote server.
--observe-only        Connect to a server as an observing-only client.
```

第一眼，命令行选项的数量看起来是巨大的，但是它们可以被简单分解成一些功能块。

显示程序版本和帮助

命令行选项--version和--help或者-h分别向控制台打印程序版本号和程序清单4-3中的帮助信息。

执行带依赖处理的构建

用目标（菜谱文件不带.bb后缀的基名称）来调用BitBake，运行由变量BB_DEFAULT_TASK定义的默认任务，通常是build：

```
$ bitbake core-image-minimal
```

BitBake评估目标和任务的所有依赖性，并且在实际构建目标前以正确的、由依赖链所强加的顺序执行所有依赖任务。没有被实现的依赖性会导致错误。

任何错误条件都导致BitBake停止执行，即使其他任务是可以被执行的。使用-k选项，

```
$ bitbake -k core-image-minimal
```

命令BitBake继续构建，即使一个或者多个任务已经失败了。BitBake尽可能地构建直到失败的任务阻止目标和任何依赖于它的事物。

使用不带版本和修订的目标总是构建软件包的最新版本或者由PREFERRED_VERSION变量指定的版本。版本和修订号码可以被加到目标名字上以构建特定的版本而不管它是否是最新版本并且不管PREFERRED_VERSION的值。如下命令

```
$ bitbake editor-1.0
$ bitbake editor-2.0-r3
```

分别构建editor软件包的1.0版本和2.0版本/修订r3。

执行不带依赖处理的构建

使用-b或者--buildfile选项和菜谱文件的名字，包括.bb后缀，执行菜谱的默认任务但是不带构建依赖：

```
$ bitbake -b core-image-minimal.bb
$ bitbake --buildfile=core-image-minimal.bb
```

如果core-image-minimal的任何依赖都不被实现，那么BitBake以错误消息退出而不尝试构建依赖。

执行特定任务

目标的特定任务可以使用-c或者--cmd选项来执行：

```
$ bitbake editor -c compile
$ bitbake editor --cmd=compile
```

由于其正在执行默认任务，所以依赖都是被遵守的，并且如果它们还没有被实现那么便会被构建。使用

```
$ bitbake -b editor.bb -c compile
$ bitbake -b editor.bb --cmd=compile
```

尝试运行编译任务而不构建任何依赖。

强制执行

BitBake为每一个成功完成的任务创建时间戳。如果，在任务的后续运行上，任务的时间戳是当前的或者是更新于那个任务所依赖的所有任务的时间戳，那么BitBake不运行这个任务。选项-C或者--clear-stamp命令BitBake运行任务而不管时间戳：

```
$ bitbake zlib -C compile
$ bitbake zlib --clear-stamp=compile
```

这些选项也可以和-b或者--buildfile选项结合。

-C或者--clear-stamp选项等同于使用

```
$ bitbake zlib -c compile -f
$ bitbake zlib -c compile --force
```

任何构建系统的重要功能是，如果任何输入变化，那么决定构建过程的什么部分必须被再次执行的能力。为了这个目的，BitBake实现了共享状态缓存。共享状态缓存在任务级别操作，并且为每个菜谱的每个任务决定它的输入是什么以及存储任务增加什么输出到构建过程。如果任务的输入没有变化，那么在后续构建上，它的输出不变。如果是那种情况，那么BitBake从共享状态缓存获取和恢复任务的输出而不是运行实际的任务。从共享状态缓存获取和恢复任务输出的操作被称为set-scene。如果set-scene是不被希望的，那么可以使用--no-setscene选项来关闭它：

```
$ bitbake zlib -c compile --no-setscene
```

选项--no-setscene也可以结合-b或者--buildfile选项。

共享状态缓存是强有力的功能，其通过重用来自缓存的已经处理过的任务输出来极大地加速构建过程。

展示元数据

为了调试目的，列出所有元数据是非常有帮助的。选项-e或者--environment在BitBake已完成解析过程后立即显示所有元数据、变量和函数。

```
$ bitbake -e
$ bitbake --environment
```

当和目标或者菜谱名字一起使用的时候，命令展示BitBake应用到构建那个包的环境设置：

```
$ bitbake -e zlib
$ bitbake --environment zlib
```

这些选项也可以和-b或者--buildfile选项结合。

因为展示包含函数的所有元数据会产生大量滚动在屏幕上的输出，所以建议使用例如grep的实用工具来过滤需要的信息。

另外有帮助的选项是-s或者--show-versions，它展示所有菜谱和版本号的列表：

```
$ bitbake -s
$ bitbake --show-versions
```

这些选项总是列出整个菜谱名单，使用过滤实用工具可能有助于缩短列表成你正在寻找的条目。

创建依赖性关系图（Dependency Graph）

BitBake可以用DOT语言创建描述包依赖性的图形。DOT是可以描述无向图和有向图并且用属性注释它们以及节点和边的普通文本图形描述语言。来自Graphviz包（www.graphviz.org）的软件应用可以读取DOT文件并且以图形形式渲染它们。如下命令

```
$ bitbake -g zlib
```

在当前工作目录中使用DOT语言创建3个描述zlib包的依赖性的文件：

·package-depends.dot：实际包级别且详细说明子包的依赖性关系图

·task-depends.dot：任务级别的依赖性关系图

·pn-depends.dot：包名级别且不详细说明子包的依赖性关系图

作为一种对-g选项的选择，--graphviz选项可以被使用。

如果Graphviz包是被安装在开发系统上的，那么你可以从这些文件创建依赖性关系图的图形化渲染。如下命令

```
$ dot -Tpng -o pn-depends.png pn-depends.dot
```

创建包含依赖性关系图的、png格式的图片。依赖性关系图往往变得相当大，因为所有依赖性，包括例如编译器和C库在内的通用依赖性，都被展示。你可以使用-I或者--ignore-deps选项来省略通用的包：

```
$ bitbake -g zlib -I expat
$ bitbake -g zlib --ignore-deps="expat"
```

这些命令把expat包从依赖性关系图中去除。不幸的是，你必须要逐个指定每一个要从依赖性关系图中去除的包，因为没有方法可以用单一选项一次性去除通用依赖性。

由BitBake生成的依赖性关系图也包括传递依赖，使得图比必要的大。你可以用Graphviz包中的tred命令去除传递依赖：

```
$ tred pn-depends.dot > pn-depends-notrans.dot
```

BitBake也包括了内置的、用于依赖性关系图的可视化工具——依赖性浏览器。使用

```
$ bitbake -g -u depexp zlib
```

启动依赖性浏览器，显示在图4-1中。

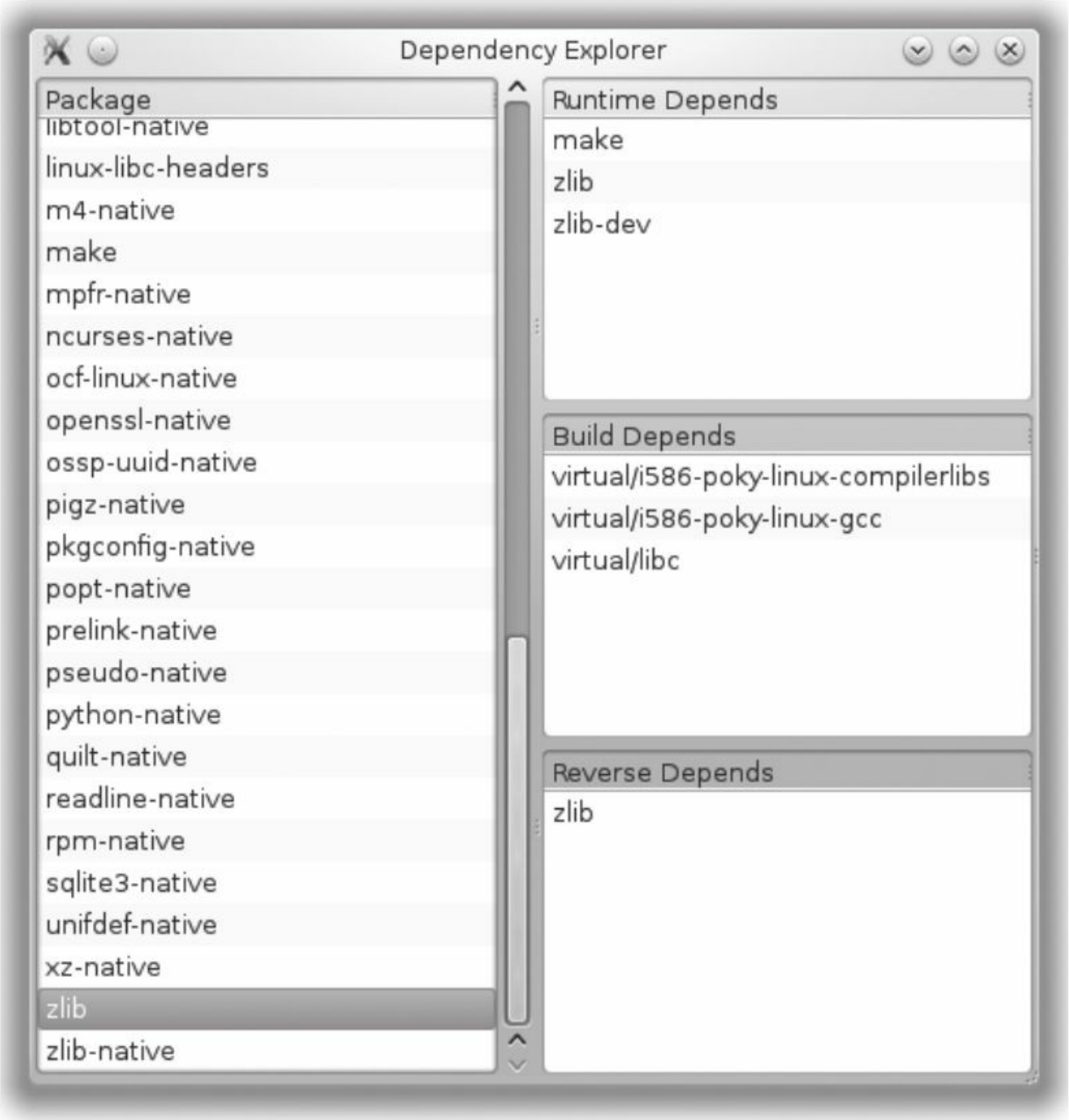

图4-1　依赖性浏览器

依赖性浏览器按照每个包列出运行时、构建时和逆向依赖性。运行时依赖性是软件包在运行时所要求的全部依赖性。构建时依赖性必须在构建软件时被实现。逆向依赖性列表显示所有依赖于这个包的包。

提供和覆盖配置数据

选项-r或者--read和-R或者--postread通过便利的方法来提供额外的配置数据或者覆盖现有的设置而不修改构建环境的任何配置文件，例如bitbake.conf或者local.conf。

作为替代通过命令上的shell直接设置变量的方法，正如前面我们所看到的，-r或者--read可以用来在工具读取任何其他文件之前向BitBake提供配置数据：

```
$ bitbake -r prefile.conf <target>
$ bitbake --read prefile.conf <target>
```

这种技术可以用于自动构建系统替代bblayers.conf文件来设置BBPATH和BBLAYERS变量以动态地设置构建环境。

使用

```
$ bitbake -R postfile.conf <target>
$ bitbake --postread postfile.conf <target>
```

允许方便地覆盖由构建环境的任何其他配置文件做出的变量设置，因为BitBake在它解析完所有配置文件以后再处理postfile.conf。例如，机器或者发行版配置可以被动态地覆盖，或者变量可以为了调试目的而被设置。

运行BitBake服务器

BitBake是客户端–服务器应用。每次你运行BitBake，它就在后台启动名为cooker的服务器或者后端进程以及用于用户界面的客户端或者前端进程。cooker后端进程进行所有元数据文件处理以及实际的构建，最终创建多个线程。当以这种多合一模式启动时，后端和前端进程使用基于管道的进程间通信（Interprocess Communication，IPC）来交换信息。

后端和前端进程也可以在不同的系统上独立地启动，使得你可以远程启动和监控构建过程，为了从构建环境中启动BitBake服务器进程，使用如下命令：

```
$ bitbake --server-only --servertype=xmlrpc --bind=<ip>:<port>
```

这个命令启动了作为后台进程、监听在IP地址<ip>和端口<port>的BitBake服务器，你可以绑定服务器到任何IP地址，例如本地（localhost）或者你构建系统的任何网络接口的IP地址。你也可以使用任何端口，只要它不是特权所有或者正在使用中。在启动BitBake时，你必须指定--servertype=xmlrpc。虽然这看起来是冗余的，因为process和xmlrpc是当前仅有的2种被支持的进程间通信方法，并且process不能和服务器模式一起使用，在未来，BitBake可能支持额外的用于服务器模式的进程间通信方法。

为了使用默认的文本终端类型的用户接口连接到运行中的BitBake服务器并且执行命令，使用如下命令：

```
bitbake --servertype=xmlrpc --remote-server=<ip>:<port> <target>
```

它在远程服务器上启动针对<target>的构建过程。

4.3 BitBake元数据

BitBake使用元数据来控制构建过程。总的来说，元数据描述软件包，也描述它们是如何被构建以及它们如何互相关联和互相依赖。BitBake区分2种类型的元数据：

·变量：变量被赋予值和等价于值的表达式。变量可以是对整个构建系统全局有效的或者对当前的上下文本地有效，例如对于特定的菜谱。许多BitBake元数据变量不仅包含单一的值，也包含由空格分隔的值的列表。

·可执行元数据：可执行元数据是嵌入在菜谱和类中的由BitBake在菜谱的上下文中执行的函数和任务。

元数据被组织进5类文件：

·配置文件（.conf）：放在配置文件中的元数据是全局的并且影响所有引用它们的菜谱。配置文件可能只包含变量而没有可执行元数据。如果相同的变量在多个配置文件中被赋值，那么由层优先级建立的顺序决定哪个设置生效。配置文件bitbake.conf有最低的优先级，构建环境的本地配置文件local.conf有最高的优先级。

·菜谱文件（.bb）：菜谱包含描述特定软件包以及如何构建该软件包的元数据。菜谱通常提供任务形式的，带有用于下载、解压、打补丁、编译、打包和安装软件包的指令的可执行元数据。

·类文件（.bbclass）：类文件提供一种简单的、用于菜谱共享相同构建指令的继承机制。BitBake在层的classes子目录中搜索类文件。菜谱可以通过使用inherit指令，用类文件的名字引用它们来包含类文件。类是全局的，这意味着，位于层中的菜谱可以从构建环境包含的任何其他

层中继承类。

·追加文件（.bbappend）：追加文件是菜谱文件的扩展。通常，层使用追加文件来扩展包含在其他层中的菜谱。追加文件必须拥有和它所扩展的菜谱相同的基名称，但其是.bbappend后缀而不是.bb后缀。追加文件也必须要有和它正在追加的菜谱相同的相对于层的根目录的路径。追加文件要么增加额外的元数据要么修改定义在菜谱中的元数据。追加文件的内容是逐字追加到原始菜谱的。如果来自不同层的追加文件追加相同的菜谱，那么层的优先级决定BitBake以何种顺序追加文件到菜谱。

·包含文件（.inc）：通过使用include和require指令，任何元数据文件可以包含其他文件。包含文件通常提供多个元数据文件共享的元数据。包含文件的内容被插入到正在包含的元数据文件的各个指令所在的位置。这当然承担着圆形夹杂的风险，BitBake会检测并对这种情况作出警告。文件包含不限于相同的层，但是层中的菜谱可以包含来自另一层的文件。.inc文件后缀是完全惯例的。一个元数据文件可以包含任何其他元数据文件；然而，包含可执行元数据的文件可能仅仅被菜谱、追加文件和类所包含。

BitBake在启动后立即解析元数据文件并且创建元数据缓存。这个缓存本质上是BitBake的元数据字典的持久形式。只要元数据没有变化，BitBake从缓存中读取它会极大地减少启动时间。

4.4 元数据语法

BitBake元数据文件使用相当简单明了的语法。在某种程度上，它类似于你可能熟悉的、来自Makefiles和shell脚本的语法。

4.4.1 注释

在元数据文件内的注释被前缀以#符号。注释必须在第一列开始，除非它们被置于shell或者Python函数内部（见程序清单4-4）。

程序清单4-4 注释

```
# This is a comment.

    # This is an invalid comment, which causes a parse error.

SUMMARY = "Sample file with comments" # this comment is also invalid

helloworld () {
   # comments in shell functions can start on any column
   # and can also be added after statements
   echo "Hello World!" # this is okay in a shell function
}

python printdate () {
   # comments in Python functions can start on any column
   # and can also be added after statements
   import time
   print time.strftime("%Y%m%d", time.gettime()) # ok too
}
```

就像任何编程语言一样，有含义注释的频繁使用使得代码对别人来说更容易被阅读和理解，并且总是被鼓励的。

4.4.2 变量

BitBake变量是无类型的。BitBake把所有赋给变量的值都视作字符串。

变量名

BitBake变量名可能包含大写和小写字母、数字以及特殊字符：下划线（_）、连接符（-）、逗号（,）、加号（+）和波浪号（～）。它们也可以以任何这些字符开头。

虽然前面的任何字符在变量名中都是被允许的，但是依据惯例，只有大写字母和下划线是被使用的，并且所有的变量名以大写字母开头。

变量范围

在配置文件（也就是，以.conf结尾的文件）中定义的变量是全局的并且在所有菜谱中可见。在菜谱中定义的变量是仅仅本地于菜谱的。菜谱由它们自己的名字空间，在菜谱中被赋值的全局变量仅仅当BitBake处理那个菜谱的时候才保持该值。

变量赋值

所有BitBake变量赋值都是带有包围值的成对分隔符的字符串字面量。分隔符要么是双引号（"）要么是单引号（'）。按照惯例，双引号是更受欢迎的。

直接赋值（=）

可以使用=符号来给变量赋值。使用

```
VAR = "value"
```

把value赋给变量VAR。在变量赋值中的引用可以通过使用反斜杠（\）作为转义字符或者单引号来实现：

```
VAR1 = "This is a \"quotation\" inside a variable assignment"
VAR2 = "This is a second 'quotation' inside a variable assignment"
```

为了可阅读性，第2种方法是更受欢迎的。

默认赋值（?=）

可以使用?=赋值操作符赋予变量默认值：

```
A ?= "value1"
B ?= "value2"
B ?= "value3"
C ?= "value4"
C = "value5"
```

如果变量以前没有被设置过，那么它被赋予默认值。如果它在默认赋值之前已经被设置过了，那么保留它的值。在前面的赋值序列中，如果A以前没有被设置过，那么A包含value1。B包含value2，因为第一次使用?=操作符设置了变量。

=操作符覆盖任何以前的默认赋值，因此，C包含value5。

弱默认赋值（??=）

当使用弱或者惰性默认赋值操作符??=时，赋值直到解析过程结束才发生，以至于针对变量的最后而不是第一个??=赋值会被使用：

```
A ??= "value1"
B ??= "value2"
B ??= "value3"
C ?= "value4"
C ??= "value5"
D = "value6"
D ??= "value7"
```

在这个例子中，如果A以前没有被设置过，那么A包含value1。如果B以前没有被设置过，那么B包含value3，因为赋值直到解析过程结束才发生。C包含value4，D包含value6，因为?=和=操作符覆盖??=操作符。

变量扩展

BitBake变量可以引用其他BitBake变量的内容：

```
VAR1 = "jumps over"
VAR2 = "The quick brown fox ${VAR1} the lazy dog."
```

可以使用${}引用操作符和变量名来引用变量的内容。在这个例子中，VAR2包含“The quick brown fox jumps over the lazy dog.”。

直接变量扩展

变量扩展直到变量实际被使用到时才发生。简单地赋值包含变量扩展的表达式不会扩展引用的变量。然而，赋值操作符:=引起了赋值上的直接扩展：

```
VAR1 = "jumps over"
VAR2 = "${VAR1} the lazy dog. "
VAR1 = "falls on"
VAR3 = "The rain in Spain ${VAR1} the plain."
VAR4 := "The quick brown fox ${VAR2}"
```

变量VAR4包含“The quick brown fox falls on the lazy dog.”因为在VAR2赋值中包含的VAR1直到VAR2的内容在使用:=操作符赋值VAR4

而被引用时才被扩展。然而，截至那时，VAR1的值已经被设置成了falls on。

Python变量扩展

BitBake可以对变量赋值中的Python表达式进行求值：

```
DATE = "${@time.strftime('%A %B %d, %Y', time.gettime())}"
TODAY := "Today is: ${DATE}."
```

@操作符告诉BitBake把随后的表达式视作Python代码。这个代码必须被求值成值。在这个例子中，TODAY将包含类似于“Today is：Friday April 1,2016.”的值。

变量追加和前新增（Prepending）

变量内容可以使用追加和前新增操作符来和其他字符串字面量以及来自其他变量的内容连接。

带空格的追加（+=）和前新增（=+）

+=和=+操作符分别追加和前新增变量，同时在值之间增加空格：

```
VAR1 = "12"
VAR1 += "34"
VAR2 = "89"
VAR2 =+ "67"
VAR3 = "5"
VAR3 += "${VAR1}"
VAR3 =+ "${VAR2}"
```

这个例子结果是，变量VAR1包含12　34，变量VAR2包含67　89以及变量VAR3包含12　34　5　67　89。

不带空格的追加（.=）和前新增（=.）

.=和=.操作符分别追加和前新增变量而不在值之间放置额外的空格：

```
VAR1 = "12"
VAR1 .= "34"
VAR2 = "89"
VAR2 =. "67"

VAR3 = "5"
VAR3 .= "${VAR1}"
VAR3 =. "${VAR2}"
```

这个例子的结果是，变量VAR1包含1234，变量VAR2包含6789，变量VAR3包含123456789。

使用_append和_prepend操作符追加和前新增

也可以使用加到变量名上的特殊的_append和_prepend操作符来被追加和前新增变量值：

```
VAR1 = "12"
VAR1_append = "34"
VAR2 = "89"
VAR2_prepend = "67"
VAR3 = "5"
VAR3_append = "${VAR1}"
VAR3_prepend = "${VAR2}"
```

这个例子的结果是，变量VAR1包含1234，变量VAR2包含6789，变量VAR3包含123456789。

_append和_prepend操作符不插入任何空格。如果空格是需要的，那么你必须用字符串字面量包括它们。

去除（_remove）

在变量中包含空格分隔的值列表的单一值可以通过_remove操作符

去除：

```
VAR1 = "123 456 789 123456789 789 456 123 123 456"
VAR1_remove = "123"
VAR1_remove = "456"
```

这个例子的结果是，变量VAR1包含字符串789　123456789　789。

条件变量赋值

变量OVERRIDES包含由冒号（:）分隔的值的列表。每个值代表需要被实现的条件：

```
OVERRIDES = "conda:condb:condc"
```

BitBake从右到左处理条件，这意味着，右边的优先级高于左边的。

条件变量设置

变量通过用下划线（_）追加条件到变量名而被有条件地设置：

```
OVERRIDES = "sun:rain:snow"
PROTECTION = "unknown"
PROTECTION_sun = "lotion"
```

在这个例子中，变量PROTECTION包含lotion，因为条件sun被包含在OVERRIDES列表中。考虑如下的例子，其中2个条件都被包含在OVERRIDES列表中：

```
OVERRIDES = "sun:rain:snow"
PROTECTION_rain = "umbrella"
PROTECTION_snow = "sweater"
```

在这个例子中，变量PROTECTION包含sweater，因为条件snow相

对于条件rain有更高的优先级。

在下面的例子中，变量PROTECTION包含umbrella，因为条件hail不被包含在OVERRIDES列表中：

```
OVERRIDES = "sun:rain:snow"
PROTECTION_rain = "umbrella"
PROTECTION_hail = "duck"
```

像任何其他元数据，变量OVERRIDES也可以引用其他变量的内容：

```
OVERRIDES = "sun:rain:snow:${OTHER}"
OTHER = "hail"
```

条件变量设置允许默认值的赋值在特殊条件被满足时用特定的值覆盖它。这个方法被构建系统频繁使用，例如，特定参数需要被传递到编译器用于编译器机器依赖的代码。

条件追加和前新增

条件也可以用来追加和前新增变量：

```
OVERRIDES = "sun:rain:snow"
PROTECTION = "sweater"
PROTECTION_append_rain = "umbrella"
```

这个例子的结果是，PROTECTION被设置成sweater umbrella。

用更高优先级（更加靠右于OVERRIDES）追加和前新增会更优先，正如条件变量赋值的情况一样。

4.4.3 包含

元数据文件可以包含其他元数据文件以允许共享的设置。常见的用例是构建相同软件包的不同版本的菜谱。包含文件提供共享的设置，例如构建指令、安装目录等，它们由特定于要构建的版本的设置以及实际菜谱来补充，例如下载位置和源文件的名字。

BitBake提供2个针对可选包含和必须包含的包含指令：

```
include optional.inc   # optional inclusion
required mandatory.inc # required inclusion
```

当用include指令实现可选包含时，BitBake尝试定位包含文件，但是即使不能找到文件，它也会悄无声息地继续操作。相反，用required指令实现的必须包含使得BitBake以错误信息退出。

对大部分用例来说，用required指令包含一个文件是受欢迎的，因为如果BitBake不能定位包含文件它便会警告你，这可以防止错误，例如在路径和文件名字中的错别字。然而，有时受欢迎的是，提供不要求直接改变元数据文件的定制化方法。对这样的情况来说，把include指令放进元数据文件会提供用于可选的定制化的机制。如果定制化是希望的，那么包含文件可以被提供，但是如果它是不必要的并且包含文件不存在，那么BitBake简单地忽略包含。

指令include与required可以和相对路径以及绝对路径一起使用：

```
include file1.inc
include meta-test/recipes-core/images/file2.inc
required /home/build/yocto/file3.inc
```

当相对路径被使用的时候，BitBake尝试使用由BBPATH变量指定的文件路径列表来定位文件。BitBake使用它找到的、有正确路径段和

文件名的第一个文件。

在BitBake定位到包含文件后，它解析包含文件的内容，并且在它遇到包含指令的位置把内容插入到起始包含文件。因此，包含文件可以覆盖原先由起始包含文件所做出的设置，反过来也一样，使得包含指令被放置到起始包含文件的恰当位置是重要的。

当然，包含文件必须符合起始包含文件的BitBake元数据语法。菜谱和类可以包含含有配置项以及可执行元数据的文件。然而，配置文件只可以包含含有配置项但不含有可执行元数据的文件，因为后者在配置文件中是不被支持的。

被包含的文件本身可以包含其他文件。那个实践存在循环包含的风险，因为BBPATH搜索顺序的原因，特别是对于被相对路径包含的文件。然而，BitBake检测循环包含并且以错误信息终止。

4.4.4 继承

通过类，BitBake提供了简单的继承机制。类可以被菜谱、追加文件和其他类用inherit指令来继承。

```
inherit myclass
```

类是带有.bbclass文件名后缀的、被放置进元数据层的classes子目录中的元数据文件。指令inherit仅仅使用类名，它是不带后缀的类文件名的基名称。它仅仅可以用在菜谱、追加文件和其他类文件中。

第一眼看来，继承似乎是非常相似并且最终冗余于包含的。然而，区别在于BitBake如何处理和解析类：

·BitBake通过它们的类名来识别类而不是通过它们的文件名和路径，这意味着类名字必须是在所有构建环境包含的元数据层中唯一的。

·BitBake在解析完配置文件之后和解析菜谱之前会解析一次类。包含文件是在BitBake遇到包含指令时被解析的。如果相同的包含文件是被多个其他文件所包含的，那么在起始包含文件的上下文中，BitBake多次解析相同的文件。这使得类对于由许多不同菜谱共享的构建指令来说是更加高效的机制。然而，对于菜谱构建相同软件包的不同版本来说，使用包含文件是好的选择，因为通常一次仅有特定软件包的一个版本是被构建的。

·BitBake的DataSmart写时复制（Copy-on-Write，COW）数据存储仅仅维护类的一个副本，即使类文件被数百个菜谱使用，使用包含文件也可能导致数据的重复。

通用类的使用简化了很多菜谱。例如，针对构建软件包的autotools

类利用GNU　Autotools配置机制可以将菜谱减少为数行代码（见程序清单4-5）。

程序清单4-5　使用autotools类

```
SUMMARY = "GNU nano - an enhanced clone of the Pico text editor"

LICENSE = "GPLv3"

LIC_FILES_CHKSUM = "file://COPYING;md5=f27defe1e96c2e1ecd4e0c9be8967949"

DEPENDS = "ncurses"
PR = "r0"

PV_MAJOR = "${@bb.data.getVar('PV',d,1).split('.')[0]}\
.${@bb.data.getVar('PV',d,1).split('.')[1]}"

SRC_URI = "http://www.nano-editor.org/dist/v${PV_MAJOR}/nano-${PV}.tar.gz\
           file://ncursesw.patch"

SRC_URI[md5sum] = "af09f8828744b0ea0808d6c19a2b4bfd"

inherit autotools gettext
RDEPENDS_${PN} = "ncurses"
```

前面的菜谱构建GNU　nano编辑器，它是自动工具化的软件包。菜谱本身仅仅指定了SRC_URI和包名字，而所有构建自动工具化的软件包的复杂度都是被隐藏在autotools类中的。

4.4.5 可执行元数据

菜谱、追加文件和类可以包含可执行元数据的定义。可执行元数据是BitBake可以执行的shell或者Python函数。

BitBake把可执行元数据和变量等同对待：函数名是和代表被赋予的值的函数代码一起存储在数据字典中的。结果就是，函数可以像普通变量一样被追加和前新增，并且可能有元数据的属性。

定义在菜谱和追加文件中的元数据函数的范围是本地于特定文件的，而定义在类中的函数是全局的。

shell函数

shell函数被定义在元数据文件中，正如你会在普通shell脚本中定义它们（见程序清单4-6）。

程序清单4-6　可执行元数据shell函数

```
helloworld () {
    echo "Hello, World!"
}
```

在函数体内的代码遵循普通shell的语法。实际上，BitBake在执行shell函数时调用shell的解释器/bin/sh。理想的是，所有shell函数应该被写成无关于系统默认的特殊的shell解释器。在所有由Yocto项目官方支持的Linux发行版上，默认的shell解释器是Bourne Again Shell（bash）。为了使你自己的代码可移植，建议避免shell特定的扩展并且遵循针对bash衍生的共同特性，例如.sh、.bash、.ksh和.zsh。

Python函数

可执行元数据也可以被定义为Python函数（见程序清单4-7）。

程序清单4-7　可执行元数据Python函数

```
python printdate () {

    import time
    print time strftime('%Y%m%d', time.gettime())
}
```

关键词python告诉BitBake其后的代码被解析成Python代码。

为可执行元数据使用Python使得你能导入任何Python模块并且利用许多可用的函数。在函数体内，你必须注意Python针对代码段的缩进方案。建议使用空格替代制表符用于缩进。

全局Python函数

函数可以通过使用def关键词而被全局定义，并且不管它们被定义在什么文件中，如程序清单4-8所示。

程序清单4-8　全局Python函数

```
def machine_paths(d):
    """List any existing machine specific filespath directories"""
    machine = d.getVar("MACHINE", True)
    filespathpkg = d.getVar("FILESPATHPKG", True).split(":")
    for basepath in d.getVar("FILESPATHBASE", True).split(":"):
        for pkgpath in filespathpkg:
            machinepath = os.path.join(basepath, pkgpath, machine)
            if os.path.isdir(machinepath):
                yield machinepath
```

语法和定义任何标准Python函数完全相同。

因为函数是全局的，我们可以从任何其他Python元数据函数中调用它。

匿名Python函数

菜谱、追加文件和类可能使用__anonymous关键词作为函数名字或者一起省略函数名来定义匿名Python函数（见程序清单4-9）。

程序清单4-9　匿名Python函数

```
python __anonymous () {
    # Anonymous function using the __anonymous keyword
    ...
}
python () {
    # Anonymous function omitting the function name
    ...
}
```

在特殊单元解析过程的结尾，BitBake执行匿名函数。例如，定义在菜谱中的匿名函数在菜谱已经被解析后执行。

任务

前面的部分描述了如何定义shell和Python函数作为可执行元数据。问题是，BitBake如何从元数据文件中执行函数？我们已经看到BitBake在完成解析菜谱或者类后如何执行匿名函数。

BitBake把特殊函数认作任务。任务是被定义在菜谱和类中的，并且可以：

·针对特殊菜谱直接从BitBake命令行调用

·被BitBake作为构建过程的部分而自动化地执行

程序清单4-10提供了例子。

程序清单4-10　定义任务

```
python do_clean () {
    # task to clean remove build output
    ...
}
addtask clean
do_build () {
    # task to build the software package
    ...
}
addtask build
do_download () {
    # task to download sources
    ...
}
addtask download before do_build
do_unpack () {
    # task to unpack sources
    ...
}
addtask unpack before do_build after do_download
do_compile () {
    # task to compile software package
    ...
}
addtask compile before do_build after do_unpack
do_install () {
    # task to install binaries, libraries, and documentation
    ...
}
addtask install after do_build
```

为了将shell或者Python函数定义为任务，它的名字必须被前缀以do_。除了这个，任务和任何其他可执行元数据完全类似。指令addtask被用来增加任务到BitBake任务列表和定义任务执行链。

在程序清单4-10中的代码定义了任务clean并且用addtask将其增加到任务列表。这个技术允许任务可以从BitBake命令行调用。让我们假设程序清单4-10的代码被放在名为myrecipe.bb的菜谱中。使用

```
bitbake myrecipe -c clean
```

调用菜谱的clean任务。菜谱使用before和after与addtask指令一起增加任务build、download、unpack、compile和install，创建执行链。运行以下命令：

```
bitbake myrecipe -c build
```

首先运行download任务，其次是unpack任务，再次是compile任务，然后是build，以及最后是install任务。

当BitBake被以菜谱作为参数调用但没有指定要运行的任务时，那么BitBake运行默认任务。默认任务是通过变量BB_DEFAULT_TASK来被定义的。通过base类设置该变量为：

```
BB_DEFAULT_TASK ?= "build"
```

其使得build成为类似于针对makefiles的all目标的默认任务。菜谱和类当然可以覆盖BB_DEFAULT_TASK以将其设置为不同的任务。

从函数中访问BitBake变量

BitBake变量可以从shell和Python函数中被访问到。

从shell函数中访问变量

从shell函数内，BitBake变量可以直接通过使用变量扩展表示法而被访问（见程序清单4-11）。

程序清单4-11　从Shell函数中访问变量

```
BPN = "myapp-v1"
MAKE = "make"
EXTRA_OEMAKE = "CFLAGS='-W -Wall -Werror -Wundef -Wshadow ${CFLAGS}'"

do_compile () {
        bbnote ${MAKE} ${EXTRA_OEMAKE} "$@"
        ${MAKE} ${EXTRA_OEMAKE} "$@" || die "oe_runmake failed"
}

do_install () {
        BPN = "myapp"
        docdir = "docs"
        install -d ${D}${docdir}/${BPN}
}
```

可以从shell函数中读取变量和写变量。然而，写变量仅仅在shell脚

本的范围内局部性地改变它的值。在程序清单4-11中，BPN是在do_install中被覆盖的BitBake变量。被覆盖的值仅仅对do_install函数有效。考虑到BitBake创建针对do_install函数的带有所有变量设置和扩展的shell脚本并且执行该脚本，那个惯例是容易理解的。

从Python函数中访问变量

从Python函数中访问BitBake比从shell函数中访问稍微复杂一点儿，因为BitBake变量不能被直接读取或者写而是必须用特殊函数从BitBake数据字典中获取并且通过BitBake数据字典操作（见程序清单4-12）。

程序清单4-12　从Python函数访问变量

```
HELLO = "Hello, World!"
DATE = ""

python printhello () {
        hello = d.getVar('HELLO', True)
        print hello
}

python setdate () {
        import time
        date = time.strftime('%A %B %d, %Y', time.gettime())
        d.setVar('DATE', date)
}
```

函数getVar和setVar通过BitBake的数据字典提供对变量的访问，它是由全局Python变量d所引用的。BitBake的数据字典是作为Python类实现的。函数getVar和setVar是该类的方法。对于两个函数的第一个参数都是要被访问的变量的名字。对getVar的第二个参数是告诉函数是否扩展变量表达式的布尔型（Boolean）值。对setVar的第二个参数是变量的新值。

创建本地的数据字典副本

使用setVar和全局数据字典全局性地修改那个变量，并且为所有后

续的操作修改了它的值。如果不需要，那么可以创建数据字典的副本：

```
localdata = d.createCopy()
```

createCopy方法创建对数据字典的新引用。因为BitBake的数据字典使用写时复制，变量的实际副本直到写操作访问时才被创建。

访问包含值列表的变量

许多BitBake变量包含值是由分隔符分隔的值列表。通常，从数据字典中获取出来后，这些列表变量被转化成Python数组（见程序清单4-13）。

程序清单4-13　访问变量列表

```
python do_download () {
        uris = (d.getVar('SRC_URI', True) or "").split(" ")
        for uri in uris:
           # process the source URIs
           ...
}
```

程序清单4-13的样例代码从数据字典中获取变量SRC_URI，它是空格分隔的URI字符串。然后它使用split操作符把字符串拆分成字符串数组。如果没有在数据字典中找到变量SRC_URI，那么术语or""返回空的字符串以使代码具有故障保护能力。

你会发现这个样例代码的变体被频繁使用在菜谱和类中以获取和处理列表。

追加和前新增函数

类似变量，可以使用_append和_prepend操作符追加和前新增函数（见程序清单4-14）。

程序清单4-14　追加和前新增函数

```
python printdate () {
    import time
    print time.strftime('%A %B %d, %Y', time.gettime())
}

python printdate_prepend () {
    print ('Today is: ')
}

python printdate_append () {
    print('MOTD: A good day for the Yocto Project.')
}
```

在程序清单4-14中的代码会产生组合的printdate函数（见程序清单4-15）。

程序清单4-15　组合函数

```
python printdate () {
    print ('Today is: ')

    import time
    print time strftime('%A %B %d, %Y', time.gettime())
    print('MOTD: A good day for the Yocto Project.')
}
```

当然，通常你不在同一个文件中追加和前新增函数。追加和前新增的典型用例是继承类和增加到由该类定义的函数的菜谱或者是扩展由其追加的菜谱定义的函数的追加文件。

追加和前新增可用于shell和Python函数。

4.4.6 元数据属性

所有BitBake元数据（也就是，变量以及函数）可以有属性（attribute），或者被称为标识（flag）。属性提供标记额外信息到元数据的方法。

BitBake元数据语法通过将括号（[]）中的属性名添加到变量和函数名来提供设置属性：

```
VAR[flag] = "flagvalue"
```

可以用+=、=+、.=和=.操作符来追加和前新增属性。用来为属性赋值的表达式可能使用变量扩展。然而，不可能通过BitBake元数据语法读取属性的值。

可以从Python函数中使用BitBake数据字典方法getVarFlag和setVarFlag来读取和写属性的值（见程序清单4-16）。

程序清单4-16　元数据属性（标识）

```
func[run] = "1"

python func () {
    run = d.getVarFlag('func', 'run')
    if run == 1:
        # do some work
        ...
        d.setVarFlag('func', 'run', 0)
    else:
        # do not run at this time
}
```

方法getVarFlag的第一个参数是变量的名字，第二个参数是标识的名字。setVarFlag使用3个参数：变量名、标识名和标识值。

4.4.7 元数据名（键）扩展

扩展也可以被应用到元数据名。实际的扩展发生在数据处理过程的正结尾，紧接在条件赋值、追加和前新增之前。

```
A${B} = "foo"
B = "2"
A2 = "bar"
```

变量A2包含foo，因为表达式A${B}="foo"的完全赋值直到表达式A2="bar"之后的解析过程的结尾才被判断。

4.5 源下载

一开始便有源，源可以在任何地方以任何格式呈现。因此，能够构建由数百个软件包组成的Linux操作系统栈的构建系统必须要能从大量源获取它们，其通常被称为上游仓库（upstream repositories）。上游仓库可以是本地的或者是远程的文件服务器或者软件配置或者例如Git、Subversion等的修订控制系统。可以例如tar的包形式打包源代码，且通常使用各种格式压缩这些包。当使用源代码控制管理系统（Source Control Management，SCM）时，通常以源树的形式一个文件一个文件地检出源代码。

除了提供用于从许多不同源获取源代码包的必要功能性，重要的是，构建系统提供一致和透明的从构建菜谱访问仓库而不要求最终用户了解仓库与协议实现的特性的方法。

BitBake通过它的fetcher架构提供了必要的框架。在BitBake术语中，获取是取得源文件的过程。来自BitBake fetch2库（bitbake/lib/bb/fetch2）的Python Fetch类通过遵循如下格式的源URI来为源代码获取呈现统一的接口：

```
<scheme>://[<user>[:<pwd>]@]<host>[:<port>]/<path>[;<key>=<value>;..]
```

就大部分而言，BitBake获取器（fetcher）URI符合IETF的RFC3986标准，除了路径的处理以外。RFC3986提供了绝对和相对路径。参考标准定义有效的绝对路径为：

```
file://hostname/absolute/path/to/file
file:///absolute/path/to/file
```

BitBake支持用于绝对路径的第二种形式，但是不识别第一种形

式。根据标准，有效的相对路径可以被指定成：

```
file:relative/path/to/file
```

BitBake确实支持使用那种格式的相对路径，同时额外识别下面这个不符合RFC3986的格式：

```
file://relative/path/to/file
```

这个格式被支持是为了带有获取器库的前期实现的向后兼容性以避免重写旧的菜谱。

BitBake URI也没有由IETF标准提供的查询（query）和片段（fragment）的标记。

Fetch基类延迟URI的实际处理和访问资源到特定的、由URI的方案（scheme）所确定的实现。可选参数列表依赖于获取器的特殊实现。某些方案可能访问需要带用户名和密码的身份验证的资源。那些可以通过使用标准标记而被包含在URI中。

Fetch基类的重要细节是，在实例化中，被传到Fetch基类的源URI参数可能不只包含单一的URI而是以前面形式的、甚至可能使用不同方案的URI列表。

4.5.1 使用Fetch类

虽然BitBake提供了获取器架构和获取器实现，但是它不提供用于获取源文件的默认任务。为了使用获取器，你必须要在菜谱中实现任务，或者更倾向于在BitBake类中实现它。程序清单4-17显示了样例do_fetch任务的实现。

程序清单4-17　样例do_fetch任务

```
1  python do_fetch() {
2
3     bb.note("Downloading sources from ${SRC_URI} ...")

4
5     src_uri = (d.getVar('SRC_URI', True) or "").split()
6     if len(src_uri) == 0:
7        bb.fatal("Empty URI")
8
9     try:
10       fetcher = bb.fetch2.Fetch(src_uri, d)
11       fetcher.download()
12    except bb.fetch2.BBFetchException:
13       bb.fatal("Could not fetch source tarball.")
14
15    bb.note("Download successful.")
16 }
17
18 addtask fetch before do_build
```

样例任务的第5行首先从BitBake数据字典中获取SRC_URI变量。这个变量被假设成包含空格分隔的、到源仓库的URI的列表。列表被转化成Python列表变量（或者数组）。然后，第10行从Fetch基类创建了fetcher对象。第11行尝试从仓库中下载源。样例任务的其他代码提供用于调试的日志信息并且处理由获取器导致的异常。

使用这个do_fetch任务的菜谱将仅仅指定SRC_URI变量。所有下载源的实际工作是由任务和获取器实现来处理的。事实上，由OpenEmbedded核心（OpenEmbedded Core）元数据层提供的base.bbclass类实现了非常类似于样例的do_fetch任务。

这留下了样例任务代码没有直接回答的问题：获取器下载源到哪里？如果你分析BitBake获取器库的代码，那么你会注意到，获取器期望变量DL_DIR来包含它们下载源文件所到的目录路径。

4.5.2 获取器实现

BitBake提供针对几乎所有被开源项目使用的常见类型的上游仓库的获取器实现。接下来的部分将分别讨论最常被用到的获取器和它们的参数。

本地文件获取器

本地文件获取器从使用file：//URI方案访问到的文件系统中获取文件。这并不意味着这些文件存在于本地主机上。它们也可能位于从远程文件服务器上被挂载的文件系统上。

包含在URI中的路径可以是绝对的或者相对的：

```
SRC_URI = "file:///absolute/path/to/file"
SRC_URI = "file://relative/path/to/file"
```

在相对路径的情况下，获取器使用变量FILESPATH和FILESDIR来定位文件：

·FILESPATH包含由冒号（：）分隔的路径列表。获取器从列表中的第一个路径开始搜索文件。搜索每个目录直到有第1次的匹配。如果存在多个包含那个名字的文件的目录，那么获取器下载它找到的第一个文件。

·如果没有任何目录被包含在FILESPATH，那么获取器检查变量FILESDIR是否被设置。如果它是被设置的，并且包含单一的有效路径，那么获取器使用那个路径来定位文件。

如果在绝对路径的情况下，文件不存在，或者在相对路径的情况下，FILESPATH和FILESDIR都没有包括包含文件的路径，那么获取器

引发错误。

本地文件获取器实际并不“下载”——也就是说，从由URI指定的路径复制文件到DL_DIR。而是它的download方法仅仅验证文件是存在的。本地文件直接从它们的原始位置被访问。

通常，SRC_URI指定单一文件。然而，获取器也可以要么通过在路径名字或者SRC_URI中的任何位置使用通配符要么通过把SRC_URI指向目录来访问多个文件：

```
SRC_URI = "file://*/*.patch"
SRC_URI = "file://path/to/dir"
```

这两个形式都同等适用于绝对和相对路径。

本地文件获取器的实现可以在文件bitbake/lib/bb/fetch2/local.py中找到。

HTTP/HTTPS/FTP获取器

经常地，源是使用HTTP、HTTPS或者FTP协议从上游文件服务器下载下来的。BitBake默认使用GNU Wget实用程序来提供对这些协议的获取器实现。然而命令，包括任何命令行选项，都可以通过设置FETCHCMD_wget变量来指定。

以下是这个获取器支持的参数：

·md5sum：用于下载验证的MD5校验和。如果提供了，那么获取器计算针对下载后文件的MD5校验和，并且和由这个参数提供的值进行比较。如果它们不同，那么获取器引发错误。

·sha256sum：用于下载验证的SHA256校验和。如果提供了，那么

获取器计算针对下载后文件的SHA256校验和，并且和由这个参数提供的值进行比较。如果它们不同，那么获取器引发错误。

·downloadfilename：如果提供了，那么获取器把下载后的文件重命名为这个文件名。

·name：对URI的符号引用。

提供用于下载验证的校验和对于获取器来说是强制的。md5sum参数和sha256sum参数必须提供其一。如果两者都提供了，那么它们必须匹配。

以下是针对SRC_URI的例子：

```
SRC_URI = "http://host.site.org/downloads/srcpkg.tar.gz;md5sum=12345"
SRC_URI = "https://host.site.org/downloads/srcpkg.tar.gz;sha256sum=6789"
SRC_URI = "ftp://user:pwd@host.site.org/repo/srcpkg.tar.gz;md5sum=12345"
SRC_URI = \
"ftp://host.site.org/srcpkg.tar.tgz;md5sum=12345;downloadfilename=file.tgz"
```

大的MD5和SHA256校验和可能使得SRC_URI难以阅读和维护。因此，它们可以使用这个语法在SRC_URI外部被指定：

```
SRC_URI = "http://host.site.org/downloads/srcpkg.tar.gz"
SRC_URI[md5sum] = "12345"
SRC_URI[sha256sum] = "6789"
```

当在SRC_URI中指定多于一个URI时，需要使用符号引用以在SRC_URI变量外部提供对所有下载的校验和：

```
SRC_URI = "http://host.site.org/downloads/srcpkg1.tar.gz;name=pkg1
           ftp://host.anothersite.org/downloads/srcpkg2.tgz;name=pkg2"
SRC_URI[pkg1.md5sum] = "12345"
SRC_URI[pkg2.sha256sum] = "6789"
```

HTTP/HTTPS/FTP获取器的实现可以在文件

bitbake/lib/bb/fetch2/wget.py中找到。

SFTP获取器

SFTP获取器允许从带有或者不带有身份认证的安全FTP站点下载文件：

```
SRC_URI = "sftp://host.site.org/downloads/srcpkg.tgz;md5sum=12345"
SRC_URI = "sftp://user@host.site.org/downloads/srcpkg.tgz;md5sum=12345"
```

在URI中使用用于身份认证的密码是不被支持的。你必须使用SSH密钥用于身份认证。

获取器期望sftp命令是在开发主机上可用的。没有针对这个获取器的BitBake变量来覆盖命令和命令行参数。

SFTP获取器支持和HTTP/HTTPS/FTP获取器相同的参数：md5sum、sha256sum、downloadfilename和name。

SFTP获取器的实现可以在文件bitbake/lib/bb/fetch2/sftp.py中找到。

Git获取器

Git已经成为众多开源项目的首选源控制管理。当然，Linux内核社区正在使用它来管理内核开发。获取器克隆仓库并且能够同时检出多个分支。

这个获取器支持的参数如下：

·protocol：用于访问仓库的传输协议。被支持的协议是git、file、ssh、http、https和rsync。如果参数是忽略的，那么获取器默认成git。

·branch：要检出的分支。如果参数是忽略的，那么获取器默认成

master。可以通过用逗号（,）分隔它们来指定多个分支。

·name：关于分支的符号名字。如果多个分支被检出，那么被这个参数提供的名字的数量必须匹配分支的数量。

·tag：要从分支获取的Git标签。如果没有提供参数，那么获取器默认成HEAD。

·rebaseable：告诉获取器，上游仓库可能在将来变基（rebase），导致标签和它们的SHA1散列变化。这个参数命令获取器保留本地缓存以使得未来合并可以被实施。为可变基Git仓库设置这个参数为rebaseable=1。如果参数没有被提供，那么默认是rebaseable=0。

·nocheckout：命令获取器在解包时不从分支检出源代码。那个指令对于使用自己的代码用于检出源代码的菜谱是有用的。如果参数没有被提供，默认值是nocheckout=0。如果菜谱使用它自己的代码用于源处理，那么设置它为nocheckout=1。

·bareclone：告诉获取器创建仓库的裸克隆，并且不从分支上检出源代码。把这个选项用在通过提供它们自己的例程来从分支检出源代码和追踪分支需求的菜谱中。如果参数没有被提供，那么默认值是bareclone=0。如果菜谱使用它自己的例程，那么设置它为bareclone=1。

以下是用于Git获取器的SRC_URI的例子：

```
SRC_URI = "git://git.site.org/git/repo.git;branch=develop"
SRC_URI = "git://git.site/org/git/repo.git;tag=0C12ABD"
SRC_URI = "git://git.site.org/git/repo.git;protocol=http"
```

常见的错误是，当尝试用不同的协议访问Git仓库时，设置在URI开头的方案为http（或者另一个协议）而不是指定protocol参数。当然，那不会工作，因为方案告诉BitBake获取器基础设施用什么获取器。方案

表示协议但是其可用protocol参数覆盖。

Git获取器的实现可以在文件bitbake/lib/bb/fetch2/git.py中找到。

Git子模块获取器

Git子模块获取器是Git获取器的扩展，处理那些仓库的树嵌入外部Git树的仓库。获取器检测Git仓库是否包含子模块，并且在克隆了主仓库后更新它们。

获取器的方案是gitsm。URI格式和参数与Git获取器的相同：

```
SRC_URI = "gitsm://git.site.org/git/repo.git;branch=develop"
```

Git子模块获取器的实现可以在文件bitbake/lib/bb/fetch2/gitsm.py中找到。

Subversion（SVN）获取器

为了从Subversion仓库检出源代码模块，BitBake提供SVN获取器。

这个获取器支持的参数如下：

·protocol：用来访问仓库的传输协议。被支持的协议是svn、svn+ssh、http和https。如果参数是被忽略的，那么获取器默认成svn。

·module：要检出的仓库模块。这个参数是必需的。

·rev：要检出的模块的修订。如果参数是被忽略的，那么HEAD被使用。

可以不带或者带身份认证使用SVN获取器：

```
SRC_URI = "svn://svn.site.org/trunk;module=library;rev=12345;protocol=http"
SRC_URI = "svn://user:passwd@svn.anothersite.org/svn;module=trunk"
```

Subversion获取器的实现可以在文件bitbake/lib/bb/fetch2/svn.py中找到。

并发版本系统（Concurrent Versions System，CVS）获取器

用CVS获取器访问使用并发版本系统的仓库。这个获取器支持的参数被展示在表4-1中。

表4-1　被CVS获取器所支持的参数

method	用于访问仓库的方法。被支持的方法是 pserver、dir 和 ext。方法 pserver 和 ext 被用于访问在远程服务器上的仓库，dir 访问在本地文件系统上的仓库。如果 method=ext，那么获取器读取环境变量 CVS_RSH 来决定访问方法，它通常是 CVS_RSH="ssh"。如果参数是忽略的，那么获取器默认成 pserver
port	用于访问在一台远程服务器上的仓库的 TCP/IP 端口
rsh	提供外部访问方法而不是使用 CVS_RSH
module	要检出的仓库模块。这个参数是必需的
tag	要检出的模块的修订。如果参数是忽略的，那么仓库的 HEAD 被检出

（续）

date	要检出的源日期，它是由参数以 YYYYMMDDHHMM 形式提供的
norecurse	仅仅在当前工作目录中运行 CVS 而不是递归地更新所有子目录。当这个参数被提供的时候，获取器向 cvs 命令的命令行增加 –l 选项
localdir	获取器检出模块到本地目录的名字。如果参数是忽略的，那么目录是模块的名字
fullpath	在检出模块后，获取器创建它的 tar 包。如果 fullpath 没有被设置，那么 tar 包的路径名仅仅包含顶级目录的名字，它通常是模块名。如果 fullpath 是被设置的，那么包的路径名包含完整路径

以下是被这个获取器支持的URI的例子：

```
SRC_URI = "cvs://user@cvs.site.org/cvs;module=src;tag=V0-23-1"
SRC_URI = "cvs://user:pwd@cvs.site.org/cvs;module=src;localdir=sitesrc"
```

CVS获取器的实现可以在文件bitbake/lib/bb/fetch2/cvs.py中找到。

其他获取器

除了在前面部分细节描述的获取器之外，BitBake支持这些不经常用到的仓库和修订控制系统：

·Bazaar：针对GNU项目修订控制系统Bazaar的获取器。方案是bzr://。获取器的实现可以在文件bitbake/lib/bb/fetch2/bzr.py中找到。

·Mercurial：针对跨平台的、分布式修订控制系统Mercurial的获取器。方案是hg://。获取器的实现可以在文件bitbake/lib/bb/fetch2/hg.py中找到。

·Open Build Service：访问由SUSE Linux发行版使用的开放构建服务（Open Build Service，OBS）管理的源的获取器。方案是osc://。获取器的实现可以在文件bitbake/lib/bb/fetch2/osc.py中找到。

·Perforce：访问由专有商业修订控制系统Perforce管理的源的获取器。方案是p4://。获取器的实现可以在文件bitbake/lib/bb/fetch2/perforce.py中找到。

·Repo：访问使用Android repo工具用于仓库管理的Git仓库的获取器。方案是repo://。获取器的实现可以在文件bitbake/lib/bb/fetch2/repo.py中找到。

·SVK：从SVK仓库中访问资源的获取器。方案是svk://。获取器的实现可以在文件bitbake/lib/bb/fetch2/svk.py中找到。

URI语法和这些获取器的使用遵循和更主流获取器一样的规则。获取器可能使用额外或者不同的参数以适应它们的特殊功能性。

4.5.3 镜像

菜谱设置SRC_URI变量为源代码的上游位置。除了SRCI_URI，BitBake支持镜像下载站点，从那里它可以替代性地获取源代码包。

BitBake使用已定义的序列来访问文件位置和站点：

·BitBake首先检查由DL_DIR指定的本地下载目录，是否在SRC_URI中提供的文件已经被下载了。如果已经被下载了，它跳过访问任何上游和镜像站点，并且使用在DL_DIR中的文件。如果一些文件是存在的，另外一些不存在，那么BitBake增量地下载文件。如果SRC_URI是源代码控制管理系统，那么它验证在DL_DIR中的正确分支和标签，并且根据需要最终更新它们。

·如果在SRC_URI中提供的文件不是本地可用的，那么BitBake尝试从PREMIRRORS变量指定的镜像站点中下载它们。

·如果前镜像站点不提供包，那么BitBake使用SRC_URI从上游项目站点直接下载文件。

·如果从上游项目站点下载是不成功的，那么BitBake使用由MIRRORS变量提供的镜像站点。

·如果下载站点都没有提供需要的文件，那么BitBake抛出错误消息。

使用镜像和前面的序列是BitBake的默认行为。如果你不想使用镜像，那么你需要将PREMIRRORS或MIRRORS变量或两者都设置成空字符串。

PREMIRRORS和MIRRORS变量指定用于键匹配SRC_URI的正则表达式和指向各自镜像的URI组成的元组列表：

```
MIRRORS = "\
ftp://.*/.*    http://downloads.mirrorsite.org/mirror/sources/ \n \
http://.*/.*   http://downloads.mirrorsite.org/mirror/sources/ \n \
https://.*/.*  http://downloads.mirrorsite.org/mirror/sources/ \n \
git://.*/.*    http://downloads.mirrorsite.org/mirror/sources/ \n \
svn://.*/.*    http://downloads.mirrorsite.org/mirror/sources/ \n"
```

元组的键和URI是由一个或者多个空格字符分隔的，元组之间是用新行符来各自分隔的。通常，镜像使用HTTP协议用于文件下载，但是其他文件下载协议，例如FTP、HTTPS和SFTP也是有效的，只要BitBake有对应的获取器。

对文件下载的操作是简单直接的。例如，如果BitBake遇到使用FTP方案的SRC_URI，那么它会寻找在MIRROR变量中的匹配键，然后为镜像的URI替换SRC_URI方案和路径。使用上面的镜像列表：

```
SRC_URI = "ftp://ftp.site.org/downloads/file.tgz"
```

有效地转化成：

```
SRC_URI = "http://downloads.mirrorsite.org/mirror/sources/file.tgz"
```

然而，如果镜像的URI指向下载站点，那么对于例如Git的源代码控制管理系统仓库镜像如何工作？在这种情况下，BitBake期望仓库被打包成tar包。它从镜像下载tar包，并且本地解压它包含的仓库。

Yocto项目于http://downloads.yoctoproject.org/mirror/sources处提供了在高可用性基础设施上的源镜像。这个镜像在所有Yocto项目构建环境中均由Poky参考发行版策略默认启用。

创建镜像

创建你自己的镜像站点有优点，例如为团队最小化网络访问和控制产品的Linux发行版从哪个源被构建。你可以通过从Yocto项目镜像下载所有源包并且把它们放在一台内部网络服务器的目录中来创建你自己的镜像站点。你也可以从Yocto项目构建环境的下载目录中创建镜像，该环境用于构建Linux发行版。本地下载目录已包含所有必要源，但是尚未采用适合镜像站点的格式。默认，并且为了节省构建时间，BitBake不为源代码控制管理系统创建源tar包。你可以通过增加如下内容到conf/local.conf文件来命令BitBake在本地下载目录中创建tar包：

```
BB_GENERATE_MIRROR_TARBALLS = "1"
```

在构建已经成功完成后，简单地复制所有在下载目录中的文件到镜像服务器。你必须复制文件。你不能使用到文件的符号链接，因为获取器不跟随符号链接。

在镜像服务器被设置后，你可以使用它的URI用于MIRRORS和PREMIRRORS变量。

4.6 HelloWorld——BitBake方式

前面的章节解释了BitBake要素并且为使用BitBake作为构建系统奠定了基础。现在是时候用简单的例子来说明问题了——BitBake的HelloWorld。这可能不是你可以用BitBake处理的最原生的例子，但是它示例了如何使用它来给作为目标的构建主机构建开源软件包。

这个例子从源构建nano文本编辑器（http://nano-editor.org）。nano使用GNU Autotools用于配置。为了这个工作在构建主机上的例子，你需要安装包含Autotools的GNU开发包。你当然也需要BitBake。

首先，我们需要为我们的BitBake HelloWorld项目设置构建环境。程序清单4-18显示了它的布局。

程序清单4-18 BitBake HelloWorld构建环境布局

```
yocto@yocto-dev:~/bitbake$ tree -L 3 bbhello
bbhello/
├── classes
│   └── base.bbclass
├── conf
│   ├── bblayers.conf
│   └── bitbake.conf
└── meta-hello
    ├── conf
    │   └── layer.conf
    └── recipes-editor
        └── nano
```

我们简单地从BitBake安装目录复制BitBake的默认文件base.bbclass和bitbake.conf到构建环境。就本例而言，它们包含所有需要的设置。

文件bblayers.conf设置构建环境并且也包含我们的meta-hello层，它最终包含用于构建nano编辑器的菜谱（见程序清单4-19）。

程序清单4-19 bblayers.conf

```
# Initialize BBPATH with the top-level directory of the
# build environment.
BBPATH := "${TOPDIR}"
# Initialize BBFILES to an empty string as it is set up
# by the layer.conf files of the layers.
BBFILES ?= ""

# Add our layer to BBLAYERS.
BBLAYERS = " \
  ${TOPDIR}/meta-hello \
  "
```

文件首先设置BBPATH变量为构建环境的顶级目录并且初始化BBFILES为空字符串。然后，它增加meta-hello层到BBLAYERS。为了简化性，这个例子是全部自包含的，并且在构建内包含meta-hello。

正如前面简要列出的，所有BitBake层都必须包含layer.conf配置文件，它被显示在程序清单4-20中。

程序清单4-20　layer.conf

```
# Add the path of this layer to BBPATH
BBPATH .= ":${LAYERDIR}"
# Add recipes and append files to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"

# Set layer search pattern and priority
BBFILE_COLLECTIONS += "hello"
BBFILE_PATTERN_test := "^${LAYERDIR}/"
BBFILE_PRIORITY_test = "5"
```

这个文件本质上是样板文件模板的副本。仅有的必要调整是搜索模式和优先级设置。

最后，我们需要菜谱来构建nano编辑器。我们把这个菜谱放进层的顶级目录meta-hello的子目录recipes-editor中。到我们层的菜谱的完整路径匹配在layer.conf中BBFILES内的表达式。程序清单4-21显示了菜谱。

程序清单4-21　nano.bb

```
SUMMARY = "Recipe to build the 'nano' editor"

PN = "nano"
PV = "2.2.6"

SITE = "http://www.nano-editor.org/dist"
PV_MAJOR = "${@bb.data.getVar('PV',d,1).split('.')[0]}"
PV_MINOR = "${@bb.data.getVar('PV',d,1).split('.')[1]}"

SRC_URI = "${SITE}/v${PV_MAJOR}.${PV_MINOR}/${PN}-${PV}.tar.gz"
SRC_URI[md5sum] = "03233ae480689a008eb98feb1b599807"
SRC_URI[sha256sum] = \
"be68e133b5e81df41873d32c517b3e5950770c00fc5f4dd23810cd635abce67a"

python do_fetch() {
    bb.plain("Downloading source tarball from ${SRC_URI} ...")
    src_uri = (d.getVar('SRC_URI', True) or "").split()
    if len(src_uri) == 0:
       bb.fatal("Empty URI")
    try:
       fetcher = bb.fetch2.Fetch(src_uri, d)
       fetcher.download()
    except bb.fetch2.BBFetchException:
       bb.fatal("Could not fetch source tarball.")
    bb.plain("Download successful.")
}

addtask fetch before do_build
```

```
python do_unpack() {
    bb.plain("Unpacking source tarball ...")
    os.system("tar x -C ${WORKDIR} -f ${DL_DIR}/${P}.tar.gz")
    bb.plain("Unpacked source tarball.")
}

addtask unpack before do_build after do_fetch
python do_configure() {
    bb.plain("Configuring source package ...")
    os.system("cd ${WORKDIR}/${P} && ./configure")
    bb.plain("Configured source package.")
}

addtask configure before do_build after do_unpack

python do_compile() {
    bb.plain("Compiling package...")
    os.system("cd ${WORKDIR}/${P} && make")
    bb.plain("Compiled package.")
}

addtask compile before do_build after do_configure

do_clean[nostamp] = "1"
do_clean() {
    rm -rf ${WORKDIR}/${P}
    rm -f ${TMPDIR}/stamps/*
}

addtask clean
```

这个菜谱使用了很多BitBake元数据语法的主要概念：变量设置、变量扩展、Python变量扩展、变量属性、从Python代码中访问BitBake变量、Python任务、shell任务等。

为了构建nano文本编辑器，从构建环境的顶级目录中简单的执行：

```
$ bitbake nano
```

如果所有东西工作正常，那么你应该看到类似于程序清单4-22的输出。

程序清单4-22　构建Nano文本编辑器

```
$ bitbake nano
NOTE: Not using a cache. Set CACHE = <directory> to enable.
Parsing recipes: 100% |#####################################| Time: 00:00:00
Parsing of 1 .bb files complete (0 cached, 1 parsed). 1 targets,
  0 skipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
NOTE: Preparing runqueue
NOTE: Executing RunQueue Tasks
Downloading source tarball from
  http://www.nano-editor.org/dist/v2.2/nano-2.2.6.tar.gz ...
Download successful.
Unpacking source tarball ...
Unpacked source tarball.
Configuring source package ...
Configured source package.
Compiling package...
Compiled package.
NOTE: Tasks Summary: Attempted 5 tasks of which 0 didn't need to be rerun and all
succeeded.
```

nano可执行程序位于你可以运行它的tmp/work/nano-2.2.6-r0/nano-2.2.6/src中。

你可以用如下命令清理构建环境：

```
$ bitbake nano -c clean
```

当然，这是非常简单的例子。构建系统，例如OpenEmbedded构建系统，构建完全的Linux操作系统栈，必须提供更多功能性并且包含数百个菜谱和很多类。另外，OpenEmbedded构建系统能够针对不同架构和许多不同机器类型来进行构建，也为它们构建必要的交叉工具链。

4.7 依赖处理

构建系统极少只构建单一的软件包。如果多个软件包被构建，那么依赖通常存在于那些包中的一些之间，意味着构建系统必须能够处理这样的依赖。构建系统必须能够检测所有对包做出的变更，重新执行任何因变更而失效的任务，然后也重新执行任何来自于依赖变更包的一个或者多个任务输出的软件包的任务。

构建系统通常区分2种类型的依赖：

·构建依赖：对成功构建的软件包来说所需要的组件。这些可以是头文件、静态库或其他组件。

·运行时依赖：对软件包正常操作所需要的组件。这些可以是库、配置文件或其他组件。

为了表达依赖，构建系统必须实现一种让软件包声明它们的名字或者功能性的方法，以及一种让其他软件包将那个名字或者功能性引用为依赖的方法。

4.7.1 配置

用于软件包的BitBake菜谱使用PROVIDES变量来声明它们的名字和功能性，然后其他菜谱可以用其来表达它们对那个软件包的依赖。BitBake给出了3种不同的配置的变体，在本书中，它们被称为隐式配置、显式配置和符号配置。

隐式配置

BitBake从菜谱的文件名中获得用于软件包、软件版本和软件修订的值。它将这些值用于配置，除非它们被菜谱的内容显式地覆盖，如在下个章节所解释的。

BitBake文件名遵循下面的惯例：

```
<package name>_<package version>_<package revision>.bb
```

惯例用下划线（_）连接包名、包版本和包修订；例如，

```
nano_2.2.6_r0.bb
```

BitBake解析文件名并且对应地设置变量PN、PV和PR：

```
PN = "nano"
PV = "2.2.6"
PR = "r0"
```

如果菜谱名不提供包修订，那么PR默认成r0。如果它不提供包版本，那么PV默认成1.0。当然，在通过菜谱名使用隐式配置时，你不能指定包修订而不指定包版本。如果你想提供包修订而不指定包版本，那么你必须使用显式配置。然而，这样做通常没有多大意义，因为包修订仅仅在包版本的上下文中才有意义。

通过赋值

```
PROVIDES_prepend = "${PN} "
```

包总是声明它的包名。

显式配置

3个变量PN、PV和PR可以由菜谱自己通过对它们赋值来显式地设置。这样的显式配置覆盖任何从文件名中获取到的设置。BitBake对应地设置PROVIDES变量。

符号配置

菜谱也可以覆盖或者添加到可用于指定该包的符号名字的PROVIDES变量。然后依赖于它的包可以引用符号名字。如果多个包或者同一个包的多个版本提供相同的功能性，那么这是有意义的：

```
PROVIDES =+ "virtual/editor"
```

这个用在菜谱中针对nano文本编辑器的声明将增加符号名字virtual/editor到名字列表。增加符号名字到PROVIDES而不是全部覆盖变量是一直被推荐的。符号名字实际上做什么不重要，但是为了避免符号名字和实际其他菜谱名字的无意冲突，开发者已经采用惯例来使用前缀virtual/用于符号名字[1]。

明显的是，如果多个包使用相同的符号名字，那么构建系统必须有一种方法来决定使用哪个菜谱。我们会在4.7.3节解决这个问题。

[1] 因为符号名字中前斜杠的使用，很明显在Linux构建主机上，符号名字永远不会与实际菜谱中的名字发生冲突，因为在UNIX系统上，前斜杠是用来分隔路径段的。

4.7.2 声明依赖

菜谱通过分别向DEPENDS和RDEPENDS变量增加它们所依赖的包名来声明构建依赖和运行时依赖：

```
DEPENDS = "libxml2-native gettext-native virtual/libiconv ncurses"
RDEPENDS = "base-files run-postinsts"
```

两个变量都包含一个它们所依赖的空格分隔的实际或者符号名字列表。当然，那些包必须通过PROVIDES变量来提供它们的名字，正如前面部分所简要描述的。

虽然依赖是被声明在菜谱级别的，但是在菜谱级别上强制它们是相当不高效的。例如，在菜谱级别强制构建依赖将意味着，构建包的菜谱的所有任务都必须在已经成功完成了之后，依赖于那个包的菜谱的第1个任务才可以开始。对于并行构建过程来说，那将是不那么优化的，因为例如获取源代码、解压它、给它打补丁等的任务是不依赖于已经成功完成构建过程的另外一个包的。然而，配置和编译包的任务依赖于包的构建过程的成功完成。因此，BitBake在任务级别强制依赖。强制是通过使用元数据属性deptask和rdeptask来实现的。每个任务有包含必须是从DEPENDS中列出的每个包中完成的空格分隔任务列表的deptask属性：

```
do_configure[deptask] = "do_populate_staging"
```

对于代码例子，包含在DEPENDS中的每个菜谱的do_populate_staging任务都必须在这个菜谱的do_configure_task可以运行之前完成。

rdeptask属性完成对运行时依赖的相同功能：

```
do_package_write[rdeptask] = "do_package"
```

在这个例子中，菜谱的do_package_write任务不能在运行时所依赖的所有包完成其打包步骤之前运行。

4.7.3 多个提供器

通过符号配置，多个包可能满足其另一个包的构建或运行时依赖。例如，可能有两个包提供用于一个电子邮件程序的文本编辑器功能。电子邮件程序本身不关心这两个文本编辑器的哪一个提供这个功能性，只要其中一个是可用的就行：

```
editor1.bb:    PROVIDES = "virtual/editor"
editor2.bb:    PROVIDES = "virtual/editor"
mailer.bb:     RDEPENDS = "virtual/editor"
```

这两个编辑器包都提供邮件包所需要的virtual/editor功能性。BitBake如何选择两个编辑器包的其中之一来构建？通过PREFERRED_PROVIDER变量，你可以选择BitBake构建的提供virtual/editor的包：

```
PREFERRED_PROVIDER_virtual/editor = "editor2"
```

最常见的是，这个变量被放进配置文件中，例如发行版策略配置文件。

4.8 版本选择

许多元数据层包含多个菜谱以构建相同软件包的不同版本。例如OpenEmbedded核心元数据层提供多个菜谱来构建Linux内核的不同版本。

默认情况下，BitBake总是选择构建包的最新版本的菜谱，正如PV变量所指示的。然而，你可以通过指定PREFERRED_VERSION变量来覆盖默认行为：

```
PREFERRED_VERSION_editor = "1.1"
```

你必须根据实际包名来设置这个变量。有时，版本号是由额外信息所追加的，例如Git标签或者一个次版本号。在这个情况下，你通过向版本字符串增加百分号（%）来告诉BitBake，额外的信息是不相关的：

```
PREFERRED_VERSION_linux-yocto = "3.10%"
```

菜谱可能设置DEFAULT_PREFERENCE变量来指示比它版本号更低或者更高的优先级：

```
editor_1.1.bb: DEFAULT_PREFERENCE = "6"
editor_1.2.bb: DEFAULT_PREFERENCE = "0"
```

默认情况下，BitBake将选择editor-1.2.bb菜谱而不是editor-1.1.bb菜谱，因为它是更新的版本。然后，通过使用DEFAULT_PREFERENCE，构建旧版本的菜谱的优先级被提高。对于DEFAULT_PREFERENCE的默认值是5。

推荐你仅仅对实验性的构建更新版本的菜谱使用DEFAULT_PREFERENCE来降低它们的优先级：

```
editor_2.0.bb: DEFAULT_PREFERENCE = "-1"
```

如果你不知道菜谱正在使用它，那么使用DEFAULT_PREFERENCE除了降低实验性菜谱的优先级外还很容易导致不良结果和混乱。

4.9 变体

菜谱通常构建用于目标系统的软件包的变体。然而，对于一些包，变体是不同应用所需要的。典型的例子是在一个目标变体和一个原生或者主机变体上所需要的编译器。为了从相同的菜谱启用一个包的多个变体，BitBake提供了BBCLASSEXTEND变量：

```
BBCLASSEXTEND += "native"
```

如果这个指令被用在一个菜谱中，那么它命令BitBake除了为目标变体也为构建主机构建软件包的一个原生变体。在这种功能性背后的机制是，BitBake又一次执行继承native类的菜谱。为了使这种机制正确工作，你必须在一个叫作native.bbclass的文件中定义natvie类。

BBCLASSEXTEND包含一个由空格分隔的列表，允许你用不同的类来按照需要多次执行菜谱。

基于正在构建的变体，在一个菜谱中包含的变量或者任务可能需要包含不同的值或者执行不同的处理步骤。这可以通过条件地覆盖变量或者任务来实现。使用以下命令：

```
VARIABLE_class-target = "target value"
VARIABLE_class-native = "native value"
```

依赖于BitBake正在构建的变体而赋予VARIABLE不同的值。所有不使用变体覆盖的变量和任务是由所有变体所共享的。

变体target是隐式的。也没有类target被定义。所有用在BBCLASSEXTEND中的其他类都必须被它们各自的.bbclass文件所定义。

4.10 默认元数据

BitBake定义、使用和依赖于一些默认元数据对象。你会发现它们在许多菜谱和类中被引用。在接下来的部分，我们讨论这个核心的变量和任务列表。

当然，OpenEmbedded、Poky和其他元数据层随着它们扩展BitBake的功能性而增加更多的元数据。当谈到在第7章中构建一个定制化Linux发行版的上下文、在第8章中开发用于软件包的菜谱、在第9章中配置和编译Linux内核、在第10章中开发板支持包的时候，我们讨论许多这些扩展及其元数据并且介绍它们。

4.10.1 变量

BitBake引用和使用数个变量。它期望设置那些变量，否则它会引发一个错误并且终止执行。BitBake从它当前的上下文中内部地获取这些变量中的一些，并且对应地设置它们。其他则被bitbake.conf文件设置成默认值。对一个第3类来说，你必须显式地为项目提供设置。

内部获取的变量

BitBake从它当前的上下文中获取针对以下变量的值，并且对应地设置它们：

·FILE：到BitBake正在处理的文件的完整路径。文件可以是一个配置文件、一个菜谱、一个类等。

·LAYERDIR：当BitBake处理那一层时，它把这个变量设置为到当前层的目录的完整路径。

·TOPDIR：BitBake设置这个变量为到你已经执行了BitBake的所在目录的完整路径。BitBake期望你要么从构建环境的顶级目录运行它，要么显式地设置这个变量为你构建环境的顶级目录，正如前面所显示的。

项目特定的变量

这些变量是特定于BitBake项目的。你需要显式地为项目设置它们。

·BBFILES：空格分隔的针对菜谱的路径列表。用于一个层的layer.conf配置文件扩展这个变量以增加它自己的菜谱。

·BBLAYERS：当BitBake处理那一层时，它把这个变量设置为到当前层的目录的完整路径。

·BBPATH：BitBake使用这个变量在一个名为classes的子目录中定位类（.bbclass文件）、在一个名为conf的子目录中定位配置文件（.conf文件）以及在其他子目录中定位文件。变量包含一个由冒号（：）分隔的目录路径列表。层通常增加它们的顶级目录到这个变量：BBPATH.=":${LAYERDIR}"。

标准运行时变量

标准运行时变量被很多BitBake模块所引用。因此，它们必须包含有效的值以使BitBake正确运行。这些变量通常被conf/bitbake.conf文件初始化。BitBake源包提供了一个包含默认设置的conf/bitbake.conf文件。如果你从头开始启动一个BitBake项目，正如我们对HelloWorld项目所做的，你可以使用这个文件作为起始点。

OpenEmbedded核心元数据层，它形成Poky构建系统的基础，包含一个初始化更大的变量列表的conf/bitbake.conf文件来满足包含在那个层中的众多类的要求。随着我们在接下来的章节中探索更多Poky的功能性，我们将在它们适合的上下文中介绍这些变量。

·B：到BitBake构建一个源包所在的目录的完整路径。通常BitBake解压源包所到的是相同目录：B=${S}。对于树外构建，B可以被设置成指向一个不同的构建目录。

·BUILD_ARCH：构建主机的CPU架构。在大部分情况下，自动使用BUILD-ARCH=${@os.uname()[4]}来获取值，在Linux系统上它返回系统的架构。

·CACHE：到用于元数据缓存的目录的完整路径，它通常存在于构

建环境的内部：CACHE=${TOPDIR}/cache。当BitBake在一个构建环境中第一次被运行时，它解析所有元数据文件和菜谱并且创建一个在后续运行时从那里读取的缓存。这改进了执行速度。一个元数据变化使缓存失效并且导致BitBake重新创建它。

·CVSDIR：到BitBake检出CVS仓库所在目录的完整路径。对于这个目录默认是下载目录的一个子目录：CVSDIR=${DL_DIR}/cvs。

·D：到一个任务（例如一个安装任务或者一个镜像创建任务）放置其输出的所在目录的完整路径。例如，一个使用make install的安装任务引用这个变量用于安装路径。

·DEPENDS：这个变量用一个空格分隔的包名来描述包依赖性。

·DEPLOY_DIR：到BitBake放置所有用于部署的文件所在的基础目录的完整路径，例如目标镜像、包源和许可程序清单（license manifest）。默认情况下，这个目录存在于构建环境中：DEPLOY_DIR=${TMPDIR}/deploy。

·DEPLOY_DIR_IMAGE：到BitBake复制目标二进制镜像所在目录的完整路径。它典型地指向DEPLOY_DIR：DEPLOY_DIR_IMAGE=${DEPLOY_DIR}/images的一个子目录。

·DL_DIR：到下载目录的完整路径。默认设置把这个目录放在构建环境中：DL_DIR=${TMPDIR}/downloads。为了避免多次下载相同的源文件和当工作于不止一个构建环境时的时间和硬盘空间的消耗，建议设置变量为构建环境外的路径。

·FILE_DIRNAME：到包含BitBake正在处理的文件的目录的路径。自动使用FILE_DIRNAME=${@os.path.dirname(bb.data.getVar('FILE'))}来获取值。

·FILESDIR：到BitBake寻找本地文件所在目录的完整路径。BitBake仅仅当它不能在任何列于FILESPATH中的目录中找到文件时使用这个变量。对这个变量的最常见的用例在追加文件中。

·FILESPATH：这个变量包含一个冒号分隔的、BitBake的本地获取器为了匹配本地文件所搜索的完整目录路径列表。默认设置是FILESPATH="${FILE_DIRNAME}/${PF}：${FILE_DIRNAME}/${P}：${FILE_DIRNAME}/${PN}：${FILE_DIRNAME}/files：${FILE_DIRNAME}"。

·GITDIR：到BitBake检出Git仓库所在目录的完整路径。对于这个目录默认是下载目录的一个子目录：GITDIR=${DL_DIR}/git。

·MKTEMPCMD：BitBake用来创建临时文件的命令：MKTMPCOMMAND="mktemp-q${TMPBASE}"。

·MKTEMPDIRCMD：BitBake用来创建临时目录的命令：MKTMPCOMMAND="mktemp-d-q${TMPBASE}"。

·OVERRIDES：冒号分隔的条件覆盖的优先级列表。BitBake从右到左处理这个列表，所以后面列出的有更高优先级。

·P：用横线（-）连接的包名、版本：P="${PN}-${PV}"。

·PERSISTENT_DIR：到BitBake持久存储文件所在目录的完整路径。默认设置是PERSISTENT_DIR="${TOPDIR}/cache"。CACHE和PERSISTENT_DIR变量可以互换使用。其中之一必须被设置。如果PERSISTENT_DIR没有被设置，BitBake退回使用CACHE。

·PF：用横线（-）连接的包名、版本和修订：PF="${PN}-${PV}-${PR}"。

·PN：从菜谱文件名获取的包名。

·PR：从菜谱名获取的或者显式设置的包修订。

·PROVIDES：空格分隔的、声明一个包提供了什么的名字列表。其他菜谱可以使用这些名字来声明它们对这个包的依赖。

·PV：从菜谱文件名获取的或者显式设置的包版本。

·S：到BitBake放置已解压文件所在目录的完整路径。默认情况下，对于包来说这是一个工作目录的子目录：S="${WORKDIR}/${P}"。

·SRC_URI：为源包下载URI。

·SRCREV：用于从源代码控制管理系统下载所用的源修订。

·SVNDIR：到BitBake检出Subversion仓库所在目录的完整路径。对于这个目录默认是下载目录的一个子目录：SVNDIR=${DL_DIR}/svn。

·T：到一个BitBake处理一个包菜谱时存储临时文件（例如任务代码和任务日志）的所在目录的完整路径。默认情况下，这个目录存在于包的工作目录中：T="${WORKDIR}/tmp"。

·TARGET_ARCH：针对BitBake正在构建的CPU架构。

·TMPBASE：到一个BitBake使用MKTMPCMD和MKTEMPDIRCMD命令来创建临时文件和目录的所在目录的完整路径。BitBake模块、类和任务根据它们的需求而设置这个变量。

·TMPDIR：到BitBake放置所有构建输出（例如包构建、根文件系统阶段、镜像和包源）的顶级目录的完整路径。它通常存在于构建环境内部：TMPDIR="${TOPDIR}/tmp"。对于这个变量名字的选择和它的默

认设置有点不那么幸运。虽然从BitBake总是可以重建它们的角度来说，在TMPDIR内的文件和目录是临时的，但是目录包含所有构建输出和制品。这使得目录实际上比它的名字所暗示的更加重要。

·WORKDIR：到BitBake构建一个包并且存储所有与包的构建过程相关的日志信息的目录的路径。默认设置是WORKDIR="${TMPDIR}/work/${PF}"。

正如你所看到的，许多变量引用其他变量，特别是针对文件和目录的路径。这创建了一个使你能够通过仅仅改变在一个配置文件的变量来简单定制构建环境的非常灵活的架构。

4.10.2 任务

BitBake代码也包含一个由base.bbclass提供的base类的默认实现。你可以使用这个类作为你自己的基于BitBake的构建系统的起点。当然，OpenEmbedded核心元数据层提供了一个扩展的base类。

BitBake的默认base.bbclass提供以下的任务：

·build：这是BitBake在执行菜谱时的默认任务，除非另一个任务是通过变量BB_DEFAULT_CLASS的。默认base类不实际实现任何有用的用于执行的东西。你需要在菜谱中覆盖它或使用它作为另一个任务的锚点，正如我们在HelloWorld例子中所做的。

·listtasks：带有任何目标执行这个任务显示所有可适用于目标的任务。其中包含目标的菜谱所定义的任务以及菜谱从类中所继承的任何任务。注意，任务必须以它们任务散列值的顺序列出，并不是按照字母排序或者以它们的执行为顺序。

4.11 总结

这一章展示了BitBake的概览——OpenEmbedded和Poky背后的构建引擎。

·BitBake是由OpenEmbedded和Yocto项目作为OpenEmbedded构建系统的核心组件而联合开发的，被2个项目共享。Yocto项目的Poky参考发行版发布包含针对特定发布、匹配Poky元数据的BitBake版本。

·BitBake要求执行或者构建环境伴随特定元数据变量被正确地设置。Poky包含shell脚本来正确设置和初始化构建环境。

·BitBake元数据区分变量和可执行元数据或者函数。元数据函数可以作为shell或者Python代码来实现。任务是特别声明的、被BitBake作为目标的构建过程的一部分执行的元数据函数，或者是可以被显式地从BitBake命令行调用的元数据函数。

·元数据被组织成配置文件、菜谱、类、追加文件和包含文件。

·BitBake的元数据语法提供各种表达式以操作变量内容。变量可能包含单一值或者由分隔符所分隔的值的列表。

·条件变量赋值以及变量追加和前新增允许基于上下文来覆盖变量。OVERRIDES变量包含条件的优先级列表。

·作为shell代码实现的元数据函数可以直接访问元数据变量，而Python函数需要通过BitBake数据字典来访问它们。

·来自Python元数据函数的返回值可以直接被赋予元数据变量。

·BitBake的依赖处理允许在包级别声明依赖。为了优化并行执行，

BitBake强制在任务级别的依赖性。

·用于包的符号名允许包基于提供的功能性而不是实现包的名字来申明依赖性。变量PREFERRED_PROVIDER允许选择实现的包。

·包可能提供多个菜谱以构建包的不同版本。BitBake总是构建带有最高版本的最新包，除非不同的版本是通过PREFERRED_VERSION指定的。

·变体或者类扩展提供一种用于针对不同应用（例如目标和构建主机）多次构建相同包的机制，而不用重写整个菜谱。

4.12 参考文献

BitBake的源包包含可以被格式化成HTML或者PDF输出的DocBook格式的文档。在线文档位于www.yoctoproject.org/docs/2.0/bitbake-user-manual/bitbake-user-manual.html。

第5章　故障排除

作为软件开发者，你应当非常清楚，不是“万一”而是“何时”会在构建软件中出问题这一疑问非常重要。复杂构建系统的故障排除可能是使人畏缩的任务。失败可能源自构建系统的很多不同方面：在菜谱中和类中的代码、配置文件设置、交叉开发、要构建的软件包、打包等。其大体归结为：

·定位和识别失败的原因

·寻找和应用解决方案

手头上有恰当的工具并且知道如何有效地使用它们，这可为正确地定位和识别问题的根本原因（root cause）节省大量时间和努力。OpenEmbedded构建系统提供了一组帮助你寻找导致构建失败的问题的工具。然而，寻找针对问题的解决方案，可能是具有挑战性的。由于潜在的大量问题及其起源，对于如何解决构建问题没有单一和简单的回答。问题解决需要许多关于识别问题模式以及修改和应用类似问题的解决方案的经验。我们不阻止你进行问题解决，但是鼓励你设置符合现实的预期。依赖于你的软件开发经验，你能相当轻松地找到对一些问题的解决方案，而其他则代表着更大的挑战。然而，有可能你不是唯一经历特定问题的人。互联网及其搜索引擎使软件开发者可非常轻松地找到对几乎所有问题的解决方案。

接下来的部分解释由OpenEmbedded构建系统提供的各种调试工具以及如何使用它们。

5.1 记日志

BitBake记录发生在构建过程中的所有事件。所记录的事件是

·插入到可执行元数据中的调试语句

·来自由任务和其他代码执行的任何命令的输出

·由任务和其他代码执行的任何命令所发出的错误消息

所有日志消息被路由到各种日志文件，从这些文件中，它们可以被获取。BitBake为它所执行的每个任务以及它自己的主构建过程创建日志文件。任何由BitBake任务和其他代码执行的命令发送到stdout和stderr的全部输出都被重定向到日志文件。在正常操作中，BitBake不显示在执行时的任何日志输出，除非有警告或者错误条件。

5.1.1 日志文件

BitBake为它的所有进程维护独立的日志文件。其中包含它的cooker进程，它是主构建进程，也是对每个菜谱的每个任务来说的主进程。cooker进程为它所运行的每个任务派生独立进程。

通用日志文件

BitBake将所有通用日志文件（例如cooker的look文件）存储在由LOG_DIR变量指向的目录中。默认情况下，这个目录是用于临时构建文件的目录中的名为log的子目录：

```
LOG_DIR = "${TMPDIR}/log"
```

在那个目录中，你为每个BitBake进程（例如cooker）找到子目录。日志文件由目标机器进一步划分。例如，如果你正在为qemux86机器构建系统，那么cooker子目录包含含有实际日志文件的qemux86子目录。虽然这个设置需要一些努力来从目录树中导航，但是它更容易使你找到相关日志文件。

BitBake在进程启动时使用时间戳来命名日志文件，这有效地维护了日志文件的历史。拥有日志文件的历史以及从后续构建中互相比较文件可以帮助你有效地追踪构建失败。BitBake总是为其时间戳使用协调世界时（Coordinated Universal Time，UTC）而不是使用本地时间，这使得它们在使用不同时区的远程构建服务器时是容易比较的。

cooker日志文件包含所有BitBake在运行时也写到控制台的日志输出。程序清单5-1显示仅仅针对来自预先构建的包中的镜像的简短cooker日志文件。

程序清单5-1　cooker日志文件

```
NOTE: Resolving any missing task queue dependencies

Build Configuration:
BB_VERSION        = "1.21.1"
BUILD_SYS         = "x86_64-linux"
NATIVELSBSTRING   = "Fedora-18"
TARGET_SYS        = "i586-poky-linux"
MACHINE           = "qemux86"
DISTRO            = "poky"
DISTRO_VERSION    = "1.5+snapshot-20140210"
TUNE_FEATURES     = "m32 i586"
TARGET_FPU        = ""
meta
meta-yocto
meta-yocto-bsp    = "master:095bb006c3dbbfbdfa05f13d8d7b50e2a5ab2af0"

NOTE: Preparing runqueue
NOTE: Executing SetScene Tasks

NOTE: Executing RunQueue Tasks
NOTE: Running noexec task 2051 of 2914 (ID: 4, /develop/yocto/yocto-git/poky/meta/
recipes-core/images/core-image-minimal.bb, do_fetch)
NOTE: Running noexec task 2052 of 2914 (ID: 0, /develop/yocto/yocto-git/poky/meta/
recipes-core/images/core-image-minimal.bb, do_unpack)
NOTE: Running noexec task 2053 of 2914 (ID: 1, /develop/yocto/yocto-git/poky/meta/
recipes-core/images/core-image-minimal.bb, do_patch)
NOTE: Running noexec task 2910 of 2914 (ID: 9, /develop/yocto/yocto-git/poky/meta/
recipes-core/images/core-image-minimal.bb, do_package_write)
NOTE: Running task 2911 of 2914 (ID: 8, develop/yocto/yocto-git/poky/meta/recipes-
core/images/core-image-minimal.bb, do_populate_lic)
NOTE: Running task 2912 of 2914 (ID: 7, develop/yocto/yocto-git/poky/meta/recipes-
core/images/core-image-minimal.bb, do_rootfs)
NOTE: recipe core-image-minimal-1.0-r0: task do_populate_lic: Started
NOTE: recipe core-image-minimal-1.0-r0: task do_populate_lic: Succeeded
NOTE: recipe core-image-minimal-1.0-r0: task do_rootfs: Started
NOTE: recipe core-image-minimal-1.0-r0: task do_rootfs: Succeeded
NOTE: Running noexec task 2914 of 2914 (ID: 12, /develop/yocto/yocto-git/poky/
meta/recipes-core/images/core-image-minimal.bb, do_build)
NOTE: Tasks Summary: Attempted 2914 tasks of which 2907 didn't need to be rerun
and all succeeded.
```

cooker日志文件中的一条重要信息是构建配置，它告诉你这个构建使用什么设置。这个信息在调试“它昨天工作，为什么今天不工作了？”这类问题时非常有价值。常见地，配置中的变化可能导致构建失败，比较成功构建的cooker日志文件和失败构建的cooker日志文件可帮助你追踪问题。下面的变量组成了列在cooker日志文件开头的构建配置：

·BB_VERSION：BitBake版本号。BitBake和元数据层一起演化。使

用元数据层的更新版本，例如OpenEmbedded核心和更老的BitBake版本，可能导致问题。这就是Poky把核心元数据层和BitBake一起打包的原因之一。然而，BitBake向后兼容，使得你可以使用更新的版本和更老的元数据层。

·BUILD_SYS：构建系统的类型。变量作为UILD_SYS="${BUILD_ARCH}${BUILD_VENDOR}-${BUILD_OS}"被定义在bitbake.conf中。BUILD_ARCH包含uname-m的输出，BUILD_OS包含uname-s的输出，BUILD_VENDOR是通常为空的定制字符串。

·NATIVELSBSTRING：用破折号连接的发行者ID和发布号码，正如用命令lsb_release获取的。

·TARGET_SYS：目标系统的类型。这个变量作为TARGET_SYS="${TARGET_ARCH}${TARGET_VENDOR}${@['-'+d.getVar（'TARGET_OS',True）,"][d.getVar（'TARGET_OS',True）==（"or'custom'）]}"被定义在bitbake.conf中。

·MACHINE：BitBake正在为其构建的目标机器。

·DISTRO：目标发行版的名字。

·DISTRO_VERSION：目标发行版的版本。

·TUNE_FEATURES：针对目标CPU架构的优化参数。

·TARGET_FPU：用于目标架构的浮点单元的身份识别。

·meta[-xxxx]：用于元数据层（如果它们是从Git仓库中检出的）的分支和提交ID。

对于程序清单5-1中的例子，在使用x86上的64位架构的Fedora18构

建主机上，作者已经使用了从Yocto项目的Git仓库中检出的Poky。目标系统是使用Poky发行版策略为QEMU构建的32位Linux系统。

任务日志文件

BitBake为它所运行的每个菜谱的每个任务创建日志文件。默认情况下，BitBake在T变量指向的目录中存储任务日志文件。默认情况下，这个目录是菜谱工作目录的子目录：

```
T = "${WORKDIR}/temp"
```

这对除了清理工作目录的clean任务以外的所有任务都适用。因为运行clean任务最终也删除工作目录和它的子目录，所以对于clean任务，T变量被有条件地设置成：

```
T_task-clean = "${LOGDIR}/cleanlogs/${PN}"
```

日志文件被存储到根据其执行菜谱的包名命名的子目录中。

用于任务的日志文件被命名为log.do_<taskname>.<pid>，其中pid是任务被BitBake运行时的进程ID。进程ID用于区分多次执行的相同任务的任务日志。这使得比较场景前和场景后以及比较成功的任务执行和失败的执行是简单直接的。更大的任务号码通常意味着更多的当前运行。名字为log.do_<taskname>的符号连接指向包含最当前运行的日志输出的日志文件。

除非在元数据文件中有不正确的语法，否则几乎所有构建失败都与菜谱任务的执行有关。当定位针对失败任务的日志文件时，通过打印它的完整路径和错误消息到控制台，BitBake提供了大量帮助。程序清单5-2显示了用于构建nano文本编辑器的菜谱当前失败的任务。

程序清单5-2　任务失败

```
NOTE: Resolving any missing task queue dependencies

Build Configuration:
BB_VERSION        = "1.21.1"
BUILD_SYS         = "x86_64-linux"
NATIVELSBSTRING   = "Fedora-18"
TARGET_SYS        = "i586-poky-linux"
MACHINE           = "qemux86"
DISTRO            = "poky"
DISTRO_VERSION    = "1.5+snapshot-20140211"
TUNE_FEATURES     = "m32 i586"
TARGET_FPU        = ""
meta-mylayer      = "<unknown>:<unknown>"
meta
meta-yocto
meta-yocto-bsp    = "master:095bb006c3dbbfbdfa05f13d8d7b50e2a5ab2af0"

NOTE: Preparing runqueue
NOTE: Executing SetScene Tasks
NOTE: Executing RunQueue Tasks
ERROR: This autoconf log indicates errors, it looked at host include and/or
library paths while determining system capabilities.
Rerun configure task after fixing this. The path was
'/develop/yocto/yocto-git/x86/tmp/work/i586-poky-linux/nano/2.3.1-r0/nano-2.3.1'
ERROR: Function failed: do_qa_configure
ERROR: Logfile of failure stored in:

.../tmp/work/i586-poky-linux/nano/2.3.1-r0/temp/log.do_configure.17865
ERROR: Task 5 (.../meta-mylayer/recipes-apps/nano/nano_2.3.1.bb,
do_configure) failed with exit code '1'
NOTE: Tasks Summary: Attempted 550 tasks of which 545 didn't need to be rerun and
1 failed.
No currently running tasks (550 of 558)

Summary: 1 task failed:
  .../meta-mylayer/recipes-apps/nano/nano_2.3.1.bb, do_configure
Summary: There were 3 ERROR messages shown, returning a non-zero exit code.
```

以ERROR开头的行包含构建失败的相关信息，例如指向问题根本原因的线索、失败的任务以及到日志文件的完整路径。你可以简单地使用日志文件的路径在你喜欢的编辑器中查看任务失败原因的额外信息。即使你清除控制台或者关闭它的窗口，你仍然可以稍后从cooker日志文件中获取这个输出。

5.1.2 使用记日志语句

一个若不是最常用到的，那就是更常用到的，用于在编程中调试的方法是向代码中插入允许遵循执行路径和通过打印变量内容来检查数据的调试消息。

BitBake提供多个级别来指示消息的严重性：

·Plain（普通）：完全按照传递的方式记录消息而不带任何额外信息。

·Debug（调试）：记录前缀了DEBUG:的消息。记日志功能也期望在1和3之间指示调试级别的数字型参数。然而，只有对于Python函数，调试级别才被实际求值。对于shell函数，所有消息都被记录而不管调试级别。

·Note（注意）：记录前缀了NOTE:的消息。它被用来通知用户需要知道的条件或者信息。

·Warn（警告）：记录前缀了WARNING:的消息。警告指示最终应该被用户关注的问题；然而，它们不导致构建失败。

·Error（错误）：记录前缀了ERROR:的消息。错误指示需要被解决以成功完成构建。然而，构建可以继续直到没有需要构建的任务。

·Fatal（致命）：记录前缀了FATAL:的消息。致命条件使得BitBake在消息已经被记录后立即终止构建过程。

依赖于上下文，所有消息被写入到各自的日志文件中。注意，警告、错误和致命消息页被输出到控制台。调试消息仅仅当BitBake的调

试级别等于或者高于消息的级别时才被写到控制台。BitBake的调试级别是通过增加-D参数到BitBake的命令行来设置的。

```
bitbake -D <target>
bitbake -DD <target>
bitbake -DDD <target>
```

分别选择调试级别1、2或者3。普通消息永远不被写到控制台。

日志消息可以被插入到定义在支持可执行元数据的文件中的任何函数：也就是，菜谱、追加文件和类。对Python和shell可执行元数据来说，提供了匹配日志级别的记日志函数（见表5-1）。

表5-1　日志函数

日志级别	Python 函数	Shell 函数
普通	bb.plain（message）	bbplain message
调试	bb.debug（level, message）	bbdebug level message
注意	bb.note（message）	bbnote message
警告	bb.warn（message）	bbwarn message
错误	bb.error（message）	bberror message
致命	bb.fatal（message）	bbfatal message

Python函数作为BitBake库的一部分实现。你可以在文件bitbake/lib/bb/__init__.py中找到实现。BitBake的Python记日志使用了和扩展了Python的记日志的类。文件bitbake/lib/bb/msg.py为日志消息提供格式化和过滤类。特别是，类BBLogFormatter提供用于不同消息级别的色彩化。如果色彩化被启用，那么警告消息以黄色显示，错误和致命消息以红色打印。

shell函数是由logging.bbclass实现的，logging.bbclass是由OpenEmbedded核心元数据层提供的。

在程序清单5-3和5-4中的伪代码说明了对于Python和shell可执行元

数据的记日志功能的使用。

程序清单5-3　Python记日志例子

```
python do_something() {
    bb.debug(2, "Starting to do something...")
    if special_condition:
        bb.note("Met special condition.")
    bb.debug(2, "Processing input")
    if warning_condition:
        bb.warn("Upper limit out of bounds, adjusting to maximum.")
    if error_condition:
        bb.error("Recoverable error, proceeding, but needs to be fixed.")
    if fatal_condition:
        bb.fatal("Division by 0, unable to proceed, exiting.")
    bb.plain("The result of doing something is 'mostly nothing'.")
    bb.debug(2, "Done with doing something.")
```

程序清单5-4　shell记日志例子

```
do_something() {
    bbdebug 2 "Starting to do something..."
    if [ special_condition ]; then
        bbnote "Met special condition."
    fi
    bbdebug 2 "Processing input"
    if [ warning_condition ]; then
        bbwarn "Upper limit out of bounds, adjusting to maximum."
    fi
    if [ error_condition ]; then
        bberror "Recoverable error, proceeding, but needs to be fixed."
    fi
    if [ fatal_condition ]; then
        bbfatal "Division by 0, unable to proceed, exiting."
    fi
    bbplain "The result of doing something is 'mostly nothing'."
    bb.debug 2 "Done with doing something."
```

在你自己的类和菜谱中有用的地方使用记日志消息是可取的和被鼓励的。指示处理过程的消息应该使用调试级别。指示警告和错误的消息应该使用恰当的级别。通常，你不应该使用致命级别而应替代性地使用错误级别，它给了BitBake完成并行运行的其他任务的处理、清理和使构建环境处在一致状态的机会。使用致命级别会导致BitBake立即终止执行，它可能使得构建环境处于BitBake无法在后续执行时自动恢复的状态。

5.2 任务执行

对于任何给定的菜谱，BitBake以由依赖性定义的特殊序列执行一系列任务，对菜谱使用命令listtasks会得到你为该菜谱定义的所有任务的列表（见程序清单5-5）。

程序清单5-5　该列出用于菜谱的任务

```
user@buildhost:~$ bitbake busybox -c listtasks

[... omitted for brevity ...]

NOTE: Preparing runqueue
NOTE: Executing RunQueue Tasks
do_build                        Default task for a recipe - depends on all
                                other normal tasks required to 'build' a
                                recipe
do_checkuri                     Validates the SRC_URI value
do_checkuriall                  Validates the SRC_URI value for all recipes
                                required to build a target
do_clean                        Removes all output files for a target
do_cleanall                     Removes all output files, shared state
                                cache, and downloaded source files for a
                                target
do_cleansstate                  Removes all output files and shared state
                                cache for a target
do_compile                      Compiles the source in the compilation
                                directory
do_compile_ptest_base           Compiles the runtime test suite included in
                                the software being built
do_configure                    Configures the source by enabling and
                                disabling any build-time and configuration
                                options for the software being built
do_configure_ptest_base         Configures the runtime test suite included
                                in the software being built
do_devshell                     Starts a shell with the environment set up
                                for development/debugging

[... omitted for brevity ...]
```

输出是以字母顺序列出所有任务，并简要说明每项任务的功能。然而，它不提供关于在一般构建过程中什么任务被运行以及以何种顺序运行的信息。

正如我们在第3章中所看到的，对任何软件包的构建很大程度上遵循下面这些标准步骤：

·获取：从下载站点或者克隆源仓库获取包源代码包，以及所有可应用的补丁和其他本地文件。

·解压：从包中解压源代码、补丁和其他文件。

·打补丁：应用补丁。

·配置：在目标环境中为构建准备资源。

·构建：编译源，归档对象进入库，以及将对象链接进可执行程序。

·安装：复制二进制和辅助文件到它们在模拟的系统环境中的目标目录中。

·打包：根据被选择的包管理系统创建安装包，包括任何程序清单。

你可以通过分析由在OpenEmbedded核心元数据层中的类创建的任务依赖性来遵循构建链，方法是在类文件中搜索关键字addtask：

```
$ grep addtask *.bbclass
```

使用输出，你可以判定任务依赖性（见程序清单5-6）。

程序清单5-6　任务依赖性

```
base.bbclass:           addtask fetch
base.bbclass:           addtask unpack after do_fetch
patch.bbclass:          addtask patch after do_unpack
base.bbclass:           addtask configure after do_patch
license.bbclass:        addtask populate_lic after do_patch before do_build
base.bbclass:           addtask compile after do_configure
base.bbclass:           addtask install after do_compile
staging.bbclass:        addtask populate_sysroot after do_install
package.bbclass:        addtask package before do_build after do_install
package_deb.bbclass:    addtask package_write_deb before do_package_write
                        after do_packagedata do_package
package_ipk.bbclass:    addtask package_write_ipk before do_package_write
                        after do_packagedata do_package
package_rpm.bbclass:    addtask package_write_rpm before do_package_write
                        after do_packagedata do_package
package.bbclass:        addtask package_write before do_build
                        after do_package
base.bbclass:           addtask build after do_populate_sysroot
```

对于程序清单5-6，我们使用了应用到典型用户空间软件包的任务。例如内核的特殊包使用不同的过程。我们也按照执行序列排序了列表并且省略了对于典型的包构建过程来说不相关的任务。

5.2.1 执行特定任务

对给定的菜谱单独执行任务和在连续执行之间做出调整，这在调试构建失败时可能是非常有帮助的。

使用

```
$ bitbake <target> -c <task>
```

为给定的<target>菜谱执行<task>。常见的场景是第一次构建包且在编译包的时候发现错误，对源代码做出调整或者修改配置，然后再次运行compile任务来验证变更是否已经解决了问题：

```
$ bitbake busybox
[...]
[make some changes to the source code]
$ bitbake busybox -c compile
```

单独重新执行任务允许你在构建过程的任何阶段做变更然后仅仅执行被变更影响的任务。因为BitBake为任务的每次执行创建日志文件，所以被进程ID所区分，你可以轻松地分析修改的效果。

一旦变更有了希望的结果，你需要把它们和一般构建过程集成起来。因为在包的构建目录中做出的修改是不永久的并且在清理构建目录时会被擦掉，所以需要通过修改菜谱、创建追加文件、包含额外的补丁等来增加它们到构建过程。

5.2.2 任务脚本文件

对一个任务的每次执行，BitBake创建包含当运行任务时其所执行的命令的脚本文件。任务脚本文件位于和任务日志文件相同的目录中：

```
T = "${WORKDIR}/temp"
```

任务脚本文件被命名为：

```
run.do_<taskname>.<pid>
```

<pid>后缀用于区分对于多次执行的相同任务的脚本文件。不带进程ID后缀的文件run.do_<taskname>是一个符号链接，其指向BitBake为任务的最新运行执行的脚本文件。

用于Python任务的脚本文件是从继承和访问BitBake数据字典的BitBake构建环境内部执行的。用于shell任务的脚本文件包含整个构建环境，并且是通过派生进程执行的。在很多情况下，它们可以直接从命令行运行。

文件log.task_order包含任务及其对应的脚本文件和用于最新执行的进程ID的列表。

5.3 分析元数据

整个BitBake构建过程是由元数据驱动的。可执行元数据提供过程步骤，它们是通过不同变量的值来配置的。这些变量中的很多依赖于特定菜谱的执行上下文。例如，类似SRC_URI的变量是由每个菜谱定义的。很多变量也引用其他变量，它们在执行上下文中扩展。条件变量设置和追加是用于动态调整执行上下文的强大概念但是在调试构建失败时也增加了复杂性。

从变量名字和设置中的简单的错别字到不正确的变量扩展和条件赋值，对构建过程来说，失败或产生不正确的输出的可能很多。因此，能够分析构建系统用于不同上下文内的变量设置是重要的。命令

```
$ bitbake -e
```

打印用于全局BitBake环境的数据字典到控制台。在其潜在地被特定菜谱上下文覆盖之前检查默认设置是有用的。用带有目标的命令显示用于那个特定目标的数据字典条目：

```
$ bitbake -e <target>
```

这个命令的缺陷是它列出变量和函数，因为变量不是别的而是可执行元数据。BitBake在相同的数据字典中存储变量和函数。列出每个函数的代码会使得输出相当冗长难以分析。不幸的是，不管BitBake还是OpenEmbedded都没有提供仅仅列出变量的功能。然而，通过包含程序清单5-7中的代码到类文件中，这个功能可以被简单地增加。

程序清单5-7　showvars任务

```
addtask showvars
do_showvars[nostamp] = "1"
python do_showvars() {
        # emit only the metadata that are variables and not functions
        isfunc = lambda key: bool(d.getVarFlag(key, 'func'))
        vars = sorted((key for key in bb.data.keys(d) \
               if not key.startswith('__')))
        for var in vars:
            if not isfunc(var):
                try:
                    val = d.getVar(var, True)
                except Exception as exc:
                    bb.plain('Expansion of %s threw %s: %s' % \
                              (var, exc.__class__.__name__, str(exc)))
                bb.plain('%s="%s"' % (var, val))
}
```

当和

```
$ bitbake <target> -c showvars
```

一起使用时，任务以字母顺序列出目标执行环境的所有变量而不列出函数。

5.4 开发shell

起源于从源到对象、库和可执行文件的编译和链接在交叉构建时调试是有挑战性的。你不能仅仅改变到源目录、输入make、检查错误消息以及改正问题。用于软件包的构建环境通常被配置成主机系统上的原生构建。交叉构建环境要求不同的并且常常是相当复杂的用于工具、头文件、库等的设置以进行正常操作。

Poky参考发行版为它自己的BitBake构建过程创建交叉构建环境。通过命令devshell，它也可以在shell中向开发者提供交叉构建环境。命令

```
$ bitbake <target> -c devshell
```

为目标启动、带有交叉构建环境的终端。交叉构建环境的设置与Poky用于完成自己构建的shell完全匹配。在那个shell内，你可以像使用构建主机上的原生构建一样使用开发工具。环境参考正确的交叉编译器以及任何头文件、库和其他被需要的文件构建软件包。

如果devshell命令所靶向的软件包的任何依赖性还没有被构建，那么Poky预先构建它们。

如果你正在从带有窗口管理器的图形用户界面上使用BitBake，那么它自动尝试判断需要用的终端程序并且在新窗口中打开它。变量OE_TERMINAL控制使用何种终端程序。它通常被设置成auto。你可以将它设置成在构建环境的conf/local.conf文件内被支持的终端程序之一。终端程序必须被安装在开发主机上。可以通过设置OE_TERMINAL="none"来完全禁止devshell的使用。

5.5 依赖性关系图

在第4章中，我们看到包是如何使用它们菜谱中的DEPENDS和RDEPENDS变量来声明对其他包的直接的构建时和运行时依赖性的。你可以简单地意识到，这个实践可以如何导致长且复杂的依赖性链。

当然，BitBake必须能够解析这些依赖性链以用正确的顺序来构建包。使用它的依赖性解析器，BitBake也可以创建用于分析和调试的依赖性关系图。

BitBake使用DOT普通文本图形描述语言来创建依赖性关系图。DOT以一种可以被人和计算机程序共同读取的语言来提供一种描述带有节点和边缘的无向和有向图的简单方法。来自Graphviz（www.graphviz.org）包的程序以及许多其他程序可以读取DOT文件并且将其渲染成图形展示。

为了为目标创建依赖性关系图，用-g或者--graphviz选项调用BitBake。使用

```
$ bitbake -g <target>
```

为<target>创建以下的依赖性文件：

·pn-buildlist：这个文件不是DOT文件但是包含以目标开始的逆向构建顺序的包列表。

·pn-depends.dot：包含首先声明节点然后声明边缘的有向图中的包依赖性。

·package-depends.dot：本质上和pn-depends.dot一样但是紧接节点之

后为节点声明边缘。这个文件可能更易于被人读取，因为它会将终结于节点的边缘和节点一起分组。

·task-depends.dot：声明在任务级别上的依赖性。

BitBake也提供了内置的用于包依赖性的用户界面——依赖性浏览器。你可以用如下的命令启动依赖性浏览器：

```
$ bitbake -g -u depexp <target>
```

依赖性浏览器使你能够在图形用户界面内分析构建时和运行时依赖性以及逆向依赖性（依赖于那个包的包），如图5-1所示。

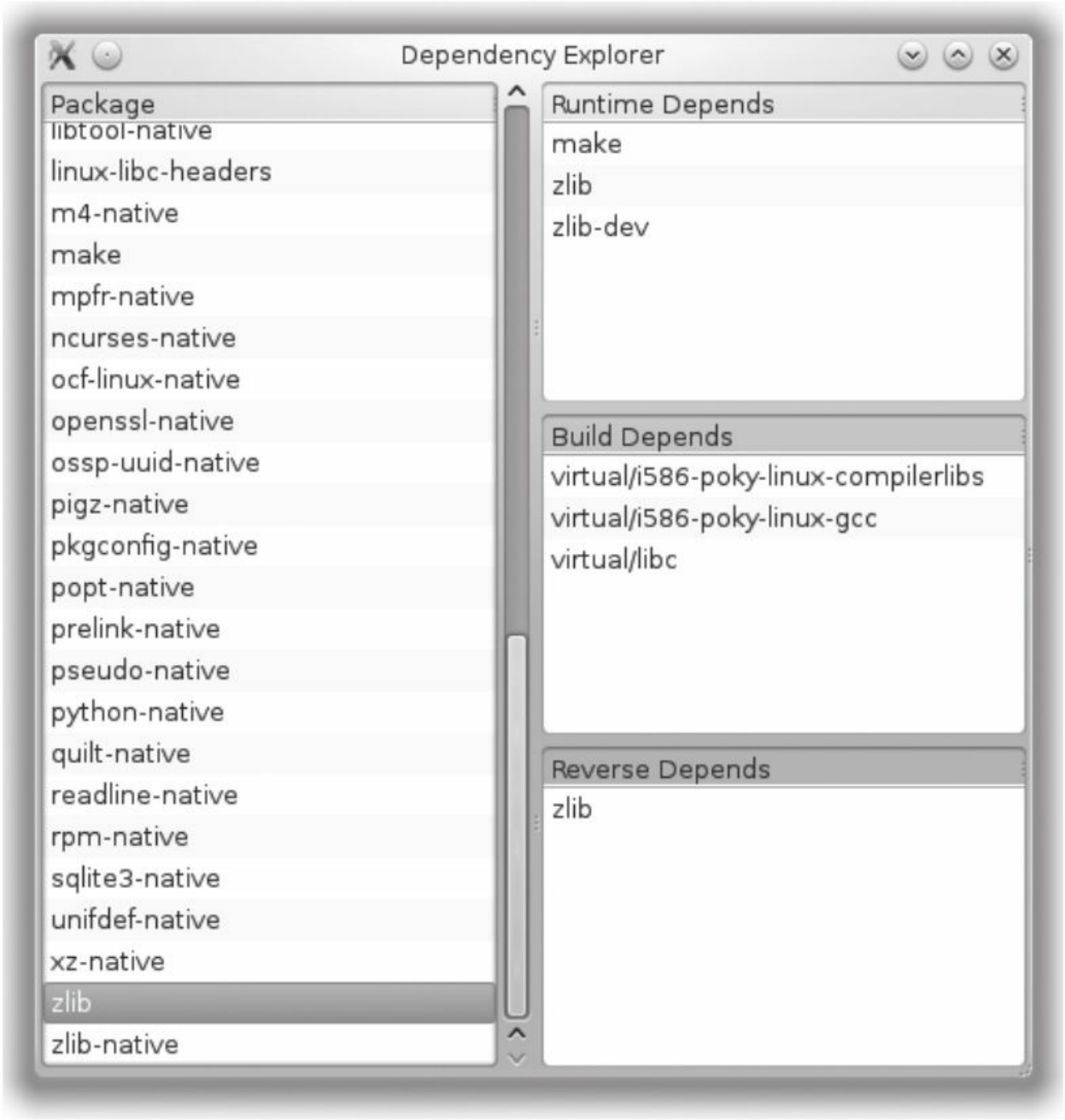

图5-1　依赖性浏览器

如果Graphviz包安装在开发系统上，那么你可以使用它来创建依赖性关系图的可视化渲染：

```
$ dot -Tpng pn-depends.dot -o pn-depends.png
```

从DOT文件创建PNG文件。

5.6 调试层

BitBake的层架构提供了优雅的组织菜谱的方法。然而，它也引入了复杂度，特别是当多个层提供相同的菜谱或用追加文件修改相同的菜谱时。

工具bitbake-layers提供一些帮助分析和调试构建环境所用的层的函数。调用

```
$ bitbake-layers help
```

提供可以和工具一起使用的命令列表：

·help：不带任何参数或者通过指定help作为参数，工具显示可用的命令列表。如果你提供命令，那么它将显示对于那个命令的额外帮助信息。

·show-layers：展示构建环境所用的层以及它们的路径和优先级的列表。

```
$ bitbake-layers show-layers
layer                 path                                      priority
==========================================================================
meta                  /path/to/poky/meta                        5
meta-yocto            /path/to/poky/meta-yocto                  5
meta-yocto-bsp        /path/to/poky/meta-yocto-bsp              5
meta-mylayer          /path/to/meta-mylayer                     1
```

·show-recipes：以字母顺序展示菜谱的列表，包括提供给它的层。

```
$ bitbake-layers show-recipes
Parsing recipes..done.
=== Available recipes: ===
acl:
  meta                 2.2.52
acpid:
  meta                 1.0.10
adt-installer:
  meta                 0.2.0
alsa-lib:
  meta                 1.0.27.2
alsa-state:
  meta                 0.2.0
alsa-tools:
  meta                 1.0.27
[...]
```

·show-overlayed：展示叠加的菜谱的列表。如果另带有相同名字的菜谱存在于一个不同的层中，那么这个菜谱是叠加的。

```
$ bitbake-layers show-overlayed
Parsing recipes..done.
```

```
 === Overlayed recipes ===
 mtd-utils:
   meta              1.5.0
   meta-mylayer      1.4.9
```

当构建叠加的菜谱时，BitBake发出告警并且构建来自带有最高优先级的层的菜谱。

·show-appends：展示带有追加文件的菜谱列表。追加文件以应用顺序显示。

```
$ bitbake-layers show-appends
Parsing recipes..done.
=== Appended recipes ===
alsa-state.bb:
  /.../meta-yocto-bsp/recipes-bsp/alsa-state/alsa-state.bbappend
psplash_git.bb:
  /.../meta-yocto/recipes-core/psplash/psplash_git.bbappend
[...]
```

·show-cross-depends：展示所有依赖于其他层中元数据的菜谱的列

表。

```
$ bitbake-layers show-cross-depends
Parsing recipes..done.
meta-yocto/recipes-core/tiny-init/tiny-init.bb RDEPENDS
           meta/recipes-core/busybox/busybox_1.22.1.bb
meta-yocto/recipes-core/tiny-init/tiny-init.bb inherits
           meta/classes/base.bbclass
meta-yocto/recipes-core/tiny-init/tiny-init.bb inherits
           meta/classes/patch.bbclass
meta-yocto/recipes-core/tiny-init/tiny-init.bb inherits
           meta/classes/terminal.bbclass
```

·flatten<directory>：通过解析菜谱叠加和追加并把输出写入由参数directory提供的单层目录来扁平化层级结构。一些规则适用：

·如果层包含叠加的菜谱，来自带有最高优先级的层的菜谱被使用。如果层具有相同的优先级，那么在构建环境的conf/bblayers.conf文件的BBLAYERS变量中的层顺序决定哪个菜谱被使用。

·如果多个层在相同的子目录中包含具有相同名字的非菜谱文件（如镜像、补丁或者类似文件），那么它们将被来自最高优先级层的那个所覆盖。

·来自列在构建环境的conf/bblayers.conf的BBLAYERS变量中的第一个层的conf/layer.conf文件被使用。

·追加文件的内容按照层的优先级或者当追加相同菜谱的层有相同优先级时按照它们在BBLAYERS变量中的顺序被简单地增加到各自菜谱。

5.7 总结

本章介绍了大量由OpenEmbedded构建系统提供的用于辅助排除构建失败故障的工具。分离问题的根本原因仅仅是第一步。第二步是找到解决方案，它可能更具挑战性。然而，在很多情况下，其他开发者可能已经遇到过相同或者类似的问题，并且搜索互联网常常引发一个或者更多关于问题和可能解决方案的讨论。

·日志文件是用于识别失败位置的好起点。任务日志文件包含任务所执行的命令的全部输出。

·插入你自己的日志消息到菜谱和类中可以帮助查明构建失败。

·多次执行特定任务允许对比结果。任务日志文件不被连续的执行所覆盖。

·打印元数据显示它们的任务上下文中的变量，包含变量扩展和条件赋值。

·开发shell允许在BitBake使用的相同的交叉构建环境中执行make目标。

·依赖性关系图支持追踪由软件包之间未被解决的依赖性导致的构建失败。

·bitbake-layers实用程序提供了一组辅助调试使用多个层的构建环境的函数。

第6章　Linux系统架构

在前面的章节中，我们介绍了Yocto项目和OpenEmbedded构建系统的核心概念和组件。在我们详细探索如何用Yocto项目构建我们自己的定制Linux发行版之前，是时候后退一步看看是什么组成了Linux系统。

理解Linux系统的架构为OpenEmbedded核心用来创建各种系统组件（例如根文件系统、内核镜像和引导加载程序）的方法提供了上下文。

我们首先看看Linux系统的剖析，然后将其分解成它的组件。

6.1 Linux或者GNU/Linux？

你可能已经注意到了，在一些上下文中，Linux操作系统被称为Linux，而在另一些上下文中则被称为GNU/Linux。这种区别背后的原因是，严格来说，名字Linux仅仅指的是作为操作系统的基础或者核心的Linux内核。

为了操作系统可以有用，更多的应用和库是被需要的——开发工具、编译器、编辑器、shell、实用程序等——它们不是内核的一部分。大量的这些应用是由Richard Stallman在1984年启动的GNU项目所提供的。可能来自GNU项目的最知名的工具是GCC编译器、GLIBC库和EMACS编辑器。关于GNU很少知道的是，它也包含被称为Hurd[1]的操作系统内核。

因为几乎所有用于桌面和服务器应用的Linux操作系统都包含很多GNU项目软件包，所以很多人认为GNU/Linux是用于操作系统的更加恰当的名字，而当仅仅指的是内核时，Linux是合适的。

嵌入式Linux操作系统通常不包含大量工具和应用，因为它们是专用于特定用例的。通常，除了对嵌入式设备来说必要的特定应用以外，你会发现有限集合的标准工具，例如shell、编辑器和少量实用程序。常见的是，那组应用是在单一的工具盒子（例如BusyBox）中提供的。

[1] https://www.gnu.org/software/hurd/hurd.html。

6.2 Linux系统的剖析

图6-1描述了Linux系统架构的高度概览。引导加载程序不严格是Linux系统的一部分，但是对于启动系统它是必需的，所以在用Yocto项目构建完整功能性的系统的上下文中其也是相关的。

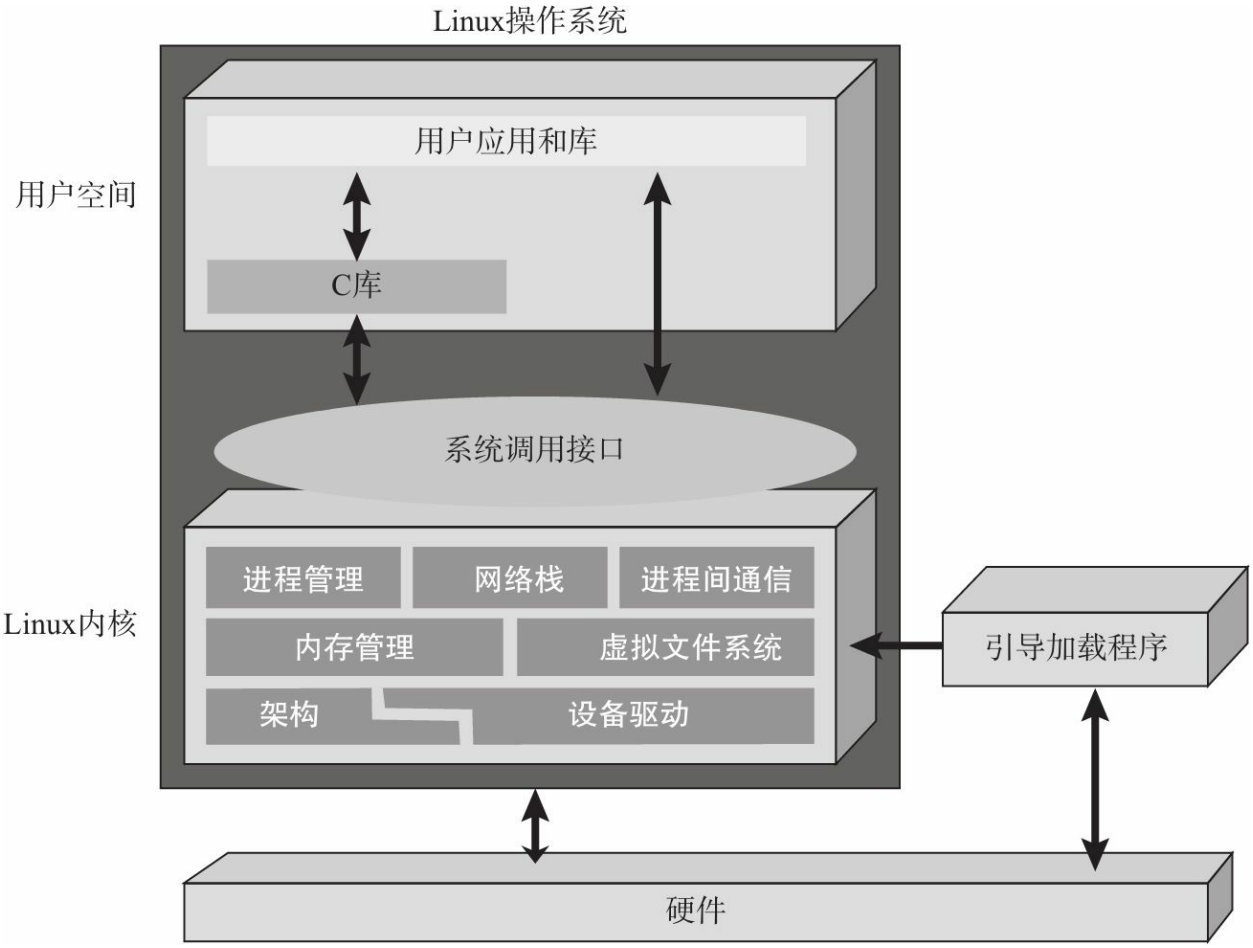

图6-1　Linux操作系统架构

Linux操作系统可以被划分成两个级别，内核空间以及用户或应用空间。这种区分不只是概念性的，而是起源于事实——来自内核空间的代码相对于来自用户空间的代码是在不同的处理器操作模式下被执行的。

所有内核代码都是在非受限制或者特权的模式[1]中被执行的。在这种模式下，架构的指令集的所有指令都可以被执行。相反，应用代码是在受限制或者用户的模式中被执行的。在这个模式下，直接访问硬件的指令——输入/输出（Input/Output，I/O）指令——或者其他可以改变机器状态的指令是不被允许的。对特定内存区域的访问通常也是被限制的。

从用户模式改变到内核模式需要CPU切换上下文，它是由依赖于架构的特殊指令来实现的。例如，在遗留的x86 CPU上，int 80h软件中断或者陷阱实现了那个目的。新的x86_64 CPU提供syscall指令，是它被使用而不是int 80h陷阱。

[1] 和几乎所有事情一样，有规则的例外。uClinux项目（http://www.uclinux.org）构建了一个针对没有内存管理系统的微控制器的Linux系统。

6.3 引导加载程序

虽然在系统的生命周期中引导加载程序仅仅在启动时扮演了非常短时间的角色，但它是非常重要的系统组件。从某种程度来说，配置引导加载程序对任何运行在标准PC方式硬件上的Linux系统来说都是常见任务。对于嵌入式系统，设置引导加载程序成为非常特殊的任务，因为用于嵌入式系统的硬件不仅仅非常异于标准PC架构，而且还以很多不同变体的形式出现。这不但对不同的CPU架构来说是成立的，对实际的CPU或者片上系统（System-on-Chip，SoC）以及很多和CPU或者片上系统组合的组成硬件平台的外设来说也是成立的。

通常，引导加载程序被划分成两类：引导器（loader）和监控器（monitor）。在前一个情况下，引导加载程序仅仅提供仅有的功能性来初始化硬件和加载操作系统。在后一个情况下，引导加载程序也包含用户可以通过其与引导加载程序交互的命令行接口，它可以用于硬件配置、再编程（reprogramming）、初始化、测试以及其他任务。使用那些功能，在开发阶段，监控器向工程师提供了大量灵活性，但是当部署产品时也带来了一些挑战。最终用户不应该能够不经意地进入监控模式。

对于Linux不缺少引导加载程序。即使仅详细讨论最常被使用的引导加载程序也可以填满一本完整的书。在这一部分，我们聚焦在引导加载程序的角色上并且介绍最流行的一些引导加载程序，强调那些被Yocto所支持的引导加载程序。

6.3.1 引导加载程序的角色

在电力被应用到处理器板后，在任何软件应用可以被执行前，大部分硬件组件需要被初始化。每个处理器架构有它自己的、在硬件可以被使用前需要被执行的初始化步骤的集合。引导加载程序通常仅仅初始化对操作系统内核启动来说必要的硬件。所有其他的硬件和外设是在稍后的启动过程阶段由操作系统本身初始化的。一旦操作系统内核控制了硬件，它可能重新初始化最初由引导加载程序所设置的硬件组件。

大部分处理器有一个默认地址，从其中可以读取第一个要被执行的指令。其他处理器从被定义的位置读取该地址。硬件设计者使用那个信息来对应地安排启动芯片（bootrom）的布局和地址范围。虽然常被称为启动芯片，但是这个设备经常是电子可擦写可编程只读存储器（Electrically Erasable Programmable Read-Only Memory，EEPROM），或者现在的闪存（flash memory）。

启动芯片包含通常被称为第一阶段引导加载程序（first-stage bootloader）的东西。这个引导加载程序可以是仅仅数行从安全数字（secure digital）芯片加载二次引导加载程序的代码或者是提供用以初始化外围设备以及从其他设备（例如硬盘或者网卡等）启动系统的功能性的更加复杂的系统。后者的例子是在标准PC中发现的BIOS。

在由引导加载程序执行的硬件前期初始化中最重要的步骤是内存和内存控制器的初始化。通常，CPU在带有逻辑和物理地址之间直接映射的实模式（real mode）中开始。在实模式中，CPU不使用内存管理单元（Memory Management Unit，MMU），该单元提供抽象、虚拟和物理内存间的映射以及内存保护和重分配。操作系统内核通常作为它的第一个硬件配置步骤来初始化内存管理单元。然而，根据平台的不同，对引

导加载程序来说，启用和配置内存管理单元可能是必要的。

在硬件初始化之后，引导加载程序定位操作系统，通常是内核，加载它进入内存，传递配置参数并且把控制转交给操作系统。此时，引导加载程序已经完成了它的责任并且终止。操作系统稍后回收所有引导加载程序使用的内存空间。

6.3.2 Linux引导加载程序

作为Linux系统开发者，对项目来说，你有很多用于引导加载程序的选择。引导加载程序在功能性、处理器和操作系统支持方面有差异。它们中的很多不但可以启动Linux也可以启动其他操作系统。另一个区别是它们可以从什么介质启动操作系统。除了软盘（floppy disk）、硬盘和USB存储设备，它们中的很多也可以通过BOOTP和TFTP从LAN启动。

当选择引导加载程序时，架构和启动介质支持是首先要考虑的。在开发中，监控功能性可以极大地加速往返开发工程化和优化启动参数。然而，当开发已经完成并且已完成产品被部署时，监控功能性可能成为惹麻烦的事，因为用户可能不经意地（或者是故意地）进入监控模式。一些提供监控能力的引导加载程序可以被重新配置以允许或者禁止对监控特性的访问或者可以用密码来保护监控模式。然而，最安全的选项，是在编译时就移除监控能力。

表6-1提供了常见被Linux使用的一些引导加载程序的概览。不是所有在表中展示的引导加载程序都有Yocto项目或者OpenEmbedded菜谱。为了更加完整的Linux引导加载程序的描述，它们是被包含的。不是所有的嵌入式系统都是同等创建的，最终你可能发现你自己正在写用于引导加载程序的、比默认被Yocto项目支持的那些更加适合应用的菜谱。

几乎所有引导加载程序都允许在默认被启动（在可配置的超时之后）之前从菜单选择不同的系统。这是对笔记本、桌面和服务器计算机来说的常见实践，给用户选择以启动失败安全或者有限的系统或者在系统被修改后的预先工作的配置。对嵌入式系统来说，这个功能性可以被用来回退到早期版本，例如，如果系统升级已经失败了。

表6-1　Linux引导加载程序

引导加载程序	描述	Yocto项目菜谱	监控器	架构	许可	主页
LILO（LInux LOader）	用于很多Linux发行版的默认引导加载程序	否	否	x86	BSD	http://lilo.alioth.debian.org
ELILO（EFI LILO）	用于基于UEFI的PC硬件的LILO	否	否	x86	GPLv2	http://elilo.sourceforge.net

（续）

引导加载程序	描述	Yocto项目菜谱	监控器	架构	许可	主页
GNU GRUB	GNU LILO 后继者	是	否	x86	GPLv3	www.gnu.org/software/grub
SYSLINUX	轻量级引导加载程序	是	否	x86	GPLv2+	www.syslinux.org
U-Boot	流行的用于嵌入式系统的统一引导加载程序	是	是	ARM、MIPS、PPC、SuperH、x86、m86k	GPLv2	www.denx.de/wiki/U-Boot
BURG	GNU GRUB 分支	否	否	x86	GPLv3	https://code.google.com/p/burg
systemd-boot	用于基于 UEFI 的 PC 硬件的小型化引导加载程序	是	否	x86	LGPL 2.1	https://wiki.archlinux.org/index.php/Systemd-boot
RedBoot	基于 eCos 实时操作系统的 Red Hat 嵌入式调试和启动固件	否	是	ARM、MIPS、PPC、SuperH、x86、m86k	eCos	http://ecos.sourceware.org/redboot

在我们更细节地讨论引导加载程序之前，关于架构的一点注意：对x86的引用总是隐含着PC平台。这个平台在内存布局、总线和外设方面大部分是标准化的。它也包含BIOS，也提供对二次引导加载程序和操作系统配置信息的第一阶段引导加载程序。这使得它是相当不同于其他架构的，例如ARM和PPC，对它们没有标准化的平台。虽然不同平台使用带有相同架构内存布局的CPU，但是总线和外设可能非常不同。那些平台通常也没有BIOS或者任何其他标准化的用于获取系统配置的方法。那要求引导加载程序被特定地改变成适合特殊平台。

LILO

LILO（Linux引导加载程序，LInux LOader）曾经是用于针对x86系统的几乎所有Linux发行版的标准引导加载程序。最初由Werner Almesberger在1992年到1998年之间开发的，John Coffman从1999年到2007年之间接管维护，而自2010年起，Joachim Wiedorn一直在维护项目。虽然自从2013年GNU GRUB已经作为用于很多Linux发行版的默认引导加载程序而成功地替代了LILO，但是项目仍然是活跃的。LILO相

对于很多其他引导加载程序的主要优势是，它是文件系统不可知的。操作系统可以存在任何文件系统上，包括NTFS、EXT4、FAT32以及相对新的BTRFS，并且LILO可以无关地启动它。LILO可以直接从软盘和硬盘启动Linux内核镜像。LILO是很好的文档化的并且作为很多应用的切实可行的选项，在这些应用中，例如GRUB的更复杂的引导加载程序是不需要的。

ELILO

ELILO（基于EFI的Linux引导加载程序，EFI-based LInux LOader）是由Hewlett-Packard做出的用于支持基于EFI的硬件的LILO的分支。另外，它通过BOOTP、DHCP和TFTP协议处理网络启动。

GRUB

GRUB（GNU GRand Unified Bootloader）最初是由Erich Stefan Boleyn设计和实现的。从2013年开始，它开始替代LILO作为用于Linux发行版的主流的引导加载程序。

GRUB最终被GRUB 2所替代，现在被称为GRUB Legacy。今天，术语GRUB通常指的是GRUB 2。

PUPA研究项目为GRUB 2创建了基础，并且最终演化成了GRUB 2。GRUB 2是完全的重写，并且它相关于GRUB Legacy仅仅是因为它的名字。GRUB 2使用来自Etherboot[1]开源软件包的驱动来处理借由BOOTP、DHCP和TFTP协议的网络启动。

SYSLINUX

Syslinux是包含多个用于不同目的轻量级引导加载程序的一揽子项目：

·SYSLINUX：用于FAT和NTFS文件系统，可以处理硬盘、软盘和USB盘的引导加载程序。

·ISOLINUX：用于可启动El Torito CD-ROM的引导加载程序。

·EXTLINUX：用于Linux EXT2/EXT3/EXT4和BTRFS文件系统的引导加载程序。

·PXELINUX：用于借由BOOTP、DHCP和TFTP使用大多数网络硬件所支持的预启动执行环境（Preboot Execution Environment，PXE）的引导加载程序。

虽然稳定，但是Syslinux是一个活跃的项目，其最新发布也支持启动基于EFI的硬件。

U-Boot

U-Boot（统一引导加载程序，Universal Bootloader），也被称为Das U-Boot，可以被认为是嵌入式引导加载程序中的瑞士军刀。基于PPCBoot和ARMBoot项目，并且最初由来自DENX软件工程的Wolfgang Denk所开发，U-Boot是最具有丰富特性、灵活的、被活跃开发的当前可用的引导加载程序之一。

对硬件平台的大量支持仅仅是U-Boot的众多特性之一。在这个写作的时间，U-Boot支持超过1000个平台，其中超过600个是基于PowerPC的以及超过300个是基于ARM的。如果项目使用任何这些架构，那么机会是硬件平台已经被U-Boot所支持。即使它不是被支持的，你将有很大可能找到一个可以派生自己的平台支持代码的相近匹配。

U-Boot也支持用于平台配置的设备树（device trees）。设备树，也被称为开放固件（open firmware）或者扁平的设备树（Flattened Device

Tree，FDT），是字节码（byte code）形式的数据结构其中包含例如寄存器位置和大小、地址、中断等平台特定的参数，该参数被Linux内核所需要以正确地访问硬件和启动。U-Boot复制设备树数据结构到内存位置。设备树背后的理念是，在运行时提供平台配置参数，允许Linux内核可以被编译成用于多个平台，而不用关于特殊平台的特定信息。

U-Boot有很好的文档记录。主要文档是The DENX U-Boot and Linux Guide（DULG）[2]。除了那个指南，有非常细节的、包含在U-Boot源中的README文件。

U-Boot源仓库位于http://git.denx.de。

BURG

BURG（新统一引导加载程序，Brand-new Universal loadeR）是从GRUB衍生的近来的引导加载程序。初衷是为了支持更广泛的操作系统和有很多主题以在启动中定制引导加载程序的外观的能力。BURG还没有被广泛使用，因为它还没有被主流Linux发行版所采用。像GRUB，它仅仅支持x86系统但是提供GRUB的所有特性，包括借由BOOTP、DHCP和TFTP的网络启动。

systemd-boot

systemd-boot是定位于UEFI系统的简单的引导加载程序。它执行位于EFI系统分区（EFI System Partition，ESP）上的镜像。所有用于引导加载程序的配置文件和操作系统需要存在于EFI系统分区上。Linux内核需要通过设置CONFIG_EFI_STUB参数来配置EFI启动，以便systemd-boot可以直接将其作为EFI镜像执行。

RedBoot

RedBoot（Red Hat嵌入式调试和启动固件，Red Hat Embedded Debug and Bootstrap Firmware）是基于eCos硬件抽象层的引导加载程序。eCos是免费的、开源的、实时的、定位于嵌入式应用的操作系统。eCos和RedBoot最初是由Red Hat所开发和维护的，但是该公司已经中断了开发，所有源因此被GPL重新授权。活跃的开发和维护已经被eCosCentric公司所接管[3]，eCosCentric是由来自Red Hat的核心eCos开发者所建立的。

因为它的eCos传统，RedBoot为几乎所有架构支持很多嵌入式硬件平台，这些架构中就有ARM、MIPS、PPC和x86。

RedBoot的网络支持包括BOOTP和DHCP。它可以通过以太网使用TFTP和通过串行连接使用X-modem或者Y-modem协议下载镜像。

带有交互命令行接口的监控功能允许RedBoot配置、镜像下载和管理以及启动脚本化。这样的特性可以用于开发以及用于部署在现场的系统以进行远程升级。

[1] http://etherboot.org/wiki/index.php。
[2] www.denx.de/wiki/DULG/Manual。
[3] www.ecoscentric.com。

6.4 内核

操作系统的内核的两个主要功能是：

·管理计算机资源

·允许其他程序执行和访问资源

计算机的核心资源通常是由以下组成的：

·CPU：CPU是用于程序的执行单元。内核负责分配程序到处理器（调度）和设置执行环境（分派）。

·内存：内存为程序存储指令和数据。多个程序竞争内存，并且程序可以要求比在系统中物理可用的更多的内存（虚拟内存）。内核负责向程序分配内存、保护内存并且决定如果程序请求了多于可用的内存时会发生什么，在大部分情况下，它是小于计算机的物理内存的。

·I/O设备：I/O设备代表用于数据的源和水槽，例如键盘、鼠标、显示器、网络接口、存储等。内核负责通过统一的、从底层硬件的规格中抽象出来的编程接口来服务程序与设备交换数据的请求。

操作系统内核架构通常被分类成单内核（monolithic kernel）和微内核（microkernel）。单内核执行所有内核功能，包括在核心内核进程和内存上下文中的设备驱动。微内核仅仅执行核心功能，例如在核心内核上下文中的进程和内存管理，并且作为用户空间进程执行设备驱动。微内核允许更简单的系统配置和维护，因为设备驱动可以在系统运行时被加载和卸载。然而，那种便利性是以性能损失为代价的，因为微内核使用进程间通信（Interprocess Communication，IPC）来在内核模块之间交换数据。

虽然Linux内核为在执行时可以被加载和卸载的设备驱动提供可加载的内核模块，但是Linux被认为是单内核，因为这些模块被直接插入进内核的执行上下文中。因为它们是运行在内核的执行上下文中，Linux内核模块拥有对所有系统资源的访问。因此，进程间通信对于交换数据是不必需的，故而没有性能损失。

微内核架构的支持者声明，它是优于单内核架构的，因为它是更加干净的设计并且故障的设备驱动不能破坏整个系统。按照他们的说法，这个优势比因为增加的上下文切换和进程间通信所导致的性能损失更重要。这个问题成了在Linus Torvalds和Minix OS创造者Andrew S.Tanenbaum之间的长期存在的辩论[1]。

[1] Andrew S.Tanenbaum，“Linux Is Obsolete,”1992年1月29日，https://groups.google.com/forum/#!msg/comp.os.minix/wlhw16QWltI/XdksC

6.4.1 主要Linux内核子系统

Linux内核被划分成一组显示在图6-2中的子系统。

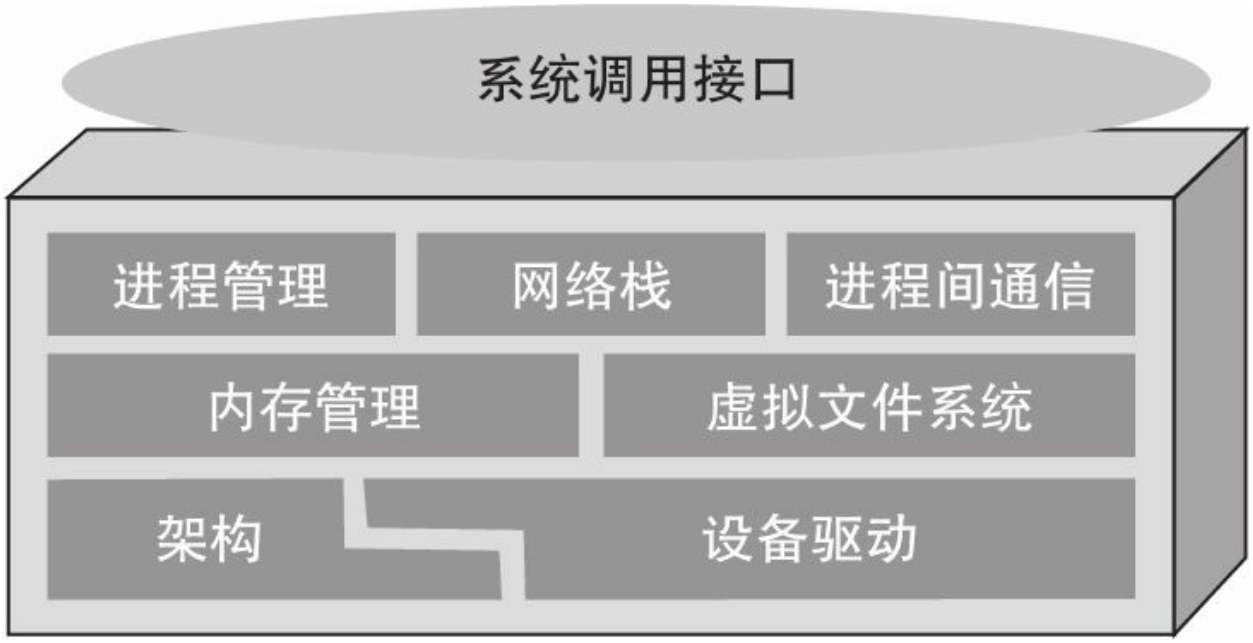

图6-2 Linux内核子系统

架构依赖代码

虽然大部分Linux内核代码独立于其执行所在的架构，但是有些部分仍然需要把CPU架构和平台考虑进去。

在Linux内核源树中，所有架构和平台特定的代码位于linux/arch子目录中。在那个子目录中，是另一个用于每个被Linux内核所支持的架构的目录。每个架构目录包含含有架构特定的内核代码的kernel子目录。汇编文件head.S（或者用于一些架构的不同的head_*.S文件）为CPU提供启动代码。

设备驱动

设备驱动处理所有Linux内核支持的设备。在大部分情况下，它们是硬件设备，但是一些驱动实现软件设备。一个例子是软件看门狗定时器（software watchdog timer）。

设备驱动组成Linux内核代码的大部分。在Linux内核源树内，设备驱动代码位于linux/drivers目录，它为特定硬件类别（例如Bluetooth、FireWire、I2C SCSI等）的各种驱动被进一步划分成子目录。

内存管理

仅有内核拥有对系统物理内存的不加限制的访问，因此负责安全地允许进程来访问它。大部分现代CPU包含提供一种通常远大于系统的实际物理内存的内存空间虚拟编址的内存管理单元。虚拟编址允许每个进程有它自己的、被内核从其他进程那里保护的私有内存空间。

当进程运行，内核必须保证进程的虚拟内存空间被映射到物理内存。内存以在Linux上通常是4k大小的页（page）或者段（segment）从虚拟地址被映射到物理地址[1]。然而，那不意味着进程每次只可以分配4k内存。Linux提供了被称为slab分配器的东西。虽然是基于4k页，但是slab分配允许更大块的内存被分配，减少由分配和释放引起的碎片，并且通过追踪slab的使用状态来重用以前分配的内存。

虚拟编址允许分配比物理存在于系统中的更多的内存。当物理内存被耗尽，内核可以把页从内存移动到例如硬盘的外部存储上。这个过程被称为交换（swapping），因为内存页从内存中被交换到了磁盘上。然而，大部分嵌入式系统没有硬盘。虽然交换到闪存设备是可能的，但是它相当不高效，并且会缩短闪存设备的寿命。

你可以在内核源树的linux/mm目录中找到内存管理代码。

虚拟文件系统

文件系统是当应用终止后用于持久数据存储的组织方案。它提供了用于写、读取、更新和擦除数据的机制并且管理在存储介质上的可用空间。

不像其他操作系统，Linux给了用户用于大量应用和存储介质的很多不同文件系统的选择。除了核心的Linux文件系统ext2、ext3和ext4，Linux提供对许多其他文件系统的支持，包括VFAT、NTFS、ZFS和新的Btrfs。

伴随着大的多样化是提供介于应用对于持久存储的需求和文件系统细节之间的抽象的挑战。对于例如创建、写、读取文件和浏览目录的基础操作，应用不需要知道它们的数据被存储在的文件系统的类型。

为了这个目的，Linux提供了用于文件操作的、被称为虚拟文件系统（Virtual File System，VFS）的通用抽象接口。虚拟文件系统是在底层文件系统实现和系统调用接口（System Call Interface，SCI）的文件操作之间的交换结构（switching fabric）。文件系统实现本质上是存在于虚拟文件系统层之上和统一的数据缓冲之下的数据管理插件。数据缓冲的目的是优化到物理存储设备的数据访问。数据缓冲层实现了一套通用的应用程序编程接口和函数集合，用于从文件系统实现到处理存储设备规格的底层设备驱动程序的数据访问。

虚拟文件系统的有趣方面是，它不限于存在于物理存储设备上的文件系统，但是同样地也为例如网络文件系统（Network File System，NFS）和服务器消息块（Server Message Block，SMB）的网络文件系统以及例如proc文件系统的伪文件系统提供相同的统一的接口。

针对虚拟文件系统和文件系统的源存在于Linux内核源树的linux/fs目录中。

进程管理

内核的进程管理负责进程的执行。应用编程领域通常区分进程和线程。在这个上下文中，进程指的是应用的执行上下文，线程指的是在进程内的独立执行路径。进程至少有一个线程，即它的主线程，从这里，它可以派生额外的线程。进程的所有线程共享相同的执行上下文、内存空间和其他资源。因此，线程也通常被称为轻量级进程（lightweight process）。

Linux内核不区分进程和线程的概念。它们都是作为代表包含代码、数据、栈和CPU寄存器的完整执行上下文的线程来被实现的。

进程管理分配计算机系统的核心资源，CPU。所有线程竞争可用CPU，调度器（scheduler）的任务是选择合适的线程来运行和分配它们可用的CPU（或者CPU核心）。Linux内核的默认调度算法是完全公平调度器（Completely Fair Scheduler，CFS）。完全公平调度器的目标是最大化全局CPU利用率同时也最大化系统的交互性能。像它所替代的以前O（1）调度器，完全公平调度器提供了调度时间，它独立于等待被执行的进程的数量。

Linux内核也提供了具有静态优先级的实时调度策略。最新增加到实时调度策略的是时限调度（deadline scheduling），使用基于最靠近过期时限的动态优先级的策略。

网络栈

Linux内核的网络栈本质上是参照著名的ISO开放系统互联（Open Systems Interconnection，OSI）的层式架构来建模的，正如由ISO/IEC 7498-1所定义的那样。

在网络层，Linux当然支持IPv4和IPv6协议，但是也支持

AppleTalk、IPX、X.25、帧中继（Frame Relay）和其他的协议。传输层协议包括TCP、UDP、SPX等。

套接字层（socket layer）提供了应用程序和在内核中的网络协议栈之间的抽象。套接字是通信端点，它是由它的域（domain）和类型（type）来定义的。域指示例如IPv4（AF_INET）或者IPv6（AF_INET6）的协议家族。类型指示通信语义（semantics），例如基于连接的双向字节流（SOCK_STREAM）或者原始协议访问（SOCK_RAW）。

网络协议栈实现可以在内核源树的linux/net中找到。

进程间通信

进程间通信（Interprocess Communication）是一套用于在进程或者线程间进行数据交换的方法。进程间通信方法通常被划分成消息传递（message passing）、共享内存（shared memory）、同步（synchronization）和数据流（data stream）。

Linux内核进程间通信方法的实现可以在linux/ipc找到。这些函数创建用于System V和可移植操作系统接口（Portable Operating System Interface，POSIX）进程间通信机制的高层抽象的核心。

系统调用接口

系统调用接口（System Call Interface，SCI）是Linux内核和运行在用户空间的应用之间的连接。通过系统调用接口，内核提供一套通用的用于进程管理、文件管理、设备管理、进程间通信和系统管理的函数调用的应用程序编程接口。

Linux内核的系统调用接口由超过300个函数组成。精确的数字依赖

于架构。大部分函数是通用的，虽然它们的实现可能是架构依赖的。一些函数可能是架构特定的并且仅仅被特定的架构所支持。系统调用接口实现与架构依赖部分可以在linux/arch的linux/kernel子目录中找到。

每个系统调用代表已定义的从用户空间进入内核的进入点。系统调用总是构成从用户模式到内核或者特权模式的CPU上下文切换。在旧的x86 CPU上，通过使用int 80h软件中断或者陷阱来调用此上下文切换。新的x86 CPU提供sysenter指令，它是比陷阱更加高效的。每个系统调用都有通过其可以被识别的唯一号码。这个号码也被称为系统调用槽（system call slot）。这个数字是从其系统调用接口通过单一应用程序编程接口来多路复用（multiplexes）和去多路复用（de-multiplexes）系统调用的关键。你可以在文件/usr/include/asm/unistd.h中找到属于特定系统调用的数字。依赖于架构，这个文件可能包含其他含有实际系统调用列表的其他文件。

如果手册页被安装在系统上，那么你可以使用man syscalls或者info syscalls来获取关于系统调用的更多信息。

用strace工具，你可以追踪系统调用和信号。例如，

```
$ strace ls
```

以它们的调用顺序显示被ls命令用以列出当前目录中的条目的系统调用。strace源也是用于针对不同架构的系统调用的好资源。你可以在Source-Forge的http://sourceforge.net/projects/strace找到项目的源。在目录linux下面的代码树中，有用于各种架构的子目录。每个这些子目录都包含一个名为syscallent.h的文件，它以系统调用的槽的顺序列出所有被那个架构支持的系统调用。

对每个架构来说，Linux内核源维护系统调用表。对于x86架构，你

可以在arch/x86/syscalls/syscall_32.tbl（用于32位）和arch/x86/syscalls/syscall_64.tbl（用于64位）找到它们。

在Linux内核源树中没有所有系统调用都在那里实现的单一位置。然而，你可以通过从Linux内核源树内使用显示在程序清单6-1中的命令来以它的名字轻松地找到特定系统调用的实现。

程序清单6-1　exit系统调用的实现

```
yocto@yocto-dev:~/linux$ grep -rA3 'SYSCALL_DEFINE.\?(exit,' *
kernel/exit.c:SYSCALL_DEFINE1(exit, int, error_code)
kernel/exit.c-{
kernel/exit.c-  do_exit((error_code&0xff)<<8);
kernel/exit.c-}
```

例子使用exit系统调用。简单地在命令行中用不同的系统调用名字替换exit以找到用于那个系统调用的实现。这个例子也显示了使用SYSCALL_DEFINE宏的系统调用函数定义。依赖于系统调用函数期望的参数数量，在它的名字中具有参数数量的不同宏被使用。

调用系统调用函数需要存储所有CPU的寄存器，在特定寄存器中传递系统调用号码和它的参数，然后发出陷阱。精确地什么需要被完成以及如何完成，依赖于架构并且经常是写在汇编代码中的。通常，用户空间应用不直接调用系统调用（虽然它们可以），而是通过包含在C库中的封装函数。

[1] Linux内核的默认页大小最长时间以来一直是4k，因为它已经是在粒度和管理开销之间的好的折衷，因为Linux为每个页维护一个64字节（依赖于架构和调试选项）的管理结构。对于具有4GB内存，这意味着64MB；对16GB，它是256MB。具有大量内存的系统通常使用大的页大小来减少被用于页表的内存量。页大小作为#define PAGE_SIZE（1UL<<PAGE_SHIFT）而被定义在asm/page.h中，并且可以通过修改PAGE_SHIFT来被改变。

6.4.2 Linux内核启动

既然我们已经讨论了主要的内核子系统，那么让我们从高层看看Linux内核启动过程。为了这个讨论的上下文，理解控制是如何从引导加载程序传到内核然后最终传到用户空间应用程序（init进程）是足够的。事实上，Linux内核经过很多用于各种硬件组件和子系统的初始化阶段。很多这些阶段依赖于硬件平台。

在引导加载程序已经复制Linux内核镜像进入内存之后，它把控制传给是内核镜像的前新增部分的启动加载器（bootstrap loader）。为了节省空间，内核镜像通常是压缩的，并且为内核创建恰当的执行环境、解压内核、在内存中重定位内核、然后传递控制给内核是启动加载器的职责。启动加载器直接把控制传给在模块中的内核进入点，对大部分架构来说，它是head.o。

模块head.o包含用于特定CPU的架构特定但是平台独立的初始化代码。这个模块是从汇编语言文件head.S中衍生出来的，head.S位于linux/arch/<ARCH>/kernel目录内，其中<ARCH>被特定架构所替换。

从高层来看，head.o模块执行下面的任务：

·验证正确的架构和CPU

·检测CPU类型和功能性，例如硬件浮点型能力

·启用CPU的内存管理单元并且创建内存页的初始表

·建立基础的错误报告和处理

·切换到在main.c中的非架构特定的内核启动函数start_kernel()

在linux/init中的main.c包括大段的Linux内核启动代码，从架构设置、内核命令行处理、第一内核线程的初始化到挂载根文件系统和执行第一个用户空间应用程序。

在执行了基础套的内核初始化之后，start_kernel()函数调用rest_init()，它派生第一个内核线程。这个线程是通过将函数kernel_init()作为第一个参数调用kernel_thread()而派生的。这个函数称为init线程。此时，有两个线程正在执行：start_kernel()和kernel_init()。前者启动调度器然后在cpu_idle()函数中无限循环。后者成为init()线程，即是带有进程ID（PID）为1的所有用户空间进程的父进程。

在最后，kernel_init()启动第一个用户空间应用。如果init命令作为内核命令行的一部分而被传递，那么kernel_init()首先尝试启动那个程序。如果没有init命令被传递，那么函数会尝试一组它从根文件系统中加载的默认程序。它以这个顺序尝试/sbin/init、/etc/init、/bin/init和/bin/sh直到成功。如果没有一个成功，那么内核以著名的错误消息“No init found.Try passing init=option to the kernel.See Linux Documentation/init.txt for guidance.”退出。

通常，第一个用户空间进程是稍后启动其他用户进程的init或者启动系统的一部分。通常由Linux桌面和服务器所使用的init系统是System V Init、systemd和Upstart。嵌入式系统尝试使用例如BusyBox的更轻量级的启动系统或者直接启动它们的核心应用。

6.5 用户空间

既然内核已经完成了它的初始化，启动了init进程并在其内执行了第一个用户空间应用，那么系统已经进入了用户区或者用户空间。用户空间是所有运行在操作系统的内核之外并且包含所有库和应用程序的代码。用户空间提供了系统所需要的用来服务其预想目的的所有功能性。

用户空间的配置和它包括的库与应用程序在每个系统之间各不相同。然而，总有几乎所有系统都包括的库：C标准库（LIBC）。正如init进程是所有进程的父进程，LIBC可以被认为是所有库的父库。即使显示在程序清单6-2中的经典的Hello World，其应用程序也需要大量的逻辑以把2个字放在屏幕上。

程序清单6-2 Hello World

```
#include <stdio.h>
int main() {
    printf("Hello World!\n");
    return 0;
}
```

这个程序调用printf()，它是由LIBC应用程序编程接口提供的众多函数之一，减轻应用程序员执行更繁重的实现核心功能性的任务的负担。LIBC应用程序编程接口是由ANSI C标准所指定的。对于UNIX系统来说，ANSI C标准是作为POSIX库的一部分而被描述的，POSIX库是ANSI C库的超集。POSIX是IEEE标准。当前的版本是POSIX.1-2008或者IEEE Std 1003.2008。

很多LIBC应用程序编程接口直接映射到内核的系统调用。事实上，经常地，函数的实现仅仅是对系统调用的封装。

对于Linux系统来说，多个LIBC的实现是可用的。它们在库本身的

内存占用、与ANSI的C标准兼容性、性能、模块化和可配置性方面有变化。表6-2提供了常见实现的概览。

表6-2　用于Linux的C标准库

库	描述	Yocto项目支持	可配置	许可	主页
GLIBC	GNU 的 LIBC 实现	是	部分	LGPL	https://www.gnu.org/software/libc
EGLIBC	为内存占用和交叉编译所优化的嵌入式版本的 GLIBC（现在被合并进了 GLIBC）	是	是	LGPL	www.eglibc.org/home
uClibc	被创建来用于支持不需要内存管理的 Linux 系统	是	否	LGPL	www.uclibc.org

（续）

库	描述	Yocto项目支持	可配置	许可	主页
Bionic libc	用于 Android 系统的 LIBC；为小的内存占用而优化	否	否	3 条款 BSD	https://android.googlesource.com/platform/bionic/
dietlibc	具有交叉构建支持的小的内存占用库	否	部分	LGPL、商业	www.fefe.de/dietlibc
musl	轻量级的 LIBC，为小的内存占用和性能而优化	是	否	MIT	www.musl-libc.org
klibc	最初被设计用于 Linux 内核初始化的 LIBC 子集	否	否	GPL、BSD	ftp：//ftp.kernel.org/pub/linux/libs/klibc
Newlib	定位于用于几乎任何架构的嵌入式系统；一些库部分的聚集	否	是	多个自由软件许可	www.sourceware.org/newlib

EGLIBC的目标是轻量级并且可配置的、支持交叉编译、与GLIBC二进制兼容的LIBC。

二进制兼容性允许为GLIBC编译的应用程序运行在具有EGLIBC的系统上而不需要重新编译。其他LIBC实现通常仅仅提供应用程序编程接口的兼容性并且需要重新编译。

虽然最初是想被用于嵌入式系统，但是正在增加的数量的桌面和服务器Linux发行版已经使得EGLIBC成为它们的默认。因此，正在努力逐步停止不同的EGLIBC和GLIBC分支，赞成在GLIBC的庇护下的共同开

发工作。在写作本书的时候，EGLIBC已经被官方地合并进了GLIBC。我们在这里提到EGLIBC是出于历史原因。

6.6 总结

Linux操作系统栈是由很多不同组件所组成的。在本章中，我们仔细检查了它的架构以为关于OpenEmbedded构建系统如何构建组件和组装它们成为工作的Linux系统的讨论创建基础。

·引导加载程序在系统启动中扮演着短暂但是重要的角色。引导加载程序的职责是初始化硬件、加载与启动操作系统内核。

·Linux内核是大型和复杂项目。它被划分成各种提供内核功能性和把它从底层硬件抽象出来的子系统。

·内核的系统调用接口（System Call Interface，SCI）是内核和用户空间应用的桥梁。

·用户空间或者用户区是指所有运行在内核以外的代码。内核启动第一用户空间进程。

·LIBC为应用程序提供了一组常用的应用程序编程接口和函数。LIBC去除了对应用开发者来说处理系统错综复杂之处的需要，并且使得应用程序在不同系统之间变得可移植。

6.7 参考文献

内核文档，https://www.kernel.org/doc/Documentation。

第7章　构建定制Linux发行版

在前面的章节中，我们为使用Yocto项目工具来构建定制Linux发行版奠定了基础。现在到了我们把知识用在工作中的时间了。

第2章简要列出了针对构建系统的先决条件和如何设置构建主机、配置构建环境以及启动一个构建，该构建创建一个准备运行在QEMU模拟器中的系统。本章我们将重用那个构建环境。如果你还没有准备构建系统，那么我们推荐你回到第2章并遵循步骤完成。执行使用Poky默认设置的构建验证设置。其中会下载大部分源代码包并且建立共享的状态缓存，这两个都加速了本章提供的例子的构建时间。

在第3章和第4章中，我们解释了OpenEmbedded构建系统和BitBake语法。本章和接下来的章节显示了使用那个语法的BitBake菜谱的例子或者片段。虽然语法大部分是简单直接并且类似于典型的脚本语言，但是有一些特定于BitBake的构造。参考第4章，你会找到语法例子和解释。

当用Yocto项目实验时，你肯定会遇到构建失败。它们可能因为各种原因而发生，并且故障排除是具有挑战性的。你可以参考第5章以了解帮助你追踪构建失败的工具。

第6章概要描述了Linux发行版的组成部分。虽然引导加载程序和Linux内核对于工作的Linux操作系统栈来说是不可分割的，但是用户空间组成了它的大部分。在本章中，我们主要关注如何通过Yocto项目提供的菜谱和OpenEmbedded项目的其他兼容层来自定义Linux操作系统栈、用户空间库和应用程序。

7.1 核心镜像——Linux发行版蓝图

OpenEmbedded核心和其他Yocto项目层包括一些样例镜像。这些镜像提供用于典型Linux操作系统栈的根文件系统配置。它们包括从仅仅启动设备到命令行提示的非常基础的镜像到包含X窗口系统（X11）服务器和图形用户界面的镜像。这些参考镜像被称作核心镜像（core images），因为它们各自菜谱的名字都以core-image开头。你可以从你构建系统的安装目录内使用find命令定位用于核心镜像的菜谱（见程序清单7-1）。

程序清单7-1 核心镜像菜谱

```
user@buildhost:~/yocto/poky$ find ./meta*/recipes*/images -name "*.bb"
                                -print
./meta/recipes-core/images/core-image-minimal-initramfs.bb
./meta/recipes-core/images/core-image-minimal-mtdutils.bb
./meta/recipes-core/images/build-appliance-image_8.0.bb
./meta/recipes-core/images/core-image-minimal-dev.bb
./meta/recipes-core/images/core-image-minimal.bb
./meta/recipes-core/images/core-image-base.bb
./meta/recipes-extended/images/core-image-full-cmdline.bb
./meta/recipes-extended/images/core-image-testmaster-initramfs.bb
./meta/recipes-extended/images/core-image-lsb-sdk.bb
./meta/recipes-extended/images/core-image-lsb-dev.bb
./meta/recipes-extended/images/core-image-lsb.bb
./meta/recipes-extended/images/core-image-testmaster.bb
./meta/recipes-graphics/images/core-image-x11.bb
./meta/recipes-graphics/images/core-image-directfb.bb
./meta/recipes-graphics/images/core-image-weston.bb
./meta/recipes-graphics/images/core-image-clutter.bb
./meta/recipes-qt/images/qt4e-demo-image.bb
./meta/recipes-rt/images/core-image-rt-sdk.bb
./meta/recipes-rt/images/core-image-rt.bb
./meta/recipes-sato/images/core-image-sato-dev.bb
./meta/recipes-sato/images/core-image-sato-sdk.bb
./meta/recipes-sato/images/core-image-sato.bb
./meta-skeleton/recipes-multilib/images/core-image-multilib-example.bb
```

可以把核心镜像视作通过扩展它们可以从其中获得你自己的发行版的Linux发行版蓝图。所有核心镜像菜谱都继承自core-image类，core-image类自己从image类继承。所有镜像设置IMAGE_INSTALL变量来指定什么包被安装进根文件系统。IMAGE_INSTALL是包和包组

（package group）的列表。包组是包的集合。定义包组减少了潜在的在IMAGE_INSTALL变量中列出数百个单一包的需求。我们在本章接下来的部分会解释包组。镜像菜谱要么显式地设置Image_INSTALL，要么扩展它自己的由core-image类提供的默认值，由core-image类提供的默认值安装packagegroup-core-boot和packagegroup-base-extended这两个包组。默认创建启动到控制台的工作的根文件系统。

让我们更进一步地看看各种核心镜像。

·core-image-minimal：这是允许设备启动到Linux命令行登录的最基础的镜像。登录和命令行解释器是由BusyBox提供的。

·core-image-minimal-initramfs：这个镜像本质上和core-image-minimal一样，但是具有包括基于RAM的初始根文件系统（initramfs）的Linux内核。

·core-image-minimal-mtdutils：基于core-image-minimal，这个镜像也包含用户空间工具来与在Linux内核中的内存技术设备（Memory Technology Device，MTD）子系统交互以在闪存设备上执行操作。

·core-image-minimal-dev：基于core-image-minimal，这个镜像也包含用于安装在根文件系统中的所有包的全部开发包（例如头文件）。如果和原生目标工具链一起部署，那么它允许在目标上的软件开发。与交叉工具链一起，它可以用于开发主机上的软件开发。

·core-image-rt：基于core-image-minimal，这个镜像目标构建Yocto项目实时内核（real-time kernel）并包含用于实时应用的测试套件与工具。

·core-image-rt-sdk：除了core-image-rt之外，这个镜像包含由所有已安装包的开发包所组成的系统开发工具包（System Development Kit，

SDK）；开发工具例如编译器、汇编程序（assembler）和链接器，以及性能测试工具和Linux内核开发包。这个镜像允许在目标上的软件开发。

·core-image-base：本质上是core-image-minimal，这个镜像也包含中间件和应用包以支持大量如WiFi、蓝牙、声卡和串行口的硬件。目标设备必须包含必要的硬件组件并且Linux内核必须为它们提供支持。

·core-image-full-cmdline：这个最小化的镜像添加典型的Linux命令行工具（bash、acl、attr、grep、sed和tar等）到根文件系统。

·core-image-lsb：这个镜像包含与Linux标准基（Linux Standard Base，LSB）规格说明保持一致性所需的包。

·core-image-lsb-dev：这个镜像和core-image-lsb相同，但是也包含安装在根文件系统上的所有包的开发包。

·core-image-lsb-sdk：除了core-image-lsb-dev之外，这个镜像包含例如编译器、汇编程序和链接器，以及性能测试工具和Linux内核开发包的开发工具。

·core-image-x11：这个基础的图形镜像包含X11服务器和X11终端应用。

·core-image-sato：这个镜像提供包含用于移动设备的OpenedHand Sato用户体验的X11支持。除了Sato屏幕管理器之外，镜像也提供一些使用Sato方案的应用，例如终端、编辑器、文件管理器和一些游戏。

·core-image-sato-dev：这个镜像和core-image-sato相同，但是也包含安装在根文件系统上的所有包的开发包。

·core-image-sato-sdk：除了core-image-sato-dev之外，这个镜像包含例如编译器、汇编程序和链接器，以及性能测试工具和Linux内核开发包的开发工具。

·core-image-directfb：将DirectFB用于图形化和输入设备管理的镜像，DirectFB可能包含图形化加速和窗口系统。因为相比于X11内存占用非常小，DirectFB是用于需要图形化支持但不需要完全的X11功能性的低端嵌入式系统的较佳选择。

·core-image-clutter：指示基于X11且包含Clutter工具包的镜像。Clutter基于OpenGL并且提供用于动画图形用户界面的功能性。

·core-image-weston：这个镜像使用Weston替代X11。Weston是使用Wayland协议和实现来与它的客户端交换数据的合成器。这个镜像也包含拥有Wayland能力的终端程序。

·qt4e-demo-image：这个镜像在完成启动过程后启动用于嵌入式Qt工具包的演示应用。用于嵌入式Linux的Qt提供直接写到框架缓冲区的图形化应用的开发框架来替代使用X11。

·core-image-multilib-example：这个镜像是支持多个库的例子，通常是在其他64位系统上对32位的支持。镜像基于核心镜像并且增加希望的多库包到IMAGE_INSTALL。

下面的3个镜像不是用于嵌入式Linux系统的参考镜像。我们为了完整性的目的而把它们包括在这个讨论中。

·core-image-testmaster、core-image-testmaster-initramfs：这些镜像是用于在真实硬件设备或者在QEMU中测试其他镜像的参考。它们被部署到要启动进入的独立分区然后使用脚本来部署要测试的镜像。这个方式对于自动化测试比较有用。

·build-appliance-image：这个菜谱创建包含Yocto项目构建系统所需要的所有东西的Yocto项目构建器具虚拟机镜像。这个镜像可以通过使用VMware Player或者VMware Workstation来启动。

学习参考镜像菜谱是学习如何构建这些镜像以及什么包组成了它们的很好的途径。核心镜像也是用于你自己的Linux操作系统栈的好起点。你可以简单地通过添加包和包组到IMAGE_INSTALL来轻松扩展它们。镜像仅仅可以被扩展，不能被收缩。为了构建具有更少功能性的镜像，你必须从更小的核心镜像开始并且仅仅添加你需要的包。没有简单的方法来去除包。它们中的大部分是通过包组来添加的，并且如果你不想安装被包组包含的包，那么你将需要拆分包组。当然，如果你正在去除包，那么也必须去除任何依赖于它的其他包。

有很多你可以添加将被包含在根文件系统中的包和包组的方法。接下来的部分将解释它们并提供为什么你希望使用某个方法而不是其他方法。

7.1.1 通过本地配置来扩展核心镜像

最简单的用于增加包和包组到镜像的方法是增加IMAGE_INSTALL到你构建环境的conf/local.conf文件中：

```
IMAGE_INSTALL_append = " <package> <package group>"
```

正如我们已经看到的，镜像菜谱设置增加包和包组的IMAGE_INSTALL变量。为了扩展镜像，你必须追加包和包组到这个变量中。你可能疑惑为什么我们使用显式的_append操作符替代+=或者.+操作符。在所有菜谱和配置文件已经被处理后，使用_append操作符无条件地追加指定的值到IMAGE_INSTALL变量。镜像菜谱通常使用=或者?=操作符来显式地设置IMAGE_INSTALL变量，这可能发生在BitBake处理了in conf/local.conf中的设置之后。

例如，增加

```
IMAGE_INSTALL_append = " strace sudo sqlite3"
```

会在根文件系统中安装strace和sudo工具以及SQLite。当使用_append操作符时，你必须要记住在第一个包或者包组的前面增加空格，因为这个操作符不会自动包含空格。

使用在构建环境的conf/local.conf中的IMAGE_INSTALL会无条件地影响你将要用这个构建环境所构建的所有镜像。如果你希望仅仅安装额外的包到特定镜像，可以使用条件追加：

```
IMAGE_INSTALL_append_pn-<image> = " <package> <package group>"
```

这仅仅向image的根文件系统安装指定的包和包组。例如，

```
IMAGE_INSTALL_append_pn-core-image-minimal = " strace"
```

仅仅向core-image-minimal的根文件系统安装strace工具。所有其他镜像是不受影响的。

使用IMAGE_INSTALL也会影响核心镜像，即从core-image类继承的镜像，以及直接从image类继承的镜像。为了便利性的目的，core-image类定义变量CORE_IMAGE_EXTRA_INSTALL。所有增加到这个变量的包和包组都被类追加到IMAGE_INSTALL。

使用

```
CORE_IMAGE_EXTRA_INSTALL = "strace sudo sqlite3"
```

增加这些包到所有继承自core-image的镜像。直接继承自image的镜像不受影响。对于核心镜像来说，使用CORE_IMAGE_EXTRA_INSTALL是比直接追加到IMAGE_INSTALL更加安全和简单的方法。

7.1.2 用QEMU测试镜像

使用QEMU模拟器可以很容易地测试镜像。虽然你最终是构建用于产品的目标硬件的系统，但由于下面的原因，使用QEMU进行测试有很好的意义。

·用于启动QEMU的往返开发时间远远快于部署镜像到实际硬件中。

·通常，当软件开发开始的时候，硬件还不可用。

·Yocto项目板支持包使得它可以简单地从QEMU切换到硬件，然后又切换回QEMU。

在第2章中，当执行我们的第一个构建时，我们使用QEMU来验证构建输出。Poky参考发行版给出了通过提供必要参数而极大简化了启动QEUM任务的脚本runqemu。在它最简化的形式下，用单一参数来启动脚本：

```
$ runqemu qemux86
```

它告诉脚本为提供的QEMU机器定位最新内核和根文件系统镜像构建，否则以默认参数启动QEMU。参数值与conf/local.conf中的QEMU机器类型相匹配。

当用不同的根文件系统镜像来工作时，你可能希望在运行QEMU之际选择特定镜像。例如，你已经使用前面的命令行构建了core-image-minimal和core-image-base，因为runqemu启动了你最后构建的任何镜像。使用如下的命令可选择镜像：

```
$ runqemu qemux86 core-image-minimal
```

脚本自动选择正确的内核并使用最新的core-image-minimal根文件系统。为了更多的控制，你可以直接指定内核镜像和根文件系统镜像文件：

```
$ runqemu <path>/bzImage-qemux86.bin <path>/core-image-minimal-qemux86.ext3
```

QEMU和runqemu脚本是用于快速往返应用开发的便利工具，我们将在第11章中探索它。

7.1.3 使用构建历史验证和比较镜像

当构建产品时，你会发现自己在频繁地修改镜像、增加新包以及去除不相关的包以剪裁内存占用。使你能够轻松验证和互相比较镜像构建的工具可以简化那个相对繁重的任务。

为了帮助维护构建输出质量并实现不同构建之间的比较，BitBake提供构建历史，由buildhistory类实现。这个类在Git仓库中记录关于被构建的所有包的内容以及关于被构建系统所创建的镜像的信息，你可以从Git仓库中检查它们。构建历史默认情况下是被禁用的。为了启用它，你需要增加

```
INHERIT += "buildhistory"
BUILDHISTORY_COMMIT = "1"
```

到你构建环境的conf/local.conf文件中。请注意，INHERIT是你必须增加buildhistory类的变量。它不同于菜谱和类为从类中继承功能性所使用的inherit指令。每次你做构建，buildhistory都会创建带有变化的提交到Git仓库。

buildhistory Git仓库存储在由BUILDHISTORY_DIR变量定义的目录中。这个变量的默认值被设置成：

```
BUILDHISTORY_DIR ?= "${TOPDIR}/buildhistory"
```

在启用buildhistory和执行构建后，你将看见被增加到构建环境的顶级目录中的buildhistory目录。目录包含两个子目录：images和packages。前者包含关于你构建的镜像的构建信息，后者含有关于包的信息。我们将在第13章中分析buildhistory Git仓库。这里我们仅仅看看

images子目录。在images子目录中，镜像按照目标机器、目标C库和镜像名字而被排序到进一步的子目录中：

```
${TOPDIR}/buildhistory/images/<machine>/<clib>/<image>
```

对于我们使用默认EGLIBC目标库用于qemux86的core-image-minimal构建，你将在下面的目录中找到镜像历史：

```
${TOPDIR}/buildhistory/images/qemux86/eglibc/core-image-mininal
```

那个目录中的文件给出了关于什么组成镜像的细节信息。

·image-info.txt：以最重要的变量（例如DISTRO、DISTRO_VERSION和IMAGE_INSTALL）的形式表现的关于镜像的概览信息。

·installed-packages.txt：安装在镜像中的包文件的列表，包括版本和目标信息。

·installed-package-names.txt：和前面的文件类似，但是仅仅包括包的名字而没有版本和目标信息。

·files-in-image.txt：带有目录名字、文件大小、文件权限和文件属主的根文件系统列表。

简单地搜索文件installed-package-names.txt会给你关于包是否已经被安装的信息。

7.1.4 用菜谱扩展核心镜像

增加包和包组到CORE_IMAGE_EXTRA_INSTALL和IMAGE_INSTALL以及conf/local.conf中可能是非常简单和快速的，但是这样做会使项目难以维护并且使重用复杂化。更好的方法是通过菜谱扩展预定义的镜像。程序清单7-2显示扩展core-image-base的菜谱。

程序清单7-2 扩展core-image-base的菜谱

```
DESCRIPTION = "A console image with hardware support for our IoT device"

require recipes-core/images/core-image-base.bb

IMAGE_INSTALL += "sqlite3 mtd-utils coreutils"
IMAGE_FEATURES = "dev-pkgs"
```

例子包括用于core-image-base的菜谱并且增加包到IMAGE_INSTALL，增加镜像特性到IMAGE_FEATURES。我们在接下来的部分解释镜像特性是什么以及如何使用它们来定制镜像。

当用菜谱扩展镜像时需要考虑的一些事情是：

·与类不同，你需要提供相对于BitBake的层的路径以找到菜谱文件来包含，并且需要添加.bb文件后缀。

·当你使用include或者require来包含你正在扩展的菜谱时，我们推荐使用require，因为如果BitBake无法定位到被包含的菜谱文件，那么它将导致BitBake以显式的错误消息退出。

·记住使用+=操作符来添加到IMAGE_INSTALL中。不使用=或者:=，因为它们覆盖由被包含文件所定义的变量的内容。

为了让BitBake能够实际使用这个菜谱作为构建目标，你必须添加

它到通过conf/bblayers.conf文件包而被包含进构建环境的层中。不建议添加菜谱到核心Yocto项目层，例如meta、meta-yocto以及其他等，因为它会使你在升级到新的Yocto项目版本时难以维护构建环境。替代的方式是，创建在其内放置菜谱的层。

为菜谱创建层看起来需要大量开销，但是几乎没有项目一直保持很小的状态。以菜谱开始的项目最终增长成复杂的、具有用于镜像、包和包组的菜谱的项目。在第3章中，我们介绍了yocto-layer，它使得创建层成为轻松的事。

7.1.5 镜像特性

镜像特性提供特定的、你可以添加到目标镜像的功能性。其可以是额外要被安装的包、配置文件的修改等。

例如，dev-pkgs镜像特性增加开发包，它通常包括安装在根文件系统中的所有包开发所需要的头文件以及其他文件。使用这个特性是一种便利的方法，以此来启用用于开发的目标镜像并且不需要显式地在IMAGE_INSTALL变量中指定开发包。为了部署，你可以简单地去除dev-pkgs镜像特性。

镜像特性的安装是由IMAGE_FEATURES和EXTRA_IMAGE_FEATURES这两个变量控制的。前者被用到镜像菜谱中以定义需要的镜像特性集。后者通常被用在conf/local.conf文件中以定义额外的镜像特性，当然该特性会影响那个构建环境所构建的所有镜像。EXTRA_IMAGE_FEATURES的内容由meta/conf/bitbake.conf配置文件简单地添加到IMAGE_FEATURES中。

镜像特性是由不同的类所定义的。当前可用的镜像特性列表如下。

·由image.bbclass定义：

·debug-tweaks：准备用于开发目的的镜像。特别地，它为控制台和安全Shell登录设置空root密码。

·package-management：按照由PACKAGE_CLASSES定义的包管理类为根文件系统安装包管理系统。

·read-only-rootfs：创建只读的根文件系统。这个镜像特性仅仅在System V Init（SysVinit）系统（而不是sytemd）被使用时才工作。

·splash：在启动中显示启动屏幕而不是启动消息。默认情况下，启动屏幕是由psplash包所提供的，它可以被定制。你也可以通过设置SPLASH变量为不同的包名来定义替代的启动屏幕包。

·由populate_sdk_base.bbclass定义：

·dbg-pkgs：为安装在根文件系统中的所有包安装包含符号的调试包。

·dev-pgks：为安装在根文件系统中的所有包安装包含头文件和其他开发文件的开发包。

·doc-pkgs：为安装在根文件系统中的所有包安装文档包。

·staticdev-pkgs：为安装在根文件系统中的所有包安装静态开发包（例如以*.a结尾的静态库文件）。

·ptest-pkgs：为安装在根文件系统中的所有包安装测试（ptest）包。

·由core-image.bbclass定义：

·eclipse-debug：安装用于和Eclipse集成开发环境集成的远程调试工具，也就是GDB调试服务器、Eclipse目标通信框架（Target Communication Framework，TCF）代理和OpenSSH SFTP服务器。

·hwcodecs：如果硬件平台提供硬件解码器和编码器，则为音频、图片和视频安装它们。

·nfs-server：安装网络文件系统（Network File System，NFS）服务器、实用程序和客户端。

·qt4-pkgs：安装Qt4框架和演示应用。

·ssh-server-dropbear：安装轻量级的SSH服务器Dropbear，它常用于嵌入式系统。这个镜像特性是与ssh-server-openssh不兼容的。两者中任一者（但不是两者同时）可以被安装。

·ssh-server-openssh：安装OpenSSH服务器。这个镜像特性是与ssh-server-dropbear不兼容的。两者中任一者（但不是两者同时）可以被安装。

·tools-debug：安装调试工具，也就是GDB调试器、GDB远程调试服务器、系统调用追踪工具以及当GLIBC库为目标库时用于GLIBC库的内存追踪工具mtrace。

·tools-profile：安装常见分析工具，例如oprofile、powertop、latencytop、lttng-ust和valgrind。

·tools-sdk：安装软件开发工具，例如GCC编译器、Make、autoconf、automake、libtool等。

·tools-testapps：安装测试应用程序进行测试，例如测试X11和中间件包（如电话管理器oFono与连接管理器ConnMan）。

·x11：安装X11服务器。

·x11-base：安装带有窗口系统的X11服务器。

·x11-sato：安装用于移动设备的OpenedHand Sato用户体验。

当创建你自己的镜像菜谱和选择要继承的镜像类时，什么类定义镜像特性是很重要的。类image继承自populate_sdk_base，因此这两个类定义的所有镜像特性都是对继承自image的镜像可用的。由core-image定义

的镜像特性仅仅对继承那个类的镜像可用，core-image依次继承自image和populate_sdk_base。

7.1.6 包组

在本次创建定制Linux发行版镜像的讨论中，我们已经数次提到了包组（package group）。包组是被名字引用的包束。在IMAGE_INSTALL变量中使用那个名字安装那个包组定义的所有包到构建镜像的根文件系统中。

Yocto项目和OpenEmbedded核心层定义通用的你可以直接用于镜像的包组的集合。你也可以创建你自己的、包含来自任何层的包的包组，包括你自己的层。我们首先描述由Yocto项目和OpenEmbedded核心层所定义的包组，然后研究关于包组是如何被定义的细节。

预定义的包组

包组是由菜谱定义的。按照惯例，菜谱文件以packagegroup-开头，并且放在各自菜谱类别的packagegroup子目录中。例如，你可以在子目录meta/recipes-qt/packagegroups中找到与Qt开发框架相关的包组菜谱。

从Yocto项目构建系统的安装目录中使用

```
find . -name "packagegroup-*" -print
```

会给你用于Yocto项目构建系统预定义的包组的所有包组菜谱列表。

以下是最常见的预定义的包组：

·packagegroup-core-ssh-dropbear：提供因为其相对于OpenSSH服务器较小的内存占用而常用于嵌入式系统的Dropbear SSH服务器的包。这个包组与packagegroup-core-ssh-openssh冲突。在镜像中只可以包含这两

者之一。ssh-server-dropbear镜像特性安装这个包组。

·packagegroup-core-ssh-openssh：提供用于标准OpenSSH服务器的包。这个包组和packagegroup-core-ssh-dropbear冲突。在镜像中只可以包含这两者之一。ssh-server-openssh镜像特性安装这个包组。

·packagegroup-core-buildessential：提供最基本的开发工具，也就是，GNU Autotools实用程序autoconf、automake和libtool；包括链接器ld、汇编程序as和其他工具的GNU二进制工具集binutils；编译器合集cpp；gcc；g++；GNU国际化和本地化工具gettext；make；带开发包的libstc++；pkgconfig。

·packagegroup-core-tools-debug：提供最基本的调试工具，也就是，GDB调试器、GDB远程调试服务器、系统调用追踪工具strace以及针对GLIBC目标库的内存追踪工具mtrace。

·packagegroup-core-sdk：这个包组把packagegroup-core-buildessential包组和额外的用于开发的工具（例如具有shell、文件和文本操作实用程序的GNU核心实用程序；动态链接器ldd等）结合起来。这个包与packagegroup-core-standalone-sdk-target一起构成tools-sdk镜像特性。

·packagegroup-core-standalone-sdk-target：提供GCC和标准C++库。这个包与packagegroup-core-sdk一起构成tools-sdk镜像特性。

·packagegroup-core-eclipse-debug：提供GDB调试服务器、Eclipse目标通信框架代理和OpenSSH SFTP服务器用于远程部署和调试而与Eclipse集成开发环境的集成。

·packagegroup-core-tools-testapps：提供测试应用程序进行测试，例如测试X11和中间件包（如电话管理器oFono与连接管理器

ConnMan）。tools-testapps镜像特性安装这个包组。

·packagegroup-self-hosted：提供用于自托管的构建系统的所有必要包。build-appliance镜像目标使用这个包组。

·packagegroup-core-boot：提供用于创建带有控制台的可启动镜像所必要的包的最小集合。所有core-image目标安装这个包组。core-image-minimal仅仅安装这个包组和安装后脚本。

·packagegroup-core-nfs：提供网络文件系统服务器、实用程序和客户端。nfs-server镜像特性安装这个包组。

·packagegroup-base：这个菜谱提供互相依赖以及依赖于机器和发行版配置的多个包组。这些包组的目的是向依赖于机器和发行版配置的镜像添加硬件、联网协议、USB、文件系统和其他支持。两个顶级包组是packagegroup-base和packagegroup-base-extended。前者仅仅当机器配置和发行版配置都需要它们时才添加对蓝牙、WiFi、3G和NFC的硬件支持。后者在发行版配置需要它们时也添加用于那些技术的配置。然而，机器配置不直接支持它们，而是提供对PCI、PCMCIA或者USB主机的支持。这个包组允许你创建带有可以被物理地添加到目标设备的设备支持的镜像，例如通过USB热插拔。最常见的是，提供硬件支持的镜像为了动态硬件支持而使用packagegroup-base-extended却不是packagegroup-base，例如core-image-base。

·packagegroup-cross-canadian：使用Canadian Cross技术为创建工具链提供软件开发工具包，它在系统A上创建一条执行在系统B上的工具链以便创建用于系统C的二进制。关于这个包组的用例是，在构建系统上用Yocto项目创建运行在镜像目标上但为具有不同于镜像目标的架构的第三方系统产生输出的工具链。

·packagegroup-core-tools-profile：提供例如oProfile、PowerTOP、LatencyTOP、LTTng-UST和Valgrind的常见调试工具。tools-profile镜像特性使用这个包组。

·packagegroup-core-device-devel：提供用于镜像的distcc支持。distcc允许在网络上跨数台机器分发编译。distcc必须安装、配置并且运行于构建主机上。在目标上，你必须定义交叉编译器变量来使用distcc替代本地编译器（例如export CC="distcc"）。

·packagegroup-qt-toolchain-target：提供用于在目标系统上构建针对基于X11版本的Qt开发工具包的应用的包。

·packagegroup-qte-toolchain-target：提供用于在目标系统上构建针对嵌入式版本的Qt开发工具包的应用的包。

·packagegroup-core-qt：为使用基于X11版本的Qt开发工具包的目标系统提供所有必要包。

·packagegroup-core-qt4e：为使用嵌入式Qt开发工具包的目标系统提供所有必要包。qt4e-demo-image安装这个包组。

·packagegroup-core-x11-xserver：仅仅提供X.Org X11服务器。

·packagegroup-core-x11：提供packagegroup-core-x11-xserver加上例如xhost、xauth、xset、xrandr的基础实用程序，以及启动时初始化。x11镜像特性安装这个包组。

·packagegroup-core-x11-base：提供packagegroup-core-x11加上用于工作的、包括Matchbox窗口管理器、Matchbox终端和字体包的X11环境的中间件与应用客户端。x11-base镜像特性安装这个包组。

·packagegroup-core-x11-sato：提供用于移动设备的OpenedHand Sato用户体验，它包括Matchbox窗口管理器、Matchbox桌面和大量应用。x11-sato镜像特性安装这个包组。为了将这个包组用于目标镜像，必须安装packagegroup-core-x11-base。

·packagegroup-core-clutter-core：提供用于Clutter图形化工具包的包。为了将工具包用于目标镜像，必须安装packagegroup-core-x11-base。

·packagegroup-core-directfb：提供用于无X11的DirectFB支持的包。这个包组包括directfb和directfb-example包，并且如果由机器配置所提供，那么它将增加触摸屏支持。

·packagegroup-core-lsb：提供所有用于Linux标准基支持的包。

·packagegroup-core-full-cmdline：通过安装完整的命令行实用程序而不是更加紧凑的BusyBox变体来为更传统的Linux系统提供包。

当解释不同的包组时，我们有点不太严格地使用术语提供（provide）和安装（install），因为包组菜谱实际上不提供或者安装任何包。它们仅仅创建会导致构建系统处理各个包菜谱的依赖性，正如我们将在下面的部分中看到的。

包组中的一些是由镜像特性所使用的，它引发了疑问：使用镜像特性还是使用镜像特性使用的包组？

包组菜谱

包组是由继承自packagegroup类的菜谱所定义的。包组菜谱不同于典型包菜谱，因为它们不构建任何东西或者创建任何输出。包组菜谱仅仅创建触发构建系统处理包组引用的包的菜谱的依赖性。

程序清单7-3显示了典型的包组菜谱。

程序清单7-3　包组菜谱

```
SUMMARY = "Custom package group for our IoT devices"
DESCRIPTION = "This package group adds standard functionality required by \
               our IoT devices."

LICENSE = "MIT"

inherit packagegroup
PACKAGES = "\
   packagegroup-databases \
   packagegroup-python \
   packagegroup-servers"

RDEPENDS_packagegroup-databases = "\
   db \
   sqlite3"

RDEPENDS_packagegroup-python = "\
   python \
   python-sqlite3"

RDEPENDS_packagegroup-servers = "\
   openssh \
   openssh-sftp-server"
RRECOMMENDS_packagegroup-python = "\
   ncurses \
   readline \
   zip"
```

包组菜谱的名字，虽然不是由构建系统所强制或者要求的，但也应该遵守惯例packagegroup-<name>.bb。你也希望把它们放在包组集成的菜谱类别的packagegroup子目录中。如果包组横跨菜谱并且可能来自多个类别的包组，那么把它们放进recipes-core类别是好的实践。

包组菜谱的基础结构相当简单。正如任何菜谱（我们在第8章中深入编写菜谱的细节）应该提供关于菜谱做什么的SUMMARY（摘要）一样，包组菜谱也应该提供它。DESCRIPTION（描述）可以提供更长且更加细节的解释），虽然它是可选的，但包括它却是好的实践。任何菜谱还需要为菜谱自己提供LICENSE（许可）。所有包组菜谱都必须继承packagegroup类。

实际包组的名字是由PACKAGES变量定义的。这个变量包含空格分隔的包组名字的列表。在程序清单7-3的案例中，它们是packagegroup-databases、packagegroup-python和packagegroup-servers。

按照惯例，包组名字以packagegroup-开头。虽然构建系统不要求这么做，但是如果你坚持将它用于你自己的包组名字，那么这会是好的实践。

对于每个包组，菜谱必须在条件变量RDEPENDS_<package-group-name>中定义它的依赖性。这些变量列出需要的依赖性，依赖性可以是包或者包组。

RRECOMMENDS_<package-group-name>定义是可选的。正如我们在第3章中所看到的，推荐是导致包仅仅在其已经被构建后才被包含的弱依赖。

你可以从例如IMAGE_INSTALL的其他变量中引用包组，当然，这会导致这些包组被安装在目标镜像中。你也可以使用它们来为层级结构的其他包组创建依赖性。你必须避免包组的循环依赖（circular dependencies）。这可能听起来很简单，很直接，但是也很容易在相当复杂的环境中由于错误而发生。然而，BitBake在循环包组依赖的情况下以错误消息终止。

包组菜谱也可以作为BitBake构建目标而被直接使用。例如，如果包组菜谱的名字是packagegroup-core-iot.bb，那么你可以使用如下命令来构建由菜谱定义的包组的所有包：

```
$ bitbake packagegroup-core-iot
```

这样做允许在通过镜像构建引用包组之前测试它们，这样可以简化调试。

7.2 从头构建镜像

7.1节详细说明了Yocto项目核心镜像以及如何通过设置在conf/local.conf和扩展预定义镜像菜谱的菜谱中的IMAGE_INSTALL、CORE_IMAGE_EXTRA_INSTALL、IMAGE_FEATURES和EXTRA_IMAGE_FEATURES来扩展它们。最终，你可能希望从头创建定制Linux发行版镜像而不依靠参考镜像。

定制镜像菜谱必须继承image或者core-image类。后者本质上是前者的扩展并且定义了额外的镜像特性，正如在7.1.5小节中所描述的。选择哪一个用于定制镜像菜谱取决于需求。然而，继承core-image通常是好建议，因为镜像特性是可用的，但是仅仅当其被显式地请求时才被安装。

程序清单7-4显示了创建可启动控制台镜像的最简单的镜像菜谱。

程序清单7-4　基础镜像菜谱

```
SUMMARY = "Custom image recipe that does not get any simpler"
DESCRIPTION = "Well yes, you could remove SUMMARY, DESCRIPTION, LICENSE."

LICENSE = "MIT"

inherit core-image
```

菜谱创建带有要启动的核心包和对目标设备的硬件支持的镜像，因为core-image类默认添加packagegroup-core-boot和packagegroup-base-extended这两个包组到IMAGE_INSTALL。也由类添加到IMAGE_INSTALL的是变量CORE_IMAGE_EXTRA_INSTALL，它允许通过conf/local.conf进行简单镜像修改，如前面所描述的。

带有package-group-core-boot和package-base-extended的基础镜像提

供了一个好起点，该起点可以简单地通过向IMAGE_INSTALL和IMAGE_FEATURES添加来扩展，如程序清单7-5中所示。

程序清单7-5　向基础镜像添加

```
SUMMARY = "Custom image recipe adding packages and features"
DESCRIPTION = "Append to IMAGE_INSTALL and IMAGE_FEATURES for \
               further customization. "

LICENSE = "MIT"

# We are using the append operator (+=) below to preserve the default
# values set by the core-image class we are inheriting.
IMAGE_INSTALL += "mtd-utils"
IMAGE_FEATURES += "splash"

inherit core-image
```

在镜像菜谱内，使用+=操作符直接追加到IMAGE_INSTALL和IMAGE_FEATURES。不要在镜像菜谱中使用EXTRA_IMAGE_FEATURES或者CORE_IMAGE_EXTRA_INSTALL。这些变量被保留以用于conf/local.conf中，在conf/local.conf中，它们被直接赋值并且覆盖任何由镜像菜谱所赋的值。

不依赖用于IMAGE_INSTALL和IMAGE_FEATURES的默认值的镜像菜谱很简单，如程序清单7-6所示。

程序清单7-6　从头开始的核心镜像

```
SUMMARY = "Custom image recipe from scratch"
DESCRIPTION = "Directly assign IMAGE_INSTALL and IMAGE_FEATURES for \
               for direct control over image contents."

LICENSE = "MIT"

# We are using the assignment operator (=) below to purposely overwrite
# the default from the core-image class.
IMAGE_INSTALL = "packagegroup-core-boot packagegroup-base-extended \
                 ${CORE_IMAGE_EXTRA_INSTALL} mtd-utils"
IMAGE_FEATURES = "${EXTRA_IMAGE_FEATURES} splash"

inherit core-image
```

第一眼看来，程序清单7-5和程序清单7-6的镜像菜谱是非常相似的。事实上，这两个菜谱产生完全相同的镜像。区别很细小但是很重要。程序清单7-5对IMAGE_INSTALL和IMAGE_FEATURES使用操作符+=来利用由core-image类所提供的默认值。程序清单7-6使用赋值操作符=来有目的地覆盖默认值。

覆盖默认值给了你对镜像内容的最大控制，但是你也必须自己处理基础。对任何镜像来说，你最有可能总是希望包含packagegroup-core-boot来获得可启动镜像。你是否希望packagegroup-base-extended提供的硬件支持依赖于需求。同样由你自由处置的是CORE_IMAGE_EXTRA_INSTALL：如果你不显式地添加它到IMAGE_FEATURES，那么你将不能在conf/local.conf中使用这个变量来本地定制目标镜像，但是对于受控的用于生产的构建环境，这么做可能是有意义的。

IMAGE_FEATURES和EXTRA_IMAGE_FEATURES也是一样的情况。如果你把赋值操作符用于IMAGE_FEATURES并且不有意添加EXTRA_IMAGE_FEATURES，那么它是不被包含的，这意味着debug-tweaks镜像特性不会被应用，并且你需要提供用于shell和SSH登录的密码。再一次，这对生产构建环境是有意义的，在其中，你不希望本地设置覆盖生产镜像的设置。

7.3 镜像选项

接下来的部分讨论影响Yocto项目构建系统如何创建根文件系统镜像的选项。

7.3.1 语言和区域

通过增加IMAGE_LINGUAS变量到镜像菜谱，针对不同领地的额外语言可以被简单地增加到根文件系统或者镜像中。使用

```
IMAGE_LINGUAS = "en-gb pt-br"
```

可增加用于英国英语和巴西葡萄牙语的特定语言包到镜像。然而，不是所有软件包都提供由语言和领地分隔的区域（设置）。它们中的一些仅仅按照语言提供区域（locale）文件。在这种情况下，构建系统默认成安装正确的语言本地文件而不管领地。

用于所有包的最小默认是en-us，它总是会被安装。另外，镜像类定义

```
IMAGE_LINGUAS ?= "de-de fr-fr en-gb"
```

当然，任何额外的区域包都会占用在根文件系统镜像中的额外空间。因此，如果设备不需要任何额外语言支持，在镜像菜谱中设置

```
IMAGE_LINGUAS = ""
```

是好的实践。

构建系统忽略不提供它们的包的语言。

7.3.2 包管理

构建系统可以使用4种不同打包格式来打包软件包，dpkg（Debian Package Management，Debian包管理）、opkg（Open Package Management，开放包管理）、RPM（Red Hat Package Manager，Red Hat包管理器）和tar。仅有前3种可以用来创建根文件系统。tar不提供必要的元数据包信息和数据库来记录什么版本的什么包已经被安装以及哪些包互相冲突等。

在构建环境的conf/local.conf中的变量PACKAGE_CLASSES控制什么包管理系统用于构建：

```
PACKAGE_CLASSES = "package_rpm package_ipk package_tar"
```

你可以声明多个打包类，但必须提供至少一个。构建系统为所有被指定的类创建包；然而，仅有在列表中的第一个打包类被用于创建发行版镜像的根文件系统。在列表中的第一个打包类一定不能是tar。

构建系统在tmp/deploy/<pms>中的各自目录里存储由包管理系统组织的包仓库，tmp/deploy/<pms>中的<pms>是各自包管理系统的名字。在那些目录中，包被进一步细分成通用（common）、架构（architecture）和机器依赖（machine-dependent）的包。

你应该为项目选择什么包管理系统？这依赖于项目的需求。这里是你可能希望顾及的一些考虑：

·opkg创建并使用比dpkg和RPM更少的包元数据，这使得构建更快并且包更加小。

·因为加强的包元数据，dpkg和RPM提供比opkg更好的依赖性处理

和版本管理。

·RPM包管理器是以Python编写的，并且需要Python被安装在目标上以便在系统运行时安装包。

默认情况下，构建系统不在目标系统上安装包管理器。如果你希望在嵌入式系统的运行时安装包，那么你必须使用它的镜像特性来增加包管理器：

```
IMAGE_FEATURES += "package_management"
```

构建系统根据PACKAGE_CLASSES的第一个条目自动安装正确的包管理器。

用于根文件系统的包管理系统最终是由变量IMAGE_PKGTYPE控制的。这个变量按照由PACKAGE_CLASSES定义的打包类的顺序被自动设置，在列表中的第一个打包类设置变量。我们建议你不直接设置这个变量。

7.3.3 镜像大小

根文件系统的最终大小依赖于多个因素并且是由构建系统使用在Python模块meta/lib/oe/image.py中的_get_rootfs_size()函数来计算的。计算会把根文件系统所要求的实际空间和以下的变量设置考虑进去。它也保证，最终的根文件系统镜像大小总是足够容纳整个镜像。因此，即使你将IMAGE_ROOTFS_SIZE设置成特定的值，最终镜像也可能比那个值大，但是却不会更小。

·IMAGE_ROOTFS_SIZE：以KB定义被创建的根文件系统镜像大小。构建系统使用这个值作为请求或者推荐。依赖于实际需要的空间，最终的根文件系统镜像大小可能更大。默认值是65536。

·IMAGE_ROOTFS_ALIGNMENT：以KB定义根文件系统镜像的对齐。如果根文件系统镜像的最终大小不是这个值的倍数，那么它将被向上舍入到它最接近的倍数。默认值是1。

·IMAGE_ROOTFS_EXTRA_SPACE：增加额外可用空间到根文件系统镜像。变量以KB指定值。例如，为了增加额外的4GB空间，设置变量为IMAGE_ROOTFS_EXTRA_SPACE="4194304"。默认值是0。

·IMAGE_OVERHEAD_FACTOR：这个变量指定用于根文件系统镜像的乘数。这个倍数在根文件系统所要求的实际空间已经确定后才被应用。默认值是1.3。

在构建系统已经于暂存区域中（由变量IMAGE_ROOTFS指定的目录）创建了根文件系统后，它使用du-ks${IMAGE_ROOTFS}来计算其以KB为单位的实际大小。函数_get_rootfs_size()计算最终的根文件系统镜像大小，如程序清单7-7以伪代码所展示的。

程序清单7-7　根文件系统镜像大小计算伪代码

```
_get_rootfs_size():
   ROOTFS_SIZE =`du -ks ${IMAGE_ROOTFS}`
   BASE_SIZE = ROOTFS_SIZE * IMAGE_OVERHEAD_FACTOR

   if (BASE_SIZE < IMAGE_ROOTFS_SIZE):
      IMG_SIZE = IMAGE_ROOTFS_SIZE + IMAGE_ROOTFS_EXTRA_SPACE
   else:
      IMG_SIZE = BASE_SIZE + IMAGE_ROOTFS_EXTRA_SPACE

   IMG_SIZE = IMG_SIZE + IMAGE_ROOTFS_ALIGNMENT - 1
   IMG_SIZE = IMG_SIZE % IMAGE_ROOTFS_ALIGNMENT

   return IMG_SIZE
```

最常见地，镜像菜谱设置IMAGE_ROOTFS_SIZE和IMAGE_ROOTFS_EXTRA_SPACE来调整最终根文件系统的镜像大小。如果你顾虑根文件系统的内存占用，那么你可能也希望减小IMAGE_OVERHEAD_FACTOR或者设置它为1以缩减镜像。

7.3.4 根文件系统类型

最终，你使用根文件系统镜像来为目标创建可启动介质或者启动QEMU模拟器。为此，构建系统提供可以为各种文件系统类型创建根文件系统的image_types类。

镜像菜谱不需要直接使用image_types类而是将变量IMAGE_FSTYPES设置成一个或者多个由类提供的文件系统类型。使用

```
IMAGE_FSTYPES = "ext3 tar.bz2"
```

创建两个根文件系统镜像，一个使用ext3文件系统，一个使用bzip2算法压缩的tar包。

image_types类定义变量IMAGE_TYPES，其中包含你可以在IMAGE_FSTYPES中指定的所有镜像类型的列表。列表显示按照核心类型排序的文件系统类型。通常，一些核心类型也以压缩的形式被使用，以保留空间。如果压缩算法被用于文件系统，那么核心类型的名字则由压缩类型来追加：<core name>.<compression type>。

·tar，tar.gz，tar.bz2，tar.xz，tar.lz3：创建非压缩和压缩的tar包形式的根文件系统。

·ext2，ext2.gz，ext2.bz2，ext2.lzma：使用不带压缩或者带压缩的ext2文件系统的根文件系统镜像。

·ext3，ext3.gz：使用不带压缩或者带压缩的ext3文件系统的根文件系统镜像。

·btrfs：B树文件系统的根文件系统镜像。

·jffs2，jffs2.sum：基于第二代日志闪存文件系统（the second generation of the Journaling Flash File System，JFFS2）的非压缩或者压缩的根文件系统。因为第二代日志闪存文件系统直接支持NAND闪存设备，所以对于嵌入式设备来说，它是流行的选择。它也支持日志（journaling）和损耗均衡（wear-leveling）。

·cramfs：使用压缩的ROM文件系统（cramfs）的根文件系统镜像。Linux内核可以挂载这个文件系统而不用先解压。压缩使用zlib算法，其一次一页地压缩文件以允许随机访问。这个文件系统是只读的以便简化它的设计，因为带压缩的随机写访问很难实现。

·iso：使用ISO 9660标准用于可启动CD-ROM的根文件系统镜像。这种文件系统类型不是独立的格式。它使用ext3作为底层文件系统类型。

·hddimg：用于可启动硬盘的根文件系统镜像。它使用ext3作为实际的文件系统类型。

·squashfs，squashfs-xz：特定用于Linux的压缩的只读根文件系统类型，与cramfs类似，但是具有更好的压缩性以及对大文件和文件系统的支持。相比于cramfs的固定的4KB块大小，SquashFS有从0.5KB～64KB的可变的块大小，它允许更大的文件和文件系统大小。SquashFS使用gzip压缩，而squashfs-xz为了更小的镜像而使用Lempel–Ziv–Markov（LZMA）压缩。

·ubi，ubifs：使用非排序块镜像（Unsorted Block Image，UBI）格式的用于原始闪存设备的根文件系统镜像。非排序块镜像文件系统（UBI File System，UBIFS）实质上是JFFS2的继承者，两者间区别在于前者支持写缓存。在IMAGE_FSTYPES中使用ubifs仅仅创建ubifs根文件系统镜像。使用ubi创建ubifs根文件系统镜像并且运行ubinize实用程序

以创建可以直接被写到一台闪存设备的镜像。

·cpio，cpio.gz，cpio.xz，cpio.lzma：使用非压缩或者压缩的复制进出（Copy in And out，CPIO）流的根文件系统镜像。

·vmdk：使用VMware虚拟机器磁盘格式的根文件系统镜像。它使用ext3作为底层文件系统格式。

·elf：用来自Coreboot项目（www.coreboot.org）的mkelfImage实用程序创建的可启动根文件系统镜像。

再说一次，使用什么镜像类型完全依赖于你项目的需求，特别是依赖目标硬件。启动设备、引导加载程序、内存限制和其他因素决定什么根文件系统是适合项目的。我们的推荐是，在菜谱中指定根文件系统类型ext3和tar，或者更好的，例如tar.bz2的压缩格式。ext3格式允许你为了测试而用QEMU模拟器启动根文件系统。tar文件系统可以简单地解压到被分区和格式化的介质上。然后，用于目标硬件的机器配置文件可以增加额外的适合它的根文件系统类型。

7.3.5 用户、组和密码

类extrausers提供了可配置的用于增加用户和组到镜像以及为用户账号设置密码的机制（见程序清单7-8）。

程序清单7-8 修改用户、组和密码

```
SUMMARY = "Custom image recipe from scratch"
DESCRIPTION = "Directly assign IMAGE_INSTALL and IMAGE_FEATURES for \
               for direct control over image contents."

LICENSE = "MIT"

# We are using the assignment operator (=) below to purposely overwrite
# the default from the core-image class.
IMAGE_INSTALL = "packagegroup-core-boot packagegroup-base-extended \
                 ${CORE_IMAGE_EXTRA_INSTALL}"

inherit core-image
inherit extrausers

# set image root password
ROOT_PASSWORD = "secret"
DEV_PASSWORD = "hackme"

EXTRA_USERS_PARAMS = "\
   groupadd developers; \
   useradd -p `openssl passwd ${DEV_PASSWORD}` developer; \
   useradd -g developers developer; \
   usermod -p `openssl passwd ${ROOT_PASSWORD}` root; \
   "
```

程序清单增加名为developers的组和名为developer的用户账号，并且把用户账号加到组。它也为root账号修改密码。用于增加和修改组、用户和密码的命令被加到变量EXTRA_USERS_PARMS中，其是由类来解释的。由类理解的命令是：

·useradd：增加用户账号

·usermod：修改用户账号

·userdel：去除用户账号

·groupadd：增加用户组

·groupmod：修改用户组

·groupdel：去除用户组

类执行具有相应名字的各个Linux实用程序。因此，选项是完全相同的，并且可以简单地在Linux手册页找到。注意，各个命令之间必须由分号分隔。

使用选项-p和命令useradd与usermod设置用户账号的密码。密码必须作为密码散列提供。你可以手动计算密码散列和增加它到菜谱，或者如例子所示的，让菜谱计算它。

关于root用户账号的一句话：如果debug-tweaks被包含在IMAGE_FEATURES中，那么构建系统为镜像设置带有空密码的root用户。去除debug-tweaks用*替换空root密码，*禁用账号，所以不再可能从控制台以root登录。对于生产使用，我们强烈推荐从构建中去除debug-tweaks。如果嵌入式系统需要控制台登录能力，你可以如前面所显示的设置root密码，或者增加sudo菜谱并且设置用户账号为sudoers。

例如，如果你希望给developer用户账号sudoer权限，可以简单地增加sudo到IMAGE_INSTALL并且增加usermod-a-G sudo developer到EXTRA_USERS_PARAMS。

7.3.6 调整根文件系统

为了在其已经被构建系统创建后和在实际的根文件系统镜像被创建前进一步地定制根文件系统，可以使用ROOTFS_POSTPROCESS_COMMAND（见程序清单7-9）。变量包含由分号分隔的shell函数列表。

程序清单7-9 ROOTFS_POSTPROCESS_COMMAND

```
SUMMARY = "Custom image recipe from scratch"
DESCRIPTION = "Directly assign IMAGE_INSTALL and IMAGE_FEATURES for \
               for direct control over image contents."

LICENSE = "MIT"

# We are using the assignment operator (=) below to purposely overwrite
# the default from the core-image class.
IMAGE_INSTALL = "packagegroup-core-boot packagegroup-base-extended \
                 ${CORE_IMAGE_EXTRA_INSTALL}"

inherit core-image

# Additional root filesystem processing
modify_shells() {
   printf "# /etc/shells: valid login shells\n/bin/sh\n/bin/bash\n" \
          > ${IMAGE_ROOTFS}/etc/shells
}
ROOTFS_POSTPROCESS_COMMAND += "modify_shells;"
```

该示例添加bash shell到/etc/shells。保证总是使用+=操作符来添加到ROOTFS_POSTPROCESS_COMMAND，因为构建系统会增加它自己的后处理命令到ROOTFS_POSTPROCESS_COMMAND。

sudo配置

如果你遵循了前面段落中关于给用户sudoer权限的例子，那么你可能注意到，除非你取消注释在/etc/sudoers中的行%sudo ALL=(ALL)ALL，否则它不会工作。当根文件系统镜像被创建时，被增加到ROOTFS_POSTPROCESS_COMMAND的简单shell函数会处理那个取消

注释的工作（见程序清单7-10）。

程序清单7-10　sudo配置

```
modify_sudoers() {
   sed 's/# %sudo/%sudo/' < ${IMAGE_ROOTFS}/etc/sudoers > \
        ${IMAGE_ROOTFS}/etc/sudoers.tmp
   mv ${IMAGE_ROOTFS}/etc/sudoers.tmp ${IMAGE_ROOTFS}/etc/sudoers
}
ROOTFS_POSTPROCESS_COMMAND += "modify_sudoers;"
```

脚本简单地使用sed取消注释行。

SSH服务器配置

所有核心镜像自动包括用于远程shell访问系统的SSH服务器。默认情况下，服务器被配置成用用户名和密码登录。使用公钥基础设施（Public Key Infrastructure，PKI）提供额外级别的安全性但是需要配置root服务器和安装密钥到根文件系统中。ROOTFS_POSTPROCESS_COMMAND可以被用来简单地完成那个任务（见程序清单7-11）。

程序清单7-11　SSH服务器配置

```
configure_sshd() {
   # disallow password authentication

   echo "PasswordAuthentication no" >> ${IMAGE_ROOTFS}/etc/ssh/sshd_config
   # create keys in tmp/deploy/keys
   mkdir -p ${DEPLOY_DIR}/keys
   if [ ! -f ${DEPLOY_DIR}/keys/${IMAGE_BASENAME}-sshroot ]; then
      ssh-keygen -t rsa -N '' \
         -f ${DEPLOY_DIR}/keys/${IMAGE_BASENAME}-sshroot
   fi
   # add public key to authorized_keys for root
   mkdir -p ${IMAGE_ROOTFS}/home/root/.ssh
   cat ${DEPLOY_DIR}/keys/${IMAGE_BASENAME}-sshroot.pub \
       >> ${IMAGE_ROOTFS}/home/root/.ssh/authorized_keys
}
ROOTFS_POSTPROCESS_COMMAND += "configure_sshd;"
```

脚本首先禁用用于SSH的带有用户名和密码的认证。然后它在构建

环境内部的tmp/deploy/keys中使用根文件系统镜像的名字（本质上是镜像菜谱的名字）创建密钥对。如果前面的构建已经创建了一套密钥，那么它们将被保留。最后，脚本增加公钥到在通常用于SSH配置的/home/root/.ssh中的authorized_keys文件。用于其他用户的登录密钥可以用类似的方法创建。

如果你不要求不同的密钥用于你构建的每个设备，那么这种方法工作得很好，因为根文件系统的每个副本当然包含相同的密钥。如果你需要不同的密钥，或者通常不同的配置用于设备，那么你需要为设备生产设计准备配置系统。

7.4 发行版配置

构建系统为应用到所有被构建的镜像的全局配置提供了一种机制。这种机制称为发行版配置（distribution configuration）或者发行版策略（distribution policy）。它仅仅是包含变量设置的配置文件。发行版配置是通过DISTRO变量设置而被包含在构建环境配置文件conf/local.conf中的：

```
DISTRO = "poky"
```

变量设置对应于其基础名字与变量的参数相同并且带有文件后缀.conf的分发配置文件。对于前面的例子，构建系统在所有被构建环境包含的元数据层的子目录conf/distro中搜索具有名字poky.conf的发行版配置文件。

7.4.1 标准发行版策略

Yocto项目提供了一些用于标准配置策略的发行版配置文件：

·poky：poky是用于Yocto项目的参考发行版Poky的默认策略。关于用Yocto项目来开始和作为你自己的发行版配置文件的模板，它是个好选择。

·poky-bleeding：这个发行版配置基于poky但是设置所有包的版本为最新修订。它通常被Yocto项目开发者用于集成测试目的。当然，你可以使用它，但是要知晓，具有不兼容版本的包可能有问题。

·poky-lsb：这个发行版配置用于符合Linux标准基的栈。它倾向于被使用在core-image-lsb镜像目标和从其衍生的镜像目标上。它继承来自poky的基础设置并且增加全局配置设置以启用安全性和包含由Linux标准基遵从性所要求的默认库。

·poky-tiny：这个发行版配置剪裁设置以产生非常紧凑的、用于嵌入式设备的Linux操作系统栈。它基于poky但是提供仅仅最简要最小化的必要功能性以支持硬件和BusyBox环境。除了仅仅有串行控制台之外，它不支持任何视频。因为它的精简配置，只有core-image-minimal镜像目标和基于core-image-minimal的镜像目标可以用poky-tiny发行版配置来构建。

标准发行版策略，特别是poky，是用于你自己的发行版配置的良好起点。让我们仔细看看poky发行版配置以理解发行版策略是如何设置的以及我们可以如何将它们用到我们自己的项目中。

7.4.2　Poky发行版策略

你可以在构建系统的meta-yocto/conf/distro目录内找到包含Poky发行版策略的文件poky.conf。我们为了方便把它的内容复制在此，重新编排了文件格式以适合页面，将变量设置分组成逻辑块，并且增加了一些注释（见程序清单7-12）。

程序清单7-12　Poky发行版策略meta-yocto/conf/distro/poky.conf

```
# Distribution Information

DISTRO = "poky"
DISTRO_NAME = "Poky (Yocto Project Reference Distro)"
DISTRO_VERSION = "1.6+snapshot-${DATE}"
DISTRO_CODENAME = "next"
MAINTAINER = "Poky <poky@yoctoproject.org>"
TARGET_VENDOR = "-poky"

# SDK Information
SDK_NAME = \
    "${DISTRO}-${TCLIBC}-${SDK_ARCH}-${IMAGE_BASENAME}-${TUNE_PKGARCH}"
SDK_VERSION := \
    "${@'${DISTRO_VERSION}'.replace('snapshot-${DATE}','snapshot')}"
SDK_VENDOR = "-pokysdk"
SDKPATH = "/opt/${DISTRO}/${SDK_VERSION}"

# Distribution Features
# Override these in poky based distros
POKY_DEFAULT_DISTRO_FEATURES = "largefile opengl ptest multiarch wayland"
POKY_DEFAULT_EXTRA_RDEPENDS = "packagegroup-core-boot"
POKY_DEFAULT_EXTRA_RRECOMMENDS = "kernel-module-af-packet"

DISTRO_FEATURES ?= "${DISTRO_FEATURES_DEFAULT} ${DISTRO_FEATURES_LIBC} \
                    ${POKY_DEFAULT_DISTRO_FEATURES}"

# Preferred Versions for Packages
PREFERRED_VERSION_linux-yocto ?= "3.14%"
PREFERRED_VERSION_linux-yocto_qemux86 ?= "3.14%"
PREFERRED_VERSION_linux-yocto_qemux86-64 ?= "3.14%"
PREFERRED_VERSION_linux-yocto_qemuarm ?= "3.14%"
PREFERRED_VERSION_linux-yocto_qemumips ?= "3.14%"
PREFERRED_VERSION_linux-yocto_qemumips64 ?= "3.14%"
PREFERRED_VERSION_linux-yocto_qemuppc ?= "3.14%"
# Dependencies
```

```
DISTRO_EXTRA_RDEPENDS += " ${POKY_DEFAULT_EXTRA_RDEPENDS}"
DISTRO_EXTRA_RRECOMMENDS += " ${POKY_DEFAULT_EXTRA_RRECOMMENDS}"

POKYQEMUDEPS = "${@bb.utils.contains( \
     "INCOMPATIBLE_LICENSE", "GPLv3", "", "qemu-config",d)}"
DISTRO_EXTRA_RDEPENDS_append_qemuarm = " ${POKYQEMUDEPS}"
DISTRO_EXTRA_RDEPENDS_append_qemumips = " ${POKYQEMUDEPS}"
DISTRO_EXTRA_RDEPENDS_append_qemuppc = " ${POKYQEMUDEPS}"
DISTRO_EXTRA_RDEPENDS_append_qemux86 = " ${POKYQEMUDEPS}"
DISTRO_EXTRA_RDEPENDS_append_qemux86-64 = " ${POKYQEMUDEPS}"

# Target C Library Configuration
TCLIBCAPPEND = ""

# Target Architectures for QEMU
# (see meta/recipes-devtools/qemu/qemu-targets.inc)
QEMU_TARGETS ?= "arm i386 mips mipsel ppc x86_64"
# Other QEMU_TARGETS "mips64 mips64el sh4"

# Package Manager Configuration
EXTRAOPKGCONFIG = "poky-feed-config-opkg"

# Source Mirrors
PREMIRRORS ??= "\
bzr://.*/.*   http://downloads.yoctoproject.org/mirror/sources/ \n \
cvs://.*/.*   http://downloads.yoctoproject.org/mirror/sources/ \n \
git://.*/.*   http://downloads.yoctoproject.org/mirror/sources/ \n \
gitsm://.*/.* http://downloads.yoctoproject.org/mirror/sources/ \n \
hg://.*/.*    http://downloads.yoctoproject.org/mirror/sources/ \n \
osc://.*/.*   http://downloads.yoctoproject.org/mirror/sources/ \n \
p4://.*/.*    http://downloads.yoctoproject.org/mirror/sources/ \n \
svk://.*/.*   http://downloads.yoctoproject.org/mirror/sources/ \n \
svn://.*/.*   http://downloads.yoctoproject.org/mirror/sources/ \n"

MIRRORS =+ "\
ftp://.*/.*      http://downloads.yoctoproject.org/mirror/sources/ \n \
http://.*/.*     http://downloads.yoctoproject.org/mirror/sources/ \n \
https://.*/.*    http://downloads.yoctoproject.org/mirror/sources/ \n"

# Build System Configuration

# Configuration File and Directory Layout Versions
LOCALCONF_VERSION = "1"
LAYER_CONF_VERSION ?= "6"
#
# OELAYOUT_ABI allows us to notify users when the format of TMPDIR changes
# in an incompatible way. Such changes should usually be detailed in the
# commit that breaks the format and have been previously discussed on the
# mailing list with general agreement from the core team.
#
OELAYOUT_ABI = "8"

# Default hash policy for distro
BB_SIGNATURE_HANDLER ?= 'OEBasicHash'

# Build System Checks
# add poky sanity bbclass
INHERIT += "poky-sanity"

# The CONNECTIVITY_CHECK_URIs are used to test whether we can successfully
# fetch from the network (and warn you if not). To disable the test, set
# the variable to be empty.
# Git example url: \
```

```
    git://git.yoctoproject.org/yocto-firewall-test;protocol=git;rev=HEAD

CONNECTIVITY_CHECK_URIS ?= " \
             https://eula-downloads.yoctoproject.org/index.php \
             http://bugzilla.yoctoproject.org/report.cgi"

SANITY_TESTED_DISTROS ?= " \
            Poky-1.4 \n \
            Poky-1.5 \n \
            Poky-1.6 \n \
            Ubuntu-12.04 \n \
            Ubuntu-13.10 \n \
            Ubuntu-14.04 \n \
            Fedora-19 \n \
            Fedora-20 \n \
            CentOS-6.4 \n \
            CentOS-6.5 \n \
            Debian-7.0 \n \
            Debian-7.1 \n \
            Debian-7.2 \n \
            Debian-7.3 \n \
            Debian-7.4 \n \
            SUSE-LINUX-12.2 \n \
            openSUSE-project-12.3 \n \
            openSUSE-project-13.1 \n \
            "

# QA check settings - a little stricter than the OE-Core defaults
WARN_QA = "textrel files-invalid incompatible-license xorg-driver-abi \
           libdir unknown-configure-option"
ERROR_QA = "dev-so debug-deps dev-deps debug-files arch pkgconfig la \
            perms useless-rpaths rpaths staticdev ldflags pkgvarcheck \
            already-stripped compile-host-path dep-cmp \
            installed-vs-shipped  install-host-path packages-list \
            perm-config perm-line perm-link pkgv-undefined \
            pn-overrides split-strip var-undefined version-going-backwards"
```

在程序清单中显示的文件来自撰写本书时Yocto项目Git仓库的头。取决于你正在使用什么版本的Yocto项目工具，这个文件可能看起来稍稍不同。文件仅仅是发行版策略的例子。它提供最常用地关联到发行版配置的变量设置。不限于只是使用显示在程序清单中的设置，如果你的项目不需要它们，你也可以去除设置。

发行版信息

这一部分的发行版策略文件包含用于有关发行版的一般信息的设置。

·DISTRO：发行版的简称。值必须匹配发行版配置文件的基名称。

·DISTRO_NAME：发行版的长名称。大量菜谱引用这个变量。它的内容显示在控制台启动提示上。

·DISTRO_VERSION：发行版版本字符串。它被大量菜谱引用并且被用在文件名的发行版制品中。它显示在控制台启动提示上。

·DISTRO_CODENAME：用于发行版的代码名称。它当前仅仅被Linux标准基菜谱所使用，并且被复制进lsb-release系统配置文件中。

·MAINTAINER：发行版维护者的名字和邮件地址。

·TARGET_VENDOR：用大量变量（尤其是目标系统，TARGET_SYS）连接的目标厂商字符串。TARGET_SYS是目标架构（TARGET_ARCH）、目标厂商（TARGET_VENDOR）和例如i586-poky-linux的目标操作系统（TARGET_OS）的连接。这3个部分由连字符（-）所分隔。TARGET_VENDOR字符串必须被前缀以连字符，而TARGET_OS一定不能被前缀以连字符。这是OpenEmbedded的很多不幸的非一致性之一。你可能希望设置这个变量为你或者你公司的名字。

软件开发工具包（SDK）信息

在本部分中的设置提供用于软件开发工具包的基础配置。

·SDK_NAME：构建系统用于软件开发工具包输出文件的基名称。它是通过用连字符连接DISTRO、TCLIBC、SDK_ARCH、IMAGE_BASENAME和TUNE_PKGARCH变量来获取的。对你来说，没有理由改变它的默认设置字符串，因为它提供用于区分不同软件开发工具包的所有信息。

·SDK_VERSION：软件开发工具包版本字符串，它通常被设置成DISTRO_VERSION。

·SDK_VENDOR：软件开发工具包厂商字符串，其目的和TARGET_VENDOR的类似。与TARGET_VENDOR一样，字符串必须被前缀以连字符。

·SDKPATH：用于软件开发工具包的默认安装路径。软件开发工具包安装程序在软件开发工具包的安装中向用户提供这个路径。用户可以接受它或者输入替代的路径。默认值/opt/${DISTRO}/${SDK_VERSION}安装软件开发工具包进入/opt系统目录，它要求root权限。可行的替代是通过设置SDKPATH="${HOME}/${DISTRO}/${SDK_VERSION}"来安装软件开发工具包进入用户的家目录。

发行版特性

这些特性提供用于发行版的特定功能性。

·DISTRO_FEATURES：为软件包中的特定功能性启用支持的发行版特性列表。在poky.conf发行版策略文件中的赋值包含DISTRO_FEATURES_DEFAULT和DISTRO_FEATURES_LIBC。这两个都包含默认发行版特性设置。我们将在接下来的两个部分讨论发行版特性，它们如何工作以及默认设置。

倾向的版本

版本设置规定用于包的特定版本而不是默认版本。

·PREFERRED_VERSION：如果你不希望使用最新版本，那么使用PREFERRED_VERSION允许为软件包设置特定版本，因为默认是最新版本。通常用于Linux内核，但也用于应用软件对其有强版本依赖性的软件包。

依赖性

这些设置是用于发行版运行时所需要的依赖性的声明。

·DISTRO_EXTRA_RDEPENDS：设置用于发行版的运行时依赖性。这个变量声明的依赖性是发行版所需要的。如果这些依赖性不被满足，那么构建发行版失败。

·DISTRO_EXTRA_RRECOMMENDS：为发行版所推荐的用于提供额外有用功能性的包。这些依赖性如果是可用的那么就被增加，但如果它们不被满足，构建发行版也不失败。

工具链配置

这些设置配置用于构建发行版的工具链。

·TCMODE：这个变量选择构建系统所使用的工具链。默认值是default，它选择由构建系统构建的内部工具链（gcc、binutils等）。变量的设置对应于配置文件tcmode-${TCMODE}.inc，构建系统在路径conf/distro/include中定位它。这允许通过包含提供了必要工具和配置文件的工具链层来用构建系统包含外部工具链。如果正在使用外部工具链，那么你必须保证它是和Poky构建系统兼容的。

·TCLIBC：指定要使用的C库。构建系统当前支持EGLIBC、uClibc和musl。变量的设置对应于构建系统在路径conf/distro/include中定位的配置文件tclibc-${TCLIBC）.inc。这些配置文件为库等设置倾向的提供者。

·TCLIBCAPPEND：构建系统追加这个字符串到其他变量以按照C库区分构建制品。如果你正在用不同的C库做实验，那么你可能希望在发行版配置中使用设置：

```
TCLIBCAPPEND = "-${TCLIBC}"
TMPDIR .= "${TCLIBCAPPEND}"
```

它为每个C库创建独立的构建输出目录结构。

镜像配置

以下设置为下载源包配置镜像。

·PREMIRRORS和MIRRORS：Poky发行版增加这些变量来设置它的镜像配置以使用Yocto项目仓库作为用于下载的源。如果你希望使用你自己的镜像，那么你可以增加它们到发行版配置文件。然而，因为镜像不是严格的发行版配置，所以你可能希望增加这些变量到你构建环境的local.conf文件中。另有替代是增加它们到定制层的layer.conf文件中。

构建系统配置

这些设置定义用于构建系统的需求。

·LOCALCONF_VERSION：为构建环境配置文件local.conf设置期望或者要求的版本。构建系统将这个值和local.conf中CONF_VERSION变量的值进行比较。如果LOCALCONF_VERSION是比CONF_VERSION更新的版本，那么构建系统可能能够自动升级local.conf为更新的版本。否则，构建系统以错误消息退出。

·LAYER_CONF_VERSION：为构建环境的bblayers.conf配置文件设置期望或者要求的版本。构建系统将这个版本和bblayers.conf设置的LCONF_VERSION的值进行比较。如果LAYER_CONF_VERSION是比LCONF_VERSION更新的版本，那么构建系统可能能够自动升级bblayers.conf为更新的版本。否则，构建系统以错误消息退出。

·OELAYOUT_ABI：为输出目录TMPDIR的布局设置期望或者要求

的版本。构建系统在TMPDIR内的文件abi_version中存储实际的布局版本。如果两者不兼容，那么构建系统以错误消息退出。这通常仅仅发生在你正在使用更新版本的构建系统和前面版本所创建的构建环境，并且布局不兼容地变化了的时候。可删除TMPDIR通过重新创建目录来解决问题。

·BB_SIGNATURE_HANDLER：签名处理器用于签署共享状态缓存条目和创建戳记文件。值引用因为其复杂度而常以Python实现的签名处理器函数。在meta/lib/oe/sstatesig.py中的代码实现了基于由bitbake/lib/bb/siggen.py定义的BitBake签名生成器SignatureGeneratorBasic和SingnatureGeneratorBasicHash的OEBasic和OEBasicHash，并且演示了如何插入你自己的签名处理器函数。这两个签名处理器大部分是相同的，但是OEBasicHash在签名中包含任务代码，它导致任何对元数据的变更以使戳记文件和共享状态缓存条目失效而不用显式地改变包的修订号码。使用OEBasicHash的默认值对于大部分应用来说通常是足够的。

构建系统检查

这些配置变量控制各种验证器来捕获构建系统配置错误。

·INHERIT+="poky-sanity"：集成类poky-sanity，它是执行构建系统检查所需要的。推荐你在发行版配置文件中包含这个指令。

·CONNECTIVITY_CHECK_URIS：构建系统尝试验证网络连通性的URI列表。在Poky的案例中，这些URI指向在Yocto项目的高可用基础设施中的文件。如果你打算使用自己的镜像来下载源包，那么你可以使用指向到镜像服务器上的文件的URI来验证正确的连通性。

·SANITY_TESTED_DISTROS：Poky构建系统已经在其上测试过的Linux发行版列表。构建系统按照这个列表验证它正运行于的Linux发行

版。如果那个发行版不在列表中，Poky展示警告消息并且不加理会地启动构建过程。Poky运行在最新的Linux发行版上，并且在大多数情况下，即使该发行版没有官方支持，构建工作也很好。

质量保证检查

质量保证（quality assurance）检查是由meta/classes/insane.bbclass定义和实现的。这个类也定义包含在构建过程中的质量保证任务。质量保证检查是在配置、打包和其他构建任务之后被执行的。下面的两个变量定义什么质量保证检查导致警告消息和什么检查导致构建系统以错误消息终止构建：

·WARN_QA：创建警告消息但是构建继续的质量保证检查列表。

·ERROR_QA：创建错误消息而且构建终止的质量保证检查列表。

前面的列表代表发行版配置所使用的最常见的变量设置。为了你自己的发行版配置，按照需要，你可以增加或省略变量。

7.4.3 发行版特性

发行版特性在软件包中启用对特定功能性的支持。增加发行版特性到变量DISTRO_FEATURES会增加这个特性的功能性到在构建时支持它的软件包中。例如，如果软件包可以被构建以用于控制台或者图形用户界面，那么增加x11到DISTRO_FEATURES则会配置软件包以使它被构建成具有X11支持。与x11镜像特性不同，这不意味着X11包被安装进目标根文件系统。发行版特性仅仅准备具有X11支持的包以使它在X11基包已被安装的系统上使用X11。

使用DISTRO_FEATURES给了你对软件包如何被构建的细粒度的控制。如果你不需要特定的功能性，那么省略启用它的发行版特性对特定软件包来说通常会产生更小的内存占用。

从构建系统的安装目录使用

```
$ grep -R DISTRO_FEATURES *
```

给了你使用DISTRO_FEATURES来条件性地修改设置或者构建过程（依赖于启用了什么发行版特性）的全部菜谱和包含文件的列表。

菜谱通常使用

```
bb.utils.contains('DISTRO_FEATURES', <feature>, <true_val>, <false_val>)
```

来扫描DISTRO_FEATURES以判断特定发行版特性是否被DISTRO_FEATURES启用。如果DISTRO_FEATURES包含feature，那么函数返回true_val；否则，函数返回false_val。这使得其对于开发者向BitBake变量赋值或者在if-then-else语句中使用函数来说是便利的。通常它被do_configure任务使用以便基于DISTRO_FEATURES修改配置。对

一些包来说，它可能向makefile提供标记（flag）。

典型的例子是用于构建EGLIBC库的菜谱。EGLIBC允许通过设置配置选项来启用功能性。文件meta/recipes-core/egligc/egilbc-options.inc是被菜谱包含的，它基于由DISTRO_FEATURES提供的发行版特性来设置配置选项。

下面的列表显示了你可以增加到DISTRO_FEATURES以在整个发行版中全局性地启用软件包中的功能性的最常见发行版特性。

·alsa：启用对高级Linux声卡架构（Advanced Linux Sound Architecture，ALSA）的支持，包括在可用时安装开源兼容性模块。

·bluetooth：启用对蓝牙的支持。

·cramfs：启用对压缩文件系统CramFS的支持。

·directfb：启用对DirectFB的支持。

·ext2：为具有内部大存储设备（例如硬盘而不仅是闪存设备）的设备启用支持和包含工具。

·ipsec：启用对使用互联网协议安全（Internet Protocol Security，IPSec）的身份认证和加密的支持。

·ipv6：启用对互联网协议版本6（Internet Protocol version 6，IPv6）的支持。

·irda：启用对如红外数据协会（Infrared Data Association，IrDA）所指定的无线红外数据通信的支持。

·keyboard：启用键盘支持，包括在系统启动中加载键盘布局。

·nfs：为在系统上挂载网络文件系统（NFS）导出而启用客户端网络文件系统支持。

·opengl：包含开放图形库（Open Graphics Library，OpenGL），它是用于渲染2D和3D图形的应用程序编程接口。OpenGL运行在不同平台上并且为大部分常见编程语言提供绑定。

·pci：启用对PCI总线的支持。

·pcmcia：启用对PCMCIA和CompactFlash的支持。

·ppp：启用用于拨号联网的点到点协议（Point-to-Point Protocol，PPP）支持。

·smbfs：为了通过网络共享远程文件系统、打印机和其他设备，对微软的服务器消息块（Server Message Block，SMB）启用支持和包含客户。

·systemd：包含对替代用于启动和关闭系统的SysVinit基于脚本的系统的系统管理守护进程（system management daemon，systemd）的支持。

·sysvinit：包含对SysVinit系统管理器的支持。

·usbgadget：启用对允许Linux设备在被连接到另一系统时像USB设备（从角色）一样表现的Linux-USB Gadget应用编程接口框架的支持。

·usbhost：启用允许例如键盘、鼠标、摄像头等被连接到系统的USB端口并且被其检测到的USB主机支持。

·wayland：启用对Wayland合成器协议（Wayland compositor protocol）的支持并且包含Weston合成器。

·wifi：启用WiFi支持。

·x11：包含X11服务器和库。

这个程序清单不包含用于C库配置的发行版特性。这些发行版特性都以libc-开头。如果C库像Yocto项目的默认C库glibc一样可配置，那么它们启用对由C库提供的功能性的支持。如果你正在使用glibc，那么你不必担心设置这些发行版特性，因为它们是从默认发行版设置中继承的，默认发行版设置将在接下来的部分讨论。

如果你已经工作于Yocto项目，那么你可能已经注意到，其中也有名为MACHINE_FEATURES的变量并且可允许的机器特性列表和发行版特性列表有很大的交集。例如MACHINE_FEATURES和DISTRO_FEATURES都提供特性bluetooth。在DISTRO_FEATURES中启用蓝牙（Bluetooth）导致用于硬件支持的蓝牙包被安装并且也为大量软件包启用蓝牙支持。然而，在MACHINE_FEATURES中启用蓝牙仅仅导致用于硬件支持的蓝牙包被安装。这给了你对于机器和发行版级别的功能性的控制。在研究Yocto项目板支持包时，我们会详细讨论机器特性。

7.4.4 系统管理器

构建系统支持传统的基于脚本的系统管理器SysVinit以及系统管理守护进程（systemd），systemd是SysVinit的替代，它提供服务间更好的优先级化和依赖性处理以及并行启动服务的能力以加速启动序列。

SysVinit是用于由Poky构建的Linux发行版的默认系统管理器。如果你希望使用SysVinit，那么不必改变配置。

为了启用systemd，你需要增加它到发行版特性并且设置它为系统管理器。增加以下命令到发行版配置文件：

```
DISTRO_FEATURES_append = " systemd"
VIRTUAL-RUNTIME_init_manager = "systemd"
```

第一行在根文件系统中安装systemd，第二行启用它作为系统管理器。如果SysVinit也包含在DISTRO_FEATURES中，那么安装和启用systemd不会从根文件系统中去除SysVinit。如果你正在使用标准发行版配置之一，例如poky，那么你可以用如下配置来从DISTRO_FEATURES中去除SysVinit：

```
DISTRO_FEATURES_BACKFULL_CONSIDERED = "sysvinit"
```

它比完全重新定义DISTRO_FEATURES更加简单。针对你自己的发行版配置，当然，你可以简单地从DISTRO_FEATURES列表中省略SysVinit。

SysVinit用以启动各个系统服务的初始化脚本（initscripts）通常是提供服务的包的一部分。如果你唯独想使用systemd，那么为了在根文件系统中节约空间，你可能不希望安装初始化脚本。使用

```
VIRTUAL-RUNTIME_initscripts = ""
```

来阻止构建系统安装SysVinit初始化脚本。

一句话警告：一些守护进程可能还没有调整以用于systemd，因此systemd服务文件是不可用的。如果你碰到这样的软件，你可能不得不自行做出调整。如果你这样做，那么请考虑向上游（upstream）提交工作。

7.4.5 默认发行版设置

OpenEmbedded核心元数据层通过文件meta/conf/distro/defaultsetup.conf和一系列被它包含的其他文件（见程序清单7-13）来提供默认发行版设置。默认发行版配置如何被包含进构建环境不是那么显而易见的，因为这个文件不是由例如poky.conf的发行版策略文件所包含的，我们前面讨论了这一点。反而，文件是由BitBake的主要配置文件bitbake.conf所包含的。

了解defaultsetup.conf并理解其设置十分重要，因为你自己的发行版策略配置可能扩展或者覆盖由它提供的一些默认变量设置。如果你不正确地设置默认发行版，那么可能不经意便失去重要的默认设置，并且发行版构建可能会失败或者不产出期望的结果。

程序清单7-13　默认发行版设置meta/conf/distro/defaultsetup.conf

```
include conf/distro/include/default-providers.inc
include conf/distro/include/default-versions.inc
include conf/distro/include/default-distrovars.inc
include conf/distro/include/world-broken.inc

TCMODE ?= "default"
require conf/distro/include/tcmode-${TCMODE}.inc

TCLIBC ?= "eglibc"
```

```
require conf/distro/include/tclibc-${TCLIBC}.inc

# Allow single libc distros to disable this code
TCLIBCAPPEND ?= "-${TCLIBC}"
TMPDIR .= "${TCLIBCAPPEND}"

CACHE = "${TMPDIR}/cache/${TCMODE}-${TCLIBC}${@['', '/' + \
         str(d.getVar('MACHINE', True))][bool(d.getVar('MACHINE', \
         True))]}${@['', '/' + str(d.getVar('SDKMACHINE', True))] \
         [bool(d.getVar('SDKMACHINE', True))]}"

USER_CLASSES ?= ""
PACKAGE_CLASSES ?= "package_ipk"
INHERIT_BLACKLIST = "blacklist"
INHERIT_DISTRO ?= "debian devshell sstate license"
INHERIT += "${PACKAGE_CLASSES} ${USER_CLASSES} ${INHERIT_DISTRO} \
            ${INHERIT_BLACKLIST}"
```

文件首先用默认设置包含3个其他文件：default-providers.inc、default-versions.inc和default-distrovars.inc。这些文件的名字对于文件内容提供什么是有指示性的。

文件default-distrovars.inc特别为DISTRO_FEATURES、DISTRO_FEATURES_DEFAULT、DISTRO_FEATURES_LIBC和DISTRO_FEATURES_LIBC_DEFAULT提供默认设置。如果你将在发行版策略配置文件中设置DISTRO_FEATURES，那么需要注意你没有不经意地通过覆盖变量而去除默认设置。这样做的安全方法是，使用类似以下的赋值：

```
DISTRO_FEATURES ?= "${DISTRO_FEATURES_DEFAULT} ${DISTRO_FEATURES_LIBC} \
                    ${MY_DISTRO_FEATURES}"
MY_DISTRO_FEATURES = "<distro features>"
```

其包含所有默认设置并且按需添加另外的变量以包含额外的发行版特性。

配置文件defaultsetup.conf也为TCMODE和TCLIBC设置默认值，并且包含它们各自的配置文件，正如前面所描述的。

7.5 外部层

在前面几节的例子中，我们使用了OpenEmbedded核心层meta和Yocto项目基层meta-yocto的软件包和包组。

随着对Yocto项目和OpenEmbedded的稳定增长的支持和贡献，现在已经有越来越多的额外层和数百个用于无数软件包的菜谱。它们中的很多被分类在OpenEmbedded网站上。如果你正在寻找菜谱以构建特定软件包，那么有可能的是，某个人已经做了这个工作。

OpenEmbedded网站的元数据索引[1]允许你按照层、菜谱和机器来搜索。例如，按照层来搜索Java会给你提供Java的层的列表。按照菜谱搜索JDK会给你构建JDK包的所有菜谱和提供该菜谱的层的列表。

元数据索引也允许你过滤出被支持的Yocto项目发布以看看菜谱或层是否和那个特定发布兼容。一旦你发现包含你正在寻找的软件包菜谱的层，所有你要做的便是下载层、包含其路径到你构建环境的conf/bblayers.conf的BBLAYERS变量中以及使用前面描述的方法之一增加期望的软件包到镜像。

[1] http://layers.openembedded.org。

7.6 Hob

Hob是由Yocto项目提供的用于BitBake的图形用户界面。它是Yocto项目的子项目之一，并且是由Yocto项目开发团队所维护的。

为什么它被称为Hob？在Hob的早期日子里，这3个字母代表Human-Oriented Builder（面向人的构建器）。然而，这听起来真的不够吸引人，现在工具的名字经常与hob（用于平底锅的英国英语单词）联系在一起。那很适合BitBake和菜谱的方案。

借助Hob，你可以用鼠标而不是编辑文本文件来便利地定制根文件系统镜像。如果是这样的情况，那么为什么我们不首先介绍Hob而是避易就难地解释如何构建你自己的定制Linux发行版？有以下一些原因：

·你可以用Hob做很多事情，但不是所有事情。

·Hob是BitBake和构建环境的前端。它操作你构建环境中的文件、启动BitBake并收集构建结果。理解这如何通过手工完成可以帮助你理解Hob做了什么，特别是如果某些东西出现错误的话。

·虽然Hob可能隐藏一些复杂性，但是你仍然需要知道术语和特定变量设置如何影响构建结果。

使用Hob相当简单。首先，设置构建环境，然后从它内部启动Hob：

```
$ source oe-init-build-env build
$ hob
```

Hob启动然后验证构建环境。在那个检查完成后，你会看到类似图7-1中的屏幕（我们已经选择了机器和镜像菜谱）。

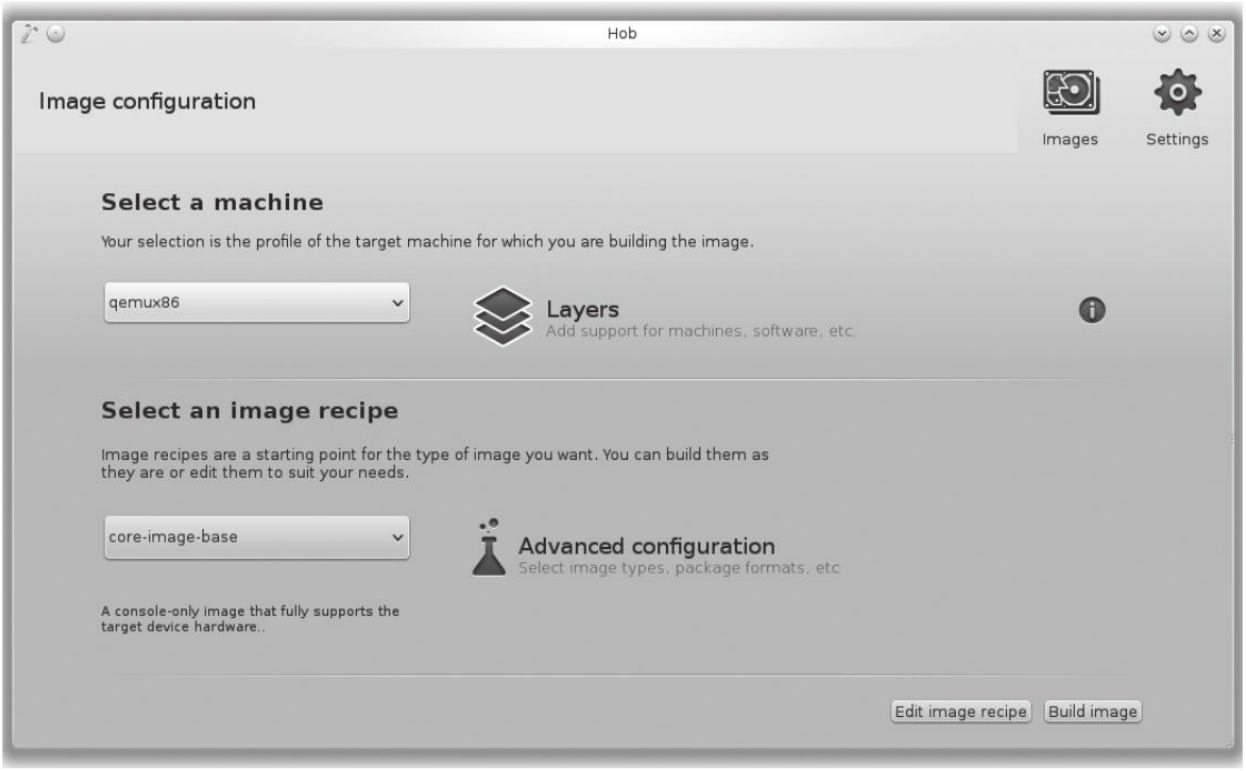

图7-1　Hob

Hob用户界面很容易理解。

·Select a machine：从下拉菜单中选择你希望为其构建的机器。列表显示所有包含在构建环境中的任何层所定义的机器。选择机器会改变在con/local.conf文件中的MACHINE变量设置。

·Layers：单击这个按钮来打开让你用构建系统包含层和从构建系统中删除它们的图形化编辑器。这样做修改了构建环境中的conf/bblayers.conf文件。

·Select an image recipe：从这个下拉菜单中，你可以选择你希望构建的镜像。与执行bitbake<image-target>类似，这向BitBake提供了镜像目标。菜单包含来自被包括在你构建环境中的所有层的镜像目标。

·Advanced configuration：单击这个按钮打开允许你选择根文件系统

类型、打包格式、发行版策略、镜像大小等的菜单，正如在7.3节和7.4节所简述的。Hob增加这些选项到构建环境的conf/local.conf文件中。

·Edit image recipe：在屏幕底部的这个按钮让你通过增加或删除包和包组来修改镜像菜谱。这样做有效地修改了镜像目标的IMAGE_INSTALL变量。然而，你不可以从Hob用户界面定义新的包组。对于那个任务，你必须按照7.1.6小节所解释的来编写自己的包组菜谱。当然，如果你编写了自己的包菜谱并且用Hob包含了其所在的层，那么你将能够从包组列表中选择它。

·Settings：在用户界面右上角的这个按钮允许你修改包含在conf/local.conf中的例如并行性、下载目录、共享状态缓存、镜像和网络代理的一般设置。使用Others标签，可以向conf/local.conf中增加任何变量并且赋值给它。

·Images：在Hob用户界面右上角靠近Settings按钮的这个按钮显示前面已构建的镜像的列表。列表是通过解析构建环境的tmp/deploy/images/<machine>子目录来创建的。你可以从列表选择镜像，如果它是QEMU镜像就运行它，否则重新构建它。

·Build image：这个按钮用已选择的配置和镜像目标启动BitBake。用户界面切换到构建视图的Log标签，从其中你可以遵循构建过程。相对于从命令行启动时的BitBake输出，这个视图有主要的优势：你不但可以看到当前正运行的任务，而且能够看到即将发生的任务和已经完成的任务。如果有任何构建问题、警告或者错误，那么它们会被记录在Issuse标签下。在那里，你可以检查构建问题并直接查看任务的完整日志文件而不用从构建环境目录结构中导航。

在构建完成后，Hob展示给你一个摘要页面，在此你可以在构建系统的文件浏览器中查看已创建的文件。你也可以检查摘要日志，该日志

显示针对每个任务的运行结果以及任何注意、警告或者错误消息的。如果你使用了Hob来为QEMU模拟器构建根文件系统镜像和Linux内核，那么你可以直接从Hob中通过单击在用户界面右下角的Run image按钮来启动QEMU以验证镜像。从摘要页面中，你也可以对配置做修改并且运行新的构建。

相对于手动地配置构建环境、定制目标镜像和启动BitBake，你是否更喜欢用Hob完全取决于你自己。对于快速原型（rapid prototyping）以及能快速使不是那么熟悉BitBake和Yocto项目的某人来构建预定义的根文件系统镜像目标来说，Hob是非常好的。Hob不允许你创建自己的镜像菜谱，你也不能用它创建自己的发行版策略文件（或者修改它们）。对于这些任务来说，你需要手动设置你自己的层并且创建必要的文件和菜谱。

从Yocto项目版本2.1以来，Hob正在被弃用而基于Web的Toaster开始变得更受欢迎，我们将在第13章中详细研究它。

7.7 总结

Linux发行版的最大组成部分是包含提供系统核心功能性的大量库和应用的用户空间。本章展示了关于Poky构建系统如何创建根文件系统镜像和你可以如何定制它们以满足需求的基础概念。

·OpenEmbedded构建系统的核心镜像提供你可以扩展和修改的发行版蓝图。

·核心镜像可以通过追加包和包组到包含在变量IMAGE_INSTALL中的列表来轻松扩展。

·QEMU模拟器是一种便利和快速的方法，可在启动实际设备上的根文件之前测试它。

·启用构建历史允许你追踪对镜像的变更以及比较构建过程的后续执行。

·创建你自己的通过包含核心镜像菜谱以在其上构建的镜像菜谱给你提供了对于根文件系统包含什么包的更多的控制。直接继承core-image类的镜像菜谱允许你从头构建根文件系统镜像。

·包组是捆绑多个包和通过单一名字引用它们的机制，它用IMAGE_INSTALL变量极大地简化了镜像定制化。Poky提供了一系列组织常见包的预定义包组。

·构建系统可以以大量输出形式生产根文件系统镜像。它们中的一些可以直接写入到例如闪存设备的存储介质中以启动系统。

·设置发行版策略允许独立于根文件系统内容的操作系统配置。它

也提供了在构建系统中使用外部工具链和改变C库的方法。

·Hob是用于BitBake的图形用户界面。从已初始化的构建环境内部被启动，它允许配置和构建根文件系统镜像而不用使用文本编辑器来修改文件。

第8章　软件包菜谱

第7章探索了如何构建你自己的Linux操作系统栈和创建根文件系统以启动它们。对于该章中的例子来说，我们使用了软件包，并为此将菜谱作为默认元数据层的一部分来提供，而默认元数据层被包含在OpenEmbedded构建系统中。默认包给了你可运行的Linux系统作为用于你自己项目的基础。它们可以如仅仅启动到交互控制台的系统一样简单和基础，也可以如带有提供用户图形界面的X窗口系统（X11）的系统一样复杂。

除非你正在为嵌入式Linux工程师构建开发板，否则最终设备会要求你增加你自己的软件包到操作系统栈。理想的情况是，你希望BitBake为你构建软件包并且能够使用在第7章中讨论的方法把它们包含在根文件系统中。

本章以展示构建软件包的菜谱的结构和描述被使用的典型元数据开始，然后我们显示如何编写直接从C文件构建的菜谱、用标准makefiles构建的菜谱、用CMake构建的菜谱以及用GNU　Autotools构建的菜谱，随后我们解释各种构建制品是如何通过使用包管理系统来被拆分成不同包的。我们以关于如何用追加文件修改现有菜谱的部分来结束本章。

8.1 菜谱布局和惯例

大部分菜谱被设计成构建软件包。在Yocto项目和OpenEmbedded惯例持续演进过程中，关于如何编写用于软件包的菜谱的指导方针和最佳实践已经被确立了。它们不是绝对的规则，并且你发现很多菜谱，特别是老旧的菜谱，不严格遵守这些指导方针。然而，社区要向着最佳实践靠拢，并且遵循它们是很有意义的。在OpenEmbedded网站[1]上，你可以发现菜谱样式指导。

[1] http://openembedded.org/wiki/Styleguide。

8.1.1 菜谱文件名

菜谱文件名遵循惯例<packagename>_<version>-<revision>.bb，其中packagename是菜谱构建的软件包的名字。下划线（_）把版本字符串和包名分隔开来，连字符（-）把版本字符串和修订分隔开来。不要在菜谱的任何其他地方使用下划线，例如用于分隔包名的部分。连字符是被允许用于包名以及包版本的，但是对于后者，是应该被避免的。包修订一定不能包含连字符。菜谱名字的例子是：

·avahi_0.6.31.bb

·linux-yocto_3.14.bb

·wpa-supplicant_2.2.bb

菜谱文件名的字段packagename、version和revision分别被BitBake赋值到变量PN、PV和PR。

当包是从软件配置管理（Software Configuration Management，SCM）系统的分支中获取的并且版本没有与标签关联时，有关菜谱名字的特定问题出现。在这个情况下，菜谱应该被命名成<packagename>_<scm>.bb，其中scm是版本系统的名字，例如git、svn或者cvs。然后，菜谱应该显式地设置PV为PV="<version>+git${SRCREV}"，其中version是最当前的发布或者标签点，并且SRCREV指向要从软件配置管理中获取的修订。遵循用于菜谱文件的命名指导方针是十分重要的，因为包名、版本和修订也被用于包管理系统。使用不正确的菜谱名可能导致包管理器在为目标根文件系统创建、安装和维护包版本时出问题。

8.1.2 菜谱布局

菜谱遵循标准布局以使菜谱文件更加可访问和更易于理解。这个布局可以被分解成数个逻辑上分组元数据的部分。我们使用来自meta/recipes-core/gettext/gettext_0.18.3.2.bb的gettext菜谱来解释结构，其显示在程序清单8-1中。

程序清单8-1　gettext菜谱gettext_0.18.3.2.bb

```
# Descriptive Meta-data
SUMMARY = "Utilities and libraries for producing multi-lingual messages"
DESCRIPTION = "GNU gettext is a set of tools that provides a framework to help
other programs produce multilingual messages. These tools include a set of
conventions about how programs should be written to support message catalogs,
a directory and file naming organization for the message catalogs themselves,
a runtime library supporting the retrieval of translated messages, and a few
standalone programs to massage in various ways the sets of translatable and
already translated strings."
HOMEPAGE = "http://www.gnu.org/software/gettext/gettext.html"

# Package Manager Meta-data
SECTION = "libs"

# Licensing Meta-data
LICENSE = "GPLv3+ & LGPL-2.1+"
LIC_FILES_CHKSUM = "file://COPYING;md5=d32239bcb673463ab874e80d47fae504"

# Inheritance Directives
inherit autotools texinfo
```

```
# Build Meta-data
DEPENDS = "gettext-native virtual/libiconv expat"
DEPENDS_class-native = "gettext-minimal-native"
PROVIDES = "virtual/libintl virtual/gettext"
PROVIDES_class-native = "virtual/gettext-native"
RCONFLICTS_${PN} = "proxy-libintl"
SRC_URI = "${GNU_MIRROR}/gettext/gettext-${PV}.tar.gz \
           file://parallel.patch \
          "

PACKAGECONFIG[msgcat-curses] = "\
    --with-libncurses-prefix=${STAGING_LIBDIR}/..,\
    --disable-curses,ncurses,"

LDFLAGS_prepend_libc-uclibc = " -lrt -lpthread "

SRC_URI[md5sum] = "241aba309d07aa428252c74b40a818ef"
SRC_URI[sha256sum] =
"d1a4e452d60eb407ab0305976529a45c18124bd518d976971ac6dc7aa8b4c5d7"

EXTRA_OECONF += "--without-lispdir \
                 --disable-csharp \
                 --disable-libasprintf \
                 --disable-java \
                 --disable-native-java \
                 --disable-openmp \
                 --disable-acl \
                 --with-included-glib \
                 --without-emacs \
                 --without-cvs \
                 --without-git \
                 --with-included-libxml \
                 --with-included-libcroco \
                 --with-included-libunistring \
                "
acpaths = '-I ${S}/gettext-runtime/m4 \
           -I ${S}/gettext-tools/m4'

# Packaging Meta-data
# these lack the .x behind the .so, but shouldn't be in the -dev package
# Otherwise you get the following results:
# 7.4M    Angstrom-console-image-glibc-ipk-2008.1-test-20080104-ep93xx.rootfs.tar.gz
# 25M     Angstrom-console-image-uclibc-ipk-2008.1-test-20080104-ep93xx.rootfs.tar.gz
# because gettext depends on gettext-dev,
# which pulls in more -dev packages:
# 15228   KiB /ep93xx/libstdc++-dev_4.2.2-r2_ep93xx.ipk
# 1300    KiB /ep93xx/uclibc-dev_0.9.29-r8_ep93xx.ipk
# 140     KiB /armv4t/gettext-dev_0.14.1-r6_armv4t.ipk
# 4       KiB /ep93xx/libgcc-s-dev_4.2.2-r2_ep93xx.ipk

PACKAGES =+ "libgettextlib libgettextsrc"
FILES_libgettextlib = "${libdir}/libgettextlib-*.so*"
FILES_libgettextsrc = "${libdir}/libgettextsrc-*.so*"

PACKAGES =+ "gettext-runtime gettext-runtime-dev gettext-runtime-doc"

FILES_${PN} += "${libdir}/${BPN}/*"

FILES_gettext-runtime = "${bindir}/gettext \
                         ${bindir}/ngettext \
                         ${bindir}/envsubst \
                         ${bindir}/gettext.sh \
                         ${libdir}/libasprintf.so* \
```

```
                          ${libdir}/GNU.Gettext.dll \
                        "
FILES_gettext-runtime_append_libc-uclibc = " ${libdir}/libintl.so.* \
                                             ${libdir}/charset.alias \
                                           "
FILES_gettext-runtime-dev += "${libdir}/libasprintf.a \
                      ${includedir}/autosprintf.h \
                    "
FILES_gettext-runtime-dev_append_libc-uclibc = " ${libdir}/libintl.so \
                                                 ${includedir}/libintl.h \
                                               "
FILES_gettext-runtime-doc = "${mandir}/man1/gettext.* \
                             ${mandir}/man1/ngettext.* \
                             ${mandir}/man1/envsubst.* \
                             ${mandir}/man1/.* \
                             ${mandir}/man3/* \
                             ${docdir}/gettext/gettext.* \
                             ${docdir}/gettext/ngettext.* \
                             ${docdir}/gettext/envsubst.* \
                             ${docdir}/gettext/*.3.html \
                             ${datadir}/gettext/ABOUT-NLS \
                             ${docdir}/gettext/csharpdoc/* \
                             ${docdir}/libasprintf/autosprintf.html \
                             ${infodir}/autosprintf.info \
                            "

# Task Overrides, Prepends and Appends
do_install_append() {
    rm -f ${D}${libdir}/preloadable_libintl.so
}
do_install_append_class-native () {
        rm ${D}${datadir}/aclocal/*
        rm ${D}${datadir}/gettext/config.rpath
        rm ${D}${datadir}/gettext/po/Makefile.in.in
        rm ${D}${datadir}/gettext/po/remove-potcdate.sin
}

# Variants / Class Extensions
BBCLASSEXTEND = "native nativesdk"
```

我们稍微重新格式化了这个菜谱，并且包含了注释以概述我们正在讨论的元数据部分。gettext是包含你在菜谱中可能遇到的大部分元数据的相当复杂的菜谱。然而，不是所有菜谱都是那么复杂。大部分实际上是相当简单的，因为构建它们各自软件包完全可以被各种类不加任何太大修改地完成。

接下来的讨论解释菜谱的部分和它们通常包含的元数据。

描述性元数据

描述性元数据（Descriptive Metadata）提供关于菜谱和它构建的软件包的信息。

·SUMMARY（摘要）：一行（最多80个字符长度），包的简短描述。

·DESCRIPTION（描述）：扩展的（可能多行长度），包和其所提供的东西的细节描述。

·AUTHOR（作者）：以AUTHOR="Santa Claus<santa@northpole.com>"形式的软件包（不是菜谱）的作者的名字和邮箱地址。这可以是多个作者的列表。

·HOMEPAGE（主页）：以http://开头的URL，软件包被托管在这里。

·BUGTRACKER（缺陷追踪）：以http://开头的URL，指向项目的缺陷追踪系统。

包管理器元数据

在这一部分的元数据为包管理系统提供主要用于包数据库（package database）维护的额外信息。然而，不是所有的包管理系统都支持这些设置。

·SECTION（部分）：这个包所归属的类别。包管理工具使用这个类别来组织包。虽然类别未被严格标准化并且各种主流Linux发行版定义它们自己的列表，但是通常被使用的类别已经演化出来了。常用部分或者类别的例子是app、audio、base、devel和libs。

·PRIORITY（优先级）：用优先级来告诉包管理工具软件包是否被

系统运行需要，以及软件包是否是可选的或者最终是与其他包冲突的。优先级仅仅被Debian包管理器dpkg和开放包管理器opkg所利用。优先级是：

·standard（标准的）：对任何Linux发行版来说都是标准的包，包括一个相当小的但是也不太限制于控制台模式的系统。

·required（必要的）：对系统的正常功能来说所必要的包。

·optional（可选的）：对可运行的系统来说不是必要的包，而是为了合理可用的系统的包。

·extra（额外的）：可能与来自更高优先级的其他包冲突或者有特殊需求的包。

许可元数据

这部分的元数据允许构建系统自动追踪开源许可需求。这个信息对所有包都是强制性的。我们将在第12章中处理Yocto项目许可管理的细节。

·LICENSE（许可）：用于这个软件包的许可（或者多个许可）的名字。在大部分情况下，仅有一个许可适用，但是一些开源软件包使用多个许可。这些可以是允许包的用户从数个许可中选择一个的双许可或者是在软件包的部分授权不同的多许可。双许可是通过以管道符号（|）连接许可名字来指定的。多许可是通过以&连接许可名字来指定的。构建系统也支持复杂的逻辑许可算术，例如GLv2&（LGPLv2.1|MPL-1.1|BSD）。

·LIC_FILES_CHECKSUM：这个变量允许追踪对许可文件本身的变更。变量包含、许可文件以及它们各自校验和的空格分隔列表。在获取

并解压了软件包的源文件后，构建系统通过对许可文件或者许可文件的部分计算校验和，以及将其与所提供校验和进行比较来验证许可。

继承指令和包含

这部分包含用于菜谱从类继承功能性的继承指令。它也包括包含（include）和要求（require）语句以在语句的位置上直接插入其他文件。在菜谱中的位置对于继承没有关系，但是对于包含文件，它可能是重要的。被包含的文件可以设置你可能希望在菜谱中覆盖的变量。

构建元数据

我们把这部分的元数据叫作构建元数据，因为它提供对于构建软件包所需的设置，例如URI、声明依赖性和定义提供（provisions）。

·PROVIDES（提供）：通常用于抽象配置的一个或者多个额外包名的空格分隔列表。

·DEPENDS（依赖）：在这个包可以被构建前必须被构建的包名的空格分隔列表。

·PN：包名。这个变量的值是由BitBake从菜谱文件的基名称获取的。对大部分包来说，这是正确且足够的。一些包可能需要调整这个值。例如，交叉工具链应用——例如gcc-cross，在它们的名字上被追加了目标架构。

·PV：包版本，它是由BitBake从菜谱文件的基名称获取的。对于除了直接从源仓库构建的包来说，这个值是正确且足够的。对于那些从软件配置管理中构建的包来说，8.1.1小节解释了如何正确设置PV。

·PR：包修订。默认修订是r0。在过去，BitBake要求你在每次菜谱

自己已经改变了的时候增加修订以触发重新构建。然而现在，新的签名处理器计算包含函数的菜谱元数据的签名。现在，构建系统完全依赖于签名来重新构建。

为了正确的包命名，增加PR的值可能仍然是必需的，因为其使得包管理器可以正确地为包升级维护数据库。以前，维护是通过使用PRINC变量来完成的。然而，这个方法已经证明是易于出错的，所以PRINC已经被弃用并且被PR service所替代。PR service是基于签名来计算PR的修订服务器[1]。

·SRC_URI：用于从其下载源代码、补丁和其他文件的URI空格分隔列表。

·SRCDATE：源代码日期。这个变量仅仅当源是从软件配置管理中获取而来时才适用。

·S：在构建环境中的、构建系统把未解压的源代码放在其中的目录位置。默认位置依赖于菜谱名和版本：${WORKDIR}/${PN}-${PV}。默认位置对于几乎所有从文件包中构建的包来说都是合适的。对于直接从软件配置管理中构建的包来说，你需要显式地设置这个变量，例如用于GIT仓库的${WORKDIR}/git。

·B：在构建环境中的、构建系统把在构建中创建的对象放入其中的目录位置。默认是与S相同的：${WORKDIR}/${PN}-${PV}。很多软件包是在树（in tree）中或者在位置（in location）中被构建的，把对象放在源树中。用GNU Autotools来构建包的菜谱、Linux内核和交叉工具链应用分离源和构建目录。

·FILESEXTRAPATHS：扩展用于由FILESPATH定义的额外本地文件的构建系统的搜索路径。这个变量最常以

FILESEXTRAPATHS_prepend：="${THISDIR}/${PN}"形式被用于追加文件中，它导致构建系统在由FILESEXTRAPATHS指定的其他目录中查找之前先在带有追加文件存在于其中的目录的包名的子目录中寻找额外文件。

·PACKAGECONFIG：这个变量允许在构建时启用和禁用软件包的特性。你用以下形式的四倍列表（quadruples of lists）定义特性：

```
PACKAGECONFIG[f1] = "--with-f1,--wo-f1,build-deps-f1,rt-deps-f1"
PACKAGECONFIG[f2] = "--with-f2,--wo-f2,build-deps-f2,rt-deps-f2"
PACKAGECONFIG[f3] = "--with-f3,--wo-f3,build-deps-f3,rt-deps-f3"
```

四倍是用逗号（,）分隔的。它们的顺序很重要：

1.如果特性是被启用的，额外参数加到configure脚本（EXTRA_OECONF）的配置列表。

2.如果特性是被禁用的，额外参数加到EXTRA_OECONF。

3.如果特性是被启用的，额外构建依赖性加到DEPENDS。

4.如果特性是被启用的，额外运行时依赖性加到RDEPENDS。

为了启用特性，你可以创建追加文件或者在配置文件中这样做：

·追加文件：在你自己的层中创建追加文件并且命名它为<packagename>.bbappend，其中packagename是你希望追加的菜谱名，然后你可以通过用PACKAGECONFIG="f2 f3"覆盖它们来完全重新定义变量或者你可以用PACKAGECONFIG_append="f2 f3"来保留原来的设置值。

·配置文件：你可以简单地使用PACKAGECONFIG_pn-<packagename>="f2 f3"或者PACKAGECONFIG_append_pn-

<packagename>="f2 f3"来增加变量到如local.conf的配置文件或者发行版配置文件。

这两个方法在结果上是完全相同的。

·EXTRA_OECONF：额外的configure脚本选项。

·EXTRA_OEMAKE：用于GNU Make的额外选项。

·EXTRA_OECMAKE：用于CMake的额外选项。

·LDFLAGS：传到链接器的选项。默认设置依赖于构建系统在构建什么：当为目标构建时，它是TARGET_LDFLAGS；当为构建主机构建时，它是BUILD_LDFLAGS；当为主机构建SDK时，它是BUILDSDK_LDFLAGS。你通常不完全覆盖这个变量而是向其增加选项。

·PACKAGE_ARCH：定义软件包的架构。默认情况下，当为目标构建时，这个变量被设置成TUNE_PKGARCH；当为构建主机构建时，这个变量被设置成BUILD_ARCH；当构建SDK时，这个变量被设置成"${SDK_ARCH}-${SDKPKGSUFFIX}"。默认值通常是足够的，除非你的软件包完全依赖于特定的机器而不是依赖于机器的架构。在这种情况下，要在菜谱中设置PACKAGE_ARCH="${MACHINE_ARCH}"。

打包元数据

菜谱的这个元数据部分定义构建输出如何使用包管理器被打包进不同的包中。打包发生在软件已经被构建并且被安装进包构建目录本地的根文件系统结构之后。我们在此介绍变量并且在接下来的部分讨论细节。

·PACKAGES：这个变量是在打包过程中被创建的包的空格分隔列表。这个变量的默认值是"${PN}-dbg${PN}-staticdev${PN}-dev${PN}-doc${PN}-locale\${PACKAGE_BEFORE_PN}${PN}"。这个列表是以从左到右的方式来被处理的，意味着，最左侧的包首先被创建，最右侧的包最后被创建。顺序是重要的，因为包消耗与它相关联的文件。如果两个或者更多包消耗相同的文件，那么仅仅第一个被处理的包包含文件。菜谱通常把额外包增加到列表的前部。

·FILES：FILES变量定义被放进特定包中的目录和文件的列表。构建系统为实际的包定义默认文件和目录列表，例如FILES_${PN}-dbg="<files>"，其中files是空格分隔的可以包含通配符的目录和文件列表。如果菜谱增加额外包到PACKAGES中的列表，那么你需要为那个包定义FILES。菜谱可能产生通常未在默认包中被发现的对象，你希望将其增加到标准包。在那个情况下，你要用带有那些文件的列表追加到FILES。

·PACKAGE_BEFORE_PN：这个变量让你在最终的包名被创建前简单地增加包。你可以简单地看到其如何工作。在PACKAGES的默认列表中，PACKAGE_BEFORE_PN变量的内容在最终的包PN之前被扩展。简单地增加包到变量PACKAGE_BEFORE_PN="${PN}-examples"。当然，你也需要为examples包定义FILES列表。

·PACKAGE_DEBUG_SPLIT_STYLE：这个变量决定当${PN}-dgb包被创建时如何拆分二进制和调试对象。有3个变体：

·".debug"：在目标上，包含调试符号的文件被放在安装有二进制的目录内的.debug目录中。例如，如果二进制被安装在/usr/bin中，那么调试符号文件被置于/usr/bin/.debug。这个选项也在.debug中安装源文件，它是默认行为。

·"debug-file-directory"：在目标上，调试文件被放在/usr/lib/debug下面，将其和二进制分开。

·"debug-without-src"：这个变体和.debug一样，但是源文件不被安装。

·PACKAGESPLITFUNCS：这个变量定义执行包拆分的函数的列表。由package.bbclass定义的默认是PACKAGESPLITFUNCS?="package_do_split_locales populate_packages"。菜谱可以前新增到这个变量以在该默认被运行前运行它们自己的包分离函数。

任务覆盖、前新增和追加

在这一部分，菜谱覆盖任务、向任务前新增（prepend）和追加以重新定义、变更或者扩展默认行为。

变体/类扩展

这个部分简单地包含BBCLASSEXTEND变量以创建例如包的原生或者软件开发工具包构建的变体。

运行时元数据

这个元数据部分定义运行时依赖性。

·RDEPENDS：该包在运行时所依赖的，为了该包正确运转而必须被安装的包列表。变量适用于正在被构建的包，因此对特定包，你需要条件性地定义它。例如，如果开发包依赖Perl以正确运行，那么你需要指定RDEPENDS_${PN}-dev+="perl"，它告诉构建系统在包管理器的程序清单中创建这个包依赖性。

·RRECOMMENDS：与RDEPENDS类似但是指示弱依赖性，因为

这些包对于正在被构建的包不是必不可少的。然而，它们确实增加可使用性。如果这些包是可用的，那么包管理器安装它们；但如果这些包是不可用的，那么包管理器也不失败。

·RSUGGESTS：与RRECOMMENDS类似，但是从即使这些包是可用的但是包管理器也不安装它们的这个意义上来说，其甚至更加弱。它们仅仅提供这样的信息——安装这些包可能是有好处的。

·RPROVIDES：用于运行时配置的包名别名列表。包的自有名字总是隐式地作为那个列表的一部分。正如对于控制包创建的所有运行时元数据一样，你需要使用条件赋值：RPROVIDES_${PN}="alias1 alias2"。

·RCONFLICTS：冲突包的名字列表。如果在安装前不是所有冲突包都被移除了，那么包管理器不安装这个包。正如对于控制包创建的所有运行时元数据一样，你需要使用条件赋值：RCONFLICTS_${PN}="conflicting-package-name"。

·RREPLACES：这个包替代的包的名字列表。包管理器使用这个变量来决定这个包替换其他哪些包。如果包可以共存，那么包管理器安装这个包，即使在这个列表的其他包是被安装的。如果包不能共存，那么该包必须也设置RCONFLICTS变量来包含那些包。正如对于控制包创建的所有运行时元数据一样，你需要使用条件赋值：RCONFLICTS_${PN}="conflicting-package-name"。构建系统支持版本化的依赖性：

```
RDEPENDS_${PN} = "<package> (<operator> <version>)"
```

这里的操作符是=、<、>、<=或者>=之一。例如，

```
RDEPENDS_${PN} = "gettext (> 0.16)"
```

你可以在RDEPENDS、RRECOMMENDS、RSUGGESTS、RCONFLICTS和RREPLACES中使用版本化的依赖性。

[1] 默认情况下，PR service是被禁用的。关于更多信息以及如何启用它，请参见在https://wiki.yoctoproject.org/wiki/PR_Service的PR service wiki。

8.1.3 格式指导方针

用于源代码、BitBake菜谱和类的格式指导方针本质上是源代码。它们的目的是跨所有制品创建一致的格式和外观以便某个正在尝试工作于Yocto项目和OpenEmbedded的人可以快速学习和理解。指导方针也简化审查维护者的贡献。

OpenEmbedded已经建立了样式指导[1]，它确立了用于格式菜谱、类和配置文件的基础规则：

赋值

·在赋值操作符的每一侧使用单一的空格。

·仅仅在赋值的右手侧使用引号：

```
VARIABLE = "VALUE"
```

延续

·为了更好的可阅读性，延续被用来拆分长的变量列表，例如SRC_URI。

·使用行延续符号（\）。

·不在行延续符号后面放置任何空格。

·缩进连续的行到值开头的水平。

·为缩进使用空格替代tab，因为开发者倾向于不相同的设置它们的tab大小。

·把结束的引号放在它自己的行上。

```
VARIABLE = "\
            value1 \
            value2 \
           "
```

Python函数

·每个缩进使用4个空格；不使用tab。

·Python对于缩进是相当挑剔的。永远不要混用空格和tab。

Shell函数

·每个缩进使用4个空格；不使用tab。

·一些层，例如OECore，为shell函数使用tab用于缩进。然而，推荐你为新层使用4个空格以和Python函数保持一致。

注释

·在菜谱、类和配置文件中，注释是被允许并且被鼓励的。

·注释必须在一行的开头使用#字符开始。

·注释不能被用在延续中。

即使你不打算向OpenEmbedded或者Yocto项目提交补丁或者贡献菜谱或层，遵循这些简单的指导方针也依旧可使为你和你的组织维护你自己的菜谱、类和配置文件变得更加简单。

[1] http://openembedded.org/wiki/Styleguide。

8.2 写新菜谱

当工作在OpenEmbedded构建系统时，为软件包写新菜谱并且增加它到构建是必不可少的任务。一开始它可能是令人气馁的，但是它不像它看起来的那样困难。大部分复杂性是由各种类来解决的。图8-1显示了一步步写菜谱的方法。

用于创建菜谱的工作流相当类似于我们在第3章中所讨论过的、用于构建包的BitBake过程。当然，那不是巧合。当创建新菜谱时，你一步一步为每个过程步骤增加必要元数据。工作流也不是像描述的那样线性，而通常是做出增加或者变更然后测试变更的迭代。因为前面步骤的结果影响后续步骤，所以使得每个步骤正确是重要的。有时，运行过程步骤（例如编译）的问题，可能相关于前面步骤中的问题，例如configure。

在写菜谱之前，我们推荐你寻找别人已经写了的并且满足或者至少接近满足需求的菜谱。OpenEmbedded元数据索引[1]是用于搜索的好起点。你可能找到你可以为了符合需求而修改的菜谱。然而，如果你正在为你自己的软件包写菜谱，那么这个方法可能不好使。在这种情况下，你从既定菜谱开始并且尝试修改它可能比从头或者从菜谱骨架创建菜谱需要更多的努力。

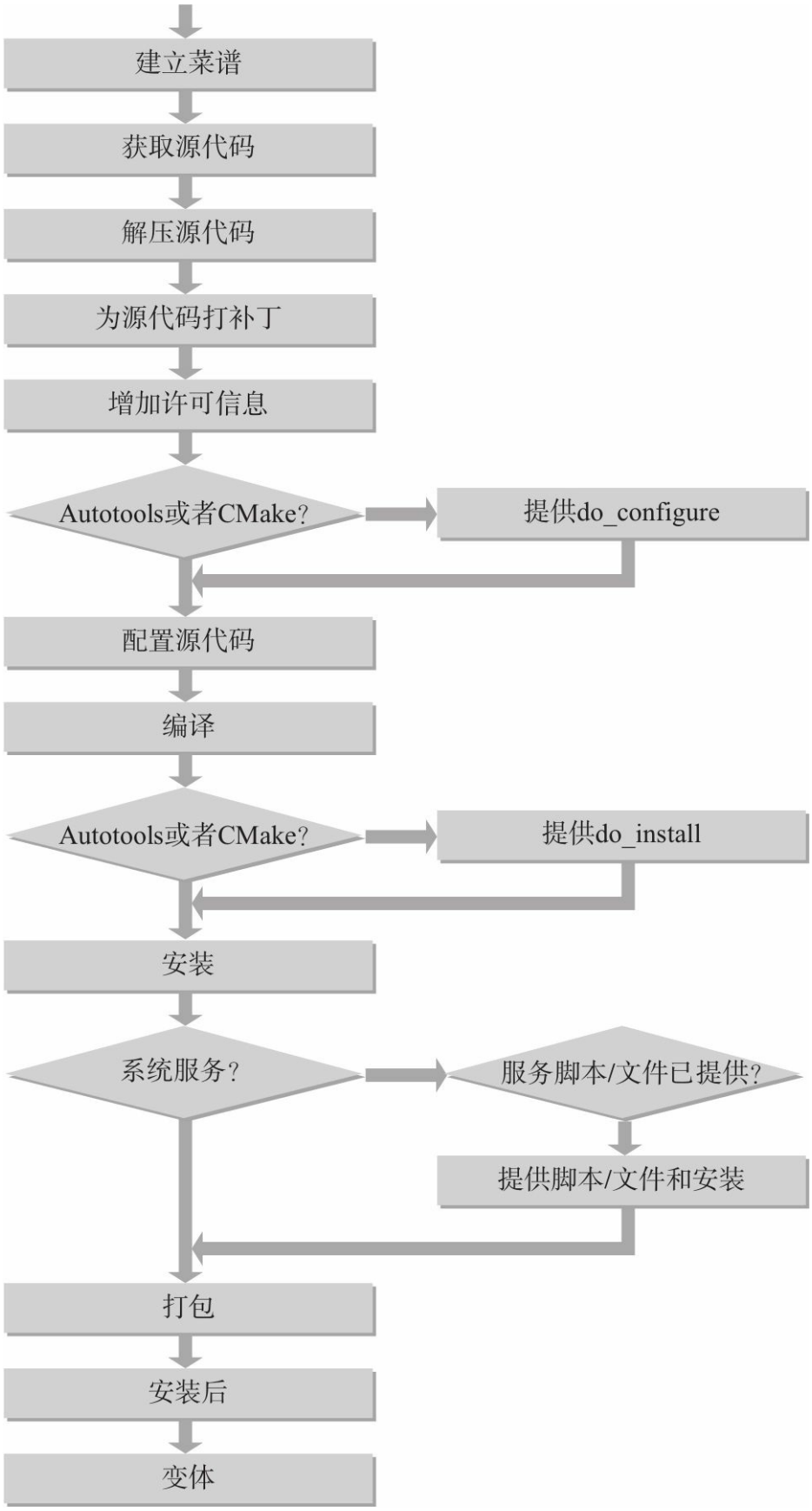
建立菜谱
获取源代码
解压源代码
为源代码打补丁
增加许可信息
Autotools或者CMake?
提供do_configure
配置源代码
编译
Autotools或者CMake?
提供do_install
安装
系统服务?
服务脚本/文件已提供?
提供脚本/文件和安装
打包
安装后
变体

图8-1　菜谱创建工作流

接下来的部分详细描述用于从头创建菜谱的工作流步骤。

[1] http://layers.openembedded.org。

8.2.1 建立菜谱

即使你仅仅正在创建单一菜谱，我们也总是推荐你把它放进你自己的层中或者为它创建新的层。为什么？几乎没有项目保持简单。大部分项目随着时间而增长，因为你要增加它们。通过使用你自己的层来建立结构极大地简化了将来的维护。如果你还没有层来放入菜谱，那么你可以简单地使用yocto-layer工具来创建：

```
$ yocto-layer create mylayer
```

该工具交互式地引导你创建层并且也为你准备好创建样例菜谱。你可以从样例菜谱或者从显示在程序清单8-2中的骨架开始。

程序清单8-2　骨架菜谱

```
SUMMARY = ""
DESCRPTION = ""
AUTHOR = ""
HOMEPAGE = ""
BUGTRACKER = ""
```

在层内创建骨架菜谱。BitBake按照你层内conf/layer.conf文件中的BBFILES变量设置来在层内定位菜谱。BBFILES定义了用于菜谱文件的搜索模式。它通常被设置成：

```
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"
```

当然，你可以为你自己的层改变搜索模式，但是使用以上的默认是有意义的，因为所有OpenEmbedded/Yocto项目层都使用它。因此，yocto-layer工具创建具有这个用于BBFILES的设置的conf/layer.conf。

例如，如果你正在为名为myapp的应用创建菜谱，那么你将希望把

菜谱放在目录recipes-apps/myapp中。

菜谱名必须遵守在8.1.1小节讨论的命名惯例并且应该至少是<packagename>_<version>.bb，例如myapp_1.0.bb。

8.2.2 获取源代码

菜谱做的第一件事是获取源代码。因此，菜谱必须提供SRC_URI以告诉构建系统从哪里获取源以及使用什么协议。获取是通过do_fetch任务来执行的。

对于大部分典型开源上游下载来说，源文件是作为压缩包来提供的。这些包可以通过使用标准文件传输协议来获取，正如我们在第3章中详细讨论BitBake获取器时看到的。例如，

```
SRC_URI = "http://ftp.gnu.org/gnu/hello/hello-2.9.tar.gz"
```

为GNU Hello World程序获取源包。然而，在SRC_URI中使用硬编码的版本号使得菜谱更不可移植。作为替代，推荐使用PV：

```
SRC_URI = "http://ftp.gnu.org/gnu/hello/hello-${PV}.tar.gz"
```

当获取源包时，构建系统要求你提供MD5或SHA256校验和来验证包是否已经被正确下载了。你可以直接指定校验和为SRC_URI的一部分，但是由于这些校验和往往有些不便的，所以我们推荐你在SRC_URI中给包一个名字，并且独立指定校验和：

```
SRC_URI = "http://ftp.gnu.org/gnu/hello/hello-${PV}.tar.gz;name=archive"
SRC_URI[archive.md5sum] = "67607d2616a0faaf5bc94c59dca7c3cb"
SRC_URI[archive.sha256sum] = \
"ecbb7a2214196c57ff9340aa71458e1559abd38f6d8d169666846935df191ea7"
```

这样做，使得SRC_URI更易于维护并且当SRC_URI包含多个URI时也工作得更好。

直接从软件配置管理中获取源代码也是常见的实践。构建系统几乎支持所有常见软件配置管理系统。当前最被频繁用于开源软件的软件配

置管理是Git。从软件配置管理中获取不要求校验和但是通常要求要检出的修订和标签：

```
SRC_URI = "git://git.lttng.org/lttng-ust.git;branch=stable-2.5 \
           file://lttng-ust-doc-examples-disable.patch \
          "
SRCREV = "ce59a997afdb7dc8af02b464430bb7e35549fa66"
```

很多菜谱也要求文件被直接从构建主机本地获取。通常，它们是集成补丁、配置文件等：

```
SRC_URI = "git://git.lttng.org/lttng-tools.git;branch=stable-2.5 \
           file://runtest-2.4.0.patch \
           file://run-ptest \
          "
```

使用file：//协议说明符命令构建系统在构建主机上寻找文件。路径是相对于由FILESPATH指定的路径的。变量通常包含多个路径。首个匹配文件的路径被使用。默认情况下，在FILESPATH中的目录列表包含BP（基包名和版本）、BPN（基包名）和files。所有这些都被假设成是菜谱所在目录的子目录。

8.2.3 解压源代码

在获取后，源代码需要被解压。仅仅当源代码作为源包被下载时解压才是需要的。如果它是从软件配置管理中获取的，那么解压不是必要的，因为文件可以被一一从仓库中检出。任务do_unpack处理解压。它几乎可以处理所有常见打包和压缩方案。

源代码被解压进由变量S指定的目录中。构建系统期望源包包含目录树，目录树具有单一的、名为${BP}或者${BPN}-${PV}的顶级或者根目录。使用GNU Hello的例子，hello-2.9.tar.gz包必须解压到单一的、名为hello-2.9的目录中。这是大部分开源包遵循的惯例。对于不是这种情况的包，你需要在菜谱中显式地设置S。例如，对于没有顶级目录的包，你需要设置S为：

```
S = "${WORKDIR}/${BP}"
```

你需要精确地设置S为什么，依赖于源代码是如何被打包的。可能最终必须要追加do_unpack任务来重命名或者复制目录或者复制以及移动目录内容。

如果菜谱正在从软件配置管理中获取源代码，那么你也需要显式地设置S。在这种情况下，S通常被设置成${WORKDIR}/<scm>，其中<scm>是软件配置管理的名字。对于Git，你设置：

```
S = "${WORKDIR}/git"
```

在你设置了用于获取和解压的新菜谱以后，运行它并且验证源已被正确下载和解压是好主意：

```
$ bitbake -c fetch <myrecipe>
$ bitbake -c unpack <myrecipe>
```

让你分别运行获取和解压任务。如果你仅仅运行解压任务而获取任务还没有被运行，那么BitBake会自动运行它，因为解压依赖于获取。

获取任务已经完成后，你应该在由DL_DIR指定的下载目录中找到源包。对于来自软件配置管理的获取，下载目录包含具有软件配置管理名字且远程仓库已经被克隆或者检出到那里的子目录。

在你运行了解压任务后，你应该能够在源目录S中找到已解压的源。

8.2.4 为源代码打补丁

如果SRC_URI包含任何补丁，也就是以.patch或者.diff结尾的文件，那么do_patch任务使用Quilt工具自动应用这些补丁到已解压的源代码。

构建系统期望补丁可以用-p1选项来应用，这意味着第一级目录是被除去的。如果补丁需要多级目录被除去，那么你需要提供striplevel选项和数字作为补丁SRC_URI的一部分。如果补丁需要被应用进特定的、不是在补丁文件本身内提供的子目录中，那么你增加patchdir选项到SRC_URI：

```
SRC_URI = "http://downloads.tld.com/pub/package-1.2.tar.gz \
           file://patch-1.patch;striplevel=3 \
           file://patch-2.diff;patchdir=src/common \
          "
```

你应该把补丁和任何其他本地获取的文件放在菜谱旁边的子目录中，子目录被命名成和菜谱的基名称BPN相同，或者被命名成基名称加版本号BP，或者被命名成files。例如，如果菜谱的名字是foo_2.3.1.bb，那么目录的名字可以是foo、foo-2.3.1和files其中之一。使用foo而不是files有助于为不同菜谱组织补丁。使用foo-2.3.1而不是foo允许为不同版本的foo菜谱分开补丁。

8.2.5 增加许可信息

所有菜谱都不要求通过设置变量LICENSE和LIC_FILES_CHKSUM来提供许可信息。这两个变量都是强制的；否则，构建系统拒绝菜谱并且不构建它。

·LICENSE：用于这个软件包的许可名字。在大部分情况下，仅有单一许可适用，但是一些开源软件包使用多个许可。这些可以是允许包用户从数个许可中选择其一的双许可或者是在其中软件包的部分授权不同的多许可。双许可是通过以管道符号（|）连接许可名字来指定的。多许可是通过以&连接许可名字来指定的。构建系统也支持复杂的逻辑许可算术，例如GLv2&（LGPLv2.1|MPL-1.1|BSD）。

·LIC_FILES_CHKSUM：这个变量允许追踪对许可文件本身的变更。变量包含许可文件以及它们各自校验和的空格分隔列表。在获取并解压了软件包的源文件后，构建系统通过对许可文件或者许可文件的部分计算校验和，以及将其与所提供校验和进行比较来验证许可。

由LIC_FILES_CHKSUM列出的许可可以有任何名字，只要名字不含有任何空格或者字符&、|、“(”和“)”。对于标准许可，我们推荐你使用meta/files/common-licenses中的许可文件名或者来自meta/conf/licenses.conf中的SPDXLICENSEMAP属性的许可名。

如果源包在文件（例如COPYING、LICENSE或者类似文件）中提供实际的许可，那么你应该在LIC_FILES_CHKSUM中指定它：

```
LIC_FILES_CHKSUM = "file://COPYING;md5=<md5sum>"
```

一些使用标准开源许可的软件包可能简单地陈述名字和许可的版本

但不包含许可文件本身。在这种情况下，使用在COMMON_LICENSE_DIR中提供的许可文件：

```
LIC_FILES_CHKSUM = \
    "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"
```

如果你没有许可文件的MD5校验和，那么你不需要手动计算它。仅仅把md5参数留空，构建系统以提供校验和的错误消息退出，然后你可以复制和粘贴这个校验和进菜谱中。

8.2.6 配置源代码

构建软件包通常依赖于构建系统和构建环境，例如开发工具以及对其他软件包和库的依赖性。这尤其发生在为不同架构交叉构建时。为了保证源包是可移植的以便它们可以在各种具有不同构建环境的构建系统上为大量目标系统构建，开发者提供用于配置源代码和它的构建系统的方法。

在最简单的情况下，这个配置是通过设置环境变量来实施的，源代码构建系统将该变量用于它的makefiles和构建脚本。不幸的是，这个方法是非常不用户友好的，因为，为了能够提供正确的设置，希望构建软件包的人需要理解变量和他们打算做的事。不同开发者也倾向于以不同方式命名变量，这增加了复杂度。虽然有一些达成一致的公分母——例如，作为用于C编译器变量的CC、用于C++编译器的CPP和用于链接器的LD——但是开发者仍然可能定义你可能需要解决的其他变量。

为了缓和问题，构建系统被开发成使源代码以标准化的方式可移植。两个常见的构建系统是GNU Autotools和CMake。这两者都旨在通过为源代码配置提供自动化工具来简化源代码的移植并使配置用户友好。这些工具通过将随源代码一起交付的配置文件和实际构建系统进行匹配并构建环境配置来决定必要的配置，然后基于输出设置需要的变量。

作为第一个步骤，你需要决定你正在为其编写菜谱的包依赖于哪些其他软件包。通常，软件包的文档应该提供给你这个信息。你需要增加这些包到DEPENDS变量以便BitBake可以首先构建这些包来满足依赖性。

第二，你需要决定你正在构建的软件包是否使用源配置系统。

·GNU Autotools：如果软件包包含configure.ac文件，那么它使用GNU Autotools。configure.ac文件包含用于检测编译器、寻找特定头文件、测试库等的宏（macros）列表。autoconf工具从这个名为configure的文件创建一个shell脚本然后执行实际的测试。GNU Autotools构建在传统的Make原生系统之上，并用于构建软件包。在configure脚本已经成功完成后，它创建makefile。

如果软件包使用GNU Autotools，那么所有你要做的事情是继承autotools类。这个类提供do_configure任务，并且在大部分情况下，你不需要进一步修改配置。

然而，一些软件包可能依然要求你对配置做调整。为了这个目的，OpenEmbedded构建系统提供允许你增加或者覆盖配置设置的变量EXTRA_OECONF。

·CMake：如果软件包包含CMakeLists.txt文件，那么它使用CMake。这个文件包含配置指令。CMake和例如Make的原生构建系统结合使用。

如果软件包使用CMake，那么所有你必须做的事情是继承cmake类。这个类提供do_configure任务，并且在大部分情况下，你不需要进一步修改配置。

如果你需要对配置做调整，那么OpenEmbedded提供用于传递任何需要的配置选项的EXTRA_OECMAKE变量。

·Other：如果源包不包含configure.ac或者CMakeLists.txt文件，那么它正在通过直接传递环境变量到构建系统来使用某个其他的配置方法。如果是这种情况，你可能必须编写你自己的do_configure任务来应用正确的设置。在简化的情况下，设置EXTRA_OEMAKE变量以传递设置是

足够的。

此时，你应该运行configure任务并且检查它的日志文件以保证配置成功了并且正确的选项已经被传递了。日志也告诉你是否你缺失DEPENDS变量中所需要的任何依赖项或者是否你可能不希望启用的选项已经被启用了。

8.2.7 编译

现在菜谱已经就绪于编译源代码。运行compile任务并且看看编译是否成功了。如果成功了，那么对这一步来说，你没有任何其他要做的了。然而，如果编译失败，那么你需要分析日志文件以得到根源。此时，最常见的失败是并行构建失败（parallel build failure）、主机泄露（host leakage）、缺失头文件或者库：

·并行构建失败：为Make使用多线程默认是由你构建环境的conf/local.conf中的PARALLEL_MAKE为所有包都启用的。有时，这可能导致竞态条件（race condition），它表现为间歇性失败（intermittent failure）。通常，编译失败是因为应该已经被创建的制品不能被找到。这些失败难以追踪，因为在检查时，你可能发现制品确实已经被创建了；然而，问题是，构建过程的部分已经以错误的顺序执行了。

作为变通方法，你可以在菜谱中设置PRALLEL_MAKE=""（空字符串）。这个设置仅仅为这个菜谱关闭并行构建。当然，它减慢构建软件包。如果你希望解决问题并且允许并行构建，那么你可能需要做进一步的调试并且最终应用补丁以改变构建过程的顺序。

·主机泄露：这个问题与为目标构建有关，或者是在构建软件开发工具包时发生。当构建过程从主机系统而不是从交叉构建环境引用头文件、库或者其他文件时，问题发生。几乎在所有案例中的根本原因是，例如/usr/include、/usr/lib等绝对路径的使用。主机泄露是由质量保证工具报告的，包括Swabber。你应该能够从质量保证消息中通过分析日志文件来简单地认出这些问题，然后创建补丁来解决它们。

·缺失头文件或者库：如果因为编译器找不到来自其他软件包的头文件或者库而编译失败，那么根本原因通常是这些包还没有被声明在

DEPENDS变量中，或者依赖性存在但是到文件的路径是不正确的，并且它没有被配置步骤正确地检测到。

在大部分情况下，增加依赖性到DEPENDS可以解决问题。然而，它可能不立即表现它自己。通常，你以已经用于构建目标根文件系统的构建环境来开始。在很多情况下，在你开始为新包增加菜谱前，依赖性可能已经为其他软件包所构建了，这导致依赖性得以实现，尽管没有在DEPENDS中声明。对这种情况，建议用全新的构建环境来测试。

很少的情况下，你可能需要通过设置变量STAGING_BINDIR、STAGING_INCDIR、STAGING_DATADIR和STAGING_BASELIBDIR等来调整到依赖性的头文件和库的路径。

8.2.8 安装构建输出

在软件包被构建后，do_install任务复制例如二进制、库、头文件、配置文件和文档文件的构建制品到镜像了目标设备的根文件系统的文件系统层级结构中。文件从S、B和WORKDIR目录中复制到D目录，其通常被设置成${WORKDIR}/image。在安装后，这个目录包含具有已安装软件包的所有子目录和文件的本地根文件系统。

因为打包过程从安装目录D中收集文件，所以你必须保证软件包已经被正确地安装了：所有你已经创建的子目录和文件位于根文件系统结构的正确位置。依赖于软件包是如何被构建的，你可能需要对安装过程做出调整：

·GNU Autotools和CMake：如果软件是使用GNU Autotools或者CMake来构建的，那么autotools和cmake类各自提供足以安装大部分软件包的do_install任务。你仅仅需要验证，do_install任务毫无问题地完成了，并且在D中的目录结构是正确的。

如果你需要安装还没有被由类提供的do_install任务复制的额外文件，那么你需要在菜谱中创建处理安装剩余文件的do_install_append函数。你必须使用install-d<source><dest>来复制文件而不是使用cp或者任何其他实用程序。

·Make：如果软件包是仅仅使用不带GNU Autotools或者CMake的Make构建系统来构建的，那么你需要在菜谱中创建do_install任务。makefile很可能已经包含了安装目标来复制文件。这个目标通常需要目的目录，它是文件系统结构的根目录并且可以通过变量来设置。这个变量是什么依赖于makefile。典型的例子是DESTDIR、PREFIX和INSTALLROOT。

OpenEmbedded构建系统提供函数oe_runmake，它用特定的目标执行Make。这个函数允许你传递变量到makefile。你简单地在do_install任务中使用那个函数，如下，假设被makefile需要的用于传到根安装目录的变量名为PREFIX：

```
:

do_install() {
    oe_runmake install PREFIX=${D}
}
```

如果makefile不提供安装目标，那么请看下一个用于手动安装的着重号。

·手动安装：软件包构建系统根本不提供安装功能，你需要创建do_install任务并且使用install-d<src><dest>来安装需要的文件：

```
do_install() {
    install -d ${B}/bin/hello ${D}${bindir}
    install -d ${B}/lib/hello.lib ${D}${libdir}
}
```

你可以使用下列用于标准安装路径的变量并且把它们和D连接起来：

```
bindir = "/usr/bin"
sbindir = "/usr/sbin"
libdir = "/usr/lib"
libexecdir = "/usr/lib"
sysconfdir = "/etc"
datadir = "/usr/share"
mandir = "/usr/share/man"
includedir = "/usr/include"
```

为了验证软件包的安装，运行安装任务并且检查D目录的子目录和文件正确性。

8.2.9 设置系统服务

如果软件包提供需要在系统启动时被启动、在系统关闭时被停止的系统服务，那么菜谱需要为软件包设置系统服务。

首先，你需要验证软件包是否提供必要启动脚本，并且安装任务把它们复制进正确的位置。这些脚本需要包含什么以及它们被安装在什么地方依赖于你正在使用的服务管理器。如果包不提供脚本或者不安装它们，那么你必须增加它们并且追加do_install任务。

构建系统支持这两种服务管理器SysVinit和systemd：

·SysVinit：System V Init是用于类UNIX系统的传统服务管理器。当Linux内核在启动时完成了它的初始化以后，它派生init进程，然后它执行服务脚本并且按照运行级别和优先级启动服务。

do_install任务必须把服务start-stop-script安装进恰当的目录中，通常是/etc/init.d。然后服务需要通过创建从资源控制目录（/etc/rc0.d到/etc/rc6.d）到脚本的链接来启用，结果就是，当系统进入特定运行级别时，服务可以被启动和停止。这是通过update-rc.d类来完成的，update-rc.d类使用菜谱需要继承的update-rc.d工具。类做所有配置工作。所有需要的事物是在菜谱中用于向类提供必要配置的3个变量：

·INITSCRIPT_PACKAGES：包含用于这个软件包init脚本的包列表。这个变量是可选的，并且默认成INITSCRIPT_PACKAGES="${PN}"。

·INITSCRIPT_NAME：init脚本的名字。

·INITSCRIPT_PARAMS：传递到update-rc.d的参数。这可以是例

如"defaults　8020"的字符串，用来在进入运行级别2、3、4、5时启动服务和在进入运行级别0、1、6时停止它。update-rc.d手册[1]提供了关于工具和它的使用的细节。

·systemd：系统管理守护进程（System Management Daemon，systemd）是作为日渐变老的SysVinit的替代物而开发的。特别是，它提供了服务间更好的优先级化和依赖处理以及并行启动服务的能力，它加速了系统的启动序列。systemd主页[2]提供了详细的信息。软件包必须提供给systemd服务脚本，并且菜谱必须在恰当的位置安装它，位置通常是/lib/systemd/system。然后菜谱通过继承systemd类和提供下面的变量来配置服务：

·SYSTEMD_PACKAGES：包含用于软件包的systemd服务文件的包列表。这个变量是可选的，并且默认成SYSTEMD_PACKAGES="${PN}"。

·SYSTEMD_SERVICE：服务文件的名字。

虽然Linux系统明显只能利用仅仅系统管理服务，但是你可以编写菜谱来支持这两者。类和变量不冲突，并且构建系统从菜谱中选择正确的设置。

为得到更多关于如何设置系统服务的信息，参见update-rc.d.bbclass和systemd.bbclass，并且查阅各自系统服务管理器的文档。

[1] www.tin.org/bin/man.cgi?section=8&topic=update-rc.d。
[2] www.freedesktop.org/wiki/Software/systemd。

8.2.10 打包构建输出

此时，软件包的所有必要文件都被构建并且被组织在目的目录D内的文件系统结构中。下一步是由do_package任务执行的，是收集文件并且为包管理系统分配它们进包中。

包拆分

把构建输出（build output）分组到不同包的过程被称为包拆分（package splitting）。大部分软件产生很多不同制品，依赖于需求，你可能只需要其中一些被安装进目标设备的根文件系统中。拆分包以只选择你需要的制品帮助你控制你系统的内存占用并且避免其安装可能对设备造成安全风险的二进制、库、调试信息以及其他文件。

两个控制包拆分的变量是PACKAGES和FILES：

·PACKAGES：这个变量是空格分隔的包名字的列表。被定义在meta/conf/bitbake.conf中的这个变量的默认值是PACKAGES="${PN}-dbg${PN}-staticdev${PN}-dev${PN}-doc${PN}-locale${PACKAGE_BEFORE_PN}${PN}"。do_package任务从左到右处理列表，首先创建最左侧的包，最后创建最右侧的包。顺序是重要的，因为包消耗与它相关联的文件。

·FILES：FILES变量定义被放进特定包中的目录和文件的列表。这个变量总是和条件赋值一起使用，例如：

```
FILES_${PN}-dbg = "<files>"
```

其中files是空格分隔的、可以包含通配符以匹配用于do_install任务在其中创建目录和安装文件的目的目录D的目录结构部分的目录和文件

列表。用于针对列在PACKAGES中的默认包的FILES默认设置被定义在meta/conf/bitbake.conf。

用于PACKAGES和FILES的默认设置满足大部分软件包的需求。然而，软件包可能创建额外的制品并且安装它们到没有被默认设置所包含的目录中。在其他情况下，虽然被默认设置所满足，但是你可能希望进一步拆分包，例如为多个被创建的二进制创建不同的包。在以上两种情况下，你需要增加包名到PACKAGES或通过增加到FILES来包含额外的制品。程序清单8-3演示了如何定制打包。

程序清单8-3　定制打包

```
SUMMARY = "Hello Universe Application"
DESCRPTION = "The ultimate hello extending beyond 'world'."
AUTHOR = "spacey@universetrotter.com"
HOMEPAGE = "http://universetrotter.com"
BUGTRACKER = "https://bugs.universetrotter.com"

PN = "hellouniverse"

# Other recipe stuff
# ...

PACKAGES =+ "graphics"
FILES_${PN}-graphics = "${datadir}/pixmaps/*"
FILES_${PN}-doc =+ "${datadir}/blurbs/*"

PACKAGE_BEFORE_PN = "examples"
FILES_${PN}-examples = "${datadir}/examples"
```

在程序清单8-3中的例子前新增名为graphics的额外包到PACKAGES并且设置FILES_${PN}-graphics到过滤器。它也追加过滤器到FILES_${PN}-doc来从非标准的${datadir}/blurb目录收集文档文件并且把它们放进doc包中。

在PACKAGES的默认列表中的最后包是${PN}，它是标准包。对FILES相关的定义最终消耗所有尚未被其之前包所声明的文件和目录：

```
SOLIBS = ".so.*"
FILES_${PN} = "${bindir}/* ${sbindir}/* ${libexecdir}/* ${libdir}/lib* \
               {SOLIBS} ${sysconfdir} ${sharedstatedir} ${localstatedir} \
               ${base_bindir}/* ${base_sbindir}/* \
               ${base_libdir}/*${SOLIBS} \
               ${base_prefix}/lib/udev/rules.d $prefix}/lib/udev/rules.d \
               ${datadir}/${BPN} ${libdir}/${BPN}/* \
               ${datadir}/pixmaps ${datadir}/applications \
               ${datadir}/idl ${datadir}/omf ${datadir}/sounds \
               ${libdir}/bonobo/servers"
```

BitBake语法仅仅允许你追加或者前新增到变量。然而，如果你需要在${PN}包之前嵌入包，那么你可以设置PACKAGE_BEFORE_PN变量并且使用对FILES的条件赋值来设置文件过滤器，正如在程序清单8-3中所示的那样。

最后两个控制打包的变量是PACKAGE_DEBUG_SPLIT_STYLE和PACKAGESPL-ITFUNCS。前者给了你对如何处理二进制和调试对象的控制。后者让你为包拆分增加你自己的函数到菜谱。它们是不常用的。我们在8.1.2小节解释了基础。

打包质量保证

insane类增加合理性和错误检查到打包过程。类定义检查函数的列表，检查函数在打包过程之前、期间和之后被调用，例如属主和文件/目录权限、用于可执行文件和库的正确架构、调试包对非调试包的依赖性等。为得到函数列表，请参考类本身或者Yocto项目参考手册[1]。

你可以使用变量WARN_QA和ERROR_QA来决定质量保证函数是否应该创建警告消息，其让构建继续，并且成功而不管警告；或者创建错误消息，其让构建失败。你通常在全局的级别设置这些变量，要么正如在前面章节中概述的，在发行版配置文件中设置，要么在构建环境的conf/local.conf中。

如果你需要为菜谱禁用特定的检查，你可以使用INSANE_SKIP变

量。例如，

```
INSANE_SKIP_${PN} += "dev-so"
```

为动态加载的库跳过符号链接检查。通常，这些链接仅仅对于开发包有用，但是一些软件包可能需要它们以正常工作。

除了由insane类执行的自动化的检查以外，你可以手动验证正确的包拆分。目录${WORKDIR}/packages-split包含用于被创建的每个包的独立目录结构。

包架构

一般来说，构建系统把所有包标记成是特定于目标架构的。那个惯例对于大部分包是恰当的。有两个你可能需要调整包架构的用例：

·机器依赖包：如果包依赖于它为其构建的特定机器，这通常发生在当它执行do_configure任务传递MACHINE变量设置到配置脚本的情况下，那么你需要显式地设置包架构为：

```
PACKAE_ARCH = "${MACHINE_ARCH}
```

·架构独立包：如果包适用于所有架构而不管它为其构建的机器是什么，例如，如果它是字体或者脚本包，那么菜谱需要继承allarch类来正确地标记包：

```
inherit allarch
```

即使你不显式地改变包架构，我们仍然推荐你尝试通过设置你构建环境的conf/local.conf文件中的MACHINE变量来为不同机器做多个构建的方法来为不同架构构建包。

[1] www.yoctoproject.org/docs/1.6.1/ref-manual/ref-manual.html#ref-classes-insane。

8.2.11　定制安装脚本

包管理器有能力在包被安装、升级或者卸载之前和之后运行定制脚本。这些脚本是包含在包中的，并且当脚本被调用以执行特定动作时，包管理系统执行它们。OpenEmbedded构建系统支持提供这样的脚本作为菜谱的一部分。它们被作为函数而加到菜谱中。package类拿出函数并且按照包管理系统增加它们到各自的包。机制对包管理系统是透明的，并且仅仅为RPM、dpkg和ipkg所支持。tar打包不支持定制安装脚本。你可以定义以下4个脚本：

·pkg_preinst_<packagename>：在包被安装前运行的安装前脚本（preinstallation script）。

·pkg_postinst_<packagename>：在包被安装后运行的安装后脚本（postinstallation script）。

·pkg_prerm_<packagename>：在包被卸载前运行的卸载前脚本（pre-uninstallation script）。

·pkg_postrm_<packagename>：在包被卸载后运行的卸载后脚本（post-uninstallation script）。

你需要用包的名字替换<packagename>，它是列在PACKAGES变量中的名字之一。指定包名允许你条件性地应用脚本到特定包。

定制脚本通常是shell脚本。程序清单8-4显示了用于主要包${PN}的安装后脚本的骨架。

程序清单8-4　安装后脚本骨架

```
pkg_postinst_${PN}() {
#!/bin/sh
# shell commands go here
}
```

这个脚本是在包被安装后执行的。为了包成功安装，脚本必须成功完成。不管包管理器是由OpenEmbedded构建系统在创建根文件系统时运行的，还是被运行在目标上，脚本都执行。在一些情况下，你可能希望仅仅通过在目标上执行包管理器将包安装在目标上时才运行脚本，或者你可能希望当安装在目标上时执行不同的命令。程序清单8-5显示了如何实现那个目的的骨架。

程序清单8-5　条件安装后脚本骨架

```
pkg_postinst_${PN}() {
#!/bin/sh
if [ x"$D" = "x" ]; then
   # shell commands for target execution
else
   # shell commands for build system execution
fi
}
```

在程序清单8-5中的骨架脚本背后的逻辑是，目的目录变量D是在构建系统的上下文中而不是当包管理器在目标上被执行时来设置的。

8.2.12 变体

所有菜谱为目标构建软件包。如果你希望菜谱为构建主机或除了目标之外的软件开发工具包而构建，那么你必须设置BBCLASSEXTEND变量：

·native：为构建主机构建。

·native-sdk：为软件开发工具包构建。

当构建变体时，你可能需要按需通过条件设置或者覆盖变量来对菜谱做调整。

8.3 菜谱例子

这部分提供如何为构建不同类型的软件包编写菜谱的例子。这些是具有目的性的简单例子，你可以立即使用你自己的构建环境来测试它们。一如既往，我们推荐你为它创建你自己的层：

```
$ yocto-layer create mylayer
```

让工具为你创建样例菜谱，或者手动创建目录结构和文件。例子是应用，我们假设它们被放在meta-mylayer/recipes-apps/<appname>下面的层内。

不要忘记通过增加层到你构建环境的conf/bblayers.conf中的BBLAYERS变量来在构建环境中包含它。

8.3.1 C文件软件包

这个例子演示了如何编写直接从软件包的源文件构建软件包的菜谱。虽然这个技术不常被使用，因为几乎所有包都使用某种构建系统，例子显示了构建系统可以被调整成构建任何包。而且，如果你有作为你原生构建一部分的必要编译器，那么你不被限于构建使用C或者C++编程语言的源代码。

用于例子的源代码由两个C源文件和C头文件组成，我们在程序清单8-6中已经复制了它们。

程序清单8-6 C文件软件包源代码

```
helloprint.h:
void printHello(void);

helloprint.c:
#include <stdio.h>
#include "helloprint.h"
void printHello(void) {
    printf("Hello, World! My first Yocto Project recipe.\n");
    return;
}

hello.c:
#include "helloprint.h"
int main() {
    printHello();
    return(0);
}
```

例如，我们假设你从你创建了3个文件的目录中使用以下命令创建名为hello-1.0.tgz并且包含3个文件的压缩tar包：

```
$ tar cvfz hello-1.0.tgz .
```

然后，复制tar文件到子目录meta-mylayer/recipes-apps/hello/hello-1.0.中。

随后，在meta-mylayer/recipes-apps/hello中创建名为hello_1.0.bb且显示在程序清单8-7中的菜谱。

程序清单8-7　用于构建C文件源包的菜谱

```
SUMMARY = "Simple Hello World Application"
DESCRIPTION = "A test application to demonstrate how to create a recipe \
               by directly compiling C files with BitBake."

SECTION = "examples"
PRIORITY = "optional"

LICENSE = "MIT"
LIC_FILES_CHKSUM = "\
   file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

SRC_URI = "file://hello-1.0.tgz"

S = "${WORKDIR}"

do_compile() {
    ${CC} -c helloprint.c
    ${CC} -c hello.c
    ${CC} -o hello hello.o helloprint.o
}

do_install() {
    install -d ${D}${bindir}
    install -m 0755 hello ${D}${bindir}
}
```

构建系统为正确的C编译器自动设置变量CC，包括用于机器架构、系统根、调优参数等的所有必要标记。关于这个菜谱要注意的唯一其他事情是设置变量S。因为S默认是${WORKDIR}/${PN}-${PV}，但是我们的tar包不包含具有包名和包版本的根目录，所以，菜谱必须调整S的设置以便构建系统可以找到源。

这个菜谱构建引用，并且也自动化创建主要包hello。所有其他包是空的，因为do_install仅仅安装hello应用文件。

8.3.2 基于makefile的软件包

为了这个例子，我们重用上个例子中的3个源文件，并且增加显示在程序清单8-8中的makefile来构建软件包。

程序清单8-8 makefile

```
CC=gcc
RM=rm

CFLAGS=-c -Wall
LDFLAGS=

DESTDIR=
BINDIR=/usr/bin

SOURCES=hello.c helloprint.c
OBJECTS=$(SOURCES:.c=.o)

EXECUTABLE=hellomake

.cpp.o:
        $(CC) $(CFLAGS) $< -o $@

all: $(SOURCES) $(EXECUTABLE)

$(EXECUTABLE): $(OBJECTS)
        $(CC) $(LDFLAGS) $^ -o $@

clean:
        $(RM) $(EXECUTABLE) *.o

install: $(EXECUTABLE)
        mkdir -p $(DESTDIR)/$(BINDIR)
        install -m 0755 $< $(DESTDIR)/$(BINDIR)
```

这是一个简单而典型的makefile，其设置了一些指定C编译器和一些标记的变量。使用这个文件，你可以在主机系统上构建软件包。然而，makefile不把交叉构建考虑在内。

把makefile放进和其他3个源文件相同的目录中，并且使用如下命令创建tar包：

```
$ tar --transform "s/^\./hellomake-1.0/" -cvzf hellomake-1.0.tgz .
```

这个命令不仅创建压缩的tar包，也以目录hellomake-1.0前缀在包中的文件来创建正确的目录结构以便菜谱不用必须调整S变量。复制tar包进子目录meta-mylayer/recipes-apps/hello/hellomake-1.0中。

然后在meta-mylayer/recipes-apps/hellomake中创建显示在程序清单8-9中的名为hellomake_1.0.bb的菜谱。

程序清单8-9　用于构建基于makefile的软件包的菜谱

```
SUMMARY = "Hello with Makefile"
DESCRIPTION = "A test application to demonstrate how to create a \
               recipe for makefile-based project."
SECTION = "examples"
PRIORITY = "optional"

LICENSE = "MIT"
LIC_FILES_CHKSUM = "\
   file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

SRC_URI = "file://${P}.tgz"

EXTRA_OEMAKE = "'CC=${CC}' 'RANLIB=${RANLIB}' 'AR=${AR}' \
   'CFLAGS=${CFLAGS} -I${S}/. -DWITHOUT_XATTR' 'BUILDDIR=${S}'"

do_install () {
   oe_runmake install DESTDIR=${D} BINDIR=${bindir} SBINDIR=${sbindir} \
      MANDIR=${mandir} INCLUDEDIR=${includedir}
}
```

函数oe_runmake由do_compile任务执行并且以makefile来调用Make工具。你通常必须通过设置EXTRA_OEMAKE变量来传递用于变量设置的参数到makefile。变量提供从构建系统变量到被makefile使用的变量和传统参数的映射（mapping）。

如果你需要提供额外选项到CFLAGS变量，那么你一定不能覆盖它，因为构建系统使用它传递交叉构建设置到编译器。替代性地，使用

```
CFLAGS_prepend = "<flags>"
```

来增加额外选项。

不需要覆盖do_compile任务。由base.bbclass定义的默认任务执行oe_runmake，其对应执行Make工具，传递EXTRA_OEMAKE变量给它。

然而，你确实需要提供你自己的do_install任务，因为由base.bbclass定义的默认什么也不做。如果makefile提供安装目标，那么任务仅仅用参数来调用oe_runmake，正如在例子中所显示的。否则，你必须显式地编写安装，正如在前面例子中所显示的。

8.3.3 基于CMake的软件包

CMake是用于构建、测试和打包软件的开源跨平台构建系统。CMake工具套件使用独立于平台和编译器的配置文件来控制和管理构建过程。CMake创建原生makefiles，甚至为各种集成开发环境（Integrated Development Environments，IDE）创建工作区。

OpenEmbedded构建系统用cmake类为使用CMake的软件包提供支持。类处理配置和构建过程，把大部分菜谱缩减到仅有描述性元数据、许可信息、源URI和用于cmake类的继承指令。程序清单8-10显示了用于构建Synergy的菜谱，Synergy是允许通过本地网络在多个计算机之间共享键盘和鼠标的客户端–服务器应用。Synergy使用CMake来控制它的构建过程。显示的菜谱是从OpenEmbedded meta-oe层复制过来的。

程序清单8-10　用于构建Synergy的菜谱——synergy_1.3.8.bb

```
SUMMARY = "Synergy - control multiple computers with one keyboard and mouse"
HOMEPAGE = "http://synergy.googlecode.com"
LIC_FILES_CHKSUM = "file://COPYING;md5=9772a11e3569985855e2ce450e56f991"
LICENSE = "GPL-2.0"
SECTION = "x11/utils"

DEPENDS = "libx11 libxtst libxinerama"

SRC_URI = "http://synergy.googlecode.com/files/synergy-${PV}-Source.tar.gz"

SRC_URI[md5sum] = "3534c65ecfa6e47d7899c57975442f03"
SRC_URI[sha256sum] = \
   "0afc83e4ed0b46ed497d4229b2b2854e8d3c581a112f4da05110943edbfacc03"

S = "${WORKDIR}/${PN}-${PV}-Source"

inherit cmake

do_install() {
    install -d ${D}/usr/bin
    install -m 0755 ${S}/bin/synergy* ${D}/usr/bin/
}
```

8.3.4　基于GNU Autotools的软件包

GNU构建系统，通常被称为GNU　Autotools，已经成为用于开发可以在类UNIX系统之间移植的应用的事实标准。

开发者频繁批评Autotools复杂并且难以使用。然而，编写可以在大量不同系统上被编译的应用是一项复杂任务，对于嵌入式系统尤其如此。Autotools可以帮助开发者理解很多不同系统的细节，以及它们在哪些方面存在差异，但要求他们提供一个可能很长的配置设置列表。

带有Autotools支持而被创建的软件包极大地简化了在不同系统上构建包的任务。通常，所有你要做的是运行配置脚本，然后构建包。大量开源包使用Autotools用于配置。OpenEmbedded构建系统通过autotools类提供很大的支持，在大部分情况下，它把编写菜谱极大地简化成了提供描述性指令、增加源URI和许可信息以及继承autotools类。程序清单8-11显示了构建nano编辑器包的菜谱。

程序清单8-11　用于构建Nano编辑器的菜谱——nano_2.3.1.bb

```
SUMMARY = "GNU nano - an enhanced clone of the Pico text editor"
DESCRIPTION = "GNU nano - an enhanced clone of the Pico text editor"

HOMEPAGE = "http://www.nano-editor.org"
BUGTRACKER = "https://savannah.gnu.org/bugs/?group=nano"

SECTION = "console/utils"
PRIORITY = "optional"

LICENSE = "GPLv3"
LIC_FILES_CHKSUM = "file://COPYING;md5=f27defe1e96c2e1ecd4e0c9be8967949"

DEPENDS = "ncurses"

PV_MAJ = "${@bb.data.getVar('PV',d,1).split('.')[0]}"
PV_MIN = "${@bb.data.getVar('PV',d,1).split('.')[1]}"

SRC_URI = "\
   http://www.nano-editor.org/dist/v${PV_MAJ}.${PV_MIN}/nano-${PV}.tar.gz"

SRC_URI[md5sum] = "af09f8828744b0ea0808d6c19a2b4bfd"
SRC_URI[sha256sum] = "\
   b7bace9a8e543b84736d6ef5ce5430305746efea3aacb24391f692efc6f3c8d3"
inherit autotools gettext
RDEPENDS_${PN} = "ncurses"
```

这个菜谱不包含用于任何步骤的任何特殊设置。所有东西都是由autotools处理的。关于这个菜谱需要注意的仅有以下：

·除了autotools，菜谱也继承gettext类。这个类用GNU gettext原生语言支持（Native Language Support，NLS）促进构建软件包。

·Nano是使用ncurses库的控制台应用。因此，菜谱声明对其的构建时和运行时依赖。

·为了使菜谱更易于为软件包的更新版本而升级，不直接把版本号写入SRC_URI是好建议。因为nano编辑器的上游源仓库在它的路径中使用部分版本号，所以菜谱拆分PV变量成它的各部分以创建版本号。

为了构建nano编辑器，简单地在meta-mylayer/recipes-apps/nano中创建名为nano_2.3.1.bb且显示在程序清单8-11中的菜谱，并且启动BitBake。

8.3.5 外部构建软件包

在一些情况下，你可能需要构建系统包含已经通过其他方法构建的软件包，并且你被提供了仅仅包含二进制、配置文件和文档等的软件包。因为你没有源代码，所以你不能使用一个获取源代码然后构建它的菜谱。

解决方案是，写获取二进制包、解压它、跳过配置和编译步骤、安装包组件、然后重新打包它们以与目标设备的根文件系统集成。

虽然这不是最理想的过程并且可能导致兼容性问题，但是它可能是仅有的无缝集成这种包的方法。

为了集成外部构建的软件包，OpenEmbedded构建系统提供了bin_package类。类使用默认的do_fetch和do_unpack任务来获取和解压它到S目录中。它跳过do_configure和do_compile任务并且定义了简单的把文件从S复制到D目录的do_install任务。在复制后，它通过设置FILES_${PN}="/"来创建包含所有文件的单一包。

程序清单8-12显示了使用bin_package的一个样本菜谱。

程序清单8-12　使用bin_package的菜谱

```
SUMMARY = "Package the Proprietary Software"
DESCRIPTION = "A sample recipe utilizing the bin_package class \
               to package the externally build Proprietary software \
               package."

LICENSE = "CLOSED"

SRC_URI = "file://proprietary-${PV}.rpm"

inherit bin_package
```

如果原软件包在被解压到S目录后具有和它在目标上完全相同的布

局，那么bin_package类可以不做任何调整的工作。

如果软件包在目标上要求有不同的布局，或者你希望拆分它为不同的包，那么你需要覆盖do_install任务并且对应设置FILES变量（至少FILES_{PN}，但最终也包括其他来自PACKAGES列表的变量）。

8.4 devtool

在本章前面的部分，我们学习了为软件包写菜谱，为它们创建层，在构建环境中包含层，以及增加包到镜像目标，构建并且最终部署镜像的过程。这些步骤中的大部分都可以通过使用devtool来简化。

devtool是工具套件，它在用OpenEmbedded构建系统进行往返开发（round-trip development）方面辅助你。它本质上就像瑞士军刀，在单一包中提供最常被用到的工具。使用--help选项给了你devtool子命令的程序清单：

```
$ devtool -help
usage: devtool [--basepath BASEPATH] [--bbpath BBPATH] [-d] [-q]
               [--color COLOR] [-h]
               <subcommand> ...

OpenEmbedded development tool

optional arguments:
  --basepath BASEPATH  Base directory of SDK / build directory
  --bbpath BBPATH      Explicitly specify the BBPATH, rather than
                       getting it
                       from the metadata
  -d, --debug          Enable debug output
  -q, --quiet          Print only errors
  --color COLOR        Colorize output (where COLOR is auto, always,
                       never)
  -h, --help           show this help message and exit

subcommands:
  <subcommand>
    create-workspace   Set up a workspace
    deploy-target      Deploy recipe output files to live target machine
    undeploy-target    Undeploy recipe output files in live target machine
    build-image        Build image including workspace recipe packages
    add                Add a new recipe
    modify             Modify the source for an existing recipe
    extract            Extract the source for an existing recipe
    sync               Synchronize the source for an existing recipe
    update-recipe      Apply changes from external source tree to recipe
    status             Show workspace status
    reset              Remove a recipe from your workspace
    search             Search available recipes
    upgrade            Upgrade an existing recipe
    build              Build a recipe

Use devtool <subcommand> --help to get help on a specific command
```

8.4.1 使用devtool的往返开发

devtool为你创建和维护工作区（workspace）层，它自动化地把工作区层和当前构建环境集成起来。在你可以使用devtool用于往返开发之前，你需要引用（source）构建环境，正如往常一样。

创建workspace层

命令

```
$ devtool create-workspace [layerpath]
```

在指定的layerpath中为你创建新的workspace层。如果你省略layerpath，那么devtool在当前位置创建名为workspace的层。被创建的workspace层包含conf/layer.conf文件。devtool自动化地增加层到当前构建环境的conf/bblayers.conf文件中，除非你指定--create-only选项。

只有当你希望显式地指定layerpath时，你才必须使用create-workspace命令，因为如果还没有用于你当前构建环境的workspace层存在，其他devtool命令会自动创建它。

对于同一个构建环境来说，同一时刻你仅仅可以拥有一个由devtool维护的workspace层。如果你在已经有了工作区的构建环境中再次使用create-workspace命令，那么devtool创建新层并且对应修改conf/bblayers.conf。然而，它不删除你以前的workspace层。

文件conf/devtool.conf包含用于devtool的配置设置，特别是到workspace层的路径。

增加新菜谱到workspace层

为软件包增加新菜谱到workspace层，使用命令

```
$ devtool add <recipe-name> <source-path>
```

其中，recipe-name是菜谱的名字，source-path是到软件包的源的路径。如果你不显式地创建workspace层，那么当使用add命令时，devtool将隐式地创建名为workspace的层。

devtool在工作区层作为recipes/<recipe-name>/<recipe-name>.bb来创建菜谱。在菜谱内的SRC_URI变量是空的，因为devtool创建带有被设置成指向到source-path的EXTERNALSRC的append/<recipe-name>.bbappend文件。如果devtool不能找到被包含在包源中的许可信息，那么它设置LICENSE="CLOSED"和LIC_FILES_CHKSUM=""，这允许菜谱在即使没有许可信息的情况下也可构建。对应地，devtool也尝试计算出如何构建源和设置菜谱。对于使用CMake和Autotools的软件包，它包含各个类。对于基于makefile的软件包，它为do_configure()、do_compile()和do_install()任务设置桩（stub）。虽然在大部分情况下，devtool产生可工作的菜谱，但是你将可能需要调整它以使其完全运转起来。

如果你可以从远程位置访问包，那么可以使用

```
$ devtool add <recipe-name> <source-path> -f <source-uri>
```

来从<src-uri>直接获取源并且解压它们到本地<source-path>。例如

```
$ devtool add nano sources/nano \
    -f http://www.nano-editor.org/dist/v2.5/nano-2.5.1.tar.gz
```

从nano编辑器下载站点获取nano-2.5.1.tar.gz源tar包，并且解压它们到sources/nano目录，在其中它初始化了Git仓库。它也在workspace/recipes/nano中创建菜谱nano.bb，显示在程序清单8-13中（我

们仅仅重新格式化了菜谱的内容以使得它适应本书的页面）。

程序清单8-13　用devtool创建的用于nano编辑器的菜谱

```
# Recipe created by recipetool
# This is the basis of a recipe and may need further editing in order
# to be fully functional.
# (Feel free to remove these comments when editing.)
#
# WARNING: the following LICENSE and LIC_FILES_CHKSUM values are
# best guesses - it is your responsibility to verify that the values are
# complete and correct.
#
# NOTE: multiple licenses have been detected; if that is correct you
# should separate these in the LICENSE value using & if the multiple
# licenses all apply, or | if there
# is a choice between the multiple licenses. If in doubt, check the
```

```
    # accompanying documentation to determine which situation is applicable.
    LICENSE = "GPLv3 Unknown"
    LIC_FILES_CHKSUM = " \
         file://COPYING;md5=f27defe1e96c2e1ecd4e0c9be8967949 \
         file://COPYING.DOC;md5=ad1419ecc56e060eccf8184a87c4285f"

    SRC_URI = "http://www.nano-editor.org/dist/v2.5/nano-2.5.1.tar.gz"
    SRC_URI[md5sum] = "f25c7da9813ae5f1df7e5dd1072de4ce"
    SRC_URI[sha256sum] =
    "e06fca01bf183f4d531aa65a28dffc0e2d10185239909eb3de797023f3453bde"

    S = "${WORKDIR}/nano-2.5.1"

    # NOTE: the following prog dependencies are unknown, ignoring: makeinfo
    # NOTE: the following library dependencies are unknown, ignoring:
    # ncursesw ncursesw ncurses curses curses magic z
    # (this is based on recipes that have previously been built and packaged)
    # NOTE: if this software is not capable of being built in a separate
    # build directory from the source, you should replace autotools with
    # autotools-brokensep in the inherit line
    inherit pkgconfig gettext autotools

    # Specify any options you want to pass to the configure script using
    # EXTRA_OECONF:
    EXTRA_OECONF = ""
```

注意，变量SRC_URI被设置成用-f（--fetch）选项指定的URI。而且，devtool创建追加文件workspace/append/nano.bbappend，显示在程序清单8-14中。

程序清单8-14　外部源nano.bbappend

```
    inherit externalsrc
    EXTERNALSRC = "/run/media/rstreif/YoctoDevelop/projects/kc/src/nano"

    # initial_rev: c0516cb63fa0d376f81aec4e75a9c3cbd80823cb
```

追加文件覆盖菜谱的SRC_URI设置，允许你修改源而不用改变菜谱。

构建菜谱

在你已经增加了菜谱、审查了它并且最终做了一些调整后，你可以使用devtool来构建它：

```
$ devtool build <recipe-name>
```

其本质上用来自构建环境的所有设置调用bitbake<recipe-name>，这些设置包括用于make的并行度（parallelism）选项。你可以通过增加-s（--disable-parallel-make）到构建命令来禁用并行make。

部署包到目标系统

现在你可以部署新鲜出炉的构建包到目标系统上。目标系统可以是实际硬件或者QEMU。唯一的要求是，目标系统必须是运行着安全Shell（SSH）的服务器。命令

```
$ devtool deploy-target <recipe-name> [user@]target-host[:destdir]
```

由do_install()任务把所有已安装文件传输到根文件系统中。你可以指定替代的用户名和文件复制到的目的目录。

一些选项修改命令的行为：

·-n,--dry-run：这个选项仅仅列出要被部署的文件而不实际复制它们到目标系统。

·-s,--show-status：如果你使用这个选项，那么命令将展示状态和过程输出。

·-c,--no-host-check：跳过SSH主机密钥验证。

从目标系统移除包

与deploy-target类似，你使用命令

```
$ devtool undploy-target <recipe-name> [user@]target-host
```

来从目标系统中删除用deploy-target部署的文件。如果在部署时你使用了destdir，那么devtool记住它并且从那个目录中移除文件。

用于undeploy-target命令的选项和用于deploy-target命令的选项是完全相同的。

构建镜像

用devtool，你也可以构建包含所有来自workspace层的菜谱的镜像。命令

```
$ devtool build-image <image-name>
```

通过增加来自工作区的菜谱到IMAGE_INSTALL_append来用其扩展由<image-name>指定的镜像，然后启动BitBake以构建镜像。

展示工作区信息

命令

```
$ devtool status
```

打印关于workspace层的状态信息。

8.4.2 用于现有菜谱的工作流

通常，你可能需要为在另一层中定义菜谱的软件包创建补丁。为了完成这个任务，你通常必须获得包的源代码，本地解压它并且创建追加文件来使用本地源代码。在你对本地源代码做了变更以后，你需要创建补丁，然后增加它到菜谱。devtool提供了通过处理用于管理包源代码和菜谱的繁重任务来极大地辅助你完成这个工作流的命令行。

增加现有菜谱到工作区

命令

```
$ devtool modify -x <recipe-name> <source-path>
```

获取用于由菜谱<recipe-name>构建的包的源代码，解压它到<source-path>指定的目录中，并在Git仓库中设置源代码。该命令不把菜谱从原始层复制到workspace层，而是简单地创建追加文件来覆盖SRC_URI。

例如，如果你从构建环境中运行命令

```
$ devtool modify -x sqlite3 src/sqlite3
```

那么它在workspace/src/sqlite3中创建源仓库和追加文件workspace/append/sqlite3.bbappend。

你现在可以对SQLite3源代码做变更并且使用

```
devtool build sqlite3
```

来以这些变更构建包。

更新菜谱

一旦你对变更满意了，那么提交它们到仓库，正如以往用Git：

```
$ git add .
$ git commit -s
```

现在你使用devtool来从提交中创建补丁并且增加它到原始菜谱：

```
$ devtool update-recipe <recipe-name>
```

这个命令直接更新在原始层中的菜谱。对于我们的SQLite3例子来说，这意味着，devtool增加补丁到poky/meta/recipes/sqlite/sqlite3并且对应地修改菜谱。

如果你不希望修改原始层而是希望增加追加文件到另一个层，那么使用以下形式的命令：

```
$ devtool update-recipe <recipe-name> -a <layer-dir>
```

其中，<layer-dir>是到你希望增加追加文件的层的顶级目录的路径。

你总是不得不首先提交变更到仓库；否则，devtool忽略它们并且不创建补丁。你可以通过后续提交变更为创建一系列补丁而使用这个行为。

8.5 总结

一开始，编写菜谱看起来是困难的，但是随着实践而变得更加容易。在OpenEmbedded网站上，差不多有数千菜谱可供你仔细研究和学习。层索引和搜索功能使其容易找到已经接近于你正在尝试完成的东西的菜谱。

在本章，我们

·研究了菜谱的结构并且讨论了典型变量。

·解释了菜谱命名和格式惯例。

·提供了一步一步的关于如何写你自己的菜谱的指导。

·显示了典型菜谱的例子。

·解释了如何使用devtool用于快速往返开发和轻松地工作在现有菜谱以及它们构建的软件包上。

8.6 参考文献

OpenEmbedded元数据层索引，http://layers.openembedded.org/layerindex/branch/master/layers/。

Yocto项目文档，https://www.yoctoproject.org/documentation/current。

第9章　内核菜谱

如果没有定制Linux内核来完整支持硬件，那么嵌入式Linux项目将不是名副其实的。虽然Linux不是一个微内核（microkernel）架构[1]，但是Linux是模块化的。功能性要么被编译进Linux内核中，要么作为可加载内核模块（loadable kernel module）在运行时被插入到内核中。从模块化的角度来看，Linux内核类似于微内核架构。然而，在微内核中，设备驱动和其他非内核核心组成部分的内核代码是作为独立但具有特权的进程来执行的。在这一点上，Linux内核和微内核是不相似的。在Linux内核中，模块和设备驱动代码总是在内核的上下文中执行，这使得Linux内核称为单内核。因为这种架构，Linux模块拥有对所有内核数据结构的访问，不管是好是坏，它不需要使用微内核所必须使用的内核进程间通信（Interprocess Communication，IPC）。然而，进程间通信对所有Linux内核模块都是可用的。

你是希望编译内核功能性到内核中还是使其成为运行时可加载模块，这依赖于你需要为你的项目所考虑的各种因素：

·作为内核模块的可用性：如果功能性是作为内核模块来提供的，那么你有这样两个选择——直接编译模块代码到内核中或者编译它成为可以在内核运行时被加载和卸载的可加载模块。如果你选择编译内核模块到内核中，那么在运行时它不能被卸载。然而，一些功能性，因为它的技术本质，永远不能在运行时被加载。如果你的项目要求那样的功能性，那么你必须编译它到内核中。

·内核内存占用：编译进内核的功能性越少就意味着可以快速从存储介质中加载的更小内核。

·启动时间：具有更少的被编译进内核的驱动的更小内核意味着在

内核启动时更少的初始化，这有助于实现一个更快的启动时间。

·启动时的硬件支持：因为内核模块在内核已经启动了用户空间之后才被插入到Linux内核中，所以在内核启动时就需要可用的所有硬件支持，例如硬盘以及最终的网络硬件，都必须被编译进内核中。替代性地，你可以使用初始内存磁盘（initial ramdisk，initrd）镜像来探测硬件，正如大部分Linux桌面和服务器发行版所做的。使用initrd保持内核更加通用，但是探测硬件需要更多时间。

·升级能力：在嵌入式设备交付之后，硬件驱动可能需要升级。如果设备驱动是被编译进内核的，那么整个内核必须被升级，在不重启系统的情况下这是不可能实现的。如果设备驱动是作为内核模块来加载的，那么旧模块可以在运行时被卸载并且新模块可以被加载以替换旧模块。

在本章中，我们解释如何用Yocto项目来使用Linux内核的配置系统以定制内核，用内核菜谱构建内核的不同方法，如何向内核打补丁以及如何构建树外内核模块（out-of-tree kernel module）。

[1] 在此，我们不深入讨论微内核架构对比单内核（monolithic kernel）架构的长处和短处。Linus Torvalds和Minix OS创造者Andrew Tanenbaum之间对此有一个非常有趣的辩论：https://groups.google.com/forum/?fromgroups=#!topic/comp.os.minix/wlhw16QWltI%5B1-25%5D。虽然辩论远远不是完全客观和基于事实的，但是它提供了对这两种架构的优点和缺点的良好见解。

9.1 内核配置

Linux内核提供了它自己的配置系统，通常被称为kconfig。kconfig本质上是以树结构组织的配置数据库。所有配置选项合并进位于内核源树的顶级目录中的单一文件.config。内核的构建系统使用那个文件来传送配置到所有内核源文件。作为文件名首字母的点（.）使得这个文件成为UNIX系统上的隐藏文件。你必须使用ls-a以便在目录程序清单中看到它。虽然你可以直接使用文本编辑器来编辑那个文件，但这样做是不推荐的。在一个逐行的平面文件中包含了超过5500个配置设置，这可能是内核开发者决定让.config成为隐藏文件的原因之一吧。

.config文件通常是被自动化地创建的，它要么是来自默认平台配置文件，要么是来自用于特定系统的现有配置文件。如果你希望编辑它，那么Linux内核配置系统提供了用于它的菜单编辑器。这些编辑器也识别配置设置的依赖性（dependency）和互依赖性（interdependency）。

如果你选择依赖于一个或者更多其他设置的配置设置，那么，如果那些被依赖的设置还没有被设置，编辑器也自动选择它们。

9.1.1 菜单配置

如果你以前用过Linux内核并且为原生环境构建过它，那么你很可能熟悉make menuconfig、make xconfig和make gconfig命令。所有这些命令都启动基于菜单的层级结构的编辑器，它允许浏览、搜索和改变配置选项。

Yocto项目的内核菜谱通过调用

```
$ bitbake -c menuconfig <kernel-recipe>
```

来提供make menuconfig的功能性。

例如，

```
$ bitbake -c menuconfig virtual/kernel
```

依赖于机器设置，它启动用于当前内核的菜单编辑器。

为了使菜单编辑器工作，有效的配置必须可用。因此，你必须至少构建过一次内核。通常是这样的，那就是你首先构建完整镜像，然后做修改。然而，如果你确实有新内核菜谱，并且不希望在可以使用菜谱编辑器修改配置之前等待内核被完整构建，那么你可以使用kernel_configme命令来运行内核构建过程，直至包括创建.config文件的配置步骤：

```
$ bitbake -c kernel_configme virtual/kernel
```

如果你正在使用图形桌面环境来开发Yocto项目，那么期待菜单编辑器打开另一个终端窗口。terminal类，它提供用于启动终端窗口的代

码，尝试在你系统配置的基础上找到合适的终端程序。在几乎所有系统上，terminal类都能成功，并代表合适的终端程序。如果菜单编辑器看起来不正确，或者你倾向于使用不同的终端类型，那么你可以通过显式地设置构建环境的conf/local.conf中的变量OE_TERMINAL来覆盖自动选择。

你可以使用菜单编辑器来对内核配置做修改。容易用QEMU模拟器测试的简单例子是，禁用对称多处理（Symmetric MultiProcessing，SMP）。用于QEMU的Yocto项目的内核配置默认启用这个特性。从子菜单Processor type and features中，取消选择条目Symmetric multi-processing support，然后单击Save and Exit。

菜单编辑器把变更的配置保存到在构建环境内核构建目录中的.config文件。为了应用新配置，你必须编译内核：

```
$ bitbake -C compile virtual/kernel
```

大写-C使指定任务的共享状态缓存条目的戳无效，然后运行默认任务。在这个情况下，它强制BitBake再次构建内核，但是并不首先获取源代码。因为使用菜单编辑器直接修改在构建目录中的.conf文件，所以如果再次获取源代码，也包含获取内核配置，那么会覆盖新配置。

现在，你可以通过启动QEMU来测试新配置：

```
$ runqemu qemux86 qemuparams="-smp 2"
```

脚本runqemu以单处理模式启动QEMU。增加qemuparams="-smp 2"用两个处理器核心启动QEMU。在QEMU启动你的Linux系统后，以root用户登录，并且从命令行中执行：

```
# cat /proc/cpuinfo | grep processor
```

虽然QEMU提供了两个处理器核心，但是只有一个处理器被显示。你可以通过使用菜单编辑器重新启用对称多处理来再次执行测试。此时，命令显示两个处理器。

对快速测试新内核配置来说，菜单编辑器是一个优秀的工具。然而，因为对内核配置的修改被直接写到内核构建环境内的.conf文件，所以那些变更是不持久的。如果你再次获取内核源代码、删除构建环境或者使用cleanall命令，那么你的变更会丢失。

你可能想，在进一步修改Linux内核配置之前，你希望设置CONFIG_LOCALVERSION[1]为一个定制字符串以识别已修改的内核。然而，那不起作用，因为构建系统通过配置变量LINUX_VERSION_EXTENSION来设置CONFIG_LOCALVERSION。

[1] CONFIG_LOCALVERSION是命令uname-r打印到控制台的字符串。

9.1.2 配置片段

当然，你不希望每次重新构建内核的时候都使用菜单编辑器来手动修改内核配置。对那个目的来说，构建系统提供了使用菜谱来合并部分配置（partial configuration）（被称为配置片段）到.config文件的机制。图9-1示例了配置片段（configuration fragment）的概念。

配置片段是这样一些文件，它们包含正如你在.config文件中找到的一行或者多行内核配置设置——例如，CONFIG_SMP=n。然后，你可以简单地增加那些文件到内核菜谱的SRC_URI。因为内核菜谱经常是由Yocto项目、OpenEmbedded或者板支持包层所提供的，所以我们推荐你创建自己的层并且使用到内核菜谱的追加文件而不是直接使用在那个原始层中的菜谱。

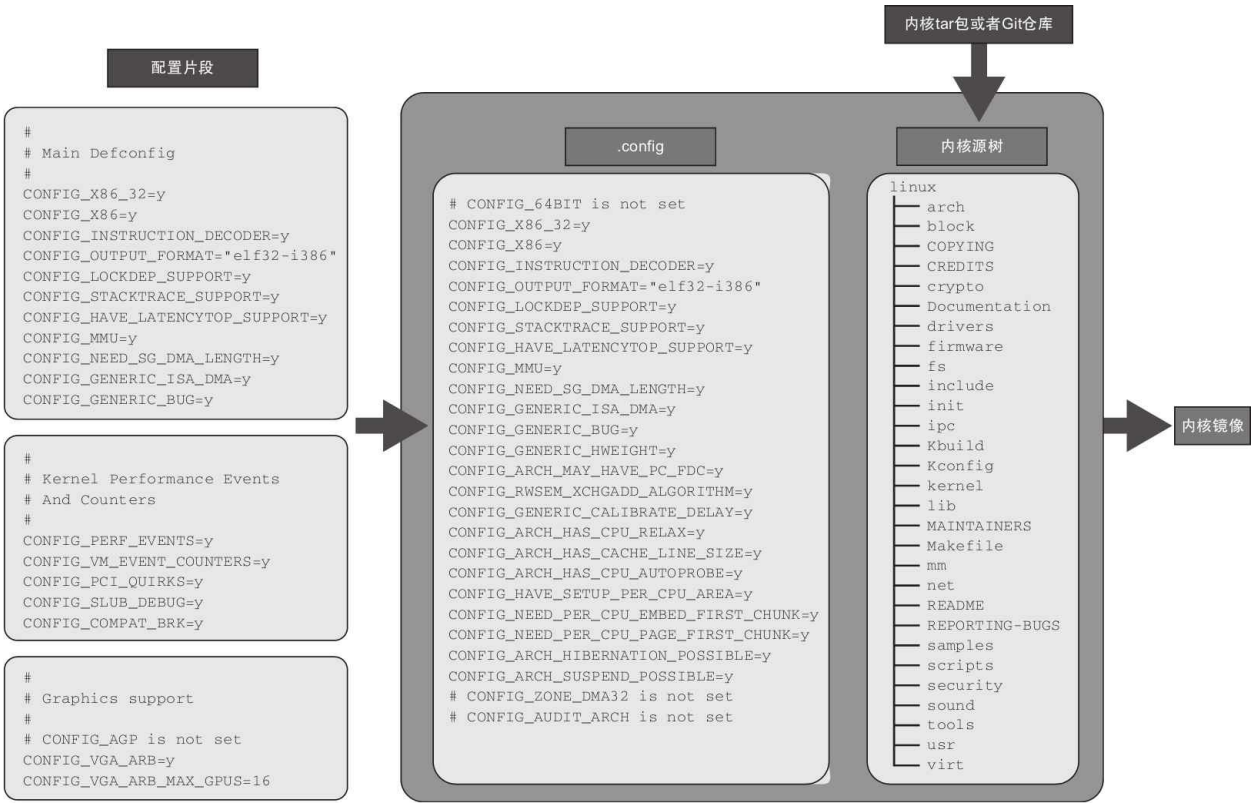

图9-1　配置片段

我们在第3章中解释了创建层。从一个已初始化的构建环境内使用

```
$ yocto-layer create ypbook
```

为你创建基础的层结构。增加你自己的新层到构建环境的conf/bblayers.conf中的BBLAYERS变量（见程序清单9-1）。

程序清单9-1　<builddir>/conf/bblayers.conf

```
# LAYER_CONF_VERSION is increased each time build/conf/bblayers.conf
# changes incompatibly
LCONF_VERSION = "6"

BBPATH = "${TOPDIR}"
BBFILES ?= ""

BBLAYERS ?= " \
  ${HOME}/yocto/poky/meta \
  ${HOME}/yocto/poky/meta-yocto \
  ${HOME}/yocto/poky/meta-yocto-bsp \
  ${HOME}/yocto/build/meta-ypbook \
  "
BBLAYERS_NON_REMOVABLE ?= " \
  ${HOME}/yocto/poky/meta \
  ${HOME}/yocto/poky/meta-yocto \
  "
```

现在为内核追加文件创建目录并且为配置片段文件创建一个子目录：

```
$ mkdir -p recipes-kernel/linux
$ mkdir -p recipes-kernel/linux/files
```

增加文件smp.cfg和内核追加文件到目录recipes-kernel/linux，如程序清单9-2所示。

程序清单9-2　配置片段

```
recipes-kernel/linux/files/smp.cfg:
# Disable SMP
CONFIG_SMP=n

recipes-kernel/linux/linux-yocto_3.19.bbappend:
# Include kernel configuration fragment
FILESEXTRAPATHS_prepend := "${THISDIR}/files:"
SRC_URL += "file://smp.cfg"
```

内核追加文件的名字依赖于当构建virtual/kernel时构建环境所用到的内核版本。你可以从内核构建输出中找到名字。在我们的例子中，内核菜谱是linux-yocto_3.19.bb。因此，我们创建的追加文件是linux-yocto_3.19.bbappend。为了使获取器能找到文件smp.cfg，到它的路径需要被增加到FILESEXTRAPATH变量。在我们的例子中，我们简单地把

文件放在和内核追加文件相同的目录中。一个菜谱或者菜谱追加文件的路径可以通过使用THISDIR变量来引用。

现在可以使用

```
$ bitbake -C fetch virtual/kernel
```

来重新构建内核。

这一次，我们显式地希望获取内核源，它现在包含我们的配置片段。在BitBake完成构建内核以后，你可以使用QEMU测试结果。

对于前面的例子，我们手动创建了包含配置片段的文件。如果你正在变更很多配置选项，特别是如果依赖设置是由菜单编辑器自动启用的，那么那种配置片段可能将难以追踪。

为了便于创建配置片段的任务，构建系统提供了diffconfig命令，它比较新旧配置并且创建配置片段。在用菜单编辑器编辑了配置后，执行：

```
$ bitbake -C diffconfig virtual/kernel
```

命令把配置片段放进${WORKDIR}。

在你已经创建了配置片段并且增加它到你的菜谱之后，你可以使用内核工具的配置验证来检查内核配置：

```
$ bitbake -C kernel_configcheck -f virtual/kernel
```

-C选项使用于kernel_configcheck的共享状态缓存失效以强制运行它，即使BitBake在之前执行过它。如果你的内核配置中有任何问题，那么构建系统把问题通知给你。

9.2 内核补丁

用菜谱应用补丁到内核源和用菜谱为一般软件包应用补丁没有不同。如果你已经有被格式化过的补丁文件，那么简单地提供文件并且增加它到用于内核菜谱的追加文件的SRC_URI。

为了从内核构建目录中的已修改内核源中创建补丁，请遵循这些概述了用于新内核驱动模块的工作流的步骤：

1.改变到内核源目录。改变你的工作目录到内核源目录。内核源目录可能有点难找。然而，和任何菜谱一样，变量${S}指向源目录。对于内核菜谱，${S}被设置成${STAGING_KERNEL_DIR}。为了找到内核源目录，使用命令：

```
$ bitbake -e virtual/kernel | grep STAGING_KERNEL_DIR
```

然后使用以上输出来改变到内核源目录。替代性地，你可以使用

```
$ bitbake -c devshell virtual/kernel
```

它在内核源目录中打开另一个终端窗口。

2.增加/修改内核源文件。对于这个例子，我们增加简单设备驱动到内核。编辑/添加文件，如下所示：

```
drivers/misc/Kconfig (add to the end of the file):
config YP_DRIVER
  tristate "Yocto Project Test Driver"
  help
     This driver does nothing but print a message.

drivers/misc/Makefile (add to the end of the file):
obj-$(CONFIG_YP_DRIVER) += yp-driver.o

drivers/misc/yp-driver.c (add new file):
#include <linux/module.h>

static int __init yocto_testmod_init(void)
{

        pr_info("Hello Kernel from the Yocto Project!");
}

static void __exit yocto_testmod_exit(void)
{
        pr_info("Gone fishing. I'll be back!");
}

module_init(yocto_testmod_init);
module_exit(yocto_testmod_exit);

MODULE_AUTHOR("Rudolf Streif <rudolf.streif@gmail.com");
MODULE_DESCRIPTION("Yocto Project Test Driver");
MODULE_LICENSE("GPL");
```

3.组织和提交变更。Yocto项目内核是从Git仓库检出的。因此，你可以简单地使用Git来创建补丁：

```
$ git status
$ git add .
$ git commit -m "Added Yocto Project Driver"
```

（注：替代性地，你可能希望使用git commit-s来把签署（signed-off-by）消息包含进补丁中。）

4.创建补丁文件。现在再次使用Git来从内核源的顶级目录中创建补丁文件：

```
$ git format-patch -n HEAD^
```

它创建文件0001-Added-Yocto-Project-Driver.patch。

5.移动补丁文件到你的层。复制或者移动补丁文件0001-Added-Yocto-Project-Driver.patch到我们在前面步骤创建的层的recipes-kernel/linux/files目录。

6.创建配置片段。因为我们正在增加一个新驱动，所以我们需要用配置片段启用它：

```
recipes-kernel/linux/files/yp-driver.cfg:
# Enable Yocto Project Driver
CONFIG_MISC_DEVICES=y
CONFIG_YP_DRIVER=y
```

7.增加配置片段和补丁到菜谱。现在需要增加配置片段和补丁到我们在前面步骤中创建的菜谱配置文件。

```
recipes-kernel/linux/linux-yocto_3.19.bbappend:
# Include kernel configuration fragment and patch
FILESEXTRAPATHS_prepend := "${THISDIR}/files:"
SRC_URI += "file://smp.cfg"
SRC_URI += "file://yp-driver.cfg"
SRC_URL += "file://0001-Added-Yocto-Project-Driver.patch"
```

8.构建内核。用以下命令来构建内核：

```
$ bitbake -C fetch virtual/kernel
```

现在，你可以通过运行QEMU并且在dmesg中寻找驱动的启动消息来验证结果。在以root登录后，执行

```
# dmesg | grep "Hello Kernel"
```

例子显示了你可以如何为了任何目的而直接为内核源打补丁。然而，仅仅当你希望把模块编译进内核时，你才需要直接为模块而给内核源打补丁。对于可以在运行时被加载的模块，你可以以树外模式编译模块。我们将在9.4节解释如何这样做。

9.3 内核菜谱

Poky发行版指定了如何用提供了必要指令的菜谱像类似构建任何其他软件包一样构建Linux内核。构建和打包Linux内核的复杂性，特别是为跨目标构建和打包，被内核类所隐藏。类kernel.bbclass是主要的类，它从各种其他类继承。内核菜谱从kernel继承，它简化开发内核菜谱成数行代码。

Yocto项目维护了自己的内核基础设施，它由用于内核源和元数据（例如配置、配置片段和补丁）的仓库构成。所有Yocto内核——也就是，用于QEMU机器的内核——以及由Poky提供的板支持包是从那个内核仓库构建出来的。很多公司正在使用Yocto项目内核用于它们的板支持包。

接下来的部分详细说明如何开发用于使用任何内核树来构建Linux内核的菜谱。9.3.2小节解释了Yocto项目内核基础设施和你可以如何使用它来用于你的项目。

9.3.1 从一个Linux内核树构建

有各种原因让你可能无法使用Yocto项目内核基础设施和它的内核版本之一用于你的嵌入式项目。不管原因是什么，你仍然可以利用构建系统提供用于构建Linux内核的工具化。

我们使用一个直接来自www.kernel.org的当前内核版本解释了从上游Linux内核树构建的过程。描述的机制应用到任何上游内核树，一个你可能已经从硬件提供者那里获得的内核树，或者一个你正在组织内部维护的内核树。描述的方法是“传统的”、用一个配置文件来组合内核源的内核方法，如图9-2所示。

你负责提供一个匹配你的硬件和内核版本的内核配置。内核树可以作为一个tar包来提供，你可以从www.kernel.org下载它，或者内核树可以直接从一个Git仓库检出。

从一个Linux内核tar包构建

从一个tar包来构建是构建一个Linux内核的经典方法。这种方法从一开始就已经被OpenEmbedded构建系统所支持。很多定制内核菜谱仍然在使用这种方法。虽然它不和新的内核工具化那样提供例如配置片段的特性，但是很多开发者倾向它是因为它的简单性以及它非常接近在不用Yocto项目时内核开发者通常使用的东西。程序清单9-3显示了一个从获取自www.kernel.org的tar包构建内核的菜谱。

在程序清单中的行因空间和可读性而被打断。特别是，SRC_URI在URI本身中不能包含任何空格或者换行。菜谱从kernel类继承，kernel类提供用于构建Linux内核的所有功能性并且保持菜谱本身相当简单。为了你自己的项目你可能需要调整下面的变量：

·LIC_FILES_CHKSUM：许可文件的名字和MD5校验和。在你第一次尝试构建菜谱时，你可以省略校验和。然后构建系统会抱怨缺少的校验和，但是也会为你计算它来复制进菜谱中。

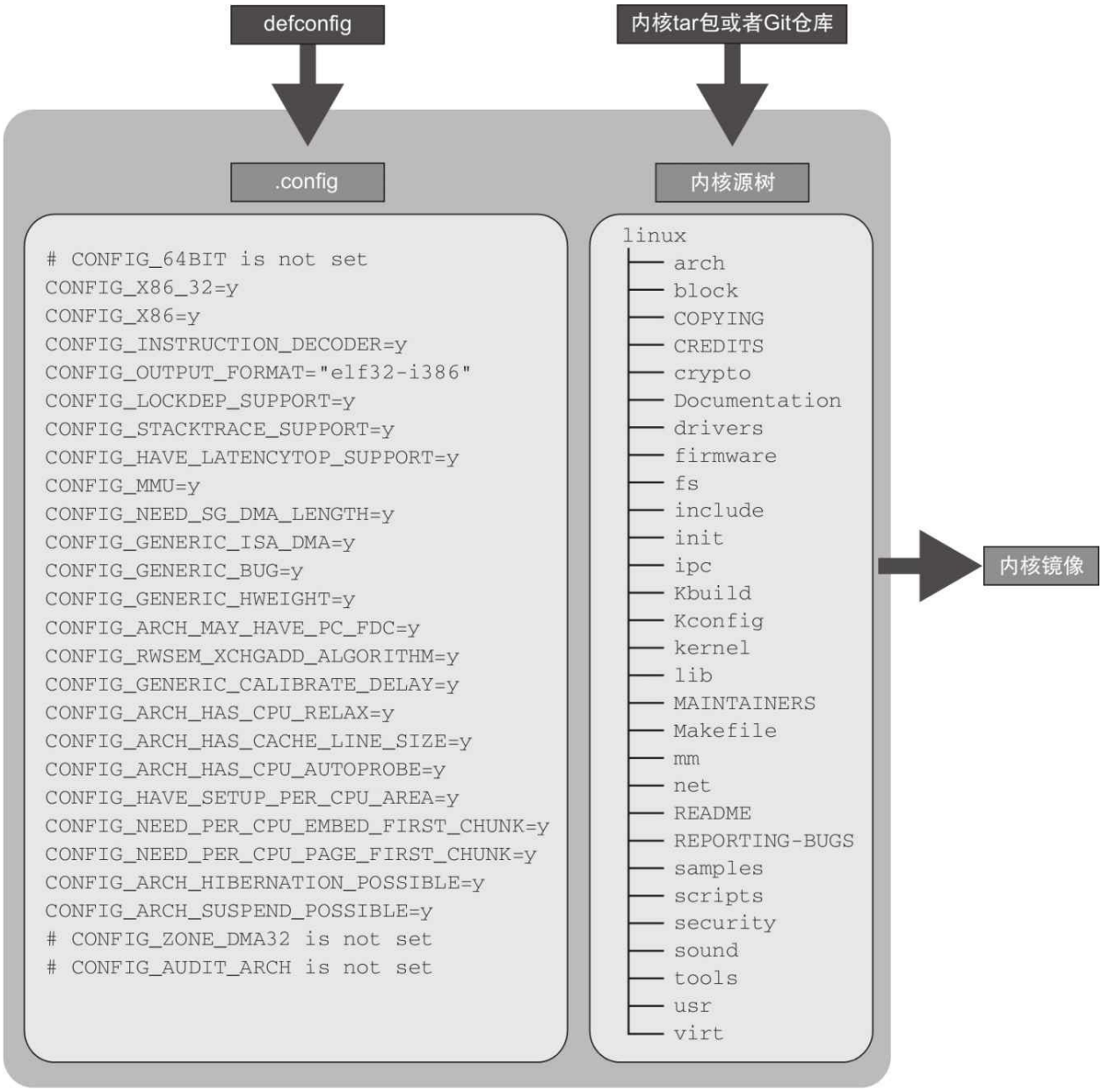

图9-2　传统内核方法

程序清单9-3　来自tar包的Linux内核（linux-ypbook_4.2.bb）

```
DESCRIPTION = "Linux Kernel from Tarball"
SECTION = "kernel"
LICENSE = "GPLv2"

inherit kernel

LIC_FILES_CHKSUM = "file://COPYING;md5=d7810fab7487fb0aad327b76f1be7cd7"

LINUX_VERSION ?= "${PV}"
LINUX_RC = "rc1"

SRC_URI = \

    "https://www.kernel.org/pub/linux/kernel/v4.x/testing/ \
     linux-${LINUX_VERSION}-${LINUX_RC}.tar.xz"
SRC_URI += "file://defconfig"

SRC_URI[md5sum] = "3e8331759af56ddd621528b2c7015ae1"
SRC_URI[sha256sum] = \
    "3c524ee0446b4ea8288708fa30acd28647317b9724f2d336052130e164c83f29"

S = "${WORKDIR}/linux-${LINUX_VERSION}-${PR}"

COMPATIBLE_MACHINE = "qemux86|qemux86-64"
```

·LINUX_VERSION：菜谱正在构建的Linux内核的版本号。

·LINUX_RC：Linux发布候选（release candidate）。

·SRC_URI：指定到Linux内核tar包的路径，它可以是远程的或者本地的。另外，变量必须指定一个包含内核配置的defconfig文件。

·SRC_URI[md5sum]，SRC_URI[sha256sum]：用于远程下载的校验和。对于你第一次尝试构建菜谱，你可以省略它。然后，构建系统会对缺失的校验和抱怨，但是会为你计算校验和，然后你可以复制它们进菜谱中。

·S：内核源被解压到的目录。它必须反映源包的名字。

·COMPATIBLE_MACHINE：被内核支持的机器的名字列表。名字由管道符（|）分隔。

把菜谱保存进你的层的recipes-kernel/linux。也增加一个defconfig文

件到recipes-kernel/linux，或者更好的是，增加到recipes-kernel/linux/linux-ypbook。这样做允许你为不同内核互相隔离defconfig文件。

在你可以用你的新内核菜谱开始构建内核之前，在你构建环境的conf/local.conf中设置

```
PREFERRED_PROVIDER_virtual/kernel ?= "linux-ypbook"
```

以告诉系统使用你的新内核菜谱来构建Linux内核。现在，你已经可以启动构建了，执行：

```
$ bitbake linux-ypbook
```

在构建完成后，用QEMU测试你的新内核。

从Linux内核Git仓库构建

如果你正在积极开发Linux内核，那么你可能正使用Git来开发。毕竟，Git是Linus Torvalds为了支持Linux内核社区的开发过程而创建的。当工作于Git时，能够直接从Git仓库而不是从一个tar包构建Linux内核是有益的。程序清单9-4显示了一个关于直接从位于www.kernel.org的Linus Torvalds的Git仓库构建Linux内核的样例菜谱。

程序清单9-4　从Git仓库构建Linux内核（linux-ypbook_git.bb）

```
DESCRIPTION = "Linux Kernel from kernel.org Git Repository"
SECTION = "kernel"
LICENSE = "GPLv2"

require recipes-kernel/linux/linux-yocto.inc

LIC_FILES_CHKSUM = "file://COPYING;md5=d7810fab7487fb0aad327b76f1be7cd7"

LINUX_VERSION ?= "4.2"
```

```
LINUX_VERSION_EXTENSION ?= "-ypbook"
PV = "${LINUX_VERSION}+git${SRCPV}"
SRC_URI = \
    "git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git;\
     protocol=git;nocheckout=1"
SRC_URI += "file://defconfig"

SRCREV = "d770e558e21961ad6cfdf0ff7df0eb5d7d4f0754"

COMPATIBLE_MACHINE = "qemux86|qemux86-64"
```

菜谱包含来自meta/recipes-kernel/linux的linux-yocto.inc。这个包含文件被所有Yocto项目内核菜谱使用并且继承kernel类和kernel-yocto类。除了kernel类的功能性，kernel-yocto类也提供了用于从Git仓库构建的工具化以及例如配置片段的其他功能性。调整菜谱的变量以满足你的需求：

·LIC_FILES_CHKSUM：许可文件的名字和MD5校验和。在你第一次尝试构建菜谱时，你可以省略校验和。然后构建系统会抱怨缺少的校验和，但是也会为你计算它来复制进菜谱中。

·LINUX_VERSION：菜谱正在构建的Linux内核的版本号。

·PV：由${LINUX_VERSION}、字面量+git和${SRCPV}变量连接起来的字符串，它包含由源仓库维护的源代码修订。这个默认设置是按照惯例的，你通常不必为自己的菜谱而修改它。

·SRC_URI：指定到Linux内核源的Git仓库的URI，它可以是本地或者远程仓库。另外，变量必须指定一个包含内核配置的defconfig文件。

·SRCREV：内核源的修订。

·COMPATIBLE_MACHINE：被内核支持的机器的名字列表。名字由管道符（|）分隔。

你存储和构建这个菜谱，正如前面部分中讲到的，你存储和构建用

于从tar包构建的菜谱。

应用配置设置和补丁

为了应用内核配置设置，你增加它们到defconfig文件，而你把内核菜谱提供给defconfig文件。如果你正通过包含来自meta/recipes-kernel/linux的linux-yocto.inc来使用Yocto内核工具化，那么你可以使用配置片段，正如9.1.2小节所示。使用Yocto项目工具化不限于从Git仓库构建菜谱。对于从Linux内核tar包构建的菜谱来说，你可以使用它。当然，你可以先用菜单编辑器测试配置变更。

补丁以在9.2节中描述的方法被应用。

使用入树配置文件

继承自kernel-yocto类的内核菜谱可以使用包含在内核源中的defconfig文件，通常被称为入树（in-tree），而不是用SRC_URI+="file：//defconfig"来提供defconfig文件（被称为树外）。这样做的一个原因是，你可能不希望在你的层中维护defconfig配置文件的副本而是希望使用来自内核树的默认配置。然后，配置片段允许你进一步定制内核配置。

为指定入树配置文件，增加如下行到你的内核菜谱：

```
KBUILD_DEFCONFIG_<KMACHINE> ?= "<defconfig file>"
```

其中，你用内核机器的名字替换<KMACHINE>，用在内核树中的defconfig文件名替换<defconfig file>。例如：

```
KBUILD_DEFCONFIG_beaglebone ?= "omap2plus_defconfig"
```

9.3.2 从Yocto项目内核仓库构建

Yocto项目维护自己的内核基础设施，它包含用于内核源的仓库和元数据以及一组强大的工具集，该工具集帮助管理内核源和配置数据。

在前面的章节中，我们演示了如何通过提供defconfig、配置片段和菜谱补丁来定制内核。尽管这个方法好用且灵活，但是它把内核源与配置分离开了，导致维护更加困难。

使用入树配置为defconfig解决了问题，但是少一些灵活，因为只有默认配置可以以那个方式来提供。额外配置和补丁依然需要用菜谱来提供。入树配置也要求移植配置到每个新版本的Linux内核中。

另一个问题是，如何对相同内核版本的不同目标硬件维护和应用不同的内核配置和补丁，以及如何跨多个硬件平台来启用内核特性和增加补丁。

理想的解决方案将是移动配置和补丁，使其靠近内核源而不是入树，并且提供工具化来灵活地为大量目标硬件选择补丁和配置。对Yocto项目内核设施来说，Yocto项目内核开发者已经设计了这样的解决方案。

Yocto项目内核基础设施

Yocto项目内核开发者为每个其采用的Linux内核版本创建一个仓库。Yocto项目Git服务器托管了仓库[1]。图9-3描述了Yocto项目内核仓库的基础结构。

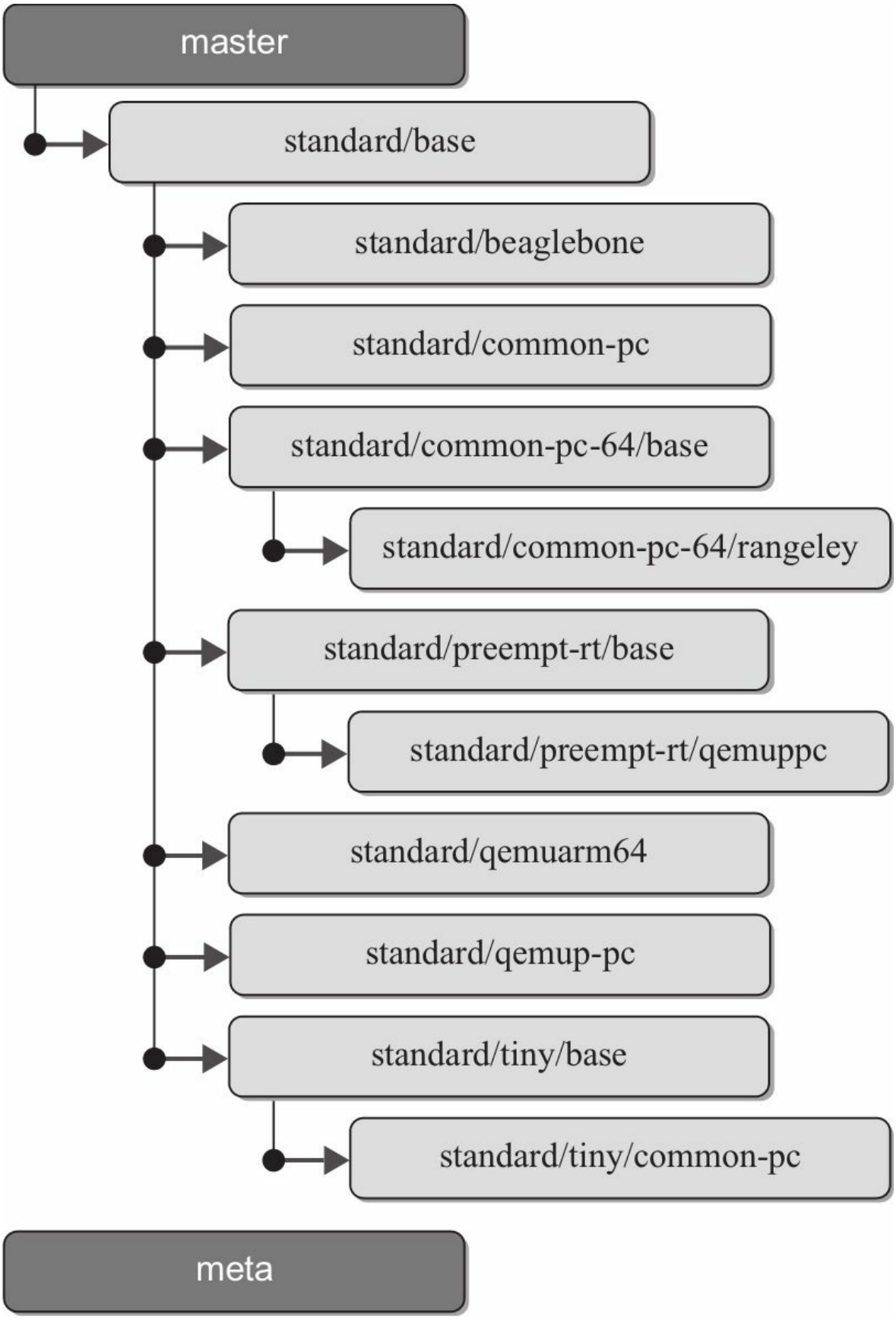
master
standard/base
standard/beaglebone
standard/common-pc
standard/common-pc-64/base
standard/common-pc-64/rangeley
standard/preempt-rt/base
standard/preempt-rt/qemuppc
standard/qemuarm64
standard/qemup-pc
standard/tiny/base
standard/tiny/common-pc
meta

图9-3　Yocto项目内核仓库结构

每个仓库有多个用于内核源的分支以及一个用于元数据的分支。内核源分支要么是基分支（base branch），要么是板支持包分支。基分支提供在各种板支持包分支中共享的通用功能性。显示在图9-3（为了更清晰，我们从该图中省略了板支持包的一些分支）中的分支如下：

·主干分支（Master Branch）：主干分支是来自www.kernel.org的各自分支的精确克隆。这个分支是用来自www.kernel.org的更新来维护的。然而，从它不包含任何Yocto项目调整的意义上来说，它保持了原始状态。所以基分支是从主干分支衍生的。

·基分支：当前，有3个直接从主干衍生出来的基分支。基分支为板支持包分支提供了基础。基分支通常会有冲突的配置，这是它们为什么分开的原因。

·标准基分支（standard/base）：这个基分支用于标准内核配置。

·实时基分支（standard/preempt-rt/base）：这个基分支用于实时内核。它应用PREEMPT-RT补丁到内核源中。

·小型基分支（standard/tiny/base）：这个分支配置一个非常紧凑的Linux内核。

·板支持包分支：可能有任何数量的板支持包分支。板支持包分支是从基分支中衍生出来的。

·通用PC（standard/common-pc）：适用于32位x86架构的板支持包分支。分别各有一个从标准基和从小型基衍生的分支。

·通用PC64（standard/common-pc-64/base）[2]：适用于64位

x86（x86_64）架构的板支持包分支。

·BeagleBone：用于BeagleBone板的板支持包分支。

·Rangeley：用于基于Intel Atom处理器C2000产品家族（代码名Rangeley）的板的板支持包分支。

·QEMU PPC：适用于PowerPC模拟的板支持包分支。分别各有一个从标准基和从实时基衍生的分支。

·QEMU ARM 64：适用于ARM 64位架构（ARMv8-A）的板支持包分支

·元分支：这个分支包含用于内核源分支的元数据。这个分支不是从任何其他分支中衍生的，而是通过提供用于其他分支的配置和补丁来补充内核源。因为这个分支是独立的，所以它也被称为孤儿分支（orphan branch）。

内核源分支的命名惯例指示其系谱。从standard/base继承的板支持包分支的分支名以standard开头。因此，standard/preempt-rt/base分支继承自standard/base，standard/tiny/base也是。如果Yocto项目内核开发者对standard/base做了变更，那么这个变更被合并进所有继承自它的分支中。

内核分支和来自元分支的数据被构建系统整合以创建有效的内核配置。图9-4示例了原则。

程序清单9-5显示了用于构建3.14实时内核的菜谱。因为它很好地演示了内核源和元数据是如何被整合的，所以我们选择这个内核菜谱。

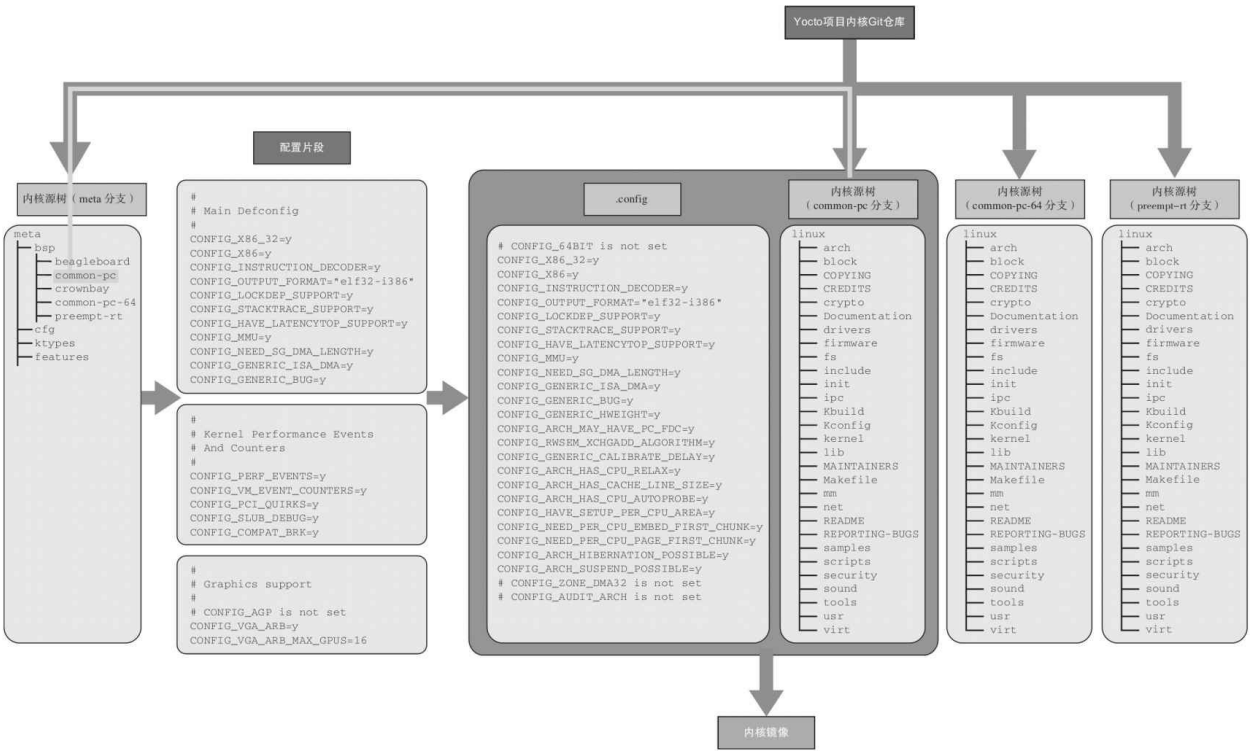

图9-4　Yocto项目内核基础设施

程序清单9-5　整合内核源和元数据（linux-yocto-rt_3.14.bb）

```
KBRANCH ?= "standard/preempt-rt/base"
KBRANCH_qemuppc ?= "standard/preempt-rt/qemuppc"

require recipes-kernel/linux/linux-yocto.inc

SRCREV_machine ?= "3428de71031ede23682dd0842b9cfc23ae465f39"
SRCREV_machine_qemuppc ?= "32c44a56a8218c3349f50d8151a10252d5e85dd1"
SRCREV_meta ?= "a996d95104b72c422a56e7d9bc8615ec4219ac74"

SRC_URI = "git://git.yoctoproject.org/linux-yocto-3.14.git; \
   bareclone=1;branch=${KBRANCH},meta;name=machine,meta"

LINUX_VERSION ?= "3.14.36"

PV = "${LINUX_VERSION}+git${SRCPV}"

KMETA = "meta"
KCONF_BSP_AUDIT_LEVEL = "2"

LINUX_KERNEL_TYPE = "preempt-rt"

COMPATIBLE_MACHINE = "(qemux86|qemux86-64|qemuarm|qemuppc|qemumips)"

# Functionality flags
KERNEL_EXTRA_FEATURES ?= "features/netfilter/netfilter.scc features/taskstats/
taskstats.scc"
KERNEL_FEATURES_append = " ${KERNEL_EXTRA_FEATURES}"
KERNEL_FEATURES_append_qemux86=" cfg/sound.scc cfg/paravirt_kvm.scc"
KERNEL_FEATURES_append_qemux86-64=" cfg/sound.scc"
```

我们以逻辑顺序而非变量在菜谱中出现的顺序来看看这些变量：

·SRC_URI：指定到Yocto项目内核Git仓库（在本例中是linux-yocto-3.14.git）的URI。两个分支通过branch参数来检出：内核源分支和元数据分支。name参数把内核源分支命名为machine，把元数据分支命名为meta。

·KBRANCH：提供用于内核源的分支。这个变量是本地于菜谱的。默认值是standard/preempt-rt/base，对于qemuppc机器，这个值被条件性地覆盖成standard/preempt-rt/qemuppc。

·SRCREV：指定用于内核源和元数据分支的源修订。

·LINUX_VERSION：设置Linux版本号。

·PV：设置包版本号。PV是通过连接LINUX_VERSION和字符串

+git以及SRCPV来获得的。

·LINUX_KERNEL_TYPE：按照基内核分支设置要构建的内核的类型。

·standard：用于构建自standard/base和衍生自standard/base的内核分支的所有内核（除了实时和小型内核外）的设置。如果LINUX_KERNEL_TYPE没有被显式地设置，那么它默认成standard。

·preempt-rt：用于构建自standard/preempt-rt/base和衍生自standard/preempt-rt/base的内核分支的所有实时内核的设置。

·tiny：用于构建自standard/tiny/base和衍生自standard/tiny/base的内核分支的所有实时内核的设置。

由LINUX_KERNEL_TYPE设置的内核类型必须匹配内核分支。例如，当KBRANCH="standard/base"时，你不能用LINUX_KERNEL_TYPE="preempt-rt"。

·KMETA：设置元数据分支名字。默认是以它自己的名为meta的孤儿分支提供元数据。替代性地，元数据可以存在于内核源树的名为meta的目录中。在这个情况下，KMETA被设置成空字符串。

·COMPATIBLE_MACHINE：由管道符（|）分隔的目标机器的名字列表，该内核可以与之一起使用。

·KERNEL_FEATURES：包含内核特性配置的文件列表。

·KCONF_BSP_AUDIT_LEVEL：kernel-yocto类可以检测和报告不正确的内核配置。这个变量设置报告什么类型的配置错误。

·KCONF_BSP_AUDIT_LEVEL="0"：不报告任何配置错误。

·KCONF_BSP_AUDIT_LEVEL="1"：报告任何指定的但不包含在最终内核中的配置设置。

·KCONF_BSP_AUDIT_LEVEL="2"：除了设置成1的功能之外，报告指定在非硬件配置中的硬件设置。

·KMACHINE：Linux内核已知的硬件机器的名字。每个内核菜谱都必须设置这个变量。虽然它没有在菜谱例子中被显式地设置，但是它被包含文件linux-yocto.inc设置成等于MACHINE变量，MACHINE是构建系统所知道的机器的名字。在大部分情况下，这个默认是足够的。在MACHINE不精确地代表Linux内核通过其识别机器的机器名的情况下，KMACHINE变量提供构建系统机器名和内核机器名之间的映射。

例如，你提供自己的目标机器配置，并且命名为excalibur。构建系统按照excalibur来引用这个机器，但是技术上，excalibur是基于Intel Core i764位CPU。Linux内核按照目标机器的CPU名字来引用这个机器：intel-corei7-64。当你让构建系统为excalibur构建时，为了告诉它构建适合Intel Core i7 CPU的内核，增加KMACHINE_excalibur="intel-corei7-64"到内核菜谱。

使用由构建系统提供的菜谱，你可以很容易地创建自己的内核菜谱，这些内核菜谱从Yocto项目仓库中构建Linux内核。你也可以克隆Yocto项目内核仓库，并且创建你自己的、继承自一个基分支的板支持包分支以包含特定于你的机器的驱动和其他代码。这个方法允许你从Yocto项目内核开发者正在基分支上做的维护工作中受益。当对特定内核版本来说原本重要的新安全补丁或者其他更新被www.kernel.org发布后，Yocto项目内核开发者合并它们到基分支并且测试它们。然后，你可以择优而取并且合并它们到你自己的板支持包分支中。

元数据语法

包含在Yocto项目内核仓库的meta分支中的数据可以被分类成以下类别：

·配置片段文件：配置片段文件是以.cfg结尾并且包含内核配置设置的文件，正如在9.1.2小节所描述的。

·补丁文件：以.patch结尾的补丁文件被应用到内核源，正如在9.2节所描述的。

·描述文件：以.scc结尾的描述文件描述和聚合配置片段和补丁。描述也可以包含其他描述。

描述文件

描述文件描述和聚合配置片段和补丁以及它们如何被包含在Linux内核的构建中。它们使用由以下关键字组成的脚本编程语言：

·define：定义变量。

·kconf：应用配置片段。

·patch：应用补丁。

·include：包含另一个SCC文件。

·if[<condition>];then<block>fi：按照<condition>的求值来条件性地执行<block>。当其他SCC包含那个文件时，条件可以包含由其他SCC文件设置的变量。

集合描述由一个描述文件与配置片段文件和补丁文件构成。有不同类别的集合描述。

配置集合描述

元数据配置集合描述由一个或者多个包含Linux内核配置参数的配置文件以及一个描述配置片段集合的配置描述文件构成。程序清单9-6显示了启用扩展固件接口（Extended Firmware Interface，EFI）支持的配置集合描述。

程序清单9-6　配置集合描述

```
cfg/efi.cfg:
   # EFI Support
   # Dependencies
   CONFIG_PCI=y
   CONFIG_ACPI=y
   # Enable basic EFI support
   CONFIG_EFI=y
   CONFIG_EFI_STUB=y
   CONFIG_EFIVAR_FS=m

efi.scc:
   define KFEATURE_DESCRIPTION "Core EFI support"
   define KFEATURE_COMPATIBILITY arch
   kconf hardware efi.cfg
```

配置描述文件定义两个变量：

·KFEATURE_DESCRIPTION：用户工具可展示给用户的简短描述。

·KFEATURE_COMPATIBILITY：配置的兼容性：

·board：与特定板兼容。

·arch：与特定架构兼容。

·all：与所有板和架构都兼容。

kconf指令被用来包含实际配置片段。hardware关键字标记配置作为启用硬件功能，而non-hardware关键字用于一般配置。这种区分对内核

构建没有影响，但是对于内核配置验证工具有影响。

补丁集合描述

补丁集合描述（Patch Collection Description）由至少一个补丁文件和描述补丁集合的SCC文件构成。如果补丁提供可配置的内核功能性，那么集合描述也可能包含启用它们的配置文件。程序清单9-7显示了用于针对ARM架构的一个补丁的元数据集合描述。

程序清单9-7　补丁集合描述

```
patches/arm.scc:
   # patches are for everyone, but the kconfig data is just for ARM builds.
   if [ "$KARCH" = "arm" ]; then
      kconf hardware arm.cfg
      include cfg/timer/hz_100.scc
   fi
   include v7-A15/v7-A15.scc
   patch arm-ARM-EABI-socketcall.patch
   patch vexpress-Pass-LOADADDR-to-Makefile.patch
```

该示例显示了如何在不考虑架构的情况下应用补丁，但是只在构建架构（KARM）被设置成arm时才启用。

特性集合描述

特性集合描述启用可能需要整合很多不同配置和补丁以及包含其他集合描述的复杂内核特性。程序清单9-8显示了启用虚构的内核测试框架的特性集合。

程序清单9-8　特性集合描述

```
features/testframework.scc:
   define KFEATURE_DESCRIPTION "Enable Kernel Test Framework"

   patch 0001-test-framework-core.patch
   patch 0002-test-framework-proc.patch

   include cfg/testframework-deps.scc
   kconfig non-hardware testframework.scc
```

特性提供了用于启用特定功能性的更高级别的聚合。在你的内核菜谱中，你通常通过增加SCC文件的相对路径和名字到KERNEL_FEATURES变量来启用由特性集合描述所描述的特性。

内核类型集合描述

内核类型集合描述为3个不同的内核类型——标准、preempt-rt和小型——聚合默认设置、补丁和特性。内核类型集合描述本质上是特性集合描述。

内核类型集合描述只用于隔离目的。在LINUX_KERNEL_TYPE和其他描述之间没有联系。为了使用它们，板支持包集合描述必须显式地包含它们。

板支持包集合描述

板支持包集合描述聚合特定硬件平台所要求的配置、补丁和特性。程序清单9-9显示了用于Intel的MinnowBoard的针对标准内核类型的板支持包集合描述。

程序清单9-9　板支持包集合描述

```
bsp/minnow/minnow-standard.scc:
  define KMACHINE minnow
  define KTYPE standard
```

```
define KARCH i386

include ktypes/standard
include minnow.scc

# Extra minnow configs above the minimal defined in minnow.scc
include cfg/efi-ext.scc
include features/media/media-all.scc
include features/sound/snd_hda_intel.scc

# The following should really be in standard.scc
# USB live-image support
include cfg/usb-mass-storage.scc
include cfg/boot-live.scc

# Basic profiling
include features/latencytop/latencytop.scc
include features/profiling/profiling.scc

# Requested drivers that don't have an existing scc
kconf hardware minnow-drivers-extra.cfg
```

所有板支持包集合描述都必须为构建系统定义KMACHINE、KTYPE和KARCH以识别适合于内核菜谱定义的需求的集合描述。

内核菜谱通过设置KMACHINE和KTYPE变量来表达它们的需求。构建系统把板支持包集合描述的KMACHINE和KTYPE与由内核菜谱设置的KMACHINE和LINUX_KERNEL_TYPE分别匹配以找到为菜谱所需的内核提供正确配置的板支持包。

元数据组织

元数据可以以菜谱内空间（in-recipe space）或者入树的形式提供。菜谱内空间配置意味着，集合描述、配置片段和补丁是由菜谱提供的。入树配置意味着，元数据是在分支中提供的，通常是内核仓库的meta分支。

菜谱内空间元数据

对菜谱内空间配置，你把文件放进FILESEXTRAPATHS下的目录层级结构中，如程序清单9-10所示。

程序清单9-10　菜谱内空间元数据

```
meta-mylayer
└── recipes-kernel
    └── linux
        ├── linux-custom
    |   ├── bsp-standard.scc
    |   ├── bsp.cfg
    |   └── standard.cfg
        └── linux-custom_4.2.bb
```

为了使菜谱知道配置集合，你必须通过增加

```
SRC_URI += "file://bsp-standard.scc"
```

来把SCC文件包含进SRC_URI。

因为在例子中的内核菜谱名字是linux-custom_4.2.bb并且FILESEXTRAPATH被自动地设置成包含${THISDIR}/${PN}，所以BitBake能找到集合描述。

入树元数据

入树元数据被存储在通常为meta的元数据层，它是内核仓库的一部分，正如程序清单9-11所示。

程序清单9-11　入树元数据

```
meta
└── cfg
    └── kernel-cache
        ├── bsp-standard.scc
        ├── bsp.cfg
        └── standard.cfg
```

路径meta/cfg/kernel-cache是内核工具所期望的，所以是强制的。在kernel-cache目录下，元数据可以被组织在任何目录层级结构中，而这正是Yocto项目内核仓库的meta分支的情况。Yocto项目内核仓库meta分支

结构如程序清单9-12所示。

程序清单9-12　Yocto项目内核仓库meta分支

```
meta
└── cfg2
    └── kernel-cache
        ├── 00-README
        ├── arch
        │   ├── arm
        │   ├── mips
        │   ├── omap
        │   ├── powerpc
        │   └── x86
        ├── backports
        ├── bsp
        │   ├── arm-versatile-926ejs
        │   ├── beagleboard
        │   ├── beaglebone
        │   ├── common-pc-64
        │   ├── common-pc
        │   └── ...
        ├── cfg
        │   └── ...
        ├── features
        │   ├── amt
        │   ├── aufs
        │   ├── bfq
        │   ├── blktrace
        │   ├── bluetooth
        │   └── ...
        ├── ktypes
        │   ├── base
        │   ├── preempt-rt
        │   ├── standard
        │   └── tiny
        ├── kver
        ├── patches
        │   ├── boot
        │   ├── build
        │   ├── debug
        │   ├── drivers
        │   ├── exports
        │   ├── misc

        │   ├── net
        │   └── patches.scc
        ├── scripts
        └── staging
```

集合描述、配置片段和补丁被组织成子目录：

·arch：用于架构的特性集合描述

·backports：用于从更新内核版本向后移植（back-port）功能性的补丁的补丁集合描述

·bsp：版支持包集合描述

·cfg：配置集合描述

·features：启用非硬件特性的特性集合描述

·ktypes：内核类型集合描述

·patches：补丁集合描述

·staging：临时补丁（staging patch）

scripts目录包含工具化脚本，kver文件包含内核版本。

为了在用内核菜谱构建Linux内核时应用入树元数据，你必须增加集合描述文件及其相对路径到KERNEL_FEATURES变量。例如，为了增加另一个联合文件系统（Another Union File System，AUFS）特性，使用以下命令：

```
KERNEL_FEATURES_append = " features/aufs/aufs.cfg"
```

变量KERNEL_FEATURES包含空格分隔的集合描述文件的列表。因为_append不增加空格，所以你必须显式地增加空格。不要直接给KERNEL_FEATURES赋予集合描述列表，因为构建系统用自己的列表填充这个变量。

在前面我们解释了，元数据分支是内核仓库中的孤儿分支。孤儿分支是与仓库的任何其他分支都没有关联的分支。如果你希望使用带有自己的元数据分支的自有内核源仓库，那么你必须在仓库中创建孤儿分支：

```
$ cd <kernel repo>
$ git checkout --orphan meta
$ git rm -rf .
$ git commit -allow-empty -m "Create orphan meta branch"
$ mkdir -p meta/cfg/kernel-cache
```

现在，你可以开始向新元数据分支增加元数据文件了。在增加和修改了文件后，你必须提交它们到仓库中。一旦你提交了它们，你需要调整你内核菜谱中的SRCREV_meta为新的提交散列；否则，构建系统获取一个更旧的版本。忘记调整SRCREV_meta是一个让人沮丧的常见错误。

元数据应用

构建系统通过以下方式来收集正确的元数据列表：

·把板支持包集合描述的KMACHINE和KTYPE分别与内核菜谱设置的KMACHINE和LINUX_KERNEL_TYPE匹配，以找到包含其所有内容的板支持包集合

·包含在SRC_URI中找到的任何集合描述

·求值变量KERNEL_FEATURES

从这个信息中，内核工具会为内核配置创建一个合并的.config文件，以及一个应用补丁到内核源的整合列表。

长期支持计划内核

Yocto项目内核开发者基于各种标准来从上游www.kernel.org选择Linux内核。其中之一就是长期支持。通常，www.kernel.org大约每90天发布新内核。一旦新内核版本已经发布了，它所替换的前面版本将不再接收任何更新。对那些可能需要用于安全性和其他已选改进的补丁的嵌

入式系统来说，这可能是有问题的，其无法升级到更新的内核版本。

为了解决这个问题，一些公司在Linux基金会的支持下已经创建了长期支持计划（Long-Term Support Initiative，LTSI）[3]。长期支持计划选择特定Linux内核版本，并且自原始内核发布日期开始的2年内，用补丁来维护它们，并且把从更新内核版本中精挑细选的特性移植到它们中去。

Yocto项目已和长期支持计划合作，并且Yocto项目内核开发者总是为Yocto项目内核选择长期支持计划内核。每个长期支持计划内核确实最终会成为Yocto项目内核。但是，不是所有Yocto项目内核都是长期支持计划内核，因为Yocto项目内核开发者通常在长期支持计划发布之间选择额外的内核版本。

如果你选择的Yocto项目内核也是一个长期支持计划，那么你会获得维护内核的好处，该内核维护时间从其最初发布日期开始长达3年。

[1] http://git.yoctoproject.org/。
[2] 在作者看来，这个分支的命名是错误的。为了和分支命名惯例一致，它的名字实际上应该是standard/common-pc-64，因为这个分支不是基分支而是用于x86_64架构的板支持包分支。
[3] http://ltsi.linuxfoundation.org/what-is-ltsi。

9.4 树外模块

当然，构建内核模块的最简单方法是入树。模块的源代码已经被集成进了Linux内核源树，只要为内核kconfig配置工具正确设置了模块，那么构建内核模块仅仅是通过配置参数启用它罢了。

然而，那不总是一个选项。你可能已经收到了作为源包的模块的源代码，这对由硬件厂商提供的设备驱动来说是常见的。即使对你自己的模块，你也可能选择树外构建而不是集成它们的源代码到Linux内核源树。为了那个目的，构建系统提供了module类，它包含用于构建树外模块的大部分逻辑[1]。

[1] 尽管如此，如果可能，总是尝试提交你的内核模块上游到Linux内核源。一旦到了上游，你的模块将自动由内核社区在各个内核版本中维护。

9.4.1 开发内核模块

如果你确实编写了自己的内核模块，那么，当然，你负责源文件。这允许你编写makefile，以便模块类可以直接使用它而不用对菜谱中的编译和安装函数做任何修改。

Yocto项目提供了关于一个模块的简单例子，该模块包括一个C源文件、一个许可文件以及makefile和用于构建模块的一个对应的菜谱。你可以在以下找到这个例子：

```
poky/meta-skeleton/recipes-kernel/hello-mod
```

菜谱是简单直接的。为了便利性，我们复制它到程序清单9-13中。

程序清单9-13　模块Recipe（hello-mod_0.1.bb）

```
SUMMARY = "Example of how to build an external Linux kernel module"
LICENSE = "GPLv2"
LIC_FILES_CHKSUM = "file://COPYING;md5=12f884d2ae1ff87c09e5b7ccc2c4ca7e"
inherit module

PR = "r0"
PV = "0.1"

SRC_URI = "file://Makefile \
           file://hello.c \
           file://COPYING \
          "

S = "${WORKDIR}"

# The inherit of module.bbclass will automatically name module packages
# with "kernel-module-" prefix as required by the oe-core build
# environment.
```

正如你为任何菜谱所做的，你必须提供SUMMARY、LICENSE、LIC_FILES_CHKSUM和SRC_URI。例如，后者仅仅包含由菜谱内空间提供的3个文件。当然，如果你有更复杂的模块，那么你会结构化和打包源文件。该示例也设置了PV，而严格来说，它是冗余的，因为构建

系统从菜谱文件的名字中获取它。菜谱从模块类中继承构建逻辑，并且也设置S为${WORKDIR}，因为获取器直接复制源文件到那里。

对菜谱来说，那就是全部了，因为复制在程序清单9-14中的makefile遵守module类的构建目标和参数惯例。

程序清单9-14　模块makefile（Makefile）

```
obj-m := hello.o

SRC := $(shell pwd)

all:
	$(MAKE) -C $(KERNEL_SRC) M=$(SRC)

modules_install:
	$(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules_install

clean:
	rm -f *.o *~ core .depend .*.cmd *.ko *.mod.c
	rm -f Module.markers Module.symvers modules.order
	rm -rf .tmp_versions Modules.symvers
```

这是一个简单而典型的Linux内核模块makefile。模块类要求一个默认目标（all）和一个名为modules_install的安装目标。内核模块必须从内核源树内构建。因此变量KERNEL_SRC和-C参数一起被传递。模块类设置KERNEL_SRC为STAGING_KERNEL_DIR，它包含构建系统保存内核源的位置。参数M[1]告诉构建系统，树外内核模块正在被构建。M必须被设置成模块的源目录。

程序清单9-15　模块类（poky/meta/classes/module.bbclass）

```
DEPENDS += "virtual/kernel"

inherit module-base kernel-module-split

addtask make_scripts after do_patch before do_compile
do_make_scripts[lockfiles] = "${TMPDIR}/kernel-scripts.lock"
do_make_scripts[deptask] = "do_populate_sysroot"

module_do_compile() {
        unset CFLAGS CPPFLAGS CXXFLAGS LDFLAGS
        oe_runmake KERNEL_PATH=${STAGING_KERNEL_DIR}   \
                   KERNEL_SRC=${STAGING_KERNEL_DIR}    \
                   KERNEL_VERSION=${KERNEL_VERSION}    \
                   CC="${KERNEL_CC}" LD="${KERNEL_LD}" \
                   AR="${KERNEL_AR}" \
                   ${MAKE_TARGETS}
}

module_do_install() {
        unset CFLAGS CPPFLAGS CXXFLAGS LDFLAGS
        oe_runmake DEPMOD=echo INSTALL_MOD_PATH="${D}" \
                   KERNEL_SRC=${STAGING_KERNEL_DIR} \
                   CC="${KERNEL_CC}" LD="${KERNEL_LD}" \
                   modules_install
}

EXPORT_FUNCTIONS do_compile do_install

# add all splitted modules to PN RDEPENDS, PN can be empty now
KERNEL_MODULES_META_PACKAGE = "${PN}"
FILES_${PN} = ""
ALLOW_EMPTY_${PN} = "1"
```

我们复制模块类在程序清单9-15中，因为理解类在编写内核模块及其菜谱时有帮助。

·类设置virtual/kernel作为构建依赖，这保证在构建系统尝试构建模块之前内核已经被构建了。

·它继承自module-base，module-base定义了make_scripts任务以在构建模块之前构建内核脚本。它在patch任务之后compile任务之前增加那个任务。Linux内核树包含一些为主机系统构建的工具。一些模块需要那些工具。然而，STAGING_KERNEL_DIR不包含这些工具的二进制版本。构建系统在为目标打包内核源之前从STAGING_KERNEL_DIR中去除了这些二进制。理由是，为主机系统构建的工具不适用于目标，当然，这意味着，在内核已经被打包好后，为了构建模块，工具必须被再

次构建。

·kernel-module-split类处理内核模块的打包。主要包被前缀以kernel-module-。对于我们的hello-mod例子，kernel-module-hello-mod包包含实际内核模块二进制。

·在module_do_compile任务中，除了KERNEL_SRC，类也传递KERNEL_PATH到makefile。传递这两个参数为最常用在模块makefile中的内核源目录处理这两个变量名。类也为来自变量KERNEL_CC、KERNEL_LD和KERNEL_AR的编译器（CC）、链接器（LD）和打包器（AR）传递命令，这当然包括用于目标架构的正确的交叉工具栏版本。

·在运行module_do_install任务时，类传递DEPMOD=echo到makefile。因为几乎所有模块都设计成构建和安装在主机系统上，安装目标通常调用depmod实用程序来创建符号图（symbol map）。当然，当在一个主机系统上为另一个目标系统构建模块时，那是不合适的。因此，depmod被替换成echo，它仅仅产生日志输出。

当使用Yocto项目来构建第三方模块时，你可能必须覆盖module_do_compile或module_do_install任务以匹配参数。

[1] 在旧内核模块的makefile中，你也可能找到变量SUBDIRS，它是为了向后兼容而保留的。

9.4.2 创建用于第三方模块的菜谱

通常，树外模块是和用于原生地构建模块的makefile一起交付的；也就是，为构建系统原生地构建模块。看一下makefile，它可以告诉你是否需要调整以及如何调整菜谱来构建内核模块。这里是一些要注意的项目：

·内核源目录：为了使过程对用户来说是简化的，模块开发者通常把逻辑构建到makefile中以自动化地检测内核源的位置。当为主机系统构建时，其工作得很好；但是对于Yocto项目构建，这不好使。因此，你需要寻找用于内核源目录的变量，例如KSRC、KERNEL_PATH或者KERNEL_SRC，并且最终在你的菜谱中赋值它。

·构建目标：大部分内核模块makefile定义编译模块的默认构建目标。如果没有目标被显式地传递到makefile，那么这个目标被调用，这是用于module_do_compile任务的默认值。对大部分情况，其工作得很好。

·安装目标：module类期望安装目标为modules_install，它是内核开发管理。然而，很多模块仅仅使用install。

·子目录结构：如果makefile不在模块源包的顶级目录中而是在一个包含源的子目录中，那么你必须对应地调整S变量。

·许可文件：构建系统期望许可文件位于模块源包的顶级目录中。如果不是这种情况，那么你的菜谱必须把它复制到那里。向你的菜谱增加一个do_configure_prepend用于复制许可文件。

程序清单9-16作为一个例子，显示了为Intel PCI-E 40千兆网络连接

[1]构建Linux驱动的菜谱。

程序清单9-16　用于Intel PCI-E 40 Linux驱动的菜谱

```
SUMMARY = "Base Driver for the Intel(R) XL710 Ethernet Controller Family"

LICENSE = "GPLv2"
LIC_FILES_CHKSUM  = "file://COPYING;md5=d181af11d575d88127d52226700b0888"

inherit module

PR        = "r0"

# Point SRC_URI to SDK tarball
```

```
SRC_URI =  "file://${TOPDIR}/../i40e-1.2.46.tar.gz"

do_configure_prepend() {
    # license file is expected to be in ${S}
    cp ${WORKDIR}/${P}/COPYING ${S}
}

module_do_compile() {
        unset CFLAGS CPPFLAGS CXXFLAGS LDFLAGS
        oe_runmake KSRC=${STAGING_KERNEL_DIR}   \
                   BUILD_KERNEL=${KERNEL_VERSION}    \
                   KVER=${KERNEL_VERSION}    \
           INSTALL_MOD_PATH="${D}" \
                   CC="${KERNEL_CC}" LD="${KERNEL_LD}" \
                   AR="${KERNEL_AR}" \
                   ${MAKE_TARGETS}
}

module_do_install() {
        unset CFLAGS CPPFLAGS CXXFLAGS LDFLAGS
        oe_runmake DEPMOD=echo INSTALL_MOD_PATH="${D}" \
                   BUILD_KERNEL=${KERNEL_VERSION}    \
                   KSRC=${STAGING_KERNEL_DIR} \
                   KVER=${KERNEL_VERSION}    \
                   CC="${KERNEL_CC}" LD="${KERNEL_LD}" \
                   install
}

S = "${WORKDIR}/${P}/src"
```

对很多模块，你可以在你的菜谱中调整来自module类的默认值以构建模块。这是更受欢迎的方法，因为你不必为模块源代码打补丁。然而，可能有一些模块，你必须对它们提供补丁才能够构建模块。对这样的模块，以菜谱内空间的方式提供补丁并且增加它们到SRC_URI。

[1] 你可以在https://downloadcenter.intel.com/download/24411/Network-Adapter-Driver-for-PCI-E-40-Gigabit-Network-Connections-under-Linux-找到用于驱动的源包。

9.4.3 把模块包含在根文件系统中

最后步骤是，在用于目标的根文件系统镜像中包含模块。绝大部分模块是用于硬件驱动的，这当然意味着，只有在目标硬件相应地配备了相关硬件的情况下，把模块包含进根文件系统才是有价值的。为了这个目的，构建系统提供了如下的变量，它们通常是被设置在机器配置文件中的[1]：

·MACHINE_ESSENTIAL_EXTRA_RDEPENDS：镜像构建所需的机器特定包列表。在构建系统可以创建镜像之前，任何你增加到这个变量的模块包都必须被构建。因为它是机器必要的变量，所以在列表中的包都被认为是机器启动的关键。变量由所有基于packagegroup-core-boot的镜像来评估。

·MACHINE_ESSENTIAL_EXTRA_RRECOMMENDS：为机器推荐的机器特定包列表，与前一个变量相似，但是构建过程不依赖于它们。这意味着，即使这个列表中的包不存在，镜像依然能成功构建。这个列表中的包是机器必要的，意味着它们是机器启动所需要的。这听起来矛盾，但是对于可能被编译进Linux内核而不是作为模块包来提供的模块来说是有意义的。

·MACHINE_EXTRA_RDEPENDS：镜像构建所需要的机器特定包列表。然而，相较于MACHINE_ESSENTIAL_EXTRA_RDEPENDS，这些包对于机器启动来说不是必要的。这个变量由所有基于packagegroup-base的镜像来评估。

·MACHINE_EXTRA_RRECOMMENDS：为机器推荐的但对启动机器不是必要的机器特定包列表。

例如，增加

```
MACHINE_EXTRA_RDEPENDS += "kernel-module-<module name>"
```

到你的机器定义文件以在你的镜像中包含内核模块。

你可能有疑问，是否可以增加模块到镜像菜谱中的IMAGE_INSTALL变量。当然，那是可能的，但是如果模块不依赖于机器特定的硬件特性的话，那么那样做是不被推荐的。

[1] 在下一章我们讨论Yocto项目板支持包时，我们将讨论机器配置。

9.4.4 模块自动加载

构建系统可以使用你目标系统上的/etc/modules-load.d和/etc/modprobe.d来配置自动但静态的模块加载。尽管依赖于由硬件识别和类似触发器基础上的udev提供的自动模块加载通常是一个比较好的主意，但是，可能在一些用例中，启动时的静态模块加载是有意义的。

当使用systemd和systemd-modules-load.service时，变量KERNEL_MODULE_AUTOLOAD指定在启动时需要被加载的模块的列表：

```
KERNEL_MODULE_AUTOLOAD += "module1 module2 module3"
```

对每个列出的模块，构建系统在in/etc/modules-load.d中创建一个以模块名字命名的、以.conf结尾的、内容包含要加载的模块名字的文件。文件的词典编撰的顺序决定模块的加载顺序。

为了通过/etc/modprobe.d来为模块提供配置参数，你使用变量KERNEL_MODULE_PROBECONF和module_conf_<module_name>。和KERNEL_MODULE_AUTOLOAD类似，KERNEL_MODULE_PROBECONF仅仅是模块名字的列表。对于在KERNEL_MODULE_PROBECONF中的每个项，构建系统要求一个module_conf_<module_name>变量，它指定如modprobe所需要的模块配置。例如，对虚构的名为foofighter且需要两个配置变量的模块，使用

```
KERNEL_MODULE_PROBECONF += "foofighter"
Module_conf_foofighter = "options foofighter foo=1 bar=2"
```

你通常在用于你模块的菜谱中放置这些设置。

9.5 设备树

简单来说，设备树是描述硬件平台的数据结构。与将设备及其配置的每个细节硬编码到内核源中不同，在启动时，一个数据结构被传递到内核。这些细节包括I/O地址、内存地址空间、中断等。设备树编译器（Device Tree Compiler，DTC）把设备树从其人类可读的层级格式编译成通常被称为扁平设备树（Flattened Device Tree，FTD）的二进制格式。

OpenEmbedded构建系统支持使用设备树编译器从设备树源文件构建扁平设备树。设备树源文件以.dts结尾。包含FTB的文件以.dtb结尾。在本节中，我们解释如何用构建系统为给定的平台或者机器构建设备树[1]。

设备树是平台或者机器特定的，因为它们描述平台的硬件配置。它们是作为内核菜谱的一部分而被构建的。为使其工作，内核菜谱必须包含：

```
require recipes-kernel/linux/linux-dtb.inc
```

或者：

```
require recipes-kernel/linux/linux-yocto.inc
```

而后者包含前者。

现在你必须通过设置变量KERNEL_DEVICETREE为用于你平台的扁平设备树的名字，以告诉构建系统为你的平台构建什么设备树。例如，

```
KERNEL_DEVICETREE = "am335x-bone.dtb am335x-boneblack.dtb"
```

命令构建系统为BeagleBone（White）和BeagleBoneBlack生成扁平设备树文件。这个变量的最佳位置是机器配置文件，其包含所有机器特定的设置。

[1] 如果你正在寻找关于设备树是如何被结构化的以及如何为你的平台开发设备树的信息，那么参考www.devicetree.org和http://elinux.org/Device_Tree。

9.6 总结

在本章中，我们聚焦在用Yocto项目构建Linux内核和内核模块上。

·使用菜单编辑器和配置片段，构建系统提供了允许快速往返测试和内核配置集成的工具化。

·用于内核源代码的补丁的应用方式，和对任何其他软件包应用补丁的方式完全相同。

·在最简单的形式下，Linux内核可以通过提供菜谱内空间或者入树配置来直接从来自源tar包或Git仓库的内核树源中构建。

·Yocto项目维护它自己的Linux内核基础设施，Linux基础设施由包含内核源以及配置元数据和补丁的仓库组成。Yocto项目内核工具化允许使用元数据特性进行灵活的配置。

·Yocto内核基础设施为3个基础配置（标准、实时和小型）提供了维护的内核源，用于板支持包的特定分支是从这3个基础配置中衍生出来的。

·Yocto项目采用在多年内提供持续支持的长期支持计划（LSTI）内核，这对嵌入式项目来说尤其有益。

·通过module类，构建系统支持树外内核模块的便利构建。

9.7 参考文献

Linux内核构建系统，Documentation/kbuild/kbuild.txt。

Yocto项目Linux内核开发手册，www.yoctoproject.org/docs/1.8/kernel-dev/kernel-dev.html。

第10章　板支持包

前面的章节为使用Yocto项目工具构建定制Linux系统铺垫了基础。我们使用软件系统模拟器QEMU来测试。在本章中，我们更进一步地探讨并且还示例了构建系统如何支持创建用于实际硬件的Linux内核和根文件系统。

硬件支持是由板支持包（Board Support Package，BSP）提供的。如果你熟悉嵌入式系统文档，那么你很可能之前已经听说过术语板支持包。然而，尽管有相同的名字，但是Yocto项目板支持包与用于典型嵌入式系统的更传统的板支持包是相当不同的。

本章首先解释Yocto项目板支持包背后的理念以及为什么Yocto项目板支持包不同于传统板支持包。然后，我们使用板支持包来为一个嵌入式评估板构建系统，把那个系统放在那个板上，最后启动板。在本章的结尾，我们概述了如何为自己的硬件创建Yocto项目板支持包。

10.1 Yocto项目板支持包理念

总体来说，板支持包是用于给定硬件（例如嵌入式设备板）的给定操作系统的特定调整。板支持包通常是由半导体和板厂商提供给他们的客户的，以便客户在厂商的硬件上构建、加载和运行操作系统。

传统板支持包通常由下面这些术语组成：

·文档：我们把文档放在首位，因为任何板支持包都应该包含文档。文档描述板支持包的内容，提供关于板支持包支持的特定硬件的信息，包含关于如何使用板支持包为硬件构建操作系统、把操作系统镜像传输到硬件以及启动硬件的使用说明。另外，文档可能包含如何调整板支持包以用于类似板支持包支持的参考硬件的硬件的信息。

·开发工具：通常，厂商包含至少一个带有编译器、汇编程序、链接器和打包器的工具链。工具链匹配被支持的硬件和源代码。一些板支持包甚至可能包含软件开发工具包以及最终的集成开发环境，例如Eclipse，这使得对目标硬件的开发更加便利。

·操作系统源代码和二进制：如果操作系统是开源的，那么一些板支持包甚至会包含用于操作系统的完整源代码。如果操作系统是专有的，那么仅仅对编译设备驱动和应用软件来说必要的源代码（例如头文件和静态连接的库）才可能被包含。所有其他操作系统文件是以二进制形式提供的。因为操作系统源和二进制可能是巨大的，所以常见的是，在构建时，所提供的开发工具使用厂商的仓库和下载站点来按需获取源代码和二进制。

·源代码补丁：如果板支持包针对的硬件需要特殊的设备驱动、配置或者位于基础操作系统软件包之上的其他模块，那么板支持包可能提

供它们。替代性地，常见的是，厂商已经为操作系统源打了补丁，以至板支持包不需要包含任何补丁。

·文件系统镜像：板支持包也可能包含用于被支持的硬件的完整文件系统镜像，这非常方便，因为它有助于调出目标，并且在构建你自己的系统时提供参考。

包含从文档到文件系统镜像的一切的板支持包非常有利于用户快速上手，因为大部分的依赖性是由包含在板支持包中的东西解决的。然而，长期维护有时候会成为挑战，特别是在厂商没有提供定期更新的情况下。我们依然可以看到很多用于片上系统和使用Linux内核版本2.6的开发板的厂商板支持包。

如果嵌入式项目希望利用那些片上系统和仅仅被更新的内核版本支持的其他硬件，那么这就成了问题。如果片上系统厂商不为板支持包提供用于后来的Linux内核的更新，那么作为系统管理员的你将面对这样一个任务：移植板支持包到更新的内核版本，或者向后移植设备支持到由板支持包支持的更旧的内核版本中。这两种方法都有潜在的陷阱。

移植板支持包到后来的Linux内核版本对于具有未知历史的板支持包来说可能变得非常困难：厂商什么时候分支了上游源？应用过什么修改？

另一方面，如果设备驱动依赖的特性仅仅在最新Linux内核中可用，那么向后移植设备驱动也许是不可能的。

Yocto项目为板支持包采用了不同的方法：

·Yocto项目板支持包不是独立的，这一点不像传统板支持包。它们依赖于例如Open-Embedded核心（meta）以及可能其他层的基础元数据层。

·Yocto项目板支持包不包含构建系统或者任何开发工具。它们是由Yocto项目本身提供，并在构建过程中被创建。

·除了菜谱和最终的补丁，Yocto项目板支持包不包含任何源代码。

·Yocto项目板支持包仅仅关心特定于特殊硬件的组件。

本质上，Yocto只不过是为了支持目标硬件而包含对核心层的增加和修改的特殊元数据层。所有Yocto项目板支持包都至少依赖OpenEmbedded核心元数据层。

Yocto项目板支持包把板支持包的维护限定为其增加的包和用于它所修改的底层的层。所有其他维护工作是底层的责任。当然，这种方法意味着Yocto项目板支持包的一个特定版本是和它所依赖的底层的特定版本绑定的。如果底层变化——例如，如果元数据层不再支持一个特定内核版本——那么板支持包必须被调整。然而，这种维护工作通常远远少于维护传统板支持包。

Yocto项目板支持包的维护工作被Poky构建系统的依赖性处理进一步缩减。大部分包并不依赖于特定目标硬件。由条件变量覆盖提供的依赖性处理保证，构建依赖于硬件的包（例如Linux内核）的菜谱会获得关于目标硬件以及架构的正确信息。

板支持包依赖性处理

构建系统的板支持包依赖性处理不但减轻了维护工作而且也使得板支持包可以很容易地互相交换，简直就是在配置文件中修改一行代码的事。对于像你一样的系统构建者来说，这是巨大的收益。它使得你能够在今天为一个板构建完全相同的根文件系统，并且在下一天为另一个甚至可能使用具有完全不同架构的片上系统的板构建完全相同的根文件系

统。我们把这个特性叫作Yocto项目板支持包的正交性（orthogonality），虽然术语从数学角度的正交性[1]来看并不完全正确，但是用户可以为下一次交换一个板支持包而不用担心变更构建系统的其他部分。

快速地看一下构建系统如何处理目标特定的依赖性是值得的，因为它们常见于依赖硬件配置的菜谱中。

在第4章的4.4.2小节，讨论了使用变量OVERRIDES实现的BitBake条件变量赋值机制。这种机制是构建系统用于板支持包的依赖性处理的基础。变量OVERRIDES包含逗号分隔的覆盖条件列表，从左到右，覆盖条件的优先级递增。Poky参考发行版设置变量为如下表达式[2]：

```
OVERRIDES="${TARGET_OS}:${TRANSLATED_TARGET_ARCH}:build-${BUILD_OS}: \
    pn-${PN}:${MACHINEOVERRIDES}:${DISTROOVERRIDES}: \
    ${CLASSOVERRIDE}:forcevariable"
```

对特定目标，对应地扩展表达式内的变量。例如，对于64位x86 Qemu目标，

```
OVERRIDES="linux:x86-64:build-linux:pn-core-image-minimal: \
    qemuall:qemux86-64:poky:class-target:forcevariable:libc-glibc"
```

板支持包特定的设置是由变量MACHINEOVERRIDES提供的，对这个例子来说，它扩展成qemuall：qemux86-64。

本质上，有两个设置：qemuall和qemux86-64。前者提供用于QEMU机器的覆盖，而后者特别为64位x86模拟提供覆盖。后者有更高优先级，因为它是列在前者之后的。如果特定变量需要机器依赖设置，那么这仅仅是把机器覆盖追加到变量罢了。例如，

```
KERNEL_FEATURES_append_qemux86-64=" cfg/sound.scc"
```

如果目标构建是针对qemux86-64，那么它增加配置描述到KERNEL_FEATURES。

像前面那个，板支持包频繁地在菜谱和追加文件中使用条件变量赋值来为特定目标微调菜谱。

对本章的后续部分来说，除非我们另有显式声明，否则我们将把Yocto项目板支持包等称为板支持包，以使得文本更加清晰。

[1] 数学上的正交性意味着两个维度是完全互不依赖的。当然，对于构建系统来说，那是不正确的，因为由板支持包提供的目标依赖变量设置覆盖构建系统的标准设置。从用户视角来看，不考虑板支持包的细节，板支持包和构建系统看起来是相互独立的。

[2] 你可以通过在构建环境中执行bitbake-e core-image-minimal|grep OVERRIDES来获得这个信息。

10.2 用板支持包构建

板支持包仅仅是一个元数据层，它包含一个配置文件形式的机器定义，其中机器的名称以.conf结尾。机器配置文件位于板支持包层的conf/machine子目录内。为了使用板支持包，你需要增加它到构建环境conf/bblayers.conf文件中的BBLAYERS变量。然后，必须设置构建环境conf/local.conf文件内的MACHINE变量为你希望为其构建的机器的名字。

技术上，贯穿前面的章节，我们一直在隐式地将板支持包用于qemux86构建。你可以在位于poky/meta/conf/machine/qemux86.conf的构建环境的OpenEmbedded核心元数据层中找到用于qemux86的机器定义。OpenEmbedded核心元数据层为不同架构的各种模拟机器提供机器定义：qemuarm、qemuarm64、qemumips、qemumips64、qemuppc、qemux86和qemux86-64。提供机器定义本质上使该层成为板支持包层。

Yocto项目也包含用于实际硬件机器的自有板支持包层：meta-yocto-bsp。这个层默认包含在每个由oe-init-build-env创建的构建环境的BBLAYERS中。它提供用于Texas Instruments BeagleBone[1]板、Ubiquity Networks EdgeRouter[2]、Freescale MPC8351E-RDB参考平台[3]和通用32位以及64位x86平台的机器定义。

为任何这些目标平台进行构建都是简单的。文件conf/local.conf已经包含用于目标平台的MACHINE设置。你仅仅需要取消注释你希望为其构建的目标平台。

当然，这是一个非常有限的硬件平台列表。Yocto项目包含它们是为了便利性和测试目的。有好多其他来自片上系统厂商、板厂商和社区的用于大量不同硬件的板支持包可用。在10.2.2小节，我们将深入探讨

如何查找和利用外部板支持包的细节。

[1] http://beagleboard.org/bone。
[2] https://www.ubnt.com/edgemax/edgerouter。
[3] http://www.nxp.com/files/32bit/doc/fact_sheet/MPC8315ERDBFS.pdf。

10.2.1 为BeagleBone构建

BeagleBone是基于Texas Instruments AM335x ARM Cortex-A8片上系统的开发板。硬件和软件是开放设计，它是由BeagleBoard.org基金会创建和支持的[1]，该基金会是设在美国的非盈利组织，它的目标是“提供嵌入式计算中开源软件、硬件的设计和使用的教育并推广之”。

BeagleBone有多个变体：原始的BeagleBone（White）、BeagleBone Black以及现在的BeagleBone Green。根据各自印制电路板（Printed Circuit Board，PCB）的颜色，它们容易被区分。BeagleBone Black使用具有1GHz时钟速度的更强劲版本的AM3358片上系统，提供512MB内存（是原始BeagleBone内存的2倍），并且提供2GB（在修订C型号中是4GB）板载嵌入式多媒体（embedded MultiMedia Card，eMMC）存储。而且，BeagleBone Black仅仅花费稍微比原始板一半多一点的成本。然而，Black版没有配备用于系统控制台的USB到串行的转换器。当运行Yocto项目构建时，串行系统控制台对于与系统交互是必要的，因为板没有显示器[2]。BeagleBone Green是基于BeagleBone Black的，但是去除了板载高清多媒体接口（High-Definition Multimedia Interface，HDMI）连接器，以便为两个承载I2C信号的连接器创造空间来与由Seeed Studios[3]提供的Grove传感器简单地对接。

构建BeagleBone镜像

为了用OpenEmbedded构建系统构建用于BeagleBone的Linux系统，在构建环境的conf/local.conf中简单地取消以下注释（以及最终注释掉前面你曾经使用过的所有其他MACHINE设置）：

```
MACHINE = "beaglebone"
```

你之前曾将相同的构建环境用于qemux86构建系统，这是没有关系的，qemux86是基于x86架构的机器。然而，如果你倾向于把东西分开，那么，你可以通过引用oe-init-build-env来创建新的构建环境。如果确实这么做了，那么要确保为你前面使用过的目录设置DL_DIR和SSTATE_DIR。它节省大量用于下载共享源包和重新创建架构间共享的构建制品的时间。

现在用以下命令开始构建：

```
$ bitbake -k core-image-minimal
```

一旦构建完成，你可以在构建环境的tmp/deploy/images/beaglebone子目录中找到用于引导加载程序、Linux内核和根文件系统的镜像。Poky整齐地把用于不同机器的镜像分开到它们自己的目录中，避免它们混合在一块。

现在你用这些镜像做什么？

理解BeagleBone启动过程

为了能够启动目标硬件，你需要理解目标硬件（在我们的例子中是BeagleBone）如何启动它的操作系统。BeagleBone以特定的方式从它的外部SD卡中启动。在BeagleBone.org上，你能够找到这个信息。对此总结如下：

1.通电复位（Power-On-Reset，POR）后，BeagleBone的片上系统从它的板载ROM中加载和运行阶段（stage）0引导加载程序。

2.阶段0引导加载程序访问名为MLO的文件，它必须位于SD卡第一个分区的第一个扇区中。MLO是阶段1引导加载程序，它是由U-Boot的二次程序加载器（Secondary Program Loader，SPL）功能性实现的。

3.U-Boot二次程序加载器MLO配置BeagleBone的片外（off-chip）内存，然后加载文件u-boot.img，它是完整的U-Boot引导加载程序。在这个过程中，U-Boot是阶段2引导加载程序。

4.然后，U-Boot加载Linux内核镜像到内存中，并且把控制传给Linux内核。U-Boot默认要求内核镜像uImage位于SD卡第二分区的/boot目录中。第二分区包含具有用于BeagleBone的完整根文件系统的Linux ext3文件系统。

5.接着，Linux内核开始它的启动过程，按照由引导加载程序提供的内核命令行的指示挂载根文件系统，并且最终启动第一个用户空间进程。

SD卡要求一个具有两个分区的特定布局：一个FAT启动分区和一个包含根文件系统的Linux分区。用于根文件系统的Linux分区可以是任何Linux支持的文件系统，例如ext3和ext4。Poky构建系统顶级目录内的文件README.hardware描述了如何创建分区和格式化它们。

创建启动SD卡

你可以通过遵循说明和使用命令fdisk、mkfs.vfat、mkfs.ext3来手动地启动BeagleBone以完成SD卡的分区和格式化。然而，用小脚本自动化这个过程是更加方便的。程序清单10-1显示了用于为BeagleBone分区和格式化SD卡的脚本。

程序清单10-1　用于分区和格式化BeagleBone SD卡的脚本

```
# (c) bbonesd.sh, 2015, Rudolf J Streif
#!/bin/sh

echo "Partitioning and formatting SD card for BeagleBone"

# test for the most common problems
test "$(id -u)" != "0" \
    && echo "You need to be 'root' to run this script." \
    && exit 1
test -z "$1" && \
    echo "No disk device specified." \
    && exit 1
test "$1" = "/dev/sda" \
    && echo "OOPS - System disk specified: ${1}" \
    && exit 1

# be sure before continuing
echo "Are you sure that you want to format ${1}? Type YES to proceed."
read RESPONSE
test "$RESPONSE" != "YES" && echo "Exiting." && exit 1

# now do the job
DRIVE=$1
if [ -b "$DRIVE" ] ; then
        dd if=/dev/zero of=$DRIVE bs=1024 count=1024
        SIZE=`fdisk -l $DRIVE | grep Disk | awk '{print $5}'`
        echo DISK SIZE - $SIZE bytes
        CYLINDERS=`echo $SIZE/255/63/512 | bc`

        echo CYLINDERS - $CYLINDERS
        {
                echo ,9,0x0C,*
                echo ,200,0x83,-
                echo ,,0x83,-
        } | sfdisk -D -H 255 -S 63 -C $CYLINDERS $DRIVE
        if [[ $1 == '/dev/sd'* ]] ; then
                mkfs.vfat -F 32 -n "beagboot" ${DRIVE}1
                mkfs.ext3 -L "beagroot" ${DRIVE}2
        else
                mkfs.vfat -F 32 -n "beagboot" ${DRIVE}p1
                mkfs.ext3 -L "beagroot" ${DRIVE}p2
        fi
fi
echo Done.
```

在你把SD卡插入进开发系统，用dmesg命令确定设备：

```
$ dmesg
[ 4389.803854] sd 9:0:0:1: [sdf] 15278080 512-byte logical blocks: \
    (7.82 GB/7.28 GiB)
[ 4389.822434]  sdf: sdf1
```

在这个例子中，SD卡已经被认作成了/dev/sdf，所以你需要用

```
$ sudo ./bbonesd.sh /dev/sdf
```

来调用脚本。

脚本工作于被认作SCSI驱动（/dev/sd*，通常，当用一个USB读卡器连接时）的SD卡以及被认作内存块设备（/dev/mmcblk*，通常，当插入计算机的一个卡槽时）的SD卡。

既然分区和格式化了SD卡，那么需要复制引导加载程序和根文件系统到它们各自的分区。以下假设启动分区被挂载在/media/beagboot，根分区被挂载在/media/beagroot。如果你的系统把它们挂载到不同的挂载点上，那么需要对应地做出调整。从构建环境的顶级目录中，改变到具有用于Beaglebone的镜像的目录：

```
$ cd tmp/deploy/images/beaglebone
```

复制引导加载程序文件到启动分区中：

```
$ sudo cp MLO /media/beagboot
$ sudo cp u-boot.img /media/beagboot
```

解压根文件系统和内核模块到根分区中：

```
$ tar x -C /media/beagroot -f core-image-minimal.tar.bz2
$ tar x -C /medai/beagroot -f modules-beaglebone.tgz
```

复制内核镜像和设备树文件到根分区的/boot目录中：

```
$ cp zImage /media/beagroot/boot
$ cp zImage-am335x-bone.dtb /media/beagroot/boot/am335x-bone.dtb
$ cp zImage-am335x-boneblack.dtb /media/beagroot/boot/am335x-boneblack.dtb
```

复制和解压文件可能要花费一些时间。即使当最后的命令返回了，进程也可能没有完成，因为Linux对硬盘操作提供了写缓冲。使用

```
$ sync
```

来冲刷所有缓冲并且在卸载和移除SD卡之前等待命令返回。如果你从命令行使用umount，那么在卸载之前，它等待文件系统缓冲被冲刷，因为在文件系统被实际卸载前sync是被umount调用的。

现在，已经准备好启动BeagleBone板。

启动BeagleBone

为了启动BeagleBone板并且遵循它的启动过程，你需要为它提供电力并且连接它的控制台串口到你的开发系统。如果你自己调用BeagleBone（White），那么启动是非常简单直接的。如果你拥有一个BeagleBone Black，那么你需要进行一些额外的步骤。

连接BeagleBone（White）

为了连接BeagleBone（White）到开发计算机，你需要一个一端是USB A插头、另一端是一个微型USB插头的USB线缆。USB A插头插进你的开发计算机，微型USB插头插进BeagleBone（White）下侧的USB端口中。USB线缆提供电力，也提供介于BeagleBone的控制台端口和你的开发系统之间的串行连接。因为BeagleBone（White）自带USB线缆，所以不需要额外的硬件。

在连接板之前，把SD卡插进BeagleBone（White）的SD卡槽中。

连接BeagleBone Black

连接BeagleBone Black要稍微更难一点儿，因为这个板没有内置的串行到USB的转换器。你需要外部的串行到USB的转换器线缆，而BeagleBone Black没有提供。这些线缆通常称为FTDI线缆，是以生产转换器芯片和线缆的公司Future Technology Devices International Ltd.首字母命名的。

BeagleBone Black在它的顶部有一个6针脚、单列0.1英寸（1英寸=25.4毫米）间距的连接器。这是用于它的串行控制台端口的连接器。为了连接到它，需要一根一端是USB A连接器而另一端是6向、单列连接器的串口到USB转换器线缆。你也需要注意BeagleBone Black串行控制台端口的信号电平是3.3V。也有用于5V信号电平的线缆，对BeagleBone使用5V线缆可能损坏板的CPU。正确的线缆是FTDI TTL-232R-3V3。你可以在FTDI的网站上找到它[4]。大部分组件配电器携带这类线缆。

为了连接线缆到板，把它设置成顶部朝上，以太网和5V电力连接器朝左。线缆连接器的黑线连到板连接器的最左侧的针脚。那个针脚在板上以一个小白点标示出来。

串行到USB的转换器线缆不为BeagleBone提供电力。你可以连接一个能够提供至少1.2A（或者6W）的5V电源到桶形连接器，或者使用一个USB A到微型USB线缆。在大部分情况下，后者可能更加容易，因为BeagleBone Black自带了这样一根线缆。USB线缆连接到BeagleBone Black底部的微型USB端口。

设置终端模拟器

为了通过串行连接与BeagleBone交互，你需要终端模拟程序。对于Linux有几个选择，我们推荐Minicom。几乎所有Linux发行版通过其包仓库提供Minicom。如果它还没有安装在你的系统上，那么使用你的发行版的包管理器来安装它。

所有当前的Linux系统都把FTDI串行到USB的转换器识别成ttyUSB设备。在你把BeagleBone插到开发计算机的USB端口后，设备节点/dev/ttyUSB0[5]由Linux内核创建。Linux内核防止设备节点受到普通用户的访问。/dev/ttyUSB设备是由root拥有的，但通常属于dialout用户

组。为了使Minicom能够于运行时在你的用户账号下访问设备，你需要增加自己到dialout用户组中。

```
$ sudo usermod -a -G dialout <username>
```

其中，可用你的实际用户名替换<username>。为了使变更生效，你需要登出并再次登录。

现在连接你的BeagleBone到计算机并且以设置模式启动Minicom[6]：

```
$ sudo minicom -s
```

在此时，你应该能够看到Minicom的设置菜单。选择Serial port setup菜单选项并改变设置：

·设置Serial Device为/dev/ttyUSB0。

·设置Bps/Par/Bits为1152008N1。

·设置Hardware Flow Control为No（如必要）。

·设置Software Flow Control为No（如必要）。

按下Enter键从Serial port setup菜单中退出，然后从主设置菜单中选择Save as dfl。最终，从Minicom退出。

现在再次用以下命令启动Minicom：

```
$ sudo minicom -o -w
```

-o选项告诉Minicom不给你的BeagleBone发送任何调制解调器初始化字符串，-w选项为长日志行打开换行。

现在你已经连接到BeagleBone的控制台。如果你的板是BeagleBone（White），那么它可能已经启动了。如果在键盘上敲击Enter键，那么应该能看到类似如下的Linux登录提示：

```
Poky (Yocto Project Reference Distro) 1.8+snapshot-20150720 beaglebone \
    /dev/ttyO0
beaglebone login:
```

如果你的板是BeagleBone Black，那么在敲击Enter键时，你可能只看到U-Boot提示。原因是，BeagleBone Black有内部eMMC存储，而BeagleBone（White）没有。默认情况下，BeagleBone Black首先从它的内部eMMC启动。为了临时告诉BeagleBone Black从SD卡中启动，遵循这些步骤：

·从板上拔除电源（依赖于你正在使用什么，有可能是微型USB或者5V桶形连接器）。

·按住板上的USER/BOOT按钮（当以太网和5V电源连接器朝向左时，这个按钮是板顶部右上部位的小按钮）。

·在按住USER/BOOT按钮的时候再次插入电源，直到来自U-Boot的第一条消息显示在你的Minicom窗口中。然后，可以释放USER/BOOT按钮。

你的BeagleBone启动并且最终显示Linux登录提示。

从U-Boot命令提示中发出这些命令，你可以永久性地把启动顺序从内部eMMC改到SD卡：

```
mmc dev 1
mmc erase 0 512
```

把内容从内部eMMC中擦除。

从Linux命令行提示中，你可以登录进正运行在BeagleBone上的系统中。

恭喜！你已经成功为嵌入式板创建了Yocto项目镜像，并用它来启动板。

[1] http://beagleboard.org/about。
[2] 对嵌入式系统来说，没有显示器是常见的，因为它们可能不需要显示器，例如在工业控制中。虽然显示器可以通过cape或者HDMI增加到BeagleBone，但是设计者决定不这样做，而是让硬件架构保持开放和可扩展。
[3] www.seeedstudio.com/wiki/Grove_System。
[4] www.ftdichip.com/Products/Cables/USBTTLSerial.htm。
[5] 除非你有不止一个串行到USB的转换器被连接到开发系统上，在这种情况下，有多个/dev/ttyUSB设备，你必须找出哪一个连接到你的BeagleBone。
[6] 以设置模式运行Minicom需要root权限。

10.2.2 外部Yocto项目板支持包

物联网和创客运动（Maker Movement）正在驱动对廉价计算机硬件的需求。专业开发者和兴趣爱好者[1]同样在寻找可以用于设计和原型并与项目集成的开发板。围绕大量片上系统构建的可用板的数量正在稳步增加。绝大部分是基于ARM架构的硅片，但是也可以找到基于x86、x86_64、xScale和PowerPC架构的。Raspberry Pi为能够运行完整Linux操作系统栈的低成本嵌入式计算机树立了新基准。其他板也纷纷效仿，其中很多只花费和在像样的餐厅里吃顿不错的饭一样的钱。

流行开发板

表10-1以字母顺序显示了一些比较流行的能够运行Linux的开发板。它们都提供各种低速I/O接口，这些接口是可通过针脚头直接访问的。对于它们中的一些来说，直接插到针脚头上的扩展板是可用的。扩展板提供各种东西，从带LED状态和中继器的并行I/O、模拟–数字转换器（Analog-to-Digital Converter，ADC）和数字–模拟转换器（Digital-to-Analog Converter，DAC），到用于机器人应用的步进电动机驱动器等。

表10-1只展示了在写作本书时市面上的部分开发板。伴随每个新一代片上系统，新的开发板也几乎同步地进入市场。

为你的板寻找Yocto项目板支持包

一旦你决定了用于开发项目的板，那么你将在哪里找到Yocto项目板支持包支持的开发板呢？最好是浏览板自己的网站。在很多情况下，你可以找到Yocto项目或者提供硬件支持的社区的链接。

表10-1　流行开发板

开发板	CPU/ 内存	输入 / 输出	图形化	主页 Yocto 项目板支持包
BeagleBoard-xM	TI DM3730 ARM Cortex-A8，720 MHz 512 MB 内存 MicroSD 卡槽	USB hub USB OTG 10/100 BaseT McBSP DSS I2C UART LCD McSPI PWM JTAG Camera	DVI-D S-Video	www.beagleboard.org git://git.yoctoproject.org/meta-ti -b dora
BeagleBone	TI AM3358 ARM Cortex-A8，720 MHz 256 MB 内存 MicroSD 卡槽	USB 2.0 client USB 2.0 host 10/100 BaseT 4 × UART 8 × PWM LCD GPMC MMC1 2 × SPI 2 × I2C ADC 2 × CAN 4 timers FTDI USB	No	www.beagleboard.org git://git.yoctoproject.org/meta-yocto -b fido
BeagleBone Black	TI AM3358 ARM Cortex-A8，1 GHz 512 MB 内存 2 GB RAM（Rev C） MicroSD 卡槽 eMMC	USB 2.0 client USB 2.0 host 10/100 BaseT 4 × UART 8 × PWM LCD GPMC MMC1 2 × SPI 2 × I2C ADC 2 × CAN 4 Timers	HDMI	www.beagleboard.org git://git.yoctoproject.org/meta-yocto -b fido
CubieBoard 2	AllWinnerTech SOC A20，ARM Cortex-A7，双核 1 GB 内存 3. 4GB NAND 内存 MicroSD 卡	USB 2.0 host USB 2.0 OTG 10/100 BaseT Infrared I2C SPI CSI/TS FM-IN ADC CVBS SPDIF-OUT	Mali 400 HDMI RGB LVDS	www.cubieboard.org https://github.com/linux-sunxi/ meta-sunxi

（续）

开发板	CPU/ 内存	输入 / 输出	图形化	主页 Yocto 项目板支持包
CubieTruck（CubieBoard 3）	AllWinnerTech SOC A 20，ARM Cortex-A7，双核 1 GB/2GB 内存 MicroSD 卡	USB 2.0 host USB 2.0 client 10/100 BaseT SATA 2.0 Infrared I2C SPI CSI/TS FM-IN TV-IN ADC CVBS SPDIF	Mali 400 HDMI	www.cubieboard.org https://github.com/linux-sunxi/ meta-sunxi
Edison	具有 500 MHz 的双核、双线程 Intel Atom CPU 和一个 32 位 Intel Quark 微控制器的片上系统 1 GB LPDDR3 4 GB eMMC	WiFi a/b/g/n dual-band Bluetooth 4.0 2 × UART 2 × I2C 1 × SPI 1 × I2S 40 × digital I/O USB 2.0 OTG	No	www.intel.com/content/www/us/en/do-it-yourself/edison.html
Galileo	Intel Quark SoC X1000，400 MHz，32 位，单核 256 MB 内存盘 MicroSD 卡	USB 2.0 host USB 2.0 client 10/100 BaseT PCI Express 2.0 JTAG 2 × UART 20 × digital I/O ADC SPI	No	www.intel.com/content/www/us/en/embedded/products/galileo/galileo-overview.html git://git.yoctoproject.org/meta-intel-gallileo -b dizzy
MinnowBoard Max	Intel Atom E38xx SoC，1.33 GHz，双核 1 GB/2 GB 内存 8 MB SPI 闪存 MicroSD 卡槽	USB 3.0 host USB 2.0 host UART 10/100/1000 BaseT I2C SPI	Intel HD graphics HDMI	www.minnowboard.org/meet-minnowboard-max git://git.yoctoproject.org/meta-intel-b fid
ODROID-XU4	Samsung Exynos 5422 ARM Cortex A-15，4 核 2 GB 内存 MicroSD 卡槽	USB 3.0 host USB 2.0 host 10/100/1000 BaseT	Mali T628 HDMI	www.hardkernel.com https://github.com/akuster/meta-odroid
Raspberry Pi 2 B	Broadcom BCM2836，ARM Cortex-A7，900 MHz，4 核 1 GB 内存 MicroSD 卡	USB 3.0 host 10/100 BaseT CSI camera port DSI display port 40 GPIO	HDMI	https://www.raspberrypi.org/products/raspberry-pi-2-model-b git://git.yoctoproject.org/meta-raspberrypi

（续）

开发板	CPU/ 内存	输入 / 输出	图形化	主页 Yocto 项目板支持包
WandBoard	Freescale i.MX6，ARM Cortex-A9，单核，双核，4 核 512 MB 内存（单） 1 GB 内存（双） 2 GB 内存（4 个)MicroSD 卡槽	USB 3.0 host USB 3.0 OTG 10/100/1000 BaseT Audio SPDIF Optical Camera Interf. SATA（quad）	Vivante GC880 Vivante GC320 HDMI	www.wandboard.org git://github.com/Freescale/meta-fsl-arm-extra.git

Yocto项目在它的网站维护了一个可搜索的板支持包[2]页面，该页面展示了开发板和Yocto项目仓库里的板支持包的一个概览。这些包是由组织提供的，有商业的和开源的，它们为Yocto项目提供资源和开发工作[3]。

一部分作为Yocto项目兼容列出，这是作为Yocto项目对支持该项目的组织的一个认可的状态。Yocto项目兼容并不意味着不具有那个状态的层是不兼容的，仅仅是说那些由Yocto项目兼容所列出的层已经经过了审查并且是由一个Yocto项目成员组织支持的。

Yocto项目Git仓库[4]托管很多板支持包层，包括那些未被Yocto项目官方认证的板支持包层。板支持包层的列表包含如下内容：

·meta-fsl-arm：用于使用基于ARM架构的片上系统的Freescale平台的板支持包层。

·meta-fsl-ppc：用于使用基于PowerPC架构的片上系统的Freescale平台的板支持包层。

·meta-intel：用于基于x86和x86_64架构的Intel平台的复合板支持包层。这个层包含多个用于实际平台的子层。

·meta-intel-galileo：用于Intel Galileo平台支持的板支持包层。

·meta-intel-quark：用于Intel Quark平台支持的板支持包层。

·meta-minnow：用于原始MinnowBoard（非MinnowBoard Max，它是由meta-intel支持的）的板支持包层。

·meta-raspberrypi：用于Raspberry Pi 1和Raspberry Pi 2设备的板支持包层。

·meta-renesas：用于Renesas设备的板支持包层。

·meta-ti：用于Texas Instruments设备的板支持包层，包括对BeagleBone的扩展的硬件支持，这种支持不由meta-yocto-bsp提供。

·meta-xilinx：用于Xilinx设备的板支持包层。

·meta-zynq：用于Zynq设备的板支持包层。

最后但并非最不重要，OpenEmbedded网站[5]提供了包含由各种社区提供的很多板支持包的可搜索层索引。搜索功能允许按照机器名搜索，并且列出为那个机器提供支持的层。有时候，用于相同机器的多个层可用，它们为板提供的支持的级别可能有差别。

用外部Yocto项目板支持包构建

Yocto项目板支持包是层，用它们来构建简单到只有3步：

1.通过增加板支持包层的路径到conf/bblayers.conf中的BBLAYERS变量来在你的构建环境中包含它。

2.把你希望从板支持包层构建的机器赋值到conf/local.conf中的MACHINE变量。

3.选择镜像目标并且开始你的构建——例如，bitbake-k core-image-minimal。以仅仅能够启动到控制台命令行的小镜像开始是好建议，例如core-image-minimal或者core-image-base。如果正常工作，那么你可以测试例如core-image-sato的更大的镜像或者开始构建你自己的定制镜像菜谱。一些板支持包包含自己的镜像目标，这也是一个好起点。

当使用Yocto项目板支持包时，需要注意，板支持包版本匹配OpenEmbedded构建系统的版本。板支持包通常用bbappend文件来扩展来自OpenEmbedded核心的菜谱。如果版本不匹配，那么有可能是，由板支持包扩展的各个菜谱的版本与在核心层中找到的版本不匹配。如果这种情况发生，那么BitBake为各个菜谱发出错误消息，但是根本原因却不总是显而易见的。

Yocto项目开发者对主要发布使用代号：1.5是Dora、1.6是Daisy、1.7是Dizzy、1.8是Fido、2.0是Jethro。虽然你可以从Yocto项目网站[6]下载发布的tar包，但是推荐你从Yocto项目Git仓库中检出发布版。使用Git仓库有利于追踪版本甚至在版本间切换。例如，如果你希望使用meta-ti层替代meta-yocto-bsp来为BeagleBone构建，那么简单地从poky和meta-ti仓库中克隆匹配的分支：

```
$ git clone git://git.yoctoproject.org/poky -b fido
$ git clone git://git.yoctoproject.org/meta-ti -b fido
```

另外一个需要注意的事情是，虽然板支持包通常不互相影响，但是存在在机器定义中有重合的板支持包，例如meta-yocto-bsp和meta-ti，它们的beaglebone机器定义重合。为了避免冲突，必须用BBLAYERS仅仅包含其中一个。简言之，在一个构建环境中同时仅包含一个板支持包层是好建议。

[1] 区分专业开发者和兴趣爱好者时，不是基于他们的能力，而是看他

们是将嵌入式系统作为工作的一部分还是作为兴趣爱好。

[2] https://www.yoctoproject.org/downloads/bsps。

[3] 在“free and open source software（FOSS）”中的“free”是指自由，而不是指成本。寻求Yocto项目兼容性证书的组织以供开发使用的资源和基础设施来支持项目。

[4] http://git.yoctoproject.org。

[5] http://openembedded.org/wiki/Main_Page。

[6] https://www.yoctoproject.org/downloads。

10.3 Yocto项目板支持包内部

Yocto项目板支持包是特殊化的BitBake层。正因为如此，它们遵循第3章讨论的用于层的惯例，也包含版支持包层特有的项。程序清单10-2描述了一般的板支持包层布局。

程序清单10-2 Yocto项目板支持包层布局

```
meta-<bspname>
├── LICENSE
├── MAINTAINERS
```

```
├── README
├── README.sources
├── binary
│   ├── <bootable-image-1>
│   ├── <bootable-image-2>
│   ├── ...
│   └── <bootable-image-n>
├── conf
│   ├── layer.conf
│   ├── machine
│   │   ├── <machine-1>.conf
│   │   ├── <machine-2>.conf
│   │   ├── ...
│   │   └── <machine-m>.conf
├── classes
│   ├── class-<1>.bbclass
│   ├── class-<2>.bbclass
│   ├── ...
│   └── class<m>.bbclass
├── recipes-bsp
│   ├── formfactor
│   │   ├── formfactor_0.0.bbappend
│   │   └── formfactor
│   │       ├── machine-1
│   │       │   └── machconfig
│   │       ├── machine-2
│   │       │   └── machconfig
│   │       ├── ...
│   │       └── machine-m
│   │           └── machconfig
│   ├── <package a>
│   │   ├── <package a>_<version 1>.bb
│   │   └── <package a>_<version 2>.bb
│   ├── <package b>
│   │   ├── <package b>_<version 1>.bb
│   │   └── <package b>_<version 2>.bb
│   ├── ...
│   └── <package z>
├── recipes-core
│   ├── images
│   │   ├── core-image-1.bb
│   │   ├── core-image-2.bb
│   │   ├── ...
│   │   └── core-image-n.bb
│   └── ...
├── recipes-graphics
│   ├── <package a>
│   │   ├── <package a>_<version 1>.bb
│   │   └── <package a>_<version 2>.bb
│   ├── <package b>
│   │   ├── <package b>_<version 1>.bb
│   │   └── <package b>_<version 2>.bb
│   ├── ...
│   └── <package z>
├── recipes-kernel
│   ├── linux
│   │   ├── linux-yocto_<version 1>.bbappend
│   │   └── linux-yocto_<version 2>.bbappend
└── recipes-<category x>
    └── ...
```

这个结构的大部分是约定俗成的。然而，构建系统期望特定目录和文件在特定位置被特殊化地命名。像任何层一样，层的顶级目录被命名为meta-<layername>，其中layername是板支持包的名字。

10.3.1 许可文件

许可文件位于板支持包的顶级目录。它们可以具有任何名字，但通常使用的是LICENSE、EULA和COPYING等名字。

从构建系统不寻找许可文件并且不强制其存在性这个意义上来说，许可文件是可选的。然而，我们强烈建议，在写你自己的板支持包时，提供许可文件。提供许可文件给了你的板支持包用户关于可以用板支持包做什么以及不能做什么的确定性。

10.3.2 维护者文件

维护者文件包含关于谁是开发者以及谁负责维护板支持包的信息。这个文件是可选的，而且维护者信息通常包含在顶级目录的README文件中。如果板支持包有多个（可能很多）维护者，那么在单独的文件中提供这个信息很有意义。

10.3.3 README文件

符合Yocto项目的板支持包必须在板支持包的顶级目录中包含README文件。这个文件至少应该包含下列信息：

·描述板支持包及其针对的硬件的部分。

·详细描述用于这个板支持包层的所有依赖性的部分。通常，这是例如meta和meta-yocto的其他层。几乎所有板支持包都依赖OpenEmbedded核心元数据层meta。虽然看起来是显而易见的，但是把它包含在依赖性列表中是好的实践。

·用于提问、报告缺陷和提交补丁的指导方针部分。这些信息使得用户和维护者可轻松地沟通和协作。

·如果单独文件没有提供，那么需要关于维护板支持包的开发者的部分。

·关于如何使用板支持包为目标硬件构建二进制镜像的说明。

·关于用板支持包构建的镜像可以如何安装在目标硬件以及如何启动目标硬件的说明。

·关于如何使用板支持包binary目录包含的预构建二进制镜像（如果板支持包提供此类镜像）的说明。

·关于板支持包用户在使用板支持包时应该知道的任何已知缺陷或者问题的部分。

在README文件内的信息越详细，对板支持包用户来说越容易成

功地在它们的项目中包含板支持包以及使用目标硬件。

10.3.4 README.sources文件

这个文件提供用户可以在哪里找到用来构建位于二进制目录中的可启动镜像的源文件的信息。通常，这些是曾被用来创建镜像的元数据层。这对于README文件中的依赖性信息似乎是冗余的。然而，它却不冗余，因为用来构建镜像的构建环境可能已经包含了其他可选但不必需的层。

10.3.5 预构建二进制

板支持包可能在binary子目录中包含预构建的可启动镜像二进制。虽然包含可启动镜像是可选的，但是它辅助板支持包的用户来用一个已知良好镜像启动目标硬件。利用预构建的镜像二进制和在README.sources文件中的信息，用户可以通过比较自己构建的镜像和预构建的镜像来更加容易地调试可能出现在镜像中的问题。当然，增加预构建的镜像二进制可能极大地增加板支持包的大小。

10.3.6　层配置文件

像任何层，板支持包必须包含层配置文件conf/layer.conf。对于板支持包来说，这个文件和普通层用到的层配置文件毫无两样。样板文件显示在程序清单10-3中。

程序清单10-3　用于板支持包的层配置文件

```
 # We have a conf and classes directory, add to BBPATH
BBPATH .= ":${LAYERDIR}"

# We have a recipes directory, add to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "bsp"
BBFILE_PATTERN_bsp = "^${LAYERDIR}/"
BBFILE_PRIORITY_bsp = "6"

LAYERDEPENDS_bsp = "<deplayer>"
```

为了使用这个样板，必须替换bsp为层的实际名字（不带meta-）。如果板支持包层依赖其他层，例如用于多个板支持包的通用层，那么层配置文件设置LAYERDEPENDS_bsp变量为由空格分隔的层的列表。

10.3.7　机器配置文件

机器配置文件用于区分板支持包层和普通层。板支持包层包含至少一个机器配置文件，但是在conf/machine子目录中可以有任何数量的、用于由板支持包支持的不同硬件的机器配置文件。机器配置文件被命名为<machinename>.conf。通过设置构建环境的conf/local.conf中的MACHINE变量为机器配置文件的名字（不带.conf结尾）来选择你希望的机器目标。

简言之，机器配置文件包含用于选择和配置由包含在板支持包或者其他元数据层中的菜谱所构建的软件包的设置。典型的例子是板支持包为特定机器使用的Linux内核类型和版本。机器配置文件包含用于PREFERRED_PROVIDER_virtual/kernel和PREFERRED_VERSION_virtual/kernel的设置，以分别选择内核类型和版本。

机器配置文件通常也包含用于目标硬件正在使用的特定CPU架构的调优参数。这些参数和在多个机器间共享的其他参数通常被分离到它们各自的文件中。这些文件被require指令包含在机器配置文件中。

10.3.8 类

板支持包在classes子目录中可能包含定制类。这些经常是以目标硬件要求的方式组装二进制镜像的镜像类。这样的镜像类和它们所构建的镜像使得从镜像创建可启动介质更加容易。

10.3.9 菜谱文件

任何板支持包都包含大量菜谱文件来构建特定于板支持包的软件包或使用.bbappend文件来为来自其他层的包扩展菜谱。板支持包的菜谱以和任何其他元数据层中完全相同的方式组织到recipes-<category>/<package>子目录中。以下是在板支持包中找到的更加常见的菜谱：

·板支持包特定菜谱文件（recipes-bsp）：各式各样的特定于板支持包的菜谱。通常，在recipes-bsp之下的一个目录结构中，你可以找到引导加载程序和规格（formfactor）元数据文件。规格文件为构建系统提供有关目标机器是否使用键盘、触摸屏、鼠标等的信息。

·核心支持文件（recipes-core）：在目录recipes-core中，通常可找到用于目标硬件的二进制镜像以及针对init-scripts、systemd、udev等其他核心菜谱的调整的菜谱。

·显示支持文件（recipes-graphics）：如果板支持包的目标硬件有特定的图形需求，那么你会在目录recipes-graphics中找到与显示支持有关的菜谱。通常，这些是用于X11服务器或者Wayland/Weston合成器的配置菜谱。

·Linux内核（recipes-kernel）：目录recipes-kernel及其子目录包含与Linux内核有关的菜谱和配置文件。通常，它们是用来增加内核配置片段到位于meta核心层的内核菜谱或内核补丁的.bbappend文件。一些板支持包提供它们自己的内核菜谱，这些内核菜谱从它们自己的仓库中构建Linux内核。在第9章中，我们详细讨论了Linux内核配置和各种参数设置。

·其他菜谱（recipes-*）：板支持包可能按照目标硬件的要求增加它自己的菜谱或扩展来自其他层的菜谱。在板支持包层内，可以在它们各自菜谱子目录中找到这些菜谱。

板支持包提供哪些菜谱完全依赖于板支持包及其支持的目标硬件。

10.4 创建Yocto项目板支持包

如果你正在开发自己的硬件，那么你希望创建Yocto项目板支持包以为其提供完整支持。原则上，你可以采用如下3种用于创建Yocto项目板支持包的方法之一：

·手动创建：你可以通过使用yocto-layer脚本创建空的层来开始，然后手动地为板支持包填充目录和文件。

·从现有板支持包层复制：如果你的板支持包针对的硬件类似于来自另一个板支持包的硬件，那么你可以复制那个层并且做调整以满足你的目标硬件的需求。

·使用Yocto项目板支持包工具：Yocto项目提供了几个简化创建板支持包任务的工具。它们是交互性的，并且允许通过回答一系列问题来设置常用板支持包参数。然后，工具为你的板支持包创建骨架，随之你可以填入缺失的细节。

手动地从头创建板支持包是最乏味的方法，因为它要求你自己来增加文件和目录。在几乎所有情况下，使用Yocto项目板支持包工具都是比较好的选项，因为其允许你交互式地剪裁板支持包层。如果你的目标硬件近似于被一个现有的板支持包所支持的硬件，那么从其复制是好的选择。你也可以扩展其他板支持包并且使其成为用于你板支持包的依赖性。这个方法避免了重复，但是，当你的板支持包依赖的其他层发生变化时，就需要维护你自己的板支持包。

10.4.1 Yocto项目板支持包工具

有两个辅助你创建Yocto项目板支持包的工具：yocto-bsp和yocto-kernel。你可能已经猜到了，前者辅助创建板支持包层，后者辅助配置Linux内核。这两个工具都有一些子命令。调用工具而不指定子命令会打印帮助消息和可用子命令的列表。

工具位于Poky参考发行版的poky/scripts目录中。为了使用工具，你必须引用oe-init-build-env脚本。然而，你不用在你的构建环境内创建板支持包。一旦你引用了脚本，你可以改变目录到你希望在其中设置板支持包的地方。

yocto-bsp工具

调用yocto-bsp或者yocto-bsp help提供给你关于可用子命令的信息：

```
$ yocto-bsp
ERROR:root:No subcommand specified, exiting
Usage:

 Create a customized Yocto BSP layer.

 usage: yocto-bsp [--version] [--help] COMMAND [ARGS]

 Current 'yocto-bsp' commands are:
    create            Create a new Yocto BSP
    list              List available values for options and BSP properties

 See 'yocto-bsp help COMMAND' for more information on a specific command.


Options:
  --version    show program's version number and exit
  -h, --help   show this help message and exit
  -D, --debug  output debug information
```

工具有两个子命令：create和list。用yocto-bsp<subcommand>--help调用工具打印关于特定子命令的进一步信息。

子命令yocto-bsp list

子命令yocto-bsp　list只显示信息。当前，这是关于被支持的内核架构的信息：

```
$ yocto-bsp list karch
Architectures available:
    i386
    mips64
    arm
    powerpc
    mips
    x86_64
    qemu
```

对每个内核架构，有一些可用的属性，在创建板支持包时，工具可以设置它们。例如，

```
$ yocto-bsp list x86_64 properties
```

以JSON格式显示可用于x86_64架构的全部属性的列表。当你为x86_64创建板支持包时，工具会交互式地过一遍那些属性，并问你它应该应用的设置。

通过指定-o<filename>或者--outfile<filename>参数，你可以转存list子命令的输出到文件。

```
$ yocto-bsp list x86_64 properties -o x84_64.prop
```

在文件中的转存也是JSON格式。

子命令yocto-bsp create

通过调用yocto-bsp　create<bsp-name><karch>来创建板支持包，其中，替换<bsp-name>为你的板支持包的名字（不带meta-），替换<karch>为希望的内核架构。子命令接受一些可选参数：

·-o<outdir>或者--outdir<outdir>：如果不带这个参数，那么工具在当前目录以名字<bsp-name>创建板支持包。为了在不同的目录创建它，使用-o或者--outfile选项。不幸的是，如果使用这些选项之一，那么你必须提供板支持包的完整路径而不仅仅是基础路径，因为在那个情况下，工具不使用<bsp-name>。

·-i<properties-file>或者--infile<properties-file>：这些参数导致工具从文件中读取用于各种属性的设置而不是交互式地询问它们。文件必须是JSON格式。

·-c codedump或者--codedump：转存代码到文件bspgen.out而不是运行板支持包创建。如果你想知道工具通过做什么来创建板支持包而不实际创建它，那么使用这个选项。

·-s或者–skip-git-check：这些参数检查到远程Git仓库的访问。提供这个选项以在创建板支持包时节省少量时间。

在10.4.2小节，我们将浏览一遍板支持包的生成。

yocto-kernel工具

yocto-kernel工具提供用于对板支持包的内核菜谱列出、增加、去除配置设置、特性和补丁的功能性。它交互式地创建第9章中讨论的内核配置片段、特性和补丁集合。和yocto-bsp类似，yocto-kernel工具有你可以通过--help选项打印出的子命令列表：

```
$ yocto-kernel --help
Usage:
```

```
Modify and list Yocto BSP kernel config items and patches.

usage: yocto-kernel [--version] [--help] COMMAND [ARGS]
Current 'yocto-kernel' commands are:
  config list       List the modifiable set of bare kernel config options
                    for a BSP
  config add        Add or modify bare kernel config options for a BSP
  config rm         Remove bare kernel config options from a BSP
  patch list        List the patches associated with a BSP
  patch add         Patch the Yocto kernel for a BSP
  patch rm          Remove patches from a BSP
  feature list      List the features used by a BSP
  feature add       Have a BSP use a feature
  feature rm        Have a BSP stop using a feature
  features list     List the features available to BSPs
  feature describe  Describe a particular feature
  feature create    Create a new BSP-local feature
  feature destroy   Remove a BSP-local feature

See 'yocto-kernel help COMMAND' for more information on a specific
command.

Options:
  --version     show program's version number and exit
  -h, --help    show this help message and exit
  -D, --debug   output debug information
```

为了使用yocto-kernel工具，你必须已经引用了你的构建环境。然后从构建环境内执行命令。大部分子命令将板支持包的名字作为参数。这个板支持包必须被包含在你的构建环境的conf/bblayers.conf的BBLAYERS变量中。

管理Linux内核配置选项

config子命令允许内核配置选项的简单管理：

·yocto-kernel config list<bsp>：列出用于被板支持包<bsp>使用的Linux内核的所有配置片段。

·yocto-kernel config add<bsp>CONFIG_<parameter>=[y|n|m]：增加修改内核配置选项<parameter>的配置片段：

·y（yes）：开启参数。

·n（no）：关闭参数。

·m（module）：构建内核模块（仅对内核模块适用）。

·yocto-kernel config rm<bsp>CONFIG_<parameter>：去除用于内核配置选项<parameter>的配置片段。

对config add和config rm子命令，你可以提供多个配置项。

管理内核补丁

你可以使用patch子命令来管理用于你的板支持包的补丁：

·yocto-kernel patch list<bsp>：列出所有用于被板支持包<bsp>使用的Linux内核的补丁。

·yocto-kernel patch add<bsp>/path/to/patchfile.patch：把补丁patchfile.patch从已提供的路径复制到板支持包层<bsp>内的recipes-kernel/linux/files中，并且增加它到内核菜谱的SRC_URI。

·yocto-kernel patch rm<bsp>patchfile.patch：把补丁patchfile.patch从板支持包<bsp>内的recipes-kernel/linux/files中去除，并且把它从内核菜谱的SRC_URI去除。

你可以一次对patch add和patch rm子命令提供多个补丁。

管理内核特性

类似于内核配置选项和补丁，你可以通过feature子命令来管理特性：

·yocto-kernel features list<bsp>：列出所有本地可用于板支持包<bsp>的内核特性。这些是以.scc结尾的位于recipes-kernel/linux/files中的板支持包内的特性文件。

·yocto-kernel feature list<bsp>：列出所有当前被板支持包<bsp>所使用的内核特性。

·yocto-kernel feature create<bsp>featurefile.scc"Feature Description"/capabilities CONFI_parameter=[y|n|m]/path/to/patchfile.patch：为板支持包<bsp>创建名为featurefile.scc、使用"Feature Description"作为描述的新的本地特性，并且增加列在capabilities之后的内核配置参数和补丁。

·yocto-kernel feature add<bsp>featurefile.scc：增加本地特性featurefile.scc到板支持包<bsp>的内核特性列表中。特性必须是以前用yocto-kernel feature creat创建的。

·yocto-kernel feature rm<bsp>：从板支持包<bsp>内核特性列表中去除一个或者多个特性。工具列出特性，然后提示要被去除的特性。这个命令不从板支持包中删除特性，仅仅从内核的特性包含列表中去除。

·yocto-kernel feature destroy<bsp>featurefile.scc：从板支持包<bsp>特性的本地列表中删除特性。这个命令不从板支持包中删除特性文件、配置片段和补丁。

feature子命令是管理包含在内核中的特性的便利方法，它不用修改特性文件和菜谱的SRC_URI。

10.4.2 用Yocto板支持包工具创建板支持包

现在是时候把这些结合起来并使用Yocto项目板支持包工具来创建一个基本的板支持包了。我们的板支持包用于信息亭设备。设备使用Intel Core i7 CPU，有板载HDMI图形输出，并且被连接到多点触控的触摸屏以提供良好的用户体验。作为用于我们设备的测试平台，我们使用MinnowBoard Max并且让它连接带HDMI和USB输入的多点触控触摸屏。我们也希望启动时板支持包在内核日志中以一个条目来识别自己。为了那个目的，我们开发了简单的设备驱动，本质上它是和显示在第9章程序清单9-3中的那个完全一样。我们仅仅把消息文本改成了“Yocto Project Book Kiosk BSP：init”和“Yocto Project Book BSP：exit.”。简单地遵循9.2节的步骤来创建补丁。

以下步骤简要概述了创建你的板支持包（我们称其为ypbkiosk）的工作流：

1.初始化构建环境。为了用Yocto项目板支持包工具创建板支持包，你需要构建环境：

```
$ source /path/to/poky/oe-init-build-env kiosk
```

在conf/local.conf中，设置构建环境变量DL_DIR和SSTATE_DIR。

2.创建板支持包层。为了简单性，我们在构建环境内创建了新的板支持包层。命令

```
$ yocto-bsp create ypbkiosk x86_64
```

启动交互式过程来为具有x86_64架构的机器ypbkiosk创建名为ypbkiosk的板支持包层。该工具现在问一些关于我们希望用在板支持包

中的特性的问题：

·Would you like to use the default（3.19）kernel?（y/n）[default：y]y

·Do you need a new machine branch for this BSP（the alternative is to re-use an existing branch）?（y/n）[default：y]n

·Please choose a machine branch to base this BSP on：[default：standard/common-pc-64/base]7（或者回车以用默认）

·Do you need SMP support?（y/n）[default：y]y（或者回车以用默认）

·Which machine tuning would you like to use?[default：tune_core2]2（Corei7调优）

·Do you need support for X?（y/n）[default：y]y（或者回车以用默认）

·Please select an xserver for this machine：[default：xserver_i915]4（fbdev xserver支持）

·Does your BSP have a touchscreen?（y/n）[default：n]y

·Does your BSP have a keyboard?（y/n）[default：y]y（或者回车以用默认）

在你回答了最后一个问题后，板支持包工具设置板支持包并且以消息“New x86_64 BSP created in meta-ypbkiosk”退出。

3.启用触摸屏驱动。对于我们的亭板支持包，需要在Linux内核中启用多点触控驱动。驱动的配置设置是CONFIG_HID_MULTITOUCH。我

们使用Yocto项目内核工具来增加启用驱动的配置片段：

```
$ yocto-kernel config add ypbkiosk
CONFIG_HID_MULTITOUCH=y
Added item:
        CONFIG_HID_MULTITOUCH=y
```

工具添加设置到位于meta-ypbkiosk/recipes-kernel/linux/files中的ypbkiosk-user-config.cfg。

4.增加板支持包补丁。作为一个特性来增加板支持包驱动补丁。特性应用补丁，也增加配置设置以启用补丁：

```
$ yocto-kernel feature create ypbkiosk ypbbspmsg.scc \
 "Yocto Project Book BSP Message" capabilities \
 CONFIG_YP_DRIVER=y 0001-Yocto-Project-Book-Kiosk-BSP-Message.patch
Added feature:
   features/ypbbspmsg.scc
$ yocto-kernel feature add ypbkiosk features/ypbbspmsg.scc
Added features:
 features/ypbbspmsg.scc
```

第一个命令yocto-kernel feature create，创建特性并且增加它到板支持包。然而，它还没被内核构建所包含，这是由第二个命令yocto-kernel feature add来实现的。

5.构建镜像。既然我们已经创建了具有希望的内核配置的板支持包，那么我开始构建镜像：

```
$ bitbake -k core-image-sato
```

我们正在使用core-image-sato镜像目标，它提供我们用于测试的图形用户界面。

6.复制镜像到可启动介质。我们正在使用MinnowBoard Max作为用于亭的测试目标系统。板从USB内存棒中启动。复制镜像到内存棒中。

```
$ dd if=tmp/deploy/images/ypbkiosk/core-image-sato-ypbkiosk.hddimg \
     of=/dev/<usbstickdevice>
```

你需要替换<usbstickdevice>为设备节点的名字，把USB棒插入进你的开发系统后，你可以使用dmesg命令来找到设备节点的名字。

7.启动目标。MinnowBoard Max有一个带shell的UEFI BIOS。在你把USB内存棒插入板中并且开机后，MinnowBoard Max启动shell。内存棒上的文件系统被识别成fs0。在shell提示上，输入fs0:，在下一个提示上，输入bootx64。然后，MinnowBoard Max应该启动到Sato用户界面。如果你把触摸屏连接到了你的板上，那么应该能够使用它了。你也可以打开终端并查找板支持包消息。

Yocto项目板支持包的这两个工具yocto-bsp和yocto-kernel提供了创建板支持包层的简单直接的方法。仅仅几个交互步骤，它们就为一个基本的板支持包设置了具有必要目录和文件的核心框架，然后，你可以进一步定制板支持包。使用yocto-kernel工具管理内核配置选项、补丁和特性成为简单任务。不需要手动修改菜谱和其他文件。

10.5 调优

如果你检查我们在前面部分创建的机器配置文件ypbkiosk.conf，那么注意下面两行：

```
DEFAULTTUNE = "corei7-64"
require conf/machine/include/tune-corei7.inc
```

第一行选择CPU架构和被构建系统使用的应用程序二进制接口（Application Binary Interface，ABI）调谐（tune）。第二行为特定的调谐提供细节的工具链设置，例如GCC编译器标志。

CPU架构提供向后兼容性，这意味着，你或许可以对一个使用更新一代架构的CPU使用用于更旧版本架构的调谐。但是，通过这样做，你当然无法从更新架构代的扩展指令集和功能性中获益。这种CPU架构代的层级结构是由调优文件的层级结构反映的。对于Intel x86架构，它尤其明显：

·tune-corei7.inc：用于带64位扩展和MMX、SSE、SSE2、SSE3、SSSE3、SSE4.1以及SSE4.2指令集支持的Intel Core i7 CPU代的调谐。基于tune-core2。

·tune-core2.inc：用于带64位扩展和MMX、SSE、SSE2、SSE3和SSSE3指令集支持的Intel Core2 CPU代的调谐。基于tune-586。

·tune-i586.inc：启用Intel i586特定处理器优化的调谐。基于arch-x86。

·arch-x86：用于Intel x8632位、x8664位和x32架构的核心架构定义。

你可以在OpenEmbedded核心元数据层meta的conf/machine目录和子目录以及子目录的子目录等找到调谐文件。

DEFAULT_TUNE变量为特定CPU架构选择TUNE_FEATURES。构建系统使用DEFAULT_TUNE和TUNE_FEATURES来判断用于以下的设置：

·TUNE_ARCH：用于特定CPU架构的GNU工具链的权威架构。TUNE_ARCH与TARGET_ARCH紧密联系，因为BitBake配置文件（meta/conf/bitbake.conf）赋予TARGET_ARCH="${TUNE_ARCH}"。

·TUNE_PKGARCH：打包系统所已知的用来定义正确的架构、应用程序二进制接口和输出包的调优的包架构。

·TUNE_ASARGS：用于特定调优架构的汇编程序标志。

·TUNE_CCARGS：用于特定调优架构的编译器标志。

·TUNE_LDARGS：用于特定调优架构的链接器标志。

构建系统验证调优设置的兼容性，特别是冲突的应用程序二进制接口设置。

每个调谐被加到由变量AVAILTUNES代表的可用调谐列表中。你可以使用以下命令来获取用于你当前设置的列表：

```
$ bitbake -e | grep AVAILTUNES
```

例如，通过由我们的ypbkioks.conf机器配置提供的设置，你将看到：

```
$ bitbake -e  | grep AVAILTUNES
# $AVAILTUNES [10 operations]
AVAILTUNES=" x86 x86-64 x86-64-x32 i586 core2-32 core2-64 core2-64-x32 \
             corei7-32 corei7-64 corei7-64-x32"
```

在细节上：

·x86：带有32位应用程序二进制接口的Intel x8632位架构

·x86-86：带有64位应用程序二进制接口的Intel x8664位架构

·x86-64-x32：带有32位应用程序二进制接口的Intel x8664位架构

·i586：带有32位应用程序二进制接口的Intel i58632位架构

·core2-32：带有32位应用程序二进制接口的Intel Core232位架构

·core2-64：带有64位应用程序二进制接口的Intel Core264位架构

·core2-64-x32：带有32位应用程序二进制接口的Intel Core264位架构

·corei7-32：带有32位应用程序二进制接口的Intel Core i732位架构

·corei7-64：带有64位应用程序二进制接口的Intel Core i764位架构

·corei7-64-x32：带有32位应用程序二进制接口的Intel Core i764位架构

用于其他CPU架构的调谐提供用于CPU架构和应用程序二进制接口的类似配置。

10.6 创建可启动介质镜像

由OpenEmbedded构建系统创建的镜像不总是能够被直接用在存储介质上来创建可启动的系统。在我们为BeagleBone创建第一个镜像时，我们看到了那样的情况。用于分区和格式化SD卡然后复制文件和镜像到各种分区的额外步骤是必要的。其他镜像依赖于目标平台以及使用的存储硬件和介质。相对于HDD，SD可能需要一个不同的镜像格式。为了促进创建可以被直接传输到存储介质的可启动镜像的过程，OpenEmbedded构建系统提供了OpenEmbedded镜像创建器（wic）[1]。

wic工具从由构建系统创建的制品中创建可启动镜像。为了使wic命令正确工作，必须从构建系统的上下文中运行它，正如以往，这是通过oe-init-build-env引用来完成的。

wic使用以.wks结尾的kickstart文件，这些文件命令工具创建什么镜像以及如何创建它们。构建系统自带一组kickstart文件。为了获得内置镜像的列表，使用以下命令：

```
$ wic list images
```

wic被设计成是可扩展的。wic kickstart文件描述用于创建硬盘镜像的必要步骤。wic源插件包含wic为各个步骤所执行的代码。为了列出可用的源插件，使用以下命令：

```
$ wic list source-plugins
```

当我们更仔细地看kickstart文件的时候，我们展示kickstart文件和源插件如何一起工作。

wic有两个操作模式：

·原始模式（Raw Mode）：你必须在wic命令行上指定kickstart文件需要的参数。

·烹制模式（Cooked Mode）：wic使用你构建环境的当前MACHINE设置来决定选项。

原始模式提供更好的灵活性并且给了你对镜像创建过程更多的控制，而烹制模式更易于使用。这两种模式本质上是以相同方式调用的：

```
$ wic create <kickstart_file> [options]
```

选项决定了wic是否运行在原始模式或者烹制模式。在接下来的两个部分，我们会解释这两种模式以及对每个模式你可以使用什么选项。如果你正是使用由wic list images显示的列表中的kickstart文件，那么你不要提供.wks后缀。

在你可以使用wic之前，你需要使用构建系统为你的开发主机构建几个工具：

```
$ bitbake parted-native dosfstools-native mtools-native
```

你不需要以root运行wic。实际上，你不应该以root运行wic。wic不写任何存储介质——它为存储介质创建镜像，然后你可以复制镜像到存储介质上。

[1] 你可能要问，为什么工具被命名为wic而不是oeic（OpenEmbedded Image Creator）。尝试快速说出二合元音oeic，你将注意到它听起来更像wic。

10.6.1 用烹制模式创建镜像

在烹制模式中，只有两个参数是用wic创建镜像所需要的：

```
$ wic create <kickstart_file> -e <image_target> [options]
```

其中，

·<kickstart_file>是OpenEmbedded kickstart文件。你可以使用由wic提供的预录的kickstart文件，或者你可以提供你自己的。

·<image_target>是构建系统镜像目标的名字，例如core-image-base或者core-image-sato或者任何你自己的镜像名字。

总的来说，所有其他设置是由wic从构建环境决定的，特别是conf/local.conf中的MACHINE设置。

在烹制模式下，这些额外的选项是可用的：

·-o PATH，--outdir=PATH：到最终镜像位置的路径。

·-c COMPRESSOR，--compress-with=COMPRESSOR：用来压缩最终镜像的压缩实用程序。wic支持gzip、bzip2和xz作为COMPRESSOR。

·-f IMAGE，--build-rootfs=IMAGE：在创建介质镜像之前，使用BitBake IMAGE来构建根文件系统镜像。

·-D，--debug：显示关于创建过程的详细的调试信息。它显示精确的命令序列并且帮助故障排除问题。

·-s，--skip-build-check：略过构建检查步骤，它是用于验证构建环

境是否已经被正确引用了的简单的健全性检查器（sanity checker）。

例如，

```
$ wic create bootimg-efi -e core-image-base
```

创建用扩展固件接口（Extended Firmware Interface，EFI）BIOS启动的镜像，它可以被直接传输到可启动介质。

一个合理的警告：仅仅因为你可以通过wic使用kickstar文件来创建镜像并不意味着这个镜像将在你的目标系统上启动。你需要保证，你选择了匹配你目标系统的正确的kickstart文件。如果在前面的例子中机器配置是MACHINE="beaglebone"，那么wic将确实创建带有用于扩展固件接口的启动分区的镜像。然而，它将不会在BeagleBone板上启动，因为BeagleBone没有扩展固件接口BIOS。

10.6.2 用原始模式创建镜像

当以原始模式使用wic时，你必须在命令行上提供必要的参数：

```
$ wic create <kickstart_file> [options]
```

其中，<kickstart_file>是OpenEmbedded kickstart文件，它可以是由wic提供的kickstart文件之一或者是你自己创建的。选项如下：

·-r ROOTFSDIR，--rootfs-dir=ROOTFSDIR：在开发主机上到用于目标的根文件系统的路径。

·-b BOOTIMGDIR，--bootimg-dir=BOOTIMGDIR：到引导加载程序制品（例如扩展固件接口和syslinux目录或者U-Boot文件）的路径。

·-k KERNELDIR，--kernel-dir=KERNEL_DIR：到Linux内核的路径。

·-n NATIVE_SYSROOT，--native-sysroot=NATIVE_SYSROOT：到例如parted、DOS文件系统工具等原生工具的路径。这些可以是由OpenEmbedded构建系统构建的工具或者由你的开发主机提供的工具。

·-o PATH，--outdir=PATH：到最终镜像位置的路径。

·-c COMPRESSOR，--compress-with=COMPRESSOR：用来压缩最终镜像的压缩实用程序。wic支持gzip、bzip2和xz作为COMPRESSOR。

·-f IMAGE，--build-rootfs=IMAGE：在创建介质镜像之前，使用BitBake IMAGE来构建根文件系统镜像。

·-D，--debug：显示关于创建过程的详细调试信息。它显示精确的

命令序列并且帮助故障排除问题。

在其最简单的形式下，如果你必须以原始模式调用wic，那么如下：

```
$ wic create bootimg-efi -r <ROOTFSDIR> -b <BOOTIMGDIR> -k <KERNELDIR>
```

该示例假设你已经在开发主机上安装了文件系统工具。

当使用烹制模式并带有-e<image_target>选项时，wic通过运行bitbake-e<image_target>来自动地从构建环境决定各种选项：

·-r，--rootfs-dir：IMAGE_ROOTFS

·-k，--kernel-dir：STAGING_KERNEL_DIR

·-n，--native-sysroot：STAGING_DIR_NATIVE

·-b，--bootimg-dir：空的；用于各种引导加载程序的源插件需要决定这个。

除非你正在使用-f（--build-rootfs）选项，否则在原始模式下，你不需要引用构建环境。

10.6.3　kickstart文件

正如我们看到的，命令

```
$ wic list images
```

提供了可用kickstart文件的列表。实际的kickstart文件位于poky/scripts/lib/wic/canned-wks。

程序清单10-4显示了一个kickstart文件，它用于为适合于BeagleBone的SD卡创建带有一个vfat启动分区和一个ext4文件系统分区的镜像。

程序清单10-4　SD卡镜像（sdimage-bootpart.wks）

```
# short-description: Create SD card image with a boot partition
# long-description: Creates a partitioned SD card image. Boot files
# are located in the first vfat partition.

part /boot --source bootimg-partition --ondisk mmcblk --fstype=vfat
     --label boot --active --align 4 --size 16
part / --source rootfs --ondisk mmcblk --fstype=ext4 --label root
     --align 4
```

part指令指示wic创建一个分区。第一个参数是分区的挂载点——在这个kickstart文件中，/boot是启动分区的挂载点，/是根分区的挂载点。source参数指定用来创建分区的源插件，bootimg-partition用于启动分区，rootfs用于根文件系统分区。剩余的参数决定分区特征，我们稍后将详细解释他们。

程序清单10-5显示了一个为启动带有老式PC　BIOS的系统创建镜像的kickstart文件。

程序清单10-5　老式PC BIOS程序清单镜像（directdisk.wks）

```
# short-description: Create a 'pcbios' direct disk image
# long-description: Creates a partitioned legacy BIOS disk image that the
# user can directly dd to boot media.

part /boot --source bootimg-pcbios --ondisk sda --label boot -active \
     --align 1024
part / --source rootfs --ondisk sda --fstype=ext4 --label platform \
     --align 1024

bootloader --timeout=0 --append="rootwait rootfstype=ext4 \
                                 video=vesafb vga=0x318 console=tty0"
```

在这个例子中，启动分区使用源插件bootimg-pcbios，它创建一个Syslinux启动分区。bootloader指令指示Syslinux立即启动内核（--timeout=0）并且传递由append指定的参数到Linux内核。

程序清单10-6显示了一个kickstart文件，它为启动一个带有扩展固件接口BIOS的系统创建镜像。

程序清单10-6　扩展固件接口BIOS启动镜像（mkefidisk.wks）

```
# short-description: Create an EFI disk image
# long-description: Creates a partitioned EFI disk image that the user
# can directly dd to boot media.
part /boot --source bootimg-efi --sourceparams="loader=grub-efi" \
     --ondisk sda --label msdos --active --align 1024

part / --source rootfs --ondisk sda --fstype=ext4 --label platform \
     --align 1024

part swap --ondisk sda --size 44 --label swap1 --fstype=swap

bootloader --timeout=10 --append="rootwait rootfstype=ext4 \
    console=ttyPCH0,115200 console=tty0 vmalloc=256MB \
    snd-hda-intel.enable_msi=0"
```

在这个例子中，启动分区是使用bootimg-efi源插件来创建的，其他使用扩展固件接口Grub引导加载程序。除了启动和根文件系统分区，还有一个交换分区被创建。bootloader指令指示Grub在启动Linux内核前等待10秒（timeout=10）并且传递由append指定的参数到Linux内核。

10.6.4 kickstart文件指令

要么从头开始，要么通过复制一个预录的文件，你可以很容易地创建自己的kickstart文件。如果你在目录poky/scripts/lib/wic/canned-wks中创建自己的kickstart文件，那么对于wic来说它们是已知的，并且你不需要提供路径和后缀。缺点是，你正在修改构建系统源，这使得更新更加困难了一点。kickstart文件当前只包含两个指令：partition和bootloader。每个指令都接受一个预定义集合的参数。提供功能性的源插件需要理解参数。

分区指令

分区指令part在介质上创建分区。格式如下：

```
Part <mountpoint> <options>
```

<mountpoint>决定分区被挂载到哪里。它可以是如下之一：

·/path：分区挂载点路径——例如/、/usr、/opt和/home等。

·swap：分区是一个交换分区。

<options>提供关于如何创建分区的必要信息：

·--source：决定用来填充分区的数据源。如果你不使用这个选项，wic创建空分区，并且你必须至少提供--size。如果你希望wic用文件系统来格式化分区，那么你需要提供--fstype。

如果你使用--source rootfs，wic创建一个根文件系统分区，其具有足够空间用于由-r（--rootfs-dir）参数提供的根文件系统。

·--size：以MB为单位的最小的分区大小。如果你不使用--source，那么你必须提供这个参数。如果你想创建比由根文件系统内容（也参见--extra-space和--overhead-factor）所决定的分区更大的分区，那么你可以和--source一起提供这个参数。

·--ondisk：在特定的设备上创建分区。

·--ondrive：与--ondisk相同。

·--fstype：用于格式化分区的文件系统类型。被支持的文件系统类型是ext2、ext3、ext4、btrfs、squashfs和swap。

·--fsoptions：将被写到/etc/tstab的选项字符串。字符串需要被包在引号中。如果你不指定这个参数，那么它被设置成"defaults"。

·--label：分区标签。

·--active：把分区标记为启动分区。

·--align：在一个n KB边界上开始分区。

·--no-table：创建、格式化并且填充分区但不增加它到分区表。

·--extra-space：增加额外的空间（以MB为单位）到分区。默认值是10MB。

·--overhead-factor：成倍放大分区大小，分区大小要么是由--size提供的要么是从由-r（--rootfs-dir）指定的根文件系统目录内容决定的，然后乘以这个因子。默认值是1.3。

·--part-type：为分区类型指定全局唯一标识符（Globally Unique Identifier，GUID）以用于GUID分区表（GPT）[1]。

·--use-uuid：为分区生成随机Linux UUID。

·--uuid：为分区指定Linux UUID。

bootloader指令

bootloader指令提供用于引导加载程序的配置：

```
bootloader --timeout=<timeout_in_seconds> --append="<kernel_parameters>
```

指令仅仅接受两个参数：

·--timeout：在引导加载程序启动默认选项前等待的时间（以s为单位）。这个参数被用在向用户展示启动选项列表的引导加载程序上。

·--append：包在引号中、被传递到Linux内核的参数字符串。

[1] 为获取关于GPT的信息，见https://wiki.archlinux.org/index.php/GUID_Partition_Table和https://en.wikipedia.org/wiki/GUID_Partition_Table#Partition_type_GUIDs。

10.6.5 插件

插件为wic功能性提供简单扩展性，它们是用Python编写的。当前，有两类插件：镜像器（imager）和源。你可以在目录poky/scripts/lib/wic/plugins中找到插件。每个插件都有自己的子目录。

镜像器插件安装整个系统到一个文件，该文件包含一个分区表和一个或者多个用文件系统格式化的分区。镜像器的输出是一个可直接传输到介质的文件。所有镜像器插件继承自ImagerPlugin Python类。当地仅有一个镜像器插件——DirectPlugin，它用于创建所有镜像。

源插件从特定源创建特定类型的分区。它们被用在kickstart文件的part指令中。源插件继承自SourcePlugin Python类。

文件poky/scripts/lib/wic/pluginbase.py定义核心插件类ImagerPlugin和SourcePlugin。

当前，用户扩展仅仅限于源插件。为了创建你自己的源插件，写一个包含插件类的Python源文件。你可以把文件放进目录poky/scripts/lib/wic/plugins/source中，或者你可以把它放进在目录meta-mylayer/scripts/lib/wic/plugins/source中的你自己的层内。层的根目录下的精确路径是重要的；否则，wic不能定位到插件。

为了编写你自己的源插件，你从基源插件类SourcePlugin衍生它并且实现执行任务的函数，如程序清单10-7所示。

程序清单10-7 源插件

```
class MyPartitionPlugin(SourcePlugin):
    name = 'mypartition'

@classmethod
def do_prepare_partiton(cls, part, ...)
    ...

@classmethod
def do_configure_partition(cls, part, ...)
    ...
@classmethod
def do_install_disk(cls, part, ...)
    ...

@classmethod
def do_stage_partition(cls, part, ...)
    ...
```

你自己的源插件总是必须设置name属性为唯一名字。这是源插件为wic所知的名字。它是你使用part指令的--source参数的名字，例如：

```
part / --source mypartition <options>
```

依赖于你的源插件，你需要实现一个或者多个方法，wic将在分区创建过程的各个阶段调用这个或者这些方法：

·do_configure_partition()：在do_prepare_partition()之前被调用，并用于为分区创建配置文件，例如引导加载程序配置文件。

·do_stage_partition()：在do_prepare_partition()之前被调用，并用于剪裁由源提供的分区内容。

·do_prepare_partition()：被调用来填充内容进分区。这个方法创建分区镜像，然后该分区镜像被集成进硬盘镜像。

·do_install_disk()：在do_prepare_partition()之后并且在分区集成进最终硬盘镜像之前，如果额外的步骤需要执行，那么它被调用。

源插件至少实现do_prepare_partition()方法，因为它就是实际从内容源rootfs-dir、kernel-dir和bootimg-dir中填充分区内容的方法。这些参数

的值被传到方法以便插件可以访问各自目录。如果你的源插件不实现一个特定方法，那么作为替代，超类方法被使用。对这4个方法，超类方法不做任何事只记录调试消息。

10.6.6 传输镜像

在wic完成了镜像的创建后，你可以简单地传输它到SD卡或者USB棒或者任何你为其构建镜像的其他介质上：

```
$ sudo dd if=<image_file> of=/dev/<device>
```

wic在创建了镜像后，它提供你镜像的名字和位置。默认情况下，输出目录是/var/tmp/wic/build。

因为你正在把镜像文件直接写入到设备，所以你必须以root权限执行命令。这可能是危险的，因为指定了错误的设备会擦除开发主机上的设备和分区。总是要两次确认，你正在使用正确的设备名。在插入了介质后立即使用dmesg来找正确的设备名总是一个好主意。

10.7 总结

Yocto项目板支持包提供调整层来用相同的核心构建系统支持很多不同的硬件平台。

·不像传统的嵌入式设备板支持包，Yocto项目板支持包不是独立的。它们需要OpenEmbedded核心以及可能的其他元数据层。

·Yocto项目板支持包不包含工具链或者开发工具。这些是由核心层提供的。

·Yocto项目板支持包是BitBake层，其通过增加它们的路径到构建环境的conf/bblayers.conf文件的BBLAYERS变量中而包含在构建环境中。

·Yocto项目板支持包必须定义至少一个提供目标平台特定设置的机器配置文件。

·Yocto项目板支持包可能增加它自己的菜谱或追加来自其他层的菜谱以按照目标硬件的需求调整包的构建。

·Yocto项目板支持包可以简单地互相交换而不涉及除构建环境的conf/local.conf文件中的MACHINE变量以外的配置设置。

·因为Yocto项目板支持包仅仅包含对构建系统的目标特定调整，所以维护被极大地缩减了。

·Yocto项目板支持包遵循特定布局。

·yocto-bsp工具允许一个基本板支持包的快速创建，该板支持包遵从板支持包惯例。

·yocto-kernel工具简化了Linux内核配置选项、补丁和特性的管理。

·OpenEmbedded镜像创建器（wic）极大地简化了可启动镜像的创建，该镜像可直接传输到各种介质。wic可通过kickstart文件和源插件而被扩展。

10.8 参考文献

Yocto项目板支持包开发者指南，www.yoctoproject.org/docs/1.8/bsp-guide/bsp-guide.html。

第11章　应用开发

在前面的章节中，我们学习了如何使用Yocto项目来构建Linux操作系统镜像、如何定制那些镜像以及如何通过板支持包来调整它们以适应特定硬件。由硬件、操作系统栈和一堆其他开源软件组成的设备不是一个产品。最终，你希望为设备开发自己的提供最终用户功能性的软件包，构建它们并且部署它们到设备。对于这样的任务来说，你需要用于你目标系统的软件开发环境，它通常在应用程序开发工具包（ADT）或者软件开发工具包（SDK）中被找到。Yocto项目使用这两者，并且通常把已经被ADT安装程序（来自从Web服务器提取的包仓库）安装的ADT叫作SDK，把已经被构建系统创建的SDK叫作SDK。实际的工具包提供相同的功能性，因为用于ADT的包仓库前面已经被构建系统创建。

使用Yocto项目构建系统，你可以构建匹配你的目标系统本身的应用程序开发工具包。然后你可以使用ADT环境来为你的设备开发和构建应用。

在本章中，我们研究Yocto项目ADT是由什么组成的以及你可以如何构建和使用它。

11.1 Yocto项目ADT内部

那么，你究竟能用Yocto项目ADT获得什么呢？由以下内容组成的完整的应用开发环境：

·交叉开发工具链：ADT交叉开发工具链由交叉编译器、交叉链接器、交叉调试器和一组用于应用开发的其他工具组成。

·系统根：ADT包含2个系统根：一个用于开发主机，它包含交叉开发工具链和其他工具；另一个用于目标，它是用于目标的完整根文件系统，也包含具有头文件和库等的开发包。

·QEMU模拟器：与内核和根文件系统镜像一起，QEMU提供用户空间应用的测试而不用实际硬件。你甚至可以在硬件可用前就开发用于目标的应用。

·环境设置：用于在你的开发主机上为使用ADT进行交叉开发而设置环境的脚本被提供。

·Yocto项目Eclipse插件：插件用于流行的Eclipse IDE[1]与ADT集成。

·分析工具：用于在你的目标系统上分析应用的各种用户空间工具被包含在ADT中。工具集包括以下：

·LatencyTOP：LatencyTOP[2]是为Linux系统测量和解决应用中的那些影响用户体验的延迟问题的工具，这样的问题例如在媒体回放时的声音和视频跳帧、在桌面界面上对用户输入的延后响应等，即使你的系统有大量CPU能力。

·PowerTOP：对几乎所有嵌入式系统来说，电量管理都是至关重要的，特别是当它们运行在电池上的时候。PowerTOP[3]是测量电量消耗并追踪电量消耗到应用、库、例程（routine）和代码片段的诊断工具。

·OProfile：OProfile[4]是用于Linux系统的系统范围分析工具，它能够分析运行中的代码而仅仅增加少量开销。它是由用于为目标系统收集采样数据的一个Linux内核驱动和一个守护进程（daemon）以及数个在主机系统上分析采样数据的离线工具构成。

·Perf：Perf[5]是一个Linux分析工具，它使用Linux内核计数器来收集各种意见和软件事件上的数据，例如CPU周期、指令、中断、缓存引用等。

·System Tap：System Tap[6]是用于收集关于运行中的Linux系统的信息的基础设施和检测。System Tap脚本允许通过放置探针（probe）来追踪几乎任何系统事件。

·Linux Trace Toolkit——Next Generation（LTTng）：LTTng[7]是用于Linux系统的开源追踪框架，它提供检测来识别系统事件，用极少开销来抽取已识别的事件，并且提供用于检查和分析的工具。

包含在ADT中的组件给应用开发者提供了使用Linux和中间件应用程序编程接口来以C和C++编写用户空间应用的所有必要工具。这些可以是基于GNU Make的、基于GNU Autotools的或者基于CMake的应用。在初始化了ADT环境后，你可以使用命令行来交叉编译应用。

然而，很多应用开发者倾向于集成开发环境（IDE）的便利性和生产率，集成开发环境允许他们从用户图形界面内编辑、构建和调试应用。为了那个目的，Yocto项目为流行且可扩展的Eclipse IDE提供了插件。插件把ADT和很多可用于Eclipse的工具（也就是，Eclipse C/C++工

具化）集成起来。通过Eclipse目标通信框架（Target Communication Framework，TCF），Yocto项目Eclipse插件也提供了靶上远程应用执行和调试。通过目标通信框架，你可以从Eclipse IDE内部直接部署应用二进制到目标设备，在目标设备上运行它并且与它交互。你也可以在位于你目标上的GNU调试服务器（gdbserver）内远程地运行你的应用。GDB交叉调试器，它是由Eclipse在你的部署主机上启动的，它连接到你目标上的gdbserver，允许你从Eclipse IDE控制调试会话，包括但不限于设置断点、在你的代码中步进以及检查变量。

为嵌入式系统开发应用不局限于编写代码，然后调试和部署它，而且通常包含为性能、电源消耗等而优化。除了System Tap以外都与Eclipse IDE集成的调试工具，还可以与ADT集成并且被增加到目标根文件系统以辅助你分析应用。

接下来的部分将带你浏览一遍构建ADT，安装它，使用它用于命令行开发，为了开发、构建和靶上分析的完整往返开发体验使其与Eclipse IDE集成的过程。

[1] https://www.eclipse.org。
[2] http://git.infradead.org/latencytop.git。
[3] https://01.org/powertop。
[4] http://oprofile.sourceforge.net。
[5] https://perf.wiki.kernel.org。
[6] https://sourceware.org/systemtap。
[7] http://lttng.org。

11.2 设置Yocto项目ADT

你可以以多种方式设置ADT：

·下载ADT安装程序：从Yocto项目下载站点[1]下载ADT安装程序tar包，解压它，配置它用于你的目标，然后运行它。接着，ADT安装程序从Yocto项目下载站点下载合适的交叉工具链、根文件系统等，并且在你的开发系统上安装它们。这个方法对于设置ADT来说是最方便的，但是它使用预创建的系统根镜像，很大可能不匹配你的目标镜像。

·构建ADT安装程序：使用Yocto项目构建环境自己构建ADT安装程序而不是下载它。在那之后，它与下载ADT安装程序就完全相同了。

·构建工具链安装程序来创建ADT：使用你的目标构建环境，创建包含交叉工具链和目标系统根的工具链安装程序，该安装程序完全匹配你的目标系统并且具有用于你可能已经增加到定制镜像中的软件包的开发包。

前两种方法已经由Yocto项目应用开发者指南[2]详细解释了。在本章中，我们聚焦在第3种方法上，使用构建环境为目标创建工具链安装程序。

[1] http://downloads.yoctoproject.org/releases/yocto/yocto-2.0/adt-installer。
[2] www.yoctoproject.org/docs/1.8/adt-manual/adt-manual.html。

11.2.1 构建工具链安装程序

如果你已经有了用于你目标系统的构建环境，可能包含板支持包和其他层，那么你可以使用它来构建带有你增加到定制根文件系统镜像中的全部软件组件的工具链安装程序。对我们的例子来说，我们正在使用来自前一章用于亭项目的构建环境和板支持包层。

我们的板支持包层meta-ypbkiosk当前不包含定制镜像目标。程序清单11-1显示了用于它的镜像菜谱。

程序清单11-1　定制镜像菜谱（ypbkiosk-image-sato.bb）

```
DESCRIPTION = "Custom image for the Yocto book Kiosk, which is based \
               on core-inmage-sato. We only replaced the Dropbear SSH \
               server with the OpenSSH server, which is necessary for \
               the ADT, and added tar.bz2 to the image types built."
IMAGE_FEATURES += "splash package-management x11-base \
                   x11-sato ssh-server-openssh hwcodecs"

LICENSE = "MIT"

inherit core-image

IMAGE_INSTALL += "packagegroup-core-x11-sato-games"

IMAGE_FSTYPES += "tar.bz2"
```

在meta-ypbkiosk层内创建目录recipes-core/images，并且向其增加程序清单11-1中的菜谱文件ypbkiosk-image-sato.bb。

现在，在引用了构建环境后，通过执行以下命令来构建工具链：

```
$ bitbake -c populate_sdk ypbkiosk-image-sato
```

也要确保你构建环境的conf/local.conf中的MACHINE变量被设置成MACHINE="ypbkiosk"。构建系统从这个变量获取用于工具链的正确的架构设置。

populate_sdk适用于所有镜像目标，并且为镜像目标创建工具链，它把类似MACHINE和EXTRA_IMAGE_FEATURES等的所有其他设置考虑在内。一旦任务完成，它把工具链安装程序放在构建环境的tmp/deploy/sdk中。工具链安装程序是以.sh结尾的单一可执行文件。它部分是shell脚本安装程序，部分是包含在主机和目标系统根中的实际工具链的有效载荷。如果你打开文件看它的内容，那么你会发现文本MARKER:，它把脚本和有效载荷分开。

除了方便分发，这个单一文件配置的目的是使得工具链完全自包含，意味着所有二进制都被连接到它们自己的libc副本，这导致不依赖于主机系统。因为在构建时，工具链的安装路径是不可知的（你可能安装它在你系统中的任何地方），并且到动态加载器的指针不能被动态地调整，所以，shell脚本部分处理重定位。

用于工具链为其构建的开发主机的架构是由配置变量SDKMACHINE决定的。在创建工具链安装程序的时候，这个变量被自动设置成构建系统正在运行于其中的主机的架构。如果你希望为不同的开发主机架构构建工具链，而不是为了你正用来构建工具链的这个东西，那么你可以显式地设置你构建环境的conf/local.conf中的SDKMACHINE变量。当前被支持用于构建的架构只有i686（x8632位）和x86_64（x8664位）。

11.2.2 安装工具链

现在，工具链的安装是相当简单的：

```
$ cd tmp/deploy/sdk
$ ./poky-glibc-x86_64-ypbkiosk-image-sato-corei7-64-toolchain-2.0.sh
```

实际工具链安装程序文件可能依赖于你的设置。工具链安装程序默认安装工具链到/opt/poky/<version>目录。如果你希望的话，那么你可以提供不同的目录。

工具链安装程序的一个特性是，你可以为具有任何架构的任何机器创建不同的安装程序，并且把它们安装到相同的目录。构建系统对每个架构仅仅创建一组cross-canadian工具链二进制[1]并且工具链安装程序分离目标系统根到不同的目录中。这是有可能的，因为描述目标硬件的细节可以作为选项被传到编译器。那些选项是由环境脚本通过赋值例如CC、LD等的变量来设置的。

程序清单11-2显示了用tree-L 3获得的工具链安装目录的布局。

程序清单11-2　工具链安装目录布局

```
/opt/poky/2.0
├── environment-setup-corei7-64-poky-linux
├── environment-setup-cortexa8hf-vfp-neon-poky-linux-gnueabi
├── site-config-corei7-64-poky-linux
├── site-config-cortexa8hf-vfp-neon-poky-linux-gnueabi
├── sysroots
│   ├── corei7-64-poky-linux
│   │   ├── bin
│   │   ├── boot
│   │   ├── dev
│   │   ├── etc
│   │   ├── home
│   │   ├── lib
│   │   ├── media
│   │   ├── mnt
│   │   ├── proc
│   │   ├── run
│   │   ├── sbin
│   │   ├── sys
│   │   ├── tmp
│   │   ├── usr
│   │   └── var
│   ├── cortexa8hf-vfp-neon-poky-linux-gnueabi
│   │   ├── bin
│   │   ├── boot
│   │   ├── dev
│   │   ├── etc
│   │   ├── home
│   │   ├── lib
│   │   ├── media
│   │   ├── mnt
│   │   ├── proc
│   │   ├── run
│   │   ├── sbin
│   │   ├── sys
│   │   ├── tmp
│   │   ├── usr
│   │   └── var
│   └── x86_64-pokysdk-linux
│       ├── etc
│       ├── lib

│       ├── sbin
│       ├── usr
│       └── var
├── version-corei7-64-poky-linux
└── version-cortexa8hf-vfp-neon-poky-linux-gnueabi
```

文件和子目录可以被分类成以下几部分：

·环境设置：environment-setup-*脚本设置用于各种架构的工具链配置。如果你希望使用特定的工具链，那么你需要引用各自脚本，类似于在设置Yocto构建环境时引用oe-init-build-env脚本。

·站点配置：在使用GNU Autotools开发软件包时，站点配置文件site-config-*包含配置设置。

·系统根：sysroots子目录包含一个带有用于每个目标架构和主机架构的系统根的子目录。例如，子目录x86_64-pokysdk-linux是用于主机的系统根，它包含交叉工具链。另外，该示例也包含用于MinnowBoard Max的Intel Core i7的目标系统根和用于BeagleBone的ARM Cortex A8的目标系统根。

·版本文件：version-*文件包含关于工具链版本的版本信息。

关于构建工具链安装程序的最后一点说明：默认情况下，创建的工具链仅仅构建动态连接的二进制。如果你希望构建静态连接的二进制，那么你需要保证在你的系统根中能找到那个包含有静态库的包。你可以通过增加这些包到IMAGE_INSTALL变量来实现这个目的。这个例子增加glibc静态库：

```
IMAGE_INSTALL_append = " glibc-static"
```

添加这一行到conf/local.conf，并且增加任何你需要的其他静态库。

[1] Cross-canadian意味着，在由HOST_ARCH定义的架构上运行的构建系统创建运行在由SDKMACHINE定义的架构上的工具链，它还可以为运行在由TARGET_ARCH定义的架构上的系统构建软件。这可能涉及3个不同的架构。然而，最常见的是，SDKMACHINE和HOST_ARCH是相同的。

11.2.3 用工具链工作

为了将Yocto项目工具链用于你的项目，首先必须使用合适的脚本来初始化环境。对我们的亭的例子，脚本如下：

```
$ source environment-setup-corei7-64-poky-linux
```

工具链初始化脚本是以与构建环境设置脚本完全相同的方式被引用的。如果看一下脚本内部，你会注意到它设置了一系列由脚本赋值的环境变量。这些环境变量按照字母顺序列在这里，而不是按照它们在脚本中出现的顺序：

·AR：用于ar来维护静态库的最小化命令和选项。

·ARCH：目标系统的架构。

·AS：用来为目标系统运行交叉汇编程序的最小化命令和选项。

·CC：用来为目标系统运行C交叉编译器的最小化命令和选项。

·CCACHE_PATH：Ccache[1]是用于C、C++、Objective-C和Objective-C++编译器的编译器缓存。在构建时，它在一个目录中缓存中间编译器输出，如在后续的构建中没有任何变化，就重用它。它可以极大地加速后续构建的时间。第一个构建是稍慢的，因为缓存被创建。Ccache仅仅工作于GNU编译器集合（GNU Compiler Collection，GCC）编译器以及具有类似行为的编译器。脚本增加路径到交叉工具链以告诉ccache在哪里找到工具链二进制。默认情况下，ccache在${HOME}/.ccache中存储缓存文件。如果希望改变位置，那么你也需要设置CCACHE_DIR环境变量。脚本没有那么做。

·CFLAGS：用于C交叉编译器的标志。

·CONFIG_SITE：用于GNU Autotools的站点配置。

·CONFIGURE_FLAGS：用于GNU Autotools configure命令的标志。

·CPP：用来为目标系统运行C预处理器的最小化命令和选项。

·CPPFLAGS：用于预处理器的标志。

·CXX：用来为目标系统运行C++交叉编译器的最小化命令和选项。

·CXXFLAGS：用于C++交叉编译器的标志。

·GDB：用来为目标系统运行GNU调试器（GNU Debugger，GDB）的最小化命令和选项。

·KCFLAGS：用于编译Linux内核的标志。

·LD：用来为目标系统运行交叉链接器的最小化命令和选项。

·LDFLAGS：用于交叉链接器的标志。

·NM：用于nm来检查二进制文件（可执行文件、对象文件、库）并展示存储在其中的元数据（特别是符号表）的最小化命令和选项。

·OBJCOPY：用于objcopy来复制和翻译对象文件的最小化命令和选项。

·OBJDUMP：用于objdump来展示关于对象文件的各种信息的最小化命令和选项。

·OECORE_ACLOCAL_OPTS：用于aclocal命令的选项，该命令是GNU Autotools Autoconfig的一部分。

·OECORE_DISTRO_VERSION：用于工具链的版本号。

·OECORE_NATIVE_SYSROOT：到主机系统根的路径。

·OECORE_TARGET_SYSROOT：到目标系统根的路径。

·PATH：向用于你开发系统的可执行文件的搜索路径增加到主机系统根内的/usr/bin目录的路径以便交叉工具链命令可以被找到并被执行。

·PKG_CONFIG_SYSROOT和PKG_CONFIG_PATH：到被pkg-config使用的目标包配置的路径。

·PYTHONHOME：到包含在主机系统根中的Python解释器的路径。

·RANLIB：用于ranlib来在静态库中增加和更新文件的最小化命令和选项。

·SDKTARGETSYSROOT：到包含开发包的目标系统根的路径。这个变量被传到例如编译器、链接器等的交叉工具链命令。

·STRIP：用于strip命令来从二进制中除去符号的最小化命令和选项。

·TARGET_PREFIX，CROSS_COMPILE：用于交叉工具链工具的工具链二进制前缀。

这些环境变量中的很多是标准变量，因为它们是makefile用来构建应用的。当你开发用于你目标系统的自有应用时，在你正在编写的makefile中利用这些变量。这些变量也被构建系统用在菜谱中，这意味

着，你不需要用EXTRA_OEMAKE来覆盖它们。

在我们深入编写应用的更多细节前，让我们构建仅仅包含一个C文件的简单程序，正如程序清单11-3所示。

程序清单11-3　计算斐波那契数列（fibonacci.c）

```
#include <stdio.h>

int main()
{
   int n, first = 0, second = 1, next, c;
   printf("Enter the number of terms: ");
   scanf("%d",&n);

   printf("First %d terms of Fibonacci series are:\n", n);

   for (c = 0 ; c < n ; c++)
   {
      if (c <= 1)
         next = c;
      else
      {
         next = first + second;
         first = second;
         second = next;
      }
      printf("%d\n",next);
   }

   return 0;
}
```

在你开发系统的任何目录中创建这个文件。然后初始化工具链并且构建应用：

```
$ source environment-setup-corei7-64-poky-linux
$ ${CC} fibonacci.c -g -o fibonacci
```

第二个命令使用工具链环境为我们的亭目标系统交叉编译fibonacci.c。我们增加-g选项来增加调试符号。如果你尝试在你的开发主机上执行应用，那么最可能看到类似如下的输出：

```
$ ./fibonacci
bash: ./fibonacci: /lib/ld-linux-x86-64.so.2: bad ELF interpreter: \
   No such file or directory
```

这毫不意外，因为你的应用是动态连接到用于我们目标的glibc库的，这几乎是确定不同于你开发系统上的库。

[1] http://ccache.samba.org。

11.2.4 目标上执行

如果你有一个MinnowBoard Max，并且它具有我们在前面章节中构建的Linux操作系统栈，那么复制二进制到目标。你可以通过内存棒来完成或者通过网络使用scp：

```
$ scp fibonacci root@<target_ip>:/usr/bin/fibonacci
```

替换<target_ip>为你的MinnowBoard Max IP地址。现在你可以在目标上通过目标的控制台或者ssh连接来执行程序：

```
$ ssh root@<target_ip>
root@ypbkiosk:~# fibonacci
Enter the number of terms: 10
First 10 terms of the Fibonacci series are:
0
1
...
root@ypbkiosk:~#
```

11.2.5 远程目标上调试

作为软件开发者，故障排除和修复缺陷是你的生计。首选的工具是这样一个调试器——它让你控制和检查运行程序的状态以及在程序崩溃之后分析它，这通常被称为事后总结调试（post-mortem debugging）。Yocto项目提供了GDB[1]作为一个用于目标的包以及作为一个用于开发主机的带有交叉版本的包。像任何交叉开发工具，交叉调试器运行在使用一个架构的开发主机上，而同时能够调试被编译成用于另一个架构的主机上的二进制、可执行程序以及库。

通过在conf/local.conf的变量EXTRA_IMAGE_FEATURES中包含tools-debug，我们已经包含了目标包。交叉版本被自动包含在SDK中。

你可以直接使用如下命令来在目标上运行GDB：

```
root@ypbkiosk:~# gdb /usr/bin/fibonacci
```

GDB启动，开始我们的斐波那契程序，并且停在第一个指令上。从这里，你可以使用GDB命令来控制执行中的程序（在GDB术语中被称为下标（inferior））、展示程序变量等。

然而，直接在目标上调试并不总是可能的。具有内存和硬盘限制的目标可能不能存储和加载被处理的二进制或者程序的调试信息。而且，GDB需要定位和处理例如函数和变量名、变量值以及栈追踪等信息，这要求你在目标上使用包含调试信息的可执行文件，这也常被叫做非精简二进制（non-stripped binary）。另外，为了能够于调试时在GDB内检查程序的源代码，你需要复制程序的所有源文件到目标上。对于简单到如我们的斐波那契例子一样的程序，那是简单和直接的，但是对于从很多不同源文件中构建的程序，它可能变得相当笨重。

为了突破这些限制，你可以用gdbserver来远程调试。gdbserver不是调试器而是运行在目标上并且控制被调试的程序或者下标的服务器进程。gdbserver不加载和处理任何与被调试的程序相关的调试信息而是中继所有信息到运行于开发主机上的GDB。在开发主机上的GDB发送控制命令到目标上的gdbserver以启动或者停止下标、设置断点、读和写目标变量、单步调试（step through）程序等。因为所有针对调试的处理都是由开发主机上的GDB来完成的，所以在目标上仅仅需要具有调试符号二进制。

如果你希望调试用在你程序中的库，那么需要通过在你目标镜像的IMAGE_INSTALL变量中包含这些库的调试包来把它们安装在目标上。按照惯例，所有调试包以-dbg结尾。如果你希望包含由你的目标镜像安装的所有包的调试包，你可以向conf/local.conf中的EXTRA_IMAGE_FEATURES增加dbg-pkgs。

如果你确实包含所有包的调试包，那么注意，你的目标根文件系统镜像的大小将明显增加，因为源文件是*-dbg包的组成部分。通过设置你构建环境的conf/local.conf文件的PACKAGE_DEBUG_SPLIT_STYLE变量为debug-without-src，你可以命令构建系统不把源文件包含进调试包中。因为这个变量全局性地控制所有包的打包行为，所以它将导致整个目标系统的重新构建。

因为在开发主机上的GDB负责加载和处理所有调试信息，所以它必须对非精简二进制（用-g选项并且不带任何优化而编译的可执行文件和所有库）拥有访问。在目标上的二进制可以是精简的，但是不能带优化编译。

GDB和gdbserver使用基于网络或者（替代性地）串行连接的命令行接口互相通信。你首先在目标上启动gdbserver和被调试的程序，然后你开发主机上启动GDB，命令GDB连接到你目标上的gdbserver。

在目标上启动gdbserver

为了远程调试，你需要把gdbserver安装在你的目标上。你可以直接在你的目标镜像中安装gdbserver，但是使用tools-debug镜像特性是更加便利的[2]。

直接在控制台中或者通过安全Shell来在你的目标上启动gdbserver。例如，对于我们的斐波那契程序，使用以下命令：

```
root@ypbkiosk:~# gdbserver localhost:2345 /usr/bin/fibonacci
Process /usr/bin/fibonacci created; pid = 810
Listening on port 2345
```

gdbserver直到被调试的进程终止才退出。端口2345是用于GDB和gdbserver的默认端口。如果你希望，那么你可以修改端口。

在开发主机上启动GDB

为了调试在你目标上运行着的进程，你需要启动交叉调试器并且命令它连接到你目标上的gdbserver。从你已经构建了斐波那契应用的目录内，正如在11.2.3小节中做的那样引用环境设置脚本，然后启动调试器：

```
$ source environment-setup-corei7-64-poky-linux
$ ${GDB} fibonacci
GNU gdb (GDB) 7.9.1
...
Reading symbols from fibonacci...done.
(gdb)
```

为了清晰，我们省略了GDB的一些初始输出。在完成了初始化后，GDB展示它的命令行提示（gdb）。命令GDB连接到目标上的gdbserver：

```
(gdb) target remote <target_ip>:2345
Remote debugging using <target_ip>:2345
...
(gdb)
```

把<target_ip>替换成你目标系统的IP地址。如果你为gdbserver指定了不同的端口，那么你必须在此处指定相同的端口。在你的目标系统上，应该看到gdbserver响应连接：

```
    Remote debugging from host <host_ip>
```

现在可以在开发主机上开始你的调试会话了。输入continue将在你的目标上运行程序。当目标上的下标进程终止后，在目标上运行着的gdbserver也将终止。然而，你开发主机上的GDB将继续运行。你可以重启目标上的gdbserver，然后重新连接GDB。

除非你是一个顽固的命令行开发者，否则使用GDB和它的命令行可能不是你的菜。有很多可用于GDB的图形前端。其中一个是由GNU项目提供的可靠的数据显示调试器（Data Display Debugger，DDD）[3]。DDD可以用任何Linux发行版的包管理系统来轻松安装。DDD有基本但是可用的用户界面而不带任何虚饰，如图11-1所示。

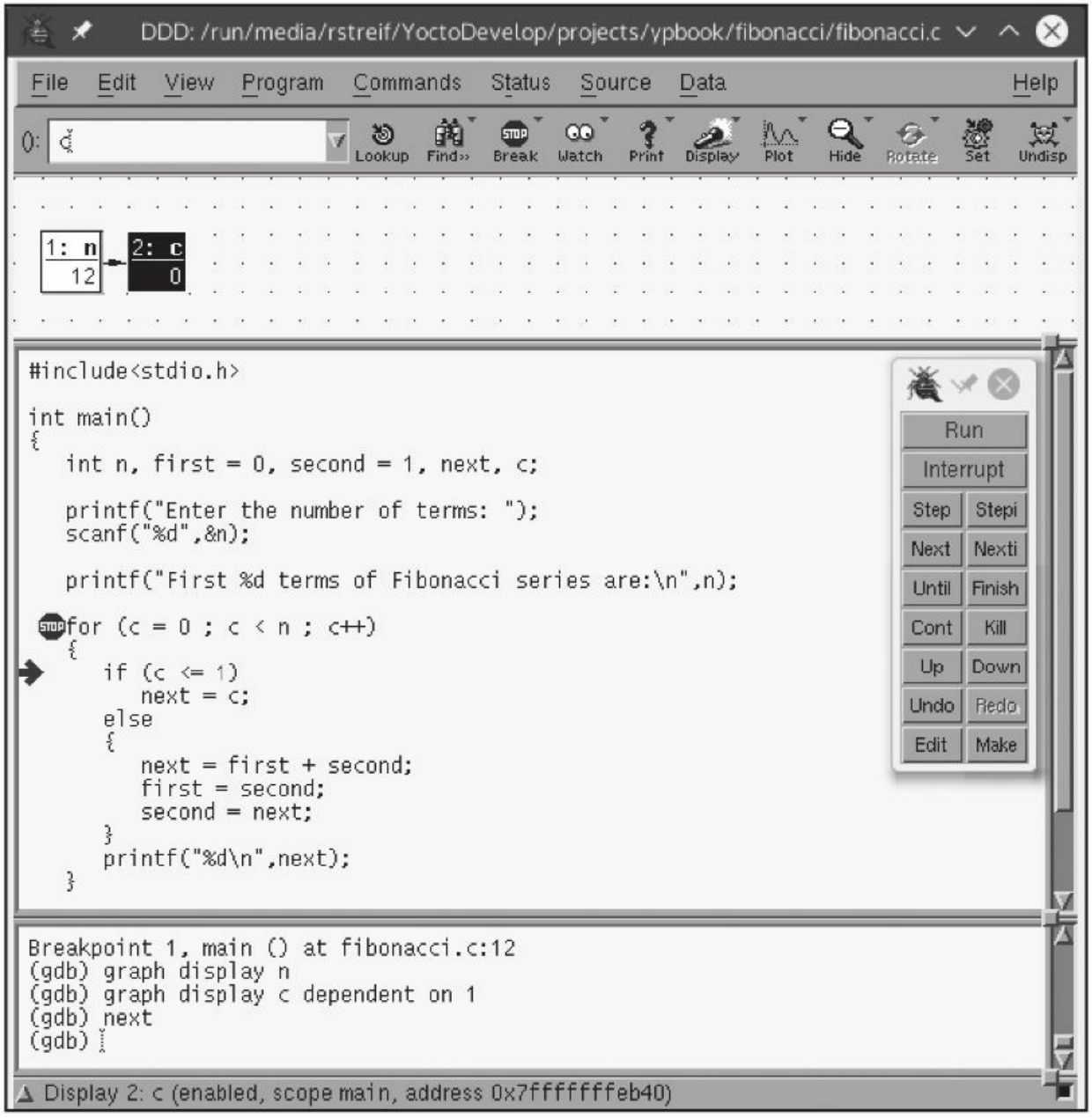

图11-1　DDD用户界面

DDD默认使用安装在你开发系统上的主机GDB。你可以命令DDD使用来自Yocto项目SDK的交叉调试器：

```
$ source environment-setup-corei7-64-poky-linux
$ ddd --debugger ${GDB}
```

DDD不提供用来连接到远程gdbserver的按钮或者菜单项。你必须向位于DDD底部的GDB命令窗口手动输入target remote命令。

Eclipse IDE也提供了用于GDB的非常友好的图形前端。在11.4节中，我们将解释如何把Eclipse和Yocto项目以及调试器的使用集成起来。

调试标准库

如果你希望用GDB追踪安装在你目标上的标准库，那么你需要告诉GDB在哪里找到调试信息以及用于库的源文件。SDK在你的开发主机上安装用于目标平台的所有调试包，包括在SDK系统根内的源文件——例如，/opt/poky/2.0/sysroots/corei7-64-poky-linux是用于我们的MinnowBoard Max的SDK。

你可以通过输入以下内容来命令GDB使用一个系统根：

```
(gdb) set sysroot /opt/poky/2.0/sysroots/corei7-64-poky-linux
(gdb) set substitute-path /usr/src/debug \
          /opt/poky/2.0/sysroots/corei7-64-poky-linux/usr/src/debug
```

然后，GDB将使用这个系统根来寻找调试信息和源文件。当启动GDB时再次重复输入这些设置是相当单调乏味的。因此，你可以增加语句到.gdbinit文件。你可以把这个文件放在你的家目录中，在这种情况下设置将应用到所有项目；或者放在项目目录中，在这种情况下，当你从那个目录内部启动GDB时，它将仅应用到当前的项目。如果你确实在项目目录中使用了一个.gdbinit文件，那么在你的家目录中，你仍然需要一个包含如下行的.gdbinit文件：

```
set auto-load safe-path /
set auto-load local-gdbinit on
```

GDB的默认安全策略仅仅允许从用户的家目录中自动加载初始化文

件。这些设置也允许从当前目录自动加载。

[1] https://www.gnu.org/software/gdb。
[2] 我们推荐使用tools-debug而不是直接安装gdbserver包，因为它只要求对conf/local.conf的修改而不是修改镜像菜谱。
[3] https://www.gnu.org/software/ddd。

11.3 构建应用

任何重要的应用都可能由很多源文件组成，这些源文件被编译和连接成库或可执行程序文件。工具链的环境设置使得开发基于makefile和基于GNU Autotools的应用相当简单。

11.3.1 基于makefile的应用

对于基于makefile的项目，最好不要在makefile中设置例如CC、AS、LD和CFLAGS的任何环境变量，因为用于那些变量的设置是由用于工具链环境的初始化脚本确立的。简言之，你不应该直接设置或者提供有关特定架构的工具链标志，因为它们可能限制在不同的架构上编译项目的能力。工具链环境以及Yocto项目构建环境提供所有架构依赖的设置。

程序清单11-4显示了一个简单的makefile，用于构建我们的计算斐波那契数列的项目。

程序清单11-4 用于斐波那契数列项目的makefile

```
# Makefile for the Fibonacci Application

# Files
SOURCES=fibonacci.c
OBJECTS=$(SOURCES:.c=.o)
EXEC=fibonacci

# Define extra flags here
EXTRACFLAGS=-ansi

all: $(SOURCES) $(EXEC)

$(EXEC): $(OBJECTS)
	$(CC) $(CFLAGS) $(EXTRACFLAGS) $(OBJECTS) -o $@

%.o : %.c
	$(CC) $(CFLAGS) $(EXTRACFLAGS) -c $<
install:
	install $(EXEC) $(DESTDIR)/usr/bin

clean:
	rm -rf *.o $(EXEC)
```

当然，它依然是一个相当简单的项目，但是它举例说明了使用由环境提供的变量和通过独立变量来提供任何额外配置设置的概念。

11.3.2 基于Autotools的应用

在第8章中，我们讨论了GNU Autotools和为自动工具化的软件包编写菜谱。工具链环境提供用于构建自动工具化的包的所有必要设置，包括站点配置文件。GNU Autotools通过检测系统配置来使得应用可移植于不同类UNIX系统。当工作在交叉开发环境时，那种检测不能正常工作：Autotools将检测开发主机配置而不是用于目标的正确配置。向Autotools提供目标配置是站点配置文件的目的。

为了举例说明在自动工具化项目中使用工具链的过程，我们将构建GNU Hello应用：

```
$ source environment-setup-corei7-64-poky-linux
$ wget http://ftp.gnu.org/gnu/hello/hello-2.10.tar.gz
$ tar xvf hello-2.10.tar.gz
$ cd hello-2.10
$ aclocal ${OECORE_ACLOCAL_OPTS} -I m4
$ autoconf
$ autoheader
$ automake -a
$ ./configure ${CONFIGURE_FLAGS}
$ make
```

前4个步骤是不需解释的：我们设置环境，然后下载用于GNU Hello应用的源包，解压它，并且改变到已解压的源目录。

然后，我们运行aclocal来基于automake宏自动生成aclocal.m4。这些宏是从由-I选项增加的目录中收集的。OECORE_ACLOCAL_OPTS为工具链配置增加那些选项，而-I m4包含源的本地m4目录。

然后，运行autoconf、autoheader和automake创建被configure使用的配置输入、头文件模板和makefile模块。

在那之后，本地configure脚本被执行来确定构建配置和生成

makefile。环境变量CONFIGURE_FLAGS为交叉编译提供主机和目标配置设置。

最后但也很重要的是，运行make构建应用。

11.4 Eclipse集成

使用命令行工具、编辑器、交叉工具链、makefile、脚本等的开发长久以来一直是嵌入式开发者的日常工作，因为用于嵌入式系统具有图形用户界面的集成开发环境（IDE）的开发一直落后于用于原生开发的它们的同伴。工作在用于个人计算机的操作系统的原生应用的应用开发者通常有大量不同的IDE选择用于他们的平台来最合适地满足他们的需求。嵌入式开发者必须一直工作在硅片厂商或者第三方工具链公司为特定硬件和软件平台提供的工具上，在很多情况下，这种工具并不多。使用Linux用于嵌入式系统开发对嵌入式开发者来说已经扩大了开发工具的选择。

这些选择之一，而不是唯一选择，是Eclipse。Eclipse最初是IBM为了Java开发而开发的[1]，是可通过插件来扩展的一种IDE，以服务很多不同开发目的，并在一个通用框架内遵循标准工作流来集成各种工具。在幕后，Equinox[2]是核心开放服务网关协议（Open Services Gateway Initiative，OSGi）[3]框架规格说明的一个实现，它为插件被安装和集成进Eclipse以及两者互相通信提供管道系统。在OSGi术语中，插件通常被称为捆绑（bundle），软件包可以被安装进OSGi框架中并且向其他捆绑提供服务。

虽然最初Eclipse是作为用于Java编程语言的IDE而开发的，但是它已经扩展成支持很多其他编程语言了，其中有Ada、C/C++、Cobol、Erlang、Fortran、Haskell、JavaScript、Lua、Perl、PHP、Python、Ruby和Scala等；扩展成连接一系列源代码控制管理，例如Git、Perforce和Subversion等；还扩展成与一套正在增加的工具集成。那些插件之一是Yocto项目插件，它集成Yocto项目工具链等。

[1] 最初的Eclipse代码来自于IBM VisualAge IDE。

[2] www.eclipse.org/equinox。
[3] www.osgi.org。

11.4.1 安装Eclipse IDE

Eclipse安装包被预先配置成用于特定任务而发布，例如针对Java开发者的Eclipse IDE，针对Java EE开发者的Eclipse IDE，针对C/C++开发者的Eclipse IDE，以此类推。在Eclipse下载站点[1]，你可以找到完整列表。为了与Yocto项目集成，针对C/C++开发者的Eclipse IDE是最佳选择，因为它已经包含了插件需要的核心组件：

·C/C++开发工具

·Eclipse Git Team Provider

·远程系统浏览器（Remote System Explorer，RSE）

Yocto项目Eclipse插件要求匹配的Eclipse版本。对于Yocto项目2.0（Jethro），其中插件可用的Eclipse版本是Juno、Kepler和Luna。在Yocto项目Eclipse网站上，你总是可以找到你需要安装的Eclipse的那个版本[2]。

几乎所有主流Linux发行版都提供用于从它们的包仓库中安装的Eclipse。那些可能工作，但是在安装它们中的任何一个之前，你需要验证用于Yocto项目的匹配版本是被提供的，你很大可能需要独立安装上面列出的3个插件[3]。如果你的发行版不提供匹配的版本，或者你倾向于手动安装，那么你需要使用来自Eclipse站点的安装包：

1.安装Java：Eclipse是Java应用，至少需要Java运行时环境（Java Runtime Environment，JRE）[4]来运行。Eclipse可以和OpenJDK以及Oracle Java一起工作。Linux发行版通常从包仓库中提供OpenJDK用以安装。简单地使用你发行版的包管理器来安装Java并且用以下命令来验证

安装：

```
$ java -version
openjdk version "1.8.0_40"
OpenJDK Runtime Environment (build 1.8.0_40-b25)
OpenJDK 64-Bit Server VM (build 25.40-b25, mixed mode)
```

2.安装Eclipse：为你的开发主机系统下载针对C/C++开发者的正确的Eclipse IDE版本。在我们的例子中，这是eclipse-cpp-luna-SR2-linux-gtk-x86_64.tar.gz，它适用于任何64位基于x86的Linux系统。安装包是一个简单的压缩的tar包，你可以解压它到你系统内的任何目录中（我们使用/opt，然而，这需要root访问）：

```
$ cd /opt
$ sudo tar xvf \
    ~/Downloads/eclipse-cpp-luna-SR2-linux-gtk-x86_64.tar.gz
```

现在启动Eclipse IDE：

```
$ /opt/eclipse/eclipse &
```

Eclipse首先显示一个对话框用以为它的工作空间选择位置。默认~/workspace是合适的，除非你想使用不同的目录。

3.安装标准的Eclipse插件：你需要安装几个通过Eclipse下载站点提供的标准插件。为了安装那些插件，首先从Eclipse workbench的Help菜单中选择Install New Software...。然后，从Work with:组合框中选择匹配你的Eclipse版本的下载站点（对Eclipse Luna来说，那是Luna——http://download.eclipse.org/releases/luna）。

现在你可以选择用于安装的各种插件（如果任何以下的项没显示，那么它们已经被安装了）：

a.展开列表Mobile Device Development并且选择以下：

·C/C++Remote Launch（Requires RSE Remote System Explorer）

·Remote System Explorer End-user Runtime

·Remote System Explorer User Actions

·Target Management Terminal（Core SDK）

·TCF Remote System Explorer add-in

·TCF Target Explorer

b.展开列表Linux Tools并且选择

·Linux Tools LTTng Tracer Control

c.展开列表Programming Languages并且选择

·C/C++Autotools Support

·C/C++Development Tools

完成安装并重启Eclipse。

4.安装Eclipse Yocto插件：我们从Yocto项目下载站点安装插件。

a.在Eclipse已经再次启动后，从Help菜单中选择Install New Software...，然后在Work with：区域单击Add...。在Location：字段中输入http://downloads.yocto-project.org/releases/eclipse-plugin/2.0/luna，在Name：字段中输入一个有意义的名字，例如Yocto Project。

b.选择以下内容旁边的复选框：

·Yocto Project ADT Plug-in

·Yocto Project BitBake Commander Plug-in

·Yocto Project Documentation plug-in

通过接受许可协议完成安装，并且重启Eclipse。

你的Eclipse安装现在已经准备好用于与Yocto项目ADT集成。

[1] www.eclipse.org/downloads。
[2] https://www.yoctoproject.org/tools-resources/projects/eclipse-ide-plug。
[3] 对Yocto项目Eclipse插件来说，Eclipse Git Team Provider不是严格必要的，但却是推荐的。
[4] 当然，你也可以使用包含JRE的Java开发工具包（Java Development Kit，JDK）。

11.4.2 集成Yocto项目ADT

现在，我们正把在11.2节中设置的ADT与Eclipse和Yocto项目Eclipse插件集成起来。过程被拆分成以下的步骤：

1.配置交叉工具链选项。

2.配置目标选项。

在接下来的步骤中你选择的配置选项将成为你用Yocto项目插件开发的所有项目的默认设置。每个你创建的新项目会继承这些设置，对每个项目，你可以稍后调整它们。

对于下面部分描述的配置步骤来说，你首先需要：

1.从Window菜单中选择Preferences以显示Preferences对话框。

2.从列表中选择Yocto Project ADT以显示配置屏幕。

配置屏幕被划分成以下部分：

·Cross-Development Profiles（交叉开发配置文件）：你可以为Yocto项目ADT开发创建具有不同设置的多个配置文件。名为Standard Profile的配置文件是默认的，它被应用到所有新项目，除非在创建项目时你选择不同的配置文件。你不能移除Standard Profile，要保持选择为Standard Profile。

·Cross Compiler Options（交叉编译器选项）：这部分配置交叉工具链。

·Toolchain Type（工具链类型）：你有如下的选择：

·Standalone Prebuilt Toolchain（独立预构建工具链）：这个选项是用于已经被提供了一个预构建和打包的Yocto项目工具链的开发者，他们已经把这个工具链安装在了开发系统上。

·Build System Derived Toolchain（构建系统衍生的工具链）：如果你希望使用来自Yocto项目构建环境的工具链，那么使用这个选项。

因为我们用Yocto项目构建了一个工具链并且把它安装在了开发系统上，所以选择第一个选项。

·Toolchain Root Location（工具链根位置）：这个选项是指向工具链安装位置的路径。这个路径依赖于Toolchain Type：

·Standalone Prebuilt Toolchain：把路径指向你安装了工具链的位置。在我们的例子中，那是/opt/poky/2.0。

·Build System Derived Toolchain：如果你打算使用构建系统衍生的工具链，那么把路径指向你的构建环境的顶级目录。

·Sysroot Location（系统根位置）：这个选项是到用于你目标的系统根所在位置的路径。对于我们使用MinnowBoard Max的亭项目，这是/opt/poky/2.0/sysroots/corei7-64-poky-linux。

·Target Options（目标选项）：这个部分配置你希望用在你的Eclipse设置中的目标。有两个选择：

·QEMU：如果你打算使用QEMU模拟器来测试你的应用，那么选择这个选项。如果你正在使用QEMU，那么你也必须提供一个你可能向其传递额外参数的内核。

·External-HW：这个选项用于外部硬件，它是我们正在使用的硬

件。

单击Apply以使Yocto项目插件验证你的设置并且保存设置。单击OK来关闭对话框。Eclipse和Yocto项目插件现在已经为应用开发准备好了。

11.4.3 开发应用

Yocto项目Eclipse插件提供用于开发C/C++应用（使用CMake和GNU Autotools）的项目模板。构建文件，例如makefile，是用满足构建系统的cmake.bbclass和autotools.bbclass类的需求的环境设置来创建的。因此，编写构建你用Eclipse和Yocto项目交叉工具链开发的项目的菜谱与提供SRC_URI和继承各自类一样简单。

你也可以开发仅仅使用makefile而不用CMake或者Autotools的应用。然而，你必须手动设置用于交叉工具链的Eclipse环境。因此，我们推荐你遵循已提供的项目模板之一。你是否选择用CMake或者Autotools来构建项目依赖于你的项目的需求以及你的个人喜好。

通过启动Eclipse并从File菜单中选择New>Project...来开始创建你的新项目。从New Project对话框中，展开C/C++文件夹，从C Project或者C++Project中选择其一。接下来的例子使用相同的源代码构建一个简单的Hello World方式的项目，所以，选择C Project或者C++Project没有什么区别。对于该例，我们选择C Project。

下一个对话框让你选择你希望创建的项目类型。选择有：

·Yocto Project ADT Autotools Project

·Yocto Project ADT CMake Project

它们是由Yocto项目Eclipse插件提供的，并且包含与Yocto项目交叉工具链集成的模板。

开发基于CMake的应用

从项目对话框开始，遵循这些步骤来创建你的项目：

1.C项目页面：展开文件夹Yocto Project ADT CMake Project，从列表中选择Hello World C CMake Project。这创建一个简单的项目，它具有一个C文件（包含一个主要函数）、CMake文件等。替代性地，你可以选择Empty Project，然后你需要手动创建所有文件。在顶部的Project Name字段处输入名字，保持Use default location选中。项目名不能包含任何特殊字符串或者空格。单击Next。

2.一般配置页面：如果你希望，那么输入你的作者信息。也可以保持页面空白，你在这里输入的信息被自动加载到源文件的一个头文件中。单击Next。

3.选择配置页面：Debug配置默认是被选中的。如果你单击Advanced Settings，项目属性对话框打开。项Yocto Project Settings包含带有交叉开发配置文件的ADT设置、交叉编译器选项和目标选项。字段用我们前面输入的设置预先填充了。通过选择Use project specific settings，你可以覆盖这些设置。在你的项目已经被创建后，你随时都可以通过从Eclipse的Project菜单中选择Properties来回到这个对话框。单击Finish。

4.C/C++透视图（Perspective）：依赖于你当前的Eclipse状态，Open Perspective提示可能出现询问你是否希望改到C/C++Perspective。确定打开C/C++Perspective。Eclipse把它的不同环境称为透视图。依赖于你的Eclipse安装，有用于Java、C/C++、调试等的透视图。你可以从Eclipse的Window窗口菜单或者使用位于Eclipse窗口右上角的按钮在视图间切换透视。每个透视可以有多个视图，它显示在透视图的工作台区域。在左侧，你看到Project Explorer（项目浏览器）视图，它显示项目结构和文件。如果你希望编辑文件，那么在Project Explorer中双击它的名字，这加载它到内置的编辑器中。

现在你可以容易地从C/C++透视图内执行所有开发任务，包括用Yocto项目ADT交叉工具链构建项目和为你的目标构建系统根。可用下面两种方法中的任何一种：

·从Eclipse的Project菜单中选择Build Project。

·在Project Explorer中右键单击项目名称并选择Build Project。

在构建过程中，当然，对于这个简单项目，这是相当迅速的，Console视图显示各个构建步骤的输出。如果有任何的构建错误，那么Eclipse切换到Problems视图。如果构建成功，那么Eclipse在Project Explorer中的Binaries下面显示二进制文件。

开发基于Autotools的应用

创建和构建基于Autotools的应用和基于CMake的应用没有太多不同。再一次，从项目对话框中开始，遵循这些步骤来创建你的项目：

1.C项目页面：展开文件夹Yocto Project ADT Autotools Project，并从列表中选择Hello World ANSI C Autotools Project。在顶部的Project Name字段中输入名字，并保持Use default location选中。项目名不可以包含任何特殊字符或者空格。单击Next。

2.基本设置页面：如果你希望，那么输入你的作者信息并选择许可。这个对话框和用于CMake项目的General Settings有点不同，但是它具有类似的目的。单击Next。

3.选择配置页面：这个对话框和用于CMake项目的完全相同。单击Finish。

4.C/C++透视图：在完成后，Eclipse直接切换到C/C++透视图或者在

这样做之前询问你，这依赖于你当前的Eclipse状态。

构建基于Autotools的应用包含两个你可以从C/C++透视图内执行的步骤：

·配置：在Project Explorer中右键单击项目名称并选择Reconfigure Project。这调用autogen.sh脚本，它又执行libtoolize、aclocal、autoconf、autoheader、automake和configure，类似于在11.3.2小节中我们在命令上所做的。

·构建：在Project Explorer中右键单击项目名称并选择Build Project。

你可以遵循在Console视图中的步骤。在成功构建后，Eclipse在Project Explorer中的Binaries下面显示二进制文件。

11.4.4　在目标上部署、运行和测试

集成Yocto项目ADT工具链和Eclipse使得创建和构建基于CMake和基于GNU　Autotools的项目简单到就像在图形用户界面中点几下鼠标。当然，你仍然需要编写代码，但是单调乏味的操作被处理了。然而，在目标系统上部署、运行和测试你的应用仍然只是手动复制二进制文件到目标。

Eclipse的TCF为完整的往返开发体验增加了缺失的连接。TCF允许复制二进制文件到目标系统，远程地在目标系统上运行可执行应用，甚至直接从Eclipse内远程地在目标系统上调试应用。

TCF是轻量级但是可扩展的网络协议，它主要用于但是不限于与嵌入式设备或者目标通信。它被设计成用于开发系统上的工具的框架，其使用标准化且不依赖于特定传输层（例如TCP/IP、串行有线连接、SSH隧道等）的通信层来与目标服务进行交互。虽然TCP/IP是标准通信通道，其他协议是可用的并且可以增加。TCF使用JSON用于数据编组（marshalling），而且也支持目标和目标上服务的自动发现。位于其核心的是，TCF由运行在目标上的普通C实现的可扩展代理以及Java客户端API组成。后者与各种Eclipse工具集成，但是也可以用在独立的应用中。

准备目标用于远程控制

为了能够使用Eclipse和TCF来远程部署、运行、测试和调试应用，我们需要通过安装所需的组件来准备目标根文件系统。

Yocto项目提供了一组镜像特性，它们使得这个任务变得相当容易和直接。所有你要做的是，增加特性tools-debug和eclipse-debug到你构

建环境的conf/local.conf文件中的EXTRA_IMAGE_FEATURES：

```
EXTRA_IMAGE_FEATURES = "debug-tweaks tools-debug eclipse-debug"
```

然后重新构建镜像：

```
$ bitbake -k ypbkiosk-image-sato
```

并且部署它到MinnowBoard Max目标系统，正如在前面章节中所解释的。在重启了板后，你可以通过在目标上执行以下命令来验证TCP代理正在运行：

```
# ps | grep tcf-agent
699 root      668m S    /usr/sbin/tcf-agent -d -L- -l0
931 root      4412 R    grep tcf-agent
```

当然，你的输出有很大可能是稍微不同的，但是包含/usr/sbin/tcf-agent的行表示TCF代理正运行在目标上。你的目标现在已准备好接受来自你的Eclipse工作区的TCF连接。

在你可以从开发系统上的Eclipse工作区建立到目标系统的TCP连接之前，这两个系统需要被连接到相同的局域网。如果组织的IT部门不允许连接你的目标系统到组织的网络，那么你可能希望使用独立的路由器和开发系统的额外网络端口。

使用Eclipse目标浏览器

用目标浏览器，你可以从Eclipse工作区检查目标系统。你可以浏览目标的文件系统、列出运行中的进程以及终止它们和向它们附加调试器、创建应用启动配置。

目标浏览器使用TCF的发现机制，这是非常好用的，因为你不必手动找出你目标的IP地址[1]。TCF发现机制扫描你的局域网以寻找监听在

其默认端口1534上的TCF代理。遵循这些步骤来用目标浏览器建立到你目标的连接：

1.打开Target Explorer Perspective：从Eclipse Window菜单选择Open Perspective，并且从子菜单中选择Other...。从对话框中的透视图列表中，选择Target Explorer。Eclipse自动切换到Target Explorer透视图，在其左侧，有System Management标签。

2.System Management标签：System Management标签包含一个3文件夹列表：Favorites、Connections和Neighborhood。如果你从来没使用过Target Explorer，那么Favorites文件夹是空的，Connections文件夹有一个名为Create New Connection...的条目。然而，Neighborhood文件夹为发现机制已经在局域网上找到的每个TCF代理显示一个条目，条目的形式是TCF Agent<ip地址>。

3.建立连接：在你希望连接到的IP地址的TCF代理上右键单击，并且从菜单中选择Connect。Eclipse打开New Connection对话框。对话框的字段被用发现机制已经检测到的连接参数预先填充了。所有你需要做的是，给连接一个有意义的Connection Name（连接名字）。在你单击Finish后，Eclipse连接到目标上的TCF代理，并且为新连接打开了一个标签。

4.Connection标签：依赖于连接状态，Connection标签有多个显示在底部的子标签：

a.Details（Overview）：显示连接细节。仅仅当连接是关闭的时候，字段才可以被修改。

b.Source Paths：当附加调试器到进程时用于源文件的搜索路径。默认情况下，这些是目标上的路径，但是开发主机上的路径可以被增加。

c.Launches：允许启动配置的创建以运行安装在目标上的应用。标准输入（stdin）、标准输出（stdout）和标准错误（stderr）通过TCF被重定向到Eclipse以便你可以与运行中的应用交互。

d.Processes：运行在目标上的进程列表。当你单击这个子标签时，列表从目标上被获取下来。通过单击右上角的Refresh按钮，你可以再次加载它。

e.File System：让你浏览目标上的文件系统。你可以创建新的文件夹和文件，移动它们，删除它们，复制它们，重命名它们。你也可以搜索目标上的文件夹和文件。

如果你曾经用Eclipse开发过原生应用（也就是，可以被执行在开发系统本身上的应用），那么你知道，在编译了之后，你可以直接从Eclipse内的开发环境（例如C/C++或者Java透视图）中执行和调试它们。用TCF，你可以在目标上做相同的事情。它要求一些配置步骤，我们在接下来的部分解释这些步骤。

在目标上运行应用

通过Eclipse的Run Configurations，你可以设置用于执行应用的环境。从Eclipse的Run菜单中选择Run Configurations...，你启动了一个对话框，使用这个对话框，你可以创建和管理你的环境。在这个对话框的左侧，Eclipse显示一个带有各种运行配置类型的列表，在每个类型下面，是具有预定义的配置的列表。为了创建用于在远程目标上执行我们的C/C++应用的运行配置，你选择C/C++Remote Application，然后单击位于列表上面左上角的New按钮。

1.Name：基于当前的项目，Eclipse为你的运行配置填入名字。如果它满足你的需求，那么接受默认；否则输入新的名字。

2.主标签：

a.Connection：通过单击New...创建新连接。

i.从列表中选择TCF并单击Next>。

ii.在Host　Name字段中输入你的目标板的IP地址，或者，如果你有本地DNS，输入主机名。

iii.在字段Connection Name中为你的连接输入名字。

iv.如果你希望，那么输入一个Description（描述）。

v.点击Finish来创建连接和关闭对话框。

b.项目：Eclipse字段自动填入你当前的项目名字。如果那不是你希望的项目，使用Browse...按钮选择一个不同的项目。

c.Build　configuration（构建配置）：Eclipse自动选择项目的当前配置。通过从列表中选择一个不同的来覆盖它。

d.C/C++application：一个项目可能构建不止一个应用。使用Search Project...来选择正确的可执行文件。

e.Remote absolute file path for C/C++application（用于C/C++应用的远程绝对文件路径）：这个字段包含目标上应用的绝对路径和名字。Eclipse复制应用到那个路径。你可以选择一个不同于由Eclipse构建过程创建的可执行文件的名字。使用Browse...来选择在目标上的路径，然后追加一个名字，例如/usr/bin/hello。

f.Commands to execute before application（在应用前执行的命令）：在这个字段中，你可以输入在Eclipse启动应用前于目标上执行的额外命

令。把它留空。

g.Skip download to target path（略过下载到目标路径）：不要选中这个框。你希望Eclipse每次下载你的应用到目标以便你的最新变更被应用。

3.Arguments Tab（参数标签）：如果你的应用要求命令行参数，那么你可以输入它们到Program Arguments（程序参数）字段。

4.Common Tab（一般标签）：用于运行配置的一般设置是通过Common标签来访问的。在大部分情况下，默认设置是合适的。如果你希望Eclipse存储你应用的输出，选中File并且提供用于存储输出的文件的路径和名字。

先后单击对话框底部的Apply和Run来使你的应用在目标上启动起来。如果这是Eclipse第一次使用这个连接在那个目标上运行应用，那么它会展示给你一个对话框询问用于执行应用的用户ID和密码。输入root作为用户ID，密码保持为空。因为标准Yocto项目镜像仅仅有一个不带密码的root用户账号。

当Eclipse首先构建你的应用、然后传输它到目标以及最后执行它的时候，观察在Eclipse内的Console窗口。Console窗口应该显示类似如下的输出：

```
root@ypbkiosk:/#
echo $PWD'>'
/>
root@ypbkiosk:/# /usr/bin/hello;exit
Hello World!
logout
```

Eclipse和TCF仅仅把标准输入（stdin）、标准输出（stdout）和标准错误（stderr）从目标重定向到位于Eclipse内的终端。如果你的目标应

用是图形化应用，那么它的输出被展示在连接到目标硬件的屏幕上。简言之，这是有意义的，因为你将希望用本地屏幕来测试你的应用。

在目标上调试应用

和用于在目标上执行应用的Run Configurations（运行配置）类似，Eclipse提供了用于在目标上调试应用的Debug Configurations（调试配置）。为了调试，应用或者程序从调试器的上下文内被启动。然后调试器控制和监控程序的运行并且提供对程序使用的变量、栈、动态存储、文件指针等的访问。

远程调试需要这样一个调试器：它能够在目标上执行程序而同时通过例如网络连接或者串行连接的链路从不同的系统（开发主机）控制和监控它。GDB是用于Linux系统的标准调试器，有一个匹配的服务器（gdbserver）用于远程调试。在主机系统上运行的GDB和在目标系统上运行的gdbserver通过使用TCP的网络连接或者通过串行连接来通信。

理解用于远程调试的主要Eclipse工作流是有用的，因为它使用TCF和用于调试器的通信协议的组合：

1.Eclipse通过TCF下载应用到目标系统。

2.通过TCF利用类似gdbserver host：2345<program><arguments>的命令，Eclipse在目标系统上启动gdbserver和被调试的应用。参数host：2345表示使用在端口2345（gdbserver默认端口）上的TCP连接，<program>是Eclipse在前面步骤中下载的应用，如果提供了<arguments>，那么它们是传递到应用的命令行参数。

3.Eclipse在开发系统或者主机上启动GDB并且切换到调试透视图。

4.通过GDB/MI接口，Eclipse使用命令target remote<target>：2345指

示GDB连接到运行在目标上的gdbserver，其中<target>是目标的主机名（如果DNS是可用的）或者是IP地址。

5.GDB在目标上启动应用，并且把执行保持在main()函数的第一个指令上。

6.现在通过由Eclipse的调试透视图提供的图形用户界面，你可以用GDB控制和监控应用了。

GDB提供了多个命令解释器和命令基础设施，用户和其他应用可以通过它们与GDB交互：

·GDB/CLI：经常被用户使用的控制台或者命令行解释器。它以人可读的形式提供了简单的命令基础设施。这是默认的解释器。

·GDB/MI：通常被其他应用（例如Eclipse和其他GDB前端）使用的机器解释器。GDB/MI有两个版本——mi1和mi2。后者是当前版本。

所有GDB解释器使用stdin来接受命令，使用stdout用于信息和数据输出，使用stderr用于错误消息。Eclipse使用最新的GDB/MI版本，mi2。

创建调试配置并不比创建运行配置困难很多。事实上，如果你已经为你的应用创建了运行配置，那么Eclipse使用它作为起点。从Eclipse的Run菜单中选择Debug　Configurations...来打开配置编辑器。该对话框和Run Configurations对话框看起来几乎一模一样。在左侧，是应用类型的列表。如果你正如在前面部分所描述的那样创建了运行配置，那么Eclipse很可能已经在列表中高亮显示了那个配置并且正在右侧显示配置细节。

Main、Arguments、Common标签让你查看和编辑与Run

Configurations对话框中各自对应标签相同的信息。在前面部分，我们解释了它们。Debugger和Source标签是新的：

·Debugger Tab（调试标签）：在这里，你可以配置GDB：

·Stop on startup at：如果选中并且在它旁边的字段中提供了函数名，那么GDB在所提供函数的第一个指令处设置断点。在启动时，GDB执行程序直到它抵达断点。默认设置是在main()时暂停执行。如果你正在调试复杂程序并且你对程序在开始发生了什么不感兴趣，那么设置它为不同的函数。

·Main Subtab（主子标签）：用于GDB的一般设置：

·GDB Debugger：这个设置用于GDB调试器的路径和名字。这个信息应该已经被Yocto项目插件正确填写为指向到用于平台的交叉调试器。

·GDB Command File（GDB命令文件）：这个设置指向用于GDB的项目命令文件。调试器在启动时读取这个文件。文件名通常是.gdbinit并且位于Eclipse项目目录内。

·Non-stop Mode（不间断模式）：如果选中，这个设置允许你调试多线程程序的已停止线程而其他线程则正常运行。

·Enable Reverse Debugging（启用反向调试）：在这个设置被选中的情况下，你可以在程序中向后步进和继续。一般情况下，你仅仅可以按照程序流的顺序步进和继续。这个GDB功能性仍然被局限到特定架构和平台，并且对远程调试来说一般是不可用的。

·Force Thread List Update on Suspend（暂停时强制线程列表更新）：当线程遇到断点时，这个设置自动在Eclipse调试透视图中更新线

程信息。

·Automatically Debug Forked Processes（自动调试派生的进程）：通常，GDB调试器附加到主进程。如果那个进程创建子进程，那么GDB继续调试主进程。新版本的GDB可以附加到派生的进程。启用这个选项以使GDB也自动附加到子进程上。

·Tracepoint Mode（追踪点模式）：对于一些应用，特别是实时应用，设置断点和检查变量是不可行的，因为暂停程序修改了时序特征（timing characteristic）。在那种情况下，开发者可以使用追踪点来让GDB自动收集和报告数据。在一般模式下（Normal mode），追踪点作为陷阱被插入到程序中；在快速模式下（Fast mode），追踪点作为跳追踪点（jump tracepoint）被插入到程序中。不是在所有平台上和全部条件下都支持跳追踪点。自动模式（Automatic mode）把决定权留给调试器。

·Shared Libraries Subtab（共享库子标签）：在这个子标签中，你可以向文本框中增加目录路径到额外的共享库。标准共享库是已知的并且被默认加载。如果你希望调试共享库，那么你需要选中Load Shared Library Symbols Automatically（自动加载共享库符号）选项。

·Gdbserver Settings Subtab（Gdbserver设置子标签）：目标上的gdbserver可执行文件的名字和路径被显示在这个子标签中。默认是gdbserver，这对Yocto项目目标镜像是足够的，因为gdbserver位于默认PATH中。port number（端口号）是用于运行在主机上的GDB和运行在目标上的gdbserver之间通信的TCP端口。默认是2345。

·Source Tab（源标签）：当在程序中步进时，调试器需要显示源代码以便你可以有效地追踪程序。为了那个目的，调试器需要能够定位和加载源文件。增加到Source Lookup Path（源查找路径）列表中的路径服

务于那个目的。Eclipse默认增加项目路径。如果你需要调试任何共享库，那么你必须增加它们的路径到那个列表以能够看到它们的源。

通过单击Apply接受修改，然后通过单击Debug启动调试器。Eclipse启动调试器并且切换到调试透视图。执行在main()函数的第一个指令处暂停。现在，你可以在运行于目标上的程序中步进代码，设置断点并检查变量，正如对本地的运行在调试器中的程序所做的一样。

追踪库函数

你可以使用调试器追踪到共享库（例如C库libc）的函数。然而，调试器不能展示源代码，因为它没有关于从哪里定位源代码的信息。当构建SDK时，构建系统自动在系统根中为所有已安装的包（包括标准共享库）增加调试和源包。为了告诉调试器在哪里找到源文件，我们必须增加必要信息到位于Eclipse工作区的项目目录中的.gdbinit文件（程序清单11-5）。

程序清单11-5　GDB启动文件.gdbinit

```
set sysroot /opt/poky/2.0/sysroots/corei7-64-poky-linux
set substitute-path /usr/src/debug \
    /opt/poky/2.0/sysroots/corei7-64-poky-linux/usr/src/debug
```

程序清单11-5显示一个样例.gdbinit文件，它为GDB设置路径以为安装在/opt/poky/2.0的SDK定位用于库调试的源文件。

·set　sysroot：指定包含被安装在目标上的库的副本的目录。这个路径使调试器能定位和加载库以及它们的符号。这是到安装在你开发系统的Yocto项目SDK的路径。

·set　substitute-path：为调试器指定替换规则以找到用于库的源文件。为Linux编译的标准的可执行文件和库用/usr/src/debug作为路径前缀

记录它们的编译目录。substitute-path命令使得GDB在定位源文件时用第二个目录片段替换第一个目录片段。

不幸的是，Eclipse不提供任何方法来赋予GDB配置设置，例如但是不限于在前面所显示的用于交叉调试的配置设置。然而，你可以使用Eclipse的文本编辑器修改.gdbinit文件。因为该文件是Linux/UNIX系统上的隐藏文件，所以默认情况下Eclipse Project Explorer不显示它。在Project Explorer旁边的菜单中单击向下的箭头，然后从菜单中选择Customize View...。然后移除.*resources旁边的选择标记。替代性地，你可以在Debug Configuration对话框中给予GDB Command File不同的文件名以使它不再是隐藏文件（文件名前没有逗号）。

[1] 并不是说在目标系统上运行ifconfig是非常困难的，而是因为它需要串行终端或者屏幕和键盘。

11.5 使用模拟目标的应用开发

很多嵌入式项目是硬件和软件工程共同努力的成果。实际的目标硬件，或者甚至早期工程样品，对软件开发来说，可能不能很好地就绪于开发周期中。为了缩短开发周期，与硬件开发并行，你可能希望在项目早期就开始在应用软件上开发软件。一种方法是使用例如BeagleBone、MinnowBoard Max、Wandboard以及其他类似于你项目的目标硬件的廉价开发板。另一种方法是使用目标模拟。目标模拟也有这样的优点——应用开发者不必处理嵌入式硬件，因为他们可以使用自己的开发系统构建和测试应用。

Yocto项目使用QEMU用于目标模拟。我们已经在全书中使用QEMU用于测试各种Yocto项目系统构建。现在，我们举例说明如何以Yocto项目SDK和Eclipse使用QEMU用于应用开发。

11.5.1 为用QEMU进行应用开发做准备

我们需要两个东西用于用模拟目标进行应用开发：

·用于QEMU的Linux内核和根文件系统

·匹配前者的ADT

当然，我们用构建系统构建这两者。我们已经有了构建环境，我们曾使用它来为用在MinnowBoard Max上的亭项目创建过镜像。我们仅仅需要把机器从ypbkiosk改到QEMU机器。使用其核心架构类似于实际硬件目标的模拟机器是很有意义的。因为MinnowBoard Max是配备了64位x86 CPU，所以我们选择qemux86-64作为机器。把构建环境的conf/local.conf的MACHINE变量改成qemux86-64，然后用以下命令启动BitBake：

```
$ bitbake -k ypbkiosk-image-sato
```

来构建Linux内核和ypbkiosk-image-sato根文件系统镜像。一旦构建完成，用以下命令创建匹配的SDK：

```
$ bitbake -c populate_sdk ypbkiosk-image-sato
```

一旦那完成了，则安装ADT，正如在11.2.2小节中所解释的。你可以安全地把ADT安装在相同的/opt/poky/<version>目录，因为工具链、系统根和启动脚本具有唯一名字并且是互相分开的。

解压根文件系统

Eclipse用从开发主机上通过网络文件系统（Network File System，

NFS）导出的根文件系统启动QEMU。为了用QEMU和NFS设置根文件系统，我们需要解压和准备在前面步骤中已创建的根文件系统。ADT包含一个执行所有必要工作的脚本。

1.改变目录到ADT安装目录：

```
$ cd /opt/poky/<version>
```

2.引用QEMU ADT环境：

```
$ source environment-setup-core2-64-poky-linux
```

3.改变目录到sysroots：

```
$ cd sysroots
```

4.从构建环境解压根文件系统：

```
$ runqemu-extract-sdk \
  /<path-to-build-environment>/tmp/deploy/images/qemux86-64/\
  ypbkiosk-image-sato-qemux86-64.tar.bz2 \
  core2-64-poky-linux
```

5.从构建环境复制Linux内核镜像：

```
$ cp /<path-to-build-envrionment>/bzImage-qemux86-64.bin \
  core2-64-poky-linux/boot
```

对NFS来说，你的开发系统必须要有rpcbind——通用地址到远程过程调用程序数字映射器（RPC Program Number Mapper）均已被安装。依赖于你的系统配置，你可能需要使用系统包管理器来安装它，例如，在Ubuntu上使用sudo apt-get install rpcbind。而且，为了使用用户空间NFS，rpcbind需要运行在非安全模式，允许来自任何主机的SET和UNSET。为了启用非安全模式，你需要增加-i选项到rpcbind启动配置。

在Ubuntu上，增加-i到文件/etc/init.d/rpcbind中的OPTIONS变量并且用命令sudo service rpcbind restart重启rpcbind。

与Eclipse集成

现在我们需要把ADT和解压用于QEMU的根文件系统与Eclipse集成。

从Eclipse Window菜单中打开Preferences对话框，单击Yocto Project ADT，并且遵循这些指令来创建用于用QEMU进行应用开发的新的交叉开发配置文件：

1.Cross Development Profiles（交叉开发配置文件）：交叉开发配置文件列表中的Standard Profile（标准配置文件）应该反映我们在前面章节中用于MinnowBoard Max的设置。保存Standard Profile为MinnowBoard Max，然后把选择改回Standard Profile。

2.Cross Compiler Options（交叉编译选项）：保持用于Standalone Pre-built Toolchain（独立预构建工具链）和Toolchain Root Location（工具链根位置）。点击Sysroot Location（系统根位置）旁的Browse...（浏览），浏览到/opt/poky/<version>/sysroots/core2-64-poky-linux，这是我们解压根文件系统所到的位置。从Target Architecture（目标架构）列表中，选择core2-64-poky-linux。

3.Target Options（目标选项）：选择QEMU，并且浏览到我们将Linux内核复制到的位置：/opt/poky/<version>/sysroots/core2-64-poky-linux/boot/bzImage-qemux86-64.bin。

现在Eclipse已经准备好用QEMU进行应用开发。

11.5.2 构建应用并在QEMU中启用它

现在你可以按照11.4.3小节中所描述的那样创建新应用项目。创建了项目后，在Project Explorer中右键单击它的名字，并选择位于列表底部的Yocto Project Settings。从Cross Development Profiles，选择QEMU配置文件，单击Apply。你的项目现在使用用于QEMU的交叉工具链来编译和调试。

为了执行和调试你的应用，你首先需要启动QEMU。在Eclipse的Run菜单中选择External Tools。子菜单的第一个条目是我们刚刚创建的QEMU集成：qemu_core2-64-poky-linux。在其上单击，Eclipse启动终端窗口，它从该窗口内启动QEMU。QEMU本身在第2个窗口中启动。

借助TCF和GDB/gdbserver，使用运行配置和调试配置在模拟的目标上部署、执行和调试你的应用，正如在11.4.4小节中所解释的。

11.6 总结

操作系统栈和中间件组成设备的基础。应用和用户软件为最终用户创造价值。Yocto项目应用开发工具包提供给应用开发者用以为他们的目标设备构建、测试和部署应用的必要工具。

·Yocto项目ADT包含交叉开发工具链、用于硬件目标设备和模拟的目标的系统根、QEMU模拟器、测试和分析工具以及便于开发环境设置的集成脚本。

·通过包含用于构建设备的Linux操作系统栈的一模一样的工具链和通过打包设备的根文件系统镜像，构建系统创建匹配目标设备的ADT。

·为了传统开发以及与构建工具的集成，ADT交叉开发工具可以直接从命令行使用。设置仅仅需要引用设置必要环境变量的脚本。

·ADT和Eclipse IDE的集成从图形用户界面的便利性方面提供了对交叉开发工具的访问。

·Eclipse的目标通信框架（TCF）允许直接从IDE对实际目标硬件以及模拟的目标进行部署、执行和调试。

11.7 参考文献

Yocto项目应用开发者指南，www.yoctoproject.org/docs/2.0/adt-manual/adt-manual.html。

Yocto项目开发手册，www.yoctoproject.org/docs/1.8/dev-manual/dev-manual.html。

第12章　许可和合规

功能齐全的Linux操作系统栈由几百个，也可能是几千个开源软件包组成。这些软件包是由它们的作者以众多开源许可的条款和条件来发布的。几乎所有这些许可都要求最终用户被赋予对许可文本的访问以把使用软件时的权利和可能的义务通知给他们。作为系统构建者，你正在构建由很多不同开源软件包所组成的完整系统，你需要收集所有许可信息并且将其提供给你系统的用户。除了提供许可信息，一些开源许可，特别是GNU通用公共许可（GNU General Public License，GPL），要求你提供用于构建软件包二进制文件的源代码。收集、管理和提供许可信息及源代码可能是耗时的任务。Yocto项目通过提供一组工具来极大地简化了这个任务，这些工具会处理开源许可和源代码管理中单调乏味的部分。

12.1 管理许可

对于任何软件产品，在其中包含最终用户许可协议（End-User License Agreement，EULA）是常见的实践，该协议把使用产品时的权利和义务通知给用户。如果包含在产品中的所有代码是完全由交付产品的组织或者个人开发的，那么这个过程是简单且直接的。只要在产品中提供和包含专有许可协议并且让客户在计算机系统上安装软件时接受协议即可。对用于个人计算机的任何软件来说，这已经成为标准实践。然而，事情可能会极大的不同，如果：

·软件产品包含来自其他提供者的软件组件（例如库）。

·软件产品是使用开源软件包来构建的。

·软件产品是作为嵌入式系统的一部分而与硬件一起提供的。

对用于嵌入式设备的Linux操作系统栈来说，所有以上的情况都符合。虽然不要求最终用户显式地接受用于包含在产品中的软件包的开源许可，但必须让他们明白，产品是用开源软件来构建的。必须使许可文本和关于如何获取源代码的信息是对最终用户可用的。

许可和源代码信息可以被包含在设备中或者通过其他方式提供。例如，在Android设备上，你可以从Settings应用的About Device菜单的Legal Information项中访问许可信息。许可信息和文本可以存储在设备上，或者替代性地，特别是对联网设备，它们可以从超链接访问到，超链接把最终用户引导到许可信息被显示的网站。对具有至少能够显示文本的用户界面的设备来说，那是方便的。对没有这种能力的设备，许可信息可以借助用户文档在制造者的网站等位置提供。

管理用于你产品的许可不是小任务。Yocto项目包含超过170个常见

许可方案。它们中的大部分是开源许可，但也有一些是用于某些软件包的商业许可。一些开源软件的开发者使用他们自己的许可，这进一步复杂化了情况，因为它们可能是不合法的。开源组织（Open Source Initiative，OSI）在许可评审过程中检查许可与开源定义[1]的合规性。OSI在其网站上列出了大约70个开源许可，这些许可通过了该组织的许可评审过程并且被批准为符合开源定义[2]。

更进一步复杂化问题的是，单一开源项目可能实际上使用多个许可方案。以下是常见的例子：

·对一些库来说，一种许可方案适用于库的源代码和任何从其衍生的工作，例如缺陷修复和加强；另一种许可方案适用于由其他软件组件通过其API对库的使用。例如，GnuTLS使用GPLv3+用于源代码，而使用LGPLv2.1+用于其他组件对库的使用。

·由多个分别由不同方案许可的组件组成的包。这种方案经常在提供插件机制的包中找到。一个例子是媒体框架，在那里，用于编码器和解码器的插件可能和框架本身以及其他编码器和解码器的许可不同，例如自由无损音频编解码（Free Lossless Audio Codec，FLAC）。

Yocto项目在几个方面辅助管理许可：

·许可追踪

·通用许可

·商业许可的包

·许可部署

在接下来的部分，我们将仔细研究这些。

[1] http://opensource.org/osd。
[2] http://opensource.org/licenses/alphabetical。

12.1.1 许可追踪

所有菜谱都必须将LICENSE变量设置为适用于菜谱构建的软件包的源许可列表。用于软件包本身的许可信息几乎在所有情况下都与用于菜谱的许可不同。后者通常是在层的LICENSE文件中指定的。你一定不能混淆这两者。

LICENSE变量可能包含一个单一的许可指派，或者，如果软件包是由多个许可所覆盖的，那么LICENSE变量包含多个许可指派的列表：

·如果许可是可供选择的，那么用管道符（|）来分隔许可指派：例如，LICENSE="LGPLv2.1+|GPLv3"。

·如果多个许可覆盖包源的不同部分，那么用&符号分隔许可指派：例如，LICENSE="MPLv2&LGPLv2.1"。

许可指派可以是任何文本字符串但是不能包含任何空格字符串。对标准许可，使用在meta/files/common-licenses中的通用许可文件的名字或者使用定义在meta/conf/licenses.conf中的软件包数据交换（Software Package Data Exchange，SPDX）许可标志作为许可指派。SPDX[1]是用于许可信息的标准格式，它是由Linux基金会的SPDX工作组创建和维护的。

除非LICENSE被设置成特殊许可指派
CLOSED（LICENSE="CLOSED"），否则菜谱也必须设置变量
LIC_FILES_CHKSUM以启用许可追踪。

软件包的作者或者著作权持有人可能改变许可本身，例如从GPLv2改为GPLv3，或者可能把许可文本从软件包的一个版本改成下一个版本

[2]。构建系统可以追踪这样的变化并且显示通知以便作为系统构建者的你可以选择是否希望采用具有更新的许可的新的包版本，这是重要的。LIC_FILES_CHKSUM变量，它是由license类判断的，并且提供了用于追踪许可变化的灵活机制。下面的例子显示了在菜谱中指定LIC_FILES_CHKSUM的各种方法：

```
LIC_FILES_CHKSUM = "file://COPYING;md5=wwww \
                    file://header.h;beginline=7;endline=34;md5=xxxx \
                    file://source.c;beginline=10;md5=yyyy \
                    file://license3.txt;endline=46;md5=zzzz"
```

变量包含空格分隔的许可文件列表，许可文件包含许可文本。构建系统对许可文本计算一个MD5校验和，并且把它和由md5参数提供的值作比较。如果参数beginline和参数endline都没有被提供，那么整个文件都被视为许可文本。这个方式对通常被命名为COPYING或者LICENSE的独立许可文件是好用的。有时，许可信息是作为源文件（例如C头文件或者类似文件）的一部分来提供的。在这种情况下，实际上只有文件内容的一个部分代表许可信息。你可以通过分别在许可文本开始处的行号上设置参数beginline和在许可文本结束处的行号上设置参数endline来指定那个部分。

构建系统默认在由变量S指定的目录中搜索许可文件，这个目录是源代码被解压的目录。你可以通过增加目录路径来显式地提供目录信息：

```
LIC_FILES_CHKSUM = "file://src/header.h;beginline=7;endline=34;md5=xxxx \
                    file://${WORKDIR}/license.txt;md5=yyyy"
```

第一行在相对于S的${S}/src中定位包含许可信息的文件header.h，而第二行参考变量WORKDIR来定位文件license.txt。

[1] https://spdx.org/。

[2] 事实上，很多软件包的更新版本，特别是那些由GNU提供的软件包

（其中那些早期版本是以GPLv2的条款和条件许可的），现在是以更新的GPLv3来许可的。

12.1.2 通用许可

OpenEmbedded核心元数据层在目录meta/files/common-licenses中包含用于通用许可的文件和许可文本。这个目录是由变量COMMON_LICENSE_DIR所引用的。你可以把这个变量和通用许可文件名用于LIC_FILES_CHKSUM，例如：

```
LIC_FILES_CHKSUM = "\
file://${COMMON_LICENSE_DIR}/GPL-2.0;md5=801f80980d171dd6425610833a22dbe6"
```

然而，我们推荐仅仅在源包实际上不包含许可文本而仅仅包含到文件中通用许可的引用时才这样做。在这种情况下，我们也推荐你包含含有对LIC_FILES_CHKSUM引用的文件。理由是，仅仅使用通用许可文件实际上禁用了许可追踪机制。包源内的许可信息变更可能不会被构建系统注意到，因为菜谱使用了通用许可文件。

通过增加一个或者多个你自己的许可目录到LICENSE_PATH变量，你可以为构建系统提供用于搜索的目录：

```
LICENSE_PATH += "/path/to/my/licenses"
```

这个变量可能包含由空格分隔的目录列表。构建系统增加变量的路径到由COMMON_LICENSE_DIR提供的路径。

12.1.3 商业许可的包

一些软件包，虽然是开源的，但却是使用商业许可或者不符合开源许可的特殊许可条款和条件来许可的。构建这种包的菜谱通过设置变量LICENSE_FLAGS来标志特殊许可需求：例如，LICENSE_FLAGS="commercial"。

LICENSE_FLAGS变量可以包含任何字符串，但是一旦菜谱设置了变量，构建系统不会构建软件包，除非它是被显式地启用的。

为了启用特殊的许可标志，你增加它到LICENSE_FLAGS_WHITELIST变量。变量包含空格分隔的许可标志列表。构建系统把由菜谱设置的许可标志和LICENSE_FLAGS_WHITELIST变量中的许可标志列表进行匹配。然而，在这样做之前，它把包名PN追加到定义在菜谱的LICENSE_FLAGS中的许可标志。例如，对于构建hello包包含如下的菜谱：

```
LICENSE_FLAGS = "commercial"
```

变量实际上成为：

```
LICENSE_FLAGS = "commercial_hello"
```

通过设置

```
LICENSE_FLAGS_WHITELIST = "commercial_hello"
```

你现在可以特殊启用hello菜谱或者通过设置

```
LICENSE_FLAGS_WHITELIST = "commercial"
```

来启用所有使用商业许可标志的菜谱，包括hello。

这种匹配方案可以让你精确控制要启用构建的包。通过在你菜谱的LICENSE_FLAGS中显式地指定包名PN和包版本PV，你可以更进一步控制。考虑构建hello包的菜谱包含：

```
LICENSE_FLAGS = "commercial_${PN}_${PV}"
```

这使得你可以仅仅启用一个特定版本的hello菜谱，例如版本1.0，通过使用以下命令：

```
LICENSE_FLAGS_WHITELIST = "commercial_hello_1.0"
```

你在构建环境的conf/local.conf中指定LICENSE_FLAGS_WHITELIST，或者甚至更好，在例如定制层的conf/distro/mydistro.conf的发行版配置文件中指定。

12.1.4 许可部署

当构建目标时，构建系统把许可信息放在构建环境内的${TMPDIR}/deploy/licenses目录中。对每个构建系统构建的菜谱，它在${TMPDIR}/deploy/licenses中创建一个子目录：

·如果菜谱构建软件包，那么子目录的名字是菜谱的名字，并且包含许可文件。

·如果菜谱构建镜像目标，例如core-image-minimal，那么子目录的名字是镜像菜谱的名字并带有一个增加的时间戳，例如，core-image-minimal-20150817224402。目录包含两个文件：package.manifest和license.manifest。前者是按字母顺序排列的、包含在镜像中的所有包名的列表。后者是按字母顺序排列的相同的包并带有包名、包版本、菜谱名字以及许可细节的列表。

·如果菜谱是用于包组的，那么构建系统以包组的名字创建子目录，但是在子目录中没有文件。

如果你希望为你的目标系统部署许可信息到根文件系统镜像中，那么

·设置变量COPY_LIC_MANIFEST="1"复制文件license.manifest到根文件系统中，直到/usr/share/common-licenses/license.manifest。

·设置变量COPY_LIC_DIRS="1"复制许可目录到/usr/share/common-licenses。

你在构建环境的conf/local.conf中指定这些变量，或者甚至更好的是，在例如定制层的conf/distro/mydistro.conf的发行版配置文件中指定。

12.1.5 黑名单许可

通过将变量INCOMPATIBLE_LICENSE设置为空格分隔的许可指派的列表，你可以把菜谱从构建中排除。对于那些它们的各自菜谱不提供对列表中的许可的替代性许可包，构建系统不构建它们。例如，设置

```
INCOMPATIBLE_LICENSE = "GPL-3.0 LGPL-3.0 AGPL-3.0"
```

有效地把所有以这些许可授权的包从构建中排除，除非有许可替代，它可能是使用不同许可的早期版本或者提供类似功能性的包。前面的例子代表了Yocto项目团队用其来测试这个功能性的设置。虽然你可以使用其他设置，但是你可能需要通过移除它们或者提供替代来自己处理依赖性以生产功能系统。

12.1.6 提供许可程序清单和文本

对开源许可的一个常见需求是，你必须提供许可信息。使用

```
COPY_LIC_MANIFEST = "1"
```

复制许可程序清单到目标镜像的目录/usr/share/common-licenses/license.manifest中。许可程序清单包含已安装在目标上的所有开源软件包及其各自许可的列表。

使用

```
COPY_LIC_DIRS = "1"
```

也复制用于已安装在目标上的所有包的许可文本到镜像目标的目录/usr/share/common-licenses中。COPY_LIC_DIRS仅仅可以和COPY_LIC_MANIFEST联合使用；否则，设置变量没有效果。

对具有能够展示许可程序清单和许可文本的用户界面的目标系统，把它们包含在目标镜像中极大地简化了这部分开源许可的合规性。如果你的嵌入式目标没有用户界面或者没有足够的存储空间来保存许可程序清单和文本，那么你需要以替代的方法提供信息，例如通过打印出的信息或者在你组织网站上的页面。

12.2 管理源代码

一些开源许可明确要求你向系统的最终用户提供你用来为目标系统构建软件栈的源代码。提供源代码是你必须执行的合规管理活动的一部分，并且在你为目标系统创建最终镜像之前，你应该考虑必要的任务。

提供完整下载目录DL_DIR是最简单的提供源代码的方法。然而，这种方法有一些问题：

·完整下载目录的大小可能是相当大的。它也包含那些你通常不部署在发布镜像中的包的源文件，例如工具链源文件。在下载目录中也有一些你永远不会部署的文件，也就是*.done文件，它们表示源包是否已被成功下载了。而且，下载目录包含子目录，子目录中包含用于源文件的未解压的仓库，这些仓库是直接从源控制管理系统中获取的。

·下载目录不包含任何用菜谱提供的补丁。为了提供最终用户可以用来构建二进制的源文件，你可能需要手动包含那些补丁。

·最重要的是，下载目录页包含用于闭源或专有软件包的源包和仓库，而你不希望发布这些给最终用户。

一个更好的提供源代码的方法是，通过archiver类来提供，这给了你对希望提供什么源代码以及用什么形式来提供它的控制的灵活性。通过增加archiver类到构建环境的conf/local.conf中的INHERIT变量，你启用使用archiver类的源打包：

```
INHERIT += "archiver"
```

通过设置一些变量和变量标志，你控制archiver类做什么以及如何做它的工作：

·ARCHIVER_MODE[src]是控制源代码如何被打包的标志。

·ARCHIVER_MODE[src]="original"分别打包源包和补丁。

·ARCHIVER_MODE[src]="patched"打包应用了补丁的源包。这是默认设置。

·ARCHIVER_MODE[src]="configured"打包应用和配置了补丁的源包。

·ARCHIVER_MODE[diff]="1"在do_unpack和do_patch任务之间打包补丁。

·ARCHIVER_MODE[diff-exclude]是空格分隔的、你希望从ARCHIVER_MODE[diff]中排除补丁的文件和目录的列表。默认设置是ARCHIVER_MODE[diff-exclude]=".pc autom4te.cache patches"。

·ARCHIVER_MODE[dumpdata]="1"包含一个文件，它包含用于特定包的环境数据，类似于bitbake-e<recipe>。默认设置是ARCHIVER_MODE[dumpdata]="0"。

·ARCHIVER_MODE[recipe]="1"包含菜谱（包含任何.bbappend在内的.bb文件）以及任何包含文件。默认设置是ARCHIVER_MODE[recipe]="0"。

·ARCHIVER_MODE[srpm]="1"除了压缩的tar包，它也以源RPM（SRPM）文件输出源文件。ARCHIVER_MODE[src]标志也适用于SRPM。默认设置是ARCHIVER_MODE[srpm]="0"。

除了各种ARCHIVER_MODE标志，archiver类允许过滤出包源是为哪些许可打包的以及打包的菜谱类型是什么：

·COPYLEFT_LICENSE_INCLUDE是空格分隔许可列表，源代码是为其打包的。用于许可指派的通配符是允许的。例如，COPYLEFT_LICENSE_INCLUDE="GPL*LGPL*"包含用于所有以任何版本的GPL和LGPL授权的包的源文件。默认设置是COPYLEFT_LICENSE_INCLUDE=""。

·COPYLEFT_LICENSE_EXCLUDE是空格分隔的许可列表，它们是显式地从源打包中排除的。例如，COPYLEFT_LICENSE_EXCLUDE="CLOSEDProprietary"显式地把所有用闭源许可或者专用许可授权的软件包排除在源打包之外。默认设置是COPYLEFT_LICENSE_EXCLUDE=""。

·COPYLEFT_TARGET_TYPES是空格分隔的菜谱类型列表，对这些菜谱类型，源打包是启用的。可能的菜谱类型是：

·target：打包为目标构建的所有包的源文件

·native：打包为构建主机构建的所有包的源文件

·nativesdk：打包为主机SDK构建的所有包的源文件

·cross：打包用于所有交叉构建包的源文件

·crosssdk：打包用于所有交叉构建SDK包的源文件

·cross-canadian：打包用于所有cross-canadian包的源文件

默认设置是包含所有以上菜谱类型。

默认情况下，COPYLEFT_LICENSE_INCLUDE和COPYLEFT_LICENSE_EXCLUDE都被设置成空字符串，archiver类不执行任何许可过滤。最小情况下，你可能希望设置

COPYLEFT_LICENSE_EXCLUDE来排除任何你自己的专有软件包。我们推荐把它设置在一个定制层的发行版策略文件（例如conf/distro/mydistro.conf）中，而不是设置在构建环境的conf/local.conf文件中，这样一来，在构建你的目标发行版时，启用archiver类将默认排除你的专有软件包。

为了进一步限制源部署的大小，考虑在conf/local.conf文件或者发行版策略配置文件中设置COPYLEFT_TARGET_TYPES="target"。

archiver类复制它创建的源包输出到${TMPDIR}/deploy/sources目录。它在子目录中先按照架构然后按照包名组织包。通过设置变量DEPLOY_DIR_SRC，你可以覆盖默认输出目录。

12.3 总结

按照开源软件包各自许可的许可要求来进行合规管理是系统构建者必须要执行的一项重要任务。整体来说，因为用于嵌入式设备和软件产品中的Linux和开源日益增加的流行度，例如自由软件基金会[1]（Software Freedom Conservancy）和自由软件管理机构[2]（Software Freedom Conservancy）的多个组织正在积极监控提供包含开源软件的产品的公司是否满足各种开源许可（特别是GPL）的要求。如果这些组织发现一个公司没有遵守许可要求，那么他们将要求其弥补这个情况。如果一个公司不做任何措施，这些组织可能代表相关软件包的作者来起诉该公司。

尽管开源码许可是宽容的，但它们是具有法律约束力的文件，因此可以由法律强制执行。最好遵守合规要求来避免任何问题，并且在交付产品前计划合适的许可文本和源代码的发布。

在本章中，我们讨论了：

·许可管理：通过用菜谱追踪许可，创建商业许可的包及其各自菜谱，以及收集和部署许可信息。

·用airchiver类实现的源代码管理，这提供了基于许可和菜谱类型控制源代码打包的便利方法。

[1] www.fsf.org。
[2] https://sfconservancy.org。

12.4 参考文献

Yocto项目开发手册，www.yoctoproject.org/docs/1.8/dev-manual/dev-manual.html。

第13章　高级主题

在本章中，我们讨论精选的主题，它们促进在团队或者生产环境中使用Yocto项目。Yocto项目构建系统的一个强大之处是，它可以轻松部署到软件和构建工程师的个人开发系统上。本章中讨论的特性会扩展构建系统的能力，使得你共享资源和追踪构建结果，这对团队开发和在生产环境中要求再现性（reproducibility）、一致性和可重复性来说是必要的。

13.1 Toaster

Toaster是构建系统的一个图形用户界面。不像Hob，Hob是原生应用，Toaster是通过Web浏览器访问的Web界面。因为它是Web界面，所以Toaster适合部署在构建集群和云服务中的远程系统上。

与Hob相似，Toaster允许你配置、启动和监控构建。Toaster也提供搜索以及向你的构建增加元数据层和菜谱的功能性。自从Yocto项目版本1.8 Fido开始，Hob已经被Yocto项目团队官方弃用了，并且开发也已经停止，取而代之的是Toaster。

用Toaster工作要比用Hob工作稍微复杂一点儿，因为它要求用于Web服务器环境的额外设置。Toaster使用Django[1]Web框架。Django是完全用Python编写的高层级Web框架。Django利用对象关系映射（Object-Relational Mapping，ORM）用于数据库支撑的对象存储。Toaster使用ORM在数据库中存储构建统计和其他数据，这允许在后续构建间容易的比较。数据库系统可以是任何Django支持的关系型数据库管理系统（Relational Database Management System，RDBMS）。默认情况下，对于本地使用，Toaster为了简单性而使用SQLite。对于在远程构建服务器上的部署，我们推荐使用实际的关系型数据库管理系统，例如MySQL、MariaDB或者PostgreSQL。

在接下来的部分，我们讨论两种Toaster操作模式，两种模式下本地开发的设置，Toaster配置以及用Toaster设置一个生产构建系统。

[1] https://www.djangoproject.com。

13.1.1 Toaster操作模式

你可以以分析模式或者构建模式两者之一来运行Toaster。

分析模式

在分析模式中，Toaster附加到你以前用oe-init-build-env构建的现有构建环境上。在这个模式中，你直接使用bitbake命令开始构建镜像。然后Toaster收集构建统计和其他信息，把它们存储在数据库中，并且使得它们可用于在Web界面中浏览和查看。在通过BitBake启动你的构建之前，你需要首先启动Toaster。

在分析模式中，Toaster提供以下的功能性：

·包括菜谱和包在内的关于构建的镜像的细节信息。

·关于什么包已经被安装进镜像中的程序清单。

·浏览镜像目录结构的能力。

·包括变量设置在内的构建配置。

·检查错误、警告和日志消息以促进调试。

·关于任务执行和共享状态使用的信息。

·依赖性浏览器。

·例如构建时间、每个任务的时间、CPU使用率等的性能信息。

构建模式

在构建模式中，Toaster创建构建环境并且管理配置、BitBake执行以及分析模式下的数据收集和分析任务。你仅仅通过Toaster的Web界面与其交互。从Toaster界面中，你可以选择镜像，配置目标机器和构建的其他方面，并且启动构建。你不直接和BitBake交互，而是当Toaster在分析模式时，你直接和BitBake交互。

在构建模式中，除了分析模式的功能性以外，Toaster也提供了以下功能性：

·浏览层并且添加层到构建配置。

·选择目标镜像、目标机器和发行版策略。

·检查和设置配置变量。

·执行构建。

在构建模式中，Toaster提供与Hob不相上下的配置和执行功能性。用Toaster通过向镜像中增加包来配置镜像不像Hob那样方便，因为它要求你从Toaster用户界面中编辑IMAGE_INSTALL_append变量。使用Hob，它仅仅是选中多选框。与Hob相比，Toaster提供了更详细的构建分析和统计。

13.1.2 Toaster设置

Toaster是基于Django Web框架的，因此要求安装一些额外的Python包，然后你才可以使用它。你可以在已经安装了构建系统的根目录（例如/home/myname/poky）的bitbake/toaster-requirements.txt文件中找到包的列表。可以直接使用这个文件来安装需要的Python包。

你可以直接安装那些包到构建主机的Python库目录中，或者你可以使用Python虚拟环境（virtual environment）。Python虚拟环境基于构建主机上的原始Python设置来创建Python沙盒。如果你对冲突的版本有所顾虑，那么这是特别有用的。我们强烈推荐使用Python虚拟环境。

设置Python虚拟环境

你用virtualenv命令设置Python虚拟环境，所有Linux发行版作为一个包来提供virtualenv命令。如果它还没有安装在你的构建主机上，那么对Fedora或者Red Hat系统，你可以使用以下命令来安装它：

```
$ sudo dnf install python-virtualenv
```

或者对Ubuntu系统，使用：

```
$ sudo apt-get install python-virtualenv
```

然后你可以用以下命令创建Python虚拟环境：

```
$ virtualenv pvenv
```

命令在当前位置的名为pvenv的目录中创建新的Python虚拟环境。然后它把所有必要文件从构建主机的Python环境中复制到新的虚拟环境

中。在构建主机上创建Python虚拟环境的位置是无关紧要的。

在你可以使用Python虚拟环境之前，你必须用以下命令来激活它：

```
$ source pvenv/bin/activate
(pvenv) $
```

为了标示你正在工作于Python虚拟环境，你的命令行提示被前缀以在括号内的虚拟环境的名字，在我们例子中是（pvenv）。

你必须从活动的Python虚拟环境内执行所有与Toaster相关的操作，包括所需Python包的安装；否则，你的构建主机使用它默认的Python环境。

当在Python虚拟环境中时，你输入以下命令退出Python虚拟环境：

```
(pvenv) $ deactivate
```

安装Toaster需求

从Python虚拟环境内，执行：

```
(pvenv) $ pip install -r bitbake/toaster-requirements.txt
```

把需要的Python包（包括Django）安装进Python虚拟环境中。

现在你的构建主机已经准备好用于本地Toaster开发了。

13.1.3 本地Toaster开发

在本地开发模式中，Toaster使用Django内置的Web服务器而不是与外部Web服务器集成；它使用SQLite而不是RDBMS，这极大地简化了安装和配置。然而，对于工作组使用和远程部署来说，它不能伸缩。对于可伸缩的部署来说，你应该考虑使用Toaster生产设置，正如在13.1.5小节中所描述的。

构建模式下的本地Toaster开发

在活动的Python虚拟环境内，改变到你的构建系统安装的根目录中，例如，

```
(pvenv) $ cd /home/myname/poky
```

然后，用如下命令启动Toaster：

```
(pvenv) $ bitbake/bin/toaster
```

如果这是你第一次启动Toaster，那么它初始化数据库，读取层和构建系统配置，并且执行一系列其他初始设置任务。一旦设置完成，Toaster打印出：

```
Starting webserver...
Webserver address: http://0.0.0.0:8000/
Starting browser...
Toaster is now running. You can stop it with Ctrl-C
```

你的构建主机的默认Web浏览器应该自动启动了，并且显示Toaster登录页面。如果Web浏览器没有自动启动，那么打开你喜欢的Web浏览器，并且在导航栏中输入：

```
http://localhost:8000
```

现在你可以创建Toaster项目、配置它并且构建它。

分析模式下的本地Toaster开发

如果你希望使用Toaster用于现有的构建环境并且通过直接调用BitBake来控制构建过程，那么首先从Python虚拟环境内引用构建环境：

```
(pvenv) $ source /home/myname/poky/oe-init-build-env tbuild
```

然后，首先从Python虚拟环境内启动Toaster：

```
(pvenv) $ source /home/myname/poky/bitbake/bin/toaster
```

现在你可以像往常一样运行BitBake，例如：

```
(pvenv) $ bitbake -k core-image-base
```

为了监控构建中的过程和在构建已经完成后查看构建统计等，把你的浏览器指向到：

```
http://localhost:8000
```

从Python虚拟环境内输入以下命令来停止Toaster：

```
(pvenv) $ source /home/myname/poky/bitbake/bin/toaster stop
```

这终止所有Toaster进程。

13.1.4 Toaster配置

Toaster配置可以通过命令行选项、环境变量和Django管理用户界面来配置和管理。

设置不同端口

默认情况下，Toaster监听在构建主机的所有网络接口的8000端口。为了改变端口，在构建模式下用webport参数启动Toaster，如下所示：

```
(pvenv) $ /home/myname/poky/bitbake/bin/toaster webport=5000
```

替代性地，你可以在分析模式下用webport参数启动Toaster：

```
(pvenv) $ source /home/myname/poky/bitbake/bin/toaster webport=5000
```

然后，Toaster监听所有网络接口上提供的端口[1]。

设置用于构建模式的Toaster目录

在构建模式下，Toaster在由环境变量TOASTER_DIR定义的目录中存储构建环境和来自远程仓库的额外层的克隆。在那个目录内，Toaster创建目录build，它包含构建环境；创建目录_toaster_clones，它包含克隆的层；创建数据库toaster.sqlite，它存储配置和构建数据。默认情况下，TOASTER_DIR被设置成当前目录（Toaster是从这里启动的）。

从不同的目录启动Toaster是设置Toaster目录的唯一方法。然后Toaster创建新的数据库，以及构建和层目录。

管理Django框架

Toaster构建在Django Web框架之上。Django Web框架提供了管理用户界面，它给了你对存储在数据库中的ORM的直接访问。为了使用用户界面，你必须首先创建Django超级用户。从Python虚拟环境内，执行：

```
(pvenv) $ /home/myname/poky/bitbake/lib/toaster/manage.py createsuperuser
```

该命令启动Django管理实用程序，它首先询问你下列项：

·超级用户的用户名（必需）

·电子邮箱地址（可选）

·密码（必需；你必须输入2次以验证）

在Django创建了超级用户之后，你可以像往常一样用以下命令启动Toaster：

```
(pvenv) $ /home/myname/poky/bitbake/bin/toaster
```

然后通过向浏览器的导航栏中输入以下地址来访问管理用户界面：

```
http://localhost:8000/admin
```

从管理用户界面中，你可以浏览Toaster　ORM，并且增加数据库条目等。

如果你从不同目录启动Toaster，那么每个目录都包含自己的toaster.sqlite数据库。对每个这些Toaster环境，你必须分别创建超级用户。

Toaster管理提供了以下类别的访问：

·Auth：定义用户和用户组的身份验证类别。本地Toaster配置除了用于超级用户之外不再使用这个功能。用于生产的Toaster环境使用这个功能来和远程以及多个用户共享Toaster示例以控制访问。

·Bldcontrol：包含关于被Toaster使用的构建环境的信息的构建控制类别。一个Toaster实例可能控制在相同构建主机上的多个构建环境，或者通过网络控制在不同构建主机上的多个构建环境。对于本地开发，通常仅仅有一个构建环境。

·Orm：包含关于BitBake版本、层源、Yocto项目发布和Toaster设置的信息的对象关系模型。Toaster设置包含各种BitBake变量，你可以借由Toaster用户界面通过点击Project Configuration屏幕内的BitBake变量来设置它们。

通常，你不需要直接访问ORM，但是可以使用Toaster用户界面进行配置。

[1] 如果你熟悉Django，你可能知道Django不但允许你设置端口而且也允许你指定网络接口。当前对Toaster来说，不能指定网络接口。

13.1.5 Toaster生产部署

Toaster的生产部署允许实例在多个及远程的用户中共享。为了伸缩到承载多个用户访问Toaster服务的负载，Toaster使用外部Web服务器代替Django的内置Web服务器，并用RDBMS代替SQLite。生产部署也通常被设置成构建模式而不是分析模式。正如我们已经看到的，构建模式允许用户直接从Web用户界面中创建Toaster项目并且启动构建。

用于Django生产部署的Web服务器首选是Apache。对于RDBMS来说，你可以在MySQL和PostgreSQL之间选择。你选择哪一个可能依赖于你的喜好以及过去的经验。对这个例子，我们选择MySQL。

几乎所有Linux发行版都已经用MariaDB[1]替换了MySQL。MariaDB是MySQL的一个直接可用的替代品，它是由原来MySQL的开发者维护的。即使Linux发行版的包管理器可能仍然在包名字中使用mysql，但是它们实际上安装MariaDB。

准备生产主机

为了准备用于Toaster设置的生产系统，执行下面的步骤：

1.为Yocto项目构建系统安装先决条件，正如第2章中所描述的。

2.安装Apache Web服务器、MySQL（MariaDB）及开发库、Python虚拟环境和Apache的Python模块。

a.在Ubuntu系统上：

```
$ sudo apt-get install apache2 libapache2-mod-wsgi mysql-server \
    virtualenv libmysqlclient-dev
```

b.在Fedora或者Red Hat系统上：

```
$ sudo dnf install httpd mod_wsgi python-virtualenv mysql-server \
    mysql mysql-devel
```

3.启动Apache Web服务器。

a.在Ubuntu系统上：

```
$ sudo service apache2 start
```

b.在Fedora或者Red Hat系统上：

```
$ sudo systemctl start httpd
```

通过把的浏览器指向生产主机来验证Web服务器是否正在运行。你应该看到默认网页，网页的内容依赖于你的发行版。

4.启动数据库服务器。

a.在Ubuntu系统上：

```
$ sudo service mysql start
```

b.在Fedora或者RedHat系统上：

```
$ sudo systemctl start mariadb
```

5.默认情况下，MySQL（MariaDB）没有为root设置密码。为了第一次设置root密码，执行以下命令：

```
$ mysqladmin -u root password <rootpassword>
```

6.验证你可以登录进MySQL（MariaDB）服务器：

```
$ mysql -u root -p
```

数据库应该询问你在前面步骤设置的密码，然后显示命令提示符。

7.为Toaster准备MySQL数据库：

```
$ mysql -u root -p
mysql> CREATE USER 'toaster'@'localhost' identified by 'password';
mysql> CREATE DATABASE toaster;
mysql> GRANT all on toaster.* to 'toaster'@'localhost';
mysql> exit
```

你的生产主机现在已经准备好用于Toaster的安装和配置了。

Toaster的安装和配置

对生产配置，作为Poky构建系统一部分的Toaster最理想的是被安装到Apache　Web服务器的文档根目录，这允许更简单的应用Apache访问规则层级结构。通常，那个目录是/var/www。你可以安装到不同的目录，但是你需要对应地调整Apache配置。执行以下步骤来在你的生产主机上安装和配置Toaster：

1.在Web服务器文档根目录中创建一个目录，并且安装Poky构建系统。我们正在使用/var/www/toaster作为安装目录和Poky的Jethro分支：

```
$ sudo mkdir /var/www/toaster
$ cd /var/www/toaster
$ sudo git clone git://git.yoctoproject.org/poky
$ cd poky
$ sudo git checkout jethro
```

2.安装Toaster和Django　Web框架用于访问MySQL数据库所需要的Python包。我们推荐再次使用Python虚拟环境来隔离Toaster设置和生产主机的Python设置。

```
$ cd /var/www/toaster
$ sudo virtualenv pvenv
```

```
$ source pvenv/bin/activate
$ sudo pip install -r poky/bitbake/toaster-requirements.txt
$ sudo pip install mysql
$ sudo pip install MySQL-python
```

3.通过按照如下编辑在文件/var/www/toaster/poky/bitbake/lib/toaster/toastermain/settings.py中的配置部分来配置Toaster：

a.修改用于MySQL的DATABASES部分：

```
DATABASES = {
'default': {
      'ENGINE': 'django.db.backends.mysql',
      'NAME': 'toaster',
      'USER': 'toaster',
      'PASSWORD': 'password',
      'HOST': 'localhost',
      'PORT': '3306',
   }
}
```

对数据库NAME、USER和PASSWORD，使用上一小节第7步中的值。移除HOST和PORT设置以使用UNIX域套接字（UNIX domain socket）来访问MySQL数据库，这是默认的情况。

b.修改SECRET_KEY为唯一的密钥：

```
SECRET_KEY = 'secretkey'
```

你可以使用OpenSSL来创建任意的密钥。下面的命令创建16个字符长度的base64编码的密钥：

```
$ openssl rand -baes64 16
```

c.修改STATIC_ROOT为

```
STATIC_ROOT = '/var/www/toaster/static/'
```

Django和Toaster两者所使用的静态服务的文件（例如HTML和JavaScript文件）需要由Apache　Web服务器提供。这些文件被收集并且复制到这个目录。

d.通过设置BUILD_MODE为下面的值来启用构建模式：

```
BUILD_MODE = True
```

4.创建数据库方案（schema），加载默认数据并且收集静态服务文件：

```
$ cd /var/www/toaster/poky
$ ./bitbake/lib/toaster/manage.py syncdb
$ ./bitbake/lib/toaster/manage.py migrate
$ TOASTER_DIR=`pwd` TOASTER_CONF=./meta-yocto/conf/toasterconf.json\
      ./bitbake/lib/toaster/manage.py checksettings
$ sudo ./bitbake/lib/toaster/manage.py collectstatic
```

与步骤相关的几个解释是恰当的：

·syncdb和migrate命令创建数据库方案。它们也安装Django的身份验证，询问你是否希望创建用于访问管理用户界面的超级用户。我们推荐你在此时这样做。替代性地，你可以在稍后创建超级用户，正如13.1.4小节中所描述的。

·checksettings命令从文件poky/meta-yocto/conf/toasterconf.json中加载Toaster配置数据。这个文件包含默认设置以及关于层源的信息。TOASTER_DIR环境变量决定Toaster在哪里创建构建环境。正如在前面例子中所看到的，使用pwd把构建环境放在/var/www/toaster/poky目录，

这可能是不可取的，因为构建环境变得非常大。你可以指定不同的目录——只要保证它是存在的并且对运行Toaster构建的用户来说是可写的。

·collectstatic命令获取静态服务的文件，正如前面所描述的。不像其他命令，你必须以root权限运行collectstatic，除非你改变static目录的权限。

Web服务器配置

这个步骤把Toaster和Apache Web服务器集成起来。在你生产主机的Apache配置目录中，你需要一个Web服务器网关接口（Web Server Gateway Interface，WSGI）配置文件。

在Ubuntu和Debian系统上，创建文件：

```
/etc/apache2/conf-available/toaster.conf
```

在Fedora和Red Hat系统上，创建文件：

```
/etc/httpd/conf.d/toaster.conf
```

并写入程序清单13-1的内容。

程序清单13-1　WSGI配置（toaster.conf）

```
Alias /static /var/www/toaster/static
     <Directory /var/www/toaster/static>
             Order allow,deny
             Allow from all
             Require all granted
     </Directory>

     WSGIDaemonProcess toaster_wsgi \
        python-path=/var/www/toaster/poky/bitbake/lib/toaster: \
        /var/www/toaster/pvenv/lib/python2.7/site-packages
     WSGIProcessGroup toaster_wsgi
WSGIScriptAlias / \
   "/var/www/toaster/poky/bitbake/lib/toaster/toastermain/wsgi.py"
```

如果你改变了例子中的Poky安装位置或Python虚拟环境，那么确保你在WSGIDaemonProcess设置中对应地调整了它们。

在Ubuntu和Debian系统上，你需要用以下命令显式地启用WSGI模块和Toaster配置：

```
$ sudo a2enmod wsgi
$ sudo a2enconf toaster
```

这个步骤在Fedora和Red Hat系统上是不需要的。

最后，你需要重启Apache Web服务器。在Ubuntu和Debian系统上，使用以下命令：

```
$ sudo service apache2 restart
```

在Fedora和Red Hat系统上，使用以下命令：

```
$ sudo systemctl restart httpd
```

现在，在你浏览器的导航栏中输入你生产主机的主机名或者IP地址，你应该看到Toaster登录页面。如果你得到错误消息，那么参考Apache Web服务器的日志文件以获得关于错误的根本原因的信息——在Ubuntu和Debian系统上，它是在/var/log/apache2目录中；在Fedora和Red Hat系统上，它是在/var/log/httpd目录中。

安装构建运行服务

构建运行服务需要运行以执行构建。服务用bblayers.conf和local.conf的配置设置来创建Yocto项目构建环境，并且用已提供的镜像目标来执行BitBake。为了启动构建运行服务，使用以下命令：

```
$ source /var/www/toaster/pvenv/bin/activate
(pvenv) $ /var/www/toaster/poky/bitbake/lib/toaster/mangage.py runbuilds
```

你可能希望把上面的命令包进一个简单的shell脚本中以简化执行，正如程序清单13-2所示的。

程序清单13-2　Toaster构建运行shell脚本（toasterbuildrunner.sh）

```
#!/bin/sh
# Launch Toaster Build Runner on a production host
pushd /var/www/toaster
source ./pvenv/bin/activate
./poky/bitbake/lib/toaster/manage.py runbuilds
```

不要以root运行构建运行服务。构建运行服务执行BitBake，BitBake拒绝以root权限运行。在安装和配置Toaster时，确保运行构建运行服务的用户对你用TOASTER_DIR指定的目录拥有完全的访问。

现在你可以从Toaster Web用户界面中启动构建。

维护你的Toaster生产实例

为了使你的Toaster生产实例总是保持最新的，你需要有规律地更新数据库中的层源信息。这保证你总是对最新的层和菜谱拥有访问。为了加载最新的层信息到Toaster数据库中，使用命令：

```
$ /var/www/toaster/poky/bitbake/lib/toaster/manage.py lsupdates
```

为了更新你的生产Toaster实例到Yocto项目的一个更新的版本，使用以下命令：

```
$ cd /var/www/toaster/poky
$ git pull
$ git checkout <branch>
$ TOASTER_DIR=`pwd` TOASTER_CONF=./meta-yocto/conf/toasterconf.json \
      ./bitbake/lib/toaster/manage.py checksettings
$ sudo ./bitbake/lib/toaster/manage.py collectstatic
```

运行checksettings用来自toasterconf.json文件的更新的发布和层信息来填充数据库。执行collectstatic保证用户界面更新是对Web服务器可用的。在更新后，确保重启Web服务器。

[1] https://mariadb.org。

13.1.6 Toaster Web用户界面

Toaster Web用户界面提供以下的功能性：

·项目管理：创建、配置和查看Toaster项目。Toaster项目类似于通过命令行引用oe-init-build-env创建的构建环境。Toaster为你创建和管理环境。对项目，你选择希望使用其来构建项目的Yocto项目发布。

·构建配置：在Toaster项目内，你可以配置机器、发行版和其他设置，正如你通过编辑构建环境的conf/local.conf文件所做的。Toaster用户界面提供对常见配置变量的直接访问，例如DISTRO、IMAGE_FSTYPES、IMAGE_INSTALL_append、PACKAGE_CLASSES和SDKMACHINE。你可以按照你希望的增加其他变量。然而，一些变量是被排除的。这些变量是那些影响构建主机的配置和设置到构建制品存储位置的路径的变量，例如SSTATE_DIR和TMPDIR。

·层管理：Toaster允许你向你的项目增加层和去除层。你也可以浏览可用层的列表。meta、yocto和yocto-bsp这3个层是默认包含在项目中的，并且被自动从Poky仓库中检出。来自OpenEmbedded层索引中关于层的信息是直接从网页中获取的并且被显示在Toaster用户界面中。通过简单地单击按钮，你即可增加那些层到你的项目中。这些层是按需从OpenEmbedded层仓库中检出的。另外，你可以从Git仓库中导入自己的层。你需要保证你正在导入的层与你为项目选择的Yocto项目发布是兼容的。

·镜像目标：Toaster从各种可用层中识别和列出镜像目标。如果镜像目标是从一个你构建配置所包含的层中可用的，那么你可以通过单击镜像目标旁边的Build recipe按钮来直接构建它。否则，单击Add layer按钮以增加层到你的构建配置中。如果层依赖于其他的层，那么Toaster把

依赖性通知给你，并且自动包含它们。

·包菜谱：Toaster维护所有层的所有菜谱列表，不管它们是否已经被你的构建所包含。搜索功能帮助寻找特定的菜谱。例如，在搜索栏中输入jdk列出所有提供Java JDK的菜谱。单击按钮，你可以增加包含菜谱的层并且构建它。然而，构建菜谱不自动增加它到IMAGE_INSTALL。你必须通过编辑BitBake变量屏幕上的IMAGE_INSTALL_append变量来显式地增加它。

·构建日志：你可以直接从Toaster用户界面中查看和检查追踪、警告及错误消息。你也可以从Toaster中下载构建日志到你的本地机器。

·构建统计和性能信息：Toaster收集例如全局构建时间、每个任务的时间、CPU使用率和硬盘输入输出等构建统计。

·镜像信息：Toaster收集和展示关于什么包已经被构建并且被包含在镜像中的信息。你可以从Toaster用户界面中浏览镜像的结构并查看菜谱和包之间的依赖性关系。

虽然Toaster允许你选择和构建特定菜谱并且包含它到你的镜像中，但如果一个层为一个菜谱提供了多个版本，那么你不能直接选择一个菜谱的特定版本来构建。通常，正如我们前面所看到的，构建系统选择一个菜谱的最新版本。例如，如果Linux　Yocto内核的最新版本是4.1，但是你想构建版本3.9，你需要增加

```
PREFERRED_VERSION_linux-yocto = "3.9%"
```

到Toaster用户界面的变量配置中。

Yocto项目Toaster团队已经制造了一系列涵盖Toaster各个方面的指导性视频。你可以在www.youtube.com上找到这些视频。仅仅输入Yocto

Project Toaster到YouTube搜索栏中。

13.2 构建历史

由构建系统处理的相当多的菜谱和配置文件决定什么包被构建、它们如何被构建、它们如何互相依赖以及最终完成的输出制品包含什么。那些输出包括但不限于二进制包、内核和根文件系统镜像以及软件开发工具包（Software Development Kit，SDK）。伴随着这么多影响构建的因素，在构建间维护可重复性和一致性以及追踪和审计变更的能力是一个强需求。

考虑升级软件包到新版本。如果那个软件包包依赖于一个新版本的库，而这个库是与很多其他包所共享的，那么看起来足够简单的东西会有重要的影响。在这种情况下，构建新版本的软件包自动拉取新版本的库，如果那个库不是向后兼容的，那么这可能对其他软件包造成问题。

构建系统的构建历史功能为维护构建质量提供了自动的支持，这是通过以下来完成的：记录关于包、镜像和SDK构建的核心信息；在文件中存储它们；以及提交那些文件到Git仓库中以创建可追踪的历史。构建历史是由buildhistory类实现的，该类是由所有菜谱全局性地继承的。它的行为是由几个配置变量控制的，这些变量允许你定义构建历史存储在哪里，从什么构建制品中收集信息等。

13.2.1 启用构建历史

构建历史默认是禁用的。你必须通过增加它到你构建环境的conf/local.conf中的INHERIT变量中以启用它：

```
INHERIT += "buildhistory"
BUILDHISTORY_COMMIT = "1"
```

第一条语句启用buildhistory类，它为所有菜谱收集构建新型。第二条语句启用提交所有对构建历史的变更到Git仓库。如果你仅仅对收集关于最后构建的信息感兴趣，那么你可以通过设置BUILDHISTORY_COMMIT="0"来禁用提交到Git仓库。

构建历史是累积的，这意味着仅仅收集那些在构建过程中已经执行了的菜谱及其任务的信息。其他隐含着，如果你正在寻找完整的构建历史，那么你必须在启动第一次构建之前就为你的构建环境启用它。

13.2.2 配置构建历史

你可以通过一组配置变量来调整构建历史的行为：

·BUILDHISTORY_DIR：这个变量指定到buildhistory类存储构建历史信息所在位置的路径。默认设置是BUILDHISTORY_DIR?="${TOPDIR}/buildhistory。如果BUILDHISTORY_COMMIT是启用的，那么buildhistory类在那个位置创建Git目录。使用这个变量来迁移构建历史。

·BUILDHISTORY_COMMIT：这个变量控制构建历史是否提交到位于BUILDHISTORY_DIR内的本地Git仓库中。如果这个变量第一次用于现有构建历史，那么buildhistory类初始化在BUILDHISTORY_DIR中的Git仓库。在每次完成BitBake运行后，对构建历史的变更被提交到仓库。如果你希望追踪对构建历史的变更，那么设置BUILDHISTORY_COMMIT="1"。默认设置是BUILDHISTORY_COMMIT?="0"。

·BUILDHISTORY_COMMIT_AUTHOR：当使用Git仓库来追踪随时间演变而产生的构建历史变更时，变量提供一个用于提交到仓库的Git用户名。Git要求这个变量的值的形式是name<email@domain>。这个变量的默认设置是BUILDHISTORY_COMMIT_AUTHOR?="buildhistory<buildhistory@${DISTRO}>。只有在BUILDHISTORY_COMMIT设置成"1"时，这个变量的设置才起作用。

·BUILDHISTORY_FEATURES：buildhistory类为以下不同类别收集分析数据：

·image（镜像）：包含已安装包在内的镜像内容的分析数据。

·package（包）：各个包内容的分析数据。

·sdk：SDK内容的分析数据。

你可以以一个空格分隔的列表来指定这些类别的任意组合。默认值是BUILDHISTORY_FEATURES?="image package sdk"。

·BUILDHISTORY_IMAGE_FILES：这个变量指定空格分隔的路径列表，该路径指向安装在根文件系统镜像中的文件，以便你可以追踪文件的内容。对系统和应用配置文件来说，这尤其有用。默认设置是BUILDHISTORY_IMAGE_FILES?="/etc/passwd/etc/group"，这允许你追踪对用户和组条目的变更。buildhistory类使用cp命令来把文件从镜像根复制到构建历史目录。你可以将通配符用于路径的最后片段来用变量中的单一条目复制多个文件。然而，子目录不是递归复制的。例如，使用BUILDHISTORY_IMAGE_FILES="/etc/*"复制所有在/etc下的文件但不包括子目录。你需要显式地指定那些希望复制的子目录。

·BUILDHISTORY_PUSH_REPO：当使用Git仓库来追踪变化（这是非常推荐的）的时候，你可以有选择地指定远程Git仓库以在构建历史已经被提交到本地仓库后推送构建历史到Git仓库服务器。为了使BUILDHISTORY_PUSH_COMMIT生效，变量BUILDHISTORY_COMMIT必须被设置成1。对变量的默认设置是BUILDHISTORY_PUSH_REPO?=""。

13.2.3 推送构建历史到Git仓库服务器

除了用构建主机上的本地Git仓库追踪构建历史，推送所有变更到Git仓库是保证构建质量和维护历史的重要工具。虽然设置基本上是简单直接的，但是我们把这个部分专门用于这个主题是因为有一些重要的事情要考虑。

为了演示设置，我们使用公共GitHub仓库服务器[1]。对这个目的而言，我们不推荐使用GitHub，但是它提供了测试功能性的简单方法，因为设置Git仓库服务器不在本书的范围内。在可以使用GitHub之前，你必须创建GitHub账号，对仅仅托管公共仓库的账号来说，这是免费的。GitHub网站也解释了该过程。然后，使用GitHub Web用户界面创建空的Git仓库——例如，yp_buildhistory。不要创建任何文件，也不要对仓库做初始提交。第一次提交来自Yocto项目构建。

虽然你可以自由地读取和克隆在GitHub上的任何公共仓库，但是推送变更到仓库需要身份验证。构建系统使用安全Shell（SSH）来推送对本地构建历史Git仓库做出的变更到远程Git仓库。当使用SSH时，GitHub要求公钥基础实施（Public Key Infrastructure，PKI）用于身份验证。PKI由一对公钥和一对私钥组成。你在构建主机上创建密钥对并且上传公钥到GitHub上。GitHub对如何完成那个任务提供了好的指导[2]。当创建SSH密钥时，你可以选择使用密码来保护私钥。如果你计划用Yocto项目做自动化的构建，那么不推荐这么多，因为构建系统停止执行并且询问要输入的密码。

一旦你已经设置了GitHub账号，增加了SSH公钥到你账号，并且创建了仓库，那么你需要设置用于构建环境的构建历史。和往常一样，那个配置是通过设置在conf/local.conf中的变量来完成的，正如程序清单

13-3所示。

程序清单13-3　构建历史配置（conf/local.conf）

```
#
# Build History Configuration
#
INHERIT += "buildhistory"
BUILDHISTORY_COMMIT = "1"
BUILDHISTORY_COMMIT_AUTHOR = "Santa Claus <santa.claus@northpole.com>"
BUILDHISTORY_DIR = "${TOPDIR}/../../yocto/buildhistory"
BUILDHISTORY_IMAGE_FILES = "/etc/passwd /etc/group"
BUILDHISTORY_PUSH_REPO = "git@github.com:sclaus/yp_buildhistory.git master"
```

用于BUILDHISTORY_PUSH_REPO的URL当然依赖于远程仓库的设置。在例子master的案例中，指定分支是可选的，但要求分支已经存在于远程仓库中，而对于空的仓库这是不成立的。当多个构建环境推送构建历史到相同的远程仓库时，使用分支并且显式地指定它们是强烈推荐的，这允许为每个构建环境使用不同的分支。BitBake变量扩展适用于BUILDHISTORY_PUSH_REPO，正如它适用于任何其他变量一样。这意味着，你可以使用例如${DISTRO}的变量来指定分支。

因为构建系统总是提交构建历史中的变更到主干分支，所以你手动地使用

```
$ git checkout -b <branchname>
```

来创建和切换到本地构建历史Git仓库的那个分支中。

在那之后，你可以在BUILDHISTORY_PUSH_REPO中使用branchname。

[1] https://github.com。
[2] https://help.github.com/articles/generating-ssh-keys。

13.2.4 理解构建历史

构建历史以由目录和文件组成的特定结构存储在BUILDHISTORY_DIR之下（见程序清单13-4）。

程序清单13-4 构建历史结构

```
$ tree -L 3 buildhistory
buildhistory/
├── metadata-revs
├── images
│   ├── qemux86_64
│   │   └── glibc
│   │       └── core-image-base
│   │           ├── build-id.txt
│   │           ├── depends.dot
│   │           ├── depends-nokernel.dot
│   │           ├── depends-nokernel-nolibc.dot
│   │           ├── depends-nokernel-nolibc-noupdate.dot
│   │           ├── depends-nokernel-nolibc-noupdate-nomodules.dot
│   │           ├── files-in-image.txt
│   │           ├── image-files
│   │           ├── image-info.txt
│   │           ├── installed-package-names.txt
│   │           ├── installed-package-sizes.txt
│   │           └── installed-packages.txt
│   └── ypbkiosk
│       └── glibc
│           ├── core-image-minimal-initramfs
│           │   ├── build-id.txt
│           │   ├── depends.dot
│           │   ├── depends-nokernel.dot
│           │   ├── depends-nokernel-nolibc.dot
│           │   ├── depends-nokernel-nolibc-noupdate.dot
│           │   ├── depends-nokernel-nolibc-noupdate-nomodules.dot
│           │   ├── files-in-image.txt
│           │   ├── image-files
│           │   ├── image-info.txt
│           │   ├── installed-package-names.txt
│           │   ├── installed-package-sizes.txt
│           │   └── installed-packages.txt
│           └── ypbkiosk-image-sato
│               ├── build-id.txt
│               ├── depends.dot
│               ├── depends-nokernel.dot
│               ├── depends-nokernel-nolibc.dot
│               ├── depends-nokernel-nolibc-noupdate.dot
│               ├── depends-nokernel-nolibc-noupdate-nomodules.dot
│               ├── files-in-image.txt
│               ├── image-files
│               ├── image-info.txt
│               ├── installed-package-names.txt
│               ├── installed-package-sizes.txt
│               └── installed-packages.txt
├── metadata-revs
├── packages
│   ├── all-poky-linux
│   │   ├── adwaita-icon-theme
│   │   │   ├── adwaita-icon-theme
│   │   │   │   ├── latest
│   │   │   │   ├── latest.pkg_postinst
```

```
|   |   |   |   └── latest.pkg_postrm
|   |   |   ├── adwaita-icon-theme-cursors
|   |   |   |   ├── latest
|   |   |   |   ├── latest.pkg_postinst
|   |   |   |   └── latest.pkg_postrm
|   |   |   ├── adwaita-icon-theme-hires
|   |   |   |   ├── latest
|   |   |   |   ├── latest.pkg_postinst
|   |   |   |   └── latest.pkg_postrm
...
|   ├── core2-64-poky-linux
|   |   ├── acl
|   |   |   ├── acl
|   |   |   |   ├── files-in-package.txt
|   |   |   |   └── latest
|   |   |   ├── acl-dbg
|   |   |   |   ├── files-in-package.txt
|   |   |   |   └── latest
|   |   |   ├── acl-dev
|   |   |   |   ├── files-in-package.txt
|   |   |   |   └── latest
|   |   |   ├── acl-doc
|   |   |   |   ├── files-in-package.txt
|   |   |   |   └── latest
...
└── sdk
    └── poky-glibc-x86_64-hagw-image-base-corei7-64
        └── hagw-image-base
            ├── files-in-sdk.txt
            ├── sdk-info.txt
            ├── host
            |   ├── depends.dot
            |   ├── installed-package-names.txt
|   ├── installed-package-sizes.txt
|   └── installed-packages.txt
└── target
    ├── depends.dot
    ├── installed-package-names.txt
    ├── installed-package-sizes.txt
    └── installed-packages.txt
```

位于顶级的是用于通过BUILDHISTORY_FEATURES启用的类别的子目录。在一个成功的构建完成后，用于镜像和包特性的子目录总是存在的。用于软件开发工具包特性的子目录仅仅在你已经用-c populate_sdk构建了SDK之后才被创建和填充。另外，构建历史的顶级目录也包含文件metadata-revs，它包含元数据层的修订信息，而这些元数据层是在构建生产时由构建系统所使用的。

构建历史镜像信息

构建历史镜像信息正如${DEPLOY_DIR}/image目录中的镜像一样

是按照机器划分类别的。在那之下是一个名为C库的子目录。除非你正在用不同的C库构建，否则那个子目录的名字就是glibc。在C库子目录下，构建系统为每个镜像目标创建一个子目录，例如core-image-base。在镜像目标子目录内是包含关于镜像的信息的各种文件和目录：

·build-id.txt：用到的构建配置，包括BitBake版本、构建主机和元数据层版本等。

·depends.dot：以文本的DOT格式表现的完整依赖性关系图，它可以被Graphviz和其他能够解析DOT格式的软件所渲染。

·depends-nokernel.dot：和前面一个相同，但不包括内核依赖性。

·depends-nokernel-nolibc.dot：和前面一个相同，但不包括C库依赖性。

·depends-nokernel-nolibc-noupdate.dot：和前面一个相同，但不包括更新依赖性。

·depends-nokernel-nolibc-noupdate-nomodules.dot：和前面一个相同，但不包括内核模块依赖性。

·files-in-image.txt：镜像中的所有文件列表，本质上是根文件系统的命令find/!-path.-printf"%M%-10u%-10g%10s%p\n"输出。

·image-files：包含由BUILDHISTORY_IMAGE_FILES指定的文件的子目录。

·image-info.txt：直接影响镜像的内容和大小的变量及其值的列表。

·installed-package-names.txt：按照字母排序的、安装在镜像中的所有包名的列表。

·installed-package-sizes.txt：按照大小（从大到小）顺序排列的、安装在镜像中的所有包的列表。

·installed-packages.txt：按照字母排序的、安装在镜像中的所有包的完整包文件名的列表。

带有概述性总结的image-info.txt文件对于追踪镜像内容中的变更提供了有用的起点。存储在文件中的变量有：IMAGE_CLASSES，其包含用于创建镜像的类的列表；IMAGE_INSTALL、IMAGE_FEATURES和ROOTFS_POSTPROCESS_COMMAND，它们直接参与编译根文件系统的内容。

构建历史包信息

针对包的构建历史按照架构被组织进子目录中并且包含为构建主机和目标所创建的包。架构子目录和用在${BASE_WORKDIR}内的子目录是相同的。

每个包都有自己的子目录，其包含名为latest的文件和用于每个在包拆分过程中创建的安装包的子目录。包顶级的latest文件包含包版本（PV）和包修订（PR）信息、构建依赖性列表（DEPENDS）以及创建的安装包的列表（PACKAGES）。如果包是从获取自版本控制系统（如Git）的源创建的，那么目录也包含文件latest_srcrev，它包含所使用分支的源修订的列表。

每个安装包子目录包含一个或者多个带有包信息的文件。

·latest：在构建中决定包的内容的变量及其值的列表。这个列表包含PV、PR、RPROVIDES、RDEPENDS、FILES以及一个PKGSIZE条目（以KB为单位的整个包的大小）。

·files-in-package.txt：包含在包中的所有文件及其路径和大小的列表。

·latest.pkg_*：在实施特定的命令之前或者之后，包含由包管理器执行的任何命令的文件，特定的命令包括install、update和remove。

当你希望为那些使用AUTOREV来自动化从仓库分支中获取最新修订的包固定源修订时，包含在包的latest_srcrev文件中的源修订信息是重要的。你可以使用脚本buildhistory-collect-srcrevs来从构建历史中以一种可直接用在例如conf/local.conf的配置文件或者发行版策略文件中的格式收集源修订。针对从版本控制系统源中获得的每个包，脚本产生以下形式的一行：

```
SRCREV_pn-<packagename> = "<versiontag>"
```

默认情况下，脚本仅仅为正在使用AUTOREV的包产生输出，除非在调用脚本的时候，你对命令行使用-a或者--report-all参数。

在配置中指定SRCREV_pn-<packagename>不会阻止变量被菜谱的追加文件或者构建配置中的其他地方所覆盖。为了阻止覆盖，必须增加forcevariable：

```
SRCREV_pn-<packagename>_forcevariable = "<versiontag>"
```

当-f或者--forcevariable被增加到命令行时，脚本自动增加forcevariable。

构建历史软件开发工具包信息

当软件开发工具包（SDK）目标被构建系统处理时，构建历史收集关于SDK的内容的信息。多个SDK按照发行版和镜像目标被分离进子目

录中。因为SDK是为构建主机和目标所构建的，所以构建历史包含用于它们每一个的信息。用于SDK的构建历史目录包含以下：

·files-in-sdk.txt：包含在SDK中的文件列表。这个列表是针对构建主机和目标的。

·sdk-info.txt：决定SDK内容的配置变量及其值的列表，以及SDK大小的一个条目：

·DISTRO：发行版策略。

·DISTRO_VERSION：发行版版本字符串。

·SDK_NAME：SDK名字字符串。

·SDK_VERSION：SDK的版本字符串。

·SDKMACHINE SDK：机器信息。

·SDKIMAGE_FEATURES：用于构建SDK根文件系统镜像特性的列表，这些特性通常是dev-pkgs和dbg-pkgs。

·SDKSIZE：SDK的大小。

·host和target：包含具有关于主机和目标SDK的信息的文件的目录。这些文件在这两个目录中创建：

·depends.dot：以文本的DOT格式表现的完整依赖性关系图，它可以被Graphviz和其他能够解析DOT格式的软件所渲染。

·installed-package-names.txt：以字母顺序排列的安装在SDK中的包名的列表。

·installed-package-sizes.txt：安装在SDK中的包名和包大小的列表，是按照从最大包到最小包的顺序排列的。

·installed-packages.txt：按照字母排序的、安装在SDK中的所有包的完整包文件名的列表。

13.3 源镜像

在本书4.5节中，我们讨论了构建系统如何访问和下载源，以及镜像站点可以如何被用来设置替代性的下载位置而不用修改菜谱中的SRC_URI。

设置你自己的镜像站点由于以下原因来变得很有意义：

·对有很多开发者的团队来说，避免从互联网下载数GB的源包。

·为了一致性和可重复性而确保一个团队的所有开发者正在从相同的源来构建。

·为产品交付控制源包版本。

13.3.1 使用源镜像

我们已经看到，在检查了一个源包是否在由DL_DIR指定的本地下载目录中可用之后，构建系统首先使用由变量PREMIRRORS指定的镜像站点用于对源包的远程访问。变量包含由换行符分隔的数组列表，数组为键指定正则表达式来匹配SRC_URI。当然，你可以自己直接在构建环境的conf/local.conf文件中设置PREMIRRORS变量。然而，更方便的是，你可以使用own-mirrors类和变量SOURCE_MIRROR_URL。正如程序清单13-5所示，own-mirrors类仅仅包含PREMIRRORS变量赋值而没有其他内容。

程序清单13-5 own-mirrors类

```
PREMIRRORS() {
cvs://.*/.*     ${SOURCE_MIRROR_URL}
svn://.*/.*     ${SOURCE_MIRROR_URL}
git://.*/.*     ${SOURCE_MIRROR_URL}
gitsm://.*/.*   ${SOURCE_MIRROR_URL}
hg://.*/.*      ${SOURCE_MIRROR_URL}
bzr://.*/.*     ${SOURCE_MIRROR_URL}
p4://.*/.*      ${SOURCE_MIRROR_URL}
osc://.*/.*     ${SOURCE_MIRROR_URL}
https?$://.*/.* ${SOURCE_MIRROR_URL}
ftp://.*/.*     ${SOURCE_MIRROR_URL}
}
```

类本质上通过将变量SOURCE_MIRROR_URL复制到构建系统获取器所支持的所有协议方案中来为你完成所有工作。所有你要做的是，继承own-mirrors类并且在构建环境的conf/local.conf文件中将SOURCE_MIRROR_URL变量设置为自己的镜像站点：

```
SOURCE_MIRROR_URL ?= "file:///path/to/directory/"
INHERIT += "own-mirrors"
```

例子使用file：方案来直接访问文件系统，它可以是本地或者任何类型的远程文件系统，例如网络文件系统（Network File System，

NFS）。替代性地，你可以为构建系统使用ftp：、http:和https:方案来访问你的镜像站点。对团队开发来说，使用你自己的发行版策略而不是本地构建环境配置来设置自己的镜像也是有意义的。

使用带有SOURCE_MIRROR_URL的own-mirrors类或者直接设置PREMIRRORS都不能阻止构建系统访问例如菜谱的SRC_URI的其他下载资源或者由MIRRORS指定的后镜像。然而，对于产品部署，关键的是，构建系统不会在无意中从非受控位置获取任何源包。可以通过设置MIRRORS为空字符串来实现禁用后镜像。然而，在菜谱中全局性地禁用SRC_URI是不可能的。

使用

```
BB_NO_NETWORK = "1"
```

禁止用于从任何下载源获取的网络访问，包括由PREMIRRORS指定的那些。如果你仅仅打算为镜像站点使用file:方案，那么这会工作得很好。然而，对一个对团队开发有用的、具有自动化构建和质量保证的设置来说，你应该使用FTP、HTTP或者HTTPS协议从受控的镜像站点来启用获取而没有不经意地从互联网上获取源包的风险。你可以通过设置以下内容而限制仅仅允许访问PREMIRRORS来实现这个目的：

```
BB_FETCH_PREMIRRORONLY = "1"
```

另外，或者是替代性地，你可以限制仅仅允许对特定主机的网络访问。变量BB_ALLOWED_NETWORKS指定空格分隔的主机列表，构建系统被允许从这些主机中获取源包：

```
BB_ALLOWED_HOSTS = "server1.acme.com server2.acme.com"
```

这个例子允许从属于acme.com域的server1和server2上获取。基础的

通配符匹配被提供来匹配主机名的开头：例如

```
BB_ALLOWED_HOSTS = "*.acme.com"
```

允许从任何属于acme.com域的主机上获取。

当使用BB_ALLOWED_HOSTS时，列在镜像变量PREMIRRORS和MIRRORS中的主机被简单跳过，并且一条日志消息被记录。用未包含在BB_ALLOWED_HOSTS的主机来访问SRC_URI会导致错误。

联合使用BB_ALLOWED_HOSTS和own-mirrors类以及SOURCE_MIRROR_URL或者直接联合使用BB_ALLOWED_HOSTS和PREMIRRORS允许你利用网络协议方案同时阻止构建系统访问任何未被列在BB_ALLOWED_HOSTS中的下载站点。增加列在BB_ALLOWED_HOSTS中的主机到SOURCE_MIRROR_URL或者PREMIRRORS会导致源只能从已授权的主机获取。如果源包是在镜像站点上缺失的，那么构建系统通过使用来自菜谱的SRC_URI来尝试获取它。这会导致失败，因为在SRC_URI中的上游主机没有被BB_ALLOWED_HOSTS列出。这正是你希望的行为，因为它把任何从非受控的站点访问资源的尝试通知给你。

13.3.2　设置源镜像

如何设置源镜像取决于你。你必须决定，是否希望在构建主机上挂载来自源镜像的文件系统导出并且使用file:方案，或者是否宁愿设置HTTP/HTTPS或者FTP服务器。在任何情况下，你必须使用构建系统下载一次源包，然后把它们从由DL_DIR指定的下载目录中复制到你的镜像主机。

直接从远程源仓库（例如Git、Apache SVN、Perforce等）中检出的源以树的形式被放在下载目录中。这使得它们不适合简单地复制到稍后可以用file:、ftp:、http:或者https:方案访问的镜像站点。在构建环境的conf/local.conf中使用

```
BB_GENERATE_MIRROR_TARBALLS = "1"
```

使得构建系统从仓库树中创建tar包，然后可以简单地把它们以及其他源包复制到你的镜像站点。为了性能的原因，BB_GENERATE_MIRROR_TARBALLS默认是被禁用的。

以下是几个简单的步骤，它们让你设置自己的源镜像：

1.设置构建环境并且启用BB_GENERATE_MIRROR_TARBALLS。

2.用bitbake-c　fetchall<target>启动构建，其中<target>是任何综合性的镜像目标，例如core-image-sato。

3.复制所有包含在下载目录中的源tar包到你的镜像主机。

4.设置一个构建环境来继承own-mirrors并且把SOURCE_MIRROR_URL指向你的新镜像。你可以禁用

BB_GENERATE_MIRROR_TARBALLS。

5.启动构建以测试从新镜像获取。

6.通过按需设置BB_NO_NETWORK或者BB_ALLOWED_HOSTS来调优你的设置。

13.4 自动构建器

Yocto项目自动构建器（Autobuilder）是基于开源持续集成框架Buildbot的自动化构建系统[1]。Buildbot是用于自动化软件构建、质量保证和发布过程的可扩展的框架。

Buildbot是以Python编写的，并且是使用Twisted Python[2]事件驱动的网络引擎来实现的。Buildbot是作业调度系统——它把作业排队，监控执行作业的必要资源，当资源就位时执行作业，并且报告结果。

Buildbot部署通常至少由一个控制器和一组工作者组成。控制器监控源代码仓库，调度作业，协调工作者并报告作业执行的结果。控制器为用户提供Web用户界面来与系统交互。工作者可以被部署在和控制器相同的系统上，或者被部署在独立的系统上，后者使得Buildbot成为分布式构建引擎。控制器分配构建器（builder）到工作者，工作者执行构建器并且把结果报告回控制器。

Buildbot配置是通过Python脚本来完成的，这和设置配置变量一样简单。然而，完整的Python功能性是可用的，这允许通过Python代码实现配置的动态生成。

Yocto项目自动构建器通过一组用于Yocto项目目标的标准构建器来扩展Buildbot。Yocto项目构建基础设施使用自动构建器用于每夜构建、持续集成和发布构建。你可以通过其位于https://autobuilder.yoctoproject.org的首页来访问Yocto项目自动构建器。除非你有Yocto项目自动构建器的登录账号，否则你不能自己调度和执行构建，但是你可以查看最新构建的当前状态和构建历史。你也可以直接从Yocto项目自动构建器的发布目录[3]下载用于各种机器的镜像构建和Eclipse插件等。

Yocto项目构建团队已经把自动构建器以及设置和执行脚本打包在一起了，这使得让一个自动构建器实例运行在你自己的系统上这一过程变得很短（只需几分钟）。

[1] http://buildbot.net。
[2] https://twistedmatrix.com/trac。
[3] http://autobuilder.yoctoproject.org/pub。

13.4.1 安装自动构建器

具有运行在相同主机上的一个控制器和一个工作者的自动构建器的基本安装和配置可以通过以下3个简单步骤来完成：

```
$ git clone git://yoctoproject.org/yocto-autobuilder
$ cd yocto-autobuilder
$ source yocto-autobuilder-setup
```

就这么简单。yocto-autobuilder-setup脚本产生大量输出，你应该注意其中一些：

·Client–Server Password（客户端–服务器密码）：这是工作者用于向控制器识别自己的密码。这个密码被用在控制配置文件yocto-controller/controller.cfg和工作者配置文件yocto-worker/buildbot.tac中。

·User Name and Password（用户名和密码）：脚本为Web用户界面创建用户名和密码，并且把它们存储在文件yocto-autobuilder/.htpasswd中。密码是以加密的形式存储的，所以你需要记录下密码。如果你忘记或者丢失了密码，那么你可以用以下命令创建一个新的并且把它写到密码文件中：

```
$ cd yocto-autobuilder
$ ./bin/htpasswd -b .htpasswd <username> <password>
```

确保你使用了自动构建器提供的./bin/htpasswd命令而不是安装在你的系统上的那个。

·Environment Variables（环境变量）：脚本增加自动构建器路径到环境变量PYTHONPATH和PATH，设置变量YOCTO_AB_CONFIG，并且打印它们的值到控制台。你可以复制和粘贴设置到你的.bashrc文件，

或者你可以在每次希望使用自动构建器的时候引用脚本。再次引用脚本是安全的。它检测前面的设置，例如现存的配置和密码文件，并且不会覆盖它们。

现在你可以用以下命令启动自动构建器：

```
$ ./auto-start-autobuilder both
```

它在相同的节点上启动了一个控制器和一个工作者。脚本接受以下参数：

·both（两者）：启动控制器和工作者。

·controller（控制器）：仅仅启动控制器。

·worker（工作者）：仅仅启动工作者。

为了关闭自动构建器，使用yocto-stop-autobuilder，它和启动脚本接受相同的参数。

在启动了自动构建器后，把你的浏览器指向：

```
localhost:8010
```

它把你带到自动构建器登录页面。从那里开始，你可以使用你的用户名和密码登录自动构建器。

单击Builders链接，它把你带到所有已配置的构建器列表。单击一个，例如nightly-x86-64。为了在其周期调度之外强制启动一个构建器，单击Force Build按钮。一旦构建器被启动，你可以在Waterfall视图中观察它的进度。

13.4.2 配置自动构建器

自动构建器配置是通过一系列配置文件来完成的。这些不是BitBake配置文件。那意味着，没有BitBake变量扩展可供你使用。

自动构建器全局配置文件

文件config/autobuilder.conf是全局的自动构建器配置文件。文件被划分为几个部分。每个部分是通过中括号内的部分名来引入的。每个部分包含一个或者多个配置变量。部分名和变量名大部分是不用解释的。这里，我们讨论你最常希望调整的参数。

·[GitSettings]：处理Git仓库。

·OPTIMIZED_GIT_CLONE：如果设置为True，那么Git仓库在使用后被移动到临时存储位置而不是从自动构建器中删除。这加速了构建过程，但是需要用定时任务来执行对过时目录的外部清理。默认设置是True。然而，这个设置是相当耗费磁盘空间的。如果你的工作者磁盘空间不多，那么你可能希望考虑禁用这个设置。

·OGIT_TRASH_DIR：过时的Git仓库被移动到的目录。

·OGIT_MIRROR_DIR：克隆Git仓库所到的目录。

·OGIT_TRASH_CRON_TIME：用于清理过时Git仓库的定时任务设置。

·OGIT_TRASH_NICE_LEVEL：清理任务的优先级。

·[BuildHistorySettings]：自动构建器是否收集构建历史以及在哪里

存储它。

·BUILD_HISTORY_COLLECT：如果设置为True，那么自动构建器收集构建历史。

·BUILD_HISTORY_DIR：构建历史被存储在的目录。

·BUILD_HISTORY_REPO：用于构建历史的远程仓库。

·[ErrorReportSettings]：自动构建器是否收集、存储和发布有关自动构建器遇到的错误的报告。

·ERROR_REPORT_COLLECT：如果设置为True，自动构建器收集错误报告。

·ERROR_REPORT_EMAIL：错误报告被发送到的邮箱地址。

·[PublishSettings]：是否以及向哪里发布构建制品。

·PUBLISH_BUILDS：如果设置成True，那么自动构建器发布镜像和包仓库到MACHINE_PUBLISH_DIR、QEMU_PUBLISH_DIR、RPM_PUBLISH_DIR、DEB_PUBLISH_DIR和IPK_PUBLISH_DIR，它们是BUILD_PUBLISH_DIR的子目录。

·PUBLISH_SOURCE_MIRROR：如果设置为True，自动构建器发布要用于一个源镜像的源文件到SOURCE_PUBLISH_DIR。

·PUBLISH_SSTATE：如果设置成True，自动构建器发布共享状态环境到SSTATE_PUBLISH_DIR。

·[BuildSettings]：用于被工作者使用的构建环境的conf/local.conf文件的设置。

·[QAEmail]：自动构建器应该发送关于构建结果的邮件所到的邮箱地址。

通常，你可能希望调整用于构建制品的目录而希望其他你需要的数据保留。默认设置把所有文件存储在/tmp/yocto-autobuilder，当系统重启时，这个目录会丢失。

控制器配置文件

文件yocto-controller/controller.cfg包含用于控制器的配置设置。这个文件使用Python语法。所有配置被存储在一个名为BuildmasterConfig的字典中。这些是由位于http://docs.buildbot.net的Buildbot文档详细解释的Buildbot配置设置。我们在此覆盖一些最重要的内容：

·c['debugPassword']：如果设置了，那么你可以使用Buildbot调试客户端来连接控制器。

·c['title']：显示在自动构建器网页上部的标题。

·c['titleURL']：嵌入在标题中的URL（通常匹配c['buildbotURL']）。

·c['buildbotURL']：自动构建器的Web服务器监听到的URL、主机和端口。

·c['workers']：被控制器识别的工作者列表。每个工作者必须有一个唯一的名字和其用于向控制器进行身份验证的密码。工作者名字和密码必须匹配工作者配置中的对应值。

·c['workerPortnum']：控制器监听用于接受工作者连接的TCP端口号。端口号必须匹配工作者配置的端口号码。

·c['status']：自动构建器向其发布构建状态报告的状态目标列表。

Buildbot提供大量状态目标，例如网页、邮件发送程序和互联网中继聊天（Internet Relay Chat，IRC）机器人。Buildbot文档包含关于如何配置各种状态目标的细节。

·c['db']：自动构建器用于存储它状态信息的数据库。默认是SQLite数据库。其他数据库（包括MySQL和PostgreSQL）是可以配置的。Buildbot文档解释了细节。为了性能原因，你可能希望为生产环境使用RDBMS而不是SQLite。

你可在文件中设置其他配置选项，例如c['multiMaster']，它允许使用多个控制器来创建更加可伸缩的构建工厂。

工作者配置文件

文件yocto-worker/buildbot.tac包含工作者信息。这个文件也使用Python语法。这些是你需要调整以创建分布式系统的设置：

·buildmaster_host：控制器正在运行的主机名或者主机的IP地址。

·port：控制器监听用以接收工作者连接的端口号。值必须匹配控制器配置的c['workerPortnum']。

·workername：用于工作者的唯一性名字。值必须匹配在控制器配置的c['workers']中的工作者的名字。

·passwd：用于和控制器进行身份验证的密码。值必须匹配用于在控制器配置的c['workers']中的工作者的密码。

构建集配置

自动构建器根目录包含多个其名字全部以buildset-开头的目录。在Buildbot语言中，构建集是一系列按照构建集定义的顺序执行的步骤。

程序清单13-6显示了用于每夜x86_64构建的自动构建器构建集。

程序清单13-6　nightly-x86-64构建集（buildset-config/nightly-x86-64.conf）

```
[nightly-x86-64]
builders: 'example-worker'
repos: [{'poky':
            {'repourl':'git://git.yoctoproject.org/poky',
             'layerversion': {'core':'meta', 'yoctobsp':'meta-yocto-bsp'},
             'branch':'master'}}]
steps: [{'SetDest': {}},
        {'CheckOutLayers': {}},
        {'RunPreamble': {}},
        {'GetDistroVersion': {'distro': 'poky'}},
        {'CreateAutoConf': {'machine': 'qemux86-64', 'SDKMACHINE': 'i686',
                            'distro': 'poky', 'buildhistory': True}},
        {'CreateBBLayersConf': {'buildprovider': 'yocto'}},
        {'SyncPersistDB': {'distro': 'poky'}},
        {'GetBitbakeVersion': {}},
        {'BuildImages': {'images': 'core-image-sato core-image-sato-dev
                                    core-image-sato-sdk core-image-minimal
                                    core-image-minimal-dev'}},
        {'RunSanityTests': {'images': 'core-image-minimal core-image-sato
                                       core-image-sato-sdk'}},
        {'CreateAutoConf': {'machine': 'genericx86-64',
                            'SDKMACHINE': 'i686',
                            'buildhistory': False, 'distro': 'poky'}},
        {'BuildImages': {'images': 'core-image-sato core-image-sato-dev
                                    core-image-sato-sdk core-image-minimal
                                    core-image-minimal-dev'}},
        {'CreateAutoConf': {'machine': 'qemux86-64',
                            'SDKMACHINE': 'i686',
                            'distro': 'poky', 'buildhistory': False}},
        {'BuildToolchainImages': {}},
        {'RunSDKSanityTests': {'images': 'core-image-sato'}},
        {'CreateAutoConf': {'machine': 'qemux86-64',
                            'SDKMACHINE': 'x86_64',
                            'distro': 'poky', 'buildhistory': False}},
        {'BuildToolchainImages': {}},
        {'RunSDKSanityTests': {'images': 'core-image-sato'}},
        {'SyncPersistDB': {'commit': True, 'distro':'poky'}},
        {'PublishLayerTarballs': {}},
        {'SendErrorReport': {}},
        {'UploadToasterEventlog': {}},
        {'PublishArtifacts': {'artifacts': ['qemux86-64', 'genericx86-64',
                              'ipk', 'toolchain', 'md5sums']}}]
```

构建集通常至少包含这些元素：

·Buildset　Name（构建集名字）：在中括号（[]）内的构建集的名字。

·Repos（仓库）：包含要监控的仓库描述的字典列表。仓库描述本身就是一个字典，它的键是repourl、layerversion和branch。

·Steps（步骤）：构建步骤字典列表。构建步骤是位于自动构建器

根目录的lib/python2.7/site-packages/autobuilder/buildsteps中的Python类。步骤可能接受参数，它们是作为字典来提供的。

Buildbot包含关于如何创建构建集和构建步骤的通用例子和解释。文件README-NEW-AUTOBUILDER解释自动构建器构建集合构建步骤是如何被配置的。

13.5 总结

在本章中，我们描述了你可以用来伸缩Yocto项目构建系统到开发团队和生产环境的工具和技术。

·Toaster用Web用户界面来扩展了构建系统，它允许远程部署和共享的构建资源。

·构建历史提供对构建配置和构建输出的追踪。它是维护构建质量和可重复性的重要工具。由一个初始构建创建的基线（baseline）开始，对配置和元数据制品的变更被保存在Git仓库中以创建因果关系的无缝历史。

·通过使用源镜像，开发团队可以共享源下载，并且生产环境可以控制用于生产部署的软件包镜像是从什么源镜像中创建的。

·自动构建器为Yocto项目构建提供自动化的持续构建和集成系统。它是一个完全开箱即用（out-of-the-box）的解决方案，它已经准备好在短时间内被部署。它的默认构建集覆盖所有标准Yocto项目构建目标。它们是一个坚实的基础，可被轻松扩展和调整到适合你自己的需求。

13.6 参考文献

Buildbot文档，http://docs.buildbot.net。

Yocto项目自动构建器，https://www.yoctoproject.org/tools-resources/projects/autobuilder。

Yocto项目参考手册，www.yoctoproject.org/docs/2.0/ref-manual/ref-manual.html。

Yocto项目Toaster手册，www.yoctoproject.org/docs/2.0/toaster-manual/toaster-manual.html。

附录A　开源许可协议

开源许可的列表是相当长的。OpenEmbedded构建系统在它的meta/files/common-licenses目录中提供用到的173种许可的文本。开源组织（Open Source Initiative，OSI）[1]分析和审查许可并且发布已批准的开源许可列表，这样做的目的是，向用户、开发者、商业和政府机构提供对开源许可的教育。作为参考，最常用的4个开源许可的逐字文本在本附录中提供。

A.1　MIT License（MIT）

The MIT License (MIT)
Copyright (c) <year> <copyright holders>
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR

OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

A.2　GNU General Public License（GPL）Version 2

GNU GENERAL PUBLIC LICENSE
Version 2, June 1991
Copyright (C) 1989, 1991 Free Software Foundation, Inc.
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change free software—to make sure the software is free for all its users. This General Public License applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by the GNU Library General Public License instead.) You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that redistributors of a free program will individually obtain patent licenses, in effect making the program proprietary. To prevent this, we have made it

clear that any patent must be licensed for everyone's free use or not licensed at all.

The precise terms and conditions for copying, distribution and modification follow.

Terms and Conditions for Copying, Distribution and Modification

0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term "modification".) Each licensee is addressed as "you".

   Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program.

   You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

   a. You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.

   b. You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.

   c. If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:

   a. Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

   b. Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

   c. Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

   The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

   If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

4. You may not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

   If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

   It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

   This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who

places the Program under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

9. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

   Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and "any later version," you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation.

10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

No Warranty

11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR

A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

> One line to give the program's name and a brief idea of what it does.
>
> Copyright (C) <year> <name of author>
>
> This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.
>
> This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANT-ABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.
>
> You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA

Also add information on how to contact you by electronic and paper mail.

If the program is interactive, make it output a short notice like this when it starts in an interactive mode:

> Gnomovision version 69, Copyright (C) year name of author Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type `` `show w' ``. This is free software, and you are welcome to redistribute it under certain conditions; type `` `show c' `` for details.

The hypothetical commands `` `show w' `` and `` `show c' `` should show the appropriate parts of the General Public License. Of course, the commands you use may be called something other than `` `show w' `` and `` `show c' ``; they could even be mouse-clicks or menu items—whatever suits your program.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the program, if necessary. Here is a sample; alter the names:

> Yoyodyne, Inc., hereby disclaims all copyright interest in the program `` `Gnomovision' `` (which makes passes at compilers) written by James Hacker.

signature of Ty Coon, 1 April 1989

Ty Coon, President of Vice

This General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Library General Public License instead of this License.

A.3 GNU General Public License（GPL）Version 3

GNU GENERAL PUBLIC LICENSE
Version 3, 29 June 2007
Copyright (C) 2007 Free Software Foundation, Inc. <http://fsf.org/>
Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program—to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed

erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

Terms and Conditions

0. **Definitions.**

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License.

If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. **Source Code.**

   The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

   A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

   The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.
   The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

   The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

   The Corresponding Source for a work in source code form is that same work.

2. **Basic Permissions.**

   All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

   You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all

material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. **Protecting Users' Legal Rights From Anti-Circumvention Law.**

   No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

   When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. **Conveying Verbatim Copies.**

   You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

   You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. **Conveying Modified Source Versions.**

   You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

   a. The work must carry prominent notices stating that you modified it, and giving a relevant date.

   b. The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".

   c. You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

d. If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. **Conveying Non-Source Forms.**

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

a. Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.

b. Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.

c. Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.

d. Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.

e. Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. **Additional Terms.**

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used

separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

a. Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or

b. Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or

c. Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

d. Limiting the use for publicity purposes of names of licensors or authors of the material; or

e. Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

f. Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. **Termination.**

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. **Acceptance Not Required for Having Copies.**

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. **Automatic Licensing of Downstream Recipients.**

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you

may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. **Patents.**

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may

not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. **No Surrender of Others' Freedom.**

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. **Use with the GNU Affero General Public License.**

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. **Revised Versions of this License.**

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU

General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. **Disclaimer of Warranty.**

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. **Limitation of Liability.**

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. **Interpretation of Sections 15 and 16.**

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to

the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

> <one line to give the program's name and a brief idea of what it does.>
> Copyright (C) <year> <name of author>
>
> This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.
>
> This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.
>
> You should have received a copy of the GNU General Public License along with this program. If not, see <http://www.gnu.org/licenses/>.

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

> <program> Copyright (C) <year> <name of author>
> This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
> This is free software, and you are welcome to redistribute it under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <http://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <http://www.gnu.org/philosophy/why-not-lgpl.html>.

A.4 Apache License Version 2.0

Apache License
Version 2.0, January 2004
http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. **Definitions.**

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf

of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. **Grant of Copyright License.**

   Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. **Grant of Patent License.**

   Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. **Redistribution.**

   You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

   a. You must give any other recipients of the Work or Derivative Works a copy of this License; and

   b. You must cause any modified files to carry prominent notices stating that You changed the files; and

   c. You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

   d. If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only

and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. **Submission of Contributions.**

   Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. **Trademarks.**

   This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. **Disclaimer of Warranty.**

   Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. **Limitation of Liability.**

   In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. **Accepting Warranty or Additional Liability.**

   While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or

other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to Apply the Apache License to Your Work

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS,WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

[1] https://opensource.org。

附录B　元数据参考

表B-1和B-2分别列出层和机器，它们显示在接下来的页面中。为了获得可搜索的参考，请访问 http://layers.openembedded.org/layerindex/branch/master/layers/。

表B-1　层

层名	描述	类型	仓库
meta-oe	Additional shared OpenEmbedded (OE) metadata	Base	git://git.openembedded.org/meta-openembedded
openembedded-core	Core metadata	Base	git://git.openembedded.org/openembedded-core
e100-bsp	Ettus E1XX series board support package (BSP)	Machine (BSP)	git://github.com/EttusResearch/meta-ettus.git
e300-bsp	Ettus E3XX Series BSP	Machine (BSP)	https://github.com/EttusResearch/meta-ettus.git
meta-aarch64	AArch64 (64-bit ARM) architecture support	Machine (BSP)	git://git.linaro.org/openembedded/meta-linaro.git
meta-acer	Acer machines support	Machine (BSP)	git://github.com/shr-distribution/meta-smartphone.git
meta-altera	Altera SoC BSP layer	Machine (BSP)	https://github.com/kraj/meta-altera
meta-amd	AMD board support common layer (official)	Machine (BSP)	git://git.yoctoproject.org/meta-amd
meta-asus	Asus machines support	Machine (BSP)	git://github.com/shr-distribution/meta-smartphone.git
meta-atmel	Official Yocto Project layer for Atmel SoCs	Machine (BSP)	git://github.com/linux4sam/meta-atmel.git
meta-baldeagle	AMD Bald Eagle platform BSP	Machine (BSP)	http://git.yoctoproject.org/cgit/cgit.cgi/meta-amd/
meta-beagleboard	Support for beagleboard.org devices	Machine (BSP)	git://github.com/beagleboard/meta-beagleboard.git
meta-bug	BugLabs bug20 machine support	Machine (BSP)	git://github.com/buglabs/meta-bug.git
meta-bytesatwork	Official BSP layer for Bytesatwork-based platform	Machine (BSP)	https://github.com/bytesatwork/meta-bytesatwork.git
meta-chip	Yocto BSP layer for CHIP boards	Machine (BSP)	https://github.com/agherzan/meta-chip.git
meta-ci20	Meta layer to support Creator CI20 MIPS boards	Machine (BSP)	https://github.com/akuster/meta-ci20
meta-crownbay	Intel Crown Bay platform BSP	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-crystalforest	Intel Crystal Forest platform BSP	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-cubox	SolidRun CuBox platform BSP	Machine (BSP)	git://github.com/dv1/meta-cubox.git
meta-digi-arm	Support for Digi ConnectCore/ConnectCard modules	Machine (BSP)	https://github.com/dgii/meta-digi.git
meta-dra7xx-evm	Tizen BSP layer for DRA7XX-EVM	Machine (BSP)	https://github.com/vitalyvch/meta-dra7xx-evm.git
meta-efikamx	Genesi Efika MX machine support	Machine (BSP)	git://github.com/kraj/meta-efikamx.git

meta-emenlow	Intel eMenlow platform BSP	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-exynos	BSP layer for Exynos-based machines	Machine (BSP)	https://github.com/slimlogic/meta-exynos.git
meta-fri2	Kontron Fish River Island 2 BSP	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-fsl-arm	Freescale ARM hardware support	Machine (BSP)	git://git.yoctoproject.org/meta-fsl-arm
meta-fsl-arm-extra	Freescale ARM hardware support (extra boards)	Machine (BSP)	git://github.com/Freescale/meta-fsl-arm-extra.git
meta-fsl-ppc	Freescale PowerPC hardware support	Machine (BSP)	git://git.yoctoproject.org/meta-fsl-ppc
meta-geeksphone	Geeksphone device support	Machine (BSP)	git://github.com/shr-distribution/meta-smartphone.git
meta-gumstix	Gumstix board support (official)	Machine (BSP)	git://github.com/gumstix/meta-gumstix.git
meta-gumstix-community	Gumstix board support (unofficial community BSP)	Machine (BSP)	https://github.com/schnitzeltony/meta-gumstix-community
meta-gumstix-extras	Gumstix board support extras (official)	Machine (BSP)	https://github.com/gumstix/meta-gumstix-extras.git
meta-handheld	Older handheld device support (Zaurus, iPAQ, etc.)	Machine (BSP)	git://git.openembedded.org/meta-handheld
meta-hipos	BSP and related Distro for embedded devices from the DResearch Fahrzeugelektronik GmbH	Machine (BSP)	git://github.com/DFE/meta-hipos.git
meta-htc	HTC smartphone support	Machine (BSP)	git://github.com/shr-distribution/meta-smartphone.git
meta-igep	IGEP board support	Machine (BSP)	git://github.com/ebutera/meta-igep.git
meta-intel	Intel board support common layer (official)	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-intel-edison-bsp	Intel Edison module support	Machine (BSP)	git://git.yoctoproject.org/meta-intel-edison
meta-intel-quark	Intel Quark platform support	Machine (BSP)	git://git.yoctoproject.org/meta-intel-quark
meta-ivi-bsp	BSP layer for IVI reference hardware	Machine (BSP)	git://git.yoctoproject.org/meta-ivi
meta-jasperforest	Intel Jasper Forest platform BSP	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-jz-mips	BSP for devices with Ingenic MIPS SoC	Machine (BSP)	https://github.com/leon-anavi/meta-jz-mips.git
meta-kirkwood	Marvell Kirkwood-based device support	Machine (BSP)	git://github.com/kelvinlawson/meta-kirkwood.git
meta-lsi	LSI Axxia Communication Processors Platform BSP	Machine (BSP)	git://git.yoctoproject.org/meta-lsi

（续）

层名	描述	类型	仓库
meta-minnow	MinnowBoard BSP layer	Machine (BSP)	git://git.yoctoproject.org/meta-minnow
meta-nanopi	FriendlyARM NanoPi board support	Machine (BSP)	git://git.kernelconcepts.de/meta-nanopi.git
meta-netbookpro	Psion NetBook Pro device support	Machine (BSP)	git://github.com/tworaz/meta-netbookpro
meta-netmodule	BSP Layer from NetModule for various boards	Machine (BSP)	https://github.com/netmodule/meta-netmodule.git
meta-nokia	Nokia N900 support	Machine (BSP)	git://github.com/shr-distribution/meta-smartphone.git
meta-nslu2	Linksys NSLU2 device support	Machine (BSP)	git://github.com/kraj/meta-nslu2
meta-nuc	Intel Next Unit of Computing BSP	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-odroid	Supports the series of Odroid boards made by Hardkernel	Machine (BSP)	https://github.com/akuster/meta-odroid
meta-openmoko	OpenMoko device support	Machine (BSP)	git://github.com/shr-distribution/meta-smartphone.git
meta-openpandora	OpenPandora machine support	Machine (BSP)	git://github.com/openpandora/meta-openpandora.git
meta-ouya	Support Ouya game console	Machine (BSP)	https://github.com/pwgen/meta-ouya
meta-palm	Palm Pre machine support	Machine (BSP)	git://github.com/shr-distribution/meta-smartphone.git
meta-parallella	Support for the Parallella board from Adapteva	Machine (BSP)	https://github.com/nathanrossi/meta-parallella.git
meta-phytec	Phytec board support (official)	Machine (BSP)	git://git.phytec.de/meta-phytec
meta-picosam9	Picosam9 board support	Machine (BSP)	https://gitorious.org/picopc-tools/meta-picosam9.git
meta-qemu-bsps	Layer for other QEMU machines	Machine (BSP)	https://github.com/akuster/meta-qemu-bsps
meta-raspberrypi	Raspberry Pi board support	Machine (BSP)	git://git.yoctoproject.org/meta-raspberrypi
meta-renesas-rza1	Official BSP layer for Renesas RZ/A1 platforms	Machine (BSP)	git://github.com/renesas-rz/meta-renesas-rza1
meta-rockchip	Layer supporting public Rockchip-based development boards and products	Machine (BSP)	git://git.yoctoproject.org/meta-rockchip
meta-romley	Intel Romley platform BSP	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-samsung	Samsung smartphone support	Machine (BSP)	git://github.com/shr-distribution/meta-smartphone.git
meta-sugarbay	Intel Sugar Bay platform BSP	Machine (BSP)	git://git.yoctoproject.org/meta-intel
meta-sunxi	Allwinner sunxi board support	Machine (BSP)	https://github.com/linux-sunxi/meta-sunxi

meta-ti	Texas Instruments board support (official)	Machine (BSP)	git://git.yoctoproject.org/meta-ti
meta-via-vab820-bsp	VIA VAB-820/AMOS-820 BSP layer	Machine (BSP)	git://github.com/viaembedded/meta-via-vab820-bsp.git
meta-xilinx	Xilinx hardware support	Machine (BSP)	git://git.yoctoproject.org/meta-xilinx
meta-xilinx-community	Additional Xilinx hardware support	Machine (BSP)	git://git.yoctoproject.org/meta-xilinx-community
meta-yocto-bsp	BSP layer for Yocto Project reference hardware	Machine (BSP)	git://git.yoctoproject.org/meta-yocto
meta-yocto-bsp-old	Former Yocto Project reference BSPs	Machine (BSP)	git://git.yoctoproject.org/meta-yocto-bsp-old
meta-angstrom	Ångström distribution	Distribution	git://github.com/Angstrom-distribution/meta-angstrom.git
meta-arago-distro	Arago/TI-SDK distribution	Distribution	git://arago-project.org/git/meta-arago.git
meta-debian	Using Debian source code to BitBake images	Distribution	git://github.com/meta-debian/meta-debian.git
meta-digi-dey	Provides the Digi Embedded Linux distribution images	Distribution	https://github.com/dgii/meta-digi
meta-eca	Mobile access point	Distribution	git://git.yoctoproject.org/meta-eca
meta-eldk	Denx ELDK distribution	Distribution	git://git.denx.de/eldk.git
meta-guacamayo	Guacamayo distribution	Distribution	git://github.com/Guacamayo/meta-guacamayo.git
meta-intel-edison-distro	Distro layer for official Intel Edison images	Distribution	git://git.yoctoproject.org/meta-intel-edison
meta-intel-iot-devkit	Distro layer for the Intel IoT Developer Kit	Distribution	git://git.yoctoproject.org/meta-intel-iot-devkit
meta-ivi	Collection of software related to in-vehicle infotainment systems	Distribution	git://git.yoctoproject.org/meta-ivi
meta-luneos	Distro layer for LuneOS	Distribution	https://github.com/webOS-ports/meta-webos-ports
meta-luv	luvOS distribution	Distribution	git://git.yoctoproject.org/meta-luv
meta-mel	Mentor Embedded Linux distribution (from Mentor Graphics Corporation)	Distribution	https://github.com/MentorEmbedded/meta-mentor
meta-micro	Micro distribution	Distribution	git://git.openembedded.org/meta-micro

（续）

层名	描述	类型	仓库
meta-mmmpi	An example distro layer customized for the Raspberry Pi	Distribution	https://bitbucket.org/mmmpi/meta-mmmpi.git
meta-overc	Full-featured, container-based distribution	Distribution	git://github.com/WindRiver-OpenSourceLabs/meta-overc.git
meta-shr-distro	SHR distribution	Distribution	git://github.com/shr-distribution/meta-smartphone.git
meta-slugos	SlugOS distribution	Distribution	git://github.com/kraj/meta-slugos
meta-tizen	Tizen reference for the Yocto Project	Distribution	git://review.tizen.org/scm/bb/meta-tizen
meta-webos	WebOS distribution (official)	Distribution	git://github.com/openwebos/meta-webos
meta-webos-ports	Distro layer for LuneOS	Distribution	https://github.com/webOS-ports/meta-webos-ports
meta-woce	WebOS Community Edition distribution	Distribution	git://github.com/kraj/meta-woce
meta-yocto	Poky reference distribution for the Yocto Project	Distribution	git://git.yoctoproject.org/meta-yocto
meta-alt-desktop-extras	Lightweight (legacy) X desktop, tools, and recipe extensions	Miscellaneous	https://github.com/sarnold/meta-alt-desktop-extras
meta-axon	Using Erlang on YP	Miscellaneous	https://github.com/joaohf/meta-axon
meta-baryon	Baryon example NAS distribution	Miscellaneous	git://git.yoctoproject.org/meta-baryon
meta-cgl	Carrier Grade Linux compliance packages	Miscellaneous	git://git.yoctoproject.org/meta-cgl
meta-darwin	OE meta layer for darwin-based SDKs	Miscellaneous	git://git.yoctoproject.org/meta-darwin
meta-gir	GObject Introspection Support	Miscellaneous	git://github.com/meta-gir/meta-gir.git
meta-intel-galileo	Intel Galileo platform support	Miscellaneous	git://git.yoctoproject.org/meta-intel-galileo
meta-kernel-dev	Kernel development extras	Miscellaneous	git://git.yoctoproject.org/meta-yocto-kernel-extras
meta-linaro	Linaro layer for OE	Miscellaneous	git://git.linaro.org/openembedded/meta-linaro.git
meta-mentor-staging	Mentor Graphics Staging Layer for Yocto/OE	Miscellaneous	https://github.com/MentorEmbedded/meta-mentor
meta-mingw	OE meta layer for mingw-based SDKs	Miscellaneous	git://git.yoctoproject.org/meta-mingw
meta-pareon	This layer automates using Pareon Verify with OE	Miscellaneous	https://github.com/vectorfabrics/meta-pareon
meta-ro-rootfs	Staging area for improvements to read-only-rootfs handling	Miscellaneous	https://github.com/MentorEmbedded/meta-ro-rootfs

meta-security-isafw	Image Security Analyser FW layer	Miscellaneous	https://github.com/01org/meta-security-isafw
meta-security-rockwell	OE layer that assists developers in hardening their Yocto-built systems, focusing on the Daisy release	Miscellaneous	https://github.com/IrdetoServices/meta-security-rockwell.git
meta-tlk	Time-limited kernel layer	Miscellaneous	git://git.yoctoproject.org/meta-intel
meta-ada	Ada support	Software	git://github.com/Lucretia/meta-ada.git
meta-android	Android-specific tools	Software	git://github.com/shr-distribution/meta-smartphone.git
meta-arago-extras	Arago/TI extra apps	Software	git://arago-project.org/git/meta-arago.git
meta-aspnet	ASP.NET 5	Software	git://github.com/Tragetaschen/meta-aspnet.git
meta-audio	Contains recipes for various audio DSP tools	Software	https://github.com/errordeveloper/oe-meta-audio
meta-aurora	Aurora UI	Software	git://github.com/shr-distribution/meta-smartphone.git
meta-beagleboard-extras	BeagleBoard extras	Software	git://github.com/beagleboard/meta-beagleboard
meta-browser	Web browsers (Chromium, Firefox, etc.)	Software	git://github.com/OSSystems/meta-browser.git
meta-buglabs	BugLabs middleware and specific OSGi components	Software	git://github.com/buglabs/meta-buglabs.git
meta-chicken	Chicken toolchain support	Software	git://github.com/OSSystems/meta-chicken
meta-ci	The OE/Yocto BSP layer for CI-related things	Software	https://github.com/koenkooi/meta-ci.git
meta-clang	Clang/LLVM alternative to gcc for C/C++ cross compiler	Software	https://github.com/kraj/meta-clang
meta-cloud-services	General package support for cloud and clustering development	Software	git://git.yoctoproject.org/meta-cloud-services
meta-clutter	Recipes for Cogl, Clutter, and friends	Software	git://github.com/Guacamayo/meta-clutter.git
meta-cpan	CPAN distributions as meant by CPAN authors	Software	https://github.com/rehsack/meta-cpan.git
meta-crosswalk	Web runtime for ambitious HTML5 applications	Software	git://github.com/crosswalk-project/meta-crosswalk
meta-efl	Enlightenment UI support	Software	git://git.openembedded.org/meta-openembedded

（续）

层名	描述	类型	仓库
meta-erlang	Erlang support	Software	https://github.com/joaohf/meta-erlang
meta-filesystems	Support for additional filesystems	Software	git://git.openembedded.org/meta-openembedded
meta-fsl-demos	OE/Yocto layer for Freescale's demonstration images	Software	https://github.com/Freescale/meta-fsl-demos.git
meta-fso	Freesmartphone.org framework support	Software	git://github.com/shr-distribution/meta-smartphone.git
meta-game-emulators	Software layer for video game system emulators	Software	https://github.com/sergioprado/meta-game-emulators
meta-games	Open source games	Software	git://github.com/cazfi/meta-games.git
meta-gnome	GNOME UI support	Software	git://git.openembedded.org/meta-openembedded
meta-go	Provides support for cross-compiling programs written in Go language	Software	https://github.com/errordeveloper/oe-meta-go
meta-gpe	GPE UI support (very minimal currently)	Software	git://git.openembedded.org/meta-openembedded
meta-gstreamer10	OE layer for GStreamer 1.0	Software	git://github.com/dv1/meta-gstreamer1.0.git
meta-hamradio	Ham radio–related software	Software	https://github.com/hambedded-linux/meta-hamradio.git
meta-initramfs	initramfs tools	Software	git://git.openembedded.org/meta-openembedded
meta-intel-iot-middleware	Shared middleware recipes for Intel IoT platforms	Software	git://git.yoctoproject.org/meta-intel-iot-middleware
meta-iot	Provides support for building an Internet of Things device manager and translator	Software	https://github.com/cablelabs/meta-iot
meta-ivi-demo	GENIVI Yocto Demo Platform	Software	git://git.yoctoproject.org/meta-ivi
meta-java	Java support	Software	git://git.yoctoproject.org/meta-java
meta-kali	Kali penetration test tools	Software	https://github.com/akuster/meta-kali
meta-kde4	KDE 4/Plasma Active	Software	https://bitbucket.org/gen_dev_sst/meta-kde4.git
meta-kf5	KDE Frameworks 5	Software	https://github.com/e8johan/meta-kf5.git
meta-linaro-toolchain	Linaro toolchain support	Software	git://git.linaro.org/openembedded/meta-linaro.git
meta-luneui	Recipes for LuneOS UI	Software	https://github.com/webOS-ports/meta-webos-ports

meta-lxcbench	Yocto layer for the LXCBENCH project	Software	git://git.projects.genivi.org/lxcbench.git
meta-maker	OE recipes for makers	Software	http://git.yoctoproject.org/git/meta-maker
meta-measured	Tools and utilities related to measuring software and trusted computing	Software	https://github.com/flihp/meta-measured.git
meta-mel-support	Mentor Embedded Linux Support Layer for Yocto/OE	Software	https://github.com/MentorEmbedded/meta-mentor
meta-mono	Mono	Software	git://git.yoctoproject.org/meta-mono
meta-multimedia	Multimedia-related software	Software	git://git.openembedded.org/meta-openembedded
meta-musl	Support for the musl C library	Software	https://github.com/kraj/meta-musl.git
meta-netmodule-extras	Additional packets used by NetModule	Software	https://github.com/netmodule/meta-netmodule-extras.git
meta-networking	Network-related software	Software	git://git.openembedded.org/meta-openembedded
meta-office	Office applications and libs	Software	git://github.com/schnitzeltony/meta-office.git
meta-oic	Support for building the Open Interconnect Consortium Iotivity framework	Software	git://git.yoctoproject.org/meta-oic
meta-openclovis	Layer to support SAFplus middleware components	Software	https://github.com/joaohf/meta-openclovis
meta-openhab	Yocto layer providing openHAB (home automation software)	Software	https://github.com/ulfwin/meta-openhab
meta-openstack	OpenStack support	Software	git://git.yoctoproject.org/meta-cloud-services
meta-openstack-compute-deploy	OpenStack compute node support	Software	git://git.yoctoproject.org/meta-cloud-services
meta-openstack-controller-deploy	OpenStack controller node support	Software	git://git.yoctoproject.org/meta-cloud-services
meta-openstack-qemu	OpenStack QEMU image support	Software	git://git.yoctoproject.org/meta-cloud-services
meta-openwrt	OE metadata for OpenWRT packages	Software	https://github.com/kraj/meta-openwrt
meta-opie	Opie UI support	Software	git://git.openembedded.org/meta-opie

（续）

层名	描述	类型	仓库
meta-oracle-java	Oracle Java support	Software	git://git.yoctoproject.org/meta-oracle-java
meta-osmocombb	OsmocomBB support	Software	git://github.com/shr-distribution/meta-smartphone.git
meta-perl	Additional Perl recipes	Software	git://git.openembedded.org/meta-openembedded
meta-pypy	Cross/Embedded PyPy (https://bitbucket.org/pypy/pypy) support	Software	https://github.com/mzakharo/meta-pypy.git
meta-python	Python support	Software	git://git.openembedded.org/meta-openembedded
meta-qt3	Qt 3.x support	Software	git://git.yoctoproject.org/meta-qt3
meta-qt4	Qt 4.x support	Software	git://git.yoctoproject.org/meta-qt4
meta-qt5	Qt5 modules	Software	git://github.com/meta-qt5/meta-qt5.git
meta-qt5-extra	Desktop environments and applications based on qt5	Software	git://github.com/schnitzeltony/meta-qt5-extra.git
meta-realtime	Additional real-time support	Software	git://git.yoctoproject.org/meta-realtime
meta-ros	Robot Operating System (ROS) support layer	Software	git://github.com/bmwcarit/meta-ros.git
meta-ruby	Ruby support	Software	git://git.openembedded.org/meta-openembedded
meta-rust	Rust compiler and package manager	Software	https://github.com/starlab-io/meta-rust.git
meta-sdr	Software-defined radio (SDR)–related recipes	Software	https://github.com/balister/meta-sdr.git
meta-security	Security tools for Internet connected devices	Software	git://git.yoctoproject.org/meta-security
meta-security-framework	Adds higher-level security middleware and tools	Software	https://github.com/01org/meta-intel-iot-security.git
meta-security-smack	Adds the Smack LSM to OE distros	Software	https://github.com/01org/meta-intel-iot-security.git
meta-selinux	SELinux support	Software	git://git.yoctoproject.org/meta-selinux
meta-shr	SHR applications	Software	git://github.com/shr-distribution/meta-smartphone.git
meta-smalltalk	GNU Smalltalk and ports	Software	git://github.com/sysmocom/meta-smalltalk.git
meta-sourcery	CodeSourcery toolchain support layer by Mentor Graphics Corporation	Software	git://github.com/MentorEmbedded/meta-sourcery.git
meta-spec	Support for the Simple PCIe FMC Carrier (SPEC)	Software	git://ohwr.org/fmc-projects/spec/spec-getting-started/meta-spec.git

meta-swupdate	Layer supporting Software Update via swupdate	Software	https://github.com/sbabic/meta-swupdate
meta-systemd	systemd support	Software	git://git.openembedded.org/meta-openembedded
meta-telephony	Telephony-related software	Software	https://github.com/sysmocom/meta-telephony
meta-telldus	TellStick support layer	Software	https://github.com/maxinbjohn/meta-telldus
meta-tracing	Layer to add necessary agent support for Sourcery Analyzer (lttng+systemtap bits)	Software	git://github.com/MentorEmbedded/meta-tracing.git
meta-uav	Software for drones	Software	https://github.com/koenkooi/meta-uav.git
meta-virtualization	Hypervisor, virtualization tool stack, and cloud support	Software	git://git.yoctoproject.org/meta-virtualization
meta-web-kiosk	Layer enabling browser-based kiosk devices with virtual keyboard support	Software	git://git.yoctoproject.org/meta-web-kiosk
meta-webkit	WebKit engines and browsers: WebKitForWayland, WebKitGTK+	Software	https://github.com/lgalia/meta-webkit
meta-webserver	Web server–related software	Software	git://git.openembedded.org/meta-openembedded
meta-wolfssl	Lightweight, portable, C-language-based SSL/TLS library	Software	https://github.com/wolfSSL/meta-wolfssl
meta-x10	x10 protocol related software	Software	git://github.com/baillaw/meta-x10.git
meta-xfce	XFCE UI support	Software	git://git.openembedded.org/meta-openembedded
toolchain-layer	Older/newer toolchains	Software	git://git.openembedded.org/meta-openembedded

表B-2　机器

机器名	描述	层
a500	Acer IconiaTab A500	**meta-acer**
akita	Sharp Zaurus SL-C1000 device	**meta-handheld**
am180x-evm	TI AM180x EVM board	**meta-ti**
am335x-evm	TI AM335x EVM	**meta-ti**
am3517-evm	TI Sitara AM3517 EVM	**meta-ti**
am37x-evm	TI AM37x EVM	**meta-ti**
am437x-evm	TI AM437x EVM	**meta-ti**
am57xx-evm	TI DRA7xx EVM	**meta-ti**
apalis-imx6	Toradex Apalis iMX6 SOM	**meta-fsl-arm-extra**
arago	Unified/fake Arago machine configuration for TI/Arago ARMv5 platforms	**meta-arago-distro**
arago-armv5	Unified/fake Arago machine configuration for TI/Arago ARMv5 platforms	**meta-arago-distro**
arago-armv7	Unified/fake Arago machine configuration for TI/Arago ARMv7 platforms	**meta-arago-distro**
arndale	Arndale board	**meta-exynos**
arndale-octa	Arndale Octa board	**meta-exynos**
arria10	Arria 10 SoC	**meta-altera**
arria5	Arria V SoC	**meta-altera**
at91sam9m10g45ek	Atmel evaluation board	**meta-atmel**
at91sam9rlek	Atmel evaluation board	**meta-atmel**
at91sam9x5ek	Atmel evaluation board	**meta-atmel**
axxiaarm	LSI Axxia ARM systems	**meta-lsi**
axxiapowerpc	LSI Axxia PowerPC systems	**meta-lsi**

b4420qds	Freescale QorIQ Qonverge B4420	**meta-fsl-ppc**
b4420qds-64b	Freescale QorIQ Qonverge B4420	**meta-fsl-ppc**
b4860qds	Freescale QorIQ Qonverge B4860	**meta-fsl-ppc**
b4860qds-64b	Freescale QorIQ Qonverge B4860	**meta-fsl-ppc**
baldeagle	Bald Eagle systems	**meta-baldeagle**
balto	Balto Board RZ/A1H	**meta-renesas-rza1**
balto-xip	Balto Board RZ/A1H	**meta-renesas-rza1**
bananapi	Banana Pi, based on Allwinner A20 CPU (http://bananapi.org)	**meta-sunxi**
beagleboard	BeagleBoard board (http://beagleboard.org)	**meta-ti**
beagleboard	BeagleBoard board	**meta-yocto-bsp-old**
beaglebone	BeagleBone board (http://beagleboard.org/bone)	**meta-ti**
beaglebone	BeagleBone and BeagleBone Black boards (http://beagleboard.org/black)	**meta-yocto-bsp**
beaglebone	BeagleBone board	**meta-beagleboard**
ben-nanonote	Qi-Hardware Ben Nanonote	**meta-handheld**
bsc9131rdb	Freescale QorIQ Qonverge BSC9131	**meta-fsl-ppc**
bsc9132qds	Freescale QorIQ Qonverge BSC9132	**meta-fsl-ppc**
bug	BUG 1.x board (imx31 based)	**meta-bug**
bug20	BUG 2.0 board (OMAP3530 based)	**meta-bug**
bytepanel	bytePANEL by bytes at work AG	**meta-bytesatwork**
c293pcie	Freescale C29x Crypto Coprocessor	**meta-fsl-ppc**
c7x0	Sharp Zaurus SL-C700, Sharp Zaurus SL-C750, Sharp Zaurus SL-C760, Sharp Zaurus SL-C860 devices	**meta-handheld**
ccardimx28js	Digi ConnectCore for MX28 JSK	**meta-digi-arm**

（续）

机器名	描述	层
ccimx51js	Digi ConnectCore for MX51 JSK	**meta-digi-arm**
ccimx53js	Digi ConnectCore for MX53 JSK	**meta-digi-arm**
cfa10036	CFA-10036	**meta-fsl-arm-extra**
cfa10037	CFA-10037	**meta-fsl-arm-extra**
cfa10049	CFA-10049	**meta-fsl-arm-extra**
cfa10055	CFA-10055	**meta-fsl-arm-extra**
cfa10056	CFA-10056	**meta-fsl-arm-extra**
cfa10057	CFA-10057, also called CFA-920	**meta-fsl-arm-extra**
cfa10058	CFA-10058, also called CFA-921	**meta-fsl-arm-extra**
cgtqmx6	Congatec QMX6 evaluation board	**meta-fsl-arm-extra**
chip	CHIP board	**meta-chip**
chromebook-snow	Arndale board	**meta-exynos**
ci20	ci20 systems	**meta-ci20**
cm-fx6	CompuLab CM-FX6 machines	**meta-fsl-arm-extra**
colibri-imx6	Toradex Colibri iMX6 SOM	**meta-fsl-arm-extra**
colibri-vf	Toradex Colibri VF50/VF61 powered by Freescale Vybrid SoC	**meta-fsl-arm-extra**
collie	SA1100 based Sharp Zaurus SL-5000 and SL-5500 devices	**meta-handheld**
creator-ci20	MIPS Creator CI20	**meta-jz-mips**
crespo	Samsung Crespo	**meta-samsung**
crownbay-noemgd	Crown Bay systems, without Intel-proprietary graphics bits	**meta-crownbay**
crystalforest	Crystal Forest Gladden systems	**meta-crystalforest**
cubieboard	Cubieboard, based on Allwinner a10 CPU (http://cubieboard.org)	**meta-sunxi**

cubieboard2	Cubieboard2, based on Allwinner A20 CPU	**meta-sunxi**
cubietruck	Cubietruck, based on Allwinner A20 CPU (www.cubietruck.com)	**meta-sunxi**
cubox	www.solid-run.com/products/cubox	**meta-cubox**
cubox-i	SolidRun CuBox-i and HummingBoard machines	**meta-fsl-arm-extra**
cyclone5	Cyclone V SoC	**meta-altera**
dra7xx-evm	TI DRA7xx EVM	**meta-ti**
dra7xx-evm	TI DRA7xx EVM	**meta-dra7xx-evm**
duovero	Gumstix DuoVero	**meta-gumstix**
edgerouter	Edgerouter	**meta-yocto-bsp**
edison	Edison systems	**meta-intel-edison-bsp**
efikamx	EFIKA MX development platform (www.powerdeveloper.org/platforms/efikamx)	**meta-efikamx**
emenlow-noemgd	eMenlow-based systems, such as the Webs-2120 box, without the Intel-proprietary graphics bits	**meta-emenlow**
ep108-zynqmp	Xilinx EP108 ZynqMP emulation platform	**meta-xilinx**
ettus-e1xx	USRP E1XX	**e100-bsp**
ettus-e300	Ettus Research E3XX SDR	**e300-bsp**
example		**meta-kernel-dev**
forfun-q88db	Forfun Q88DB tablet with A13 CPU	**meta-sunxi**
fri2-noemgd	Fish River Island 2 systems, without Intel-proprietary graphics bits	**meta-fri2**
gcw0	Game Consoles Worldwide (GCW) Zero	**meta-handheld**
geeksphone-one	Geeksphone One	**meta-geeksphone**
generic-armv4t	ARMv4T-based boards	**meta-eldk**
generic-armv5te	ARMv5TE-based boards	**meta-eldk**

（续）

机器名	描述	层
generic-armv6	ARMv6-based boards	**meta-eldk**
generic-armv7a	ARMv7a-based boards	**meta-eldk**
generic-armv7a-hf	ARMv7a-based boards	**meta-eldk**
generic-mips	MIPS-based boards	**meta-eldk**
generic-nios2	Nios II machines	**meta-altera**
generic-powerpc	Generic PowerPC machine with FPU	**meta-eldk**
generic-powerpc-4xx	APM PPC4xx-based boards (with FPU)	**meta-eldk**
generic-powerpc-4xx-softfloat	APM PPC4xx-based boards (without FPU)	**meta-eldk**
generic-powerpc-e500v2		**meta-eldk**
generic-powerpc-softfloat	Generic PowerPC machine without FPU	**meta-eldk**
genericarmv7a	Generic machine to be used by `linaro-media-create`	**meta-linaro**
genericarmv7ab	Generic machine to be used by `linaro-media-create`	**meta-linaro**
genericarmv8	Generic machine to be used by `linaro-media-create`	**meta-aarch64**
genericarmv8b	Generic machine to be used by `linaro-media-create`	**meta-aarch64**
genericx86	Generic X86 (32-bit) PCs; supports moderately wide range of drivers that should boot and be usable on typical hardware	**meta-yocto-bsp**
genericx86-64	generic X86_64 (64-bit) PCs and servers; supports moderately wide range of drivers that should boot and be usable on typical hardware	**meta-yocto-bsp**
grouper	Asus Grouper	**meta-asus**
guruplug	ARM-based Marvell SheevaPlug	**meta-kirkwood**
h1940	HP iPAQ h1930 and h1940	**meta-handheld**
h3600	Compaq iPAQ 36xx, Compaq iPAQ 37xx, and Compaq iPAQ 38xx devices	**meta-handheld**

hpveer	HP Veer handset	**meta-palm**
htcdream	HTC Dream phone (aka T-Mobile G1 and Google ADP-1)	**meta-htc**
htcleo	HTC Leo smartphone (aka HTC HD2)	**meta-htc**
hx4700	hx4700 iPAQ with a PXA27x CPU	**meta-handheld**
i9300	Samsung Galaxy SIII	**meta-samsung**
igep0020	IGEPv2 AM/DM37x processor board	**meta-igep**
igep0030	IGEP COM module AM/DM37x processor board	**meta-igep**
igep0033	IGEP COM AQUILA AM335x processor board	**meta-igep**
imx233-olinuxino-maxi	OLIMEX iMX233-OLinuXino-Maxi	**meta-fsl-arm-extra**
imx233-olinuxino-micro	OLIMEX iMX233-OLinuXino-Micro	**meta-fsl-arm-extra**
imx233-olinuxino-mini	OLIMEX iMX233-OLinuXino-Mini	**meta-fsl-arm-extra**
imx233-olinuxino-nano	OLIMEX iMX233-OLinuXino-Nano	**meta-fsl-arm-extra**
imx23evk	Freescale i.MX23 Evaluation Kit (EVK)	**meta-fsl-arm**
imx28evk	Freescale i.MX28 EVK	**meta-fsl-arm**
imx51evk	Freescale i.MX51 EVK	**meta-fsl-arm**
imx53ard	Freescale i.MX53 SABRE Automotive Board	**meta-fsl-arm**
imx53qsb	Freescale i.MX53 Quick Start Board	**meta-fsl-arm**
imx6dl-riotboard	i.MX6S RIoTboard	**meta-fsl-arm-extra**
imx6dlsabreauto	Freescale i.MX6DL SABRE Automotive	**meta-fsl-arm**
imx6dlsabresd	Freescale i.MX6DL SABRE Smart Device	**meta-fsl-arm**
imx6qpsabreauto	Freescale i.MX6QP SABRE Automotive	**meta-fsl-arm**
imx6qsabreauto	Freescale i.MX6Q SABRE Automotive	**meta-fsl-arm**

（续）

机器名	描述	层
imx6qsabrelite	Boundary Devices i.MX6Q SABRE Lite	**meta-fsl-arm-extra**
imx6qsabresd	Freescale i.MX6Q SABRE Smart Device	**meta-fsl-arm**
imx6qvab820	VAB-820	**meta-via-vab820-bsp**
imx6sl-warp	i.MX6SL WaRP board	**meta-fsl-arm-extra**
imx6slevk	Freescale i.MX6SL EVK	**meta-fsl-arm**
imx6solosabreauto	Freescale i.MX6Solo SABRE Automotive	**meta-fsl-arm**
imx6solosabresd	Freescale i.MX6Solo SABRE Smart Device	**meta-fsl-arm**
imx6sxsabreauto	Freescale i.MX6SoloX Sabre Automotive	**meta-fsl-arm**
imx6sxsabresd	Freescale i.MX6SoloX Sabre Smart Device	**meta-fsl-arm**
imx6ulevk	Freescale i.MX6UL EVK	**meta-fsl-arm**
imx7dsabresd	Freescale i.MX7D SABRE Smart Device	**meta-fsl-arm**
intel-core2-32	32-bit Intel Core 2 CPU (and later) with MMX, SSE, SSE2, SSE3, and SSSE3 instruction set support; supports moderately wide range of drivers that should boot and be usable on typical hardware	**meta-intel**
intel-corei7-64	64-bit Intel Core i7 CPU (and later) with MMX, SSE, SSE2, SSE3, and SSSE3 instruction set support; supports moderately wide range of drivers that should boot and be usable on typical hardware	**meta-intel**
intel-quark	Quark systems	**meta-intel**
jasperforest	Jasper Forest Picket Post	**meta-jasperforest**
k2e-evm	TI Keystone 2 K2E Evaluation Module (EVM)	**meta-ti**
k2g-evm	TI Keystone 2 K2G EVM	**meta-ti**
k2hk-evm	TI Keystone 2 K2HK EVM	**meta-ti**
k2l-evm	TI Keystone 2 K2L EVM	**meta-ti**
kc705-trd-microblazeel	Machine support for Xilinx KC705 Embedded Kit targeted reference design (TRD)	**meta-xilinx**

lng-rt-x86-64		meta-linaro
lng-x86-64		meta-linaro
ls1021atwr	LS1021ATWR in 32-bit mode	meta-fsl-arm
m28evk	DENX M28 SoM EVK	meta-eldk
m28evk	DENX M28 SoM EVK	meta-fsl-arm-extra
m53evk	DENX M53 SoM EVK	meta-eldk
m53evk	DENX M53 SoM EVK	meta-fsl-arm-extra
maguro	Samsung Tuna	meta-samsung
mele	Mele a1000 and a2000, based on Allwinner a10 CPU	meta-sunxi
meleg	Mele a1000g and a2000g, based on Allwinner a10 CPU	meta-sunxi
microzed-zynq7	Machine support for microZed (www.microzed.org)	meta-xilinx-community
microzed-zynq7	Machine support for microZed (www.microzed.org)	meta-xilinx
minnow	MinnowBoard v1 (Intel Atom E640T); for the MinnowBoard MAX (Intel Atom E38xx), use the meta-intel intel-corei7-64 BSP (www.elinux.org/Minnowboard:MinnowMaxYoctoProject)	meta-minnow
minnow-emgd	MinnowBoard v1 (Intel Atom E640T); for the MinnowBoard MAX (Intel Atom E38xx), use the meta-intel intel-corei7-64 BSP (www.elinux.org/Minnowboard:MinnowMaxYoctoProject)	meta-minnow
ml405-virtex4-ppc405	Xilinx ML405 FPGA development platform with a Virtex-4 PowerPC 405 processor (with APU FPU)	meta-xilinx-community
ml507-virtex5-ppc440	Xilinx ML507 FPGA development platform with a Virtex-5 PowerPC 440 processor (with APU FPU)	meta-xilinx-community
mpc5200		meta-eldk
mpc5200xenomai		meta-eldk
mpc8315e-rdb		meta-yocto-bsp
nanopi	FriendlyARM NanoPi board	meta-nanopi
netbookpro	Psion Teklogix NetBook Pro	meta-netbookpro

（续）

机器名	描述	层
netstora	ARM-based Netgear Stora	**meta-kirkwood**
nexusone	Nexus One smartphone	**meta-htc**
nitrogen6sx	Boundary Devices Nitrogen6SX	**meta-fsl-arm-extra**
nitrogen6x	Boundary Devices Nitrogen6X	**meta-fsl-arm-extra**
nitrogen6x-lite	Boundary Devices Nitrogen6X Lite	**meta-fsl-arm-extra**
nokia900	Nokia 900	**meta-nokia**
nslu2be	NSLU2 in big-endian mode	**meta-nslu2**
nslu2le	NSLU2 in little-endian mode	**meta-nslu2**
nuc	Intel NUC model DC3217IYE	**meta-nuc**
odroid-c1	ODROID-C1 by Hardkernel	**meta-odroid**
odroid-u2	ODROID-U2 by Hardkernel	**meta-odroid**
odroid-ux3	ODROID-UX3 systems	**meta-odroid**
odroid-xu	ODROID-XU by Hardkernel	**meta-odroid**
olinuxino-a10lime	Olimex A10-OLinuXino Lime Board, based on Allwinner A10 CPU	**meta-sunxi**
olinuxino-a10s	Olimex A10S-OLinuXino-MICRO Board, based on Allwinner A10s CPU	**meta-sunxi**
olinuxino-a13	Olime A13-OLinuXino Board, based on Allwinner a13 CPU	**meta-sunxi**
olinuxino-a13som	Olimex A13-SOM Evaluation Board, based on Allwinner A13 CPU	**meta-sunxi**
olinuxino-a20	Olimex A20-OLinuXino Board, based on Allwinner A20 CPU	**meta-sunxi**
olinuxino-a20lime	Olimex A20-OLinuXino Lime Board, based on Allwinner A20 CPU	**meta-sunxi**
olinuxino-a20lime2	Olimex A20-OLinuXino Lime2 Board, based on Allwinner A20 CPU	**meta-sunxi**
olinuxino-a20som	Olimex A20-SOM Evaluation Board, based on Allwinner A20 CPU	**meta-sunxi**
om-gta01	OM Neo1973 GSM phone	**meta-openmoko**

om-gta02	Openmoko NeoFreerunner GSM phone	**meta-openmoko**
om-gta04	Goldelico GTA04 GSM phone	**meta-openmoko**
omap3evm	Texas Instruments OMAP3 EVM	**meta-ti**
omap5-evm	Texas Instruments OMAP5 uEVM	**meta-ti**
openpandora	Pandora handheld console (www.openpandora.org)	**meta-openpandora**
ouya	ASUS tf201	**meta-ouya**
overo	Gumstix Overo	**meta-gumstix**
overo	Gumstix Overo	**meta-gumstix-community**
p1010rdb	Freescale QorIQ P1010 Reference	**meta-fsl-ppc**
p1020rdb	Freescale QorIQ P1020 Reference	**meta-fsl-ppc**
p1021rdb	Freescale QorIQ P1021 Reference	**meta-fsl-ppc**
p1022ds	Freescale QorIQ P1022 Development	**meta-fsl-ppc**
p1023rdb	Freescale QorIQ P1023 Reference	**meta-fsl-ppc**
p1025twr	Freescale QorIQ P1025 MPU Tower	**meta-fsl-ppc**
p2020rdb	Freescale QorIQ P2020 Reference	**meta-fsl-ppc**
p2041rdb	Freescale QorIQ P2041 Reference	**meta-fsl-ppc**
p3041ds	Freescale QorIQ P3041 Development	**meta-fsl-ppc**
p4080ds	Freescale QorIQ P4080 Development	**meta-fsl-ppc**
p5020ds	Freescale QorIQ P5020 Development	**meta-fsl-ppc**
p5020ds-64b	Freescale QorIQ P5020 Development	**meta-fsl-ppc**
p5040ds	Freescale QorIQ P5040 Development	**meta-fsl-ppc**
p5040ds-64b	Freescale QorIQ P5040 Development	**meta-fsl-ppc**

（续）

机器名	描述	层
palmpre	Palm Pre handset	**meta-palm**
palmpre2	Palm Pre 2 handset	**meta-palm**
pandaboard	OMAP4430 Panda	**meta-ti**
parallella	Adapteva Parallella Board, headless	**meta-parallella**
parallella-hdmi	Adapteva Parallella Board with HDMI support	**meta-parallella**
pcm052	Phytec phyCORE Vybrid Development Kit	**meta-fsl-arm-extra**
pepper	Gumstix Pepper	**meta-gumstix**
picosam9	Picosam9 board (http://arm.mini-box.com)	**meta-picosam9**
picozed-zynq7	Machine support for picoZed (http://zedboard.org/product/picozed)	**meta-xilinx**
poodle	PXA250 based Sharp Zaurus SL-B500 and SHarp Zaurus SL-5600 devices	**meta-handheld**
qemuarm	arm_versatile_926ejs	**openembedded-core**
qemuarm64	Generic ARMv8	**openembedded-core**
qemumicroblaze	MicroBlaze QEMU machine support (Petalogix-ml605 model)	**meta-xilinx**
qemumicroblaze-s3adsp1800	MicroBlaze QEMU machine support (Petalogix-s3adsp1800 model)	**meta-xilinx**
qemumips	mti_malta32_be	**openembedded-core**
qemumips64	mti-malta64-be	**openembedded-core**
qemuppc	PPC system under QEMU emulation	**openembedded-core**
qemuppc64	PPC system under QEMU emulation	**meta-qemu-bsps**
qemux86	Common x86	**openembedded-core**
qemux86-64	Common x86	**openembedded-core**
qemuzynq	Zynq QEMU machine support (xilinx-zynq-a9 model)	**meta-xilinx**
quark	Quark systems	**meta-intel-quark**

raspberrypi	RaspberryPi board (www.raspberrypi.org)	**meta-raspberrypi**
raspberrypi0	RaspberryPi Zero board (https://www.raspberrypi.org/blog/raspberry-pi-zero)	**meta-raspberrypi**
raspberrypi2	RaspberryPi 2	**meta-raspberrypi**
rk3066a-marsboard	The Marsboard-RK3066 is powered by the popular Rockchip dual-core SoC combining a power dual-core ARM Cortex-A9 CPU and quad-core Mali-400 MP4 GPU	**meta-rockchip**
rk3188-radxarock	Radxa Rock is a single-board computer with quad-core ARM Cortex-A9 processor, 80 pin headers, which makes it easy to connect other I/O or GPIO	**meta-rockchip**
rk3288-firefly	Firefly-RK3288 is a high-performance platform with strong computing power, graphic processing, and video decoding ability	**meta-rockchip**
romley	Romley systems	**meta-romley**
romley-ivb	Romley systems	**meta-romley**
routerstationpro	mti_malta32_be	**meta-yocto-bsp-old**
rskrza1	RSKRZA1 board	**meta-renesas-rza1**
rskrza1-xip	RSKRZA1 board	**meta-renesas-rza1**
sama5d2-xplained	Atmel evaluation board	**meta-atmel**
sama5d3-xplained	Atmel evaluation board	**meta-atmel**
sama5d3xek	Atmel evaluation board	**meta-atmel**
sama5d4-xplained	Atmel evaluation board	**meta-atmel**
sama5d4ek	Atmel evaluation board	**meta-atmel**
sheevaplug	ARM-based Marvell SheevaPlug	**meta-kirkwood**
spitz	Sharp Zaurus SL-C3000 device	**meta-handheld**
sugarbay	Sugar Bay systems	**meta-sugarbay**
t1023rdb	Freescale QorIQ T1023 Reference	**meta-fsl-ppc**
t1023rdb-64b	Freescale QorIQ T1023 Reference	**meta-fsl-ppc**

（续）

机器名	描述	层
t1024rdb	Freescale QorIQ T1024 Reference	meta-fsl-ppc
t1024rdb-64b	Freescale QorIQ T1024 Reference	meta-fsl-ppc
t1040d4rdb	Freescale QorIQ T1040D4 Reference	meta-fsl-ppc
t1040d4rdb-64b	Freescale QorIQ T1040D4 Reference	meta-fsl-ppc
t1042d4rdb	Freescale QorIQ T1042D4 Reference	meta-fsl-ppc
t1042d4rdb-64b	Freescale QorIQ T1042D4 Reference	meta-fsl-ppc
t2080qds	Freescale QorIQ T2080 Development	meta-fsl-ppc
t2080qds-64b	Freescale QorIQ T2080 Development	meta-fsl-ppc
t2080rdb	Freescale QorIQ T2080 Reference	meta-fsl-ppc
t2080rdb-64b	Freescale QorIQ T2080 Reference	meta-fsl-ppc
t4160qds	Freescale QorIQ T4160 Development	meta-fsl-ppc
t4160qds-64b	Freescale QorIQ T4160 Development	meta-fsl-ppc
t4240qds	Freescale QorIQ T4240 Development	meta-fsl-ppc
t4240qds-64b	Freescale QorIQ T4240 Development	meta-fsl-ppc
t4240rdb	Freescale QorIQ T4240 Reference	meta-fsl-ppc
t4240rdb-64b	Freescale QorIQ T4240 Reference	meta-fsl-ppc
tilapia	Asus Tilapia	meta-asus
toro	Samsung Toro	meta-samsung
toroplus	Samsung Toro Plus	meta-samsung
tosa	PXA255-based Sharp Zaurus SL-6000 device	meta-handheld
twr-vf65gs10	Freescale Vybrid TWR-VF65GS10	meta-fsl-arm
tx6q-10x0	Ka-Ro electronics TX6Q Computer-On-Module	meta-fsl-arm-extra

tx6q-11x0	Ka-Ro electronics TX6Q Computer-On-Module	**meta-fsl-arm-extra**
tx6s-8034	Ka-Ro electronics TX6S Computer-On-Module	**meta-fsl-arm-extra**
tx6s-8035	Ka-Ro electronics TX6S Computer-On-Module	**meta-fsl-arm-extra**
tx6u-8033	Ka-Ro electronics TX6DL Computer-On-Module	**meta-fsl-arm-extra**
tx6u-80x0	Ka-Ro electronics TX6DL Computer-On-Module	**meta-fsl-arm-extra**
tx6u-81x0	Ka-Ro electronics TX6DL Computer-On-Module	**meta-fsl-arm-extra**
ventana	Gateworks Ventana boards	**meta-fsl-arm-extra**
vexpressa9	VExpress A9 board	**meta-ivi-bsp**
veyron-speedy	Google Veyron Speedy Rev 1+ board (formally known as Chromebook Asus C201)	**meta-rockchip**
wandboard	i.MX6 Wandboard Quad	**meta-fsl-arm-extra**
zc702-base-trd-zynq7	Machine support for ZC702 Base TRD	**meta-xilinx-community**
zc702-zynq7	Machine support for ZC702 Evaluation Board	**meta-xilinx**
zc706-pcie-trd-zynq7	Machine support for ZC706 PCIe TRD	**meta-xilinx-community**
zc706-zynq7	Machine support for ZC706 Evaluation Board	**meta-xilinx**
ze7000-zynq7	Machine configuration for the ze7000 from NetModule device	**meta-xilinx-community**
zedboard-zynq7	Machine support for ZedBoard (www.zedboard.org)	**meta-xilinx**
zx3-pm3-zynq7	Enclustra PM3 evaluation board with ZX3 module	**meta-xilinx-community**
zybo-zynq7	Machine support for ZYBO	**meta-xilinx**
zynq-ze7000	Poky image generated by NetModule AG (www.netmodule.com) for the ZE7000 (Z4E) Platform	**meta-netmodule**
zynq-zx3-pm3	Poky image generated by NetModule AG (www.netmodule.com) for the Mars ZX3 Module + Mars PM3 development kit from Enclustra AG (www.enclustra.com)	**meta-netmodule**
zynq-zx3-starter	Poky image generated by NetModule AG (www.netmodule.com) for the Mars ZX3 Module + Mars Starter development kit from Enclustra AG (www.enclustra.com)	**meta-netmodule**